STUDY GUIDE

SQL & DATABASES MASTERY

A practical mini-textbook: from your first SELECT to window functions, query tuning, schema design, analytics patterns and the wider database world.

Taught the IIT Professor Way — Real Numbers. Real Problems. Real Understanding.

CHAPTER 1 Databases and the Relational Model	**CHAPTER 2** SELECT, WHERE, ORDER BY, LIMIT, DISTINCT and NULL Logic	**CHAPTER 3** Aggregation, GROUP BY, HAVING and FILTER	**CHAPTER 4** Joins	**CHAPTER 5** Subqueries and CTEs (Including Recursive CTEs)	**CHAPTER 6** CASE, Strings, Dates and Times, JSON and Arrays
CHAPTER 7 Set Operations, Pivot and Unpivot, GROUPING SETS, ROLLUP and CUBE	**CHAPTER 8** Window Functions in Depth	**CHAPTER 9** DDL, DML, Constraints, Upserts and Transactions	**CHAPTER 10** Indexes, EXPLAIN and Query Optimization	**CHAPTER 11** Schema Design, Normalization and Data Warehouse Modeling	**CHAPTER 12** Analytics SQL Patterns
CHAPTER 13 Beyond SQL	**CHAPTER 14** SQL Dialect Differences and the 40-Problem Interview Bank	**CHAPTER 15** Project Lab, Part 1: Funnels, Cohorts, RFM and a Sales Database	**CHAPTER 16** Project Lab, Part 2: An Analytics Case Study and a Query-Optimization Lab	**APPENDIX** Cheat Sheet	

A Note From Your Professor

Dear learner, welcome. SQL is the craft of asking clear questions of data: how many orders came in, which delivery was slowest, which students passed. A database is a system with queues, caches and limits, like a busy bank or hospital, and a SQL query is a request that you can measure and tune. Everyday examples (shop orders, delivery times, exam marks, hospital appointments) will carry you through, especially in the chapters on indexes and query plans.

SQL looks easy, and the first week is easy. The difficulty hides in a few ideas: NULL, joins that repeat rows, grouping, and window functions. People who skip them write queries that run without errors and return wrong numbers. This book teaches those ideas slowly, with a small shop database that you can check by hand, and with real output from real runs.

Study about one hour a day. Type every example yourself. Before you run a query, predict the output. If your prediction is wrong, that is the best moment to learn. And remember the one rule that matters most: **never move on until the current topic feels obvious.** If a chapter feels foggy, repeat its practice set before you open the next chapter. SQL rewards patience, and a clear mind about the basics is worth more than a long list of tricks. I am glad you are here. Let us begin.

What Every Topic Contains

1. **Intuition**: the problem the concept solves, in everyday language, often with a performance-testing analogy.
2. **Formal definition**: how it works, precise but plain, with every new word explained.
3. **Worked examples**: real SQL with the exact output, with Predict, Compute, Check and Meaning steps.

4. **The picture**: an ASCII diagram, table or image, explained completely.
5. **Connection to the real job**: where it shows up in analytics, data engineering and ML work.
6. **Applications**: grouped by role, each with a concrete example.
7. **Common mistakes**: what beginners get wrong and how to catch it.
8. **Check yourself**: three short questions with answers.

Every chapter also ends with a recap and cheat-sheet table, a practice set (8 to 12 problems with answers) and a prompt pack of 5 prompts to paste into any AI assistant.

Contents

1. **Databases and the Relational Model**: what a database is, tables, keys, and how to set up SQLite and PostgreSQL.
2. **SELECT, WHERE, ORDER BY, LIMIT, DISTINCT and NULL Logic**: reading one table and the three-valued logic of NULL.
3. **Aggregation, GROUP BY, HAVING and FILTER**: turning many rows into summaries.
4. **Joins**: inner, left, right, full, cross, self, anti- and semi-joins, and the duplicate-row traps.
5. **Subqueries and CTEs**: queries inside queries, named steps, and recursive CTEs.
6. **CASE, Strings, Dates and Times, JSON and Arrays**: cleaning and reshaping values.
7. **Set Operations, Pivot and Unpivot, GROUPING SETS, ROLLUP, CUBE**: stacking, comparing and subtotalling.
8. **Window Functions in Depth**: ranking, LAG/LEAD, frames, top-N per group, running totals, gaps and islands.
9. **DDL, DML, Constraints, Upserts, Transactions, ACID, Isolation Levels and Locks**: changing data safely.
10. **Indexes, EXPLAIN / EXPLAIN ANALYZE and Query Optimization**: making queries fast, with a measure, change one thing, re-measure mindset.
11. **Schema Design**: ER diagrams, normalization 1NF to BCNF, denormalization, OLTP vs OLAP, star and snowflake, slowly changing dimensions.
12. **Analytics SQL Patterns**: funnels, cohort retention, RFM, MoM and YoY, sessionization, deduplication, data-quality queries.
13. **Beyond SQL**: MongoDB, Redis, Cassandra and DynamoDB, time-series, graph, search engines, vector databases, CAP, and when to use which.
14. **SQL Dialect Differences and the 40-Problem Interview Bank**: PostgreSQL, MySQL, SQL Server, BigQuery and Snowflake, with answers.
15. Project Lab, Part 1: Funnels, Cohorts, RFM and a Sales Database
16. Project Lab, Part 2: An Analytics Case Study and a Query-Optimization Lab

An appendix (`99_cheatsheet.md`) collects the key patterns and ends with an honest closing note.

How To Study This Document

Use this learning loop for every chapter.

1. **Build first.** Create the shop database and type the first example before reading the theory. Seeing a result makes the explanation easier to hold.
2. **Ask.** Write down what you do not understand. Use a prompt from the chapter's prompt pack to ask an AI assistant to explain it a second way.
3. **Just-in-time theory.** Read the definitions only when you need them to explain what you saw.
4. **Rebuild.** Close the book and redo the worked examples from memory on a new small table of your own (for example, your own shop orders or exam marks).
5. **Teach.** Explain the idea aloud, in simple words, as if to a colleague. If you cannot, repeat step 3.

Suggested pace: one chapter per week, about one hour a day. Days 1 to 2: read and run the worked examples. Days 3 to 4: rebuild and do the practice set. Day 5: prompt pack (quiz and mock interview). Weekend: redo the problems you got wrong. Chapters 8 and 10 deserve two weeks each.

Using the prompt packs: paste one prompt into any AI assistant. Replace `[PASTE HERE]` with your own query or table design for the review prompts. Always try the problem yourself first, and treat the assistant as a coach, not an answer machine. Check every number it gives by running the query.

The mastery tests from your skill map are a good finish: solve 300 or more SQL problems (about 50 hard) on LeetCode, StrataScratch or DataLemur; speed up a slow query from 10 seconds to under 1 second using `EXPLAIN ANALYZE` ; and design an e-commerce schema with all its analytics queries.

Prerequisites

- Comfortable use of a computer and a terminal (opening a shell, running a command).
- Basic arithmetic and percentages. No programming is required, though a little Python helps.
- The learner's Math PDF is not needed for this book.

Tools you need

All free.

- **SQLite** (command `sqlite3`, version 3.39 or newer; this book was run on 3.51). Included on macOS; installable on Windows and Linux.
- **PostgreSQL** (any recent version) through Docker or a native install, plus `psql`. Examples marked "PostgreSQL syntax, not run here" were not executed in this book.
- A SQL client such as DBeaver, pgAdmin, DB Browser for SQLite or a VS Code extension.
- A text editor.
- Optional for Chapter 13: Docker images for MongoDB and Redis.

CHAPTER 1

DATABASES AND THE RELATIONAL MODEL

What a database is, how tables and keys work, and how to set up SQLite and PostgreSQL on your own laptop.

Think about the last app you used to order food. Somewhere, a program stored your name, your address, your order and the restaurant's menu, and it found them again in a fraction of a second while thousands of other people did the same. That program is a database. Almost every job in data, analytics, machine learning and software depends on reading from one. In this chapter you learn what a database is, how a relational database organises data in tables linked by keys, and how to install two free ones (SQLite and PostgreSQL) so that you can practise on your own laptop from today.

Ten words before we start

This chapter uses a handful of words over and over. Read this table once now. If a word later feels fuzzy, come back here. Nothing in the table needs any earlier knowledge.

Word	Plain-English meaning	Everyday picture
Data	Facts written down: a name, a price, a date.	One cell in a spreadsheet.
Database	An organised collection of data that a program looks after for you.	A well-run filing room.
DBMS	The program that looks after the database (PostgreSQL, SQLite, MySQL).	The clerk who runs the filing room.
Table	One list of similar things, laid out as rows and columns.	One drawer: "customers".
Row	One thing in the list (one customer).	One card in the drawer.
Column	One property that every row has (the city).	One printed field on every card.
Key	A value that picks out exactly one row.	A passport number.
Query	A written question you send to the database.	A note to the clerk: "bring me all cards where city is Pune".
Constraint	A rule the database enforces on every change.	The clerk refuses a card with no name on it.
Transaction	A group of changes that either all happen or none happen.	A bank transfer: money leaves one account and arrives in the other, or nothing moves.

WHAT YOU WILL BE ABLE TO DO

After this chapter you can: (1) say what any table's rows and key are, in one sentence; (2) choose the type and the rules for each column; (3) split a messy flat file into clean linked tables and prove nothing was lost; (4) predict, without running it, whether an insert, update or delete succeeds or fails; (5) open a database you have never seen and map it in ten minutes; (6) answer the usual interview questions about keys, cardinality and normalization. Sections 1.1 to 1.4 are the base. Sections 1.5 to 1.14 turn that base into working skill.

1.1 Why Databases Exist

INTUITION

Imagine you save your shop's orders in a CSV file. It works for 100 rows. At 100 million rows, two people edit the file at once, one breaks the format, and a search takes minutes. A **database** is a program whose only job is to store data safely, let many people use it at the same time, and find rows fast. A **DBMS** (Database Management System) is that program. Examples: PostgreSQL, MySQL, SQLite, SQL Server, Oracle.

A **relational database** stores data in **tables**. You talk to it with **SQL** (Structured Query Language, said "S-Q-L" or "sequel"). SQL is *declarative*: you say **what** you want, not **how** to get it. The database engine decides the how. This is the most important mental shift for a programmer.

Compare a Python loop and a SQL query for "total of all paid orders":

```
orders = [{"status": "paid", "amount": 500}, {"status": "cancelled", "amount": 300}]
total = 0
for order in orders:
    if order["status"] == "paid":
        total += order["amount"]
```

```
SELECT SUM(amount) FROM orders WHERE status = 'paid';
```

The Python tells the computer every step. The SQL states the result. The engine may use an index, run in parallel, or read from cache. You do not care.

Real-world mapping

Idea	Example	Where it appears in the job
Table	`orders`	a fact table in a warehouse
Row	one order	one event in an event log
Column	`order_date`	a feature column for ML
SQL query	`SELECT ... WHERE ...`	a dashboard tile, a data-pull for a model

What a DBMS gives you that a file does not

Need	CSV file	Database
Fast search	Read whole file	Indexes (Chapter 10)
Many users at once	Corruption risk	Transactions and locks (Chapter 9)
Rules on data	None	Constraints (Chapter 9)
Crash safety	Half-written file	Write-ahead log, recovery
Joining two datasets	Manual code	`JOIN` (Chapter 4)

The picture: where SQL sits

```
  You / Python / BI tool
          |   SQL text
          v
  +-------------------+
  | Parser            |  checks the syntax
  | Planner/Optimizer |  chooses the fastest plan
  | Executor          |  runs the plan
  | Storage engine    |  pages on disk, cache, log
  +-------------------+
          |
        files on disk
```

Worked examples

Problem 1.1.1 A delivery app produces 5 million delivery-tracking rows per day. After 30 days, how many rows must the database hold, and why is a plain file a poor choice?

Step 1	Multiply: `5,000,000 × 30 = 150,000,000` rows.
Step 2	A file must be read from the start to find the rows of one day. 150 million rows at, say, 100 bytes each is about 15 GB to scan.
Step 3	A database index finds the rows for one day without reading the other 15 GB.
Check	`150,000,000 × 100 bytes = 15,000,000,000 bytes = 15 GB`, using 1 GB = 1,000,000,000 bytes.
Answer	150 million rows (about 15 GB at 100 bytes per row). A database with an index avoids scanning all of it.
Meaning	Reading 15 GB takes tens of seconds even on a fast disk. Doing that for every question is too slow, which is why databases keep indexes.

Problem 1.1.2 Is `SELECT * FROM orders WHERE status = 'paid'` imperative or declarative? What does the engine decide?

Step 1	The query says which rows (`status = 'paid'`) and which columns (`*`), but no loop and no order of steps.
Step 2	So it is declarative.
Answer	Declarative. The engine decides whether to scan the table, use an index, and how to read the pages.
Check	Ask: "Did I write a loop or an order of steps?" No, so it is declarative.

Connection to the real job

Every analyst, data engineer and ML engineer reads data from a database daily. Feature tables for ML, dashboards for managers and the order history of every online shop all live in databases. SQL is the one skill that appears in almost every data job description.

Applications

- Business reporting (sales, churn, finance)
- Application back ends (user accounts, orders)
- ML feature stores and training sets
- Storing delivery records and monitoring data

COMMON MISTAKE	Thinking SQL is "just for queries". It also defines structure (DDL), changes data (DML) and controls safety (transactions). Later chapters cover all four.

Check yourself

1. What does *declarative* mean?

Answer: You describe the result you want; the engine chooses the steps.

1. Name two things a DBMS gives that a CSV file does not.

Answer: For example indexes, transactions, constraints, crash recovery.

1. What does DBMS stand for?

Answer: Database Management System.

1.2 Tables, Rows, Columns and Keys

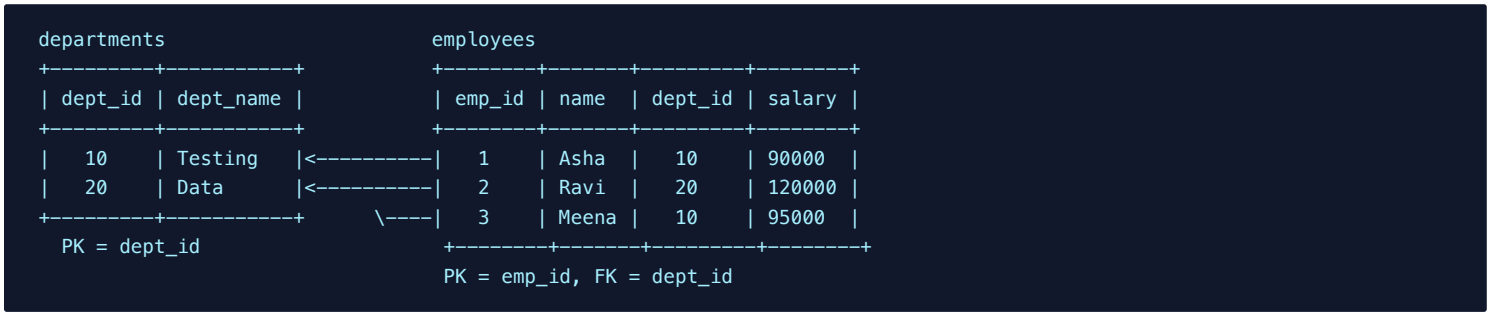
INTUITION

A table is a very strict spreadsheet. Every column has a fixed name and type. Every row is one "thing" (one customer, one order). The strictness is the point: strict data can be searched and combined safely.

Vocabulary

- **Table** (also *relation*): a named grid of data.
- **Row** (also *record*, *tuple*): one item.
- **Column** (also *field*, *attribute*): one property, with a **data type** such as integer, text, date.
- **Primary key (PK)**: a column (or group of columns) whose value is **unique** and **not NULL** for every row. It identifies the row.
- **Foreign key (FK)**: a column that holds a primary-key value of **another** table. It links rows across tables.
- **Natural key**: a real-world unique value (email, passport number). **Surrogate key**: an artificial number the database makes (1, 2, 3). Surrogate keys are usually safer because real-world values change.
- **NULL**: "unknown or missing". It is not zero and not an empty string. Chapter 2 treats NULL in depth.
- **Schema**: the set of table definitions. (In PostgreSQL a *schema* is also a named folder of tables inside a database.)

The picture: two tables linked by a key



Real-world mapping

Concept	Example value	Where it appears in the job
Primary key	`emp_id = 1`	`user_id` that identifies a user in every table
Foreign key	`employees.dept_id = 10`	`orders.customer_id` linking to a customer
Grain	one row = one employee	the first question to ask about any dataset
NULL	no manager assigned	missing values you must handle in ML

Worked examples (runnable)

Before you run each example, predict: will it succeed or fail? Then compare with the real output.

We create two tables, insert rows, and read them back. The `sqlite3` tool shows output with a header row.

Problem 1.2.1 Create `departments` and `employees`, insert data, and list all employees.

Step 1	Create the parent table first (the one that is referred to).
Step 2	Create the child table with a foreign key.
Step 3	Insert rows and `SELECT`.

```
PRAGMA foreign_keys = ON;   -- SQLite only: FK checks are off by default

CREATE TABLE departments (
  dept_id   INTEGER PRIMARY KEY,
  dept_name TEXT NOT NULL UNIQUE
);
CREATE TABLE employees (
  emp_id  INTEGER PRIMARY KEY,
  name    TEXT NOT NULL,
  dept_id INTEGER REFERENCES departments(dept_id),
  salary  INTEGER
);
INSERT INTO departments VALUES (10,'Testing'),(20,'Data');
INSERT INTO employees VALUES
  (1,'Asha',10,90000),
  (2,'Ravi',20,120000),
  (3,'Meena',10,95000);

SELECT * FROM employees;
```

```
emp_id  name   dept_id  salary
------  -----  -------  ------
1       Asha   10       90000
2       Ravi   20       120000
3       Meena  10       95000
```

- **Check** We inserted three employees and the printed table has three rows. Departments 10 and 20 exist, so the foreign keys are valid.
- **Answer** The table holds three rows, as shown above.
- **Meaning** The two tables are now linked: Asha and Meena belong to Testing (10), Ravi to Data (20).

Problem 1.2.2 What happens if we insert a second employee with `emp_id = 1`, or an employee in department 99 (which does not exist)?

Step 1	The primary key rejects duplicates.
Step 2	The foreign key rejects a parent that does not exist.

```
PRAGMA foreign_keys = ON;
INSERT INTO employees VALUES (1,'Dup',10,50000);
INSERT INTO employees VALUES (4,'Ghost',99,50000);
```

```
Runtime error near line 3: UNIQUE constraint failed: employees.emp_id (19)
Runtime error near line 4: FOREIGN KEY constraint failed (19)
```

- **Answer** Both inserts fail: `UNIQUE constraint failed: employees.emp_id` for the duplicate key and `FOREIGN KEY constraint failed` for the missing department. (SQLite says UNIQUE for primary keys; PostgreSQL says `duplicate key value violates unique constraint`.)
- **Check** Run `SELECT COUNT(*) FROM employees;` afterwards: still 3.
- **Meaning** The database protects the data for you. Your application code does not need to remember to check.

A note on data types

Common types: `INTEGER`, `NUMERIC(p,s)` / `DECIMAL` (exact decimals, use for money), `REAL` / `DOUBLE` (approximate, do not use for money), `TEXT` / `VARCHAR`, `BOOLEAN`, `DATE`, `TIMESTAMP`. SQLite is unusual: it has *type affinity*, so it is lenient and stores almost anything in any column. PostgreSQL is strict. Learn the strict rules; they are what you will meet at work.

Connection to the real job

Keys are how you join tables (Chapter 4). A wrong key assumption is the number one cause of wrong dashboard numbers. Before using any table, ask: "What is one row? What is its primary key?" This is called the table's **grain**.

Applications

- Customer, order, product tables in e-commerce
- Dimension and fact tables in data warehouses (Chapter 11)
- Feature tables keyed by `user_id` for ML

COMMON MISTAKE	Using a column like `email` or `name` as primary key. Names repeat and emails change. Use a surrogate key plus a `UNIQUE` constraint on the natural value.

Check yourself

1. Can a primary key be NULL?

Answer: No. It must be unique and not NULL.

1. What does a foreign key do?

Answer: It forces a value to exist as a primary key in another table.

1. What is the "grain" of a table?

Answer: What one row represents.

1.3 Setting Up SQLite and PostgreSQL

INTUITION	To learn swimming you need water. To learn SQL you need a database you can break. We use two: **SQLite** (zero setup, one file, perfect for learning) and **PostgreSQL** (the real server database used in industry).

SQLite

SQLite is a database in a single file. It has no server. macOS already includes the `sqlite3` command. Check the version. Window functions need version 3.25 or newer, and `RIGHT` / `FULL JOIN` need 3.39 or newer.

```
sqlite3 --version
sqlite3 shop.db          # opens (or creates) the file shop.db
```

Useful commands inside the shell (they start with a dot, they are not SQL):

Command	Meaning
`.tables`	list tables
`.schema orders`	show the CREATE statement
`.headers on` and `.mode column`	readable output
`.read file.sql`	run a SQL file
`.import file.csv table`	load a CSV
`.timer on`	show query time

Command	Meaning
`.quit`	leave

Let us use a few of them. In this book each chapter runs on a fresh file database, so the output below is real.

```
SELECT sqlite_version() AS version;
.tables
.schema employees
```

```
version
-------
3.51.0
departments  employees
CREATE TABLE employees (
  emp_id  INTEGER PRIMARY KEY,
  name    TEXT NOT NULL,
  dept_id INTEGER REFERENCES departments(dept_id),
  salary  INTEGER
);
```

PostgreSQL

PostgreSQL ("Postgres") is a server. The easy free ways to run it:

```
# Option A: Docker (works on any OS)
docker run --name pg -e POSTGRES_PASSWORD=devpass -p 5432:5432 -d postgres
docker exec -it pg psql -U postgres

# Option B: macOS with Homebrew
brew install postgresql
brew services start postgresql
psql postgres
```

Not run here: this book was tested with SQLite, so PostgreSQL commands are shown in standard form. The password above is a throw-away example for a local test container. Never reuse such a password.

Useful `psql` commands: `\l` (list databases), `\dt` (list tables), `\d tablename` (describe), `\i file.sql` (run a file), `\copy` (load a CSV), `\timing` (time queries), `\q` (quit).

Graphical clients (free): DBeaver, pgAdmin, DB Browser for SQLite, and VS Code extensions.

SQLite vs PostgreSQL at a glance

Topic	SQLite	PostgreSQL
Architecture	Library, one file	Client-server
Types	Flexible	Strict
Many writers	One at a time	Many (MVCC)
Rich features	Fewer (no `ROLLUP`, `LATERAL`, `DATE_TRUNC`)	Very rich
Best use	Learning, apps, tests	Production

Worked examples

Problem 1.3.1 Load a CSV of delivery times into SQLite. Create `deliveries.csv` with the rows `Pune,230`, `Delhi,410`, `Mumbai,980` (city, minutes).

Step 1	Create the file with `printf`.
Step 2	Create a table, then import. `.import` is a dot-command.
Step 3	Query to confirm.

```
printf 'Pune,230\nDelhi,410\nMumbai,980\n' > /tmp/deliveries.csv
```

```
CREATE TABLE deliveries (city TEXT, minutes INTEGER);
.mode csv
.import /tmp/deliveries.csv deliveries
.mode column
SELECT city, minutes FROM deliveries ORDER BY minutes DESC;
```

```
city    minutes
------  -------
Mumbai  980
Delhi   410
Pune    230
```

- **Answer** The three rows, slowest first, as printed above.

Problem 1.3.2 Find out how many rows are in a table without opening a GUI.

```
SELECT COUNT(*) AS n_employees FROM employees;
```

```
n_employees
-----------
3
```

- **Answer** The count is 3. In any tool the idea is the same: `SELECT COUNT(*)`.
- **Check** It matches the three rows we inserted in 1.2.

Connection to the real job

You will use a server database (Postgres, MySQL, SQL Server, a cloud warehouse) at work and SQLite or DuckDB on your laptop for quick analysis. All speak almost the same SQL for the first 90 percent of tasks.

Applications

- Practice and interviews (SQLite or a browser tool)
- Local prototyping before deploying to Postgres
- Mobile apps and browsers embed SQLite

COMMON MISTAKE	Forgetting `PRAGMA foreign_keys = ON;` in SQLite. Without it foreign keys are silently ignored. PostgreSQL always enforces them.

Check yourself

1. Which SQLite version added window functions?

Answer: 3.25.

1. Which is a client-server database, SQLite or PostgreSQL?

Answer: PostgreSQL.

1. What does `\dt` do in psql?

Answer: Lists the tables.

1.4 Relational Thinking: Relations, Cardinality and Why We Split Tables

INTUITION	Do not write the customer's address on every order. If she moves, you must fix 500 orders. Write it once in `customers`, and let each order point to the customer by key. This is "one fact in one place".

Relationship types (cardinality)

- **One-to-many (1:N)**: one department has many employees. The FK is on the "many" side. By far the most common.
- **Many-to-many (M:N)**: one order has many products and one product is in many orders. Needs a **bridge table** (also *junction* or *association* table). In our shop, `order_items` is the bridge.
- **One-to-one (1:1)**: rare; usually extra detail split out of a big table.

```
customers 1 ----< orders 1 ----< order_items >---- 1 products
  (PK customer_id)  (FK customer_id)   (FK order_id, FK product_id)
```

Real-world mapping

Relationship	Example	Where it appears in the job
One-to-many	one customer, many orders	user and events tables
Many-to-many	orders and products via `order_items`	students and courses, tags and articles
One-to-one	user and user_profile	splitting rarely used columns

The shop database used in this book

From Chapter 2 onward we use the same small shop database, so you can compare chapters easily. It has 8 customers, 6 products, 14 orders and 20 order lines. Customers 7 and 8 have never ordered. Product 6 (Mug) never sold. Those gaps are deliberate: they make joins and anti-joins interesting.

Worked examples

Problem 1.4.1 An `employees` table also needs "skills" (an employee has many skills, a skill belongs to many employees). Design the tables.

Step 1	Skills and employees are many-to-many, so we need a bridge table.
Step 2	The bridge has two FKs and a composite primary key.

```
PRAGMA foreign_keys = ON;
CREATE TABLE skills (skill_id INTEGER PRIMARY KEY, skill TEXT NOT NULL UNIQUE);
CREATE TABLE employee_skills (
  emp_id   INTEGER REFERENCES employees(emp_id),
  skill_id INTEGER REFERENCES skills(skill_id),
  PRIMARY KEY (emp_id, skill_id)
);
INSERT INTO skills VALUES (1,'Excel'),(2,'SQL'),(3,'Python');
INSERT INTO employee_skills VALUES (1,1),(1,2),(2,2),(2,3),(3,1);

SELECT e.name, s.skill
FROM employees e
JOIN employee_skills es ON es.emp_id = e.emp_id
JOIN skills s ON s.skill_id = es.skill_id
ORDER BY e.name, s.skill;
```

```
name   skill
-----  ------
Asha   Excel
Asha   SQL
Meena  Excel
Ravi   Python
Ravi   SQL
```

- **Check** We inserted 5 bridge rows, so 5 result rows should appear. Employee 1 (Asha) has skills 1 and 2, employee 2 (Ravi) has 2 and 3, employee 3 (Meena) has 1.
- **Answer** Five employee-skill pairs. Asha knows Excel and SQL, Meena knows Excel, Ravi knows Python and SQL. (Joins are explained in Chapter 4; read `JOIN` for now as "attach the matching rows".)

Problem 1.4.2 How many rows does the bridge table need for 1,000 employees who each know 4 skills?

Step 1	One row per pair: `1,000 × 4 = 4,000`.
Answer	4,000 rows. Storing skills as a comma list inside `employees` would break search and rules.
Meaning	4,000 small rows are cheap and fast to search. One long text cell per person is not.

Connection to the real job

Reading an unfamiliar database starts with drawing this picture: which tables are parents, which are children, which are bridges. Data engineers call it "understanding the data model".

Applications

- E-commerce (orders and items)
- Social networks (follows is M:N on users)
- Healthcare (patients, visits, procedures)

COMMON MISTAKE	Putting a list in one cell ("Java, SQL, Python"). You cannot index it or join on it. Use a bridge table.

Check yourself

1. Which side of a 1:N relationship holds the foreign key?

Answer: The "many" side.

1. What solves many-to-many?

Answer: A bridge table.

1. Why did we split address out of orders?

Answer: One fact in one place, so one update fixes it.

1.5 Types and Constraints: Teaching the Database to Refuse Bad Data

INTUITION	A good database is a strict bouncer. Your application will have bugs, a colleague will run a careless script at night, a CSV will arrive with a blank where a price should be. If the rules live only in application code, one slip lets bad data in, and bad data stays forever. A **constraint** is a rule written into the table itself, so the database checks it on every insert and update, whoever sends it.

The six rules you can attach to a column

Rule	What it forbids	Example	Typical job use
`NOT NULL`	A missing value	`name TEXT NOT NULL`	Mandatory fields such as an order date
`UNIQUE`	Two rows with the same value	`sku TEXT UNIQUE`	Natural keys: email, SKU, passport number
`PRIMARY KEY`	Duplicates and NULL (UNIQUE plus NOT NULL, one per table)	`product_id INTEGER PRIMARY KEY`	Identifying a row in every join
`FOREIGN KEY`	A value with no parent row	`REFERENCES departments(dept_id)`	No orphan orders
`CHECK (condition)`	Any row where the condition is false	`CHECK (price > 0)`	Ranges, allowed status values
`DEFAULT value`	Nothing; it fills in a value when you omit the column	`category TEXT DEFAULT 'General'`	Created-at timestamps, status flags

Four of these (NOT NULL, UNIQUE, CHECK, FOREIGN KEY) are rules that reject rows. DEFAULT is a convenience. PRIMARY KEY is shorthand for two rules at once. Learn to see every table definition as a list of promises about its data.

Worked examples

Problem 1.5.1 Build a `products` table with a mandatory name, a unique SKU, a positive price and a default category. Send it one good row and four bad rows. Predict each result before you read the output.

Step 1	Write the table so that each promise has its own rule: `NOT NULL` on name and price, `UNIQUE` on sku, `CHECK (price > 0)`, `DEFAULT 'General'`.
Step 2	Put the test statements in a list with a label, so every outcome is printed.
Step 3	Wrap each insert in `try/except sqlite3.Error` so one failure does not stop the script.
Step 4	Predict: the good row passes; missing name fails (NOT NULL); duplicate sku fails (UNIQUE); price 0 fails (CHECK); duplicate id fails (PRIMARY KEY).

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("""
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  name       TEXT    NOT NULL,
  sku        TEXT    UNIQUE,
  price      INTEGER NOT NULL CHECK (price > 0),
  category   TEXT    NOT NULL DEFAULT 'General'
)""")
tests = [
 ("good row",     "INSERT INTO products(product_id,name,sku,price) VALUES (1,'Keyboard','KB-1',1500)"),
 ("missing name", "INSERT INTO products(product_id,sku,price) VALUES (2,'MS-1',600)"),
 ("duplicate sku","INSERT INTO products(product_id,name,sku,price) VALUES (3,'Mouse','KB-1',600)"),
 ("price zero",   "INSERT INTO products(product_id,name,sku,price) VALUES (4,'Free','F-1',0)"),
 ("duplicate id", "INSERT INTO products(product_id,name,sku,price) VALUES (1,'Pen','P-1',20)"),
]
for label, sql in tests:
    try:
        con.execute(sql)
        print(f"{label:14} OK")
    except sqlite3.Error as e:
        print(f"{label:14} REJECTED: {e}")
print(con.execute("SELECT * FROM products").fetchall())
```

```
good row       OK
missing name   REJECTED: NOT NULL constraint failed: products.name
duplicate sku  REJECTED: UNIQUE constraint failed: products.sku
price zero     REJECTED: CHECK constraint failed: price > 0
duplicate id   REJECTED: UNIQUE constraint failed: products.product_id
[(1, 'Keyboard', 'KB-1', 1500, 'General')]
```

1. `sqlite3.connect(":memory:")` makes a throw-away database that lives in RAM and disappears when the script ends.
2. The `CREATE TABLE` text is one multi-line string. `product_id INTEGER PRIMARY KEY` is the identity of each row. `name TEXT NOT NULL` forbids a missing name. `sku TEXT UNIQUE` forbids a repeated SKU but allows NULL. `CHECK (price > 0)` is evaluated for every inserted or updated row. `DEFAULT 'General'` supplies the category when the insert does not mention it.
3. `tests` is a list of (label, SQL) pairs. Notice that the second test leaves out `name` entirely, so name would be NULL.
4. The loop runs each statement. If the database raises an error, `except sqlite3.Error as e` catches it and prints the database's own message. `{label:14}` pads the label to 14 characters so the output lines up.
5. The last line reads the table back as a list of tuples. Only the good row is there, and its `category` is the default `'General'`.

Answer	Exactly one row survives. All four bad rows are refused, each by a different rule.
Check	The table holds one row, and its category shows the DEFAULT worked even though the insert never mentioned it.
Meaning	You never wrote an `if` statement. The table definition is the validation code, and it runs for every program that touches the table. PostgreSQL phrases the messages differently (for example `null value in column "name" violates not-null constraint`) but refuses the same rows.

Problem 1.5.2 A table has `qty INTEGER CHECK (qty > 0)`. Does the row with `qty = NULL` get in? Then fix the table so it does not.

Idea	A `CHECK` rejects a row only when the condition is *false*. With NULL the condition `NULL > 0` is neither true nor false but *unknown* (three-valued logic, Chapter 2). Unknown is not false, so the row passes.
Fix	Add `NOT NULL` next to the `CHECK`. The two rules cover different cases: CHECK covers bad numbers, NOT NULL covers absent numbers.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE lines_a (qty INTEGER CHECK (qty > 0))")
con.execute("CREATE TABLE lines_b (qty INTEGER NOT NULL CHECK (qty > 0))")
for t in ("lines_a", "lines_b"):
    for value in (5, 0, None):
        try:
            con.execute(f"INSERT INTO {t} VALUES (?)", (value,))
            print(t, value, "accepted")
        except sqlite3.Error as e:
            print(t, value, "rejected:", e)
```

```
lines_a 5 accepted
lines_a 0 rejected: CHECK constraint failed: qty > 0
lines_a None accepted
lines_b 5 accepted
lines_b 0 rejected: CHECK constraint failed: qty > 0
lines_b None rejected: NOT NULL constraint failed: lines_b.qty
```

1. Two tables differ by one word, `NOT NULL`. `lines_a` has only a CHECK.
2. The nested loops try the values 5 (fine), 0 (violates the check) and `None`, which Python sends to SQL as NULL.
3. `?` is a placeholder: the value travels separately from the SQL text. Section 1.10 explains why this matters for safety. The table name in the f-string comes from our own constant list, never from a user.

Answer	In `lines_a` the NULL is accepted. In `lines_b` it is rejected by NOT NULL. The value 0 is rejected in both.
Meaning	A NULL slips past a CHECK in every major database. Always pair a CHECK on a column with NOT NULL when the value is mandatory.

COMMON MISTAKE	Believing `CHECK (price > 0)` also forces a price to exist. Wrong: `NULL > 0` is unknown, and a CHECK only rejects false. Right: `price INTEGER NOT NULL CHECK (price > 0)`.

Problem 1.5.3 Why is a float column dangerous for money? Add up ten payments of 0.10 and compare with 1.00. Then store the same payments as whole paise.

Idea	Computers store `REAL` / `FLOAT` numbers in binary. The decimal 0.1 has no exact binary form, just as 1/3 has no exact decimal form. Small errors appear, and they grow when you add many numbers.
Plan	Show the error in plain Python and inside SQLite. Then show two safe options: exact decimals (`NUMERIC` , Python `Decimal`) and integers counting the smallest unit (paise or cents).

```
import sqlite3
from decimal import Decimal

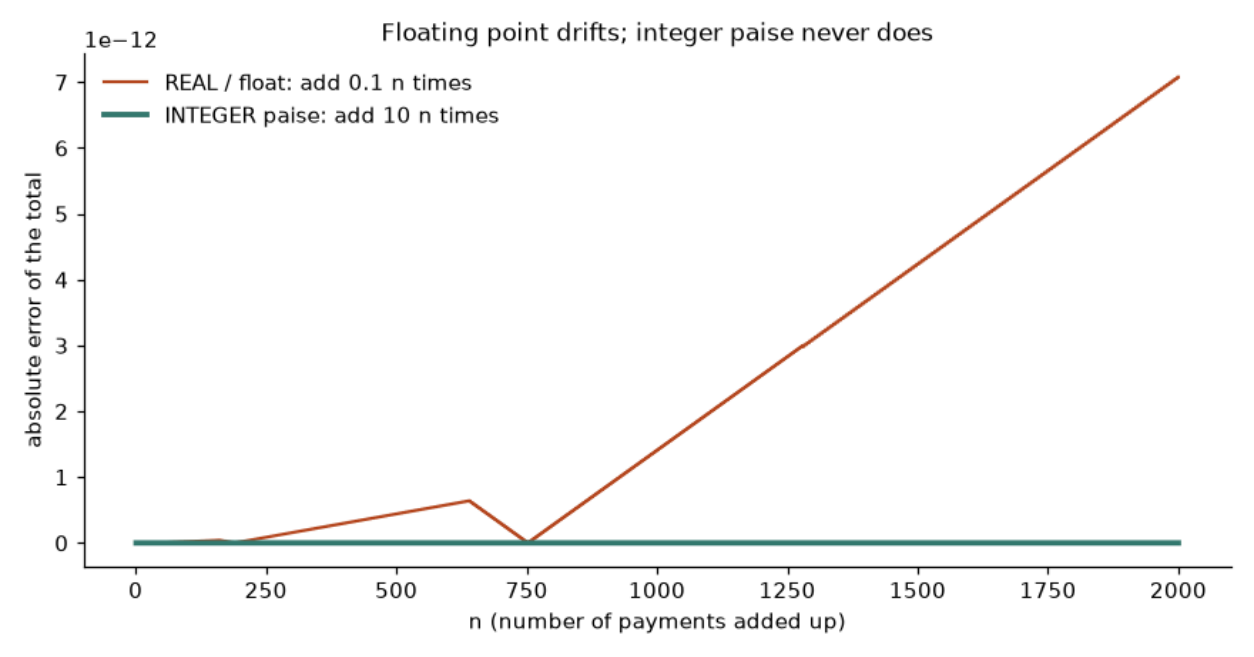
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
total = 0.0
for _ in range(10):
    total += 0.1
print(total, total == 1.0)

con = sqlite3.connect(":memory:")
print(con.execute("SELECT 0.1 + 0.2 = 0.3, 10 + 20 = 30").fetchone())

print(Decimal("0.1") + Decimal("0.2"))
paise = [10] * 10
print(sum(paise), sum(paise) / 100)
```

```
0.30000000000000004 False
0.9999999999999999 False
(0, 1)
0.3
100 1.0
```

1. Line 4: the sum of two floats is 0.30000000000000004, so the comparison with 0.3 is False.
2. Lines 5 to 8: adding 0.1 ten times does not give exactly 1.0.
3. Line 11: SQLite uses the same binary floats for REAL, so `0.1 + 0.2 = 0.3` is 0 (false) in SQL too, while the integer version is 1 (true).
4. Line 13: `Decimal` works in base 10 and is exact. In PostgreSQL the same idea is `NUMERIC(12,2)`.
5. Lines 14 and 15: ten payments of 10 paise add up to exactly 100 paise, which is 1.00 rupee.



Adding 0.1 up to 2,000 times. The orange line is the absolute error of a float total (tiny, about 7e-12 at the worst point, but never reliably zero). The teal line is the same sum in integer paise: always exactly zero.

How to read it: the horizontal axis counts how many payments were added. The vertical axis is how far the computed total is from the true total. The float error is small, which is why the bug survives testing, but equality checks such as `total = 1.00` and reconciliation reports that must match to the paisa fail at random. The integer line is flat at zero.

Answer	Floats give 0.30000000000000004 and 0.9999999999999999; exact types give 0.3 and 100 paise.
Meaning	Use `NUMERIC(p,s)` or integer paise for money, never `REAL` or `FLOAT`. Floats are fine for measurements such as temperature or a model score.

COMMON MISTAKE	Storing prices as `FLOAT` and filtering `WHERE total = 19.99`. The stored value may be 19.989999999999998 and the row never matches. Store integer paise (1999), or `NUMERIC(10,2)`.

Problem 1.5.4 You declare `n INTEGER` in SQLite and insert the text `'abc'`. What happens? How does PostgreSQL behave, and how do you make SQLite strict?

Idea	SQLite columns have a type *affinity* (a preference), not a hard type. It tries to convert the value to the preferred type and, if it cannot, stores the value as it is. `typeof()` reveals what was really stored.
Fix	Create the table with the `STRICT` keyword (SQLite 3.37 or newer). Then the declared type is enforced, as in PostgreSQL.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE loose (n INTEGER)")
con.execute("INSERT INTO loose VALUES ('42'), ('abc'), (3.0), (3.5)")
print(con.execute("SELECT n, typeof(n) FROM loose").fetchall())

con.execute("CREATE TABLE tight (n INTEGER) STRICT")
try:
    con.execute("INSERT INTO tight VALUES ('abc')")
except sqlite3.Error as e:
    print("STRICT says:", e)
```

```
[(42, 'integer'), ('abc', 'text'), (3, 'integer'), (3.5, 'real')]
STRICT says: cannot store TEXT value in INTEGER column tight.n
```

1. The first table is an ordinary SQLite table with INTEGER affinity.
2. The four inserted values are the text '42', the text 'abc', the real 3.0 and the real 3.5.
3. `typeof(n)` reports the storage class of each stored value. Text that looks like an integer is converted (42 becomes an integer), 3.0 is converted to the integer 3 because it loses nothing, 3.5 stays real, and 'abc' stays text inside an INTEGER column.
4. The second table adds `STRICT`, so inserting 'abc' is refused with a datatype error.

Answer	Plain SQLite accepts 'abc' into an INTEGER column. PostgreSQL would refuse it (`invalid input syntax for type integer`). A STRICT SQLite table refuses it too.
Meaning	Practice on SQLite but think in the strict rules of PostgreSQL. A query that works on sloppy data in SQLite may fail on a real server, or worse, may sum text as zero.

Types you will use all the time

You want to store	PostgreSQL type	SQLite practice	Warning
Whole numbers, ids, counts	`INTEGER` , `BIGINT`	`INTEGER`	Use `BIGINT` if you may pass 2.1 billion rows
Money	`NUMERIC(12,2)`	integer paise	Never float
Measurements, scores	`DOUBLE PRECISION`	`REAL`	Approximate
Names, descriptions	`TEXT` or `VARCHAR(n)`	`TEXT`	Length limits are optional in PostgreSQL
Yes or no	`BOOLEAN`	`INTEGER` 0 or 1	SQLite stores true as 1
Calendar day	`DATE`	text `'2024-01-05'`	ISO format sorts correctly as text
Moment in time	`TIMESTAMPTZ`	text or integer	Store UTC, convert at display

Connection to the real job

A data engineer's first defence against bad dashboards is a table that cannot hold bad rows. When you inherit a table full of impossible values (negative ages, orders from the year 1900), the usual root cause is a missing constraint. When you design a new table, write the constraints on day one: adding them later means cleaning every bad row first.

Applications

- Mandatory and unique business identifiers (email, SKU, invoice number)
- Allowed-value lists with `CHECK (status IN ('paid','refunded','cancelled'))`
- Safe money columns for finance data
- Audit columns such as `created_at DEFAULT CURRENT_TIMESTAMP`

Check yourself

1. Which rule makes a column mandatory?

Answer: `NOT NULL`.

1. A column has `CHECK (age >= 0)` and no NOT NULL. Can a row have a NULL age?

Answer: Yes. The check is unknown for NULL, and unknown passes.

1. Which type should hold a price in rupees with two decimals?

Answer: `NUMERIC(12,2)`, or integer paise. Never `REAL`.

1.6 Keys in Depth: Surrogate, Composite, Self-Referencing and What Happens on Delete

INTUITION

A key is a name tag. A **primary key** is the tag the table prints itself; a **foreign key** is a tag copied onto another table so that the two can find each other. Two questions cover almost every key problem: *What makes one row different from every other row?* and *Which row in another table does this row belong to?*

Four kinds of key and when each is right

Kind	Definition	Good for	Watch out
Natural key	A real-world value that is unique (passport number, ISBN, country code)	Small stable reference lists	Values change (emails) or turn out not to be unique (names)
Surrogate key	A meaningless number the database hands out (1, 2, 3)	Almost every business table	Add a `UNIQUE` on the natural value too, or duplicates creep in
Composite key	Two or more columns that are unique *together*	Bridge tables such as `order_items(order_id, product_id)`	Every child that references it must carry all the columns
Candidate key	Any column set that could serve as the key	Choosing the primary key	You pick one as primary; the rest get `UNIQUE`

Worked examples

Problem 1.6.1 What id does the next row get after you delete the row with the highest id? Compare `INTEGER PRIMARY KEY` with `INTEGER PRIMARY KEY AUTOINCREMENT`.

Step 1	In SQLite an `INTEGER PRIMARY KEY` column is an alias for the internal row number (`rowid`). A new row gets the current maximum plus one.
Step 2	So if you delete the maximum, the next insert reuses that number.
Step 3	`AUTOINCREMENT` keeps a counter that never goes down, so the number is never reused.
Predict	Table `a` (plain): ids 1,2,3, delete 3, insert, expect 3. Table `b` (AUTOINCREMENT): expect 4.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE a (id INTEGER PRIMARY KEY, v TEXT)")
con.execute("CREATE TABLE b (id INTEGER PRIMARY KEY AUTOINCREMENT, v TEXT)")
for t in ("a", "b"):
    for v in ("x", "y", "z"):
        con.execute(f"INSERT INTO {t}(v) VALUES (?)", (v,))
    con.execute(f"DELETE FROM {t} WHERE id = 3")
    cur = con.execute(f"INSERT INTO {t}(v) VALUES ('new')")
    print(t, "new row got id", cur.lastrowid)
```

```
a new row got id 3
b new row got id 4
```

1. Table `a` uses the plain form, table `b` the AUTOINCREMENT form.
2. The inner loop inserts three rows without naming an id, so the database assigns 1, 2, 3.
3. The `DELETE` removes the row with the highest id, 3.
4. `cur.lastrowid` is the id the database assigned to the row we just inserted.

Answer	Table a reuses 3. Table b gives 4.
Meaning	If another system saved "order 3" in a log, a reused id can attach that log to the wrong row. PostgreSQL identity columns and sequences (`GENERATED ALWAYS AS IDENTITY`) never reuse a number, even after deletes or rolled-back inserts, though they can leave gaps. Never assume ids are consecutive.

Problem 1.6.2 Students enrol in courses. A student can take many courses and a course has many students, but a student may enrol in the same course only once. Design the key and test it.

Step 1	Many-to-many, so a bridge table `enrolments` .
Step 2	One row is one (student, course) pair. The pair is what must be unique, so the primary key is the composite `(student_id, course_id)` .
Step 3	Each part is also a foreign key to its parent table.
Step 4	Test: the same pair twice must fail; the same student with a different course must pass.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE students (student_id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE courses  (course_id  INTEGER PRIMARY KEY, title TEXT NOT NULL);
CREATE TABLE enrolments (
  student_id INTEGER NOT NULL REFERENCES students(student_id),
  course_id  INTEGER NOT NULL REFERENCES courses(course_id),
  grade      INTEGER,
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO students VALUES (1,'Asha'),(2,'Ravi');
INSERT INTO courses  VALUES (10,'SQL'),(20,'Python');
""")
attempts = [(1,10), (1,20), (1,10), (2,10), (3,10)]
for s, c in attempts:
    try:
        con.execute("INSERT INTO enrolments(student_id, course_id) VALUES (?,?)", (s, c))
        print((s, c), "enrolled")
    except sqlite3.Error as e:
        print((s, c), "refused:", e)
```

```
(1, 10) enrolled
(1, 20) enrolled
(1, 10) refused: UNIQUE constraint failed: enrolments.student_id, enrolments.course_id
(2, 10) enrolled
(3, 10) refused: FOREIGN KEY constraint failed
```

1. `PRAGMA foreign_keys = ON` must run on every new SQLite connection, or the REFERENCES rules are silently ignored.
2. Two parent tables are created, each with its own single-column key.
3. `PRIMARY KEY (student_id, course_id)` after the column list declares the composite key. The two columns together must be unique; either may repeat on its own.
4. The attempts: (1,10) first time, (1,20) same student new course, (1,10) the duplicate pair, (2,10) new student same course, (3,10) a student who does not exist.

Answer	Attempts 1, 2 and 4 pass. Attempt 3 fails the composite key. Attempt 5 fails the foreign key.
Check	Three rows are stored. Student 1 appears twice and course 10 appears twice, which proves each column alone is *not* unique, only the pair is.

Problem 1.6.3 You delete department 10, which still has employees. What can the database do? Test the three common foreign-key actions.

Idea	When a parent row is deleted, the foreign key `ON DELETE` action decides the fate of the child rows. Default (`NO ACTION` / `RESTRICT`): refuse the delete. `CASCADE` : delete the children too. `SET NULL` : keep the children, blank the link.
Plan	Build the same two tables three times, once per action, delete department 10, and list the employees that remain.

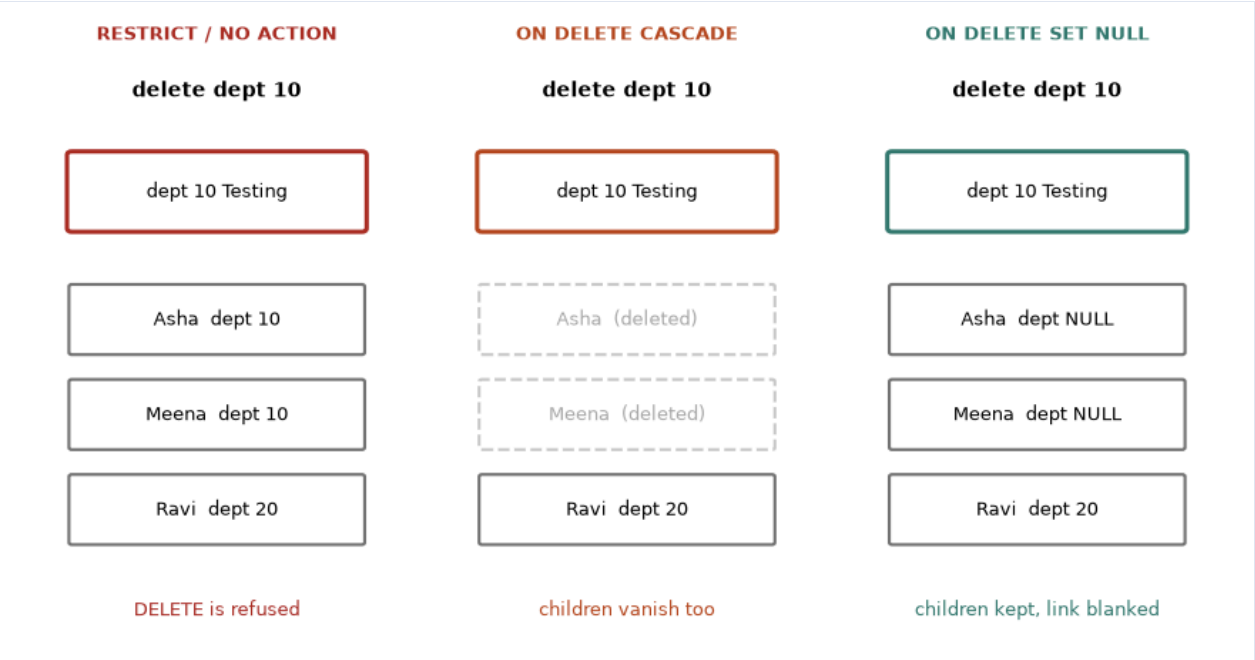
```
import sqlite3

def trial(action):
    con = sqlite3.connect(":memory:")
    con.execute("PRAGMA foreign_keys = ON")
    con.executescript(f"""
    CREATE TABLE departments (dept_id INTEGER PRIMARY KEY, dept_name TEXT);
    CREATE TABLE employees (
      emp_id INTEGER PRIMARY KEY, name TEXT,
      dept_id INTEGER REFERENCES departments(dept_id) {action});
    INSERT INTO departments VALUES (10,'Testing'),(20,'Data');
    INSERT INTO employees VALUES (1,'Asha',10),(2,'Meena',10),(3,'Ravi',20);
    """)
    try:
        con.execute("DELETE FROM departments WHERE dept_id = 10")
        result = "delete ok"
    except sqlite3.Error as e:
        result = "delete refused: " + str(e)
    rows = con.execute("SELECT name, dept_id FROM employees ORDER BY emp_id").fetchall()
    return result, rows

for action in ("", "ON DELETE CASCADE", "ON DELETE SET NULL"):
    print(repr(action or "default"), trial(action))
```

```
'default' ('delete refused: FOREIGN KEY constraint failed', [('Asha', 10), ('Meena', 10), ('Ravi', 20)])
'ON DELETE CASCADE' ('delete ok', [('Ravi', 20)])
'ON DELETE SET NULL' ('delete ok', [('Asha', None), ('Meena', None), ('Ravi', 20)])
```

1. `trial(action)` builds a fresh database so the three experiments cannot affect each other. The `{action}` text is pasted into the foreign key definition.
2. Departments 10 and 20 exist. Asha and Meena are in 10, Ravi in 20.
3. The delete is wrapped in `try` because the default action raises an error.
4. After the attempt we read the employees back, so the output shows what happened to the children.



The three foreign-key delete actions applied to department 10. The same data, three different outcomes.

How to read it: each column starts with the same department row on top and three employees below. Under RESTRICT nothing changes because the delete is refused. Under CASCADE Asha and Meena are removed together with their department. Under SET NULL both stay but their `dept_id` becomes NULL, so they are now unassigned.

Answer	Default refuses and keeps all 3 employees. CASCADE leaves only Ravi. SET NULL leaves 3 employees, two with a NULL department.
Meaning	Choose by business meaning. Cascade suits true parts (order lines die with their order). Restrict suits things you must not lose silently (a customer with orders). SET NULL suits optional links (an employee between departments).

COMMON MISTAKE	Using `ON DELETE CASCADE` on a link to a big shared table "to make deletes work". One careless `DELETE` on a customer then silently erases years of orders and payments. Prefer the default refusal for business history and delete deliberately, child first.

Problem 1.6.4 Employees have a manager who is also an employee. Where does the foreign key point, and how does the boss of everyone fit in?

Idea	A table can reference itself. `manager_id` holds the `emp_id` of another row of the same table. The top person has nobody above, so that cell is NULL, which is exactly why a foreign key column is allowed to be NULL.
Query	To show each employee beside the manager's name we use the table twice under two aliases (`e` and `m`) and attach the manager row with a `LEFT JOIN` . Read it as "for each employee, add the manager's row if there is one; leave blanks if not". Chapter 4 teaches joins fully.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE employees (
  emp_id     INTEGER PRIMARY KEY,
  name       TEXT NOT NULL,
  manager_id INTEGER REFERENCES employees(emp_id)
);
INSERT INTO employees VALUES (1,'Kavya',NULL),(2,'Asha',1),(3,'Ravi',1),(4,'Meena',2);
""")
q = """SELECT e.name, m.name AS manager
       FROM employees e LEFT JOIN employees m ON m.emp_id = e.manager_id
       ORDER BY e.emp_id"""
for row in con.execute(q):
    print(row)
```

```
('Kavya', None)
('Asha', 'Kavya')
('Ravi', 'Kavya')
('Meena', 'Asha')
```

1. `REFERENCES employees(emp_id)` sits inside the `employees` table itself. The row must exist before another row points to it, which is why Kavya (manager NULL) is inserted first.
2. In the query, `e` plays the employee and `m` plays that employee's manager. The condition `m.emp_id = e.manager_id` connects them.
3. `LEFT JOIN` keeps Kavya even though no manager row matches her NULL. An ordinary `JOIN` would silently drop her.

Answer	Four rows: Kavya has manager None (NULL), Asha and Ravi report to Kavya, Meena reports to Asha.
Meaning	Org charts, category trees and comment threads are all self-references. Walking several levels needs a recursive query (Chapter 5).

Problem 1.6.5 You receive a table and a colleague says "order_id is the key". Prove or disprove it with a query, then find the real key.

Idea	A column set is a valid key only if the number of rows equals the number of distinct values of that set. Two counts settle it.
Step 1	Compare `COUNT(*)` with `COUNT(DISTINCT order_id)`.
Step 2	If they differ, try the pair: count distinct `(order_id, product_id)` combinations.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE order_items (order_id INT, product_id INT, qty INT);
INSERT INTO order_items VALUES (101,1,1),(101,3,2),(102,2,1),(103,5,1),(103,4,1),(104,3,5);
""")
print(con.execute("SELECT COUNT(*), COUNT(DISTINCT order_id) FROM order_items").fetchone())
print(con.execute("""SELECT COUNT(*) FROM
                     (SELECT DISTINCT order_id, product_id FROM order_items)""").fetchone())
```

```
(6, 4)
(6,)
```

1. The table has no constraints at all, so nothing stops duplicates; we must check the data ourselves.
2. First query: 6 rows but only 4 distinct order ids. So `order_id` is *not* a key: order 101 appears twice.
3. Second query: the inner `SELECT DISTINCT` keeps one copy of each (order_id, product_id) pair, and the outer query counts them. 6 pairs for 6 rows means the pair is unique.

Answer	Not `order_id` alone. The composite `(order_id, product_id)` is unique on this data.
Caution	Passing on today's data does not prove a rule. It only proves nothing currently contradicts it. Decide the key from what a row *means* (one product on one order) and then enforce it with `PRIMARY KEY` .

COMMON MISTAKE	Guessing the grain from a handful of rows, then joining on a "key" that repeats. The join multiplies rows and every total is inflated. Always run the two-count check (`COUNT(*)` versus `COUNT(DISTINCT key)`) before trusting a key.

Connection to the real job

In a warehouse nobody hands you a diagram. You inherit tables called `fct_orders` or `dim_user` and must work out the keys yourself. The two-count check, the composite key test and the question "what does one row mean?" are used on day one of every project. Foreign key actions matter when you design deletion rules for personal data requests (a user asks to be forgotten and their rows in six tables must go in the right order).

Applications

- Surrogate `customer_id` plus `UNIQUE(email)` in user tables
- Composite keys in bridge tables such as tags, enrolments, follows
- Self-references for org charts, referral trees, comment replies
- Delete policies for GDPR style erasure requests

Check yourself

1. Why does the pair `(order_id, product_id)` identify an order line but `order_id` alone does not?

Answer: An order has many lines, so order_id repeats. One product appears once per order, so the pair is unique.

1. What does `ON DELETE SET NULL` do?

Answer: When the parent is deleted, child rows stay and their foreign key column becomes NULL.

1. Why can a self-referencing foreign key be NULL?

Answer: The top row of the hierarchy has no parent. NULL means "no parent".

1.7 The Relational Model Itself: Relations, Sets and Three Core Operations

INTUITION	In 1970 Edgar Codd proposed describing data with one simple idea from mathematics: a **relation**, which is just a set of rows that all have the same columns. Everything else (SQL, joins, keys) is built on that. The payoff: because a table is a set, you can combine tables with a few well-defined operations and always get another table back. That closure is why SQL queries nest and compose.

Vocabulary: three names for the same thing

Theory word	SQL word	Spreadsheet word	Precise meaning
Relation	Table	Sheet	A set of tuples that share the same attributes
Tuple	Row	Row	One combination of values, one per attribute
Attribute	Column	Column	A named property with a domain (its allowed values)
Domain	Data type + CHECK	Validation rule	The set of legal values for an attribute
Degree / cardinality	Number of columns / rows	Width / height	Do not confuse with the relationship word cardinality (1:N) from Section 1.4

Three consequences of "a table is a set" explain many surprises:

1. **Rows have no order.** A table is not a list. Without `ORDER BY` the database may return rows in any order, and the order may change after an index or a vacuum.
2. **Columns have no meaningful position.** You identify a column by name. `SELECT *` in a program that reads columns by position breaks when someone adds a column.
3. **Rows should be unique.** A mathematical set has no repeated members. SQL is looser: a table without a key can hold two identical rows (it is then a "bag"). A primary key restores set behaviour.

The three core operations

Operation	Symbol	What it does	SQL
Selection (restrict)	σ	Keeps the *rows* that satisfy a condition	`WHERE`
Projection	π	Keeps chosen *columns* and removes repeated rows	column list (plus `DISTINCT`)
Join	⋈	Pairs up rows from two tables that match on a condition	`JOIN ... ON`

A join is conceptually two steps: form every possible pair of rows (the **Cartesian product**), then keep the pairs that satisfy the match condition. Real engines do not build the full product, but thinking in that order lets you predict row counts.

Worked examples

Problem 1.7.1 Using the employees table (Asha 90000, Ravi 120000, Meena 95000) write selection, projection and join once each, in algebra and in SQL.

Selection	$\sigma_{\text{salary} \geq 95000}$(employees) is `SELECT * FROM employees WHERE salary >= 95000`. Expect Ravi and Meena.
Projection	$\pi_{\text{dept_id}}$(employees) is `SELECT DISTINCT dept_id FROM employees`. Three employees but two departments, so expect two rows.
Join	employees ⋈ departments is `FROM employees e JOIN departments d ON d.dept_id = e.dept_id`. Each employee finds exactly one department, so expect three rows.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE departments (dept_id INTEGER PRIMARY KEY, dept_name TEXT NOT NULL);
CREATE TABLE employees (emp_id INTEGER PRIMARY KEY, name TEXT, dept_id INTEGER, salary INTEGER);
INSERT INTO departments VALUES (10,'Testing'),(20,'Data');
INSERT INTO employees VALUES (1,'Asha',10,90000),(2,'Ravi',20,120000),(3,'Meena',10,95000);
""")
print("selection :", con.execute("SELECT name FROM employees WHERE salary >= 95000").fetchall())
print("projection:", con.execute("SELECT DISTINCT dept_id FROM employees").fetchall())
print("join      :", con.execute("""SELECT e.name, d.dept_name
                                   FROM employees e JOIN departments d ON d.dept_id = e.dept_id
                                   ORDER BY e.emp_id""").fetchall())
```

```
selection : [('Ravi',), ('Meena',)]
projection: [(10,), (20,)]
join      : [('Asha', 'Testing'), ('Ravi', 'Data'), ('Meena', 'Testing')]
```

1. The two `executescript` statements build and fill the tables in one call.
2. *Selection* returns only rows. Ravi (120000) and Meena (95000) pass the test, Asha (90000) does not. Note that the condition uses `>=`, so exactly 95000 passes.
3. *Projection* keeps one column. Without `DISTINCT` you would get 10, 20, 10. `DISTINCT` collapses repeats, as a mathematical projection does.
4. *Join* glues the department name onto each employee. `e` and `d` are short table aliases.

Answer	Selection: Ravi, Meena. Projection: 10 and 20. Join: three pairs, one per employee.
Check	Row counts 2, 2 and 3 equal the predictions.

Problem 1.7.2 Show that a join is "Cartesian product, then selection" by counting rows at each stage.

Step 1	Product: every employee paired with every department: 3 × 2 = 6 pairs.
Step 2	Selection keeps only pairs where the department ids agree. Each employee agrees with exactly one department, so 3 pairs survive.
Step 3	Compare with the join: also 3.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE departments (dept_id INTEGER PRIMARY KEY, dept_name TEXT);
CREATE TABLE employees (emp_id INTEGER PRIMARY KEY, name TEXT, dept_id INTEGER);
INSERT INTO departments VALUES (10,'Testing'),(20,'Data');
INSERT INTO employees VALUES (1,'Asha',10),(2,'Ravi',20),(3,'Meena',10);
""")
product = con.execute("SELECT COUNT(*) FROM employees e CROSS JOIN departments d").fetchone()[0]
matched = con.execute("""SELECT COUNT(*) FROM employees e CROSS JOIN departments d
                         WHERE d.dept_id = e.dept_id""").fetchone()[0]
joined  = con.execute("""SELECT COUNT(*) FROM employees e
                         JOIN departments d ON d.dept_id = e.dept_id""").fetchone()[0]
print(product, matched, joined)
```

```
6 3 3
```

1. `CROSS JOIN` is the Cartesian product: no condition, every pairing. 3 × 2 = 6.
2. Adding `WHERE d.dept_id = e.dept_id` filters the pairs: 3 remain.
3. `JOIN ... ON` states the same thing in one clause and also gives 3.

Answer	6, 3, 3.
Meaning	The row count of an inner join can never exceed (rows in A) × (rows in B). If a join returns more rows than either table has, the match column repeats on both sides, and totals will be inflated. Chapter 4 shows this fan-out in detail.

COMMON MISTAKE	Writing `FROM a, b` and forgetting the matching condition. The result is the full product: 1,000 customers by 50,000 orders is 50 million rows, and the query appears to hang. Always put the matching condition in `ON`.

Problem 1.7.3 Does `SELECT city FROM t` return rows in the order you inserted them? Test it, then add an index and run the same query again.

Idea	The relational model promises no order. The engine returns rows in whatever order is cheapest. Often that is insertion order, which tempts people to rely on it.
Plan	Insert Pune, Delhi, Mumbai, Chennai. Run the query. Create an index on `city`. The engine can now answer from the index alone, which is stored sorted by city.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, city TEXT, note TEXT)")
con.executemany("INSERT INTO t(city, note) VALUES (?, ?)",
                [("Pune", "a" * 40), ("Delhi", "b" * 40), ("Mumbai", "c" * 40), ("Chennai", "d" * 40)])
print("before index:", [r[0] for r in con.execute("SELECT city FROM t")])
con.execute("CREATE INDEX t_city ON t(city)")
print("after index :", [r[0] for r in con.execute("SELECT city FROM t")])
print("with ORDER BY:", [r[0] for r in con.execute("SELECT city FROM t ORDER BY city")])
```

```
before index: ['Pune', 'Delhi', 'Mumbai', 'Chennai']
after index : ['Chennai', 'Delhi', 'Mumbai', 'Pune']
with ORDER BY: ['Chennai', 'Delhi', 'Mumbai', 'Pune']
```

1. The table has an id, a city and a wide `note` column (40 characters), and four rows are inserted in a deliberate non-alphabetical order. The wide column matters: it makes the table bigger than a small index on city.
2. First query: the engine scans the table, so rows come back in insertion order.
3. `CREATE INDEX` builds a sorted side structure on `city`. The planner sees that the narrow index already contains every column the query needs (a *covering index*), reads it instead of the wide table, and so the rows come out sorted by city. Nothing in the data changed. (Whether the planner makes this choice depends on table size and statistics, which is exactly why you cannot rely on it.)
4. Only the last query guarantees order, because it says so.

Answer	Same query, same data, different order before and after the index. Only `ORDER BY` promises an order.
Meaning	A report that "worked for a year" can reorder itself after a DBA adds an index or after the data grows. Always write `ORDER BY` when order matters, and never use "the first row" without it.

COMMON MISTAKE

Using `SELECT ... LIMIT 1` without `ORDER BY` to get "the latest" or "the first" row. It returns an arbitrary row that happens to look right in testing. Add `ORDER BY created_at DESC` (and a tie-breaker).

Three kinds of integrity

Integrity	Promise	Enforced by
Entity integrity	Every row is identifiable and unique	Primary key (unique, not NULL)
Referential integrity	Every foreign key points to a real parent (or is NULL)	Foreign key constraint
Domain integrity	Every value is legal for its column	Data type, NOT NULL, CHECK, DEFAULT

Connection to the real job

Every pandas operation has a relational twin: `df[df.salary >= 95000]` is selection, `df[["dept_id"]].drop_duplicates()` is projection, `df.merge(depts, on="dept_id")` is a join. Thinking in these three operations lets you move between SQL, pandas and Spark without relearning, and it lets you estimate result sizes before running anything.

Applications

- Translating a business question into selection, projection and join steps
- Predicting row counts to catch accidental fan-out
- Explaining why a result order changed after a deployment

Check yourself

1. Which SQL clause is selection and which is projection?

Answer: `WHERE` filters rows (selection). The column list in `SELECT` chooses columns (projection).

1. How many rows does `CROSS JOIN` of a 5-row and a 7-row table give?

Answer: 35.

1. Is the row order of a table defined?

Answer: No. Only `ORDER BY` defines an order in the result.

1.8 Why We Split Tables: Anomalies, Atomic Values and a Lossless Split

INTUITION	Section 1.4 said "one fact in one place". Here you will see what goes wrong when a fact lives in many places, by breaking a flat table on purpose. A flat file is perfect for reading and terrible for changing. The three ways it breaks have names: the **update**, **insert** and **delete** anomalies.

The flat file and its three anomalies

Your manager sends `sales.csv` : one line per order, with the customer's details and the product copied onto every line. We load it as one table and then try three ordinary jobs.

Problem 1.8.1 Asha moves from Pune to Mumbai, a new customer Zoya registers but has not ordered, and Tom's only order is cancelled and deleted. What goes wrong in the flat table?

Update anomaly	Asha's city is stored on every one of her orders. If the update touches only one of them, the data contradicts itself.
Insert anomaly	The table's rows are orders. A customer with no order has no row to live in (the order columns would have to be blank, and `order_id` is mandatory).
Delete anomaly	Deleting Tom's only order deletes everything we know about Tom: his name, city and email.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE sales (
  order_id INTEGER PRIMARY KEY, customer_name TEXT NOT NULL, customer_city TEXT,
  customer_email TEXT, product_name TEXT NOT NULL, price INTEGER NOT NULL);
INSERT INTO sales VALUES
 (101,'Asha','Pune','asha@x.com','Keyboard',1500),
 (103,'Asha','Pune','asha@x.com','Desk Lamp',900),
 (106,'Asha','Pune','asha@x.com','Mouse',600),
 (102,'Ravi','Mumbai','ravi@x.com','Mouse',600),
 (120,'Tom','Pune','tom@x.com','Pen Pack',120);
""")
# 1. update anomaly: the careless UPDATE touches only the order we were looking at
con.execute("UPDATE sales SET customer_city = 'Mumbai' WHERE order_id = 101")
print("update:", con.execute("""SELECT DISTINCT customer_name, customer_city
                              FROM sales WHERE customer_name = 'Asha'""").fetchall())
# 2. insert anomaly: a customer with no order cannot be stored
try:
    con.execute("INSERT INTO sales(customer_name, customer_city, customer_email) VALUES ('Zoya','Delhi','z@x.com')")
except sqlite3.Error as e:
    print("insert:", e)
# 3. delete anomaly: removing Tom's only order removes Tom
con.execute("DELETE FROM sales WHERE order_id = 120")
print("delete:", con.execute("SELECT COUNT(*) FROM sales WHERE customer_name = 'Tom'").fetchone()[0], "rows left for Tom")
```

```
update: [('Asha', 'Mumbai'), ('Asha', 'Pune')]
insert: NOT NULL constraint failed: sales.product_name
delete: 0 rows left for Tom
```

1. The table repeats name, city and email on every Asha line. Three Asha rows means the city is stored three times.
2. `UPDATE ... WHERE order_id = 101` fixes one row only. The query below lists the distinct (name, city) pairs for Asha and finds two cities.
3. The insert for Zoya names no `order_id` and no product. The primary key could be auto-assigned, but `product_name` and `price` are NOT NULL, so the database refuses. Making them nullable would "work" but pollute the table with rows that are not orders.
4. After deleting order 120 no row mentions Tom at all. His email is gone for good.

Answer	Asha now lives in both Mumbai and Pune. Zoya cannot be stored. Tom is erased with his order.
Meaning	All three failures disappear if customers, products and orders are separate tables, because each fact is stored once and orders merely point at it.

Problem 1.8.2 Split the flat `sales` table into `customers`, `products` and `orders`, then prove that no information was lost.

Step 1	Find what each fact is "about". Name, city, email are about the customer. Product name and price are about the product. Only the link (which customer ordered which product on which order) belongs to the order.
Step 2	Create `customers` with `INSERT ... SELECT DISTINCT`, so each customer is stored once, with a new surrogate id.
Step 3	Create `products` the same way.
Step 4	Create `orders` holding the order id and two foreign keys. Find the foreign key values by joining back on the natural values (email for customers, name for products).
Step 5	**Prove it is lossless**: join the three tables back together and compare with the original rows. If they are identical sets, nothing was lost and nothing was invented.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE sales (order_id INTEGER PRIMARY KEY, customer_name TEXT, customer_city TEXT,
                    customer_email TEXT, product_name TEXT, price INTEGER);
INSERT INTO sales VALUES
 (101,'Asha','Pune','asha@x.com','Keyboard',1500),
 (103,'Asha','Pune','asha@x.com','Desk Lamp',900),
 (106,'Asha','Pune','asha@x.com','Mouse',600),
 (102,'Ravi','Mumbai','ravi@x.com','Mouse',600),
 (120,'Tom','Pune','tom@x.com','Pen Pack',120);

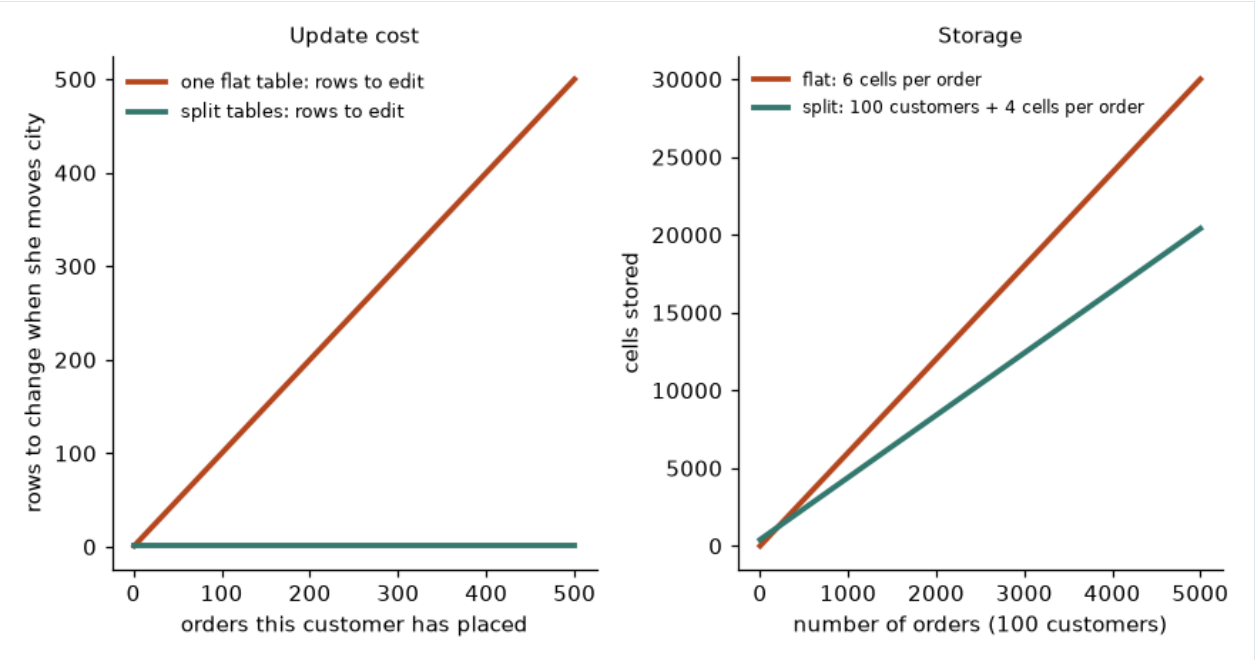
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT NOT NULL,
                        city TEXT, email TEXT NOT NULL UNIQUE);
INSERT INTO customers(name, city, email)
  SELECT DISTINCT customer_name, customer_city, customer_email FROM sales;

CREATE TABLE products (product_id INTEGER PRIMARY KEY, name TEXT NOT NULL UNIQUE, price INTEGER NOT NULL);
INSERT INTO products(name, price) SELECT DISTINCT product_name, price FROM sales;

CREATE TABLE orders (order_id INTEGER PRIMARY KEY,
                     customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
                     product_id  INTEGER NOT NULL REFERENCES products(product_id));
INSERT INTO orders
  SELECT s.order_id, c.customer_id, p.product_id
  FROM sales s
  JOIN customers c ON c.email = s.customer_email
  JOIN products  p ON p.name  = s.product_name;
""")
for t in ("sales", "customers", "products", "orders"):
    print(t, con.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0])
original = con.execute("SELECT order_id, customer_name, customer_city, customer_email, product_name, price FROM
sales").fetchall()
rebuilt = con.execute("""SELECT o.order_id, c.name, c.city, c.email, p.name, p.price
                         FROM orders o JOIN customers c USING (customer_id)
                                       JOIN products  p USING (product_id)""").fetchall()
print("lossless:", sorted(original) == sorted(rebuilt))
```

```
sales 5
customers 3
products 4
orders 5
lossless: True
```

1. The flat table now has a consistent Asha (all Pune). The Mouse price is the same on both lines (600); if two lines for the same product had different prices, `SELECT DISTINCT name, price` would make two products with one name and the `UNIQUE` on name would stop the split. That failure is useful: it tells you the flat file already contradicted itself.
2. `INSERT INTO customers(name, city, email) SELECT DISTINCT ...` copies the unique combinations. The id column is omitted, so SQLite numbers the rows. Asha's three rows collapse into one.
3. The same pattern builds `products`: Keyboard, Desk Lamp, Mouse, Pen Pack.
4. For `orders`, the join looks up the new surrogate ids using the old natural values. This is how you migrate real data: keep a natural value to match on until the foreign keys are filled.
5. The loop prints row counts: 5 flat rows become 3 customers, 4 products and still 5 orders.
6. The last part rebuilds the flat shape with two joins and compares it with the original after sorting both lists. `USING (customer_id)` is shorthand for `ON o.customer_id = c.customer_id`.



Left: how many rows you must edit when one customer changes city. Right: how many cells are stored when 100 customers place up to 5,000 orders.

How to read it: on the left, the orange line is the flat table. A customer with 500 orders means 500 edits, and missing one creates the contradiction you saw above. The teal line is the split design: one edit, always. On the right, splitting also stores fewer cells once orders pile up, because the customer details are written 100 times instead of once per order.

Answer	Counts: sales 5, customers 3, products 4, orders 5. The check prints `lossless: True`.
Meaning	A good split is one you can undo with joins and get the original back. Always run this proof when you restructure data. A decomposition that fails it has lost rows or invented new ones.

COMMON MISTAKE	Splitting a table and never comparing it with the original. Typical silent losses: a customer whose email was typed in two spellings becomes two customers; a join on a column with NULLs drops those rows. Count rows before and after, and compare the rebuilt result with the source.

Problem 1.8.3 A column `skills` holds `'Java, SQL'`, `'NoSQL, Python'` and `'Excel'`. You want everyone who knows SQL. What does `LIKE '%SQL%'` return, and what is the clean design?

Idea	First normal form (1NF) says every cell holds *one* atomic value. A comma list breaks that, so every query must parse text.
Predict	`'%SQL%'` means "contains the letters SQL anywhere". 'NoSQL, Python' contains them, so Meena will be wrongly counted.
Fix	Use a bridge table with one row per (employee, skill) and test equality: `skill = 'SQL'`.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE people (name TEXT, skills TEXT);
INSERT INTO people VALUES ('Asha','Java, SQL'),('Meena','NoSQL, Python'),('Ravi','Excel');
CREATE TABLE person_skill (name TEXT, skill TEXT, PRIMARY KEY (name, skill));
INSERT INTO person_skill VALUES ('Asha','Java'),('Asha','SQL'),('Meena','NoSQL'),('Meena','Python'),('Ravi','Excel');
""")
print("LIKE on the list :", [r[0] for r in con.execute("SELECT name FROM people WHERE skills LIKE '%SQL%'")])
print("bridge, equality :", [r[0] for r in con.execute("SELECT name FROM person_skill WHERE skill = 'SQL'")])
```

```
LIKE on the list : ['Asha', 'Meena']
bridge, equality : ['Asha']
```

1. `people` is the bad design: one text cell per person.
2. `person_skill` is the good design. The composite key stops the same skill being listed twice for a person.
3. The first query matches Asha and Meena, because 'NoSQL' contains 'SQL'. The second matches only Asha.

Answer	The list design returns Asha and Meena (wrong). The bridge design returns Asha (right).
Meaning	Besides wrong answers, a list cell cannot be indexed, cannot carry a foreign key, and cannot enforce uniqueness. Whenever you see "tags", "roles" or "categories" in one column, expect a bug.

COMMON MISTAKE	Storing several values in one cell "to keep it simple" and searching with `LIKE`. It matches substrings (NoSQL contains SQL) and cannot be indexed. Use one row per value in a bridge table.

The next level: normal forms in two minutes

Form	Rule in plain words	Violation you can spot
1NF	One value per cell; no repeating groups; rows identifiable	A skills list in one cell; columns skill1, skill2, skill3
2NF	Every non-key column depends on the *whole* key (matters for composite keys)	`order_items(order_id, product_id, product_name)`: the product name depends on product_id alone
3NF	Non-key columns depend on the key and nothing else	`employees(emp_id, dept_id, dept_name)`: dept_name depends on dept_id, not on emp_id

A memory line: every non-key column should depend on "the key, the whole key and nothing but the key". Chapter 11 develops this fully, including when to break the rules on purpose in a data warehouse.

Connection to the real job

You will meet both worlds. Application databases are normalised so that updates are safe. Warehouses and dashboards often use wide, flat, deliberately repeated tables so that reads are fast. A data analyst reads flat tables; a data engineer builds the pipeline that turns the normalised source into the flat one. Knowing what each choice costs lets you explain a number that does not match between the two.

Applications

- Cleaning a vendor CSV into proper tables
- Explaining why a customer attribute differs between two reports
- Reviewing a design in an interview (spot 1NF, 2NF, 3NF violations)

Check yourself

1. Name the three anomalies of a flat table.

Answer: Update (inconsistent copies), insert (cannot store a fact without an unrelated one), delete (losing a fact by deleting an unrelated one).

1. How do you prove a split lost nothing?

Answer: Join the pieces back together and compare with the original rows.

1. Why does a comma list in a cell break first normal form?

Answer: A cell must hold one atomic value so that it can be compared, indexed and constrained.

1.9 Reading an Unfamiliar Database: Introspection, Counts and Orphans

INTUITION	On your first day at a company nobody gives you a clean diagram. You get a connection string and a vague "the data is in there". A skilled analyst does not guess. The database describes itself: it keeps a catalogue of every table, column, key and index. You query the catalogue first, then the data, then you draw the map.

A ten-minute routine for any new database

1. **List the tables.** What are the nouns of this business?
2. **List the columns and types** of each table, and which are keys.
3. **List the foreign keys.** They are the edges of the map.
4. **Count the rows** of each table. Tiny tables are lookups; huge ones are events.
5. **Check the grain** of the biggest tables with the two-count test from Problem 1.6.5.
6. **Check NULL rates** of the columns you plan to use.
7. **Look for orphans:** child rows whose parent is missing.

Question	SQLite	PostgreSQL (not run here)	MySQL (not run here)
List tables	`SELECT name FROM sqlite_master WHERE type='table'` or `.tables`	`\dt` or `SELECT table_name FROM information_schema.tables WHERE table_schema='public'`	`SHOW TABLES`
Columns and types	`PRAGMA table_info(t)`	`\d t` or `information_schema.columns`	`DESCRIBE t`
Foreign keys	`PRAGMA foreign_key_list(t)`	`\d t` (shows "Foreign-key constraints")	`SHOW CREATE TABLE t`
Indexes	`PRAGMA index_list(t)`	`\di` or `pg_indexes`	`SHOW INDEX FROM t`
Find orphans	`PRAGMA foreign_key_check`	anti-join query (below)	anti-join query (below)

Worked examples

Problem 1.9.1 You are handed a database file. Without any documentation, list its tables, the columns of `orders` with their keys, and the foreign keys of `order_items`.

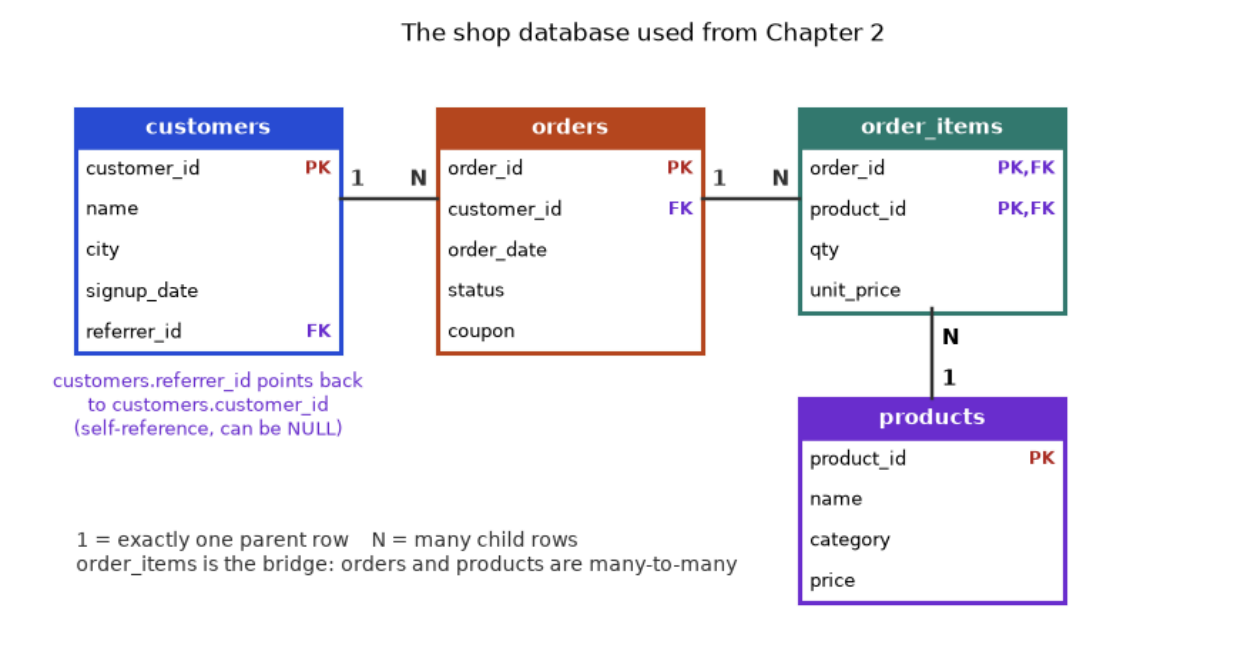
Step 1	Read `sqlite_master`, the catalogue table that holds one row per table and index.
Step 2	`PRAGMA table_info(orders)` returns one row per column: position, name, type, not-null flag, default, and primary-key position.
Step 3	`PRAGMA foreign_key_list(order_items)` returns one row per foreign key column: the parent table and the parent column.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT NOT NULL, city TEXT);
CREATE TABLE products  (product_id INTEGER PRIMARY KEY, name TEXT NOT NULL, price INTEGER NOT NULL);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY,
                     customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
                     order_date TEXT NOT NULL, status TEXT NOT NULL, coupon TEXT);
CREATE TABLE order_items (order_id INTEGER NOT NULL REFERENCES orders(order_id),
                          product_id INTEGER NOT NULL REFERENCES products(product_id),
                          qty INTEGER NOT NULL, PRIMARY KEY (order_id, product_id));
""")
print("tables:", [r[0] for r in con.execute("SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name")])
print("orders columns (cid, name, type, notnull, default, pk):")
for row in con.execute("PRAGMA table_info(orders)"):
    print("  ", row)
print("order_items foreign keys (from column -> parent table.column):")
for row in con.execute("PRAGMA foreign_key_list(order_items)"):
    print("  ", row[3], "->", f"{row[2]}.{row[4]}")
```

```
tables: ['customers', 'order_items', 'orders', 'products']
orders columns (cid, name, type, notnull, default, pk):
   (0, 'order_id', 'INTEGER', 0, None, 1)
   (1, 'customer_id', 'INTEGER', 1, None, 0)
   (2, 'order_date', 'TEXT', 1, None, 0)
   (3, 'status', 'TEXT', 1, None, 0)
   (4, 'coupon', 'TEXT', 0, None, 0)
order_items foreign keys (from column -> parent table.column):
   product_id -> products.product_id
   order_id -> orders.order_id
```

1. We rebuild a four-table shop in memory so the example is self-contained. In real life the file already exists and you only run the queries.
2. `sqlite_master` is a built-in table. Filtering `type = 'table'` hides indexes.
3. Each `table_info` row is a tuple. Read it as (position, name, type, notnull flag, default value, primary-key position). `order_id` has pk 1; the others have 0.
4. Each `foreign_key_list` row has the child column at index 3, the parent table at index 2 and the parent column at index 4. We print them as `child -> parent.column`.

Answer	Four tables; `orders` has five columns with `order_id` as its key and `coupon` the only nullable column besides the key; `order_items` links to orders and products.
Meaning	From these three outputs alone you can draw the map in the figure below, which is the shop database we use from Chapter 2.



The shop database. Boxes are tables, PK marks the primary key, FK marks a foreign key, and the 1 and N labels show the direction of each relationship.

How to read it: follow a line from the "1" end to the "N" end to go from parent to children. One customer has many orders; one order has many order lines; one product appears in many order lines. `order_items` sits between orders and products, so it is the bridge that turns many-to-many into two one-to-many links. The self-pointing `referrer_id` on customers means a customer can be referred by another customer.

Problem 1.9.2 For each table report the row count, and for `orders.coupon` report how many values are missing. Then explain why `COUNT(*)` and `COUNT(coupon)` differ.

Idea	`COUNT(*)` counts rows. `COUNT(column)` counts rows where that column is *not* NULL. Their difference is the NULL count.
Plan	Loop over table names from the catalogue so the same code works for any database, then one query for the NULL count.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER, coupon TEXT);
INSERT INTO customers VALUES (1,'Asha'),(2,'Ravi'),(3,'Meena');
INSERT INTO orders VALUES (101,1,NULL),(102,2,'SAVE10'),(103,1,NULL),(104,3,NULL),(105,2,'SAVE10');
""")
names = [r[0] for r in con.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name")]
for n in names:
    print(n, con.execute(f"SELECT COUNT(*) FROM {n}").fetchone()[0], "rows")
total, with_coupon = con.execute("SELECT COUNT(*), COUNT(coupon) FROM orders").fetchone()
print("orders:", total, "rows,", with_coupon, "with a coupon,", total - with_coupon, "NULL")
```

```
customers 3 rows
orders 5 rows
orders: 5 rows, 2 with a coupon, 3 NULL
```

1. The catalogue query returns the table names as a list, so no name is typed by hand. The f-string is safe here because the names come from the database, not from a user.
2. `COUNT(*)` gives 3 customers and 5 orders.
3. `COUNT(coupon)` skips the three NULLs and returns 2.
4. Subtracting gives the NULL count: 5 - 2 = 3.

Answer	customers 3 rows, orders 5 rows; 2 orders have a coupon and 3 do not.
Meaning	60 percent of orders have no coupon. If an analysis filters `WHERE coupon = 'SAVE10'` the other three rows are not "no coupon" in that test; they are simply absent from it. Know your NULL rates before you filter or average.

Problem 1.9.3 Someone loaded data with foreign keys switched off. Find the orphan orders (orders whose customer does not exist) in two ways.

Way 1	SQLite's built-in `PRAGMA foreign_key_check` lists every violating row.
Way 2	The portable **anti-join**: `LEFT JOIN` children to parents and keep the rows where no parent was found (`parent key IS NULL`). It works in every database.

```
import sqlite3
con = sqlite3.connect(":memory:")          # foreign_keys is OFF by default
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY,
                     customer_id INTEGER REFERENCES customers(customer_id));
INSERT INTO customers VALUES (1,'Asha'),(2,'Ravi');
INSERT INTO orders VALUES (101,1),(102,2),(103,9),(104,1),(105,7);
""")
print("pragma  :", con.execute("PRAGMA foreign_key_check").fetchall())
anti = con.execute("""SELECT o.order_id, o.customer_id
                      FROM orders o LEFT JOIN customers c ON c.customer_id = o.customer_id
                      WHERE c.customer_id IS NULL ORDER BY o.order_id""").fetchall()
print("anti-join:", anti)
```

```
pragma  : [('orders', 103, 'customers', 0), ('orders', 105, 'customers', 0)]
anti-join: [(103, 9), (105, 7)]
```

1. No `PRAGMA foreign_keys = ON` , so SQLite accepts orders 103 and 105 although customers 9 and 7 do not exist. This is exactly how orphans appear in real databases.
2. `foreign_key_check` returns tuples of (child table, child rowid, parent table, foreign key index): the rows with rowid 103 and 105 in `orders` violate the rule (here the rowid equals `order_id`).
3. The anti-join pairs every order with its customer where one exists. Orders 103 and 105 find no customer, so `c.customer_id` is NULL, and the `WHERE` keeps exactly those.
4. Test for `IS NULL` , never `= NULL` ; equals-NULL is never true (Chapter 2).

Answer	Orders 103 (customer 9) and 105 (customer 7) are orphans.
Check	Both methods agree. The pragma reports rowids 103 and 105, because `order_id INTEGER PRIMARY KEY` is the rowid.
Meaning	Revenue "by customer" silently drops these two orders in any inner join. Finding them is the first step of reconciling a dashboard to the source.

COMMON MISTAKE	Writing `WHERE c.customer_id = NULL` to find orphans. It returns no rows, because nothing equals NULL. Use `IS NULL` .

Connection to the real job

The routine above is the first hour of every data task: onboarding to a new warehouse, auditing a vendor feed, answering "why does the dashboard disagree with finance?". You will also use the same catalogue ideas in PostgreSQL through `information_schema` , a standard view that works in most databases. Writing the introspection into a small script that you run on every new source pays back every time.

Applications

- Data discovery on a new project (what tables exist, how big, how related)

- Data-quality checks: orphan counts, NULL rates, duplicate keys
- Generating documentation and diagrams from the catalogue

Check yourself

1. Where does SQLite store the list of tables?

Answer: In the built-in table `sqlite_master`.

1. Why can `COUNT(coupon)` be smaller than `COUNT(*)`?

Answer: `COUNT(column)` ignores NULLs; `COUNT(*)` counts every row.

1. How do you find child rows that have no parent?

Answer: `LEFT JOIN` to the parent and keep rows where the parent key `IS NULL` (an anti-join), or use `PRAGMA foreign_key_check` in SQLite.

1.10 Using SQLite from Python: Connections, Parameters and Transactions

INTUITION	At work you rarely type SQL into a shell all day. You send it from Python, a notebook or a pipeline, and you get rows back as Python objects. Three habits make that safe: **pass values as parameters** (never paste them into the SQL text), **group related changes in a transaction**, and **close or commit deliberately**.

The sqlite3 module in six lines

Call	What it does
`con = sqlite3.connect("shop.db")`	Open (or create) a database file. `":memory:"` makes a temporary one.
`cur = con.execute(sql, params)`	Run one statement; `params` fill the `?` placeholders.
`cur.fetchall()`, `fetchone()`	Get all rows (a list of tuples) or one row.
`con.executemany(sql, list_of_params)`	Run the same statement for many rows (fast bulk insert).
`con.commit()` / `con.rollback()`	Make the open transaction permanent / undo it.
`con.row_factory = sqlite3.Row`	Return rows you can read by column name.

Worked examples

Problem 1.10.1 A login form passes whatever the user typed straight into the SQL text. What happens when the user types `x' OR '1'='1`? Fix it.

Idea	If you build SQL by gluing strings, the user's text becomes part of the SQL code. The quote characters let the attacker close your string and add their own condition. This is called **SQL injection** and it has caused some of the largest data breaches on record.
Fix	Use a placeholder `?` and pass the value separately. The database treats it as pure data, never as code.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE users (name TEXT, secret TEXT);
INSERT INTO users VALUES ('asha','pw-1'),('ravi','pw-2'),('meena','pw-3');
""")
typed = "x' OR '1'='1"                    # what the attacker typed

unsafe_sql = f"SELECT name FROM users WHERE name = '{typed}'"
print(unsafe_sql)
print("unsafe:", con.execute(unsafe_sql).fetchall())

safe = con.execute("SELECT name FROM users WHERE name = ?", (typed,)).fetchall()
print("safe  :", safe)
```

```
SELECT name FROM users WHERE name = 'x' OR '1'='1'
unsafe: [('asha',), ('ravi',), ('meena',)]
safe  : []
```

1. The attacker's text contains a single quote, followed by `OR '1'='1`.
2. The f-string puts it inside quotes. The printed query shows the problem: `WHERE name = 'x' OR '1'='1'`. The condition `'1'='1'` is always true, so the filter matches every row.
3. The unsafe query therefore returns all three users, although nobody is called "x".
4. The safe query sends the SQL text and the value separately. The database looks for a user whose name is literally `x' OR '1'='1`, finds none, and returns an empty list.

Answer	Unsafe: all three names leak. Safe: an empty list.
Meaning	Rule with no exceptions: *values* go in parameters. Only identifiers (table or column names) cannot be parameters; if you must vary them, choose from a fixed allow-list in your code. PostgreSQL drivers use `%s` or `$1` instead of `?`, same idea.

COMMON MISTAKE	Building SQL with f-strings, `%` formatting or `+` from user input, "just for a quick script". Quick scripts become production. Always use placeholders for values.

Problem 1.10.2 Move 500 rupees from account A to account B. The second step fails halfway. Show that without a transaction money disappears, and with one it does not.

Idea	A transaction groups statements so that they all take effect or none do. In Python's `sqlite3`, `with con:` commits if the block finishes normally and rolls back if an exception escapes.
Setup	Account A has 1000 and B has 200, total 1200. The program debits A, then crashes (we raise an error on purpose) before it credits B. Whatever the final balances are, the total should still be 1200.

```
import sqlite3

def balances(con):
    return dict(con.execute("SELECT id, balance FROM accounts ORDER BY id"))

def setup():
    con = sqlite3.connect(":memory:")
    con.execute("CREATE TABLE accounts (id TEXT PRIMARY KEY, balance INTEGER NOT NULL CHECK (balance >= 0))")
    con.executemany("INSERT INTO accounts VALUES (?,?)", [("A", 1000), ("B", 200)])
    con.commit()
    return con

def transfer(con, amount, fail_midway):
    con.execute("UPDATE accounts SET balance = balance - ? WHERE id = 'A'", (amount,))
    if fail_midway:
        raise RuntimeError("network dropped before the credit")
    con.execute("UPDATE accounts SET balance = balance + ? WHERE id = 'B'", (amount,))

# 1. no transaction handling: the debit sticks, the credit never happens
con = setup()
try:
    transfer(con, 500, fail_midway=True)
except RuntimeError as e:
    print("error:", e)
print("without protection:", balances(con), "total =", sum(balances(con).values()))

# 2. with a transaction: the failure undoes the debit
con = setup()
try:
    with con:
        transfer(con, 500, fail_midway=True)
except RuntimeError:
    pass
print("with transaction  :", balances(con), "total =", sum(balances(con).values()))

# 3. the CHECK constraint also protects: overdraw A
con = setup()
try:
    with con:
        transfer(con, 5000, fail_midway=False)
except sqlite3.Error as e:
    print("overdraw refused:", e)
print("after overdraw    :", balances(con))
```

```
error: network dropped before the credit
without protection: {'A': 500, 'B': 200} total = 700
with transaction  : {'A': 1000, 'B': 200} total = 1200
overdraw refused: CHECK constraint failed: balance >= 0
after overdraw    : {'A': 1000, 'B': 200}
```

1. `setup()` gives a fresh pair of accounts: A has 1000, B has 200. The CHECK stops any balance going negative. The total of all accounts is 1200.
2. `transfer()` debits A first, then optionally raises an error, then credits B. The error simulates a crash between the two steps.
3. Case 1 has no protection. The first UPDATE ran and nobody undid it, so A shows 500 and B still 200. The total is 700. Five hundred rupees vanished.
4. Case 2 wraps the call in `with con:`. The exception rolls the open transaction back, so the debit is undone and the balances are unchanged.
5. Case 3 tries to move 5000 out of A. The debit would make A negative, so the CHECK constraint raises an error, `with con:` rolls back, and both balances stay as they were.

Answer	Case 1 ends with total 700 (money lost). Cases 2 and 3 end with total 1200.
Meaning	This is the "A" in ACID, *atomicity*: all or nothing. Together with constraints (consistency), locking between users (isolation) and the log that survives a crash (durability), it is what a CSV file cannot give you. Chapter 9 covers transactions in depth. In case 1 the debit is merely uncommitted on this connection. If any later line of the program calls `commit()`, the loss becomes permanent, so never rely on luck.

Problem 1.10.3 Read rows by column name and hand a query result to pandas.

Step 1	Set `row_factory = sqlite3.Row`, then `row["name"]` works.
Step 2	`pandas.read_sql_query(sql, con)` returns a DataFrame; pass parameters with `params=`.

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE employees (emp_id INTEGER PRIMARY KEY, name TEXT, dept_id INTEGER, salary INTEGER);
INSERT INTO employees VALUES (1,'Asha',10,90000),(2,'Ravi',20,120000),(3,'Meena',10,95000);
""")
con.row_factory = sqlite3.Row
row = con.execute("SELECT name, salary FROM employees WHERE emp_id = ?", (2,)).fetchone()
print(row["name"], row["salary"], tuple(row))

df = pd.read_sql_query("SELECT dept_id, salary FROM employees WHERE salary >= ?", con, params=(95000,))
print(df)
print(df.groupby("dept_id")["salary"].mean())
```

```
Ravi 120000 ('Ravi', 120000)
   dept_id  salary
0       20  120000
1       10   95000
dept_id
10     95000.0
20    120000.0
Name: salary, dtype: float64
```

1. With `sqlite3.Row` a row behaves like both a tuple and a dictionary, so the code survives someone reordering columns.
2. `fetchone()` returns a single row; `tuple(row)` shows its values.
3. `read_sql_query` runs the query and builds a DataFrame with the column names from the SQL. The `?` is filled by `params`, with the same safety as before.
4. The last line does the same `GROUP BY` you will learn in Chapter 3, this time in pandas.

Answer	Ravi's row by name, a two-row DataFrame (Ravi, Meena), and the mean salary per department.
Meaning	A common pattern: let the database filter and join (it is fast and close to the data), then finish the analysis in pandas.

Connection to the real job

Every data pipeline is "connect, parameterised query, transform, write". Airflow tasks, dbt tests and Jupyter notebooks all do this. Reviewers look for three things in your code: parameters instead of string building, one connection reused sensibly, and a transaction around any multi-step write. These habits also carry over unchanged to PostgreSQL drivers (psycopg, SQLAlchemy).

Applications

- Loading CSV files into a local database for analysis
- Backend services that read and write orders

- Unit tests that run against an in-memory database

Check yourself

1. What does the `?` in `WHERE name = ?` do?

Answer: It is a placeholder. The value is sent separately, so it can never be read as SQL code.

1. What does `with con:` do when an exception happens inside it?

Answer: It rolls the transaction back, undoing every change made inside the block.

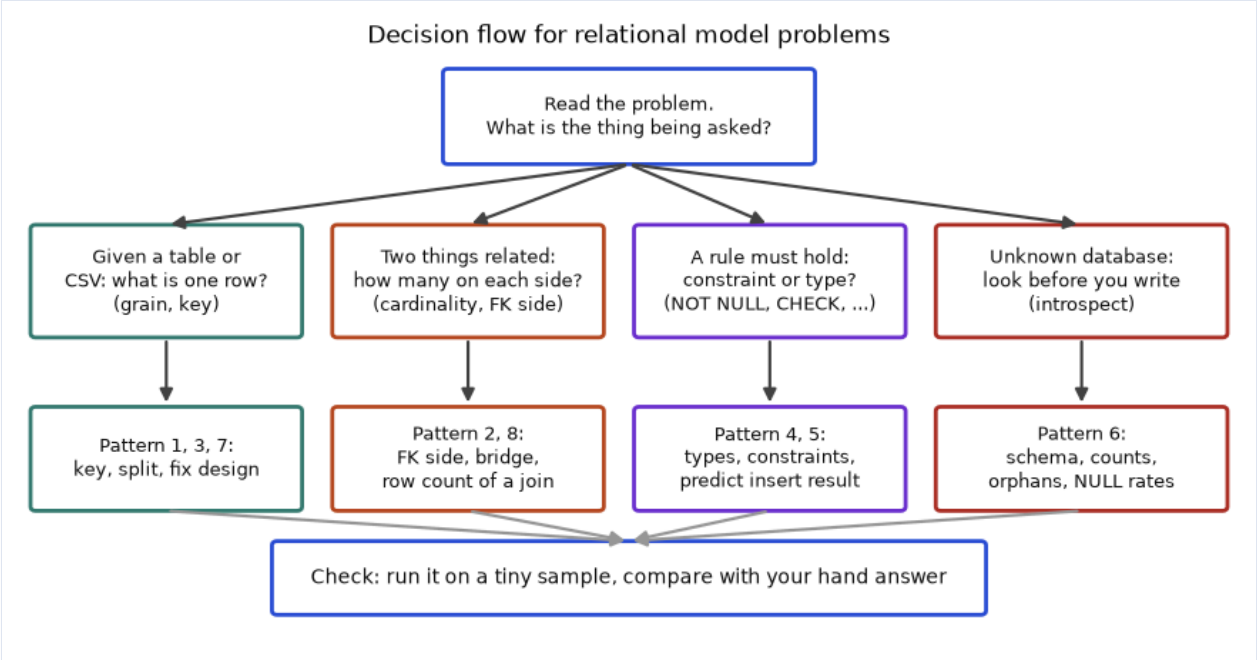
1. Can a table name be a parameter?

Answer: No. Only values can. Pick table names from a fixed list in your own code.

1.11 How to solve any databases and the relational model problem

INTUITION	Every problem in this chapter, from an interview question to a Monday-morning "why is this number wrong", is one of eight shapes. Learning to *name the shape* is half the solution, because each shape has a short recipe. This section is the recipe book. Read it once, then use it as a checklist whenever you are stuck.

Step 0: the decision flow



Start at the top box, pick the branch that matches what you are given, follow it to the pattern, and finish by checking on a tiny sample.

How to read it: the four middle boxes are the four things a problem can hand you. A *table or file* means you must find its row meaning and key. *Two things that relate* means you decide cardinality and where the foreign key goes. *A rule that must hold* means types and constraints. *An unknown database* means look before you write. The bottom box is the same for all: test on a tiny sample whose answer you can compute by hand.

If the problem says...	It is pattern	First move
"what is the key / is this column unique / what does a row mean"	1 Find the grain	Two-count test
"how are A and B related / where does the foreign key go"	2 Cardinality	Ask the two "can one have many?" questions

If the problem says...	It is pattern	First move
"design tables for ... / here is a CSV"	3 Design from requirements	List nouns, then verbs, then keys
"make sure that ... can never happen"	4 Types and constraints	Turn each sentence of the rule into one constraint
"will this insert / update / delete succeed?"	5 Predict a change	Walk the rules in order: type, NOT NULL, CHECK, UNIQUE, FK
"explore this database / why do the numbers differ"	6 Introspect	Tables, columns, keys, counts, orphans
"what is wrong with this design"	7 Spot a bad design	Hunt lists in cells, repeated facts, missing keys
"how many rows will this join return"	8 Estimate row counts	Multiply matches per key, do not guess

Pattern 1: find the grain and the key

Recipe. (a) Say in one sentence what one row means. (b) Guess a key from that sentence. (c) Test it with `COUNT(*)` versus `COUNT(DISTINCT key)`. (d) If it fails, add a column to the guess, or admit there is no natural key and add a surrogate one.

Problem 1.11.1 A table `page_views(user_id, page, viewed_at)` has these rows. Is `(user_id, page)` a key? Is `(user_id, page, viewed_at)` ?

Sentence	One row = one time a user looked at a page.
Guess 1	(user_id, page): but the same user can view the same page twice, so it should fail.
Guess 2	(user_id, page, viewed_at): fails only if two views land in the same second.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE page_views (user_id INT, page TEXT, viewed_at TEXT);
INSERT INTO page_views VALUES
 (1,'/home','2024-05-01 10:00:00'),(1,'/home','2024-05-01 10:05:00'),
 (1,'/cart','2024-05-01 10:06:00'),(2,'/home','2024-05-01 10:00:00'),
 (2,'/home','2024-05-01 10:00:00');
""")
def test(cols):
    n  = con.execute("SELECT COUNT(*) FROM page_views").fetchone()[0]
    nd = con.execute(f"SELECT COUNT(*) FROM (SELECT DISTINCT {cols} FROM page_views)").fetchone()[0]
    return n, nd, "KEY" if n == nd else "not a key"
print("user_id, page           :", test("user_id, page"))
print("user_id, page, viewed_at:", test("user_id, page, viewed_at"))
```

```
user_id, page           : (5, 3, 'not a key')
user_id, page, viewed_at: (5, 4, 'not a key')
```

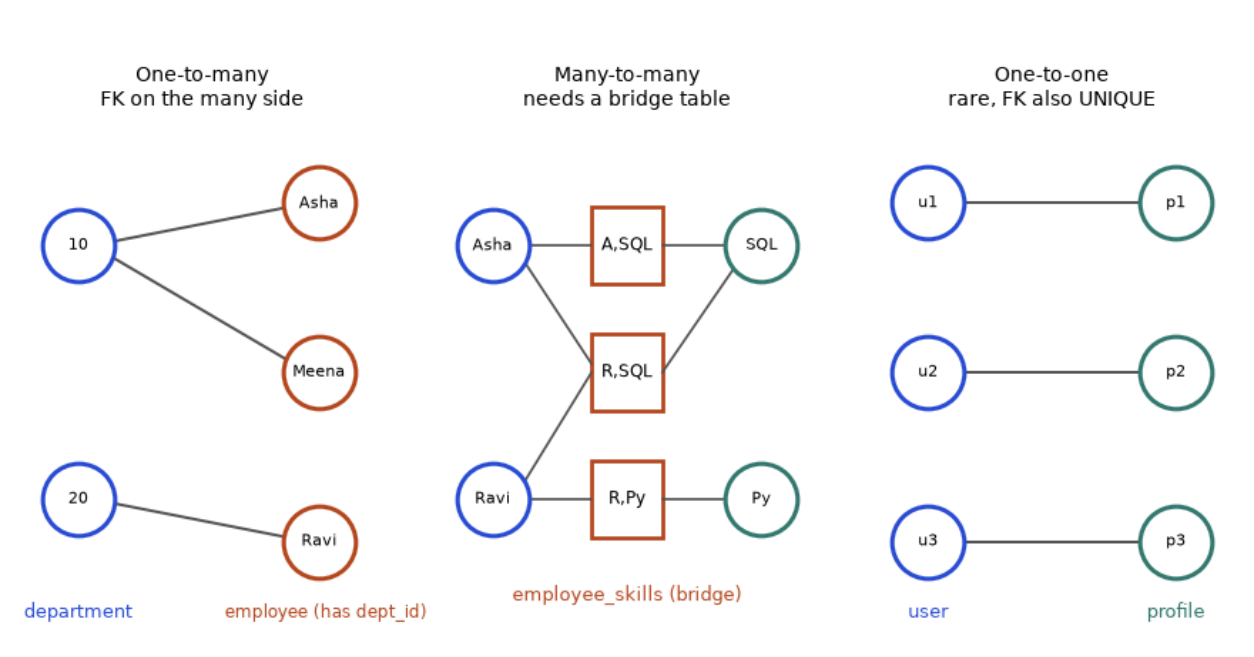
1. The data includes a repeated page for user 1 (different times) and an exact duplicate row for user 2 (two views logged in the same second).
2. `test()` counts all rows, then counts distinct combinations of the columns we pass in. `SELECT DISTINCT` in a subquery gives the distinct combinations; the outer `COUNT(*)` counts them. The column list is pasted into the SQL from our own code, never from a user.
3. Guess 1 gives 5 rows but only 3 distinct pairs: (1,/home), (1,/cart) and (2,/home). Not a key.
4. Guess 2 gives 5 rows but 4 distinct triples, because user 2's two views are identical.

Answer	Neither is a key. This table has no natural key.
Decision	Add a surrogate `view_id INTEGER PRIMARY KEY`. Two identical-looking rows are two real events (two clicks), so merging them would lose data.

Pattern 2: cardinality and the foreign key side

Recipe. Ask two questions about A and B: "Can one A have many Bs?" and "Can one B have many As?" Then use the table.

One A has many B?	One B has many A?	Relationship	Where the foreign key goes
Yes	No	A to B is one-to-many	A column in B pointing to A
No	Yes	B to A is one-to-many	A column in A pointing to B
Yes	Yes	Many-to-many	New bridge table with a foreign key to each, primary key on the pair
No	No	One-to-one	Foreign key in either, plus `UNIQUE` on it



The three relationship shapes. Lines are the links a foreign key records.

Problem 1.11.2 A library lends books to members. A book has several authors. A member can borrow many books over time and a book is borrowed many times. A book has exactly one publisher. Name every relationship and where each foreign key goes.

Publisher and book	One publisher has many books; a book has one publisher: one-to-many. Foreign key `publisher_id` in `books`.
Book and author	A book has many authors and an author has many books: many-to-many. Bridge `book_authors(book_id, author_id)`.
Member and book	Looks many-to-many, but a loan has its own facts (date out, date back). The bridge table is `loans(loan_id, member_id, book_id, loaned_on, returned_on)`. It needs a surrogate key because the same member can borrow the same book twice.
Answer	Tables: publishers, books, authors, book_authors, members, loans. Two bridge tables; one is a pure link, the other is an event table.

COMMON MISTAKE	Using the pair `(member_id, book_id)` as the primary key of `loans`. It forbids borrowing the same book twice, ever. When the relationship has history, the key needs a surrogate id or a date: `(member_id, book_id, loaned_on)`.

Pattern 3: design tables from requirements

Recipe. (a) Underline the nouns: each is a candidate table. (b) Underline the verbs between nouns: each is a relationship. (c) For each relationship apply Pattern 2. (d) Give each table a key (Pattern 1). (e) Add the mandatory, unique and range rules (Pattern 4). (f) Insert a few sample rows and test.

Problem 1.11.3 A clinic: "Patients book appointments with doctors. Each appointment has a date and time, a status (booked, done, cancelled) and optional notes. A doctor cannot be booked twice at the same time. Each doctor has one specialty chosen from a fixed list." Write the tables and prove the double-booking rule works.

Nouns	patient, doctor, specialty, appointment.
Relationships	specialty to doctors one-to-many; patient to appointments one-to-many; doctor to appointments one-to-many. So `appointments` holds two foreign keys.
Rules	status limited by CHECK; a doctor cannot have two appointments at the same time: `UNIQUE (doctor_id, starts_at)` ; notes optional (nullable).

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE specialties (specialty_id INTEGER PRIMARY KEY, label TEXT NOT NULL UNIQUE);
CREATE TABLE doctors (doctor_id INTEGER PRIMARY KEY, name TEXT NOT NULL,
                      specialty_id INTEGER NOT NULL REFERENCES specialties(specialty_id));
CREATE TABLE patients (patient_id INTEGER PRIMARY KEY, name TEXT NOT NULL, phone TEXT UNIQUE);
CREATE TABLE appointments (
  appt_id    INTEGER PRIMARY KEY,
  patient_id INTEGER NOT NULL REFERENCES patients(patient_id),
  doctor_id  INTEGER NOT NULL REFERENCES doctors(doctor_id),
  starts_at  TEXT    NOT NULL,
  status     TEXT    NOT NULL DEFAULT 'booked' CHECK (status IN ('booked','done','cancelled')),
  notes      TEXT,
  UNIQUE (doctor_id, starts_at)
);
INSERT INTO specialties VALUES (1,'Cardiology'),(2,'Dermatology');
INSERT INTO doctors VALUES (1,'Dr Rao',1),(2,'Dr Iyer',2);
INSERT INTO patients VALUES (1,'Asha','9000000001'),(2,'Ravi','9000000002');
""")
tests = [
 ("T1 Asha with Dr Rao at 10:00",     (1, 1, "2024-06-01 10:00", "booked")),
 ("T2 Ravi with Dr Rao at 10:00",     (2, 1, "2024-06-01 10:00", "booked")),
 ("T3 Ravi with Dr Iyer at 10:00",    (2, 2, "2024-06-01 10:00", "booked")),
 ("T4 status 'maybe'",                (2, 2, "2024-06-02 09:00", "maybe")),
 ("T5 patient 9 does not exist",      (9, 2, "2024-06-03 09:00", "booked")),
]
for label, args in tests:
    try:
        con.execute("INSERT INTO appointments(patient_id,doctor_id,starts_at,status) VALUES (?,?,?,?)", args)
        print("ok      ", label)
    except sqlite3.Error as e:
        print("REFUSED ", label, "|", e)
print(con.execute("SELECT appt_id, patient_id, doctor_id, status FROM appointments").fetchall())
```

```
ok       T1 Asha with Dr Rao at 10:00
REFUSED  T2 Ravi with Dr Rao at 10:00 | UNIQUE constraint failed: appointments.doctor_id, appointments.starts_at
ok       T3 Ravi with Dr Iyer at 10:00
REFUSED  T4 status 'maybe' | CHECK constraint failed: status IN ('booked','done','cancelled')
REFUSED  T5 patient 9 does not exist | FOREIGN KEY constraint failed
[(1, 1, 1, 'booked'), (2, 2, 2, 'booked')]
```

1. Reference tables first (`specialties` , then `doctors` which refers to them), then `patients` , then `appointments` which refers to both. Parents are always created before children.
2. `UNIQUE (doctor_id, starts_at)` is a composite unique rule: the pair may not repeat, although the same doctor can have many times and the same time can hold many doctors.
3. Status uses both a DEFAULT and a CHECK with `IN (...)` , a list of allowed values.
4. The tests pass the four columns as parameters. T1 is fine. T2 books Dr Rao at the same time for a second patient and breaks the unique pair. T3 books the other doctor at that time, which is fine. T4 uses a bad status. T5 uses patient 9, who does not exist.

Answer	T1 and T3 succeed; T2, T4 and T5 are refused by UNIQUE, CHECK and FOREIGN KEY. Two appointments are stored.
Check	Each of the clinic's sentences maps to exactly one structure. If a sentence has no structure, something is missing.

Pattern 4: choose types and constraints

Recipe. Take the rule in plain English and cut it into short claims: "is it mandatory?" (NOT NULL), "must it be unique?" (UNIQUE or key), "which values are legal?" (type plus CHECK), "what if omitted?" (DEFAULT), "must it point to something?" (FOREIGN KEY). One claim, one constraint.

Problem 1.11.4 "Users rate movies from 1 to 5 stars. A user can rate a movie only once. The comment is optional. The time of rating is recorded automatically." Write the table.

Claims	stars: mandatory, integer, 1 to 5. Once per user and movie: composite primary key. Comment: nullable. Time: DEFAULT to the current time, mandatory.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE ratings (
  user_id  INTEGER NOT NULL,
  movie_id INTEGER NOT NULL,
  stars    INTEGER NOT NULL CHECK (stars BETWEEN 1 AND 5),
  comment  TEXT,
  rated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, movie_id)
);
""")
for args in [(1, 10, 5, None), (1, 10, 4, "changed my mind"), (1, 11, 6, None), (2, 10, 3, "ok")]:
    try:
        con.execute("INSERT INTO ratings(user_id, movie_id, stars, comment) VALUES (?,?,?,?)", args)
        print("accepted", args)
    except sqlite3.Error as e:
        print("refused ", args, "|", e)
print(con.execute("SELECT user_id, movie_id, stars, comment, rated_at IS NOT NULL FROM ratings").fetchall())
```

```
accepted (1, 10, 5, None)
refused  (1, 10, 4, 'changed my mind') | UNIQUE constraint failed: ratings.user_id, ratings.movie_id
refused  (1, 11, 6, None) | CHECK constraint failed: stars BETWEEN 1 AND 5
accepted (2, 10, 3, 'ok')
[(1, 10, 5, None, 1), (2, 10, 3, 'ok', 1)]
```

1. `CHECK (stars BETWEEN 1 AND 5)` allows 1, 2, 3, 4, 5 and nothing else; `NOT NULL` closes the NULL loophole from Problem 1.5.2.
2. `comment TEXT` has no rule, so it may be NULL.
3. `DEFAULT CURRENT_TIMESTAMP` fills `rated_at` with the current UTC time when we do not give one. The last query only prints whether it is filled, because the actual time changes every run.
4. The second attempt repeats (user 1, movie 10) and is refused by the primary key. Changing the rating later would be an `UPDATE` (or an upsert, Chapter 9), not a second insert.

Answer	Attempts 1 and 4 are accepted; 2 (duplicate) and 3 (6 stars) are refused. The auto-filled time exists on both stored rows.

Pattern 5: predict the result of a change

Recipe. A useful mental order for checking a change (real engines may test the rules in a different order, so when two rules are broken the message may name either, but accept or refuse is the same): (1) is the type acceptable? (2) is a mandatory column missing (NOT NULL)? (3) does CHECK fail? (4) does UNIQUE / PRIMARY KEY fail? (5) does the FOREIGN KEY fail, either because the parent is missing on insert and update, or because children still exist on parent delete and parent key update? Walk the list and stop at the first failure. Also remember: a failed statement changes nothing.

Problem 1.11.5 Given the tables below, predict each of six statements (accept or refuse, and why) before reading the output. Departments: 10 Testing, 20 Data. Employees: Asha(10), Ravi(20). The `employees.dept_id` key is declared `ON DELETE CASCADE` with no ON UPDATE action. Salary has `CHECK (salary > 0)`. Department names are `UNIQUE`.

S1	Insert employee in department 30. Parent 30 does not exist, so the FK refuses.
S2	Update Asha's salary to -5. CHECK fails, refused.
S3	Delete department 20. Cascade removes Ravi, accepted.
S4	Insert a department named 'Data' with id 50. The name looks taken, but S3 already deleted department 20, which was the only 'Data'. So the name is free and S4 is accepted. Order matters.
S5	Change department 10's id to 11 while Asha points to 10. No ON UPDATE action, so the default refuses.
S6	Delete department 99. Nothing matches; accepted, 0 rows deleted.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE departments (dept_id INTEGER PRIMARY KEY, dept_name TEXT NOT NULL UNIQUE);
CREATE TABLE employees (
  emp_id  INTEGER PRIMARY KEY, name TEXT,
  dept_id INTEGER REFERENCES departments(dept_id) ON DELETE CASCADE,
  salary  INTEGER CHECK (salary > 0));
INSERT INTO departments VALUES (10,'Testing'),(20,'Data');
INSERT INTO employees VALUES (1,'Asha',10,90000),(2,'Ravi',20,120000);
""")
steps = [
 ("S1", "INSERT INTO employees VALUES (3,'Tom',30,50000)"),
 ("S2", "UPDATE employees SET salary = -5 WHERE name = 'Asha'"),
 ("S3", "DELETE FROM departments WHERE dept_id = 20"),
 ("S4", "INSERT INTO departments VALUES (50,'Data')"),
 ("S5", "UPDATE departments SET dept_id = 11 WHERE dept_id = 10"),
 ("S6", "DELETE FROM departments WHERE dept_id = 99"),
]
for label, sql in steps:
    try:
        cur = con.execute(sql)
        print(label, "accepted, rows changed:", cur.rowcount)
    except sqlite3.Error as e:
        print(label, "refused:", e)
print(con.execute("SELECT name FROM employees").fetchall(), con.execute("SELECT dept_id, dept_name FROM departments ORDER BY
1").fetchall())
```

```
S1 refused: FOREIGN KEY constraint failed
S2 refused: CHECK constraint failed: salary > 0
S3 accepted, rows changed: 1
S4 accepted, rows changed: 1
S5 refused: FOREIGN KEY constraint failed
S6 accepted, rows changed: 0
[('Asha',)] [(10, 'Testing'), (50, 'Data')]
```

1. The steps run in order against one connection, so each statement sees the effect of the earlier ones. That is why S4 is accepted: S3 already removed the old 'Data' department.
2. `cur.rowcount` reports how many rows the statement changed (S3 counts only the department row; the cascaded employee deletion is not included).
3. S5 is refused: the parent key 10 is referenced by Asha, and the foreign key has no `ON UPDATE CASCADE`.
4. The final query shows who and what remains: Asha and departments 10 and 50.

Answer	S1 refused (FK), S2 refused (CHECK), S3 accepted (Ravi cascades away), S4 accepted, S5 refused (FK), S6 accepted with 0 rows changed.
Meaning	When you predict a sequence of statements you must track the state after each one. S4 is only legal because S3 ran first. If you had judged S4 against the starting data you would have wrongly predicted a UNIQUE failure.

Pattern 6: introspect an unknown database

Recipe. The routine from Section 1.9. Wrap it in a function you reuse.

Problem 1.11.6 Write a reusable `audit(con)` that prints, per table, the row count, any column with NULLs and the orphan count, and run it on a database that has one NULL problem and one orphan problem.

Step 1	Get table names from the catalogue.
Step 2	For each table, get column names from `PRAGMA table_info` and count NULLs per column with `COUNT(*) - COUNT(col)`.
Step 3	Run `PRAGMA foreign_key_check` once and count violations per child table.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT, city TEXT);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER REFERENCES customers(customer_id), amount INTEGER);
INSERT INTO customers VALUES (1,'Asha','Pune'),(2,'Ravi',NULL),(3,'Meena',NULL);
INSERT INTO orders VALUES (101,1,500),(102,2,NULL),(103,9,300);
""")
def audit(con):
    orphans = {}
    for child, rowid, parent, fk in con.execute("PRAGMA foreign_key_check"):
        orphans[child] = orphans.get(child, 0) + 1
    for (t,) in con.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name").fetchall():
        n = con.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0]
        cols = [r[1] for r in con.execute(f"PRAGMA table_info({t})")]
        nulls = {}
        for c in cols:
            k = n - con.execute(f"SELECT COUNT({c}) FROM {t}").fetchone()[0]
            if k: nulls[c] = k
        print(f"{t}: rows={n}, null counts={nulls}, orphans={orphans.get(t, 0)}")
audit(con)
```

```
customers: rows=3, null counts={'city': 2}, orphans=0
orders: rows=3, null counts={'amount': 1}, orphans=1
```

1. The sample data hides two problems: two customers have no city, and order 103 points to customer 9 who does not exist. One order also has a missing amount.
2. The first loop reads `foreign_key_check` and tallies violations per child table in a dictionary.
3. The second loop visits every table, counts rows, and for each column subtracts the non-NULL count from the row count. Only columns with at least one NULL are kept.
4. The f-strings are safe because table and column names come from the catalogue.

Answer	customers: 3 rows, 2 NULL cities, 0 orphans. orders: 3 rows, 1 NULL amount, 1 orphan.
Meaning	Run something like this on every new data source before you write a single analytic query. Surprises found here cost minutes; surprises found in a board meeting cost trust.

Pattern 7: spot and fix a bad design

Recipe. Scan the column list for five smells: (1) numbered columns (`phone1, phone2`) or lists in cells, which break 1NF; (2) a fact that repeats on many rows (city, department name), which needs its own table; (3) a column that is determined by another non-key column (pincode decides city and state), which breaks 3NF; (4) a table with no key; (5) type mistakes (money as float, dates as free text).

Problem 1.11.7 Critique `students(id, name, phone1, phone2, phone3, city, pincode, state, fee REAL)` and give a corrected design.

Smell 1	`phone1..phone3` : a repeating group. A student with four phones does not fit; searching "who has this number" needs three comparisons. Fix: table `student_phones(student_id, phone)` .
Smell 3	pincode decides city and state, but none of the three is the key. Updating a city name means editing many students. Fix: table `pincodes(pincode, city, state)` and keep only `pincode` in students.
Smell 5	`fee REAL` for money. Fix: integer paise.
Smell 4?	There is an `id` , so a key exists; confirm it is declared `PRIMARY KEY` .

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE pincodes (pincode TEXT PRIMARY KEY, city TEXT NOT NULL, state TEXT NOT NULL);
CREATE TABLE students (
  student_id INTEGER PRIMARY KEY, name TEXT NOT NULL,
  pincode TEXT REFERENCES pincodes(pincode), fee_paise INTEGER NOT NULL CHECK (fee_paise >= 0));
CREATE TABLE student_phones (
  student_id INTEGER NOT NULL REFERENCES students(student_id),
  phone TEXT NOT NULL, PRIMARY KEY (student_id, phone));
INSERT INTO pincodes VALUES ('411001','Pune','Maharashtra'),('560001','Bengaluru','Karnataka');
INSERT INTO students VALUES (1,'Asha','411001',250000),(2,'Ravi','411001',250000),(3,'Meena','560001',300000);
INSERT INTO student_phones VALUES (1,'9000000001'),(1,'9000000002'),(3,'9000000003');
""")
con.execute("UPDATE pincodes SET city = 'Pune City' WHERE pincode = '411001'")   # ONE edit
print(con.execute("""SELECT s.name, p.city, p.state, s.fee_paise / 100.0 AS fee
                     FROM students s JOIN pincodes p ON p.pincode = s.pincode ORDER BY s.student_id""").fetchall())
print(con.execute("""SELECT s.name, COUNT(sp.phone) FROM students s
                     LEFT JOIN student_phones sp ON sp.student_id = s.student_id
                     GROUP BY s.student_id ORDER BY s.student_id""").fetchall())
```

```
[('Asha', 'Pune City', 'Maharashtra', 2500.0), ('Ravi', 'Pune City', 'Maharashtra', 2500.0), ('Meena', 'Bengaluru',
'Karnataka', 3000.0)]
[('Asha', 2), ('Ravi', 0), ('Meena', 1)]
```

1. `pincodes` now holds each city and state exactly once.
2. `students` stores only the pincode, and the fee as integer paise with a CHECK.
3. `student_phones` allows any number of phones per student, with a composite key so a number cannot repeat for the same student.
4. One `UPDATE` renames the city for both Pune students. The first query joins back and shows both rows changed.
5. The second query counts phones per student with a `LEFT JOIN` , so a student with no phone would show 0 (Ravi does). `GROUP BY` is Chapter 3.

Answer	The corrected design above. One edit updated both Pune students, and phone counts are 2, 0, 1.

Pattern 8: estimate the row count of a join

Recipe. For an inner join on a key, the result has one row per matching pair. Count, per key value, (rows on the left) times (rows on the right) and add up. If the right side's join column is a key (unique), the result has at most as many rows as the left. If neither side is unique, rows multiply. A `LEFT JOIN` adds one row for every left row with no match.

Problem 1.11.8 In the shop there are 8 customers (7 and 8 never ordered), 14 orders and 20 order lines. How many rows come back from (a) customers INNER JOIN orders, (b) customers LEFT JOIN orders, (c) that LEFT JOIN extended with order_items? Predict, then confirm.

(a)	Every order has exactly one customer (`customer_id` is NOT NULL and a foreign key). Pairs = number of orders = 14.
(b)	The 14 matched rows plus one row each for customers 7 and 8 with NULL order columns: 14 + 2 = 16.
(c)	Joining order lines multiplies each order by its number of lines. Orders with 2 lines (101, 103, 106, 110, 113, 114) count twice. 14 orders give 20 lines in total, so the 14 order rows become 20 rows; add the 2 customers without orders: 22.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER NOT NULL);
CREATE TABLE order_items (order_id INTEGER, product_id INTEGER, PRIMARY KEY (order_id, product_id));
""")
con.executemany("INSERT INTO customers VALUES (?)", [(i,) for i in range(1, 9)])
con.executemany("INSERT INTO orders VALUES (?,?)", [(101,1),(102,2),(103,1),(104,3),(105,4),(106,1),(107,5),
    (108,2),(109,5),(110,6),(111,1),(112,4),(113,5),(114,2)])
con.executemany("INSERT INTO order_items VALUES (?,?)", [(101,1),(101,3),(102,2),(103,5),(103,4),(104,3),
    (105,1),(106,2),(106,3),(107,4),(108,5),(109,1),(110,3),(110,4),(111,2),(112,5),(113,3),(113,4),(114,1),(114,2)])
q = {
 "a inner  customers-orders        ": "SELECT COUNT(*) FROM customers c JOIN orders o ON o.customer_id = c.customer_id",
 "b left   customers-orders        ": "SELECT COUNT(*) FROM customers c LEFT JOIN orders o ON o.customer_id =
c.customer_id",
 "c left   ... then order_items    ": """SELECT COUNT(*) FROM customers c
        LEFT JOIN orders o ON o.customer_id = c.customer_id
        LEFT JOIN order_items i ON i.order_id = o.order_id""",
}
for name, sql in q.items():
    print(name, con.execute(sql).fetchone()[0])
```

```
a inner  customers-orders         14
b left   customers-orders         16
c left   ... then order_items     22
```

1. The script rebuilds only the key columns of the shop, which is all a row count needs: 8 customers, 14 orders, 20 order lines.
2. Query a is an inner join. Each order matches one customer, so it returns one row per order: 14.
3. Query b is a left join, so customers 7 and 8 stay with NULLs: 16.
4. Query c adds the many side of the next relationship. Each order row is repeated once per order line; two customers still have a single NULL row: 20 + 2 = 22.

Answer	14, 16 and 22, matching the prediction.
Meaning	If you now run `SUM(order_total)` over query c, any order-level amount is counted once per line, so it is inflated. This "fan-out" is the most common cause of a revenue figure that is too high. Chapter 4 shows the cure.

COMMON MISTAKE	Joining a one-to-many table and then summing a column from the "one" side. Right after the join check `COUNT(*)` against the row count you expected (here 14 versus 20). If it grew, aggregate first or sum a column from the "many" side.

How to check an answer

1. **Run it on a tiny sample.** Three to six rows whose answer you can compute on paper. If the code and your hand answer differ, one of them is wrong; find out which.
2. **Check row counts at every step.** After each join or filter, ask what the count should be. A surprise count is the earliest warning.
3. **Try to break your own design.** Insert a duplicate, a NULL, a child with a missing parent, a delete of a parent. Does each attempt behave as your promises say?
4. **Round trip.** After restructuring data, join it back and compare with the original (Problem 1.8.2).
5. **Explain it in one sentence.** "One row of this table is ...; it links to ... through ...". If the sentence is clumsy, the design probably is too.

How to debug when you are stuck

Symptom	Most likely cause	What to do
An insert is refused and you do not know why	A constraint you forgot or a missing parent	Read the message: it names the rule and column. Query the parent table for the value.
Foreign keys seem to do nothing (SQLite)	`PRAGMA foreign_keys = ON` missing on this connection	Run `PRAGMA foreign_keys;` ; 0 means off.
Totals too high after a join	Fan-out from a one-to-many join	Compare `COUNT(*)` before and after the join.
Rows missing after a join	Inner join dropped rows with no match or NULL key	Use `LEFT JOIN` and look for NULL columns.
A "unique" column has duplicates	No `UNIQUE` declared, or NULLs, or case/space differences	Count distinct, then compare with `LOWER(TRIM(col))` .
Strange values in a typed column (SQLite)	Type affinity let text in	Check `typeof(col)` ; use `STRICT` tables.
Result order changed overnight	No `ORDER BY`	Add an `ORDER BY` , with a tie-breaker.

Error messages are the database's attempt to help. This block triggers the common ones on purpose so that you recognise them later:

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.execute("CREATE TABLE p (a INTEGER, b TEXT)")
con.execute("CREATE TABLE c (x INTEGER REFERENCES p(a))")
for sql in ["SELECT * FROM nothere", "SELECT zzz FROM p", "CREATE TABLE p (a)",
            "SELEC * FROM p", "INSERT INTO p VALUES (1)", "INSERT INTO c VALUES (1)"]:
    try:
        con.execute(sql)
    except sqlite3.Error as e:
        print(f"{sql:28} -> {e}")
```

```
SELECT * FROM nothere        -> no such table: nothere
SELECT zzz FROM p            -> no such column: zzz
CREATE TABLE p (a)           -> table p already exists
SELEC * FROM p               -> near "SELEC": syntax error
INSERT INTO p VALUES (1)     -> table p has 2 columns but 1 values were supplied
INSERT INTO c VALUES (1)     -> foreign key mismatch - "c" referencing "p"
```

1. `no such table` and `no such column` : a typo, the wrong database file, or a missing schema prefix. List the tables and columns (Section 1.9).
2. `table p already exists` : you ran the create script twice. Use `CREATE TABLE IF NOT EXISTS` or drop the table first.
3. `near "SELEC": syntax error` : SQLite points at the first word it could not parse; look just before and at that spot.

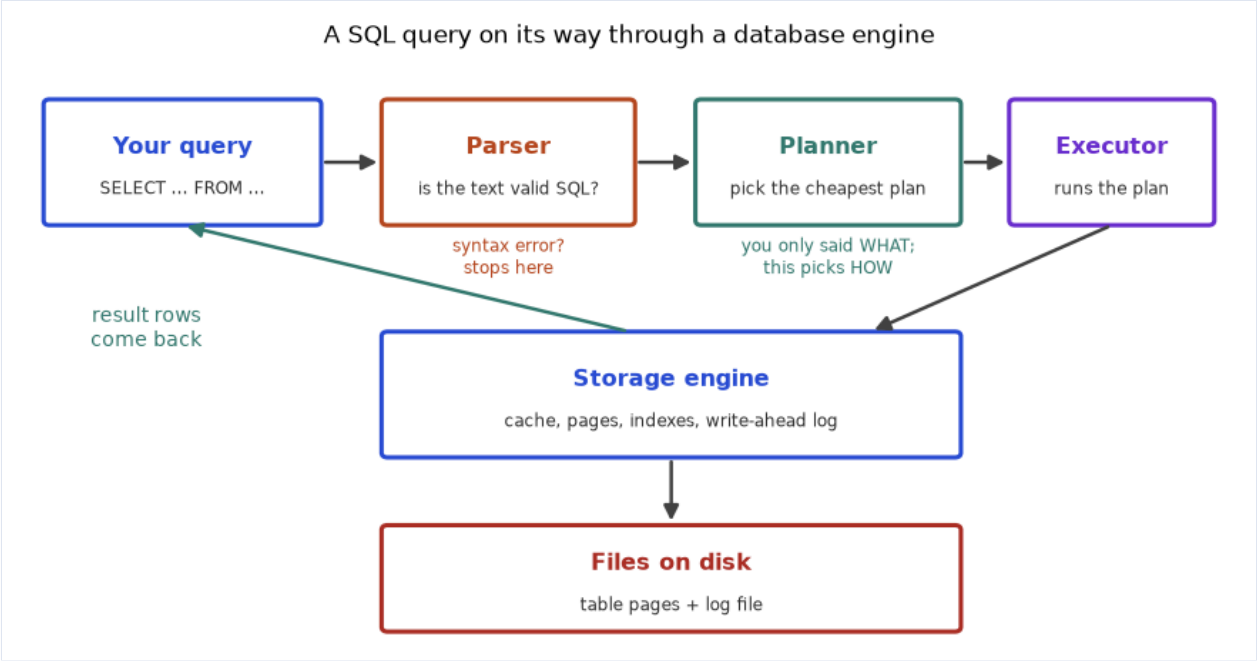
4. `table p has 2 columns but 1 values were supplied` : name the columns in every insert, `INSERT INTO p(a) VALUES (1)` , so that adding a column later does not break it.
5. `foreign key mismatch` : the parent column `p.a` is not declared `PRIMARY KEY` or `UNIQUE` . A foreign key must point at a unique column. This is a design error, not a data error.

1.12 Going Deeper: How a Database Stores, Finds and Protects Rows

You can skip this section on a first read and return after the practice set. It is what separates someone who writes queries from someone who can explain why a query is slow, why a constraint behaved oddly, and what an interviewer is really testing.

How rows are stored: pages and trees

A database does not store rows one by one on disk. It groups them into fixed-size blocks called **pages** (SQLite default 4,096 bytes; PostgreSQL default 8,192 bytes). A page is the smallest unit the engine reads or writes, so "reading one row" always means reading at least one whole page. In SQLite, an ordinary table is a **B-tree** keyed by an internal row number called the `rowid` . A column declared `INTEGER PRIMARY KEY` is that rowid, which is why looking up by it is the fastest possible access. An **index** is a second B-tree, sorted by the indexed column, whose entries hold the column value and the rowid to fetch. PostgreSQL stores tables as unordered **heap** pages and keeps the primary key as a separate B-tree index that points into the heap.



The journey of one query. Only the planner decides the how: it chooses scan or index, join order and so on.

How to read it: arrows go left to right through the three stages that work on the text (parse, plan, execute), then down into the storage engine, which owns the cache, the pages and the write-ahead log. Result rows travel back up to the caller. When you read `EXPLAIN` output in Chapter 10 you are looking at the planner's decision.

Problem 1.12.1 Make a table of 100,000 rows. Compare how the engine finds one row by primary key, by an unindexed column, and by the same column after you add an index. Measure the work with the plan and with a count of virtual-machine steps.

Idea	Without an index the only way to find `user_id = 777` is to look at every row (a *scan*, work proportional to n). With a B-tree the engine descends a few levels (a *search*, work proportional to log n).
Measure	`EXPLAIN QUERY PLAN` prints the plan. `set_progress_handler(f, 1)` calls `f` once per virtual-machine instruction, so counting calls measures the work independently of how fast your laptop is.

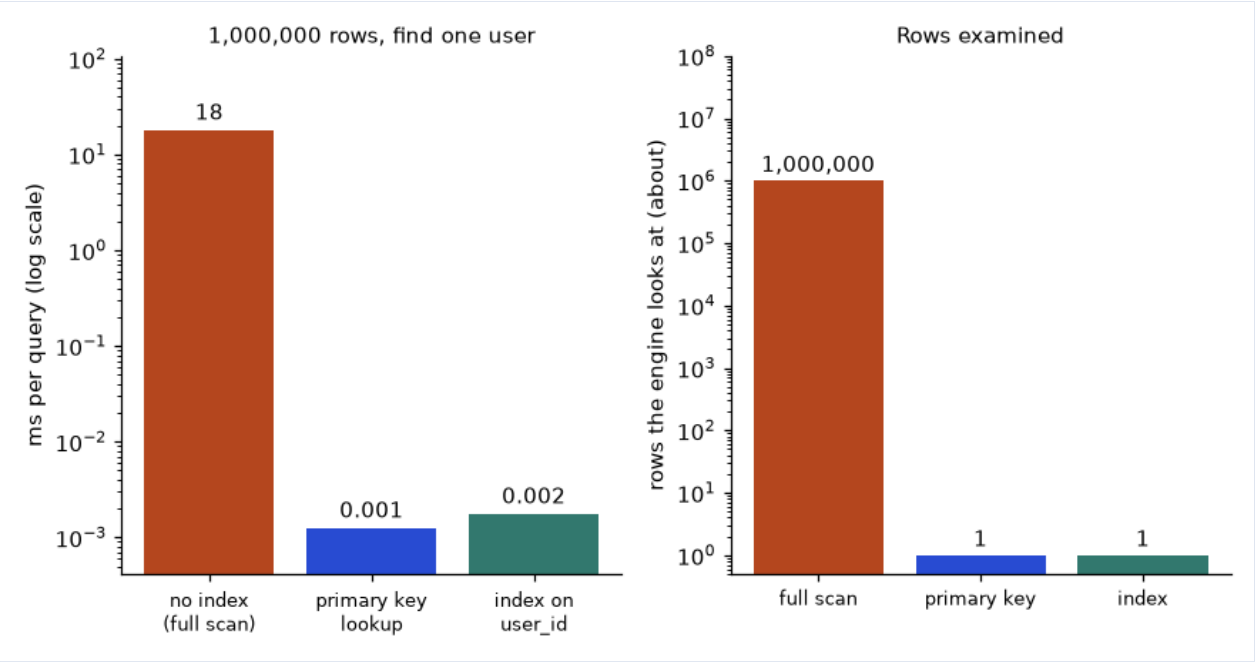
```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE ev (id INTEGER PRIMARY KEY, user_id INTEGER, note TEXT)")
con.executemany("INSERT INTO ev VALUES (?,?,?)",
                ((i, (i * 7919) % 50000, "x" * 20) for i in range(1, 100001)))
con.commit()
print("page size:", con.execute("PRAGMA page_size").fetchone()[0], "bytes")
print("pages used:", con.execute("PRAGMA page_count").fetchone()[0])

def work(sql, arg):
    n = [0]
    def tick():
        n[0] += 1
        return 0
    con.set_progress_handler(tick, 1)
    rows = con.execute(sql, (arg,)).fetchall()
    con.set_progress_handler(None, 1)
    plan = con.execute("EXPLAIN QUERY PLAN " + sql, (arg,)).fetchone()[3]
    return plan, len(rows), n[0]

print(work("SELECT * FROM ev WHERE id = ?", 777))
print(work("SELECT * FROM ev WHERE user_id = ?", 777))
con.execute("CREATE INDEX ev_user ON ev(user_id)")
print(work("SELECT * FROM ev WHERE user_id = ?", 777))
```

```
page size: 4096 bytes
pages used: 794
('SEARCH ev USING INTEGER PRIMARY KEY (rowid=?)', 1, 11)
('SCAN ev', 2, 300016)
('SEARCH ev USING INDEX ev_user (user_id=?)', 2, 25)
```

1. The generator creates 100,000 rows. `(i * 7919) % 50000` spreads user ids over 0 to 49,999, so every user id occurs twice (777 occurs twice).
2. `PRAGMA page_size` and `page_count` show the storage: roughly 794 pages of 4 KB, about 3.2 MB for the table.
3. `work()` runs a query while counting instructions, then asks for the plan. It returns (plan text, rows found, instructions executed).
4. Primary key lookup: plan `SEARCH ... USING INTEGER PRIMARY KEY`, 1 row, 11 instructions.
5. Unindexed column: plan `SCAN ev`, 2 rows, about 300,000 instructions. Every one of the 100,000 rows was examined.
6. After `CREATE INDEX`: plan `SEARCH ... USING INDEX ev_user`, 2 rows, 25 instructions.



Measured on the author's laptop with 1,000,000 rows: a full scan is about 10,000 times slower than a lookup. Timings vary by machine; the ratio is the lesson.

How to read it: the left chart is time per query on a logarithmic axis, so each gridline is ten times the previous. The scan is thousands of times slower than either lookup. The right chart is the reason: how many rows the engine has to touch.

Answer	Scan: about 300,000 steps. Search by key or index: 11 to 25 steps. A factor of roughly 12,000 to 27,000.
Meaning	This is why Chapter 10 exists. The index costs space and slows writes slightly (every insert must also update the tree), so you add indexes for the lookups you actually do, not for every column.

Edge cases that interviewers love

Problem 1.12.2 Predict, then check, four surprising behaviours: NULLs in a UNIQUE column, the empty string versus NULL, case in uniqueness, and a composite foreign key containing a NULL.

E1	UNIQUE treats each NULL as different (NULL is "unknown", and two unknowns are not known to be equal). So many NULLs are allowed. This holds in SQLite, PostgreSQL, MySQL and Oracle for single columns; SQL Server is the exception and allows only one.
E2	The empty string `''` is a real value. It is not NULL, and `'' IS NULL` is false. Two empty strings *do* collide in a UNIQUE column.
E3	SQLite compares text case-sensitively by default, so `'Asha@x.com'` and `'asha@x.com'` are different values and both pass UNIQUE. Fix with `COLLATE NOCASE` on the column (PostgreSQL: a unique index on `lower(email)` or the `citext` type).
E4	A foreign key with several columns is checked only when none of its columns is NULL (the default rule, called MATCH SIMPLE). A child row with one NULL part is accepted even if no parent exists.

```
import sqlite3
con = sqlite3.connect(":memory:", isolation_level=None)   # autocommit, so PRAGMA works
con.execute("PRAGMA foreign_keys = ON")
print("foreign_keys is", con.execute("PRAGMA foreign_keys").fetchone()[0])

def attempt(label, sql, args=()):
    try:
        con.execute(sql, args); print(f"{label:34} accepted")
    except sqlite3.Error as e:
        print(f"{label:34} refused: {e}")

con.execute("CREATE TABLE u (email TEXT UNIQUE)")
attempt("E1 first NULL", "INSERT INTO u VALUES (NULL)")
attempt("E1 second NULL", "INSERT INTO u VALUES (NULL)")
attempt("E2 empty string", "INSERT INTO u VALUES ('')")
attempt("E2 empty string again", "INSERT INTO u VALUES ('')")
print("E2 '' IS NULL ->", con.execute("SELECT '' IS NULL").fetchone()[0])
attempt("E3 asha@x.com", "INSERT INTO u VALUES ('asha@x.com')")
attempt("E3 ASHA@x.com (other case)", "INSERT INTO u VALUES ('ASHA@x.com')")
con.execute("CREATE TABLE u2 (email TEXT UNIQUE COLLATE NOCASE)")
attempt("E3 nocase first", "INSERT INTO u2 VALUES ('asha@x.com')")
attempt("E3 nocase other case", "INSERT INTO u2 VALUES ('ASHA@x.com')")

con.executescript("""
CREATE TABLE par (a INT, b INT, PRIMARY KEY (a, b));
CREATE TABLE chi (a INT, b INT, FOREIGN KEY (a, b) REFERENCES par (a, b));
INSERT INTO par VALUES (1, 1);
""")
attempt("E4 child (1,1) parent exists", "INSERT INTO chi VALUES (1, 1)")
attempt("E4 child (2,2) no parent", "INSERT INTO chi VALUES (2, 2)")
attempt("E4 child (2,NULL) no parent", "INSERT INTO chi VALUES (2, NULL)")
```

```
foreign_keys is 1
E1 first NULL                      accepted
E1 second NULL                     accepted
E2 empty string                    accepted
E2 empty string again              refused: UNIQUE constraint failed: u.email
E2 '' IS NULL -> 0
E3 asha@x.com                      accepted
E3 ASHA@x.com (other case)         accepted
E3 nocase first                    accepted
E3 nocase other case               refused: UNIQUE constraint failed: u2.email
E4 child (1,1) parent exists       accepted
E4 child (2,2) no parent           refused: FOREIGN KEY constraint failed
E4 child (2,NULL) no parent        accepted
```

1. `isolation_level=None` puts the Python module in autocommit mode. This matters: `PRAGMA foreign_keys` is silently ignored while a transaction is open, and Python's default mode opens one before the first data change. Without this line a script that inserts data and then sets the pragma would run with foreign keys still off. The first print confirms the setting is 1.
2. `attempt()` runs one statement and prints accepted or the exact error.
3. E1: the second NULL is accepted. E2: the first empty string is accepted, the second is refused, and `'' IS NULL` prints 0.
4. E3: both case variants are accepted in the plain table, but the `NOCASE` column refuses the second.
5. E4: (1,1) has a parent. (2,2) has no parent and is refused. (2,NULL) is accepted even though no parent (2,anything) exists, because one part is NULL.

Answer	E1 two NULLs both accepted; E2 second '' refused; E3 plain table accepts both cases, NOCASE refuses the second; E4 only (2,2) is refused.
Meaning	Rule of thumb: *NULL turns most rules off*. If a rule matters, add NOT NULL next to it.

COMMON MISTAKE	Running `PRAGMA foreign_keys = ON` after the connection has already started a transaction, then wondering why orphans still get in. The pragma does nothing inside a transaction. Run it immediately after `connect()`, before any data statement.

What a database promises: ACID

Letter	Promise	Mechanism	You saw it in
Atomicity	A transaction happens fully or not at all	Undo information, rollback	Problem 1.10.2
Consistency	Every committed state obeys the constraints	Primary, foreign, CHECK and NOT NULL rules	Sections 1.5 and 1.6
Isolation	Concurrent transactions do not see each other's half-finished work	Locks or MVCC (multiple row versions)	Chapter 9
Durability	Once committed, it survives a crash	Write-ahead log flushed to disk	Chapter 9

The **write-ahead log** (WAL) idea is simple: before changing a data page, the engine writes a short description of the change to a sequential log file and forces it to disk. If the power fails, on restart the engine replays the log. This is cheaper than writing every changed data page immediately, because a sequential append is fast. It is also why "a database survives a crash" is true while "a half-saved CSV" is not.

SQLite versus PostgreSQL internals

Topic	SQLite	PostgreSQL
Process model	A library inside your program; the database is one file	Server process; each client connection has a backend process
Concurrency	Many readers, one writer at a time (WAL mode lets readers continue during a write)	MVCC: readers do not block writers and vice versa; row versions are cleaned by VACUUM
Table storage	B-tree keyed by rowid	Heap of 8 KB pages; indexes are separate
Types	Affinity (flexible); STRICT optional	Strict, rich (arrays, JSONB, ranges, UUID)
Scale sweet spot	Up to a few GB per file, embedded and local analytics	Terabytes, many concurrent users, replication

Row stores and column stores

Everything above describes an **OLTP** (online transaction processing) database: many small reads and writes of whole rows. The rows of a table are stored together, which is ideal for "fetch order 101 with all its fields". Analytical warehouses (BigQuery, Snowflake, Redshift, DuckDB, ClickHouse) are **OLAP** (online analytical processing): they store each column together, so a query such as `SUM(amount)` over a billion rows reads only the one column it needs and compresses it well. The SQL you write is the same; the physical layout differs. Chapter 11 and Chapter 12 build on this distinction.

What interviewers probe about this chapter

Question	A strong answer includes
Primary key versus unique key?	One primary key per table, never NULL; many unique constraints, and NULL behaviour varies by engine (Problem 1.12.2).
Why do we normalise? Why denormalise?	Normalise to avoid update, insert and delete anomalies; denormalise in warehouses to avoid joins at read time and accept redundancy.
Natural versus surrogate key?	Surrogate for stability and compact joins, plus UNIQUE on the natural value.

Question	A strong answer includes
What does a foreign key guarantee, and what does it cost?	No orphans; costs a lookup on child writes and on parent deletes; index the child column for fast parent deletes.
Is a row order guaranteed?	No; only ORDER BY.
Can a foreign key be NULL? Can it point to a non-key column?	Yes it can be NULL; it must point to a unique column (usually the primary key).
What is atomicity? Give an example.	The transfer: debit and credit both happen or neither does.
How would you model many-to-many with attributes?	A bridge table whose primary key is the pair or a surrogate, plus the relationship's own columns (quantity, role, date).
Why can float be dangerous for money?	Binary approximation; use NUMERIC or integer minor units.
Why is a query slow, and what do you try first?	Look at the plan; is it a scan where an index could serve; is the filter sargable; are statistics fresh (Chapter 10).

Connection to the real job

The internals above explain real incidents: a nightly job that suddenly takes hours because a table passed the size where the planner stopped using an index; a deadlock between two services that update the same parents and children in different order; a "unique" email column that allowed 'Asha@x.com' and 'asha@x.com'. When you can name the mechanism, you can fix the cause instead of rerunning the job.

Check yourself

1. Why is looking up by `INTEGER PRIMARY KEY` the fastest lookup in SQLite?

Answer: That column is the rowid, so the table's own B-tree is keyed by it. No second structure is needed.

1. Why does UNIQUE allow several NULLs?

Answer: NULL means unknown, and two unknowns are not known to be equal, so no duplicate is proven.

1. What does the write-ahead log buy you?

Answer: Durability and fast commits: changes are appended to a sequential log first and replayed after a crash.

1.13 Mixed-Practice Drill and Self-Test

The practice set at the end of the chapter goes from easy to expert on one topic at a time. This section does the opposite: it mixes everything, the way an interview or a real task does. Try each item on paper first. The answers follow right after each one.

Ten-minute mixed drill

Drill 1 A table `tickets(ticket_id, user_id, opened_on, closed_on)`. What is one row, what is the key, and which column is allowed to be NULL?
Answer: One row is one support ticket. The key is `ticket_id`. `closed_on` is NULL while the ticket is open, so it must not be declared NOT NULL; the others are mandatory.

Drill 2 A teacher teaches many classes, but each class has exactly one teacher. Where does the foreign key go?
Answer: In `classes`, a column `teacher_id` referencing `teachers(teacher_id)`: the foreign key lives on the "many" side.

Drill 3 Students and clubs: a student can join many clubs and a club has many students. Give the bridge table and its primary key.
Answer: `memberships(student_id, club_id, joined_on)` with `PRIMARY KEY (student_id, club_id)`, each column a foreign key. `joined_on` is a fact about the pair, so it belongs in the bridge.

Drill 4 Table A has 4 rows and table B has 6. How many rows does `A CROSS JOIN B` return, and what is the maximum for an inner join on B's primary key?

Answer: 24 for the cross join. If the join column is B's primary key, each A row matches at most one B row, so at most 4 rows.

Drill 5 Choose a type and rule for each: a postal pincode, a star rating 1 to 5, a price with paise, an "is active" flag, the moment an order was placed.

Answer: Pincode: `TEXT` (leading zeros matter, and you never add pincodes). Rating: `INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5)`. Price: `NUMERIC(12,2)` or integer paise, never float. Flag: `BOOLEAN` (SQLite: integer 0 or 1 with a CHECK). Order moment: `TIMESTAMP` (store in UTC), with `NOT NULL` and a default.

Drill 6 Spot three problems: `CREATE TABLE orders (order_id INT, customer VARCHAR(50), items TEXT, total FLOAT);`

Answer: (1) No primary key, so duplicate orders are possible. (2) `items` as one text cell breaks first normal form; use an `order_items` bridge. (3) `total FLOAT` for money; use `NUMERIC` or integer paise. Also (4) `customer` as a name instead of a foreign key to a customers table repeats a fact.

Drill 7 Pick the delete action: invoice lines when an invoice is deleted; invoices when a customer is deleted; the "assigned_to" link of a ticket when a staff member leaves.

Answer: Invoice lines: CASCADE (they are parts of the invoice). Invoices of a customer: RESTRICT (never lose financial history silently). Ticket assignee: SET NULL (the ticket remains, unassigned).

Drill 8 A column `x` contains 1, NULL, 3. What do `COUNT(*)` and `COUNT(x)` return? And how do rows with a NULL key behave in an inner join and in a left join?

Answer: 3 and 2. In joins, NULL never equals anything, so a row with a NULL key matches nothing: an inner join drops it, a left join keeps it with NULLs on the right.

Drill 9 The query `SELECT SUM(o.total) FROM customers c JOIN orders o ON ... JOIN order_items i ON i.order_id = o.order_id` gives a revenue twice the real number. Why, and what is the quickest check?

Answer: Joining the lines repeats each order once per line, so `o.total` is added once per line (fan-out). Quick check: compare `COUNT(*)` after the join with the number of orders. Fix: sum `qty * unit_price` from the lines instead, or aggregate lines before joining.

Drill 10 A teammate says "the foreign key is slowing our inserts, drop it". How do you respond?

Answer: A foreign key adds one lookup per child insert, usually a cheap index probe. Dropping it trades a small cost for the risk of orphans that corrupt every join and report. Measure first. If bulk loads really are the problem, load into a staging table, validate with an anti-join, then insert, or disable checks only for the load and run `foreign_key_check` after.

The two numeric answers in the drill can be checked in a few lines:

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE a (id INTEGER PRIMARY KEY);
CREATE TABLE b (id INTEGER PRIMARY KEY);
CREATE TABLE t (x INTEGER);
""")
con.executemany("INSERT INTO a VALUES (?)", [(i,) for i in range(1, 5)])
con.executemany("INSERT INTO b VALUES (?)", [(i,) for i in range(1, 7)])
con.executemany("INSERT INTO t VALUES (?)", [(1,), (None,), (3,)])
print("cross join rows       :", con.execute("SELECT COUNT(*) FROM a CROSS JOIN b").fetchone()[0])
print("inner join on b's key :", con.execute("SELECT COUNT(*) FROM a JOIN b ON b.id = a.id").fetchone()[0])
print("COUNT(*), COUNT(x)    :", con.execute("SELECT COUNT(*), COUNT(x) FROM t").fetchone())
```

```
cross join rows       : 24
inner join on b's key : 4
COUNT(*), COUNT(x)    : (3, 2)
```

1. Tables a (4 rows) and b (6 rows) are filled with ids 1 to 4 and 1 to 6.
2. The cross join pairs everything: 4 × 6 = 24.

3. The inner join on the key finds a match for ids 1 to 4 only once each: 4 rows, the stated maximum.
4. `t` holds 1, NULL, 3, so `COUNT(*)` is 3 and `COUNT(x)` is 2.

Self-test: 15 questions

Write your answers on paper (letters only), then check the key below. Each question is worth one point.

1. A DBMS is: (a) a spreadsheet, (b) the program that stores data and enforces rules for many users, (c) a query language, (d) a file format.
2. "Declarative" means: (a) you write loops, (b) you state the result and the engine chooses the steps, (c) you cannot use indexes, (d) queries run slowly.
3. A primary key must be: (a) text, (b) unique and not NULL, (c) unique but may be NULL, (d) a number that never repeats across tables.
4. A foreign key value may be: (a) any text, (b) a primary key value of the parent table, or NULL, (c) only NULL, (d) a new value the parent will create later.
5. Which design needs a bridge table? (a) one department, many employees, (b) one passport per person, (c) students and courses, (d) a country and its capital.
6. A CHECK constraint `CHECK (age >= 0)` on a nullable column: (a) rejects NULL, (b) accepts NULL, (c) accepts negative numbers, (d) is ignored.
7. Best type for money in PostgreSQL: (a) REAL, (b) FLOAT, (c) NUMERIC(12,2), (d) TEXT.
8. In SQLite, foreign keys are enforced: (a) always, (b) only after `PRAGMA foreign_keys = ON` on that connection, (c) never, (d) only for primary keys.
9. A join of a 3-row and a 5-row table with no condition returns: (a) 8 rows, (b) 15 rows, (c) 5 rows, (d) 3 rows.
10. Which is a symptom of an update anomaly? (a) a query is slow, (b) the same customer appears with two different cities, (c) a table has no index, (d) a column is NULL.
11. A column holds `'Java, SQL'`. This breaks: (a) second normal form, (b) first normal form, (c) referential integrity, (d) nothing.
12. `ON DELETE CASCADE` on `orders.customer_id` means: (a) deleting a customer is refused, (b) deleting a customer also deletes the orders, (c) orders are set to NULL, (d) nothing happens.
13. Without `ORDER BY`, the row order of a result is: (a) insertion order always, (b) alphabetical, (c) not guaranteed, (d) by primary key always.
14. Two NULLs in a column with a `UNIQUE` constraint (SQLite, PostgreSQL): (a) are refused, (b) are accepted, (c) crash the server, (d) are converted to 0.
15. Rows with `customer_id` values that have no parent customer are called: (a) duplicates, (b) orphans, (c) tuples, (d) surrogates.

ANSWER KEY	1 b (Section 1.1). 2 b (1.1). 3 b (1.2). 4 b (1.2, 1.6). 5 c (1.4, 1.11). 6 b (1.5). 7 c (1.5). 8 b (1.3). 9 b (1.7). 10 b (1.8). 11 b (1.8). 12 b (1.6). 13 c (1.7). 14 b (1.12). 15 b (1.9).

Score	What it means	What to reread
14 to 15	Solid. You can move on.	Do the expert items of the practice set (P.22 to P.28), then start Chapter 2.
11 to 13	Good, with gaps.	Reread the sections named in the key for each question you missed, and redo their worked problems without looking at the steps.
7 to 10	The ideas are not yet automatic.	Reread 1.2 (keys), 1.4 (cardinality) and 1.5 (constraints), then redo Problems 1.6.1 to 1.6.5 and the drill.
0 to 6	Start the chapter again, slowly.	Read 1.1 and 1.2, run every code block yourself, change one value, predict the effect, then rerun. Come back to this test after a day.

1.14 Where This Chapter Fits: Before and After

This is the first chapter, so nothing comes before it except curiosity. What it gives to the chapters after it:

Chapter	What it takes from here	What it adds
2 SELECT, WHERE, ORDER BY	Tables, rows, columns, the shop database (Figure in 1.9)	Reading data from one table; NULL logic in depth (Problem 1.5.2 previews it)
3 Aggregation and GROUP BY	Grain: one row means one what	COUNT, SUM, grouping; `COUNT(*)` versus `COUNT(col)` (Problem 1.9.2)
4 Joins	Foreign keys, cardinality, row-count estimates (Pattern 8), anti-join (Problem 1.9.3)	All join types, fan-out cures
9 DDL, DML and transactions	Constraints (1.5), foreign key actions (1.6), the transfer example (1.10)	ALTER, upsert, isolation levels, locking
10 Indexes and EXPLAIN	Pages, B-trees, scan versus search (1.12)	Reading plans, index design, tuning
11 Schema design and warehouse modeling	Anomalies, 1NF to 3NF (1.8)	BCNF, star schemas, slowly changing dimensions
13 Beyond SQL	Row store versus column store (1.12), what a relational database promises	Document, key-value and graph stores; CAP

If you remember only three things from this chapter: **know what one row means**, **let the database enforce the rules**, and **store each fact once**. Chapter 2 now teaches you to ask questions of the tables you have just learned to build.

Chapter recap

- A database stores data safely, fast and for many users; SQL is declarative: you state the result and the engine chooses the steps.
- A table is a strict grid: fixed columns, typed values, one row per thing. In theory a table is a set, so rows have no order and no duplicates; only `ORDER BY` gives an order.
- A primary key identifies a row; a foreign key points to another table's key and may be NULL. Composite keys identify bridge-table rows; surrogate keys are stable, so add `UNIQUE` on the natural value.
- Ask "what is the grain?" before using any table, and test a key with `COUNT(*)` versus `COUNT(DISTINCT key)`.
- Constraints (`NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`, `DEFAULT`) make the database refuse bad data. A NULL slips past CHECK and past UNIQUE, so pair them with NOT NULL when a value is mandatory.
- Money is never float: use `NUMERIC` or integer paise. SQLite types are flexible; use `STRICT` tables and think in PostgreSQL's strict rules.
- Foreign key delete actions decide what happens to children: RESTRICT refuses, CASCADE deletes, SET NULL blanks the link. In SQLite turn on `PRAGMA foreign_keys = ON` right after connecting, before any transaction.
- The three core operations are selection (`WHERE`), projection (column list) and join; a join is a Cartesian product followed by a selection, and its row count can be estimated before running it.
- Flat tables cause update, insert and delete anomalies. Split them, then prove the split lossless by joining back and comparing. A comma list in a cell breaks first normal form.
- Explore an unknown database in ten minutes: tables, columns, foreign keys, row counts, NULL rates, orphans.
- From Python, pass values as parameters (never string-build SQL) and wrap multi-step writes in a transaction.
- An index turns a scan of n rows into a B-tree search; always write `ORDER BY` when order matters.
- Many-to-many relationships need a bridge table. The shop database (customers, products, orders, order_items) is used from Chapter 2.

Command / pattern	Meaning
`sqlite3 file.db`	open a SQLite database
`.tables`, `.schema t`	list tables, show a table definition
`\dt`, `\d t` (psql)	list tables, describe a table

Command / pattern	Meaning
`PRIMARY KEY`, `REFERENCES t(col)`	key and foreign key
`NOT NULL CHECK (...)`	mandatory value in a legal range
`ON DELETE CASCADE / SET NULL`	what happens to children when a parent is deleted
`COUNT(*)` vs `COUNT(DISTINCT key)`	is this column set a key
`LEFT JOIN ... WHERE parent.key IS NULL`	anti-join: find orphans
`PRAGMA table_info(t)`, `foreign_key_list(t)`, `foreign_key_check`	SQLite introspection
`con.execute(sql, (value,))`	safe parameterised query
`with con:`	commit on success, rollback on error
rows A × rows B	size of a Cartesian product (upper bound of a join)

Practice set

Problem 1.P.1 (easy) Name the primary key of `order_items` in the shop design. **Answer:** The pair `(order_id, product_id)`, a composite key.

Problem 1.P.2 (easy) True or false: a foreign key value can be NULL. **Answer:** True, unless the column is also `NOT NULL`. NULL means "no parent".

Problem 1.P.3 (what is the output?) After the earlier inserts, what does this print?

```
SELECT COUNT(*) AS n FROM employees WHERE dept_id = 10;
```

```
n
-
2
```

Answer: Shown above: 2 (Asha and Meena).

Problem 1.P.4 (find the bug) A table is defined `CREATE TABLE t (id INTEGER, name TEXT);` and has duplicate ids. What is missing? **Answer:** A `PRIMARY KEY` (or `UNIQUE`) constraint on `id`.

Problem 1.P.5 (easy) Why is `REAL` a bad type for money? **Answer:** It is approximate; `0.1 + 0.2` is not exactly `0.3`. Use `NUMERIC` / `DECIMAL` or store integer paise.

Problem 1.P.6 Show the sum of salaries in `employees`.

```
SELECT SUM(salary) AS total_salary FROM employees;
```

```
total_salary
------------
305000
```

Answer: 305000 (`90000 + 120000 + 95000`).

Problem 1.P.7 Insert a new department `(30,'Cloud')` and show the table.

```
INSERT INTO departments VALUES (30,'Cloud');
SELECT * FROM departments;
```

```
dept_id  dept_name
-------  ---------
10       Testing
20       Data
30       Cloud
```

Answer: Three departments are shown. Department 30 has no employees, which is allowed in a 1:N link.

Problem 1.P.8 (scenario) Your manager gives you `sales.csv` with columns `order_id, customer_name, customer_city, product_name, price`. What tables would you create? **Answer:** `customers`, `products`, `orders` and `order_items`, with keys. Customer name and city repeat for every order in the CSV; they belong in `customers` once.

Problem 1.P.9 (interview) Explain primary key vs unique key. **Answer:** Both enforce uniqueness. A table has only one primary key and it cannot be NULL. A table can have many unique constraints, and in most databases a unique column may contain NULL values (details vary by database).

Problem 1.P.10 (interview) Why is a surrogate key often preferred? **Answer:** It never changes, is small and fast to join. Natural keys such as email or phone can change, and changing a key means updating all child rows.

Problem 1.P.11 (easy) In the shop database, what does one row of `order_items` represent, and why is `order_id` alone not its key? **Answer:** One row is one product on one order (one order line). An order with two products has two rows with the same `order_id`, so the key must be the pair `(order_id, product_id)`.

Problem 1.P.12 (easy) Name the relationship between `customers` and `orders` and the column that implements it. **Answer:** One-to-many: one customer has many orders. The column is `orders.customer_id`, a foreign key to `customers.customer_id`, on the many side.

Problem 1.P.13 (what is the output?) What does this print? Work it out first.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE t (a INTEGER UNIQUE)")
con.executemany("INSERT INTO t VALUES (?)", [(None,), (None,), (1,)])
print(con.execute("SELECT COUNT(*), COUNT(a), COUNT(DISTINCT a) FROM t").fetchone())
```

```
(3, 1, 1)
```

Answer: `(3, 1, 1)`. UNIQUE allows both NULLs, so three rows are stored. `COUNT(a)` ignores the two NULLs and counts 1. `COUNT(DISTINCT a)` counts distinct non-NULL values: 1.

Problem 1.P.14 (what is the output?) SQLite divides integers differently from what many people expect. Predict the six values.

```
import sqlite3
con = sqlite3.connect(":memory:")
print(con.execute("SELECT typeof(1), typeof(1.0), typeof('1'), typeof(NULL), 1/2, 1/2.0").fetchone())
```

```
('integer', 'real', 'text', 'null', 0, 0.5)
```

Answer: `('integer', 'real', 'text', 'null', 0, 0.5)`. Dividing two integers gives an integer (0), as in PostgreSQL. Make one side real (`1/2.0` or `CAST(1 AS REAL)/2`) to get 0.5. This catches people computing percentages: `qty * 100 / total` with integers truncates.

Problem 1.P.15 (find the bug) The team defined `order_items` with `PRIMARY KEY (order_id)`. What happens when order 101 has two lines, and what is the fix?

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE order_items (order_id INTEGER, product_id INTEGER, qty INTEGER, PRIMARY KEY (order_id))")
con.execute("INSERT INTO order_items VALUES (101, 1, 1)")
try:
    con.execute("INSERT INTO order_items VALUES (101, 3, 2)")
except sqlite3.Error as e:
    print("bug:", e)
con.execute("CREATE TABLE order_items2 (order_id INTEGER, product_id INTEGER, qty INTEGER, PRIMARY KEY (order_id,
product_id))")
con.execute("INSERT INTO order_items2 VALUES (101, 1, 1)")
con.execute("INSERT INTO order_items2 VALUES (101, 3, 2)")
print("fixed rows:", con.execute("SELECT COUNT(*) FROM order_items2").fetchone()[0])
```

```
bug: UNIQUE constraint failed: order_items.order_id
fixed rows: 2
```

Answer: The second line of order 101 is refused because `order_id` alone must be unique, so an order could never hold two products. The key must be the composite `(order_id, product_id)`, which allows many lines per order but only one line per product within an order.

Problem 1.P.16 (medium) A hotel must never book the same room for the same night twice. Design a table where the database itself enforces this, and prove it. **Answer:** Store one row per room per night and make the pair the primary key. A booking of three nights becomes three rows pointing to one booking id.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE room_nights (
  room_id    INTEGER NOT NULL,
  night      TEXT    NOT NULL,
  booking_id INTEGER NOT NULL,
  PRIMARY KEY (room_id, night)
);
INSERT INTO room_nights VALUES (101,'2024-07-01',1),(101,'2024-07-02',1),(101,'2024-07-03',1);
""")
try:
    con.execute("INSERT INTO room_nights VALUES (101,'2024-07-03',2)")   # overlaps the last night
except sqlite3.Error as e:
    print("overlap refused:", e)
con.execute("INSERT INTO room_nights VALUES (101,'2024-07-04',2)")       # next free night is fine
print(con.execute("SELECT booking_id, COUNT(*) FROM room_nights GROUP BY booking_id").fetchall())
```

```
overlap refused: UNIQUE constraint failed: room_nights.room_id, room_nights.night
[(1, 3), (2, 1)]
```

The second booking tried night 3 July, which booking 1 already holds, and was refused by the primary key. 4 July is free and accepted. The point: a rule such as "no double booking" becomes a key when you pick the right grain (one row per room-night). PostgreSQL can also express "no overlapping date ranges" directly with an exclusion constraint on a range type.

Problem 1.P.17 (medium) Blog posts have many tags and a tag labels many posts. Write the three tables, insert two posts and three tags, and list each post with its tags. **Answer:**

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE posts (post_id INTEGER PRIMARY KEY, title TEXT NOT NULL);
CREATE TABLE tags  (tag_id  INTEGER PRIMARY KEY, label TEXT NOT NULL UNIQUE);
CREATE TABLE post_tags (
  post_id INTEGER NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
  tag_id  INTEGER NOT NULL REFERENCES tags(tag_id)   ON DELETE CASCADE,
  PRIMARY KEY (post_id, tag_id)
);
INSERT INTO posts VALUES (1,'Intro to SQL'),(2,'Keys explained');
INSERT INTO tags  VALUES (1,'sql'),(2,'beginner'),(3,'design');
INSERT INTO post_tags VALUES (1,1),(1,2),(2,1),(2,3);
""")
for row in con.execute("""SELECT p.title, t.label FROM posts p
                          JOIN post_tags pt ON pt.post_id = p.post_id
                          JOIN tags t ON t.tag_id = pt.tag_id
                          ORDER BY p.post_id, t.label"""):
    print(row)
```

```
('Intro to SQL', 'beginner')
('Intro to SQL', 'sql')
('Keys explained', 'design')
('Keys explained', 'sql')
```

The bridge table `post_tags` has a composite primary key (a post cannot carry the same tag twice) and cascading deletes (deleting a post removes only its links, never the tags themselves). Post 1 has two tags, post 2 has two tags, and the tag `sql` is shared.

Problem 1.P.18 (medium) Choose RESTRICT, CASCADE or SET NULL and justify: (a) `order_items.order_id` when an order is deleted; (b) `orders.customer_id` when a customer is deleted; (c) `employees.manager_id` when a manager is deleted; (d) `sessions.user_id` when a user is deleted under a privacy rule that demands erasure. **Answer:** (a) CASCADE: lines have no meaning without their order. (b) RESTRICT: never silently destroy order history; to remove a customer you decide deliberately (anonymise or archive). (c) SET NULL: the employees remain, temporarily without a manager. (d) CASCADE: when erasure is required, session rows must go with the user; document that choice and log the deletion.

Problem 1.P.19 (medium) Three customers have 2, 0 and 3 orders; every order has exactly 2 lines. Without running anything, how many rows do (a) customers JOIN orders, (b) customers LEFT JOIN orders, (c) customers LEFT JOIN orders LEFT JOIN order_items return? Then confirm. **Answer:** Orders: 2 + 0 + 3 = 5. (a) 5. (b) 5 matched plus 1 for the customer with no orders = 6. (c) The 5 orders become 5 × 2 = 10 rows, plus 1 for the customer with no orders = 11.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER);
CREATE TABLE order_items (order_id INTEGER, line INTEGER, PRIMARY KEY (order_id, line));
INSERT INTO customers VALUES (1),(2),(3);
INSERT INTO orders VALUES (1,1),(2,1),(3,3),(4,3),(5,3);
INSERT INTO order_items SELECT order_id, 1 FROM orders UNION ALL SELECT order_id, 2 FROM orders;
""")
a = "SELECT COUNT(*) FROM customers c JOIN orders o ON o.customer_id = c.customer_id"
b = "SELECT COUNT(*) FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id"
c = b + " LEFT JOIN order_items i ON i.order_id = o.order_id"
print([con.execute(q).fetchone()[0] for q in (a, b, c)])
```

```
[5, 6, 11]
```

The output `[5, 6, 11]` matches the hand count. (`UNION ALL`, used only to generate the data, stacks two result sets; Chapter 7.)

Problem 1.P.20 (hard) A flat table `enrolment_flat(student, student_email, course, course_title, instructor, instructor_office, grade)` has primary key `(student, course)`. Find the partial dependencies and the transitive dependency, and decompose it. **Answer:** `student_email` depends on `student` only (partial dependency, breaks 2NF). `course_title` and `instructor` depend on `course` only (partial). `instructor_office` depends on `instructor`, which is not a key (transitive, breaks 3NF). Only `grade` depends on the whole key. Decomposition: `students(student_id, email)`, `instructors(instructor_id, name, office)`, `courses(course_id, title, instructor_id)` and `enrolments(student_id, course_id, grade)`. Check that it builds and rejoins:

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE students    (student_id INTEGER PRIMARY KEY, email TEXT NOT NULL UNIQUE);
CREATE TABLE instructors (instructor_id INTEGER PRIMARY KEY, name TEXT NOT NULL, office TEXT);
CREATE TABLE courses     (course_id INTEGER PRIMARY KEY, title TEXT NOT NULL,
                          instructor_id INTEGER NOT NULL REFERENCES instructors(instructor_id));
CREATE TABLE enrolments  (student_id INTEGER NOT NULL REFERENCES students(student_id),
                          course_id  INTEGER NOT NULL REFERENCES courses(course_id),
                          grade INTEGER, PRIMARY KEY (student_id, course_id));
INSERT INTO students VALUES (1,'asha@x.com'),(2,'ravi@x.com');
INSERT INTO instructors VALUES (1,'Dr Rao','B-12');
INSERT INTO courses VALUES (10,'Databases',1),(20,'Statistics',1);
INSERT INTO enrolments VALUES (1,10,88),(1,20,75),(2,10,91);
""")
print(con.execute("""SELECT s.email, c.title, i.name, i.office, e.grade
                     FROM enrolments e JOIN students s USING (student_id)
                                       JOIN courses c USING (course_id)
                                       JOIN instructors i USING (instructor_id)
                     ORDER BY s.student_id, c.course_id""").fetchall())
```

```
[('asha@x.com', 'Databases', 'Dr Rao', 'B-12', 88), ('asha@x.com', 'Statistics', 'Dr Rao', 'B-12', 75), ('ravi@x.com',
'Databases', 'Dr Rao', 'B-12', 91)]
```

The office 'B-12' is stored once. If Dr Rao moves office, one row changes instead of one row per enrolment.

Problem 1.P.21 (hard) After a migration, some `orders.customer_id` values point to customers that were never loaded. Write the query that lists them with a count per missing customer, and give two defensible ways to repair the data. **Answer:** Anti-join, then group the orphans by the missing id.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER);
INSERT INTO customers VALUES (1,'Asha'),(2,'Ravi');
INSERT INTO orders VALUES (101,1),(102,9),(103,9),(104,2),(105,7);
""")
print(con.execute("""SELECT o.customer_id AS missing_customer, COUNT(*) AS orphan_orders
                     FROM orders o LEFT JOIN customers c ON c.customer_id = o.customer_id
                     WHERE c.customer_id IS NULL
                     GROUP BY o.customer_id ORDER BY o.customer_id""").fetchall())
```

```
[(7, 1), (9, 2)]
```

Output `[(7, 1), (9, 2)]`: customer 9 is missing for two orders and customer 7 for one. Repair option 1: obtain the missing customer rows from the source system and insert them, which restores the truth. Option 2, when the source cannot supply them: insert placeholder customers (for example name

'Unknown customer 9') flagged so that reports can exclude or highlight them. Deleting the orphan orders is the worst option because it silently removes revenue. After repair, turn foreign keys on or add the constraint so it cannot recur.

Problem 1.P.22 (hard) How do you stop an employee being their own manager, and what can a simple constraint not prevent? **Answer:** A CHECK that compares two columns of the same row: `CHECK (manager_id IS NULL OR manager_id <> emp_id)`. The `IS NULL` part keeps the top person legal, because a CHECK with NULL passes anyway but being explicit documents intent. A CHECK cannot prevent a longer cycle (A manages B, B manages A), because it sees only one row. That needs a trigger, a recursive check in application code, or a model such as a closure table.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("""CREATE TABLE emp (emp_id INTEGER PRIMARY KEY, manager_id INTEGER REFERENCES emp(emp_id),
               CHECK (manager_id IS NULL OR manager_id <> emp_id))""")
con.execute("INSERT INTO emp VALUES (1, NULL)")
for row in [(2, 1), (3, 3)]:
    try:
        con.execute("INSERT INTO emp VALUES (?,?)", row); print(row, "accepted")
    except sqlite3.Error as e:
        print(row, "refused:", e)
con.execute("UPDATE emp SET manager_id = 2 WHERE emp_id = 1")     # 1 -> 2 -> 1 is a cycle
print("cycle created:", con.execute("SELECT emp_id, manager_id FROM emp ORDER BY emp_id").fetchall())
```

```
(2, 1) accepted
(3, 3) refused: CHECK constraint failed: manager_id IS NULL OR manager_id <> emp_id
cycle created: [(1, 2), (2, 1)]
```

The self-manager row (3,3) is refused. The two-person cycle is accepted, which shows the limit of a row-level check.

Problem 1.P.23 (interview) A teammate proposes `email` as the primary key of `users` because "it is already unique". Give a balanced answer. **Answer:** Pros: it is a real identifier users recognise, and no extra column is needed. Cons: emails change (marriage, new job, typo fixes), and a changed primary key must be copied into every child table and every log; emails are longer than an integer, making every index and join bigger; case and whitespace variants can sneak in. Recommended: a surrogate `user_id` as the primary key plus `UNIQUE` on a normalised email (lower-cased, trimmed), and `NOT NULL` since UNIQUE alone permits several NULLs.

Problem 1.P.24 (expert) A nightly job loads `orders.csv` and one night it runs twice, doubling the rows. Make the load safe to repeat (*idempotent*). **Answer:** Give the table a primary key on the business identifier and insert with a conflict rule: SQLite `INSERT OR IGNORE` or the standard `ON CONFLICT DO NOTHING` (PostgreSQL: `INSERT ... ON CONFLICT (order_id) DO NOTHING`). To update changed rows instead, use `DO UPDATE` (an upsert, Chapter 9).

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE orders (order_id INTEGER PRIMARY KEY, amount INTEGER NOT NULL)")
batch = [(101, 500), (102, 300), (103, 700)]
for run in (1, 2):
    con.executemany("INSERT INTO orders VALUES (?,?) ON CONFLICT (order_id) DO NOTHING", batch)
    print("after run", run, ":", con.execute("SELECT COUNT(*), SUM(amount) FROM orders").fetchone())
```

```
after run 1 : (3, 1500)
after run 2 : (3, 1500)
```

Both runs leave 3 rows and a total of 1500. Without the key and conflict rule, run 2 would give 6 rows and 3000, a classic double-counted revenue incident.

Problem 1.P.25 (expert) A table has 50 million rows of about 100 bytes each, in 8 KB pages, and an index with about 400 entries per page. Estimate the table size and the number of page reads to find one row through the index. **Answer:** About 81 rows fit in a page (8,192 / 100, ignoring page overhead), so 50,000,000 / 81 is about 617,000 table pages, roughly 5 GB. The index has 50 million entries at 400 per page: leaves = 50,000,000 / 400 = 125,000

pages; the next level up = 125,000 / 400 = 313 pages; the next = 1 page (the root). So 3 levels. One lookup reads about 3 index pages plus 1 table page = 4 page reads, against about 617,000 for a full scan.

```
import math
rows, row_bytes, page_bytes, fanout = 50_000_000, 100, 8192, 400
rows_per_page = page_bytes // row_bytes
table_pages = math.ceil(rows / rows_per_page)
print("rows per page:", rows_per_page, " table pages:", table_pages, " size GB:", round(table_pages * page_bytes / 1e9, 2))
level, nodes = 1, math.ceil(rows / fanout)
print("index level", level, "pages:", nodes)
while nodes > 1:
    level += 1
    nodes = math.ceil(nodes / fanout)
    print("index level", level, "pages:", nodes)
print("page reads for one lookup:", level + 1, "vs full scan:", table_pages)
```

```
rows per page: 81  table pages: 617284  size GB: 5.06
index level 1 pages: 125000
index level 2 pages: 313
index level 3 pages: 1
page reads for one lookup: 4 vs full scan: 617284
```

The code gives the same numbers. Real indexes hold fewer entries per page than the ideal, so expect 3 to 4 levels: depth grows with the logarithm of the data, which is why a table 100 times bigger still needs only one or two more reads.

Problem 1.P.26 (expert) Two users sign up at the same moment with the name 'asha'. The application first checks "does this name exist?" and then inserts. Show how both succeed, and the fix. **Answer:** Check-then-insert is a race: both checks happen before either insert. The only safe guard is a `UNIQUE` constraint, which the database enforces atomically, plus code that treats the constraint error as "name taken". The script below replays the interleaving by hand on two connections:

```
import sqlite3, tempfile, os
path = os.path.join(tempfile.mkdtemp(), "race.db")

def run(with_unique):
    setup = sqlite3.connect(path, isolation_level=None)
    setup.execute("DROP TABLE IF EXISTS users")
    setup.execute("CREATE TABLE users (name TEXT" + (" UNIQUE" if with_unique else "") + ")")
    setup.close()
    a = sqlite3.connect(path, isolation_level=None)
    b = sqlite3.connect(path, isolation_level=None)
    exists_a = a.execute("SELECT COUNT(*) FROM users WHERE name='asha'").fetchone()[0]   # A checks
    exists_b = b.execute("SELECT COUNT(*) FROM users WHERE name='asha'").fetchone()[0]   # B checks
    a.execute("INSERT INTO users VALUES ('asha')")                                       # A inserts
    try:
        b.execute("INSERT INTO users VALUES ('asha')")                                   # B inserts
        outcome = "B inserted too"
    except sqlite3.Error as e:
        outcome = "B refused: " + str(e)
    n = a.execute("SELECT COUNT(*) FROM users").fetchone()[0]
    return exists_a, exists_b, outcome, n

print("no constraint  :", run(False))
print("with UNIQUE    :", run(True))
```

```
no constraint  : (0, 0, 'B inserted too', 2)
with UNIQUE    : (0, 0, 'B refused: UNIQUE constraint failed: users.name', 1)
```

Without the constraint both checks saw 0 rows, both inserts succeeded, and the table now has two 'asha' rows. With `UNIQUE` the database refuses the second insert even though the application's check was fooled. Never rely on "check then insert" for uniqueness.

Problem 1.P.27 (expert) What does `ON UPDATE CASCADE` do, and why do surrogate keys make it rare? **Answer:** If the parent's key value changes, the matching foreign key values in the children are rewritten to the new value, keeping the link. It is needed only when keys change, which happens with natural keys (a product code renamed). A surrogate key never changes, so the action seldom matters. Demonstration:

```
import sqlite3
con = sqlite3.connect(":memory:", isolation_level=None)
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE products (code TEXT PRIMARY KEY);
CREATE TABLE sales (sale_id INTEGER PRIMARY KEY, code TEXT REFERENCES products(code) ON UPDATE CASCADE);
INSERT INTO products VALUES ('KB-1');
INSERT INTO sales VALUES (1,'KB-1'),(2,'KB-1');
""")
con.execute("UPDATE products SET code = 'KB-100' WHERE code = 'KB-1'")
print(con.execute("SELECT sale_id, code FROM sales ORDER BY sale_id").fetchall())
```

```
[(1, 'KB-100'), (2, 'KB-100')]
```

Both sales rows now say `KB-100`. With a surrogate `product_id` the rename would be a single update of a non-key column and no cascade would be needed.

Problem 1.P.28 (interview, design) Design a schema for a ride-hailing app: riders, drivers, vehicles, trips and payments. State each table's key, the foreign keys and three constraints, then test one. **Answer:** `riders(rider_id)`, `drivers(driver_id)`, `vehicles(vehicle_id, driver_id, plate UNIQUE)` (a driver can have several vehicles; a vehicle belongs to one driver), `trips(trip_id, rider_id, driver_id, vehicle_id, started_at, ended_at, fare_paise)` and `payments(payment_id, trip_id UNIQUE, amount_paise, status)` (one payment per trip). Constraints: `fare_paise >= 0`; `ended_at` is NULL or not before `started_at`; payments status in a fixed list; a trip's vehicle belongs to its driver (this one needs a composite foreign key `(driver_id, vehicle_id)` referencing `vehicles(driver_id, vehicle_id)`).

```
import sqlite3
con = sqlite3.connect(":memory:", isolation_level=None)
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE riders  (rider_id  INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE drivers (driver_id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE vehicles (
  vehicle_id INTEGER PRIMARY KEY, driver_id INTEGER NOT NULL REFERENCES drivers(driver_id),
  plate TEXT NOT NULL UNIQUE, UNIQUE (driver_id, vehicle_id));
CREATE TABLE trips (
  trip_id INTEGER PRIMARY KEY,
  rider_id INTEGER NOT NULL REFERENCES riders(rider_id),
  driver_id INTEGER NOT NULL, vehicle_id INTEGER NOT NULL,
  started_at TEXT NOT NULL, ended_at TEXT, fare_paise INTEGER NOT NULL CHECK (fare_paise >= 0),
  CHECK (ended_at IS NULL OR ended_at >= started_at),
  FOREIGN KEY (driver_id, vehicle_id) REFERENCES vehicles(driver_id, vehicle_id));
CREATE TABLE payments (
  payment_id INTEGER PRIMARY KEY, trip_id INTEGER NOT NULL UNIQUE REFERENCES trips(trip_id),
  amount_paise INTEGER NOT NULL CHECK (amount_paise >= 0),
  status TEXT NOT NULL CHECK (status IN ('pending','paid','failed')));
INSERT INTO riders VALUES (1,'Asha');
INSERT INTO drivers VALUES (1,'Ravi'),(2,'Meena');
INSERT INTO vehicles VALUES (1,1,'MH12AB1234'),(2,2,'KA01XY9999');
""")
for label, args in [("ok: driver 1 in own vehicle", (1,1,1)),
                    ("bad: driver 1 in driver 2's vehicle", (1,1,2))]:
    try:
        con.execute("INSERT INTO trips(rider_id,driver_id,vehicle_id,started_at,fare_paise) VALUES (?,?,?,'2024-06-01
10:00',25000)", args)
        print(label, "-> accepted")
    except sqlite3.Error as e:
        print(label, "-> refused:", e)
```

```
ok: driver 1 in own vehicle -> accepted
bad: driver 1 in driver 2's vehicle -> refused: FOREIGN KEY constraint failed
```

The composite foreign key makes the database reject a trip that pairs driver 1 with a vehicle owned by driver 2. Note the extra `UNIQUE (driver_id, vehicle_id)` on `vehicles`: a foreign key must reference a unique column set, and `vehicle_id` alone being unique is not enough for the pair.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data and AI roles, and English is my second language. Teach me Chapter 1 topic: "relational model, tables, primary and foreign keys". Explain step by step in simple English with a small example of customers and orders. After each step, ask me one question and wait for my answer. At the end, check whether I truly understand keys and cardinality, and tell me which part to revisit.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: database keys and relationships (primary, foreign, surrogate, natural, composite, one-to-many, many-to-many). Give me 8 graded exercises, easy to hard, where I design tables for a scenario such as a library or a hospital. Do not give solutions yet. After I reply with my attempts, review them and point out mistakes in plain English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a beginner in SQL. Quiz me, one question at a time, on SQL basics from Chapter 1: what a DBMS does, declarative vs imperative, tables, rows, keys, NULL, data types, SQLite vs PostgreSQL. Wait for my answer before the next question. Start easy and increase the difficulty. After 10 questions, give a score and a study plan.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning SQL. Review my table design below for keys, data types, and normalization mistakes. Explain every problem in simple English and show a corrected CREATE TABLE script. My design: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data-analyst interviews. Run a mock interview on database fundamentals: relational model, primary vs foreign keys, why we split tables, SQLite vs PostgreSQL, choosing data types. Ask one question at a time, grade each answer out of 10, show a model answer, and give me one improvement tip after each.
```

CHAPTER 2

SELECT, WHERE, ORDER BY, LIMIT, DISTINCT AND NULL LOGIC

Reading data from one table: filtering, sorting, removing duplicates, and understanding the strange behaviour of NULL.

Every data job starts with the same request: "show me the rows that matter." Which customers live in Pune? Which three products cost the most? Which orders have no coupon? In this chapter you learn the five tools that answer these questions on a single table: `SELECT`, `WHERE`, `ORDER BY`, `LIMIT` and `DISTINCT`. Then you meet the one idea that fools even experienced engineers, `NULL`. By the end you can read any simple table, and you can predict what a query returns before you run it.

2.1 SELECT, Columns, Expressions and Aliases

INTUITION

`SELECT` is a window cut into a table. You choose which columns you want to look at, and you can even make new columns by calculating. Think of a school mark sheet: you pick which columns to show (name, maths, science), and add a calculated one such as percentage.

Before we start, create the **shop database** used in the whole book. It is small on purpose, so you can check every answer by eye. Customers 7 and 8 never ordered. Product 6 never sold. Order 104 is cancelled and 109 is refunded. Several `NULL` values exist (`referrer_id`, `coupon`). The script below is loaded automatically before every example from here on.

```
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  name        TEXT NOT NULL,
  city        TEXT,
  signup_date TEXT,
  referrer_id INTEGER
);
INSERT INTO customers VALUES
 (1,'Asha','Pune','2024-01-05',NULL),
 (2,'Ravi','Mumbai','2024-01-12',1),
 (3,'Meena','Pune','2024-01-20',1),
 (4,'John','Delhi','2024-02-02',2),
 (5,'Priya','Bengaluru','2024-02-14',NULL),
 (6,'Kabir','Mumbai','2024-03-01',5),
 (7,'Sana','Delhi','2024-03-15',NULL),
 (8,'Tom','Pune','2024-04-01',3);

CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  name       TEXT NOT NULL,
  category   TEXT NOT NULL,
  price      INTEGER NOT NULL
);
INSERT INTO products VALUES
 (1,'Keyboard','Electronics',1500),
 (2,'Mouse','Electronics',600),
 (3,'Notebook','Stationery',80),
 (4,'Pen Pack','Stationery',120),
 (5,'Desk Lamp','Home',900),
 (6,'Mug','Home',250);

CREATE TABLE orders (
  order_id    INTEGER PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
  order_date  TEXT NOT NULL,
  status      TEXT NOT NULL,
  coupon      TEXT
);
INSERT INTO orders VALUES
 (101,1,'2024-01-10','paid',NULL),
 (102,2,'2024-01-15','paid','SAVE10'),
 (103,1,'2024-02-03','paid',NULL),
 (104,3,'2024-02-10','cancelled',NULL),
 (105,4,'2024-02-20','paid','SAVE10'),
 (106,1,'2024-03-05','paid',NULL),
 (107,5,'2024-03-07','paid',NULL),
 (108,2,'2024-03-18','paid',NULL),
 (109,5,'2024-03-25','refunded',NULL),
 (110,6,'2024-03-30','paid','SAVE10'),
 (111,1,'2024-04-02','paid',NULL),
 (112,4,'2024-04-09','paid',NULL),
 (113,5,'2024-04-15','paid',NULL),
 (114,2,'2024-04-20','paid',NULL);

CREATE TABLE order_items (
  order_id   INTEGER NOT NULL REFERENCES orders(order_id),
  product_id INTEGER NOT NULL REFERENCES products(product_id),
  qty        INTEGER NOT NULL,
  unit_price INTEGER NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
INSERT INTO order_items VALUES
 (101,1,1,1500),(101,3,2,80),
 (102,2,1,600),
 (103,5,1,900),(103,4,1,120),
```

```
  (104,3,5,80),
  (105,1,1,1500),
  (106,2,2,600),(106,3,1,80),
  (107,4,3,120),
  (108,5,1,900),
  (109,1,1,1500),
  (110,3,4,80),(110,4,1,120),
  (111,2,1,600),
  (112,5,2,900),
  (113,3,2,80),(113,4,2,120),
  (114,1,1,1500),(114,2,1,600);
```

If you want to follow along yourself, save that script as `shop.sql`, then run `sqlite3 shop.db < shop.sql`.

Words you need

- **Query**: one SQL question sent to the database.
- **Clause**: one part of a query, such as `FROM` or `WHERE`.
- **Expression**: anything that produces a value: a column name, a number, `qty * unit_price`.
- **Alias**: a nickname for a column or table, given with `AS`.

Real-world mapping

SQL idea	Example	Where it appears in the job
Column list	`SELECT name, city`	choosing report columns
Expression	`qty * unit_price AS line_total`	revenue, margin, error-rate columns
`LIMIT 10`	first ten rows	quick look at a new table

Syntax

```
SELECT  <expressions>
FROM    <table>;
```

- `*` means "all columns". Handy for a quick look. Avoid it in production code (Chapter 10 explains why).
- `AS` gives an **alias**, a new name for a column or table. Aliases make results readable.
- An expression can use `+ - * /`, functions and `||` (text join in SQLite and PostgreSQL).

Worked examples

Problem 2.1.1 List each customer's name and city. Predict first: how many rows should come back? (There are 8 customers, so 8.)

Step 1	Pick the two columns after `SELECT`.
Step 2	Name the table after `FROM`.

```
SELECT name, city FROM customers;
```

```
name   city
-----  ---------
Asha   Pune
Ravi   Mumbai
Meena  Pune
John   Delhi
Priya  Bengaluru
Kabir  Mumbai
Sana   Delhi
Tom    Pune
```

- **Answer** Eight rows, one per customer.
- **Check** The count of printed rows is 8, the same as the customers table.
- **Meaning** `SELECT` with no `WHERE` returns every row, only fewer columns.

Problem 2.1.2 For every order line, compute the line total `qty × unit_price`. Show the first five lines.

Step 1	Write the expression and give it the alias `line_total`.
Step 2	`LIMIT 5` keeps five rows.

```
SELECT order_id, product_id, qty, unit_price,
       qty * unit_price AS line_total
FROM order_items
LIMIT 5;
```

```
order_id  product_id  qty  unit_price  line_total
--------  ----------  ---  ----------  ----------
101       1           1    1500        1500
101       3           2    80          160
102       2           1    600         600
103       5           1    900         900
103       4           1    120         120
```

- **Compute** Order 101: line 1 is `1 × 1500 = 1500`. Line 2 is `2 × 80 = 160`. Order 102: `1 × 600 = 600`.
- **Answer** The printed `line_total` column matches these hand values.
- **Meaning** The database computes a new column for each row, but stores nothing new. The alias only names the result.

Problem 2.1.3 Show a sentence for each product, like `Keyboard costs Rs 1500`.

```
SELECT name || ' costs Rs ' || price AS label FROM products;
```

```
label
----------------------
Keyboard costs Rs 1500
Mouse costs Rs 600
Notebook costs Rs 80
Pen Pack costs Rs 120
Desk Lamp costs Rs 900
Mug costs Rs 250
```

- **Answer** The `||` operator joins text, and the number is converted automatically.
- **Check** `'Keyboard' || ' costs Rs ' || 1500` is `Keyboard costs Rs 1500`, as printed.

The picture: what SELECT does

```
products                    SELECT name, price*2 AS double_price
+----+----------+-------+               |
| id | name     | price |               v
+----+----------+-------+     +----------+--------------+
| 1  | Keyboard | 1500  |  => | name     | double_price |
| 2  | Mouse    |  600  |     +----------+--------------+
+----+----------+-------+     | Keyboard | 3000         |
                             | Mouse    | 1200         |
```

Connection to the real job

Almost every dashboard tile is a `SELECT` with a calculated column. Good analysts give clear aliases (`revenue_inr` , not `col1`) because the alias becomes the column title that business users read.

Applications

- Reports with calculated metrics (margin, tax, discount)
- Feature engineering columns for ML
- Quick data profiling (`SELECT * ... LIMIT 10`)

By role: **Data Analytics** builds KPI columns (margin, discount); **Data Science and ML** builds feature columns; **Data Engineering** selects only needed columns to save cost.

COMMON MISTAKE	Integer division. In SQLite and PostgreSQL, `7 / 2` is `3` , not `3.5` , when both are integers. Write `7 * 1.0 / 2` or `CAST(7 AS REAL) / 2` .

Predict: what will each of these two columns print?

```
SELECT 7 / 2 AS int_div, 7 * 1.0 / 2 AS real_div;
```

```
int_div  real_div
_______  ________
3        3.5
```

The first is `3` (both numbers are integers, so the answer is an integer and the `.5` is thrown away). The second is `3.5` , because `1.0` makes the whole expression decimal.

Check yourself

1. What does `AS` do?

Answer: It gives a column or table a new name (alias).

1. What is `10 / 4` with integers in SQLite?

Answer: 2 (integer division).

1. Why avoid `SELECT *` in production?

Answer: It reads unneeded columns and breaks when columns change.

2.2 Filtering Rows with WHERE

INTUITION	`WHERE` is a bouncer at the door. Each row is tested. If the test says true, the row enters the result. If the test says false or unknown, the row stays out. (Unknown is the key to NULL, see 2.4.)

Words you need

- **Condition** (or *predicate*): a test that is TRUE, FALSE or UNKNOWN for each row, such as `city = 'Pune'`.
- **String literal**: text written in single quotes, like `'Pune'`. Double quotes are for column names in standard SQL.

Real-world mapping

Condition	Example	Where it appears in the job
`status = 'paid'`	only successful orders	revenue reports
`order_date >= '2024-03-01'`	a time window	monthly dashboards
`city IN (...)`	a region group	regional analysis
`name LIKE 'M%'`	a text pattern	searching logs and names

Operators

Operator	Meaning	Example
`= <> < <= > >=`	compare (`!=` also works in most databases)	`price >= 600`
`AND OR NOT`	combine tests	`city = 'Pune' AND ...`
`IN (a,b,c)`	equals any listed value	`city IN ('Pune','Delhi')`
`BETWEEN a AND b`	inclusive range	`price BETWEEN 100 AND 900`
`LIKE`	text pattern: `%` any text, `_` one character	`name LIKE 'M%'`
`ILIKE`	case-insensitive LIKE (PostgreSQL). SQLite's `LIKE` is already case-insensitive for English letters	
`IS NULL / IS NOT NULL`	test for missing	`coupon IS NULL`

Precedence: `NOT`, then `AND`, then `OR`. Use brackets to be clear.

Worked examples

Problem 2.2.1 Customers in Pune or Delhi who signed up in February or later.

Step 1	Write `city IN ('Pune','Delhi')`.
Step 2	Add `signup_date >= '2024-02-01'`. Dates stored as `YYYY-MM-DD` text compare correctly in order.

```
SELECT name, city, signup_date
FROM customers
WHERE city IN ('Pune','Delhi') AND signup_date >= '2024-02-01';
```

```
name  city   signup_date
----  -----  -----------
John  Delhi  2024-02-02
Sana  Delhi  2024-03-15
Tom   Pune   2024-04-01
```

- **Check** Pune customers are Asha (Jan 5), Meena (Jan 20), Tom (Apr 1). Delhi customers are John (Feb 2) and Sana (Mar 15). Only the February-or-later ones remain.
- **Answer** John (Delhi, Feb 2), Sana (Delhi, Mar 15) and Tom (Pune, Apr 1). Asha and Meena signed up in January.
- **Meaning** Both tests must be TRUE for a row to pass, because of `AND`.

Problem 2.2.2 Products priced from 100 to 900 inclusive whose name contains the letter `p` (any case).

```
SELECT name, price FROM products
WHERE price BETWEEN 100 AND 900 AND name LIKE '%p%';
```

```
name       price
---------  -----
Pen Pack   120
Desk Lamp  900
```

- **Check** Products in range 100 to 900: Mouse (600), Pen Pack (120), Desk Lamp (900), Mug (250). Of these, names with a `p`: Pen Pack and Desk Lamp (`Lamp`). Mouse and Mug have none.
- **Answer** Desk Lamp (900) and Pen Pack (120). Notebook (80) is below 100 and Keyboard (1500) is above 900.
- **Meaning** `LIKE '%p%'` ignores case in SQLite, so the capital `P` in `Pen Pack` also matched. In PostgreSQL you would write `ILIKE`.

Problem 2.2.3 The precedence trap. Find customers who are in Pune, or who are in Delhi and signed up after March 1.

```
SELECT name, city, signup_date FROM customers
WHERE city = 'Pune' OR city = 'Delhi' AND signup_date > '2024-03-01';
```

```
name   city   signup_date
-----  -----  -----------
Asha   Pune   2024-01-05
Meena  Pune   2024-01-20
Sana   Delhi  2024-03-15
Tom    Pune   2024-04-01
```

- **Answer** Because `AND` binds tighter, this means `Pune OR (Delhi AND after Mar 1)`. John (Delhi, February) is dropped, and Sana (Delhi, March 15) stays. All three Pune customers stay.
- **Check** If you add brackets as `(Pune OR Delhi) AND after Mar 1` you get only Sana and Tom. Different brackets, different answer.
- **Meaning** Always write the brackets you mean.

Connection to the real job

Every real question has filters: "paid orders in March", "users from India on Android". Business rules like "exclude test accounts" live in `WHERE`. A missing filter is a silent wrong number.

Applications

- Cohort selection for experiments
- Removing test or bot traffic from logs
- Selecting training rows for a model

COMMON MISTAKE	Mixing `AND` and `OR` without brackets. Always bracket mixed conditions: `WHERE (a OR b) AND c`.

Check yourself

1. Is `BETWEEN 1 AND 5` inclusive?

Answer: Yes, both 1 and 5 match.

1. What does `'a_c'` match with LIKE?

Answer: Three-letter text with any middle character, like `abc`.

1. Which binds first, AND or OR?

Answer: AND.

2.3 ORDER BY, LIMIT/OFFSET and DISTINCT

INTUITION	A table has no order. If you do not say `ORDER BY`, the database may return rows in any order, and that order can change tomorrow. `ORDER BY` sorts, `LIMIT` takes the top few, `DISTINCT` removes repeats. Together they answer "top 3 most expensive products" or "which cities do we serve?".

Words you need

- **Sort key**: the column used to order rows.
- **Pagination**: showing results in pages (page 1, page 2).
- **Duplicate**: two result rows that are identical in every selected column.

Real-world mapping

Tool	Example	Where it appears in the job
`ORDER BY price DESC LIMIT 3`	top 3 products	leaderboards, slowest-requests lists
`LIMIT 20 OFFSET 40`	page 3	website lists, API responses
`SELECT DISTINCT city`	unique cities	profiling a column

Rules

- `ORDER BY col [ASC|DESC]` sorts. Default is `ASC`. You can sort by several columns, by an alias, or by position (`ORDER BY 2`, avoid).
- `LIMIT n` keeps `n` rows. `OFFSET k` skips `k` first. `LIMIT 10 OFFSET 20` is "page 3 of size 10". (SQL Server uses `OFFSET ... FETCH` or `TOP`.) Large offsets are slow because the database still counts the skipped rows.
- `DISTINCT` removes duplicate **whole rows** of the selected columns.
- Where NULLs sort: SQLite puts NULL first in `ASC`. PostgreSQL puts NULL last in `ASC`. Use `NULLS LAST` in PostgreSQL to be explicit.

The logical order SQL reads your query

You write `SELECT` first, but the engine thinks in this order. This explains many errors.

```
1 FROM / JOIN      -> find the tables, combine them
2 WHERE            -> keep rows
3 GROUP BY         -> make groups
4 HAVING           -> keep groups
5 SELECT           -> compute columns, aliases are born here
6 DISTINCT
7 ORDER BY         -> may use aliases
8 LIMIT / OFFSET
```

So `WHERE` cannot use a `SELECT` alias (the alias is not born yet), but `ORDER BY` can.

Worked examples

Problem 2.3.1 The three most expensive products.

Step 1	Sort by price descending.
Step 2	Keep three.

```
SELECT name, price FROM products ORDER BY price DESC LIMIT 3;
```

```
name       price
---------  -----
Keyboard   1500
Desk Lamp  900
Mouse      600
```

- **Check** Sorted prices: 1500, 900, 600, 250, 120, 80. The top three are the first three.
- **Answer** Keyboard 1500, Desk Lamp 900, Mouse 600.
- **Meaning** This is a leaderboard. The same shape gives "top 3 slowest deliveries" in a courier report.

Problem 2.3.2 The distinct cities, and a count of customers and cities.

```
SELECT DISTINCT city FROM customers ORDER BY city;
```

```
city
---------
Bengaluru
Delhi
Mumbai
Pune
```

```
SELECT COUNT(*) AS rows_total, COUNT(DISTINCT city) AS cities FROM customers;
```

```
rows_total  cities
----------  ------
8           4
```

- **Check** By hand: Pune (3 customers), Mumbai (2), Delhi (2), Bengaluru (1). That is 4 cities and 8 customers.
- **Answer** Four cities: Bengaluru, Delhi, Mumbai, Pune. Eight customers share them.
- **Meaning** `COUNT(DISTINCT city)` is the standard way to count unique values.

Problem 2.3.3 Page 2 of customers when each page has 3 rows, ordered by `customer_id`.

Step 1	Page 2 starts after the first 3 rows, so `OFFSET 3`.

```
SELECT customer_id, name FROM customers
ORDER BY customer_id LIMIT 3 OFFSET 3;
```

```
customer_id  name
-----------  -----
4            John
5            Priya
6            Kabir
```

- **Check** The skipped rows are 1, 2, 3 (offset 3). The next three are 4, 5, 6.
- **Answer** Customers 4, 5, 6 (John, Priya, Kabir).

Problem 2.3.4 Using an alias in `ORDER BY` (allowed) and in `WHERE` (not allowed).

```
SELECT name, price * 2 AS double_price FROM products ORDER BY double_price DESC LIMIT 2;
```

```
name       double_price
---------  ------------
Keyboard   3000
Desk Lamp  1800
```

```
SELECT name, price * 2 AS double_price FROM products WHERE double_price > 1000;
```

```
name       double_price
---------  ------------
Keyboard   3000
Mouse      1200
Desk Lamp  1800
```

- **Answer** The first works. The second fails in PostgreSQL (`column "double_price" does not exist`). Look at the output: SQLite is lenient and accepted it. That is a SQLite quirk; do not rely on it. Repeat the expression: `WHERE price * 2 > 1000` .

Connection to the real job

"Top N" is the most common analyst request. In APIs, pagination uses `LIMIT/OFFSET` or, better for big tables, **keyset pagination** (`WHERE id > last_seen_id ORDER BY id LIMIT 10`), which stays fast on deep pages.

Applications

- Leaderboards and top-N reports
- Dashboard pagination
- Listing unique values to profile a column

By role: **Data Analytics** shows top-N tables; **Data Engineering** uses keyset pagination for APIs; **Logistics** lists the slowest deliveries.

COMMON MISTAKE	Using `LIMIT` without `ORDER BY` . The "first 5 rows" are then random-ish and may differ between runs. Always sort when you limit.

Check yourself

1. What does `LIMIT 5 OFFSET 10` return?

Answer: Rows 11 to 15 of the sorted result.

1. Can `ORDER BY` use a SELECT alias?

Answer: Yes. `WHERE` cannot.

1. What does `DISTINCT city, name` remove?

Answer: Rows where both city and name repeat.

2.4 NULL and Three-Valued Logic

INTUITION	NULL is "I do not know". Is an unknown number equal to another unknown number? You cannot say yes. You cannot say no. The honest answer is "unknown". SQL has three truth values: **TRUE, FALSE, UNKNOWN** (written NULL). `WHERE` keeps only TRUE rows.

Words you need

- **NULL**: a marker meaning "no value here" (missing or unknown).
- **Three-valued logic**: a test can be TRUE, FALSE or UNKNOWN.
- **Aggregate**: a function (like `SUM`) that combines many rows into one value. Chapter 3 teaches these fully.

Real-world mapping

Column	NULL means	Where it appears in the job
`customers.referrer_id`	no one referred this customer	marketing attribution
`orders.coupon`	no coupon used	discount analysis
a lost parcel's `delivery_days`	the parcel never arrived	courier records

The rules

- Any comparison with NULL gives UNKNOWN: `NULL = NULL`, `NULL <> 5`, `NULL > 3` are all UNKNOWN.
- Test with `IS NULL` / `IS NOT NULL`, never `= NULL`.
- Logic tables: `TRUE AND UNKNOWN` = UNKNOWN. `FALSE AND UNKNOWN` = FALSE. `TRUE OR UNKNOWN` = TRUE. `NOT UNKNOWN` = UNKNOWN.
- Arithmetic with NULL gives NULL: `5 + NULL` is NULL.
- Aggregates (`SUM`, `AVG`, `COUNT(col)`) ignore NULLs. `COUNT(*)` counts rows.
- `COALESCE(a, b, c)` returns the first non-NULL value. `NULLIF(a, b)` returns NULL if `a = b`, else `a`. Great for avoiding divide-by-zero: `x / NULLIF(y, 0)`.
- `DISTINCT` and `GROUP BY` treat all NULLs as the same group. `UNIQUE` constraints in most databases allow many NULLs.

The truth picture

```
AND   | T  F  U        OR    | T  F  U        NOT
------+---------       ------+---------       ---------
 T    | T  F  U         T    | T  T  T         T -> F
 F    | F  F  F         F    | T  F  U         F -> T
 U    | U  F  U         U    | T  U  U         U -> U
```

Worked examples

Problem 2.4.1 Which customers have no referrer? Then show why `= NULL` fails.

```
SELECT name FROM customers WHERE referrer_id IS NULL;
```

```
name
-----
Asha
Priya
Sana
```

```
SELECT name FROM customers WHERE referrer_id = NULL;
```

```
(no rows)
```

- **Predict** Which one returns rows? Then run both.
- **Answer** `IS NULL` returns Asha, Priya and Sana. `= NULL` returns nothing, because the test is UNKNOWN for every row.
- **Check** The customers table shows NULL in `referrer_id` for customers 1, 5 and 7: Asha, Priya, Sana.

Problem 2.4.2 The silent loss. How many customers have `referrer_id <> 1`? Eight customers exist. Two were referred by customer 1 (Ravi and Meena), so you might expect 6.

Step 1	Run the query.
Step 2	Count again by hand. Three customers have NULL referrer, so their test is UNKNOWN and they vanish.

```
SELECT name, referrer_id FROM customers WHERE referrer_id <> 1;
```

```
name   referrer_id
-----  -----------
John   2
Kabir  5
Tom    3
```

```
SELECT name, referrer_id FROM customers
WHERE referrer_id <> 1 OR referrer_id IS NULL;
```

```
name   referrer_id
-----  -----------
Asha   NULL
John   2
Priya  NULL
Kabir  5
Sana   NULL
Tom    3
```

- **Answer** The first query returns only 3 rows (John, Kabir, Tom). The fixed query returns 6, because NULL rows must be asked for explicitly.
- **Check** `3 rows with referrer 2, 5, 3` plus `3 NULL rows` is 6. And the 2 customers with referrer 1 are the only ones excluded: `8 − 2 = 6`.
- **Meaning** Without the NULL clause the query silently dropped three real customers. No error was shown.

Problem 2.4.3 `COUNT(*)` vs `COUNT(col)`, `COALESCE` and `NULLIF`.

```
SELECT COUNT(*) AS all_orders,
       COUNT(coupon) AS with_coupon,
       COUNT(*) - COUNT(coupon) AS without_coupon
FROM orders;
```

```
all_orders  with_coupon  without_coupon
----------  -----------  --------------
14          3            11
```

```
SELECT order_id, coupon, COALESCE(coupon, 'NO COUPON') AS coupon_label
FROM orders LIMIT 4;
```

```
order_id  coupon  coupon_label
--------  ------  ------------
101       NULL    NO COUPON
102       SAVE10  SAVE10
103       NULL    NO COUPON
104       NULL    NO COUPON
```

```
SELECT 100 / NULLIF(0, 0) AS safe_divide, 100 / NULLIF(4, 0) AS normal_divide;
```

```
safe_divide  normal_divide
-----------  -------------
NULL         25
```

- **Check** Coupons appear on orders 102, 105 and 110: 3, and `14 - 3 = 11`.
- **Answer** 14 orders, 3 with coupons, 11 without. `COALESCE` replaces NULL with a label. `NULLIF(0,0)` is NULL, so `100 / NULL` is NULL instead of an error.
- **Meaning** Use `COALESCE` for display and `NULLIF` for safe division.

Problem 2.4.4 The `NOT IN` trap, a famous interview question. Find customers who never referred anyone.

Step 1	Natural attempt: `customer_id NOT IN (SELECT referrer_id FROM customers)`.
Step 2	The list contains NULL. `5 NOT IN (1, 2, NULL)` becomes `5<>1 AND 5<>2 AND 5<>NULL`, which is UNKNOWN. No rows pass.
Step 3	Fix: filter the NULLs out, or use `NOT EXISTS` (Chapter 4).

```
SELECT name FROM customers
WHERE customer_id NOT IN (SELECT referrer_id FROM customers);
```

```
(no rows)
```

```
SELECT name FROM customers
WHERE customer_id NOT IN (SELECT referrer_id FROM customers WHERE referrer_id IS NOT NULL);
```

```
name
-----
John
Kabir
Sana
Tom
```

- **Answer** The first returns zero rows (wrong). The second returns the customers nobody referred: John, Kabir, Sana, Tom (customers 4, 6, 7, 8).

Connection to the real job

Real data is full of NULLs: missing phone numbers, pending shipments, optional fields. Most "my numbers do not add up" bugs are NULL bugs. Always ask: "can this column be NULL, and what should that mean in my metric?"

Applications

- Data-quality checks (how many NULLs per column)
- Safe ratio metrics (`NULLIF` in denominators)
- Filling missing values before ML (imputation starts with understanding NULL)

By role: **Data Analytics** writes NULL-safe KPIs; **Data Science and ML** decides how to impute missing values; **Data Engineering** adds `NOT NULL` constraints and NULL-count checks to pipelines.

COMMON MISTAKE

Writing `WHERE col <> 'x'` and expecting NULL rows to appear. They do not. Add `OR col IS NULL`, or use `IS DISTINCT FROM` (PostgreSQL; SQLite spells it `IS NOT`).

Check yourself

1. What is `NULL = NULL`?

Answer: UNKNOWN (NULL), not TRUE.

1. What does `COUNT(col)` do with NULLs?

Answer: It skips them. `COUNT(*)` counts all rows.

1. What is `COALESCE(NULL, NULL, 7)`?

Answer: 7.

2.5 NULL-Safe Comparison, Ordering NULLs, Collation and Type Traps

INTUITION

Section 2.4 taught that NULL means 'unknown', so comparing anything with NULL gives unknown, not true or false. Four everyday situations follow from that and surprise beginners: inequality silently drops NULL rows, NULLs sort to an end you did not choose, text sorts differently from what a person expects, and numbers stored as text compare in odd ways. This section shows each one on real rows.

Example 1: not-equal that loses rows, and the NULL-safe fix

Problem 2.5.1 Count orders whose coupon is not 'SAVE10'. Of 14 orders, 3 use SAVE10 and 11 do not. Predict first: a careless `<>` returns far fewer than 11, because the NULL coupons vanish. Every non-NULL coupon here is SAVE10, so it may return 0.

```
SELECT order_id, coupon FROM orders WHERE order_id IN (101, 102) ORDER BY order_id;
SELECT count(*) AS plain_not_equal FROM orders WHERE coupon <> 'SAVE10';
SELECT count(*) AS null_safe_not_equal FROM orders WHERE coupon IS NOT 'SAVE10';
SELECT count(*) AS with_or_null FROM orders WHERE coupon <> 'SAVE10' OR coupon IS NULL;
```

```
order_id  coupon
--------  ------
101       NULL
102       SAVE10
plain_not_equal
---------------
0
null_safe_not_equal
-------------------
11
with_or_null
------------
11
```

Step 1	The 3 orders with a coupon all have SAVE10, so `coupon <> 'SAVE10'` is false for them. For the 11 orders with a NULL coupon it is unknown, which `WHERE` treats as not true. So the first count is 0, not 11.
Step 2	`IS NOT` is SQLite's null-safe not-equal: it treats NULL as an ordinary value. The count is 11. PostgreSQL, SQL Server 2022 and MySQL spell it `IS DISTINCT FROM` (MySQL: `NOT (a <=> b)`); not run here.
Step 3	The portable fix `coupon <> 'SAVE10' OR coupon IS NULL` also gives 11.
Meaning	The first query was wrong in a way that gives no error. Always ask: can this column be NULL?

COMMON MISTAKE	Using `NOT IN` with a subquery that can return NULL: `WHERE id NOT IN (SELECT referrer_id FROM customers)`. If even one `referrer_id` is NULL, the whole `NOT IN` is unknown for every row and the query returns nothing. Use `NOT EXISTS`, or add `WHERE referrer_id IS NOT NULL` inside the subquery.

Example 2: where do NULLs sort?

Problem 2.5.2 Sort customers by referrer. Predict first: customers 1, 5 and 7 have no referrer. SQLite treats NULL as the smallest value, so ascending puts them first; PostgreSQL treats NULL as the largest, so ascending puts them last.

```
SELECT referrer_id, name FROM customers ORDER BY referrer_id, name LIMIT 4;
SELECT referrer_id, name FROM customers ORDER BY referrer_id DESC, name LIMIT 4;
SELECT referrer_id, name FROM customers ORDER BY referrer_id NULLS LAST, name LIMIT 4;
SELECT referrer_id, name FROM customers ORDER BY referrer_id DESC NULLS LAST, name LIMIT 4;
```

```
referrer_id  name
-----------  -----
NULL         Asha
NULL         Priya
NULL         Sana
1            Meena
referrer_id  name
-----------  -----
5            Kabir
3            Tom
2            John
1            Meena
referrer_id  name
-----------  -----
1            Meena
1            Ravi
2            John
3            Tom
referrer_id  name
-----------  -----
5            Kabir
3            Tom
2            John
1            Meena
```

Step 1	Ascending in SQLite: the NULL referrers (Asha, Priya, Sana) come first. In PostgreSQL and Oracle the default is the opposite, NULLs last on ascending and first on descending.
Step 2	Descending reverses it: the largest referrer 5 (Kabir) comes first, and NULL rows are last in SQLite.
Step 3	`NULLS FIRST` and `NULLS LAST` (SQLite 3.30+, PostgreSQL, Oracle) make the choice explicit, so the query returns the same order on any database. SQL Server and MySQL do not have the clause; use `ORDER BY (col IS NULL), col` there.
Answer	Always state NULL placement when the order matters: a report's 'top 10' will otherwise differ between databases.

Example 3: case and collation

A **collation** is the rule that says how text compares and sorts. SQLite's default is binary: capital letters sort before lowercase, and `'Apple' = 'apple'` is false. PostgreSQL uses the operating-system locale (usually case-aware, ignoring case in the first pass). MySQL's default collations are case-insensitive. Moving a query between systems can therefore change both results and row counts.

Problem 2.5.3 Sort and match five words with and without case-insensitivity. Predict first: binary order puts Apple and Banana before apple, banana and cherry.

```
CREATE TABLE tag (word TEXT);
INSERT INTO tag VALUES ('Apple'), ('apple'), ('Banana'), ('banana'), ('cherry');
SELECT word FROM tag ORDER BY word;
SELECT word FROM tag ORDER BY word COLLATE NOCASE, word;
SELECT count(*) AS exact_match FROM tag WHERE word = 'apple';
SELECT count(*) AS nocase_match FROM tag WHERE word = 'apple' COLLATE NOCASE;
```

```
word
------
Apple
Banana
apple
banana
cherry
word
------
Apple
apple
Banana
banana
cherry
exact_match
-----------
1
nocase_match
------------
2
```

Step 1	Binary order: all capitals first (Apple, Banana), then lowercase (apple, banana, cherry), because 'A' is code 65 and 'a' is 97.
Step 2	`COLLATE NOCASE` sorts Apple next to apple (the extra `word` sort key then breaks ties in a fixed order).
Step 3	Exact match for 'apple' finds 1 row; with NOCASE it finds 2. In PostgreSQL write `lower(word) = 'apple'` or use `ILIKE`, or a `citext` column (not run here).

Example 4: type traps in SQLite

SQLite lets a column hold any type unless the table is `STRICT` (Chapter 9). Its comparison rules convert values in ways PostgreSQL would reject, which hides bugs.

Problem 2.5.4 A phone column holds the text '0551', '551' and '5.5'. Look for phone number 551 as a number and as text. Predict first: the numeric literal 551 matches the text '551' only, because the column has TEXT affinity, so 551 is converted to the text '551'.

```
CREATE TABLE ph (id INTEGER, phone TEXT, age INTEGER);
INSERT INTO ph VALUES (1, '0551', 30), (2, '551', 31), (3, '5.5', 32);
SELECT id, typeof(phone) AS t_phone, typeof(age) AS t_age FROM ph;
SELECT id FROM ph WHERE phone = 551;
SELECT id FROM ph WHERE phone = '551';
SELECT 10 / 4 AS int_div, 10 / 4.0 AS real_div;
```

```
id  t_phone  t_age
--  -------  -------
1   text     integer
2   text     integer
3   text     integer
id
--
2
id
--
2
int_div  real_div
-------  --------
2        2.5
```

Step 1	`typeof` shows what is really stored: text for phone, integer for age.
Step 2	`phone = 551` converts the number to text and matches only id 2 ('551'). '0551' is not equal to '551' as text, and a phone number with a leading zero is exactly why phone numbers must be text.
Step 3	The last row shows that `10 / 4` is 2 (integer division) and `10 / 4.0` is 2.5. PostgreSQL does the same for integers; MySQL's `/` always returns a decimal (use `DIV` for integers). That difference silently changes percentages written as `count / total`.
Answer	Know the stored type, and multiply by `1.0` (or cast) before dividing.

COMMON MISTAKE

Computing a percentage as `sum(flag) / count(*) * 100`. With integers the division gives 0 whenever the ratio is below 1, so every percentage is 0 or 100. Write `100.0 * sum(flag) / count(*)`, with the decimal first, as the rest of this book does.

Practice with answers

Problem 2.5.5 `SELECT count(*) FROM customers WHERE referrer_id <> 1`. Customers have referrer_ids NULL, 1, 1, 2, NULL, 5, NULL, 3. What is returned? *(what is the output?)*

Answer: The count is 3: the ids 2, 5 and 3 are not equal to 1. The three NULLs are unknown and the two 1s are false, so the answer is 3, not 6.

Problem 2.5.6 Make the previous query also count customers without a referrer. *(easy)*

Answer: `WHERE referrer_id <> 1 OR referrer_id IS NULL`, or `IS NOT 1` in SQLite, or `IS DISTINCT FROM 1` in PostgreSQL. The answer is 6.

Problem 2.5.7 A teammate ports a query from MySQL to PostgreSQL and the number of rows matching `WHERE city = 'pune'` falls from 3 to 0. Why? *(interview)*

Answer: MySQL's default collation compares case-insensitively, so 'Pune' matched 'pune'. PostgreSQL compares exactly, so it does not. Fix with `lower(city) = 'pune'` (and index `lower(city)`), or `ILIKE`, or store a normalised column.

Chapter recap

- `SELECT` chooses columns and can compute new ones; use aliases.
- Integer division truncates; make one side decimal.
- `WHERE` keeps only rows where the test is TRUE.
- `AND` binds tighter than `OR`; use brackets.
- A table has no order unless you use `ORDER BY`; always sort before `LIMIT`.
- The engine runs FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT, in that order.
- NULL means unknown; comparisons with NULL are UNKNOWN; use `IS NULL`.
- `COALESCE` fills NULLs; `NULLIF` prevents divide-by-zero.
- `NOT IN` with a NULL in the list returns nothing: a classic trap.

Pattern	Syntax
Top N	`ORDER BY col DESC LIMIT n`
Page k of size n	`LIMIT n OFFSET (k-1)*n`
Missing value test	`col IS NULL` / `IS NOT NULL`
Fill NULL	`COALESCE(col, default)`
Safe divide	`x / NULLIF(y, 0)`
Unique values	`SELECT DISTINCT col`

Practice set

Problem 2.P.1 (easy) List products in the Home category.

```
SELECT * FROM products WHERE category = 'Home';
```

```
product_id  name       category  price
----------  ---------  --------  -----
5           Desk Lamp  Home      900
6           Mug        Home      250
```

Answer: Desk Lamp and Mug.

Problem 2.P.2 (easy) List customers sorted by signup date, newest first, top 2.

```
SELECT name, signup_date FROM customers ORDER BY signup_date DESC LIMIT 2;
```

```
name  signup_date
----  -----------
Tom   2024-04-01
Sana  2024-03-15
```

Answer: Tom (2024-04-01) and Sana (2024-03-15).

Problem 2.P.3 (what is the output?) What does this return, and why?

```
SELECT COUNT(*) AS n FROM customers WHERE referrer_id = referrer_id;
```

```
n
-
5
```

Answer: 5, not 8. For NULL rows `NULL = NULL` is UNKNOWN, so they are filtered out.

Problem 2.P.4 (find the bug) The query `SELECT name FROM orders WHERE coupon = NULL` is meant to find orders without coupon, but `orders` has no `name` column and the test is wrong. Fix it.

```
SELECT order_id FROM orders WHERE coupon IS NULL LIMIT 3;
```

```
order_id
--------
101
103
104
```

Answer: Use `IS NULL`. (And pick a column that exists.) Here the first three are 101, 103, 104.

Problem 2.P.5 Show orders from March 2024 using `BETWEEN`.

```
SELECT order_id, order_date FROM orders
WHERE order_date BETWEEN '2024-03-01' AND '2024-03-31';
```

```
order_id  order_date
--------  ----------
106       2024-03-05
107       2024-03-07
108       2024-03-18
109       2024-03-25
110       2024-03-30
```

Answer: Orders 106, 107, 108, 109, 110. Note the end value `'2024-03-31'` works only because dates have no time part. With timestamps use `>= '2024-03-01' AND < '2024-04-01'`.

Problem 2.P.6 Products with price between 100 and 1000 in Stationery or Home, sorted by price descending.

```
SELECT name, category, price FROM products
WHERE price BETWEEN 100 AND 1000 AND category IN ('Stationery','Home')
ORDER BY price DESC;
```

```
name       category    price
---------  ----------  -----
Desk Lamp  Home        900
Mug        Home        250
Pen Pack   Stationery  120
```

Answer: Desk Lamp 900, Mug 250, Pen Pack 120.

Problem 2.P.7 Show every order with a coupon label: the coupon, or `none` if missing, only for paid orders, first 5.

```
SELECT order_id, COALESCE(coupon,'none') AS coupon
FROM orders WHERE status = 'paid' ORDER BY order_id LIMIT 5;
```

```
order_id  coupon
--------  ------
101       none
102       SAVE10
103       none
105       SAVE10
106       none
```

Answer: Orders 101, 102, 103, 105, 106 (104 is cancelled and skipped).

Problem 2.P.8 (scenario) A manager asks, "Who are the customers without a referrer, but only those from Pune or Delhi?" Write the query.

```
SELECT name, city FROM customers
WHERE referrer_id IS NULL AND city IN ('Pune','Delhi');
```

```
name  city
----  -----
Asha  Pune
Sana  Delhi
```

Answer: Asha (Pune) and Sana (Delhi). Priya is in Bengaluru.

Problem 2.P.9 (interview) Explain why `SELECT * FROM t WHERE x NOT IN (SELECT y FROM u)` can return zero rows when `u.y` contains a NULL, and give two fixes. **Answer:** `x NOT IN (...)` expands into many `<>` tests joined by `AND`. A comparison with NULL is UNKNOWN, so the whole `AND` is never TRUE. Fixes: add `WHERE y IS NOT NULL` inside the subquery, or use `NOT EXISTS`.

Problem 2.P.10 (interview) What is the difference between `COUNT(*)`, `COUNT(col)` and `COUNT(DISTINCT col)`? **Answer:** `COUNT(*)` counts rows. `COUNT(col)` counts rows where col is not NULL. `COUNT(DISTINCT col)` counts distinct non-NULL values. Example on our data: `COUNT(*)` of customers is 8, `COUNT(referrer_id)` is 5.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my second language. Teach me Chapter 2 topic: "NULL and three-valued logic in SQL". Explain step by step with tiny tables, show why NULL = NULL is not true, then explain the NOT IN trap. After each step ask me one check question and wait. Finish by testing whether I can predict query results with NULLs.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning SQL. Topic: SELECT, WHERE, ORDER BY, LIMIT, OFFSET and DISTINCT in PostgreSQL. Give me 8 graded exercises on a small orders and customers schema that you define, from easy to hard, without solutions. When I paste my answers, review each one, explain mistakes in simple English and show a better version.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a beginner in SQL, a working professional moving into data roles. Quiz me one question at a time on the logical order of SQL execution (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT) and why aliases work in ORDER BY but not WHERE. Start easy, then give "what is the output?" and "find the bug" questions. After 10 questions give me a score and what to revise.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data-analyst interviews. Review the SQL query I paste below for NULL handling mistakes, missing brackets around AND/OR, missing ORDER BY with LIMIT, and misuse of SELECT star. Explain each issue in simple English and give a corrected query. My query: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am preparing for SQL interviews as a working professional moving into data roles. Run a mock interview on basic SQL: filtering, sorting, pagination, DISTINCT, and NULL traps (NOT IN, = NULL, COUNT star vs COUNT column). Ask one question at a time, grade my answer out of 10, show a model answer, and give a one-line tip before the next question.
```

CHAPTER 3

AGGREGATION, GROUP BY, HAVING AND FILTER

Turning many rows into a few numbers: counts, sums, averages, and the rules of grouping.

Nobody reads a million rows. Managers ask short questions: "What was revenue last month?", "Which product sells best?", "How many customers ordered more than twice?" Every answer is a *summary*, and SQL makes summaries with **aggregate functions** and **GROUP BY**. By the end of this chapter you can turn the raw order lines of our shop into revenue per product, orders per month, refund rates per customer, and you will know how to check that your summary adds up to the raw total. We reuse the same shop tables all the way, so you only learn the new idea each time.

From now on every example runs on the shop database from section 2.1 (it is loaded for you in each chapter). Quick reminder of the tables: `customers`, `products`, `orders` (with `status` and `coupon`), and `order_items` (with `qty` and `unit_price`).

HOW TO USE THIS CHAPTER

Sections 3.1 to 3.4 teach the four ideas once. Sections 3.5 to 3.9 deepen each idea with a glossary, more worked examples, figures and traps. Section 3.10 is a problem-solving playbook you can apply to any new question. Section 3.11 is the expert layer. Sections 3.12 and 3.13 are a drill and a self-test with a scoring guide. If aggregation feels fuzzy, read Section 3.5 first.

3.1 Aggregate Functions

INTUITION

A school result summary takes thousands of exam marks and gives you one row: count, average, lowest, highest. SQL aggregate functions do the same. They squeeze many rows into one value.

The main functions

Function	Returns	NULL handling
`COUNT(*)`	number of rows	counts every row
`COUNT(col)`	number of non-NULL values	skips NULL
`COUNT(DISTINCT col)`	number of different non-NULL values	skips NULL
`SUM(col)`	total	skips NULL; all NULL gives NULL
`AVG(col)`	mean = sum / count of non-NULL	skips NULL
`MIN(col)`, `MAX(col)`	smallest, largest (work on text and dates too)	skip NULL

A query with aggregates and no `GROUP BY` returns exactly **one row**, even on an empty table.

Worked examples

Problem 3.1.1 Summarise all order lines: number of lines, total units, and the revenue (sum of `qty × unit_price`).

Step 1	Count rows with `COUNT(*)`.
Step 2	Sum `qty`.
Step 3	Sum the expression `qty * unit_price`.
Predict	Before you run it: there are 20 order lines. What do you expect for `n_lines`? (20.) For revenue, add the 14 order totals worked out by hand: 1660, 600, 1020, 400, 1500, 1280, 360, 900, 1500, 440, 600, 1800, 400, 2100.

```
SELECT COUNT(*)             AS n_lines,
       SUM(qty)             AS total_units,
       SUM(qty * unit_price) AS revenue
FROM order_items;
```

```
n_lines  total_units  revenue
_______  ___________  _______
20       34           14560
```

- **Compute** Running total of the 14 hand-made order totals: 1660, 2260, 3280, 3680, 5180, 6460, 6820, 7720, 9220, 9660, 10260, 12060, 12460, 14560.
- **Answer** 20 lines, 34 units and revenue 14560.
- **Check** The hand total 14560 equals the printed `revenue`. Two independent routes agree.
- **Meaning** All orders together, including cancelled and refunded ones, are worth 14560 rupees. (Section 3.3 removes the unpaid ones.)

Problem 3.1.2 The AVG trap. (Predict first: will `AVG` divide by 3 or by 2?) Three payment amounts are 200, 300 and NULL (a payment whose amount was never recorded). What are `AVG`, the average if NULL counted as 0, and `COUNT`?

Step 1	`AVG` skips NULL: `(200 + 300) / 2 = 250`.
Step 2	If NULL were 0: `(200 + 300 + 0) / 3 = 166.67`.

```
CREATE TABLE payments (id INTEGER, amount INTEGER);
INSERT INTO payments VALUES (1,200),(2,300),(3,NULL);
SELECT AVG(amount) AS avg_skip_null,
       AVG(COALESCE(amount,0)) AS avg_null_as_zero,
       COUNT(*) AS n_rows, COUNT(amount) AS n_values
FROM payments;
```

```
avg_skip_null  avg_null_as_zero  n_rows  n_values
_____________  _________________  ______  ________
250.0          166.666666666667  3       2
```

- **Check** `(200+300)/2 = 250` and `500/3 = 166.67`; both match the output.
- **Answer** 250 versus 166.67.
- **Meaning** If a missing amount means "no data", the honest average is 250. If a missing amount should count as zero (for example, a free item), use 166.67. Choosing between them is a business decision, not a SQL decision.

Problem 3.1.3 Min and max also work on dates and text. What are the first and last signup dates and the alphabetically first and last customer names?

```
SELECT MIN(signup_date) AS first_signup, MAX(signup_date) AS last_signup,
       MIN(name) AS first_name, MAX(name) AS last_name
FROM customers;
```

```
first_signup  last_signup  first_name  last_name
____________  ___________  __________  _________
2024-01-05    2024-04-01   Asha        Tom
```

- **Answer** First signup 2024-01-05, last 2024-04-01; names from Asha to Tom.

Real-world mapping

Aggregate	Example value	Where it appears in the job
`COUNT(*)`	20 order lines	shop traffic: number of orders, number of customers
`SUM(qty * unit_price)`	14560	revenue, total bytes, total cost
`AVG(amount)`	250.0	average payment amount, average exam mark
`MIN` / `MAX`	first and last signup	oldest record, slowest delivery

The picture

```
rows                  aggregate              one row
+-----+               SUM(x), COUNT(*)
| 200 |  \
| 300 |   >--------------------------------->  | 500 | 2 |
+-----+  /
```

Connection to the real job

Every KPI (revenue, active users, error rate) is an aggregate. Percentiles such as p90 and p95, the numbers used in exam and delivery reports, come from window or special aggregate functions: see Chapters 8 and 12.

Applications

- KPIs on dashboards
- Data profiling (count, min, max, distinct count of each column)
- Exam-result summaries over mark tables

COMMON MISTAKE

`AVG` of integers is a decimal in SQLite, but in some databases (SQL Server) `AVG` of an integer column is an integer. Cast to decimal when it matters. Also: averaging averages is wrong. If group A has 1 row and group B has 100, the average of their two averages is not the overall average.

Check yourself

1. What does `SUM` return if every value is NULL?

Answer: NULL.

1. How many rows does `SELECT COUNT(*) FROM t` return on an empty table?

Answer: One row, with the value 0.

1. Does `AVG` count NULL rows in the denominator?

Answer: No.

3.2 GROUP BY

INTUITION

`GROUP BY` is "for each". For each product, total revenue. For each month, number of orders. Sort the rows into piles by a key, then aggregate each pile. Think of a school report: marks grouped by subject.

Rules

- Every column in `SELECT` must be either in `GROUP BY` or inside an aggregate function. PostgreSQL enforces this. (MySQL used to allow cheating; SQLite allows it too and picks an arbitrary row. Do not rely on it.)
- Group by several columns to get finer piles.
- `NULL`s form one group.
- You can group by an expression such as `substr(order_date,1,7)` (the year-month text).

Worked examples

Problem 3.2.1 Number of orders per status.

Step 1	Pile rows by `status`.
Step 2	Count each pile.

```
SELECT status, COUNT(*) AS n_orders
FROM orders
GROUP BY status
ORDER BY n_orders DESC;
```

```
status     n_orders
---------  --------
paid       12
refunded   1
cancelled  1
```

- **Answer** 12 paid, 1 cancelled, 1 refunded. Total 14, matches the table.

Problem 3.2.2 Revenue and units per product, best first. (We use a `JOIN` here only to get the product name; Chapter 4 explains it.)

```
SELECT p.name, SUM(oi.qty) AS units, SUM(oi.qty * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY revenue DESC;
```

```
name       units  revenue
---------  -----  -------
Keyboard   4      6000
Desk Lamp  4      3600
Mouse      5      3000
Notebook   14     1120
Pen Pack   7      840
```

- **Compute** Keyboard: sold in orders 101, 105, 109, 114, one unit each, so `4 × 1500 = 6000`. Desk Lamp: `1 + 1 + 2 = 4` units, `4 × 900 = 3600`. Mouse: `1 + 2 + 1 + 1 = 5` units, `5 × 600 = 3000`. Notebook: `2 + 5 + 1 + 4 + 2 = 14` units, `14 × 80 = 1120`. Pen Pack: `1 + 3 + 1 + 2 = 7` units, `7 × 120 = 840`.
- **Answer** Keyboard 6000, Desk Lamp 3600, Mouse 3000, Notebook 1120, Pen Pack 840.
- **Check** `6000 + 3600 + 3000 + 1120 + 840 = 14560`, the grand total from 3.1.1.
- **Meaning** The Keyboard earns the most money. The Notebook sells the most units (14) but earns little, because it is cheap. The Mug does not appear: it has no rows in `order_items`, and an inner join keeps only matches. (This also includes cancelled and refunded orders; Section 3.3 filters those.)

Problem 3.2.3 Orders per month (group by an expression) and per month and status (two columns).

```
SELECT substr(order_date,1,7) AS month, COUNT(*) AS n_orders
FROM orders
GROUP BY substr(order_date,1,7)
ORDER BY month;
```

```
month    n_orders
-------  --------
2024-01  2
2024-02  3
2024-03  5
2024-04  4
```

```
SELECT substr(order_date,1,7) AS month, status, COUNT(*) AS n
FROM orders
GROUP BY month, status
ORDER BY month, status;
```

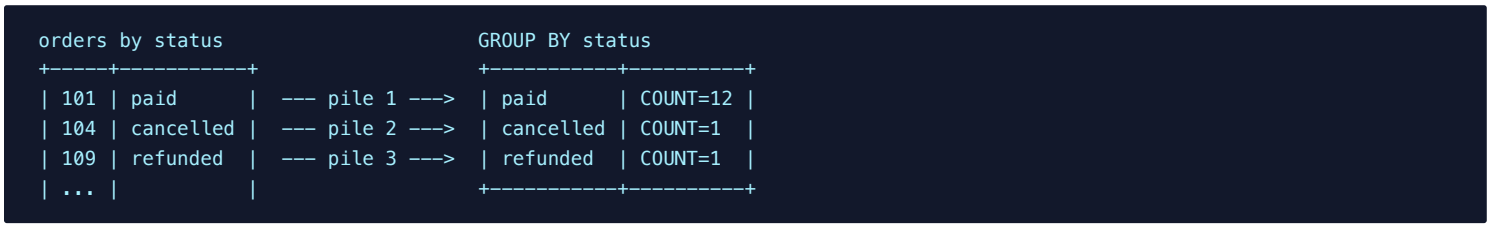
```
month    status     n
-------  ---------  -
2024-01  paid       2
2024-02  cancelled  1
2024-02  paid       2
2024-03  paid       4
2024-03  refunded   1
2024-04  paid       4
```

- **Check** `2 + 3 + 5 + 4 = 14` orders, the full table size.
- **Answer** Jan 2, Feb 3, Mar 5, Apr 4. The second table splits February into paid 2 and cancelled 1, and March into paid 4 and refunded 1. (Most databases, including PostgreSQL, allow `GROUP BY month` using the alias; SQL Server does not.)

Real-world mapping

Grouping key	Example value	Where it appears in the job
`status`	paid, refunded	order funnel, payment reports
`substr(order_date,1,7)`	2024-03	monthly revenue and trend lines
`customer_id`	1	per-user features for ML
`subject`	maths, science	per-subject exam summary report

The picture



Connection to the real job

Group-by queries are 70 percent of analyst work. The most important question is **the grain**: after the join and the group by, what does one result row mean? "One row per product per month" is a statement you should be able to write before the query.

Applications

- Sales by region, month, channel
- Error counts per API endpoint
- Feature aggregation per user for ML ("number of orders in the last 30 days")

COMMON MISTAKE	Selecting a column that is not grouped. In PostgreSQL this raises an error: `column must appear in the GROUP BY clause or be used in an aggregate function`. In SQLite it silently returns a random row. Fix the query; do not rely on the quirk.

Check yourself

1. What does a NULL key do in GROUP BY?

Answer: All NULLs go to one group.

1. Can you group by two columns?

Answer: Yes; you get one group per distinct combination.

1. Why join order_items to products by `product_id` and not by name?

Answer: The key is unique and stable; names can repeat or change.

3.3 HAVING: Filtering Groups

INTUITION	`WHERE` filters rows before piling. `HAVING` filters piles after counting. "Customers with more than 2 orders" cannot be decided row by row; you need the count first.

Rule

`WHERE` runs before `GROUP BY` and cannot use aggregates. `HAVING` runs after and can. Prefer `WHERE` for row conditions (it is faster, since fewer rows enter grouping).

Worked examples

Problem 3.3.1 Customers with at least 3 orders.

Step 1	Group `orders` by `customer_id`.
Step 2	Keep groups where `COUNT(*) >= 3`.

```
SELECT customer_id, COUNT(*) AS n_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 3
ORDER BY n_orders DESC;
```

```
customer_id  n_orders
-----------  --------
1            4
5            3
2            3
```

- **Answer** Customer 1 (4 orders: 101, 103, 106, 111), customers 2 and 5 (3 orders each).

Problem 3.3.2 `WHERE` and `HAVING` together: among **paid** orders only, which customers spent more than 1500? Use `order_items` joined to `orders`.

Step 1	`WHERE o.status = 'paid'` removes cancelled and refunded rows first.
Step 2	Group by customer and sum.
Step 3	`HAVING` keeps sums above 1500.

```
SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
HAVING SUM(oi.qty * oi.unit_price) > 1500
ORDER BY spent DESC;
```

```
customer_id  spent
-----------  -----
1            4560
2            3600
4            3300
```

- **Compute** Customer 1, paid orders 101, 103, 106, 111: `1660 + 1020 + 1280 + 600 = 4560`. Customer 2, paid orders 102, 108, 114: `600 + 900 + 2100 = 3600`. Customer 4, orders 105 and 112: `1500 + 1800 = 3300`. Customer 5, paid orders 107 and 113: `360 + 400 = 760`, below 1500, so dropped. (Order 109 is refunded, so `WHERE` removed it first.)
- **Answer** Customers 1, 2 and 4, with 4560, 3600 and 3300.
- **Check** These match the printed table. Customer 3 had only a cancelled order and customer 6 spent 440, so neither appears.
- **Meaning** Three "big spenders" account for 11460 of the 12660 paid revenue, about 90 percent. That is a typical concentration business teams care about.

Problem 3.3.3 Where to put the condition: show that filtering on a non-aggregate in `HAVING` and in `WHERE` gives the same result, but `WHERE` is the better place.

```
SELECT status, COUNT(*) AS n FROM orders GROUP BY status HAVING status = 'paid';
```

```
status  n
------  --
paid    12
```

```
SELECT status, COUNT(*) AS n FROM orders WHERE status = 'paid' GROUP BY status;
```

```
status  n
------  --
paid    12
```

- **Answer** Both give 12. The second builds only one pile, so it does less work on big tables.
- **Meaning** Use `WHERE` for any condition that does not need an aggregate.

Connection to the real job

"Customers with more than N orders", "products that sold fewer than 5 units", "endpoints with error rate above 2 percent": all are `HAVING`. It is like filtering a school report to the subjects whose fail rate exceeds a threshold.

Applications

- Finding frequent buyers or heavy users
- Detecting duplicate keys (`HAVING COUNT(*) > 1`), see Chapter 12
- Alerting on groups that cross a threshold

COMMON MISTAKE	Using an alias of an aggregate in `HAVING` in databases that do not allow it (SQL Server, older Oracle). Repeat the expression for portability.

Check yourself

1. Which runs first, WHERE or HAVING?

Answer: WHERE.

1. Can WHERE contain `COUNT(*) > 1` ?

Answer: No; use HAVING.

1. Which query finds duplicate emails?

Answer: `SELECT email, COUNT(*) FROM t GROUP BY email HAVING COUNT(*) > 1` .

3.4 FILTER and Conditional Aggregation

INTUITION	What if you want, in one row per customer, the number of paid orders **and** the number of refunded orders side by side? Do not run two queries. Count only the rows that meet a condition inside each aggregate. This is also the first step of a **pivot** (Chapter 7).

Two ways

- **Standard SQL** `FILTER` (PostgreSQL, SQLite 3.30+): `COUNT(*) FILTER (WHERE status = 'paid')` .
- **Portable** `CASE` **trick** (works everywhere): `SUM(CASE WHEN status = 'paid' THEN 1 ELSE 0 END)` .

Worked examples

Problem 3.4.1 For each month, the number of all orders, paid orders and non-paid orders in one table.

```
SELECT substr(order_date,1,7) AS month,
       COUNT(*) AS all_orders,
       COUNT(*) FILTER (WHERE status = 'paid') AS paid,
       COUNT(*) FILTER (WHERE status <> 'paid') AS not_paid
FROM orders
GROUP BY month
ORDER BY month;
```

```
month    all_orders  paid  not_paid
-------  ----------  ----  --------
2024-01  2           2     0
2024-02  3           2     1
2024-03  5           4     1
2024-04  4           4     0
```

- **Check** In every row `paid + not_paid = all_orders` (2, 3, 5, 4).
- **Answer** Jan 2/2/0, Feb 3/2/1, Mar 5/4/1, Apr 4/4/0.
- **Meaning** One query gave three columns side by side. The FILTER clause is like a small WHERE that lives inside one aggregate.

Problem 3.4.2 The same with `CASE`, plus a paid-share percentage. Revenue per month using conditional sums of line totals.

```
SELECT substr(o.order_date,1,7) AS month,
       SUM(CASE WHEN o.status = 'paid' THEN 1 ELSE 0 END) AS paid_lines,
       ROUND(100.0 * SUM(CASE WHEN o.status = 'paid' THEN 1 ELSE 0 END) / COUNT(*), 1) AS paid_pct_of_lines,
       SUM(oi.qty * oi.unit_price) FILTER (WHERE o.status = 'paid') AS paid_revenue
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY month
ORDER BY month;
```

```
month    paid_lines  paid_pct_of_lines  paid_revenue
-------  ----------  -----------------  ------------
2024-01  3           100.0              2260
2024-02  3           75.0               2520
2024-03  6           85.7               2980
2024-04  6           100.0              4900
```

- **Compute** Paid revenue per month from the order totals: January `1660 + 600 = 2260`. February `1020 + 1500 = 2520` (order 104 is cancelled). March `1280 + 360 + 900 + 440 = 2980` (order 109 is refunded). April `600 + 1800 + 400 + 2100 = 4900`.
- **Check** `2260 + 2520 + 2980 + 4900 = 12660`, the paid total we found earlier.
- **Answer** Paid revenue grows from 2260 in January to 4900 in April. The column `paid_pct_of_lines` is the share of order *lines* (not orders) that were paid: 100, 75, 85.7 and 100 percent.
- **Meaning** April is more than twice January. The share column shows that February and March lost some lines to cancellation and refund.

Problem 3.4.3 Conversion-style rate: the refund rate per customer who has at least one order (refunded orders divided by all orders).

```
SELECT customer_id,
       COUNT(*) AS orders,
       COUNT(*) FILTER (WHERE status = 'refunded') AS refunded,
       ROUND(1.0 * COUNT(*) FILTER (WHERE status = 'refunded') / COUNT(*), 2) AS refund_rate
FROM orders GROUP BY customer_id ORDER BY refund_rate DESC, customer_id;
```

```
customer_id  orders  refunded  refund_rate
-----------  ------  --------  -----------
5            3       1         0.33
1            4       0         0.0
2            3       0         0.0
3            1       0         0.0
4            2       0         0.0
6            1       0         0.0
```

- **Check** `1 / 3 = 0.333`, rounded to 0.33. Customer 7 and 8 are missing because they have no orders; a `GROUP BY` on `orders` cannot invent rows (Chapter 4 shows how to include them).
- **Answer** Customer 5 has 3 orders with 1 refunded: `1/3 = 0.33`. Everyone else is 0.
- **Meaning** One in three of Priya's orders was refunded, a signal worth a look.

The picture

```
 customer 5's orders:  107 paid | 109 refunded | 113 paid
 COUNT(*)                       = 3
 COUNT(*) FILTER (refunded)     = 1     -> rate = 1/3
```

Connection to the real job

Conditional aggregation is how analysts build "one row per user" feature tables: `orders_last_30d`, `refunds`, `avg_basket`. It is also the base of funnel and retention queries (Chapter 12).

Applications

- Pivot-style reports without pivot syntax
- Rates and ratios (error rate, conversion, refund rate)
- ML features per entity

COMMON MISTAKE

Computing a rate with integers: `refunded / orders` gives 0. Multiply by `1.0` first. And when the denominator can be 0 use `NULLIF(orders, 0)`.

Check yourself

1. What does `FILTER (WHERE ...)` do?

Answer: It restricts which rows an aggregate sees.

1. Write the portable form of `COUNT(*) FILTER (WHERE x>5)`.

Answer: `SUM(CASE WHEN x>5 THEN 1 ELSE 0 END)`.

1. Why `1.0 *` in the rate?

Answer: It forces decimal division.

3.5 Glossary and the Mental Model (read this first if aggregation feels fuzzy)

INTUITION

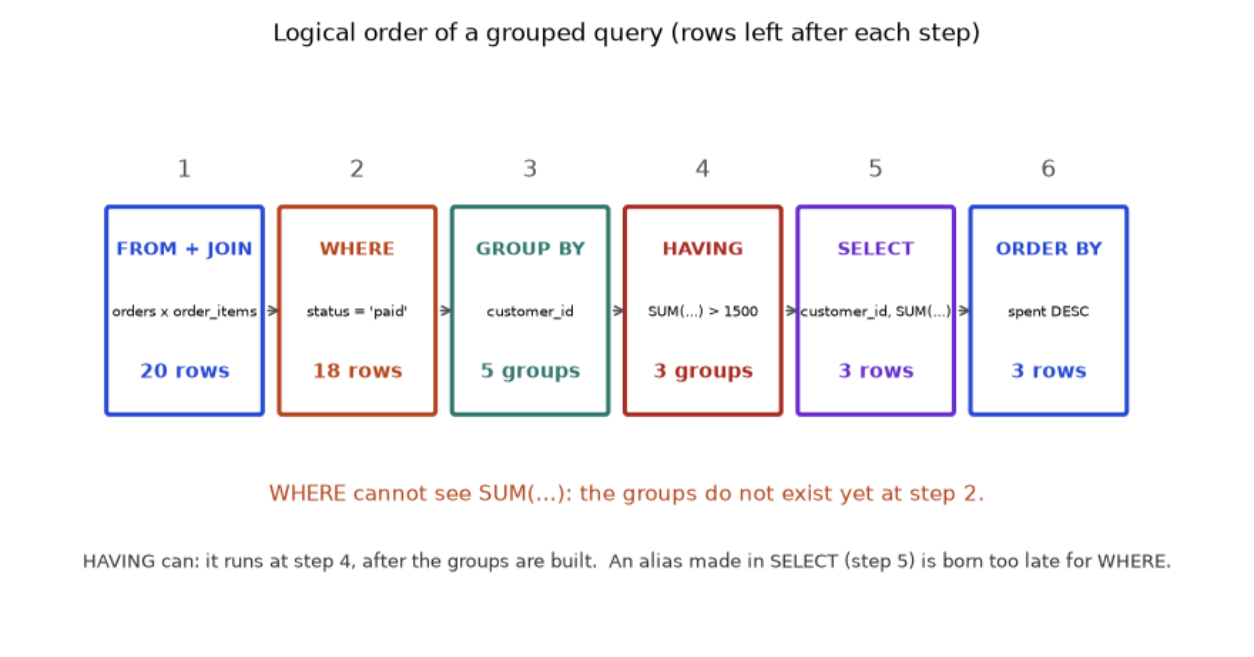
Picture a school attendance register with 40 names. The principal does not want 40 names. She wants one number: how many are present today. Taking many rows and answering with one number is *aggregation*. If she asks 'how many present *in each class*', you first sort the register into class piles, then count each pile. That second step is `GROUP BY`. If she then says 'only show classes with fewer than 30 present', you throw away whole piles after counting. That is `HAVING`. Every query in this chapter is some mix of sort into piles, collapse each pile to a number, and keep or drop piles.

The words below are used all through the chapter. Skim them now and come back whenever a term feels slippery.

Word	Plain meaning	Example in our shop
row	one record in a table; one line of the register	order line (order 101, product 3, qty 2)
aggregate function	a function that reads many rows and returns *one* value	`SUM(qty)`, `COUNT(*)`, `MAX(price)`
group (pile)	all rows that share the same value of the grouping key	all rows with `status = 'paid'`
grouping key	the column or expression you sort rows into piles by	`customer_id`, `substr(order_date,1,7)`
grain	what one row of the *result* stands for, written as 'one row per ___'	one row per customer per month
NULL	'value unknown or missing', not zero and not an empty string	an order with no coupon
predicate	a condition that is true, false or unknown	`status = 'paid'`
conditional aggregate	an aggregate that only reads the rows meeting a condition	`COUNT(*) FILTER (WHERE status='refunded')`
fan-out	a join that repeats one row several times, inflating counts and sums	an order repeated once per item line
scalar	a single value (one row, one column)	the number 14560
reconcile	check that parts add up to the whole	per-product revenue adds to 14560

The mental model in five sentences

1. The database first decides *which rows* exist: `FROM` and `JOIN` build the rows, `WHERE` throws away the ones you do not want.
2. `GROUP BY` sorts the surviving rows into piles by the key. With no `GROUP BY` the whole table is one pile.
3. Each aggregate function turns each pile into one value.
4. `HAVING` throws away whole piles, using those values.
5. Only then does `SELECT` produce the output columns, followed by `ORDER BY` and `LIMIT`.



The logical order in which the database processes a grouped query, with the real row counts for 'paid spend per customer, only above 1500'.

How to read it: read left to right. The first box builds 20 order lines. `WHERE` drops the 2 lines that belong to non-paid orders (order 104 has one line, order 109 has one line), leaving 18. `GROUP BY` sorts those 18 lines into 5 piles, one per paying customer. `HAVING` keeps the 3 piles whose sum exceeds 1500. `SELECT` and `ORDER BY` only format what is left. The red sentence under the boxes is the key rule of the chapter: a clause can only use things that already exist when it runs.

Problem 3.5.1 Trace the query 'paid spend per customer, only customers above 1500' step by step and write down how many rows or groups remain after each stage. Predict first: the shop has 20 order lines, order 104 (cancelled) has 1 line and order 109 (refunded) has 1 line.

```
SELECT COUNT(*) AS lines_after_join
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id;
```

```
lines_after_join
----------------
20
```

```
SELECT COUNT(*) AS lines_after_where
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid';
```

```
lines_after_where
-----------------
18
```

```
SELECT COUNT(*) AS n_groups FROM (
  SELECT o.customer_id
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY o.customer_id);
```

```
n_groups
--------
5
```

```
SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
HAVING SUM(oi.qty * oi.unit_price) > 1500
ORDER BY spent DESC;
```

```
customer_id  spent
-----------  -----
1            4560
2            3600
4            3300
```

Stage 1	`FROM ... JOIN` gives one row per order line: 20.
Stage 2	`WHERE status = 'paid'` removes the lines of order 104 (cancelled) and order 109 (refunded): 20 - 1 - 1 = 18.
Stage 3	`GROUP BY customer_id` sorts the 18 lines into piles. Customers 1, 2, 4, 5 and 6 have paid lines (customer 3 had only the cancelled order, so there is no pile): 5 groups.
Stage 4	`HAVING SUM(...) > 1500` keeps customers 1 (4560), 2 (3600) and 4 (3300). Customer 5 has 760 and customer 6 has 440: 3 groups.
Answer	The final output has 3 rows, one per surviving customer, sorted by spent.
Check	The counts go 20, 18, 5, 3 and each step can only shrink or keep the count. If a count ever grows after a stage, something multiplied rows.

Problem 3.5.2 Do `GROUP BY` by hand on a tiny table before trusting the database. The table `marks` has 8 rows (4 students times 2 subjects). One mark, Ravi's science mark, is NULL because he was absent. Find the number of rows, the number of real marks, the total, the lowest and the highest mark per subject.

```
CREATE TABLE marks (student TEXT, subject TEXT, mark INTEGER);
INSERT INTO marks VALUES
 ('Asha','maths',78),('Asha','science',85),
 ('Ravi','maths',62),('Ravi','science',NULL),
 ('Meena','maths',91),('Meena','science',74),
 ('John','maths',55),('John','science',68);
SELECT * FROM marks;
```

```
student  subject  mark
-------  -------  ----
Asha     maths    78
Asha     science  85
Ravi     maths    62
Ravi     science  NULL
Meena    maths    91
Meena    science  74
John     maths    55
John     science  68
```

```
SELECT subject, COUNT(*) AS n_rows, COUNT(mark) AS n_marks, SUM(mark) AS total,
       MIN(mark) AS lowest, MAX(mark) AS highest
FROM marks
GROUP BY subject;
```

```
subject  n_rows  n_marks  total  lowest  highest
-------  ------  -------  -----  ------  -------
maths    4       4        286    55      91
science  4       3        227    68      85
```

Step 1	Make two piles by the key `subject`. Maths pile: 78, 62, 91, 55. Science pile: 85, NULL, 74, 68.
Step 2 (maths)	4 rows and 4 real marks. Sum: 78 + 62 + 91 + 55 = 286. Lowest 55, highest 91.
Step 3 (science)	4 rows but only 3 real marks (85, 74, 68). The NULL is skipped by `COUNT(mark)`, `SUM`, `MIN` and `MAX`. Sum: 85 + 74 + 68 = 227. Lowest 68, highest 85.
Check	The printed table matches both hand piles. `COUNT(*)` counted the absent student, `COUNT(mark)` did not.
Meaning	Two different questions hide here: 'how many students were enrolled' (4) and 'how many marks were recorded' (3). Pick the function that matches the question.

Problem 3.5.3 A teammate writes `WHERE COUNT(*) > 2` to find customers with more than two orders. What does the database say, and why?

```
SELECT customer_id, COUNT(*) FROM orders WHERE COUNT(*) > 2 GROUP BY customer_id;
```

```
Error: misuse of aggregate: COUNT()
```

Step 1	Follow the pipeline. `WHERE` is stage 2, applied to one row at a time.
Step 2	At that moment no pile exists, so there is no `COUNT(*)` for a customer yet.
Step 3	The database refuses: an aggregate is not allowed where only single rows are visible.
Fix	Move the test to `HAVING COUNT(*) > 2`, which runs at stage 4, after the piles exist.
Answer	The error is a pipeline-order error, not a typing error. When you see 'misuse of aggregate', ask: which stage am I in?

COMMON MISTAKE

Putting an aggregate in `WHERE`. Wrong: `WHERE COUNT(*) > 2` (error: misuse of aggregate). Right: `GROUP BY customer_id HAVING COUNT(*) > 2`. The reverse mistake also exists: using `HAVING status = 'paid'` for a plain row condition. It works, but it makes the database build piles for every status first and then discard them. Put row conditions in `WHERE`.

Check yourself. (1) In which stage is a column alias from `SELECT` created? Answer: stage 5, after `HAVING`, which is why strict databases refuse the alias earlier. (2) With no `GROUP BY`, how many piles are there? Answer: one, the whole table. (3) What is the grain of Problem 3.5.2's result? Answer:

one row per subject.

3.6 Aggregate Functions in Depth: COUNT, SUM, AVG, MIN, MAX and String Aggregation

Section 3.1 listed the functions. Here you learn to *choose* the right one, to predict the answer with NULL and empty tables in play, and to avoid the five silent errors that produce a believable but wrong number. We climb a ladder: counting first, then sums and averages, then min and max, then joining text together.

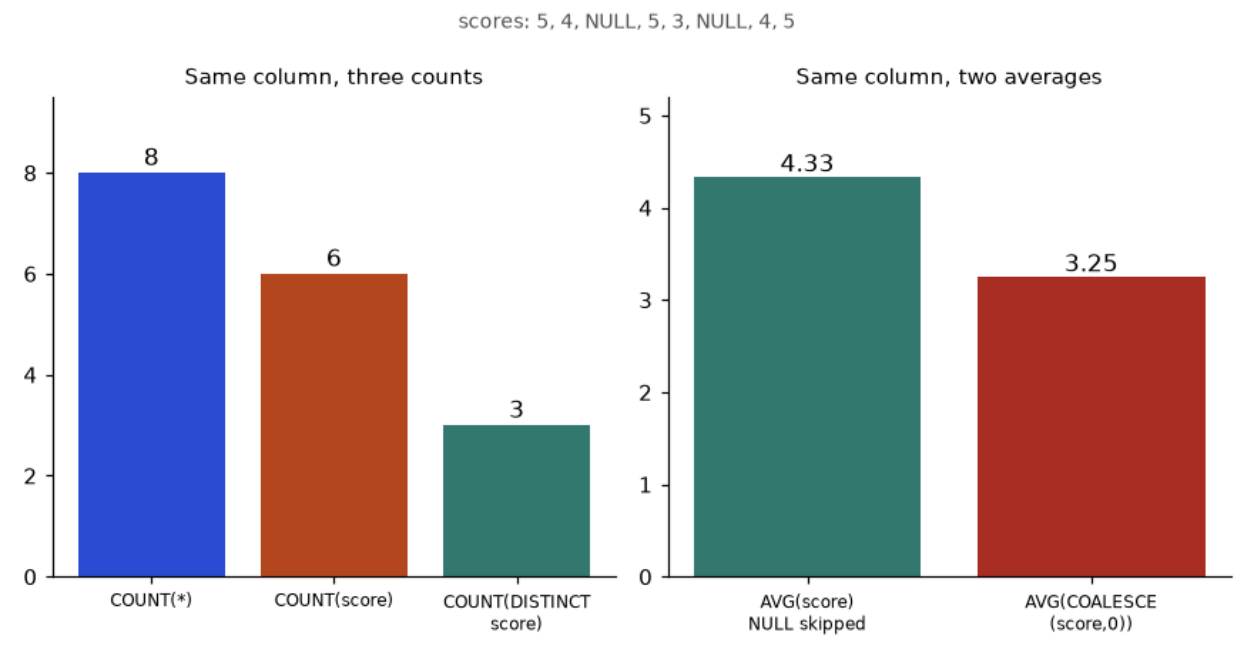
Counting: three functions, three different questions

KEY IDEA `COUNT(*)` asks 'how many rows?'. `COUNT(col)` asks 'in how many rows is `col` filled in?'. `COUNT(DISTINCT col)` asks 'how many different filled-in values?'. They only agree when the column has no NULLs and no repeats.

Problem 3.6.1 (trivial) Eight reviewers gave a score from 1 to 5, but two skipped the question (NULL). Find the number of rows, the number of answers, the number of different scores, the sum, and two averages. Predict first: scores are 5, 4, NULL, 5, 3, NULL, 4, 5.

```
CREATE TABLE reviews (reviewer TEXT, score INTEGER);
INSERT INTO reviews VALUES ('A',5),('B',4),('C',NULL),('D',5),('E',3),('F',NULL),('G',4),('H',5);
SELECT COUNT(*)              AS all_rows,
       COUNT(score)          AS answered,
       COUNT(DISTINCT score) AS different_scores,
       SUM(score)            AS total,
       AVG(score)            AS avg_skip_null,
       AVG(COALESCE(score,0)) AS avg_null_as_zero
FROM reviews;
```

```
all_rows  answered  different_scores  total  avg_skip_null      avg_null_as_zero
--------  --------  ----------------  -----  -----------------  ----------------
8         6         3                 26     4.333333333333333  3.25
```



Three counts and two averages of the same eight-row column.

How to read it: on the left the same column gives 8, 6 and 3 depending on the function: eight rows, six answers, three different values (3, 4 and 5). On the right the average is 4.33 if the two blanks mean 'no answer' and 3.25 if they mean 'zero points'. Neither is a database bug; they answer different questions, and you must decide which question is yours.

Step 1	`COUNT(*)` counts rows: 8.
Step 2	`COUNT(score)` skips the 2 NULLs: 6.
Step 3	`COUNT(DISTINCT score)`: the values 5, 4, 3 are the only different ones: 3.
Step 4	`SUM(score)` skips NULL: 5 + 4 + 5 + 3 + 4 + 5 = 26.
Step 5	`AVG(score)` divides by the 6 real answers: 26 / 6 = 4.3333.
Step 6	With `COALESCE(score,0)` the NULLs become 0 and the divisor is 8: 26 / 8 = 3.25.
Answer	6 answers, 4.33 average among those who answered, 3.25 if non-answers count as zero.
Meaning	'Average satisfaction of people who responded' is 4.33. A manager who reads 3.25 would think customers are unhappier than they are.

Problem 3.6.2 (easy) On the orders table, how many orders have a coupon, how many different coupon codes exist, how many different statuses, and how many different customers placed orders?

```
SELECT COUNT(*)               AS all_orders,
       COUNT(coupon)          AS with_coupon,
       COUNT(DISTINCT coupon)  AS kinds_of_coupon,
       COUNT(DISTINCT status)  AS kinds_of_status,
       COUNT(DISTINCT customer_id) AS buyers
FROM orders;
```

```
all_orders  with_coupon  kinds_of_coupon  kinds_of_status  buyers
----------  -----------  ---------------  ---------------  ------
14          3            1                3                6
```

Step 1	`COUNT(*)` = 14 orders, the table size.
Step 2	`COUNT(coupon)` counts only non-NULL coupons: the 3 orders 102, 105 and 110.
Step 3	`COUNT(DISTINCT coupon)`: all three used the same code SAVE10, so 1 kind.
Step 4	Statuses are paid, cancelled and refunded: 3 kinds. Customers who ever ordered: 1, 2, 3, 4, 5, 6: 6 buyers.
Check	The number of orders without a coupon is `COUNT(*) - COUNT(coupon)` = 14 - 3 = 11, matching Problem 3.P.7.

Problem 3.6.3 (realistic) For each product category, how many order lines, how many different orders and how many different customers? Here each number has a different grain, so each needs its own function. Predict: Electronics is bought in many orders, but a customer may buy two electronics items in the same order.

```
SELECT p.category,
       COUNT(*)                    AS lines,
       COUNT(DISTINCT oi.order_id) AS orders,
       COUNT(DISTINCT o.customer_id) AS buyers
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN orders   o ON o.order_id   = oi.order_id
GROUP BY p.category
ORDER BY p.category;
```

```
category     lines  orders  buyers
-----------  -----  ------  ------
Electronics  8      7       4
Home         3      3       3
Stationery   9      7       4
```

Step 1	Join items to products (for the category) and to orders (for the customer). Now one row is one order line.
Step 2	`COUNT(*)` counts lines. `COUNT(DISTINCT oi.order_id)` counts each order once even if it has two lines of that category. `COUNT(DISTINCT o.customer_id)` counts each person once.
Step 3	Read the Electronics row: 8 lines in 7 orders bought by 4 customers (1, 2, 4 and 5). Order 114 contains both a Keyboard and a Mouse, so it contributes 2 lines but 1 order.
Step 4	Stationery has 9 lines in 7 orders bought by customers 1, 3, 5 and 6. Orders 110 and 113 each contain two stationery products, so lines exceed orders by 2.
Check	Lines add up to 8 + 3 + 9 = 20, the number of order lines. Orders do *not* add up to 14 across categories (7 + 3 + 7 = 17), because an order with items from two categories is counted once in each.
Meaning	'How many customers buy Stationery?' needs `COUNT(DISTINCT customer_id)`. Using `COUNT(*)` here would overstate the audience (9 instead of 4).

Look closely at the output above: the lines, orders and buyers columns are different numbers for the same category. Whenever a number must describe a *thing* (customers, orders), count that thing's key with `DISTINCT` after a join, or aggregate before joining (Section 3.7).

SUM and AVG: empty sets, integer division and the DISTINCT trap

Two special cases surprise people: what `SUM` returns for no rows at all, and what happens when two integers are divided.

Problem 3.6.4 (medium) What do the aggregates return when no row passes the `WHERE` (here: no order line has more than 1000 units)? Predict first: how many rows does the result have?

```
SELECT COUNT(*) AS n, SUM(qty) AS total_qty, AVG(qty) AS avg_qty,
       MAX(qty) AS max_qty, COALESCE(SUM(qty), 0) AS total_or_zero
FROM order_items
WHERE qty > 1000;
```

```
n  total_qty  avg_qty  max_qty  total_or_zero
-  ---------  -------  -------  -------------
0  NULL       NULL     NULL     0
```

```
SELECT SUM(qty) AS sum_empty, TOTAL(qty) AS total_empty
FROM order_items WHERE qty > 1000;
```

```
sum_empty  total_empty
---------  -----------
NULL       0.0
```

Step 1	An aggregate query without `GROUP BY` always returns exactly one row, even when the input is empty.
Step 2	`COUNT` of nothing is 0, because zero is a perfectly good count.
Step 3	`SUM` , `AVG` , `MAX` of nothing have no sensible number, so SQL returns NULL.
Step 4	`COALESCE(SUM(qty), 0)` converts that NULL to 0 when a report needs a number. SQLite's own `TOTAL()` does the same but returns a decimal (0.0).
Answer	One row: n = 0, the rest NULL (or 0 where you coalesced).
Meaning	A dashboard card that shows 'revenue: NULL' on a quiet day is a bug waiting to be reported. Decide whether empty means 0 or 'no data'.

Problem 3.6.5 (medium) Integer division. Compute 7 / 2 four ways, then the average units per order line from `SUM(qty) / COUNT(*)` and from `AVG(qty)` .

```
SELECT 7 / 2      AS int_div,
       7 / 2.0    AS dec_div,
       1.0 * 7 / 2 AS forced_dec,
       CAST(7 AS REAL) / 2 AS cast_dec;
```

```
int_div  dec_div  forced_dec  cast_dec
-------  -------  ----------  --------
3        3.5      3.5         3.5
```

```
SELECT SUM(qty)               AS units,
       COUNT(*)               AS n_lines,
       SUM(qty) / COUNT(*)    AS wrong_int,
       1.0 * SUM(qty) / COUNT(*) AS right_dec,
       AVG(qty)               AS avg_builtin
FROM order_items;
```

```
units  n_lines  wrong_int  right_dec  avg_builtin
-----  -------  ---------  ---------  -----------
34     20       1          1.7        1.7
```

Step 1	Two integers divided with `/` in SQLite (and in SQL Server, and in PostgreSQL) give an integer: 7 / 2 = 3.
Step 2	Make either side a decimal and you get 3.5: write `7 / 2.0` , `1.0 * 7 / 2` or `CAST(7 AS REAL) / 2` .
Step 3	Total units 34 divided by 20 lines: `34 / 20` as integers is 1 (the 0.7 is thrown away), but the right answer is 1.7.
Step 4	`AVG(qty)` already works in decimals in SQLite and gives 1.7.
Answer	Average units per line is 1.7.
Check	20 x 1.7 = 34, the total units. Always multiply the average back by the count to see if you get the total.

COMMON MISTAKE	Dividing two sums or counts without making one decimal: `SUM(qty) / COUNT(*)` returns 1 here, not 1.7. MySQL's `/` always returns a decimal, but PostgreSQL, SQL Server and SQLite truncate integer division, so a query that looks fine on one engine is wrong on another. Habit: write `1.0 * SUM(x) / COUNT(*)` or use `AVG(x)` .

Problem 3.6.6 (medium) The DISTINCT trap. What is the total of `unit_price` over all order lines, and what does `SUM(DISTINCT unit_price)` return? Which one is revenue-relevant?

```
SELECT SUM(unit_price)          AS sum_all,
       SUM(DISTINCT unit_price) AS sum_distinct,
       COUNT(DISTINCT unit_price) AS different_prices
FROM order_items;
```

```
sum_all  sum_distinct  different_prices
-------  ------------  ----------------
11980    3200          5
```

Step 1	`SUM(unit_price)` adds the price on all 20 lines: 11980. (It is not revenue, since it ignores quantity; it is only a demonstration.)
Step 2	`SUM(DISTINCT unit_price)` first removes repeated prices, leaving 1500, 600, 80, 120, 900, then adds them: 3200.
Step 3	Five different prices, so `COUNT(DISTINCT unit_price)` is 5.
Answer	11980 versus 3200.
Meaning	Two different lines selling a 600 mouse are two real sales. `SUM(DISTINCT amount)` counts only one of them and silently under-reports revenue.

COMMON MISTAKE

Using `SUM(DISTINCT amount)` to 'fix' double counting after a join. It removes real sales that merely have the same amount. The safe fixes are to aggregate before joining (Section 3.7) or to sum a per-row key-safe value. `DISTINCT` belongs with `COUNT` of an *identifier*, almost never with `SUM`.

Average of averages and the weighted average

An average is a sum divided by a count. If you average two averages you give a group of 1 order the same voice as a group of 100. The correct recipe: add all numerators, add all denominators, divide once.

Problem 3.6.7 (realistic) For paid orders compute each customer's average order value, then the plain mean of those averages, and the true average order value (AOV) across all 12 paid orders.

```
SELECT o.customer_id,
       COUNT(DISTINCT o.order_id)  AS n_orders,
       SUM(oi.qty * oi.unit_price) AS spent,
       ROUND(1.0 * SUM(oi.qty * oi.unit_price) / COUNT(DISTINCT o.order_id), 1) AS aov
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
ORDER BY o.customer_id;
```

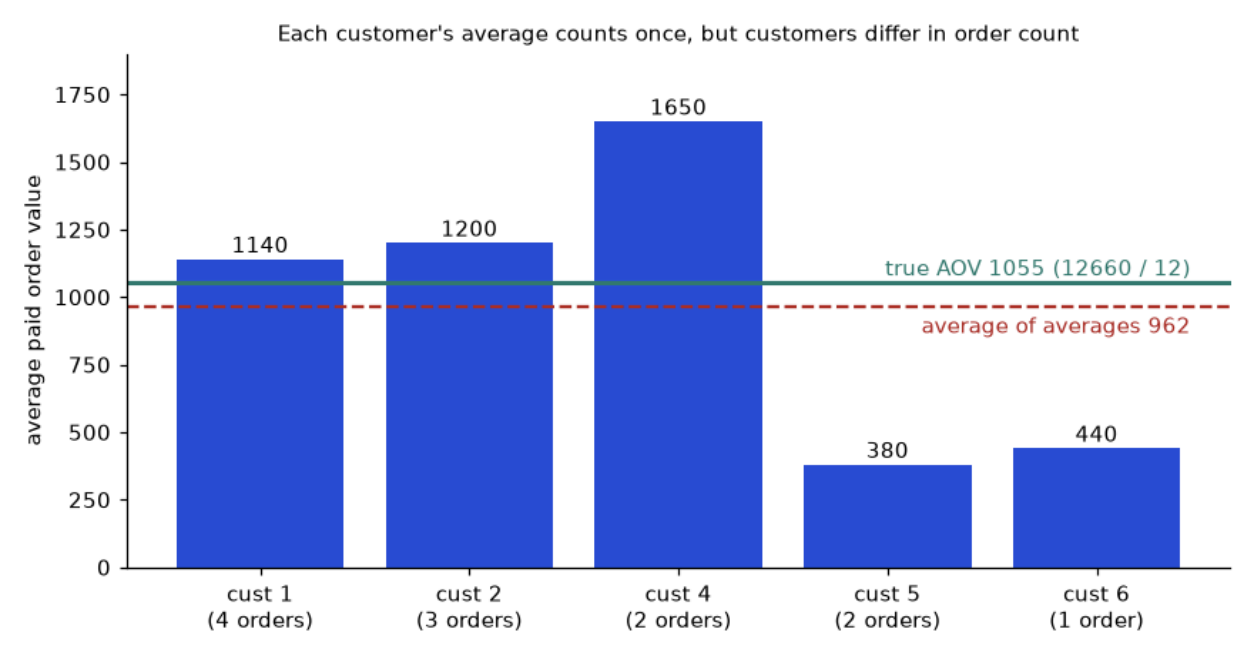
```
customer_id  n_orders  spent  aov
-----------  --------  -----  ------
1            4         4560   1140.0
2            3         3600   1200.0
4            2         3300   1650.0
5            2         760    380.0
6            1         440    440.0
```

```
SELECT ROUND(AVG(aov), 1) AS avg_of_averages
FROM (SELECT o.customer_id, 1.0 * SUM(oi.qty * oi.unit_price) / COUNT(DISTINCT o.order_id) AS aov
      FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
      WHERE o.status = 'paid'
      GROUP BY o.customer_id);
```

```
avg_of_averages
---------------
962.0
```

```
SELECT ROUND(1.0 * SUM(oi.qty * oi.unit_price) / COUNT(DISTINCT o.order_id), 1) AS true_aov
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid';
```

```
true_aov
--------
1055.0
```



Per-customer average order value against the average of averages (962) and the true AOV (1055).

How to read it: each bar is one customer's own average order value. Customer 4 (two orders) and customer 6 (one order) pull the plain mean of the five bars down to 962, because each customer counts once. The true AOV is 1055 because customer 1 placed four orders and customer 2 three, so their high averages carry more weight in the real total of 12 orders.

Step 1	Per customer: 4560 / 4 = 1140, 3600 / 3 = 1200, 3300 / 2 = 1650, 760 / 2 = 380, 440 / 1 = 440.
Step 2	Average of those five: (1140 + 1200 + 1650 + 380 + 440) / 5 = 4810 / 5 = 962.
Step 3	True AOV: total paid revenue 12660 divided by total paid orders 12 = 1055.
Answer	962 (wrong) versus 1055 (right) for 'average order value'.
Check	12 x 1055 = 12660, the real revenue. 12 x 962 = 11544 does not match the revenue, so 962 cannot be the AOV.
Rule	Never average averages unless every group has the same size. Carry the sum and the count up and divide once.

MIN and MAX: more than numbers

Problem 3.6.8 (easy) For each product, the smallest and largest quantity ever bought in one line, and the spread (max minus min).

```
SELECT product_id,
       MIN(qty) AS min_qty, MAX(qty) AS max_qty,
       MAX(qty) - MIN(qty) AS spread
FROM order_items
GROUP BY product_id
ORDER BY product_id;
```

```
product_id  min_qty  max_qty  spread
----------  -------  -------  ------
1           1        1        0
2           1        2        1
3           1        5        4
4           1        3        2
5           1        2        1
```

Step 1	Pile order lines by product.
Step 2	Keyboard (product 1) was always bought one at a time: min = max = 1, spread 0.
Step 3	Notebook (product 3) ranges from 1 to 5: spread 4. That product has the most variable order size.
Answer	The table above; the notebook has the largest spread (4).

Problem 3.6.9 (medium) `MIN` and `MAX` also work on dates and text. Find the first and last order date, the alphabetically first and last status, and the minimum coupon. Predict the coupon: only 3 orders have one.

```
SELECT MIN(order_date) AS first_order, MAX(order_date) AS last_order,
       MIN(status) AS first_status, MAX(status) AS last_status,
       MIN(coupon) AS min_coupon
FROM orders;
```

```
first_order  last_order  first_status  last_status  min_coupon
-----------  ----------  ------------  -----------  ----------
2024-01-10   2024-04-20  cancelled     refunded     SAVE10
```

Step 1	ISO dates (YYYY-MM-DD) sort the same way as time, so `MIN` and `MAX` give the earliest and latest date: 2024-01-10 and 2024-04-20.
Step 2	Text compares letter by letter: cancelled < paid < refunded.
Step 3	`MIN(coupon)` ignores the 11 NULL coupons and returns SAVE10.
Answer	Dates 2024-01-10 to 2024-04-20, statuses cancelled and refunded, coupon SAVE10.
Meaning	Dates stored as text work only if they are always in the same sortable format. 'DD-MM-YYYY' text would give nonsense.

String aggregation: joining the values of a pile into one cell

Sometimes the pile should become a list, not a number: 'which customers live in each city'. Every database has a function, with a different name.

Database	Function	Example
SQLite, MySQL	`group_concat(x, ', ')`	`group_concat(name, ', ')`
PostgreSQL, SQL Server 2017+	`string_agg(x, ', ')`	`string_agg(name, ', ' ORDER BY name)`

Database	Function	Example
Oracle	`LISTAGG(x, ', ')`	`LISTAGG(name, ', ') WITHIN GROUP (ORDER BY name)`
BigQuery, Snowflake	`STRING_AGG` / `LISTAGG`	check the dialect page in Chapter 14

Problem 3.6.10 (medium) List the customers living in each city in one cell, with a count. Predict: Pune has three customers.

```
SELECT city,
       COUNT(*)             AS n,
       group_concat(name, ', ') AS customers
FROM customers
GROUP BY city
ORDER BY city;
```

```
city       n  customers
---------  -  ----------------
Bengaluru  1  Priya
Delhi      2  John, Sana
Mumbai     2  Ravi, Kabir
Pune       3  Asha, Meena, Tom
```

Step 1	`GROUP BY city` makes four piles.
Step 2	`group_concat(name, ', ')` glues the names of one pile with a comma and space.
Step 3	`COUNT(*)` counts the pile: Pune 3 (Asha, Meena, Tom), Delhi 2, Mumbai 2, Bengaluru 1.
Answer	The table above.
Caution	The order of names inside the cell is not guaranteed in SQLite. PostgreSQL lets you write `string_agg(name, ', ' ORDER BY name)`. Never rely on an unordered list in a report.

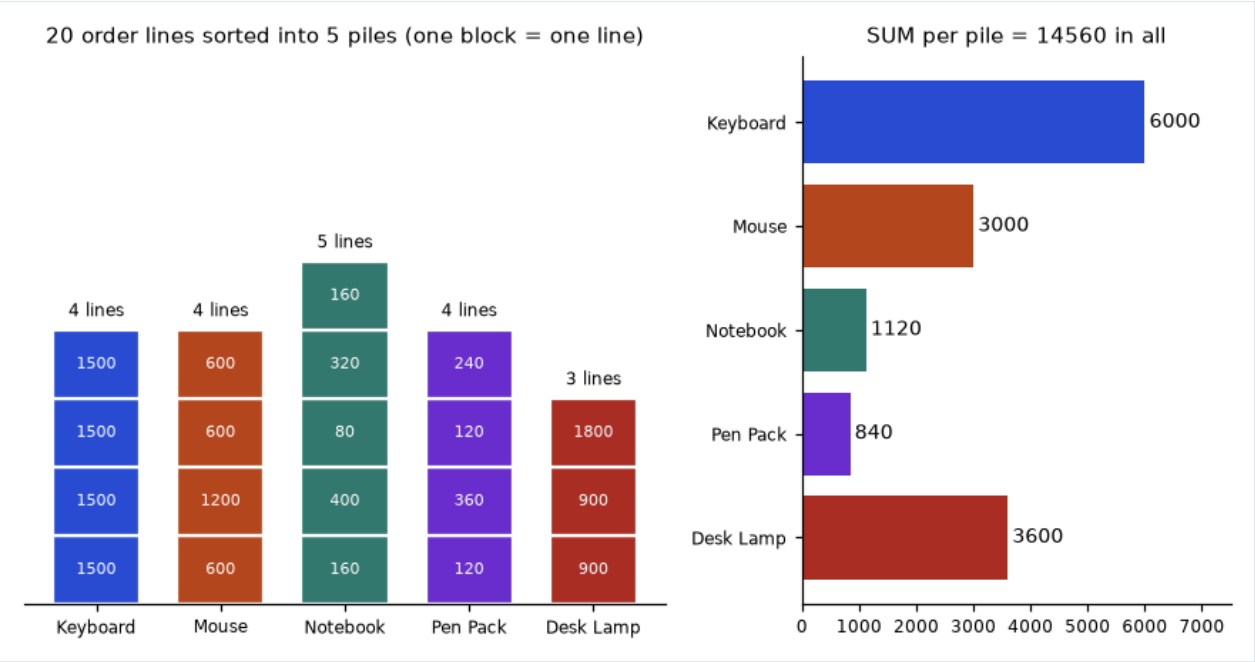
REMEMBER

Pick the function by the question: rows (`COUNT(*)`), filled values (`COUNT(col)`), distinct things (`COUNT(DISTINCT id)`), total (`SUM`), typical value among non-NULL (`AVG`), extremes (`MIN` / `MAX`), list (`group_concat` / `string_agg`).

3.7 GROUP BY in Depth: Grain, Several Keys, NULL Groups and Fan-out

INTUITION

Think of `GROUP BY` as a sorting machine with labelled bins. You choose the label (the key). Every row drops into the bin with its label. The result has exactly one row per bin that received at least one row. So the first thing to decide is never 'which function' but 'what is one row of my answer'. That sentence is the **grain**.



Left: the 20 order lines of the shop sorted into five piles by product. Right: the sum of each pile.

How to read it: each coloured block is one order line and its number is `qty * unit_price`. `GROUP BY product_id` builds the five stacks (Keyboard 4 lines, Mouse 4, Notebook 5, Pen Pack 4, Desk Lamp 3). `SUM` adds the numbers inside each stack and the bars show the result. Every block lands in exactly one stack, which is why the five bars always add up to the ungrouped total 14560.

Two guarantees you can rely on

- **Partition.** Every row goes into exactly one group. So the group counts add up to the table count, and the group sums of a column add up to the total of that column. This is your best check.
- **Grain.** The result has one row per distinct combination of key values, nothing more and nothing less. Rows with no match do not create a group, so a group with zero rows can never appear (you need a join from a table that lists all the keys, Chapter 4).

Problem 3.7.1 (easy) How many orders has each customer placed, including customers who never ordered? Show what happens with `COUNT(*)` versus `COUNT(o.order_id)` after a `LEFT JOIN`. Predict: Sana (7) and Tom (8) have no orders.

```
SELECT c.customer_id, c.name,
       COUNT(*)          AS wrong_count_star,
       COUNT(o.order_id) AS right_n_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY c.customer_id;
```

```
customer_id  name   wrong_count_star  right_n_orders
-----------  -----  ----------------  --------------
1            Asha   4                 4
2            Ravi   3                 3
3            Meena  1                 1
4            John   2                 2
5            Priya  3                 3
6            Kabir  1                 1
7            Sana   1                 0
8            Tom    1                 0
```

Step 1	`LEFT JOIN` keeps every customer. A customer with no order gets one joined row with NULL in all order columns.
Step 2	`COUNT(*)` counts that placeholder row, so Sana and Tom show 1 instead of 0.
Step 3	`COUNT(o.order_id)` counts only non-NULL order ids, so the placeholder adds nothing: Sana and Tom show 0.
Answer	Asha 4, Ravi 3, Meena 1, John 2, Priya 3, Kabir 1, Sana 0, Tom 0.
Check	4 + 3 + 1 + 2 + 3 + 1 + 0 + 0 = 14, the number of orders.

COMMON MISTAKE	`COUNT(*)` after a `LEFT JOIN`. The unmatched customer still produces one row, so you report 1 order instead of 0. Count a column from the *right-hand* table (`COUNT(o.order_id)`) when you want 'how many matches'. Chapter 4 covers LEFT JOIN in full.

Several keys

Each extra key makes the bins finer. The number of result rows is at most the product of the number of distinct values of each key, and usually far smaller because many combinations never occur.

Problem 3.7.2 (medium) Paid revenue by month and by category. Predict: 4 months and 3 categories could give 12 rows, but January has no Home sale.

```
SELECT substr(o.order_date,1,7) AS month, p.category,
       SUM(oi.qty * oi.unit_price)   AS paid_revenue
FROM orders o
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products    p  ON p.product_id  = oi.product_id
WHERE o.status = 'paid'
GROUP BY month, p.category
ORDER BY month, p.category;
```

```
month    category     paid_revenue
-------  -----------  ------------
2024-01  Electronics  2100
2024-01  Stationery   160
2024-02  Electronics  1500
2024-02  Home         900
2024-02  Stationery   120
2024-03  Electronics  1200
2024-03  Home         900
2024-03  Stationery   880
2024-04  Electronics  2700
2024-04  Home         1800
2024-04  Stationery   400
```

Step 1	Grain: one row per (month, category). Join three tables to get the month (orders), the category (products) and the line total (order_items).
Step 2	`WHERE o.status = 'paid'` removes orders 104 and 109 first.
Step 3	`GROUP BY month, p.category` : a pile for every pair that occurs.
Step 4	Count the rows: Jan 2, Feb 3, Mar 3, Apr 3 = 11, not 12, because Home has no sale in January.
Check	The 11 sums add to 12660: 2100 + 160 + 1500 + 900 + 120 + 1200 + 900 + 880 + 2700 + 1800 + 400 = 12660.
Meaning	A missing row is not a zero. To draw a chart you need a row of 0 for (2024-01, Home); that takes a calendar-style grid joined with `LEFT JOIN` . Section 3.9 shows the pivot alternative.

Ways to name the key

The same grouping can be written with a column name, an expression, an alias or a position number. They produce the same piles.

```
SELECT status, COUNT(*) AS n FROM orders GROUP BY status ORDER BY status;
```

```
status     n
---------  --
cancelled  1
paid       12
refunded   1
```

```
SELECT status, COUNT(*) AS n FROM orders GROUP BY 1 ORDER BY 1;
```

```
status     n
---------  --
cancelled  1
paid       12
refunded   1
```

Style	Example	Works in
column name	`GROUP BY status`	everywhere
expression	`GROUP BY substr(order_date,1,7)`	everywhere (repeat the expression in SELECT)
alias of a SELECT item	`GROUP BY month`	SQLite, MySQL, PostgreSQL; not SQL Server, not older Oracle
position number	`GROUP BY 1`	SQLite, MySQL, PostgreSQL, others; fine for quick work, fragile when you edit the column list

Rule for production code: group by the real expression or column, and give the key a clear alias in `SELECT`. In daily exploration `GROUP BY 1, 2` is a harmless time-saver.

NULL is a legal key

Problem 3.7.3 (medium) Orders per coupon, with the number paid. Predict: coupons are SAVE10 on three orders and NULL on eleven.

```
SELECT COALESCE(coupon, '(none)') AS coupon_used,
       COUNT(*) AS n_orders,
       COUNT(*) FILTER (WHERE status = 'paid') AS n_paid
FROM orders
GROUP BY coupon
ORDER BY n_orders DESC;
```

```
coupon_used  n_orders  n_paid
-----------  --------  ------
(none)       11        9
SAVE10       3         3
```

Step 1	In `GROUP BY`, all NULLs are treated as equal, so the 11 orders without a coupon form one bin. (Compare with `WHERE coupon = NULL`, which is never true. Grouping uses 'same bucket', not 'equal'.)
Step 2	`COALESCE(coupon, '(none)')` in SELECT is only a readable label for the NULL bin. It is allowed because it is built from the grouped column alone.
Step 3	`FILTER (WHERE status = 'paid')` counts the paid ones inside each bin: the 11 orders without a coupon include 1 cancelled and 1 refunded, so 9 paid; the 3 coupon orders are all paid.
Check	11 + 3 = 14 orders; 9 + 3 = 12 paid, matching the 12 paid orders overall.

What may appear in SELECT?

Only the group keys, aggregates, constants, and expressions built from those. A plain column that is neither is ambiguous: a pile has many values of it, and the database must pick one. PostgreSQL, SQL Server and Oracle refuse such a query. SQLite and (old) MySQL accept it and pick a value from some row of the pile.

Problem 3.7.4 (medium) What does SQLite return for `SELECT category, name, COUNT(*) ... GROUP BY category`? Then the same with `MAX(price)`. Predict whether the name is meaningful.

```
SELECT category, name, COUNT(*) AS n
FROM products
GROUP BY category
ORDER BY category;
```

```
category     name       n
-----------  ---------  -
Electronics  Keyboard   2
Home         Desk Lamp  2
Stationery   Notebook   2
```

```
SELECT category, MAX(price) AS top_price, name
FROM products
GROUP BY category
ORDER BY category;
```

```
category     top_price  name
-----------  ---------  ---------
Electronics  1500       Keyboard
Home         900        Desk Lamp
Stationery   120        Pen Pack
```

Step 1	In the first query `name` is not grouped, not aggregated. SQLite returns the name from an arbitrary row of each pile (Keyboard, Desk Lamp, Notebook here). Another SQLite version or a different table order could return another name.
Step 2	In the second query there is a single `MAX` or `MIN`. SQLite has a special documented rule: the bare column comes from the row that holds the max. So Keyboard (1500) and Desk Lamp (900) are correct; Pen Pack (120) is the dearest stationery item.
Step 3	PostgreSQL rejects both: `column products.name must appear in the GROUP BY clause or be used in an aggregate function`.
Answer	Never rely on the SQLite rule in code that must be portable. Use a window function (Chapter 8) or a join back to the max, shown in Pattern 5 of Section 3.10.

GROUP BY without an aggregate is DISTINCT

```
SELECT DISTINCT status FROM orders ORDER BY status;
```

```
status
---------
cancelled
paid
refunded
```

That is the same answer as `SELECT status FROM orders GROUP BY status`. Use `DISTINCT` when you only need the unique values, and `GROUP BY` when you also need aggregates. And remember the order of result rows is not promised by either: add `ORDER BY` whenever the order matters.

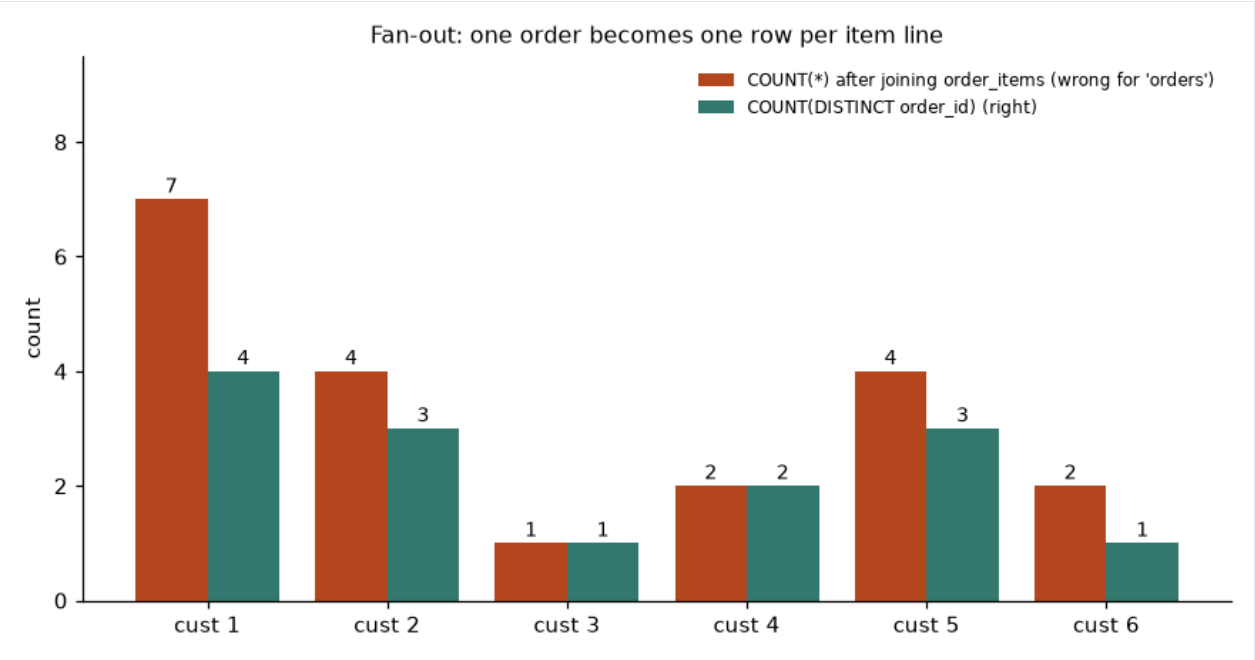
The fan-out trap: grouping after a join

INTUITION	A join repeats a row once for every partner it matches. An order with 3 item lines appears 3 times after the join. If you now count or sum an *order-level* fact, you count it 3 times. The group-by was fine; the rows it received were already inflated.

Problem 3.7.5 (medium) For each customer, count the rows after joining customers, orders and order items, and compare with the true number of orders. Predict for Asha: 4 orders but 7 item lines.

```
SELECT c.customer_id,
       COUNT(*)                   AS rows_after_join,
       COUNT(DISTINCT o.order_id)  AS real_orders
FROM customers c
JOIN orders      o  ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
GROUP BY c.customer_id
ORDER BY c.customer_id;
```

```
customer_id  rows_after_join  real_orders
-----------  ---------------  -----------
1            7                4
2            4                3
3            1                1
4            2                2
5            4                3
6            2                1
```



Rows after the join versus the real number of orders for each customer.

How to read it: the orange bar is what `COUNT(*)` reports after joining to `order_items`; the teal bar is the true number of orders. They match only for customers whose every order has exactly one item line (customers 3 and 4). Customer 1 shows 7 against 4 because three of its four orders have two item lines each (2 + 2 + 2 + 1 = 7). See the steps below.

Step 1	Grain after the join: one row per order line, not per order. Customer 1 has orders 101 (2 lines), 103 (2 lines), 106 (2 lines) and 111 (1 line): 2 + 2 + 2 + 1 = 7 rows.
Step 2	`COUNT(DISTINCT o.order_id)` collapses repeats of the order id: 4. That is the right 'number of orders'.
Step 3	Customer 3 has one order with one line, so both numbers are 1; customer 5 has orders 107 (1 line), 109 (1 line), 113 (2 lines): 4 rows, 3 orders.
Answer	Use `COUNT(DISTINCT order_id)` for orders and `COUNT(*)` only for lines.
Check	Total real orders: 4 + 3 + 1 + 2 + 3 + 1 = 14. Total rows: 7 + 4 + 1 + 2 + 4 + 2 = 20, the number of lines.

The cleaner cure: aggregate to the right grain first, then join. Below, a temporary named result `order_totals` (a CTE, taught in Chapter 5; read it as 'a named mini-table') first collapses lines to one row per order. Joining that to customers cannot multiply anything, because each order now appears once.

Problem 3.7.6 (realistic) Paid orders, revenue and average order value per city. First the correct way, then the broken way, and compare the order counts.

```
WITH order_totals AS (
  SELECT order_id, SUM(qty * unit_price) AS total
  FROM order_items
  GROUP BY order_id
)
SELECT c.city,
       COUNT(*)      AS paid_orders,
       SUM(t.total)  AS paid_revenue,
       ROUND(1.0 * SUM(t.total) / COUNT(*), 1) AS aov
FROM customers c
JOIN orders o        ON o.customer_id = c.customer_id
JOIN order_totals t  ON t.order_id    = o.order_id
WHERE o.status = 'paid'
GROUP BY c.city
ORDER BY c.city;
```

```
city       paid_orders  paid_revenue  aov
---------  -----------  ------------  ------
Bengaluru  2            760           380.0
Delhi      2            3300          1650.0
Mumbai     4            4040          1010.0
Pune       4            4560          1140.0
```

```
SELECT c.city,
       COUNT(*) AS wrong_orders,
       SUM(oi.qty * oi.unit_price) AS revenue_ok_but_count_wrong
FROM customers c
JOIN orders o       ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
WHERE o.status = 'paid'
GROUP BY c.city
ORDER BY c.city;
```

```
city       wrong_orders  revenue_ok_but_count_wrong
---------  ------------  --------------------------
Bengaluru  3             760
Delhi      2             3300
Mumbai     6             4040
Pune       7             4560
```

Step 1	`order_totals` has 14 rows, one per order. Joining it keeps the grain at order level, so `COUNT(*)` really counts orders: Bengaluru 2, Delhi 2, Mumbai 4, Pune 4. Total 12 paid orders.
Step 2	Revenue per city is the sum of order totals: Bengaluru 760, Delhi 3300, Mumbai 4040, Pune 4560. Total 12660.
Step 3	AOV = revenue / orders: Bengaluru 380.0, Delhi 1650.0, Mumbai 1010.0, Pune 1140.0.
Step 4	In the broken query the revenue column is still right (sum of line totals does not double count), but `COUNT(*)` counts lines: Bengaluru 3 instead of 2, Mumbai 6 instead of 4, Pune 7 instead of 4 (Delhi happens to be unaffected, which is how such bugs hide). Dividing the correct revenue by these wrong counts would give wrong AOVs.
Check	The correct order counts add to 2 + 2 + 4 + 4 = 12 paid orders. The broken counts add to 3 + 2 + 6 + 7 = 18, which is the number of paid lines, not orders.
Lesson	The same query can be right for one column and wrong for another. Check every column against a number you can compute independently.

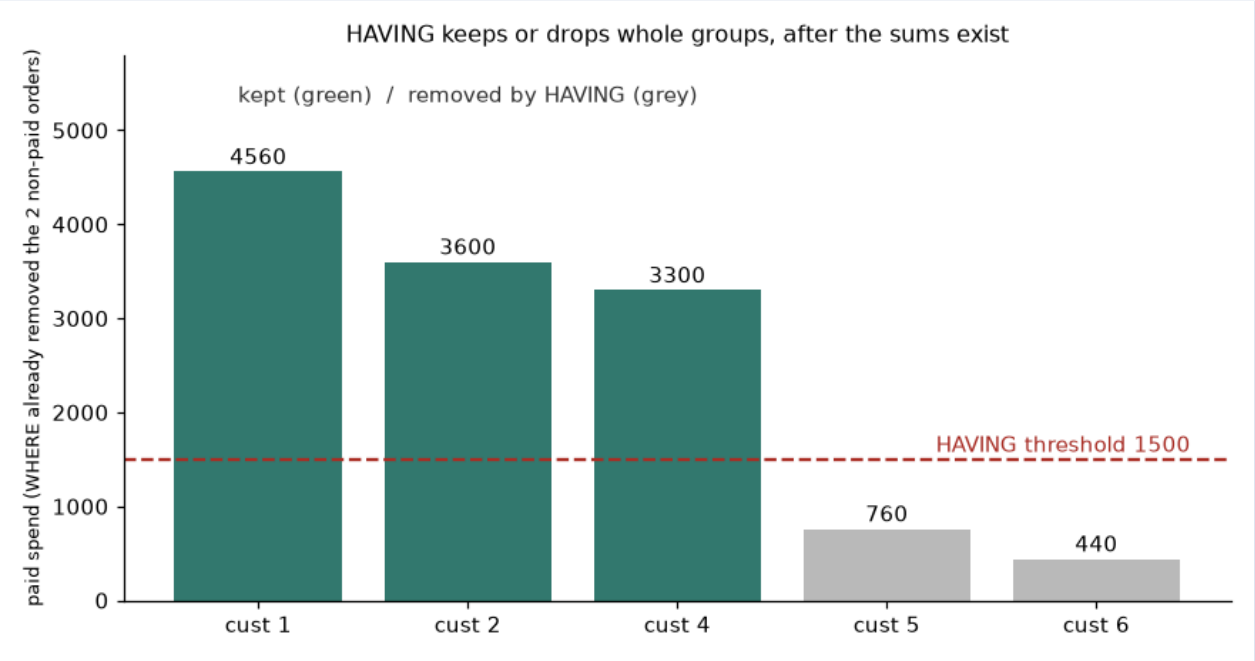
COMMON MISTAKE

Joining a detail table (order_items) just to filter or label, then counting rows. Every join to a 'many' side changes the grain. After each join ask: how many result rows per entity now? Cures in order of preference: aggregate before joining; use `COUNT(DISTINCT id)`; do not join the detail table at all (use `EXISTS`, Chapter 5).

Check yourself. (1) Which rows form the NULL group? Answer: all rows whose key is NULL, together. (2) A query groups by `customer_id` and selects `name`. Safe in PostgreSQL? Answer: no, unless `name` is also grouped, or customer_id is the table's primary key (PostgreSQL then allows it). (3) What does `GROUP BY` guarantee about row order? Answer: nothing.

3.8 HAVING in Depth: Filtering Groups Correctly

`HAVING` is a `WHERE` for piles. Section 3.3 showed the basics. Here are the shapes you will meet in real work: several conditions, a condition on an aggregate you do not display, a condition on the whole table, a condition that compares a group with all groups, and the 'all rows in the pile satisfy X' pattern.



Paid spend per customer with the HAVING threshold at 1500. Green groups survive, grey groups are removed.

How to read it: each bar is one customer pile after `WHERE status = 'paid'` and `SUM`. The dashed line is the `HAVING` threshold. A group is kept or dropped as a whole; `HAVING` never removes single rows from a kept group.

Problem 3.8.1 (easy) Customers who placed at least 2 paid orders *and* spent more than 1000 on them. Predict: customer 6 has one paid order of 440; customer 5 has two paid orders but only 760.

```
SELECT o.customer_id,
       COUNT(DISTINCT o.order_id)  AS paid_orders,
       SUM(oi.qty * oi.unit_price) AS spent
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
HAVING COUNT(DISTINCT o.order_id) >= 2
   AND SUM(oi.qty * oi.unit_price) > 1000
ORDER BY spent DESC;
```

```
customer_id  paid_orders  spent
-----------  -----------  -----
1            4            4560
2            3            3600
4            2            3300
```

Step 1	`WHERE` keeps paid lines.
Step 2	Group by customer. Per group compute the number of different orders and the total.
Step 3	`HAVING` with `AND` keeps groups meeting both: customer 1 (4 orders, 4560), 2 (3 orders, 3600) and 4 (2 orders, 3300).
Step 4	Customer 5 fails the money test (760), customer 6 fails both.
Check	Without the money test the first condition alone would keep 1, 2, 4 and 5. Each added condition can only remove groups.

Problem 3.8.2 (medium) Products that appear in at least 4 different orders. The count is used only to filter, not displayed. Predict: Notebook is popular; Desk Lamp appears in only 3 orders.

```
SELECT product_id, COUNT(DISTINCT order_id) AS in_orders
FROM order_items
GROUP BY product_id
HAVING COUNT(DISTINCT order_id) >= 4
ORDER BY product_id;
```

```
product_id  in_orders
----------  ---------
1           4
2           4
3           5
4           4
```

Step 1	`COUNT(DISTINCT order_id)` per product counts different orders (a product appears once per order because of the primary key, but DISTINCT states the intention).
Step 2	`HAVING ... >= 4` keeps Keyboard (product 1, 4 orders), Mouse (2, 4 orders), Notebook (3, 5 orders) and Pen Pack (4, 4 orders). Desk Lamp (product 5) is in orders 103, 108 and 112 only: 3, so it is dropped.
Check	Count by hand for the Mouse: orders 102, 106, 111, 114 = 4. The Mug (product 6) has no order lines, so it never forms a group.
Answer	Products 1, 2, 3 and 4.

Problem 3.8.3 (medium) `HAVING` without `GROUP BY`: the whole table is one group. Run a 'revenue alert' style test: return the order count only if there are more than 10 orders, then more than 20.

```
SELECT COUNT(*) AS n_orders FROM orders HAVING COUNT(*) > 10;
```

```
n_orders
--------
14
```

```
SELECT COUNT(*) AS n_orders FROM orders HAVING COUNT(*) > 20;
```

```
n_orders
--------
```

Step 1	With no `GROUP BY` there is one group, the entire table, so `HAVING` can keep it or drop it.
Step 2	14 > 10 is true: the single row survives. 14 > 20 is false: zero rows come back, not a row with 0.
Meaning	This is a handy health-check shape ('return a row only if something is wrong'), and a reminder that an empty result is different from a result containing 0.

Problem 3.8.4 (medium) The 'every row in the pile' pattern. Which customers have all their orders in the same status? Use `MIN(x) = MAX(x)`. Then find customers with at least one refund, and customers whose orders are *all* paid.

```
SELECT customer_id, COUNT(*) AS n_orders, MIN(status) AS status
FROM orders
GROUP BY customer_id
HAVING MIN(status) = MAX(status)
ORDER BY customer_id;
```

```
customer_id  n_orders  status
-----------  --------  ---------
1            4         paid
2            3         paid
3            1         cancelled
4            2         paid
6            1         paid
```

```
SELECT customer_id,
       COUNT(*) AS orders,
       COUNT(*) FILTER (WHERE status = 'refunded') AS refunded
FROM orders
GROUP BY customer_id
HAVING COUNT(*) FILTER (WHERE status = 'refunded') > 0;
```

```
customer_id  orders  refunded
-----------  ------  --------
5            3       1
```

```
SELECT customer_id, COUNT(*) AS orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) = COUNT(*) FILTER (WHERE status = 'paid')
ORDER BY customer_id;
```

```
customer_id  orders
-----------  ------
1            4
2            3
4            2
6            1
```

Step 1	If the smallest and largest status in a pile are equal, every value in the pile is that same value. Customers 1, 2, 3, 4 and 6 qualify; customer 5 has paid and refunded, so MIN is paid and MAX is refunded.
Step 2	'At least one refund' is a count of refunded rows above zero: only customer 5.
Step 3	'All orders paid' says `COUNT(*)` equals the count of paid rows. Customer 3 has one cancelled order and customer 5 has a refund, so both fail: 1, 2, 4, 6 remain.
Answer	Same status: 1, 2, 3, 4, 6. At least one refund: 5. All paid: 1, 2, 4, 6.
Check	Customer 3 passes the 'same status' test (all cancelled) but not the 'all paid' test, which shows the two questions are different.

Problem 3.8.5 (expert-ish) A group compared with all groups: customers whose paid spend is above the average paid spend per customer. Compute that average first, then use it inside `HAVING`.

```
SELECT AVG(s) AS avg_customer_spend FROM (
  SELECT SUM(oi.qty * oi.unit_price) AS s
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY o.customer_id);
```

```
avg_customer_spend
__________________
2532.0
```

```
SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
HAVING SUM(oi.qty * oi.unit_price) >
       (SELECT AVG(s) FROM (SELECT SUM(oi2.qty * oi2.unit_price) AS s
                            FROM orders o2 JOIN order_items oi2 ON oi2.order_id = o2.order_id
                            WHERE o2.status = 'paid'
                            GROUP BY o2.customer_id))
ORDER BY spent DESC;
```

```
customer_id  spent
___________  _____
1            4560
2            3600
4            3300
```

Step 1	Inner query: paid spend per customer: 4560, 3600, 3300, 760, 440.
Step 2	Average of those five: 12660 / 5 = 2532.
Step 3	`HAVING SUM(...) > (subquery)` keeps spends above 2532: customers 1, 2 and 4.
Check	Customers with spend 760 and 440 are below the average. Because the average of a skewed list is pulled up by big spenders, only 3 of 5 customers (60 percent) are above it here.
Meaning	The subquery returns exactly one number, so it can sit on the right of a comparison. Chapter 5 teaches subqueries properly.

Pitfalls with row conditions and NULL

Problem 3.8.6 (medium) Count the non-paid orders by status using `WHERE status <> 'paid'`. What happens to an order whose status is NULL? (The shop has none, but real tables do.)

```
SELECT status, COUNT(*) AS n FROM orders WHERE status <> 'paid' GROUP BY status;
```

```
status     n
---------  -
cancelled  1
refunded   1
```

Step 1	The two non-paid orders are 104 (cancelled) and 109 (refunded).
Step 2	`status <> 'paid'` is unknown when status is NULL, and `WHERE` keeps only rows where the test is true. So an order with a NULL status would be missing from both `= 'paid'` and `<> 'paid'` counts.
Step 3	If NULL should count as not paid, write `WHERE status IS DISTINCT FROM 'paid'` (PostgreSQL, SQLite 3.39+) or `WHERE status <> 'paid' OR status IS NULL`.
Answer	cancelled 1, refunded 1.
Meaning	Revisit Chapter 2, Section 2.4: three-valued logic follows you into every aggregate filter.

COMMON MISTAKE

Three `HAVING` errors. (1) Using a `SELECT` alias in `HAVING`: `HAVING spent > 1500` works in SQLite and MySQL but fails in PostgreSQL and SQL Server; repeat the expression. (2) Writing `HAVING` with a plain column that is not grouped, such as `HAVING city = 'Pune'` with no `GROUP BY city`: ambiguous. (3) Expecting `HAVING` to be fast: it filters after the work of grouping is done, so move every row condition to `WHERE`.

Check yourself. (1) Can `HAVING` refer to an aggregate that is not in `SELECT`? Answer: yes, as in Problem 3.8.2. (2) How do you say 'at least one row in the pile matches'? Answer: `HAVING COUNT(*) FILTER (WHERE cond) > 0`, or `MAX(CASE WHEN cond THEN 1 ELSE 0 END) = 1`. (3) What does `HAVING COUNT(*) = 0` return after `GROUP BY`? Answer: nothing, since a group has at least one row.

3.9 FILTER and Conditional Aggregation in Depth: Pivots, Rates, Funnels and Buckets

INTUITION

A plain `SUM` reads every row of its pile. Conditional aggregation gives each aggregate its own pair of glasses: 'you only see the paid rows', 'you only see the refunded rows'. Several aggregates with different glasses sit side by side in one `SELECT`, so one pass over the data fills a whole row of a report. That is how you build a pivot table with nothing but `GROUP BY`.

FILTER and CASE are the same idea

- `agg(x) FILTER (WHERE cond)` is standard SQL (PostgreSQL 9.4+, SQLite 3.30+, DuckDB). It reads like English.
- `agg(CASE WHEN cond THEN x END)` works in every database. Rows failing the condition become NULL, and aggregates skip NULL, so they are ignored. This equivalence is the whole trick.
- MySQL, SQL Server and Oracle have no `FILTER`: use `CASE`. BigQuery has `COUNTIF` and Snowflake `COUNT_IF` for the counting case. Check your version's documentation.

Problem 3.9.1 (easy) For every customer, the number of paid, cancelled and refunded orders, written twice: with `FILTER` and with `CASE`. Do they agree? Predict: customer 3 has one cancelled order; customer 5 has two paid and one refunded.

```
SELECT customer_id,
       COUNT(*) FILTER (WHERE status = 'paid')      AS f_paid,
       COUNT(*) FILTER (WHERE status = 'cancelled') AS f_cancelled,
       COUNT(*) FILTER (WHERE status = 'refunded')  AS f_refunded,
       SUM(CASE WHEN status = 'paid'      THEN 1 ELSE 0 END) AS c_paid,
       SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS c_cancelled,
       SUM(CASE WHEN status = 'refunded'  THEN 1 ELSE 0 END) AS c_refunded
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

```
customer_id  f_paid  f_cancelled  f_refunded  c_paid  c_cancelled  c_refunded
-----------  ------  -----------  ----------  ------  -----------  ----------
1            4       0            0           4       0            0
2            3       0            0           3       0            0
3            0       1            0           0       1            0
4            2       0            0           2       0            0
5            2       0            1           2       0            1
6            1       0            0           1       0            0
```

Step 1	One pile per customer (grain: one row per customer).
Step 2	Each `FILTER (WHERE status = ...)` shows only one status to its `COUNT`.
Step 3	`SUM(CASE WHEN ... THEN 1 ELSE 0 END)` adds a 1 for each matching row and a 0 for others, which is also a count.
Answer	The f_ and c_ columns are identical in every row. Customer 3: 0 paid, 1 cancelled, 0 refunded. Customer 5: 2, 0, 1.
Check	For every customer the three counts add up to the order count (customer 1: 4 + 0 + 0; customer 5: 2 + 0 + 1 = 3).

The CASE details that change the answer

Problem 3.9.2 (medium) Three nearly identical expressions, three different meanings: `COUNT(CASE WHEN status='paid' THEN 1 END)`, `COUNT(CASE WHEN status='paid' THEN 1 ELSE 0 END)` and `SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END)`. Which are correct for 'number of paid orders'?

```
SELECT COUNT(CASE WHEN status = 'paid' THEN 1 END)        AS count_case_no_else,
       COUNT(CASE WHEN status = 'paid' THEN 1 ELSE 0 END)  AS count_case_else_0_WRONG,
       SUM(CASE WHEN status = 'paid' THEN 1 ELSE 0 END)    AS sum_case_else_0
FROM orders;
```

```
count_case_no_else  count_case_else_0_WRONG  sum_case_else_0
------------------  -----------------------  ---------------
12                  14                       12
```

Step 1	Without `ELSE`, a non-matching row yields NULL, and `COUNT` skips NULL: only the 12 paid rows are counted.
Step 2	With `ELSE 0` inside `COUNT`, the zero is a real value, not NULL, so every one of the 14 rows is counted. The result is simply `COUNT(*)` in disguise.
Step 3	`SUM` with `ELSE 0` adds 1 for paid and 0 for the rest: 12.
Answer	The first and third are correct (12). The second is a bug (14).
Rule	Pair `COUNT` with no `ELSE` (or `ELSE NULL`), and `SUM` with `ELSE 0`.

COMMON MISTAKE	`COUNT(CASE WHEN cond THEN 1 ELSE 0 END)`. It counts every row because 0 is not NULL. Use `COUNT(CASE WHEN cond THEN 1 END)`, `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` or `COUNT(*) FILTER (WHERE cond)`.

Problem 3.9.3 (medium) The same `ELSE` choice changes an *average*. For customers 3 and 5, compute the average paid-order value two ways: with non-paid orders ignored (no ELSE) and with non-paid orders counted as 0 (ELSE 0). Predict: customer 5 has orders of 360 (paid), 900 (refunded) and 400 (paid).

```
WITH order_totals AS (
  SELECT order_id, SUM(qty * unit_price) AS total FROM order_items GROUP BY order_id
)
SELECT o.customer_id,
       ROUND(AVG(CASE WHEN o.status = 'paid' THEN t.total END), 1)         AS aov_paid_only,
       ROUND(AVG(CASE WHEN o.status = 'paid' THEN t.total ELSE 0 END), 1)  AS aov_else_zero
FROM orders o JOIN order_totals t ON t.order_id = o.order_id
WHERE o.customer_id IN (3, 5)
GROUP BY o.customer_id
ORDER BY o.customer_id;
```

```
customer_id  aov_paid_only  aov_else_zero
-----------  -------------  -------------
3            NULL           0.0
5            380.0          253.3
```

Step 1	Order totals are needed first, one per order. A CTE (Chapter 5) computes them.
Step 2	Without ELSE: customer 5 has two paid orders, (360 + 400) / 2 = 380.0. Customer 3 has no paid order, so every CASE value is NULL and `AVG` of nothing is NULL.
Step 3	With ELSE 0: customer 5 averages over all three orders, (360 + 0 + 400) / 3 = 253.3. Customer 3 becomes (0) / 1 = 0.0.
Answer	380.0 versus 253.3 for customer 5; NULL versus 0.0 for customer 3.
Meaning	'Average paid order value' wants no ELSE. 'Revenue per order placed, counting failures as zero' wants ELSE 0. State which one you mean in a column name.

COMMON MISTAKE	Writing `AVG(CASE WHEN cond THEN x ELSE 0 END)` when you wanted the average of the matching rows only. The zeros enter the denominator and drag the average down. Remove the `ELSE` to skip non-matching rows.

Pivots: turning values into columns

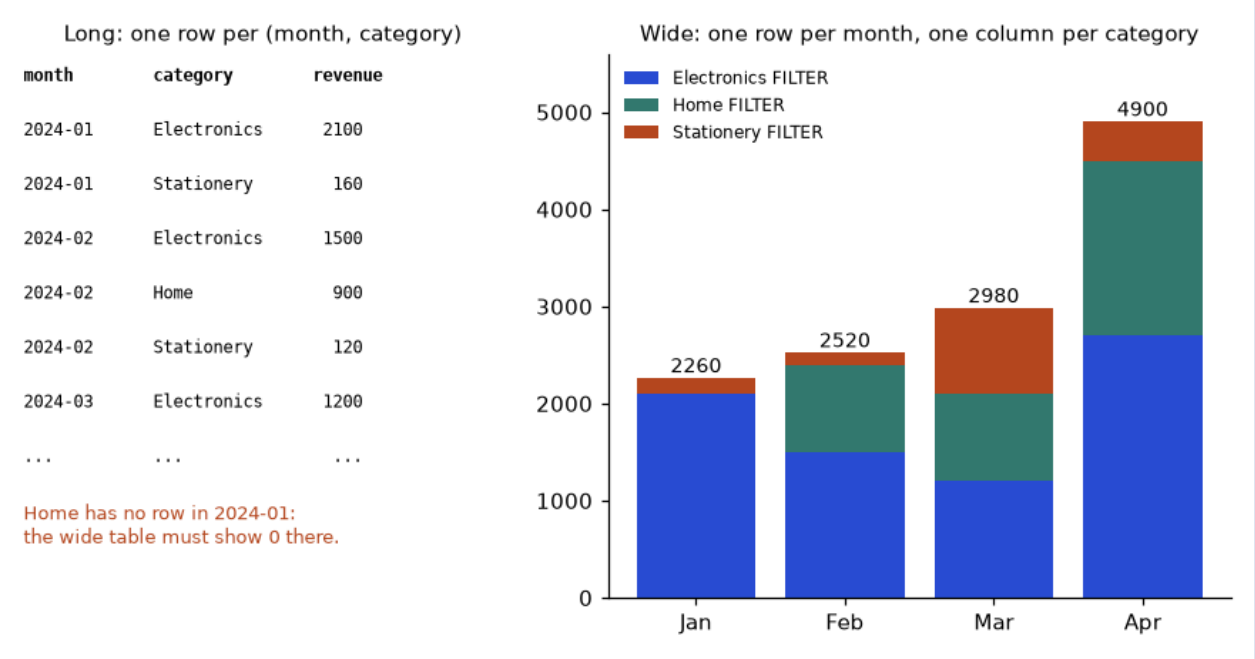
Problem 3.9.4 (medium) A month-by-category revenue grid of paid orders: one row per month, one column per category, plus a total. Predict: January has no Home sale. What will that cell show without extra care?

```
SELECT substr(o.order_date,1,7) AS month,
       COALESCE(SUM(oi.qty*oi.unit_price) FILTER (WHERE p.category = 'Electronics'), 0) AS electronics,
       COALESCE(SUM(oi.qty*oi.unit_price) FILTER (WHERE p.category = 'Home'), 0)        AS home,
       COALESCE(SUM(oi.qty*oi.unit_price) FILTER (WHERE p.category = 'Stationery'), 0)  AS stationery,
       SUM(oi.qty*oi.unit_price) AS total
FROM orders o
JOIN order_items oi ON oi.order_id  = o.order_id
JOIN products    p  ON p.product_id = oi.product_id
WHERE o.status = 'paid'
GROUP BY month
ORDER BY month;
```

```
month    electronics  home  stationery  total
-------  -----------  ----  ----------  -----
2024-01  2100         0     160         2260
2024-02  1500         900   120         2520
2024-03  1200         900   880         2980
2024-04  2700         1800  400         4900
```

```
SELECT substr(o.order_date,1,7) AS month,
       SUM(oi.qty*oi.unit_price) FILTER (WHERE p.category = 'Home') AS home_raw
FROM orders o
JOIN order_items oi ON oi.order_id  = o.order_id
JOIN products    p  ON p.product_id = oi.product_id
WHERE o.status = 'paid' AND substr(o.order_date,1,7) = '2024-01'
GROUP BY month;
```

```
month    home_raw
-------  --------
2024-01  NULL
```



Long format (one row per month and category) turned into wide format (one row per month) with FILTER, drawn as stacked bars.

How to read it: the left side is the output of Problem 3.7.2: many rows, one number each. The right side is the pivoted result: four rows, each month a stacked bar whose three parts are the category columns, and whose total is the printed total column (2260, 2520, 2980, 4900). The pivot does not create information; it rearranges it so a person can read across.

Step 1	Group by month only: grain is one row per month.
Step 2	Each category column is `SUM(line total) FILTER (WHERE category = ...)`, a sum over only that category's lines in the month.
Step 3	In January there is no Home line, so the filtered `SUM` sees no rows and returns NULL (the second output shows `home_raw` = NULL). `COALESCE(..., 0)` turns it into 0 for the grid.
Step 4	The unfiltered `SUM` is the total column.
Check	In each row the three categories add to the total: Jan 2100 + 0 + 160 = 2260; Feb 1500 + 900 + 120 = 2520; Mar 1200 + 900 + 880 = 2980; Apr 2700 + 1800 + 400 = 4900. The totals add to 12660.
Meaning	The column list is fixed in the query. A new category needs a new column. For a truly dynamic pivot you need your BI tool or dynamic SQL (Chapter 7).

Shares, buckets and first/last

Problem 3.9.5 (medium) The share of each category in paid revenue, as a percentage. The denominator is the grand total, a number from a different grain, so we compute it in a scalar subquery.

```
SELECT p.category,
       SUM(oi.qty * oi.unit_price) AS revenue,
       ROUND(100.0 * SUM(oi.qty * oi.unit_price) /
             (SELECT SUM(oi2.qty * oi2.unit_price)
              FROM order_items oi2 JOIN orders o2 ON o2.order_id = oi2.order_id
              WHERE o2.status = 'paid'), 1) AS pct_of_paid
FROM orders o
JOIN order_items oi ON oi.order_id  = o.order_id
JOIN products    p  ON p.product_id = oi.product_id
WHERE o.status = 'paid'
GROUP BY p.category
ORDER BY revenue DESC;
```

```
category     revenue  pct_of_paid
-----------  -------  -----------
Electronics  7500     59.2
Home         3600     28.4
Stationery   1560     12.3
```

Step 1	Numerator: paid revenue per category: 7500, 3600, 1560.
Step 2	Denominator: total paid revenue 12660, computed once by the subquery (it has no `GROUP BY`, so it returns one number).
Step 3	`100.0 * 7500 / 12660 = 59.2`; `100.0 * 3600 / 12660 = 28.4`; `100.0 * 1560 / 12660 = 12.3`.
Check	59.2 + 28.4 + 12.3 = 99.9: the shares add to 100 only up to rounding. Always say 'rounded'.
Meaning	Electronics makes about 59 percent of paid revenue. A share is a ratio of an aggregate at one grain to an aggregate at a coarser grain; Chapter 8 shows `SUM() OVER ()` as a neater way to get that denominator.

Problem 3.9.6 (medium) `CASE` as a grouping key (bucketing). Bucket paid orders by size: small under 500, medium 500 to 1500, large above 1500, with order counts and revenue.

```
WITH order_totals AS (
  SELECT order_id, SUM(qty * unit_price) AS total FROM order_items GROUP BY order_id
)
SELECT CASE WHEN t.total <  500  THEN '1 small  (< 500)'
            WHEN t.total <= 1500 THEN '2 medium (500-1500)'
            ELSE                      '3 large  (> 1500)' END AS size_band,
       COUNT(*)       AS n_orders,
       SUM(t.total)   AS revenue
FROM orders o JOIN order_totals t ON t.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY size_band
ORDER BY size_band;
```

```
size_band           n_orders  revenue
------------------  --------  -------
1 small  (< 500)    3         1200
2 medium (500-1500) 6         5900
3 large  (> 1500)   3         5560
```

Step 1	Order totals of paid orders: 1660, 600, 1020, 1500, 1280, 360, 900, 440, 600, 1800, 400, 2100.
Step 2	The `CASE` turns each total into a label. Small (< 500): 360, 440, 400 = 3 orders, revenue 1200. Medium (500 to 1500): 600, 1020, 1500, 1280, 900, 600 = 6 orders, revenue 5900. Large (> 1500): 1660, 1800, 2100 = 3 orders, revenue 5560.
Step 3	`GROUP BY size_band` groups by the label.
Check	3 + 6 + 3 = 12 paid orders; 1200 + 5900 + 5560 = 12660 revenue.
Meaning	Only 3 of 12 orders are large, but they bring 44 percent of revenue (5560 / 12660). Bucketing is how histograms and segmentation are made in SQL.

Problem 3.9.7 (medium) First paid order, last refund, counts of refunds and cancellations per customer in one row. Predict: most customers have no refund, so `MAX` of nothing is NULL.

```
SELECT customer_id,
       MIN(order_date) FILTER (WHERE status = 'paid')     AS first_paid,
       MAX(order_date) FILTER (WHERE status = 'refunded') AS last_refund,
       COUNT(*) FILTER (WHERE status = 'refunded')        AS refunds,
       COUNT(*) FILTER (WHERE status = 'cancelled')       AS cancels
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

```
customer_id  first_paid  last_refund  refunds  cancels
-----------  ----------  -----------  -------  -------
1            2024-01-10  NULL         0        0
2            2024-01-15  NULL         0        0
3            NULL        NULL         0        1
4            2024-02-20  NULL         0        0
5            2024-03-07  2024-03-25   1        0
6            2024-03-30  NULL         0        0
```

Step 1	Each aggregate gets its own filter, so one group scan yields all four columns.
Step 2	`MIN(order_date)` over only paid rows is the first purchase date. Customer 3 has no paid order: NULL.
Step 3	Only customer 5 has a refund (2024-03-25). Everyone else has NULL there.
Answer	The table above. NULL means 'no such rows', which is different from 0 in a count column.
Check	Both count columns are consistent with Problem 3.9.1.

Rates, ratios and the divide-by-zero guard

Problem 3.9.8 (medium) What happens with division by zero? SQLite returns NULL; PostgreSQL and SQL Server raise an error. Write a portable ratio of refunds per cancellation for customers 3 and 5.

```
SELECT 1.0 / 0 AS sqlite_div_zero,
       1.0 / NULLIF(0, 0) AS nullif_form;
```

```
sqlite_div_zero  nullif_form
---------------  -----------
NULL             NULL
```

```
SELECT customer_id,
       COUNT(*) FILTER (WHERE status = 'refunded')  AS refunds,
       COUNT(*) FILTER (WHERE status = 'cancelled') AS cancels,
       1.0 * COUNT(*) FILTER (WHERE status = 'refunded')
           / NULLIF(COUNT(*) FILTER (WHERE status = 'cancelled'), 0) AS refunds_per_cancel
FROM orders
WHERE customer_id IN (3, 5)
GROUP BY customer_id
ORDER BY customer_id;
```

```
customer_id  refunds  cancels  refunds_per_cancel
-----------  -------  -------  ------------------
3            0        1        0.0
5            1        0        NULL
```

Step 1	Customer 3 has 0 refunds and 1 cancellation: ratio 0.0 / 1 = 0.0.
Step 2	Customer 5 has 1 refund and 0 cancellations. Dividing by 0 would be an error in PostgreSQL.
Step 3	`NULLIF(x, 0)` returns NULL when x equals 0, otherwise x. Dividing by NULL gives NULL (unknown), never an error.
Answer	Customer 3: 0.0; customer 5: NULL (undefined, not infinity and not 0).
Meaning	Reports should show 'n/a' for undefined ratios, not fake zeros that average into other numbers.

Problem 3.9.9 (realistic) A funnel. Of the 8 customers who signed up, how many ordered at least once, at least twice and at least three times? Then the conversion percentages. Predict: customers 7 and 8 never ordered.

```
WITH per_customer AS (
  SELECT c.customer_id, COUNT(o.order_id) AS n_orders
  FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
  GROUP BY c.customer_id
)
SELECT COUNT(*)                              AS signed_up,
       COUNT(*) FILTER (WHERE n_orders >= 1)   AS ordered_once,
       COUNT(*) FILTER (WHERE n_orders >= 2)   AS ordered_twice,
       COUNT(*) FILTER (WHERE n_orders >= 3)   AS ordered_3_plus
FROM per_customer;
```

```
signed_up  ordered_once  ordered_twice  ordered_3_plus
---------  ------------  -------------  --------------
8          6             4              3
```

```
WITH per_customer AS (
  SELECT c.customer_id, COUNT(o.order_id) AS n_orders
  FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
  GROUP BY c.customer_id
)
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE n_orders >= 1) / COUNT(*), 1) AS pct_ordered,
       ROUND(100.0 * COUNT(*) FILTER (WHERE n_orders >= 2)
                   / NULLIF(COUNT(*) FILTER (WHERE n_orders >= 1), 0), 1) AS pct_repeat_of_buyers
FROM per_customer;
```

```
pct_ordered  pct_repeat_of_buyers
-----------  --------------------
75.0         66.7
```

Step 1	Inner query: one row per customer with their order count. `LEFT JOIN` keeps customers 7 and 8 with 0 (the `COUNT(o.order_id)` trick from Problem 3.7.1).
Step 2	Outer query: one pile (no GROUP BY) with four counts, each seeing a different subset.
Step 3	Orders per customer are 4, 3, 1, 2, 3, 1, 0, 0. At least one: 6. At least two: 4 (customers 1, 2, 4, 5). At least three: 3 (customers 1, 2, 5).
Step 4	6 / 8 = 75.0 percent ordered. Repeat buyers as a share of buyers: 4 / 6 = 66.7 percent.
Check	The counts never increase down the funnel: 8, 6, 4, 3.
Meaning	Choose the denominator on purpose: 'of all sign-ups' (50 percent repeat) or 'of buyers' (66.7). Different denominators give different headlines; write which one you used.

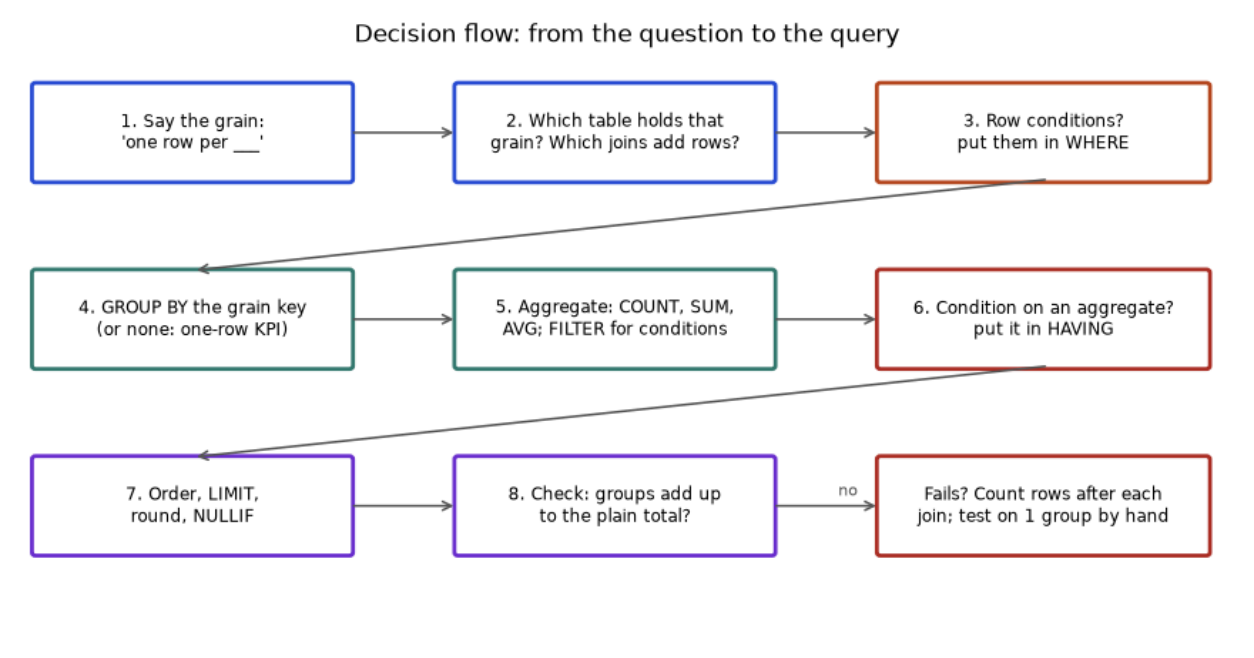
COMMON MISTAKE	Forgetting that a filtered `SUM` with no matching rows is NULL, not 0, and that NULL added to a number gives NULL: `jan_home + jan_stationery` would be NULL in the pivot. Wrap each filtered sum in `COALESCE(..., 0)` if you add them, or use the unfiltered total column.

Check yourself. (1) Rewrite `SUM(x) FILTER (WHERE y > 3)` without FILTER. Answer: `SUM(CASE WHEN y > 3 THEN x END)`. (2) What does `AVG(CASE WHEN c THEN x ELSE 0 END)` compute? Answer: the average over all rows with non-matching rows counted as zero. (3) How many columns can a pivot have? Answer: as many as you write; the list is fixed in the SQL text.

3.10 How to solve any aggregation, group by, having and filter problem

Knowing the clauses is not the same as being able to solve a problem you have never seen. This section is a repeatable method. It has five parts: a decision flow, a table that translates words in the question into SQL, eight recurring patterns each with a fully solved example, a way to check any answer, and a way to debug when the answer is wrong or the query will not run.

Part 1: the decision flow



The eight-step decision flow for any grouped query.

How to read it: start at box 1 and follow the arrows. Boxes 1 and 2 decide *what a row of the answer means* and which table can supply it. Boxes 3 to 6 follow the same order as the database's own pipeline from Section 3.5. Boxes 7 and 8 are the finish: format, then prove it. If the check in box 8 fails, the red box tells you where to look first.

1. **Say the grain out loud.** Finish the sentence 'one row per ___'. If you cannot, you do not yet understand the question. Example: 'revenue by month and category' means one row per month per category.
2. **Find the table whose rows already have that grain,** or decide which joins you need. For each join to a 'many' side, note that the row count grows (Section 3.7).
3. **Row conditions go to `WHERE`:** 'only paid orders', 'in 2024', 'in Pune'. If the condition mentions no count, sum or average, it is a row condition.
4. **`GROUP BY` the grain key.** If the question has no 'per', 'each' or 'by', there is no GROUP BY: it is a single-row KPI.
5. **Choose the aggregates,** using `FILTER` or `CASE` when one column of the answer needs only some of the rows.
6. **A condition on an aggregate goes to `HAVING`:** 'more than 3 orders', 'revenue above average'.
7. **Order and shape:** `ORDER BY`, `LIMIT`, `ROUND`, readable aliases, `NULLIF` for divisions.
8. **Prove it** with the checks in Part 4.

Part 2: translate the words

The question says	Think	SQL ingredient
per, each, by, for every	one pile per value	`GROUP BY key`
how many orders / rows	count rows of that grain	`COUNT(*)` or `COUNT(DISTINCT order_id)` after joins
how many different customers	count unique things	`COUNT(DISTINCT customer_id)`

The question says	Think	SQL ingredient
total, revenue, units	add up	`SUM(...)`
average	sum over count; beware weights	`AVG`, or `SUM(x)/COUNT(...)` over the right rows
only paid, in 2024, in Pune	a condition on single rows	`WHERE`
more than 3, at least 2, above 1500 (about a total or count)	a condition on a pile	`HAVING`
split into, side by side, paid vs refunded columns	several filtered aggregates	`FILTER / CASE`
rate, share, percent, conversion	one aggregate divided by another	`100.0 * a / NULLIF(b, 0)`
top, best, highest, most	sort the groups and take some	`ORDER BY agg DESC LIMIT n` (or join back to the max)
never, none, without, no orders	a count of zero or a missing match	`LEFT JOIN` + `COUNT(right.id) = 0`, or `NOT EXISTS`
duplicates, appears twice	a pile bigger than one	`GROUP BY key HAVING COUNT(*) > 1`
above the average, compared with all	compare a pile with a number from the whole set	subquery / CTE with the overall aggregate

Part 3: the eight recurring patterns, each solved

Almost every aggregation question in analysis work and interviews is one of the eight shapes below, or a combination of two. For each shape: recognise it, apply the recipe, check the result.

Pattern 1: the single KPI card (aggregates with no GROUP BY)

Problem 3.10.1 The shop owner wants one row of headline numbers: orders, distinct buyers, paid orders, percent of orders with a coupon, and the first and last order dates.

```
SELECT COUNT(*)                                    AS orders,
       COUNT(DISTINCT customer_id)                 AS buyers,
       COUNT(*) FILTER (WHERE status = 'paid')     AS paid_orders,
       ROUND(100.0 * COUNT(coupon) / COUNT(*), 1)  AS pct_orders_with_coupon,
       MIN(order_date)                             AS first_day,
       MAX(order_date)                             AS last_day
FROM orders;
```

```
orders  buyers  paid_orders  pct_orders_with_coupon  first_day   last_day
------  ------  -----------  ----------------------  ----------  ----------
14      6       12           21.4                    2024-01-10  2024-04-20
```

Recognise	No 'per' in the question: the whole table is one pile. Grain: one row total.
Recipe	Several aggregates in one `SELECT`, no `GROUP BY`. Use `FILTER` for the paid orders. Use `COUNT(DISTINCT ...)` for people.
Compute	14 orders, 6 buyers, 12 paid. Coupons on 3 of 14 orders: 100 * 3 / 14 = 21.4. Dates 2024-01-10 to 2024-04-20.
Check	12 paid out of 14 is correct against Problem 3.2.1 (12 paid, 1 cancelled, 1 refunded). Percentages lie between 0 and 100.

Pattern 2: one metric per group, including empty groups

Problem 3.10.2 Units sold for every product, listing products that never sold with 0. Predict: the Mug has no order lines.

```
SELECT p.name,
       COALESCE(SUM(oi.qty), 0) AS units_sold
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.product_id
GROUP BY p.product_id, p.name
ORDER BY units_sold DESC, p.name;
```

```
name       units_sold
---------  ----------
Notebook   14
Pen Pack   7
Mouse      5
Desk Lamp  4
Keyboard   4
Mug        0
```

Recognise	'For every product' and 'including those never sold': the list of groups comes from the products table, not from order_items.
Recipe	Start `FROM products`, `LEFT JOIN` the detail, group by the product key, and wrap a `SUM` in `COALESCE(..., 0)` (or count a right-side column).
Compute	Notebook 14, Pen Pack 7, Mouse 5, Keyboard 4, Desk Lamp 4, Mug 0. Total 34.
Check	34 equals the total units from Problem 3.1.1. Six products in, six rows out: the Mug survived because the join started from products.

Pattern 3: keep only groups above a threshold (HAVING)

Problem 3.10.3 Which cities have at least 3 paid orders?

```
SELECT c.city, COUNT(*) AS paid_orders
FROM orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'paid'
GROUP BY c.city
HAVING COUNT(*) >= 3
ORDER BY paid_orders DESC, c.city;
```

```
city    paid_orders
------  -----------
Mumbai  4
Pune    4
```

Recognise	'At least 3' of something counted per city: a condition on a pile.
Recipe	`WHERE` status is paid; group by city; `HAVING COUNT(*) >= 3`. Here `orders` is joined only to customers, so one row is one order and `COUNT(*)` is safe.
Compute	Mumbai 4 (orders 102, 108, 110, 114) and Pune 4 (101, 103, 106, 111). Delhi has 2 and Bengaluru 2.
Check	The cities left out total 4 paid orders, and 8 + 4 = 12.

Pattern 4: rate or share per group

Problem 3.10.4 For each month, the share of paid orders that used a coupon.

```
SELECT substr(order_date,1,7) AS month,
       COUNT(*) AS paid_orders,
       COUNT(coupon) AS with_coupon,
       ROUND(100.0 * COUNT(coupon) / COUNT(*), 1) AS coupon_pct
FROM orders
WHERE status = 'paid'
GROUP BY month
ORDER BY month;
```

```
month    paid_orders  with_coupon  coupon_pct
-------  -----------  -----------  ----------
2024-01  2            1            50.0
2024-02  2            1            50.0
2024-03  4            1            25.0
2024-04  4            0            0.0
```

Recognise	'Share' = part / whole inside each group.
Recipe	Numerator `COUNT(coupon)` (counts only non-NULL coupons), denominator `COUNT(*)`, multiply by `100.0` to avoid integer division. Both are over the same filtered rows.
Compute	January: 2 paid orders, 1 with coupon (102): 50.0. February: 2 paid, 1 coupon (105): 50.0. March: 4 paid, 1 coupon (110): 25.0. April: 4 paid, none: 0.0.
Check	Total coupons across months: 1 + 1 + 1 + 0 = 3, equal to the 3 coupon orders, all of which are paid.

Pattern 5: the top group, and the row that holds the maximum

Problem 3.10.5 (a) Which product earns the most revenue? (b) Which product is the dearest in each category? Part (b) is harder: you need the whole row (the name), not just the maximum.

```
SELECT p.name, SUM(oi.qty * oi.unit_price) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY revenue DESC
LIMIT 1;
```

```
name      revenue
--------  -------
Keyboard  6000
```

```
SELECT p.category, p.name, p.price
FROM products p
JOIN (SELECT category, MAX(price) AS top_price FROM products GROUP BY category) m
  ON m.category = p.category AND m.top_price = p.price
ORDER BY p.category;
```

```
category     name       price
-----------  ---------  -----
Electronics  Keyboard   1500
Home         Desk Lamp  900
Stationery   Pen Pack   120
```

```
SELECT category, name, price FROM (
  SELECT category, name, price,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY price DESC, name) AS rn
  FROM products)
WHERE rn = 1
ORDER BY category;
```

```
category     name       price
-----------  ---------  -----
Electronics  Keyboard   1500
Home         Desk Lamp  900
Stationery   Pen Pack   120
```

Recognise (a)	'Top one' by an aggregate: group, aggregate, sort descending, `LIMIT 1`. Danger: if two groups tie for first, `LIMIT 1` hides one. Add a tie-breaker in `ORDER BY` or use `RANK` (Chapter 8) if ties must all show.
Recipe (b)	A bare `name` next to `MAX(price)` is not portable (Section 3.7). Two safe ways: join back to the per-group maximum (first output), or number the rows within each category with a window function and keep number 1 (second output).
Compute	(a) Keyboard, 6000. (b) Electronics: Keyboard 1500; Home: Desk Lamp 900; Stationery: Pen Pack 120.
Check	Both methods in (b) agree. The join version returns every tied product if two share the maximum; the `ROW_NUMBER` version returns exactly one per group (name order breaks ties). Pick on purpose.

Pattern 6: duplicates and uniqueness

Problem 3.10.6 A contacts table has repeated emails. List each email that appears more than once with its number of copies, and say how many rows a de-duplication would delete.

```
CREATE TABLE contacts (id INTEGER, email TEXT);
INSERT INTO contacts VALUES (1,'a@x.com'),(2,'b@x.com'),(3,'a@x.com'),(4,'c@x.com'),(5,'b@x.com'),(6,'a@x.com');
SELECT email, COUNT(*) AS copies, MIN(id) AS keep_id
FROM contacts
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY email;
```

```
email    copies  keep_id
-------  ------  -------
a@x.com  3       1
b@x.com  2       2
```

```
SELECT COUNT(*) AS rows_total,
       COUNT(DISTINCT email) AS unique_emails,
       COUNT(*) - COUNT(DISTINCT email) AS rows_to_delete
FROM contacts;
```

```
rows_total  unique_emails  rows_to_delete
----------  -------------  --------------
6           3              3
```

Recognise	'Appears more than once' means a pile with more than one row.
Recipe	`GROUP BY email HAVING COUNT(*) > 1`. `MIN(id)` gives a stable row to keep.
Compute	a@x.com appears 3 times (ids 1, 3, 6), b@x.com 2 times (2, 5). c@x.com is unique and is not listed. Rows 6; unique emails 3; rows to delete 6 - 3 = 3 (2 copies of a, 1 of b).
Check	2 + 1 = 3 extra rows. The identity `COUNT(*) − COUNT(DISTINCT key)` = number of surplus rows is a one-line duplicate detector.

Pattern 7: distinct people across a join

Problem 3.10.7 For each category, how many different customers bought it, and what percent of all buyers is that?

```
SELECT p.category,
       COUNT(DISTINCT o.customer_id) AS buyers,
       ROUND(100.0 * COUNT(DISTINCT o.customer_id) /
             (SELECT COUNT(DISTINCT customer_id) FROM orders), 1) AS pct_of_buyers
FROM orders o
JOIN order_items oi ON oi.order_id  = o.order_id
JOIN products    p  ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY buyers DESC, p.category;
```

```
category     buyers  pct_of_buyers
-----------  ------  -------------
Electronics  4       66.7
Stationery   4       66.7
Home         3       50.0
```

Recognise	'How many different customers' after a join to a many-side table: fan-out danger, so `COUNT(DISTINCT customer_id)`.
Recipe	Numerator by category; denominator is the number of distinct buyers overall, a scalar subquery (6).
Compute	Electronics 4 of 6 = 66.7; Stationery 4 of 6 = 66.7; Home 3 of 6 = 50.0.
Check	Percentages add to more than 100 (about 183) because a customer can buy from several categories. That is not an error; it tells you the categories overlap.

Pattern 8: compare a group with the overall average

Problem 3.10.8 Which months had paid revenue above the average month?

```
WITH monthly AS (
  SELECT substr(o.order_date,1,7) AS month, SUM(oi.qty * oi.unit_price) AS revenue
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY month
)
SELECT month, revenue, (SELECT AVG(revenue) FROM monthly) AS avg_month
FROM monthly
WHERE revenue > (SELECT AVG(revenue) FROM monthly)
ORDER BY month;
```

```
month    revenue  avg_month
-------  -------  ---------
2024-04  4900     3165.0
```

Recognise	'Above the average': a pile is compared with a number from all piles.
Recipe	Compute the monthly totals once in a CTE, then compare each with the average of the CTE.
Compute	Monthly revenue 2260, 2520, 2980, 4900; average (2260 + 2520 + 2980 + 4900) / 4 = 12660 / 4 = 3165. Only April (4900) is above.
Check	Three months are below 3165. The big April pulls the average up, so even 2980 is below it. For growth month over month you need `LAG` (Chapter 8).

Part 4: how to check an answer

Run at least three of these checks on every query before you trust it.

1. **Partition check.** Group counts add up to the table count; group sums add up to the plain total (when no filter hides rows).
2. **Grain check.** Compare `COUNT(*)` with `COUNT(DISTINCT key)` on the joined rows. If they differ and you did not expect it, a join multiplied rows.
3. **Hand-compute one group.** Pick the smallest group, add it up with a pencil from the raw rows, and compare.
4. **Boundary cases.** What if a group is empty, a value is NULL, two groups tie, the denominator is zero?
5. **Second method.** Compute the same number a different way (`FILTER` versus `CASE` , subquery versus join). Two methods that agree are rarely both wrong.
6. **Plausibility.** Shares between 0 and 100? Averages between min and max? A count larger than the table is impossible.

Problem 3.10.9 Write the partition check for 'revenue per product' as a single query that returns the sum of the groups, the plain total, the number of groups and the number of distinct keys.

```
WITH by_product AS (
  SELECT product_id, SUM(qty * unit_price) AS revenue FROM order_items GROUP BY product_id
)
SELECT (SELECT SUM(revenue) FROM by_product)           AS sum_of_groups,
       (SELECT SUM(qty * unit_price) FROM order_items) AS plain_total,
       (SELECT COUNT(*) FROM by_product)               AS n_groups,
       (SELECT COUNT(DISTINCT product_id) FROM order_items) AS n_distinct_keys;
```

```
sum_of_groups  plain_total  n_groups  n_distinct_keys
-------------  -----------  --------  ---------------
14560          14560        5         5
```

Setup	A CTE holds the per-product revenue (the thing you want to verify).
Compute	The sum of groups, 14560, equals the plain total 14560, and the group count 5 equals the number of distinct product ids in order_items.
Answer	Both pairs agree, so no row was lost or counted twice.
Habit	If the pairs disagree, the first suspect is a join (rows lost by INNER JOIN, rows repeated by one-to-many) or a WHERE that dropped rows.

Part 5: how to debug when you are stuck

Symptom	Most likely cause	First test
Numbers too big, counts larger than the table	fan-out from a one-to-many join	`SELECT COUNT(*), COUNT(DISTINCT key)` after each join
A group you expected is missing	INNER JOIN or WHERE removed it; or it has no rows	switch to LEFT JOIN from the table that lists the groups; test the WHERE on its own
Zero shows as 1 in a count	`COUNT(*)` after LEFT JOIN	use `COUNT(right.id)`

Symptom	Most likely cause	First test
Result NULL where you expected 0	`SUM` / `MAX` of an empty or all-NULL pile	wrap in `COALESCE` or decide NULL is right
Average looks too low	NULLs counted as zero, or `ELSE 0` in a CASE	compare `COUNT(*)` with `COUNT(col)`
Percent is 0 or 100 only	integer division	multiply by `100.0` first
Error: column must appear in GROUP BY	a plain column neither grouped nor aggregated	add it to GROUP BY or aggregate it
Error: misuse of aggregate	aggregate in WHERE	move that test to HAVING
Totals do not match another report	different filter (cancelled/refunded), time zone, or duplicates	reproduce their filter exactly, then diff by group
Query slow	grouping a huge joined set	filter earlier with WHERE, aggregate before joining, see Chapter 10

A reliable debugging routine: (1) shrink the problem: filter to one customer or one month where you can verify by hand; (2) remove clauses from the end (`HAVING` , then `GROUP BY`) and look at the raw rows; (3) print the row count after each join; (4) add clauses back one at a time.

Problem 3.10.10 (debug) A teammate wants customers who never had a paid order and writes `LEFT JOIN orders ... WHERE o.status <> 'paid'`. Find what is wrong and fix it. The correct answer is Meena (only a cancelled order), Sana and Tom (no orders).

```
SELECT c.name
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status <> 'paid';
```

```
name
-----
Meena
Priya
```

```
SELECT c.customer_id, c.name,
       COUNT(o.order_id) AS orders,
       COUNT(*) FILTER (WHERE o.status = 'paid') AS paid
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(*) FILTER (WHERE o.status = 'paid') = 0
ORDER BY c.customer_id;
```

```
customer_id  name   orders  paid
-----------  -----  ------  ----
3            Meena  1       0
7            Sana   0       0
8            Tom    0       0
```

Symptom	The first query returns Meena and Priya (Priya's refunded order matched the test), but misses Sana and Tom.
Cause 1	Priya has paid orders too. Row-level `WHERE` sees her refunded order alone and keeps it. 'Never paid' is a statement about a whole pile, so it needs a group.
Cause 2	Sana and Tom have NULL status after the LEFT JOIN, and `NULL <> 'paid'` is unknown, so `WHERE` drops them.
Fix	Group by customer and demand a paid count of 0: `HAVING COUNT(*) FILTER (WHERE o.status = 'paid') = 0`. The filtered count is 0 (not NULL) for customers with no paid rows and for those with no rows at all.
Answer	Meena (1 order, 0 paid), Sana (0, 0), Tom (0, 0).
Lesson	'Never' and 'none' questions are pile questions. A condition on rows cannot say 'no row in the pile satisfies this'.

COMMON MISTAKE

Answering a 'never' question with `WHERE status <> 'paid'`. It finds customers who have *some* non-paid order, which is a different question. 'No paid order' is: group by customer, count the paid rows, keep groups where that count is 0 (or use `NOT EXISTS`, Chapter 5).

KEY IDEA

Method beats memory. Say the grain, put row conditions in WHERE, group by the key, aggregate, put pile conditions in HAVING, then reconcile with a plain total. Eight patterns cover almost everything: KPI card, metric per group, threshold groups, rate per group, top row, duplicates, distinct people, compare with overall.

3.11 Going Deeper: Internals, Edge Cases, Performance and What Interviewers Probe

This part is for when the basics feel solid. You do not need it to write correct everyday queries, but each idea here removes a class of surprises, and interviewers love them.

How the database really groups

The pipeline in Section 3.5 is *logical*: the order in which the meaning of the query is defined. The engine is free to do the work differently as long as the answer is the same. Two physical strategies do the grouping:

- **Hash aggregate.** Read the rows once; keep a hash table from key to running aggregate (for example key 'paid' to running count). Memory is proportional to the number of *groups*, not rows. Fast when there are few groups.
- **Sort (stream) aggregate.** Sort the rows by key (or read them from an index that is already sorted), then walk through: when the key changes, emit a row. Needs little memory when the input is already ordered, and delivers sorted output for free.

SQLite shows its choice with `EXPLAIN QUERY PLAN`. Without an index on `status` it builds a temporary sorted structure; with an index it reads the already-ordered index.

Problem 3.11.1 See the grouping strategy. Show the plan for counting orders per status, first without and then with an index on `status`.

```
EXPLAIN QUERY PLAN
SELECT status, COUNT(*) AS n FROM orders GROUP BY status;
```

```
id  parent  notused  detail
--  ------  -------  ----------------------------
6   0       216      SCAN orders
8   0       0        USE TEMP B-TREE FOR GROUP BY
```

```
CREATE INDEX idx_orders_status ON orders(status);
EXPLAIN QUERY PLAN
SELECT status, COUNT(*) AS n FROM orders GROUP BY status;
```

```
id  parent  notused  detail
--  ------  -------  ----------------------------------------------------
6   0       212      SCAN orders USING COVERING INDEX idx_orders_status
```

Step 1	Without an index the engine scans the whole table and uses a temporary B-tree to collect the groups (`USE TEMP B-TREE FOR GROUP BY`).
Step 2	After `CREATE INDEX` on `status` the plan scans the index only (`COVERING INDEX`), because the index already holds the statuses in sorted order. Equal keys sit next to each other, so no temporary structure is needed.
Step 3	On 14 rows the difference is invisible. On 100 million rows the index avoids a big sort or hash table, which is where the speed comes from (Chapter 10).
Answer	The exact wording of plans differs between SQLite versions and between databases; the idea does not.

```
EXPLAIN QUERY PLAN
SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id;
```

```
id  parent  notused  detail
--  ------  -------  ------------------------------------------------------------
9   0       62       SEARCH o USING INDEX idx_orders_status (status=?)
14  0       62       SEARCH oi USING INDEX sqlite_autoindex_order_items_1 (order_id=?)
19  0       0        USE TEMP B-TREE FOR GROUP BY
```

For a grouped join, filter early. The plan above shows the engine can use the primary key of `order_items` to find the lines of each order. Whatever the join method, rows removed by `WHERE` never reach the grouping step, which is why Section 3.3 said to prefer `WHERE`.

NULL: equal for grouping, unknown for comparison

The database has two different notions of 'same'. In a comparison, `NULL = NULL` is unknown. In `GROUP BY`, `DISTINCT` and set operators, NULLs are treated as *not distinct from each other*, so they form one group or one row. And `COUNT(DISTINCT col)` does not count the NULL group at all.

Problem 3.11.2 Show the three NULL behaviours on the coupon column: the NULL group exists in `GROUP BY` and `DISTINCT`, the distinct count ignores it, and `= NULL` matches nothing.

```
SELECT coupon, COUNT(*) FROM orders GROUP BY coupon;
```

```
coupon  COUNT(*)
------  --------
NULL    11
SAVE10  3
```

```
SELECT DISTINCT coupon FROM orders ORDER BY coupon;
```

```
coupon
------
NULL
SAVE10
```

```
SELECT COUNT(*) AS rows_, COUNT(DISTINCT coupon) AS distinct_non_null FROM orders;
```

```
rows_  distinct_non_null
-----  -----------------
14     1
```

```
SELECT COUNT(*) AS equals_null FROM orders WHERE coupon = NULL;
```

```
equals_null
-----------
0
```

Step 1	`GROUP BY coupon` shows a NULL group of 11 rows and a SAVE10 group of 3.
Step 2	`DISTINCT coupon` lists NULL and SAVE10: NULLs collapse into one row.
Step 3	`COUNT(DISTINCT coupon)` is 1: aggregates skip NULL, so the NULL 'value' is not counted.
Step 4	`WHERE coupon = NULL` matches 0 rows. The correct test is `IS NULL`.
Meaning	If you need the NULL group counted as a kind, use `COUNT(DISTINCT COALESCE(coupon, '(none)'))`, which gives 2.

Aggregates can be combined in pieces (and some cannot)

Big engines (Spark, BigQuery, Snowflake, Redshift, ClickHouse) split data across many machines. Each machine computes a *partial* aggregate over its slice and a final step merges them. That works for `SUM`, `COUNT`, `MIN`, `MAX`. It works for `AVG` only if each machine sends `(sum, count)` and the final step divides. It fails for the median and exact distinct counts, which need all the values or a sketch. This is the same reason you cannot average averages.

```
# why AVG cannot be computed from partial AVGs: carry (sum, count) instead
part_a = [100]                # one machine holds 1 row
part_b = [200, 200, 200]      # another holds 3 rows
avg_a = sum(part_a) / len(part_a)
avg_b = sum(part_b) / len(part_b)
print("average of averages:", (avg_a + avg_b) / 2)
s = sum(part_a) + sum(part_b)
n = len(part_a) + len(part_b)
print("sum of sums / sum of counts:", s / n)
```

```
average of averages: 150.0
sum of sums / sum of counts: 175.0
```

The average of averages is 150.0 but the correct answer is 175.0: 100 + 200 + 200 + 200 = 700 over 4 rows. This is Problem 3.P.10 in code. Because exact `COUNT(DISTINCT)` is expensive at scale, warehouses offer approximate versions (`APPROX_COUNT_DISTINCT` in BigQuery, Snowflake and Spark; `approx_distinct` in Trino) that use a sketch called HyperLogLog and are accurate to about one or two percent with tiny memory. Use them when a dashboard needs speed and tolerates a small error; never for billing.

Numbers: floats, money and overflow

Problem 3.11.3 (expert) Floating-point surprises. Check whether 0.1 + 0.2 equals 0.3 in SQL, and how to compare safely.

```
SELECT 0.1 + 0.2 AS sum_float, (0.1 + 0.2) = 0.3 AS equals_point3,
       ROUND(0.1 + 0.2, 2) = 0.3 AS rounded_equals;
```

```
sum_float           equals_point3  rounded_equals
------------------  -------------  --------------
0.30000000000000004  0              1
```

```
total = 0.0
for _ in range(10):
    total += 0.1
print(total, total == 1.0)
cents = sum([10] * 10)      # money in whole cents: exact
print(cents, cents / 100)
```

```
0.9999999999999999 False
100 1.0
```

Step 1	Decimal fractions like 0.1 have no exact binary form, so the sum is 0.30000000000000004 and the equality is false (0).
Step 2	Rounding to 2 decimals first makes the comparison true.
Step 3	Adding 0.1 ten times, one addition at a time, gives 0.9999999999999999, not 1.0. Some engines (and Python's own `sum` in recent versions) compensate for this inside the function, others do not, so do not depend on it.
Step 4	Money should be stored as integer cents or a fixed-point `DECIMAL(p, s)` column. The shop stores whole rupees, so its sums are exact.
Answer	Never test floats for equality in HAVING or WHERE; round, or compare with a small tolerance.

COMMON MISTAKE

Trusting the type of an aggregate. In SQL Server and some others `SUM` of an `INT` column returns an `INT` and can overflow on large tables (cast to `BIGINT` first). In PostgreSQL `SUM` of `INT` returns `BIGINT`, and `AVG` of integers returns a decimal. SQLite raises an 'integer overflow' error when a sum of integers exceeds 64 bits. Know your engine's rule before summing billions.

SQL aggregation versus pandas aggregation

If you move between SQL and pandas, three differences cause silent bugs.

```
import pandas as pd
import numpy as np
marks = pd.DataFrame({
    "student": ["Asha","Asha","Ravi","Ravi","Meena","Meena","John","John"],
    "subject": ["maths","science","maths","science","maths","science","maths","science"],
    "mark":    [78, 85, 62, np.nan, 91, 74, 55, 68],
})
g = marks.groupby("subject")["mark"]
print(g.agg(n_rows="size", n_marks="count", total="sum", avg="mean", lo="min", hi="max"))
print()
print("sum of an all-NaN group:", pd.Series([np.nan, np.nan]).sum())
print("SQL would give NULL; pandas gives 0.0 unless min_count=1:", pd.Series([np.nan, np.nan]).sum(min_count=1))
coupon = pd.DataFrame({"coupon": ["SAVE10", None, None, "SAVE10", None]})
print(coupon.groupby("coupon").size().to_dict())
print(coupon.groupby("coupon", dropna=False).size().to_dict())
```

```
         n_rows  n_marks  total       avg    lo    hi
subject
maths         4        4  286.0  71.500000  55.0  91.0
science       4        3  227.0  75.666667  68.0  85.0

sum of an all-NaN group: 0.0
SQL would give NULL; pandas gives 0.0 unless min_count=1: nan
{'SAVE10': 2}
{'SAVE10': 2, nan: 3}
```

Step 1	`size` counts rows (like `COUNT(*)`), `count` counts non-missing values (like `COUNT(col)`). The science line shows 4 and 3.
Step 2	The sum of an all-missing group is `0.0` in pandas, but NULL in SQL. Pass `min_count=1` to get the SQL behaviour.
Step 3	`groupby` drops missing keys by default, so the coupon-less rows vanish and you see only SAVE10: 2. SQL keeps the NULL group. Pass `dropna=False` to keep it (pandas shows it under key `nan`).
Meaning	When you check a pandas result against SQL, compare these three behaviours first.

Subtotals and grand totals

Reports often want group rows *and* a total row. PostgreSQL, MySQL, SQL Server and Oracle have `GROUP BY ROLLUP (category)` and `GROUPING SETS`, covered in Chapter 7. SQLite has neither, but `UNION ALL` gives the same result: one grouped query stacked on a one-row total query.

Problem 3.11.4 Products and total price per category with a grand total row.

```
SELECT category, COUNT(*) AS n_products, SUM(price) AS total_price
FROM products
GROUP BY category
UNION ALL
SELECT 'ALL CATEGORIES', COUNT(*), SUM(price)
FROM products
ORDER BY 1;
```

```
category        n_products  total_price
--------------  ----------  -----------
ALL CATEGORIES  6           3450
Electronics     2           2100
Home            2           1150
Stationery      2           200
```

Step 1	The first query has grain 'one row per category'; the second has grain 'one row'. `UNION ALL` stacks them because both have three columns of the same kinds.
Step 2	The literal 'ALL CATEGORIES' fills the label column of the total row.
Step 3	`ORDER BY 1` sorts by the first column, which puts 'ALL CATEGORIES' first alphabetically. For a total at the bottom, add a sort column (0 for groups, 1 for the total).
Check	2 + 2 + 2 = 6 products and 3450 total price, equal to the grand total row.

```
-- PostgreSQL / MySQL 8 / SQL Server / Oracle (not run here: SQLite has no ROLLUP)
SELECT category, COUNT(*) AS n_products, SUM(price) AS total_price
FROM products
GROUP BY ROLLUP (category);
```

Expected output in those databases: the three category rows plus one extra row where `category` is NULL, with 6 and 3450. Use `GROUPING(category)` to tell that NULL from a real NULL category.

Aggregate versus window: keep the rows

An aggregate collapses rows. A *window* aggregate (`SUM(...) OVER (...)`) computes the same number but keeps every row, so you can put the group total beside each detail row. Chapter 8 teaches windows in full; here is a preview so you can see the difference.

Problem 3.11.5 Show each line of orders 101 and 103 next to its order total, without collapsing the lines.

```
SELECT oi.order_id, oi.product_id, oi.qty * oi.unit_price AS line_total,
       SUM(oi.qty * oi.unit_price) OVER (PARTITION BY oi.order_id) AS order_total
FROM order_items oi
WHERE oi.order_id IN (101, 103)
ORDER BY oi.order_id, oi.product_id;
```

```
order_id  product_id  line_total  order_total
--------  ----------  ----------  -----------
101       1           1500        1660
101       3           160         1660
103       4           120         1020
103       5           900         1020
```

Step 1	`PARTITION BY order_id` is the window's `GROUP BY`: it makes the piles.
Step 2	But every input row is kept. Order 101's two lines (1500 and 160) both show the order total 1660; order 103's lines (900, 120) both show 1020.
Meaning	Use `GROUP BY` when the answer has one row per group. Use a window when you need detail and group figures side by side (percent of total, rank within group, running totals).

```
SELECT DISTINCT p.category,
       SUM(oi.qty * oi.unit_price) OVER (PARTITION BY p.category) AS category_revenue,
       SUM(oi.qty * oi.unit_price) OVER ()                        AS grand_total
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
ORDER BY p.category;
```

```
category     category_revenue  grand_total
-----------  ----------------  -----------
Electronics  9000              14560
Home         3600              14560
Stationery   1960              14560
```

The same window syntax gives the grand total column without a scalar subquery. `OVER ()` means 'one pile: the whole result'.

Dialect cheat table

Feature	SQLite	PostgreSQL	MySQL	SQL Server
alias in GROUP BY	yes	yes	yes	no
alias in HAVING	yes	no	yes	no
non-grouped column in SELECT	allowed (arbitrary row)	error	error unless ONLY_FULL_GROUP_BY is off (on by default since 5.7)	error
`FILTER (WHERE ...)`	3.30+	9.4+	no (use CASE)	no (use CASE)
integer `/` integer	integer	integer	decimal	integer

Feature	SQLite	PostgreSQL	MySQL	SQL Server
string list aggregate	`group_concat`	`string_agg`	`GROUP_CONCAT`	`STRING_AGG` (2017+)
`ROLLUP`	no	yes	yes (`WITH ROLLUP`)	yes
division by zero	NULL	error	NULL (usually)	error

Treat the table as a guide for what to look up first, not a guarantee: versions and settings change behaviour.

Performance rules of thumb

- Filter with `WHERE` before grouping, and filter on indexed, sargable conditions (Chapter 10).
- Aggregate to the needed grain *before* joining to other large tables (Section 3.7): joining 14 order totals is cheaper and safer than joining 20 lines.
- A covering index on `(status)` or `(customer_id, status)` can answer a grouped count without touching the table.
- `COUNT(*)` , `COUNT(1)` mean the same thing. `COUNT(col)` is not the same as `COUNT(*)` when col has NULLs, regardless of speed.
- `COUNT(DISTINCT x)` needs a sort or hash set of x; on huge data try a sketch or a pre-aggregated table.
- `HAVING` cannot use an index on the aggregate; if a threshold is needed often, store the pre-aggregated table (a summary table or materialised view).

What interviewers probe

Question	Short strong answer
WHERE versus HAVING?	WHERE filters rows before grouping and cannot use aggregates; HAVING filters groups after aggregation.
COUNT(*) versus COUNT(col) versus COUNT(DISTINCT col)?	rows; non-NULL values; different non-NULL values.
What does SUM return on no rows?	NULL, while COUNT returns 0; COALESCE if you need 0.
Why is average of averages wrong?	It weights groups equally regardless of size. Carry sum and count, divide once.
How do you find duplicates?	GROUP BY the key HAVING COUNT(*) > 1; or COUNT(*) - COUNT(DISTINCT key).
How do you pivot without PIVOT?	One FILTER or CASE aggregate per output column.
Your count doubled after a join. Why?	Fan-out from a one-to-many join; aggregate first or COUNT(DISTINCT key).
Order of evaluation of a query?	FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT.
How is NULL handled in GROUP BY?	All NULLs form a single group.
Can you use a SELECT alias in WHERE?	Not in standard SQL (SELECT runs later); some engines allow it, do not rely on it.

INTERVIEW Interviewers often give a small table and ask for a result by hand. Practise the habit from this chapter: write the grain, draw the piles on paper, compute each pile, then write the SQL. The answer you derive by hand is also the answer you check the query against.

3.12 Mixed-Practice Drill

Do these without looking at the answers. For each, write the grain first, then the query, then check it against a total you know (14560 revenue, 12660 paid, 34 units, 20 lines, 14 orders). Compare with the output and the answer below it.

Problem 3.12.1 (warm-up) Which orders have two or more item lines?

```
SELECT order_id, COUNT(*) AS n_lines
FROM order_items
GROUP BY order_id
HAVING COUNT(*) >= 2
ORDER BY order_id;
```

```
order_id  n_lines
--------  -------
101       2
103       2
106       2
110       2
113       2
114       2
```

Answer: Orders 101, 103, 106, 110, 113 and 114, two lines each. Grain: one row per order. The condition counts lines, so it is a pile condition: HAVING.

Problem 3.12.2 Units bought in paid orders, per customer name, largest first.

```
SELECT c.name, SUM(oi.qty) AS units
FROM customers c
JOIN orders o       ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
WHERE o.status = 'paid'
GROUP BY c.customer_id, c.name
ORDER BY units DESC, c.name;
```

```
name   units
-----  -----
Asha   9
Priya  7
Kabir  5
Ravi   4
John   3
```

Answer: Asha 9, Priya 7, Kabir 5, Ravi 4, John 3, total 28. Check: total units are 34; the 6 units in the cancelled and refunded orders (5 notebooks in order 104 and 1 keyboard in order 109) are excluded, and 34 - 6 = 28. Meena has only a cancelled order and so is absent.

Problem 3.12.3 Average line total per category, rounded to 1 decimal.

```
SELECT p.category, ROUND(AVG(oi.qty * oi.unit_price), 1) AS avg_line
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY p.category;
```

```
category     avg_line
-----------  --------
Electronics  1125.0
Home         1200.0
Stationery   217.8
```

Answer: Electronics 1125.0 (9000 / 8 lines), Home 1200.0 (3600 / 3), Stationery 217.8 (1960 / 9). Each is a sum divided by the count of lines in that same pile, so it is a legitimate average.

Problem 3.12.4 Paid orders split by coupon use: number of orders, revenue and average order value. Pre-aggregate lines to orders first.

```
WITH order_totals AS (
  SELECT order_id, SUM(qty * unit_price) AS total FROM order_items GROUP BY order_id
)
SELECT COALESCE(o.coupon, '(none)') AS coupon, COUNT(*) AS n_orders,
       SUM(t.total) AS revenue, ROUND(AVG(t.total), 1) AS aov
FROM orders o JOIN order_totals t ON t.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.coupon;
```

```
coupon  n_orders  revenue  aov
------  --------  -------  ------
(none)  9         10120    1124.4
SAVE10  3         2540     846.7
```

Answer: No coupon: 9 orders, 10120, AOV 1124.4. SAVE10: 3 orders, 2540, AOV 846.7. Check: 9 + 3 = 12 orders and 10120 + 2540 = 12660. Coupon orders are smaller by about 25 percent, a finding worth a note (with only 3 orders, do not over-read it).

Problem 3.12.5 Which months had at least one order that was not paid?

```
SELECT substr(order_date,1,7) AS month,
       COUNT(*) FILTER (WHERE status <> 'paid') AS not_paid
FROM orders
GROUP BY month
HAVING COUNT(*) FILTER (WHERE status <> 'paid') > 0
ORDER BY month;
```

```
month    not_paid
-------  --------
2024-02  1
2024-03  1
```

Answer: February (the cancelled order 104) and March (the refunded order 109), one each. January and April have none and are dropped by HAVING.

Problem 3.12.6 Customers whose first order was in January 2024.

```
SELECT customer_id, MIN(order_date) AS first_order
FROM orders
GROUP BY customer_id
HAVING MIN(order_date) < '2024-02-01'
ORDER BY customer_id;
```

```
customer_id  first_order
-----------  -----------
1            2024-01-10
2            2024-01-15
```

Answer: Customer 1 (2024-01-10) and customer 2 (2024-01-15). ISO text dates compare correctly as strings. A WHERE version (`WHERE order_date < '2024-02-01'`) gives the same two customers on this data, but it answers 'who ordered before February', which is a different question and differs as soon as older data exists.

Problem 3.12.7 The three customers who bought the most different products in paid orders.

```
SELECT o.customer_id, COUNT(DISTINCT oi.product_id) AS kinds
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
ORDER BY kinds DESC, o.customer_id
LIMIT 3;
```

```
customer_id  kinds
-----------  -----
1            5
2            3
4            2
```

Answer: Customer 1 with 5 products (everything but the Mug), customer 2 with 3 (Mouse, Desk Lamp, Keyboard), customer 4 with 2 (Keyboard, Desk Lamp). Tie-break by customer id keeps the output deterministic. Customers 5 and 6 have 2 and 2 as well, so the cut at 3 rows decides by id; note this in a report.

Problem 3.12.8 What percent of all units sold are Notebooks?

```
SELECT ROUND(100.0 * SUM(qty) FILTER (WHERE product_id = 3) / SUM(qty), 1) AS pct_notebook_units
FROM order_items;
```

```
pct_notebook_units
------------------
41.2
```

Answer: 14 of 34 units, 41.2 percent. Both the numerator and denominator come from the same pile (all lines), so one pass with FILTER is enough. Note that Notebooks are only 7.7 percent of revenue (1120 / 14560): units and money tell different stories.

Scoring yourself. Count a problem as correct only if your query returned the shown result without edits. 8 of 8: move on. 6 or 7: reread the section named in the playbook table for the ones you missed. 5 or fewer: redo Sections 3.7 to 3.9 and rerun every example by hand.

3.13 Self-Test and Scoring Guide

Fifteen short questions, one point each. Answer in your head or on paper first, then read the answer under each.

1. What does `SELECT SUM(qty) FROM order_items WHERE qty > 100` return?
   Answer: One row containing NULL (not 0, not no rows).
2. A column has values 4, 6, NULL. What are `COUNT(*)`, `COUNT(col)` and `AVG(col)`?
   Answer: 3, 2 and 5.0.
3. Why does `SUM(qty) / COUNT(*)` give 1 for 34 units over 20 lines in SQLite?
   Answer: Integer division truncates 1.7 to 1. Multiply by `1.0`.
4. Which clause can contain `COUNT(*) > 2`: WHERE or HAVING?
   Answer: HAVING.
5. Where do you put `status = 'paid'` for speed and clarity?
   Answer: WHERE.
6. Why is the average of customer averages (962) not the AOV (1055)?
   Answer: Customers with more orders deserve more weight; divide total revenue by total orders.
7. What is the grain of `SELECT month, category, SUM(x) ... GROUP BY month, category`?
   Answer: One row per month and category that occurs in the data.
8. After joining orders to order_items, what does `COUNT(*)` count?
   Answer: Item lines. Use `COUNT(DISTINCT order_id)` for orders.

9. How do GROUP BY and DISTINCT treat NULLs?
   Answer: All NULLs form one group or one row.
10. Why is `COUNT(CASE WHEN c THEN 1 ELSE 0 END)` wrong?
    Answer: 0 is not NULL, so every row is counted.
11. Write a portable replacement for `SUM(x) FILTER (WHERE y = 1)`.
    Answer: `SUM(CASE WHEN y = 1 THEN x END)`.
12. How do you avoid division by zero in a rate?
    Answer: Divide by `NULLIF(denominator, 0)`; the result is NULL when undefined.
13. How do you list customers with *no* paid order, including those with no orders at all?
    Answer: LEFT JOIN from customers, group by customer, `HAVING COUNT(*) FILTER (WHERE o.status='paid') = 0`.
14. Per-product revenue sums to 14500 but the plain total is 14560. What do you check first?
    Answer: Rows lost by an inner join or a WHERE (here 60 is missing), or rows with NULL keys that your grouping dropped.
15. Name two ways to get the whole row (the name) of the maximum in each group.
    Answer: Join back to a subquery of the group maximum; or number rows with `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ... DESC)` and keep 1.

Score	What it means	What to reread
14 to 15	Solid. You can solve most grouped-query problems.	Do the practice set under time pressure; then Chapter 4 (joins).
11 to 13	Good, with a few leaks.	Reread the sections for the questions you missed: 1 to 3 and 12: Section 3.6; 4, 5 and 14: Sections 3.5 and 3.8; 7 to 9 and 13: Section 3.7; 10 and 11: Section 3.9; 15: Section 3.10.
7 to 10	Shaky on the mechanics.	Reread Sections 3.5 to 3.9 slowly, running every example. Redo the drill.
0 to 6	Start again from the pipeline.	Read Section 3.5 and Problem 3.5.1 until you can draw the pipeline from memory, then redo Sections 3.1 to 3.4 with a pencil.

3.14 Where This Chapter Sits: What Came Before and What Comes Next

- **Before (Chapter 2).** Chapter 2 gave you `WHERE`, `ORDER BY`, `LIMIT`, `DISTINCT` and NULL logic. Every aggregate rule here rests on it: aggregates skip NULL, and `WHERE` keeps only rows where the test is true (Section 2.4).
- **This chapter.** Collapse rows into piles and numbers, and filter piles.
- **Next (Chapter 4).** Joins bring several tables together. You now know why every join changes the grain and how to guard against fan-out; Chapter 4 teaches the join types, and you will meet the aggregate-before-join cure again.
- **Later.** Chapter 5 (subqueries, CTEs, `EXISTS`) for the comparisons in Patterns 5 and 8; Chapter 7 for `ROLLUP`, `CUBE` and pivots; Chapter 8 for window functions (top-N per group, running totals, percent of total); Chapter 10 for indexes and plans; Chapter 12 for retention, funnels and cohorts built from the patterns in this chapter.

Chapter recap

- Aggregates collapse many rows into one value; most skip NULL, `COUNT(*)` does not.
- `AVG` ignores NULL: decide whether missing means zero or unknown.
- `GROUP BY` makes one row per distinct key; know the grain of your result.
- Every selected column must be grouped or aggregated.
- `WHERE` filters rows before grouping; `HAVING` filters groups after.
- `FILTER` or `CASE` inside aggregates gives conditional counts and sums.
- Never average averages; sum and divide.
- Cast to decimal before dividing integers.

- Always check the total of a grouped query against the ungrouped total.
- The logical order is FROM and JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT; a clause can only use what already exists when it runs (Section 3.5).
- Choose the counting function by the question: `COUNT(*)` rows, `COUNT(col)` filled values, `COUNT(DISTINCT id)` different things (Section 3.6).
- Empty or all-NULL piles give NULL for `SUM`, `AVG`, `MIN`, `MAX` and 0 for `COUNT`; integer division truncates; `SUM(DISTINCT x)` is almost always a bug.
- Every join to a many-side table changes the grain. Count with `COUNT(DISTINCT key)` or aggregate to the right grain before joining; after a `LEFT JOIN` count a right-side column (Section 3.7).
- `HAVING` handles thresholds, 'at least one' (`COUNT(*) FILTER`), 'every row' (`MIN = MAX`, or count equals filtered count) and comparisons with the overall average (Section 3.8).
- `COUNT(CASE ... ELSE 0 END)` counts everything; pair COUNT with no ELSE and SUM with `ELSE 0`. A filtered `SUM` with no rows is NULL, so `COALESCE` it in pivots (Section 3.9).
- Problem-solving method: say the grain, row conditions in WHERE, GROUP BY the key, aggregate (FILTER/CASE), pile conditions in HAVING, then reconcile with a plain total. Eight patterns cover most questions (Section 3.10).
- Expert layer: hash versus sort aggregation, NULLs equal in GROUP BY but unknown in comparisons, partial aggregates need (sum, count), floats and money, pandas differences, ROLLUP and windows (Section 3.11).

Pattern	Syntax
Count rows / non-NULL / distinct	`COUNT(*)`, `COUNT(c)`, `COUNT(DISTINCT c)`
Per group	`SELECT k, SUM(v) FROM t GROUP BY k`
Filter groups	`... GROUP BY k HAVING COUNT(*) > 2`
Conditional count	`COUNT(*) FILTER (WHERE s='paid')` or `SUM(CASE WHEN s='paid' THEN 1 ELSE 0 END)`
Safe rate	`1.0 * a / NULLIF(b, 0)`
Count real orders after a join	`COUNT(DISTINCT o.order_id)`
Count matches after LEFT JOIN	`COUNT(r.id)`, not `COUNT(*)`
Pivot column	`COALESCE(SUM(x) FILTER (WHERE k='a'), 0)`
All rows in the pile satisfy	`HAVING MIN(c) = MAX(c)` or `COUNT(*) = COUNT(*) FILTER (WHERE cond)`
Duplicates	`GROUP BY k HAVING COUNT(*) > 1`
Share of total	`100.0 * SUM(x) / (SELECT SUM(x) FROM t)`
Weighted average	`SUM(x) / SUM(w)`, never an average of averages
Compare with overall average	`HAVING SUM(x) > (SELECT AVG(s) FROM (...))`

Practice set

Problem 3.P.1 (easy) How many distinct customers placed orders?

```
SELECT COUNT(DISTINCT customer_id) AS buyers FROM orders;
```

```
buyers
------
6
```

Answer: 6 (customers 7 and 8 never ordered).

Problem 3.P.2 (easy) What is the average product price, rounded to 2 decimals?

```
SELECT ROUND(AVG(price), 2) AS avg_price FROM products;
```

```
avg_price
---------
575.0
```

Answer: `(1500+600+80+120+900+250)/6 = 3450/6 = 575.0`.

Problem 3.P.3 (what is the output?) What does this return?

```
SELECT category, COUNT(*) AS n, MAX(price) AS top_price
FROM products GROUP BY category ORDER BY category;
```

```
category     n  top_price
-----------  -  ---------
Electronics  2  1500
Home         2  900
Stationery   2  120
```

Answer: One row per category, as printed: Electronics 2 / 1500, Home 2 / 900, Stationery 2 / 120.

Problem 3.P.4 (find the bug) A teammate wrote `SELECT category, name, MAX(price) FROM products GROUP BY category;`. What is wrong?
Answer: `name` is not grouped or aggregated. PostgreSQL rejects it. SQLite picks a name from the max row for `MAX` (a special SQLite rule) but this is not portable. Use a window function (Chapter 8) to fetch the name of the max row.

Problem 3.P.5 Total quantity sold for each product, only products with 3 or more units.

```
SELECT product_id, SUM(qty) AS units
FROM order_items GROUP BY product_id HAVING SUM(qty) >= 3
ORDER BY product_id;
```

```
product_id  units
----------  -----
1           4
2           5
3           14
4           7
5           4
```

Answer: Products 1 to 5 with 4, 5, 14, 7 and 4 units. The Mug (product 6) has no rows.

Problem 3.P.6 Average order value (AOV) over paid orders: total paid revenue divided by number of paid orders.

```
SELECT SUM(oi.qty*oi.unit_price) AS revenue,
       COUNT(DISTINCT o.order_id) AS n_orders,
       ROUND(1.0*SUM(oi.qty*oi.unit_price)/COUNT(DISTINCT o.order_id), 2) AS aov
FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
WHERE o.status='paid';
```

```
revenue  n_orders  aov
-------  --------  ------
12660    12        1055.0
```

Answer: Revenue 12660 over 12 orders gives 1055.0. Note `COUNT(DISTINCT order_id)`: counting rows would count lines, not orders.

Problem 3.P.7 Coupon usage: for each coupon value (including no coupon) count orders.

```
SELECT COALESCE(coupon,'(none)') AS coupon, COUNT(*) AS n
FROM orders GROUP BY coupon ORDER BY n DESC;
```

```
coupon  n
------  --
(none)  11
SAVE10  3
```

Answer: (none) 11, SAVE10 3. NULLs form one group.

Problem 3.P.8 (scenario) The marketing team asks for "categories whose total revenue is over 3000". Write it.

```
SELECT p.category, SUM(oi.qty*oi.unit_price) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category HAVING SUM(oi.qty*oi.unit_price) > 3000
ORDER BY revenue DESC;
```

```
category     revenue
-----------  -------
Electronics  9000
Home         3600
```

Answer: Electronics 9000 (`6000 + 3000`) and Home 3600. Stationery has `1120 + 840 = 1960`, below 3000.

Problem 3.P.9 (interview) What is the difference between WHERE and HAVING? **Answer:** WHERE filters individual rows before grouping and cannot use aggregates. HAVING filters groups after aggregation and can use aggregates.

Problem 3.P.10 (interview) Why is "average of averages" wrong? Give a number example. **Answer:** Group A has 1 order of 100, group B has 3 orders of 200 each. Average of averages is `(100 + 200)/2 = 150`. True average is `(100 + 600)/4 = 175`. Always sum the numerators and denominators, then divide.

Problem 3.P.11 (easy) How many products cost more than 100, and how many per category?

```
SELECT category, COUNT(*) AS n
FROM products
WHERE price > 100
GROUP BY category
ORDER BY category;
```

```
category     n
-----------  -
Electronics  2
Home         2
Stationery   1
```

Answer: Electronics 2, Home 2, Stationery 1 (the Pen Pack at 120; the Notebook at 80 is excluded). Total 5. WHERE is right because price is a row-level fact.

Problem 3.P.12 (easy) The latest order date for every customer.

```
SELECT customer_id, MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

```
customer_id  last_order
-----------  ----------
1            2024-04-02
2            2024-04-20
3            2024-02-10
4            2024-04-09
5            2024-04-15
6            2024-03-30
```

Answer: Customer 1: 2024-04-02, 2: 2024-04-20, 3: 2024-02-10, 4: 2024-04-09, 5: 2024-04-15, 6: 2024-03-30. Six rows because only six customers ever ordered.

Problem 3.P.13 (what is the output?) Predict the three numbers, then run.

```
SELECT COUNT(*), COUNT(coupon), SUM(coupon IS NULL) FROM orders;
```

```
COUNT(*)  COUNT(coupon)  SUM(coupon IS NULL)
--------  -------------  -------------------
14        3              11
```

Answer: 14, 3 and 11. `coupon IS NULL` is 1 for orders without a coupon and 0 otherwise (SQLite and MySQL treat comparisons as 0/1), so summing it counts them: 14 - 3 = 11. In PostgreSQL write `COUNT(*) FILTER (WHERE coupon IS NULL)`.

Problem 3.P.14 (find the bug) A teammate wants customers with more than 2 paid orders and writes the query below. What does it return, and why is it wrong?

```
SELECT customer_id, COUNT(*)
FROM orders
WHERE status = 'paid'
HAVING COUNT(*) > 2;
```

```
customer_id  COUNT(*)
-----------  --------
1            12
```

Answer: It returns a single row, customer 1 with 12, which is silently wrong: there is no GROUP BY, so all 12 paid orders form one pile, and SQLite fills the bare `customer_id` from an arbitrary row. PostgreSQL would refuse (column must appear in GROUP BY). Fix: add `GROUP BY customer_id`; the result is customers 1 (4), 2 (3) only.

Problem 3.P.15 (medium) Average units per paid order for each customer (units divided by distinct orders).

```
SELECT o.customer_id,
       COUNT(DISTINCT o.order_id) AS orders,
       SUM(oi.qty)                AS units,
       ROUND(1.0 * SUM(oi.qty) / COUNT(DISTINCT o.order_id), 2) AS units_per_order
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
ORDER BY o.customer_id;
```

```
customer_id  orders  units  units_per_order
-----------  ------  -----  ---------------
1            4       9      2.25
2            3       4      1.33
4            2       3      1.5
5            2       7      3.5
6            1       5      5.0
```

Answer: Customer 1: 9 / 4 = 2.25; 2: 4 / 3 = 1.33; 4: 3 / 2 = 1.5; 5: 7 / 2 = 3.5; 6: 5 / 1 = 5.0. The denominator must be distinct orders, because the join produces one row per line (Section 3.7).

Problem 3.P.16 (medium) Rank the months by paid revenue, highest first.

```
SELECT substr(o.order_date,1,7) AS month, SUM(oi.qty * oi.unit_price) AS revenue
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY month
ORDER BY revenue DESC;
```

```
month    revenue
-------  -------
2024-04  4900
2024-03  2980
2024-02  2520
2024-01  2260
```

Answer: April 4900, March 2980, February 2520, January 2260; total 12660. ORDER BY may use the aggregate alias because ORDER BY runs after SELECT.

Problem 3.P.17 (medium) For each category: lines, lines with two or more units, and the percent.

```
SELECT p.category,
       COUNT(*) AS lines,
       COUNT(*) FILTER (WHERE oi.qty >= 2) AS multi_unit,
       ROUND(100.0 * COUNT(*) FILTER (WHERE oi.qty >= 2) / COUNT(*), 1) AS pct
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY p.category;
```

```
category     lines  multi_unit  pct
-----------  -----  ----------  ----
Electronics  8      1           12.5
Home         3      1           33.3
Stationery   9      6           66.7
```

Answer: Electronics 8 lines, 1 multi-unit (order 106 buys 2 mice): 12.5 percent. Home 3 lines, 1 (order 112, 2 lamps): 33.3. Stationery 9 lines, 6: 66.7. Stationery is bought in multiples, as expected for cheap items. Lines add to 20.

Problem 3.P.18 (hard) Customers for whom at least one third of their orders were refunded.

```
SELECT customer_id, COUNT(*) AS orders,
       COUNT(*) FILTER (WHERE status = 'refunded') AS refunded
FROM orders
GROUP BY customer_id
HAVING 3.0 * COUNT(*) FILTER (WHERE status = 'refunded') >= COUNT(*);
```

```
customer_id  orders  refunded
-----------  ------  --------
5            3       1
```

Answer: Only customer 5 (1 refunded of 3). The test is written as `3 * refunded >= orders` instead of `refunded / orders >= 1/3` to avoid a division and a rounding error on 0.3333. Cross-multiplying is a good habit for ratio thresholds.

Problem 3.P.19 (hard) Who bought from all three categories? Count the categories per customer and keep the customers that reach 3.

```
SELECT o.customer_id, COUNT(DISTINCT p.category) AS categories
FROM orders o
JOIN order_items oi ON oi.order_id  = o.order_id
JOIN products    p  ON p.product_id = oi.product_id
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.category) = 3;
```

```
customer_id  categories
-----------  ----------
1            3
```

Answer: Only customer 1. A 'bought every one of N things' question is `HAVING COUNT(DISTINCT thing) = N` where N is the total number of things (here 3; check it with `SELECT COUNT(DISTINCT category) FROM products` rather than hard-coding). All orders count here, including the cancelled one.

Problem 3.P.20 (hard) Each customer's share of paid revenue, largest first.

```
SELECT o.customer_id,
       SUM(oi.qty * oi.unit_price) AS spent,
       ROUND(100.0 * SUM(oi.qty * oi.unit_price) /
             (SELECT SUM(i2.qty * i2.unit_price)
              FROM orders o2 JOIN order_items i2 ON i2.order_id = o2.order_id
              WHERE o2.status = 'paid'), 1) AS pct
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY o.customer_id
ORDER BY spent DESC;
```

```
customer_id  spent  pct
-----------  -----  ----
1            4560   36.0
2            3600   28.4
4            3300   26.1
5            760    6.0
6            440    3.5
```

Answer: Customer 1 36.0 percent, 2 28.4, 4 26.1, 5 6.0, 6 3.5; 36.0 + 28.4 + 26.1 + 6.0 + 3.5 = 100.0. The subquery repeats the same WHERE as the outer query: a share is only meaningful if numerator and denominator use the same filter.

Problem 3.P.21 (hard) New buyers per month: customers whose first paid order falls in that month.

```
SELECT substr(first_paid, 1, 7) AS month, COUNT(*) AS new_buyers
FROM (SELECT customer_id, MIN(order_date) AS first_paid
      FROM orders
      WHERE status = 'paid'
      GROUP BY customer_id)
GROUP BY month
ORDER BY month;
```

```
month    new_buyers
-------  ----------
2024-01  2
2024-02  1
2024-03  2
```

Answer: January 2 (customers 1 and 2), February 1 (customer 4), March 2 (customers 5 and 6), April 0 (no row). Total 5 paying customers. This is a two-level aggregation: first one row per customer, then one row per month. Two levels solve most cohort questions.

Problem 3.P.22 (expert) A dashboard shows average order value as `AVG(qty * unit_price)` over `order_items`. Is it right? Compare with revenue divided by the number of orders (all statuses).

```
SELECT ROUND(AVG(qty * unit_price), 1) AS avg_line_value,
       ROUND(1.0 * SUM(qty * unit_price) / COUNT(DISTINCT order_id), 1) AS aov_all_orders
FROM order_items;
```

```
avg_line_value  aov_all_orders
--------------  --------------
728.0           1040.0
```

Answer: No. `AVG` over lines gives 728.0, the average *line*. The AOV is revenue per *order*: 14560 / 14 = 1040.0. The dashboard understates AOV by 30 percent because orders have 1.43 lines on average (20 / 14). Match the grain of the average to the noun in its name.

Problem 3.P.23 (expert) Customers who ordered in at least 2 different months (a simple repeat-over-time test).

```
SELECT customer_id,
       COUNT(DISTINCT substr(order_date, 1, 7)) AS active_months,
       COUNT(*) AS orders
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT substr(order_date, 1, 7)) >= 2
ORDER BY customer_id;
```

```
customer_id  active_months  orders
-----------  -------------  ------
1            4              4
2            3              3
4            2              2
5            2              3
```

Answer: Customer 1 (4 months), 2 (3), 4 (2: February and April) and 5 (2: March and April). Customer 3 and 6 ordered once. Counting months, not orders, matters: a customer with 5 orders in one day is one active month.

Problem 3.P.24 (expert) Build a complete month-by-category grid for Home with zeros for missing months. Missing groups need a driver table that lists every month.

```
WITH months(m) AS (VALUES ('2024-01'), ('2024-02'), ('2024-03'), ('2024-04')),
sales AS (
  SELECT substr(o.order_date,1,7) AS m, SUM(oi.qty * oi.unit_price) AS rev
  FROM orders o
  JOIN order_items oi ON oi.order_id  = o.order_id
  JOIN products    p  ON p.product_id = oi.product_id
  WHERE o.status = 'paid' AND p.category = 'Home'
  GROUP BY m
)
SELECT months.m AS month, COALESCE(sales.rev, 0) AS home_revenue
FROM months LEFT JOIN sales ON sales.m = months.m
ORDER BY months.m;
```

```
month    home_revenue
-------  ------------
2024-01  0
2024-02  900
2024-03  900
2024-04  1800
```

Answer: January 0, February 900, March 900, April 1800. A plain GROUP BY cannot produce January because no row exists for it. The month list comes from a driver table and the aggregate is joined on with LEFT JOIN, then COALESCE turns the NULL into 0 (Pattern 2 of Section 3.10).

Problem 3.P.25 (expert, pandas) Translate to pandas: the number of rows, the number of non-missing marks and the total mark per subject, keeping missing as missing. Data are the `marks` table. Predict first which pandas function matches `COUNT(*)`.

```
import pandas as pd
import numpy as np
marks = pd.DataFrame({
    "subject": ["maths","science","maths","science","maths","science","maths","science"],
    "mark":    [78, 85, 62, np.nan, 91, 74, 55, 68],
})
out = marks.groupby("subject").agg(
    n_rows=("mark", "size"),
    n_marks=("mark", "count"),
    total=("mark", lambda s: s.sum(min_count=1)),
)
print(out)
```

```
         n_rows  n_marks  total
subject
maths         4        4  286.0
science       4        3  227.0
```

Answer: `size` matches `COUNT(*)`, `count` matches `COUNT(col)`, and `sum(min_count=1)` reproduces SQL's NULL for an all-missing group. Maths: 4, 4, 286; science: 4, 3, 227, the same as Problem 3.5.2.

Problem 3.P.26 (expert, interview) You are given a 2-billion-row `events(user_id, event_date, event_type)` table in a warehouse and asked for the number of distinct users per day. The query is slow. What do you try, in order?

Answer: (1) Filter dates with `WHERE` first so only needed partitions are read. (2) Check that `event_date` is the partition or clustering key. (3) Replace the exact `COUNT(DISTINCT user_id)` by the engine's approximate function (`APPROX_COUNT_DISTINCT`) if one or two percent error is acceptable. (4) Pre-aggregate once to one row per (day, user) into a summary table, then count rows in it: the expensive de-duplication is paid once, not on every query. (5) Never fix it with `SUM(DISTINCT ...)` or by sampling without saying so. Explain the trade-off between exactness and cost; that is the point of the question.

Problem 3.P.27 (expert, design) A report must show, for each customer, orders, paid revenue, and refund rate, in one row, even for customers with no orders. Write it with the grain stated.

```
-- grain: one row per customer (8 rows)
WITH order_totals AS (
  SELECT order_id, SUM(qty * unit_price) AS total FROM order_items GROUP BY order_id
)
SELECT c.customer_id, c.name,
       COUNT(o.order_id)                                      AS orders,
       COALESCE(SUM(t.total) FILTER (WHERE o.status = 'paid'), 0) AS paid_revenue,
       ROUND(1.0 * COUNT(*) FILTER (WHERE o.status = 'refunded')
             / NULLIF(COUNT(o.order_id), 0), 2)               AS refund_rate
FROM customers c
LEFT JOIN orders o        ON o.customer_id = c.customer_id
LEFT JOIN order_totals t  ON t.order_id    = o.order_id
GROUP BY c.customer_id, c.name
ORDER BY c.customer_id;
```

```
customer_id  name   orders  paid_revenue  refund_rate
-----------  -----  ------  ------------  -----------
1            Asha   4       4560          0.0
2            Ravi   3       3600          0.0
3            Meena  1       0             0.0
4            John   2       3300          0.0
5            Priya  3       760           0.33
6            Kabir  1       440           0.0
7            Sana   0       0             NULL
8            Tom    0       0             NULL
```

Answer: Orders are counted with `COUNT(o.order_id)` so Sana and Tom show 0 (not 1). Pre-aggregating to order totals keeps the join at order grain so nothing is double counted. `COALESCE` turns the NULL sum into 0 for customers without paid revenue. The refund rate divides by `NULLIF(orders, 0)`, so Sana and Tom show NULL (undefined), and customer 5 shows 0.33. Reconcile: paid revenue column adds to 12660; orders column to 14. One subtle point: `COUNT(*) FILTER` counts the placeholder row, but the filter on status removes it since its status is NULL.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my second language. Teach me Chapter 3 topic: "GROUP BY and aggregate functions in SQL". Use a tiny sales table and explain the idea of "one row per group", the grain of a result, and how NULL behaves in COUNT, SUM and AVG. After each step ask one check question and wait for my reply.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning SQL. Topic: WHERE versus HAVING and conditional aggregation with FILTER or CASE. Give me 8 graded exercises on an orders table that you design, from easy to hard, without solutions. After I send my answers, review them, explain each mistake in simple English, and show an improved query.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Quiz me one question at a time on SQL aggregation: COUNT star vs COUNT column vs COUNT DISTINCT, SUM and AVG with NULL, GROUP BY rules, HAVING, and averages of averages. Mix "what is the output" and "find the bug" questions. After 10 questions give a score and a revision list.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for analyst interviews. Review the GROUP BY query I paste below. Check the grain, columns that are not grouped, wrong use of WHERE versus HAVING, integer division in rates, and double counting after joins. Explain each problem in simple English and give a corrected query. My query: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am preparing for SQL interviews as a working professional moving into data roles. Run a mock interview on aggregation: KPIs per group, top groups with HAVING, rates with FILTER, and conditional sums. Ask one question at a time with a small table, grade me out of 10, show a model solution, and tell me one habit to improve.
```

CHAPTER 4

JOINS

Combining rows from several tables: inner, left, right, full, cross and self joins, plus anti-joins, semi-joins and the duplicate traps that give wrong totals.

Real questions rarely live in one table. "Which customers have never ordered?" needs `customers` and `orders`. "Revenue by city" needs `order_items`, `orders` and `customers`. A **join** is how SQL glues tables together using keys. It is also the most common source of silently wrong numbers: one careless join can double your revenue and no error message appears. By the end of this chapter you can choose the right join type, find customers with no orders, and check every join for duplicate rows before you trust a total.

4.1 INNER JOIN and the ON Condition

INTUITION

Imagine two class lists. One list has student names and roll numbers. The other has roll numbers and marks. To see "name with marks" you put the two lists side by side and match each line by roll number. That matching is a join. The roll number is the **join key**. An **inner join** keeps only the students who appear in both lists.

Words you need

- **Join key**: the column(s) used to match rows (`orders.customer_id = customers.customer_id`).
- **Parent / child table**: the parent holds the primary key (`customers`), the child holds the foreign key (`orders`).
- **Table alias**: a short nickname such as `o` for `orders`, so you can write `o.order_id`.
- **Qualified column**: `table.column`, needed when two tables have a column with the same name.

How it works

```
SELECT ...
FROM   child c
JOIN   parent p  ON c.parent_id = p.parent_id;
```

Conceptually the database looks at every pair of rows, keeps the pairs where the `ON` test is TRUE, and glues each pair into one wide row. (It does not really test every pair. It uses indexes or hash tables, see Chapter 10. But the *result* is the same as if it did.) Rows with no match on either side disappear. If one parent matches three children, the parent appears three times.

Real-world mapping

Join	Example	Where it appears in the job
`orders JOIN customers`	add customer name to each order	enriching events with user attributes
`order_items JOIN products`	add product name and category	enriching logs with a lookup table
3+ tables chained	items to orders to customers	star-schema reporting (Chapter 11)

Worked examples

Problem 4.1.1 Show the first five orders with the customer name. Predict first: the result has one row per order, so 5 rows, each with a name.

Step 1	The child table is `orders` (holds `customer_id`). The parent is `customers`.
Step 2	Join on `o.customer_id = c.customer_id`.
Step 3	Order and limit so the output is stable.

```
SELECT o.order_id, c.name, o.order_date, o.status
FROM orders AS o
JOIN customers AS c ON c.customer_id = o.customer_id
ORDER BY o.order_id
LIMIT 5;
```

```
order_id  name   order_date  status
--------  -----  ----------  ---------
101       Asha   2024-01-10  paid
102       Ravi   2024-01-15  paid
103       Asha   2024-02-03  paid
104       Meena  2024-02-10  cancelled
105       John   2024-02-20  paid
```

- **Check** From the base data, order 101 belongs to customer 1 (Asha), 102 to 2 (Ravi), 103 to 1, 104 to 3 (Meena), 105 to 4 (John). The output matches.
- **Answer** Five rows, each order now carries its customer's name.
- **Meaning** The orders table only stored a number. The join turned `customer_id = 1` into a human-readable name.

Problem 4.1.2 Chain three tables: for order 101, list each line with product name and quantity.

Step 1	Start with `order_items` (child of both `orders` and `products`).
Step 2	Join `products` on `product_id`.
Step 3	Filter to order 101.

```
SELECT oi.order_id, p.name AS product, oi.qty, oi.unit_price,
       oi.qty * oi.unit_price AS line_total
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
WHERE oi.order_id = 101;
```

```
order_id  product   qty  unit_price  line_total
--------  --------  ---  ----------  ----------
101       Keyboard  1    1500        1500
101       Notebook  2    80          160
```

- **Check** Order 101: one Keyboard at 1500 and two Notebooks at 80 each: `1 × 1500 = 1500` and `2 × 80 = 160`, total 1660.
- **Answer** Two lines: Keyboard 1500 and Notebook 160.
- **Meaning** One order has two lines, so a join from `orders` to `order_items` will return two rows for order 101. Remember this; Section 4.4 depends on it.

Problem 4.1.3 Combine: paid revenue per customer city. This needs `order_items`, `orders` and `customers` together.

Step 1	Items give money. Orders give status and the customer. Customers give the city.
Step 2	Filter `status = 'paid'`.
Step 3	Group by city and sum.

```
SELECT c.city, SUM(oi.qty * oi.unit_price) AS paid_revenue
FROM order_items oi
JOIN orders o    ON o.order_id    = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'paid'
GROUP BY c.city
ORDER BY paid_revenue DESC;
```

```
city       paid_revenue
---------  ------------
Pune       4560
Mumbai     4040
Delhi      3300
Bengaluru  760
```

- **Compute** Pune: customer 1 paid `4560` (customer 3's only order is cancelled, Tom has none). Mumbai: customer 2 `3600` plus customer 6 `440` is `4040`. Delhi: customer 4 `3300`. Bengaluru: customer 5 `760`.
- **Check** `4560 + 4040 + 3300 + 760 = 12660`, the paid total from Chapter 3.
- **Answer** Pune 4560, Mumbai 4040, Delhi 3300, Bengaluru 760.
- **Meaning** Pune brings 36 percent of paid revenue (`4560 / 12660`).

The picture

```
customers (parent)           orders (child)
+----+-------+               +-----+-------------+
| 1  | Asha  |<-----+--------| 101 | customer_id=1
| 2  | Ravi  |<-----|---+---| 102 | customer_id=2
| 7  | Sana  |   (no match)  | 103 | customer_id=1  ---+
+----+-------+      |   |    +-----+                   |
                    +---|--------------------------------+
INNER JOIN keeps: (Asha,101) (Ravi,102) (Asha,103) ...
Sana (7) has no order, so she is not in the result.
```

Read it like this: each arrow is one matching pair. Asha has two arrows, so she appears twice in the joined result. Sana has none, so she disappears.

Connection to the real job

Enriching raw events with attributes (user plan, country, product category) is the daily job of an analytics engineer. For example: a delivery table has only `product_id`, and a lookup table gives the product name for it; you join to make the report readable.

Applications

- **Data Analytics:** revenue by city, category or channel.
- **Data Engineering:** building a denormalised reporting table from normalised sources.
- **Machine Learning:** attaching customer attributes to each transaction to create training rows.
- **Healthcare:** attaching doctor and clinic details to raw appointment records.

COMMON MISTAKE

Forgetting the `ON` condition, or writing `ON 1=1`. That gives every row paired with every row (a **cross join**, Section 4.2). Another one: `NATURAL JOIN` or relying on identical column names; it breaks when someone adds a column. Always write `ON` explicitly.

Check yourself

1. What does an inner join do with a row that has no match?

Answer: It drops the row.

1. If parent row P matches 3 child rows, how many times does P appear?

Answer: 3 times.

1. Why use table aliases?

Answer: Shorter code, and they separate columns with the same name.

4.2 LEFT, RIGHT, FULL, CROSS and SELF Joins

INTUITION

An inner join is a strict teacher: "only students with marks are listed". A **left join** is a kind teacher: "list all students, and write 'absent' if there are no marks". Use it when the left table is your *list of things you care about*, such as all customers, all products, all scheduled stops.

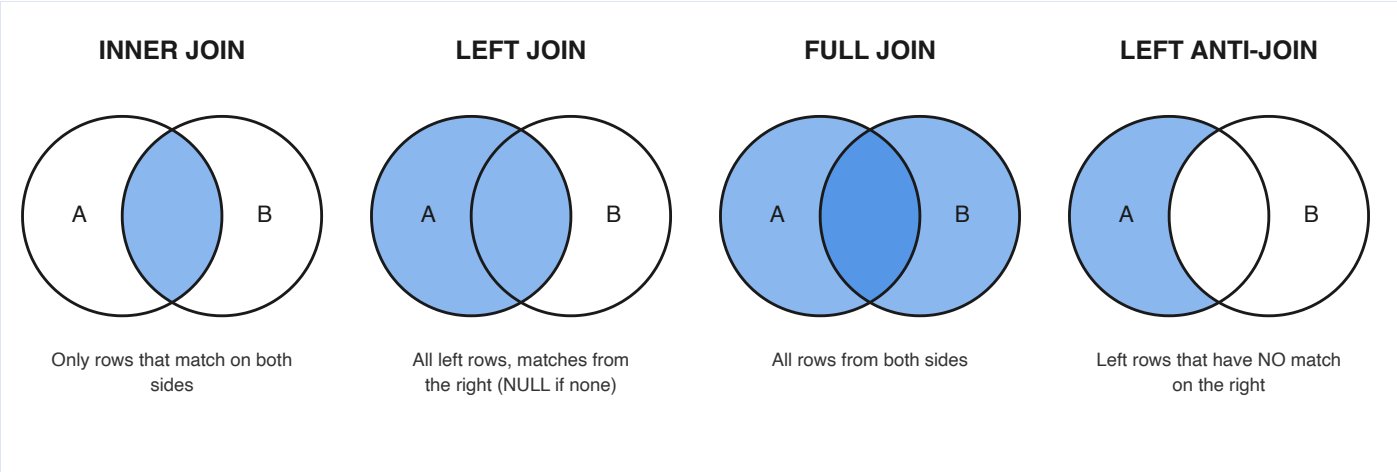
Words you need

- **Left table**: the one written before `JOIN`. **Right table**: the one after.
- **Preserved side**: the table whose every row survives.
- **Cartesian product**: all possible pairs, `rows_A × rows_B`.

Definitions

Join	Keeps	Unmatched side filled with
`INNER JOIN`	matched pairs only	(not applicable)
`LEFT JOIN`	all left rows + matched right	NULL on the right
`RIGHT JOIN`	all right rows + matched left	NULL on the left
`FULL JOIN`	all rows from both	NULL on whichever side is missing
`CROSS JOIN`	every pair (no `ON`)	not applicable
Self join	a table joined to itself (use two aliases)	depends on join type

`RIGHT JOIN` is just a `LEFT JOIN` with the tables swapped; most people always write `LEFT`. `RIGHT` and `FULL JOIN` work in SQLite 3.39 and newer (this book uses 3.51) and in PostgreSQL, but not in old MySQL (no `FULL`).



The four most common join shapes as overlapping circles: A is the left table, B the right table. Shaded area shows the rows kept.

How to read the picture: each circle is a table. The overlap holds the rows that match on the key. The shaded part is what the query returns. In the left anti-join panel the overlap is white, so only A-rows without a partner remain.

Worked examples

Problem 4.2.1 List every customer with their number of orders, including customers who never ordered.

Predict: Sana (7) and Tom (8) have no orders. Will an inner join show them? (No.) What should their counts be with a left join? (0.)

- **Step 1** Put `customers` on the left because we want all of them.
- **Step 2** `LEFT JOIN orders` on the key.
- **Step 3** Count `o.order_id`, **not** `COUNT(*)`. For a customer with no orders, the left join still produces one row (with NULLs), so `COUNT(*)` would say 1. `COUNT(o.order_id)` skips NULL and says 0.

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS n_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY c.customer_id;
```

```
customer_id  name   n_orders
-----------  -----  --------
1            Asha   4
2            Ravi   3
3            Meena  1
4            John   2
5            Priya  3
6            Kabir  1
7            Sana   0
8            Tom    0
```

Now the wrong version, with `COUNT(*)`:

```
SELECT c.name, COUNT(*) AS wrong_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id IN (6, 7, 8)
GROUP BY c.name
ORDER BY c.name;
```

```
name   wrong_count
-----  -----------
Kabir  1
Sana   1
Tom    1
```

- **Check** Counts from the base data: Asha 4, Ravi 3, Meena 1, John 2, Priya 3, Kabir 1, Sana 0, Tom 0. They add to 14, the number of orders.
- **Answer** The first query is correct. The wrong version reports Sana and Tom as 1.
- **Meaning** `COUNT(*)` counts rows, and a lonely left row is still a row. This is the most frequent left-join bug.

Problem 4.2.2 The `ON` versus `WHERE` trap. Count only **paid** orders per customer, but still list all customers.

Step 1	Wrong way: put `o.status = 'paid'` in `WHERE`. After the left join, Sana has `o.status = NULL`. `NULL = 'paid'` is UNKNOWN, so the row is removed. The left join turned back into an inner join.
Step 2	Right way: put the condition in `ON`, so it decides which orders *match*, not which customers survive.

```
SELECT c.name, COUNT(o.order_id) AS paid_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'paid'
GROUP BY c.name
ORDER BY c.name;
```

```
name   paid_orders
-----  -----------
Asha   4
John   2
Kabir  1
Priya  2
Ravi   3
```

```
SELECT c.name, COUNT(o.order_id) AS paid_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status = 'paid'
GROUP BY c.name
ORDER BY c.name;
```

```
name   paid_orders
-----  -----------
Asha   4
John   2
Kabir  1
Meena  0
Priya  2
Ravi   3
Sana   0
Tom    0
```

- **Check** Paid orders per customer from the data: Asha 4, Ravi 3, Meena 0 (her one order is cancelled), John 2, Priya 2 (109 is refunded), Kabir 1, Sana 0, Tom 0. The second output has exactly these 8 rows. The first lost Meena, Sana and Tom.
- **Answer** Use `ON` for conditions on the right table in a left join.
- **Meaning** A report that must list "all customers" must never lose the zero rows.

Problem 4.2.3 Which products never sold? And what are the units sold per product, with zero for the Mug?

```
SELECT p.name, COALESCE(SUM(oi.qty), 0) AS units
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.product_id
GROUP BY p.product_id, p.name
ORDER BY units, p.name;
```

```
name       units
---------  -----
Mug        0
Desk Lamp  4
Keyboard   4
Mouse      5
Pen Pack   7
Notebook   14
```

- **Check** From Chapter 3 the units are Notebook 14, Pen Pack 7, Mouse 5, Keyboard 4, Desk Lamp 4, and the Mug 0. Total units `14 + 7 + 5 + 4 + 4 + 0 = 34`.
- **Answer** The Mug sold 0 units.
- **Meaning** `SUM` of no rows is NULL, so we use `COALESCE(..., 0)` to display 0.

Problem 4.2.4 FULL JOIN: compare a train timetable with what actually happened. A tiny example from railways.

Step 1	`scheduled` lists the stops the train was meant to make. `arrived` lists the stops it actually reached, with the minutes it was late.
Step 2	`FULL JOIN` shows scheduled-but-not-reached (NULL on the right) and reached-but-not-scheduled (NULL on the left).

```
CREATE TABLE scheduled (stop TEXT);
CREATE TABLE arrived (stop TEXT, minutes_late INTEGER);
INSERT INTO scheduled VALUES ('Pune'),('Lonavala'),('Kalyan'),('Thane');
INSERT INTO arrived VALUES ('Pune',23),('Lonavala',41),('Kalyan',98),('Dadar',15);

SELECT s.stop AS scheduled_stop, a.stop AS arrived_stop, a.minutes_late
FROM scheduled s
FULL JOIN arrived a ON a.stop = s.stop;
```

```
scheduled_stop  arrived_stop  minutes_late
--------------  ------------  ------------
Pune            Pune          23
Lonavala        Lonavala      41
Kalyan          Kalyan        98
Thane           NULL          NULL
NULL            Dadar         15
```

- **Check** 3 matches (Pune, Lonavala, Kalyan) + 1 only in scheduled (Thane) + 1 only in arrived (Dadar) = 5 rows.
- **Answer** The train never reached `Thane` and it called at `Dadar` without being scheduled to.
- **Meaning** A FULL JOIN is the standard tool for reconciliation: comparing two lists and finding what is missing on either side.

Problem 4.2.5 CROSS JOIN: build a complete grid so that missing combinations show as zero. List every (category, month) pair, then sum the paid revenue.

Step 1	Make the list of categories (3) and months (4). `CROSS JOIN` gives `3 × 4 = 12` pairs.
Step 2	`LEFT JOIN` the real numbers onto the grid.

```
WITH cats AS (SELECT DISTINCT category FROM products),
     months AS (SELECT DISTINCT substr(order_date,1,7) AS month FROM orders)
SELECT c.category, m.month, COALESCE(SUM(oi.qty * oi.unit_price), 0) AS revenue
FROM cats c
CROSS JOIN months m
LEFT JOIN products p     ON p.category = c.category
LEFT JOIN orders o       ON substr(o.order_date,1,7) = m.month AND o.status = 'paid'
LEFT JOIN order_items oi ON oi.order_id = o.order_id AND oi.product_id = p.product_id
GROUP BY c.category, m.month
ORDER BY c.category, m.month;
```

```
category     month    revenue
-----------  -------  -------
Electronics  2024-01  2100
Electronics  2024-02  1500
Electronics  2024-03  1200
Electronics  2024-04  2700
Home         2024-01  0
Home         2024-02  900
Home         2024-03  900
Home         2024-04  1800
Stationery   2024-01  160
Stationery   2024-02  120
Stationery   2024-03  880
Stationery   2024-04  400
```

(`WITH ... AS` is a CTE, a named temporary result; Chapter 5 teaches it. Read it as "two small helper tables".)

- **Check** The result has 12 rows, `3 × 4`. For example Home in 2024-03 is Desk Lamp in paid order 108: `1 × 900 = 900`. Electronics in 2024-01: Keyboard in 101 (1500) plus Mouse in 102 (600) is 2100.
- **Answer** A complete grid with zeros where nothing was sold.
- **Meaning** Charts need every month present, even with zero. A cross join builds the "date spine" or grid first, then the data is attached.

Problem 4.2.6 SELF JOIN: show each customer with the name of the person who referred them.

Step 1	`customers` appears twice, as `c` (the customer) and `r` (the referrer).
Step 2	Use `LEFT JOIN` because some customers have no referrer.

```
SELECT c.name AS customer, r.name AS referred_by
FROM customers c
LEFT JOIN customers r ON r.customer_id = c.referrer_id
ORDER BY c.customer_id;
```

```
customer  referred_by
--------  -----------
Asha      NULL
Ravi      Asha
Meena     Asha
John      Ravi
Priya     NULL
Kabir     Priya
Sana      NULL
Tom       Meena
```

- **Check** Referrers from the base data: Ravi by 1 (Asha), Meena by 1 (Asha), John by 2 (Ravi), Kabir by 5 (Priya), Tom by 3 (Meena). Asha, Priya, Sana have none.
- **Answer** Five customers have a referrer, three show NULL.

- **Meaning** Self joins are used for hierarchies: employee and manager, category and parent category. (For deep hierarchies Chapter 5 uses recursive CTEs.)

Connection to the real job

LEFT JOIN is the workhorse of analytics ("all users, with their purchases if any"). FULL JOIN powers data reconciliation between a source system and a warehouse. CROSS JOIN builds date and category grids. Self joins model org charts, referrals and "compare to previous row" problems (which window functions in Chapter 8 do more cleanly).

Applications

- **Data Analytics:** funnel and cohort tables that must include zero rows.
- **Data Engineering:** reconciliation of source vs target row sets after a migration.
- **Data Science:** building a complete user-by-day grid before computing rolling features.
- **Operations:** planned vs actual train stops, scheduled vs delivered orders.

COMMON MISTAKE

Putting a filter on the right table in `WHERE` after a `LEFT JOIN`. It deletes the NULL rows and quietly changes your join to inner. Put right-side conditions in `ON`. The second most common bug: `COUNT(*)` after a left join; use `COUNT(right_key)`.

Check yourself

1. Which join lists all customers even if they have no orders?

Answer: `customers LEFT JOIN orders`.

1. How many rows does `CROSS JOIN` give for 8 and 6 rows?

Answer: 48.

1. Why `COUNT(o.order_id)` and not `COUNT(*)`?

Answer: To skip the NULL row of unmatched customers, so they count as 0.

4.3 Anti-joins and Semi-joins (and LATERAL)

INTUITION

Sometimes you do not want to *attach* data. You only want to *test* whether a match exists. "Which customers have never ordered?" is a test for "no match". "Which customers have ordered at least once?" is a test for "some match". These tests are called the **anti-join** (no match) and the **semi-join** (some match). They never add columns and never duplicate rows.

Definitions

- **Semi-join**: keep a left row if **at least one** right row matches. Written with `EXISTS` (or `IN`).
- **Anti-join**: keep a left row if **no** right row matches. Written with `NOT EXISTS`, or `LEFT JOIN ... WHERE right.key IS NULL`. Avoid `NOT IN` when NULLs are possible (Chapter 2, Problem 2.4.4).
- **Correlated subquery**: an inner query that refers to a column of the outer query, such as `o.customer_id = c.customer_id`. It runs "for each outer row" (conceptually).

Real-world mapping

Pattern	Question	Where it appears in the job
Anti-join	customers with no orders	churn lists, orphan-record checks
Anti-join	orders whose customer_id is missing in customers	data-quality (referential integrity)

Pattern	Question	Where it appears in the job
Semi-join	customers who bought electronics	audience selection

Worked examples

Problem 4.3.1 Find customers who have never ordered, three ways. Predict: all three must return Sana and Tom.

```
-- 1. NOT EXISTS
SELECT c.name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
name
----
Sana
Tom
```

```
-- 2. LEFT JOIN ... IS NULL
SELECT c.name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

```
name
----
Sana
Tom
```

```
-- 3. NOT IN (safe here because orders.customer_id is NOT NULL)
SELECT c.name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders);
```

```
name
----
Sana
Tom
```

- **Check** `8 customers − 6 customers with orders = 2`. Customers 7 and 8.
- **Answer** Sana and Tom, in all three queries.
- **Meaning** `NOT EXISTS` and `LEFT JOIN ... IS NULL` are always safe. `NOT IN` is safe only if the subquery column can never be NULL. Most databases plan the first two equally well.

Problem 4.3.2 Semi-join: customers who ordered at least one product in the Electronics category. Compare with the inner-join version, which produces duplicates.

Step 1	`EXISTS` asks "does at least one such line exist?" for each customer.
Step 2	The join version returns one row per matching *line*, so the same customer repeats.

```
SELECT c.name FROM customers c
WHERE EXISTS (
  SELECT 1
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  JOIN products p ON p.product_id = oi.product_id
  WHERE o.customer_id = c.customer_id AND p.category = 'Electronics'
)
ORDER BY c.name;
```

```
name
-----
Asha
John
Priya
Ravi
```

```
SELECT c.name
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE p.category = 'Electronics'
ORDER BY c.name;
```

```
name
-----
Asha
Asha
Asha
John
Priya
Ravi
Ravi
Ravi
```

- **Check** Electronics lines: Keyboard in orders 101 (Asha), 105 (John), 109 (Priya), 114 (Ravi); Mouse in 102 (Ravi), 106 (Asha), 111 (Asha), 114 (Ravi). That is 8 lines, but only 4 different people.
- **Answer** `EXISTS` returns 4 names: Asha, John, Priya, Ravi. The join returns 8 rows, with repeats.
- **Meaning** Use `EXISTS` when the question is "who has at least one", and a join when you need the details of each match.

Problem 4.3.3 `IN` versus `EXISTS`: do they differ? For a non-NULL subquery they return the same rows.

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE status = 'refunded');
```

```
name
-----
Priya
```

- **Answer** Priya, the only customer with a refunded order (109).
- **Check** Orders table: only 109 is refunded, and it belongs to customer 5.

LATERAL (not in SQLite)

A `LATERAL` join lets the right side refer to the current left row, like a loop. The classic use: "the latest 2 orders of each customer". PostgreSQL syntax, standard in PostgreSQL, MySQL 8 and Snowflake; SQL Server calls it `CROSS APPLY` / `OUTER APPLY`.

```
-- PostgreSQL syntax, not run here
SELECT c.name, x.order_id, x.order_date
FROM customers c
LEFT JOIN LATERAL (
  SELECT o.order_id, o.order_date
  FROM orders o
  WHERE o.customer_id = c.customer_id
  ORDER BY o.order_date DESC
  LIMIT 2
) x ON TRUE;
```

SQLite has no `LATERAL`. The portable way is a window function with `ROW_NUMBER` (Chapter 8). Here is the same answer in SQLite (a preview, explained later):

```
SELECT name, order_id, order_date FROM (
  SELECT c.name, o.order_id, o.order_date,
         ROW_NUMBER() OVER (PARTITION BY c.customer_id ORDER BY o.order_date DESC) AS rn
  FROM customers c JOIN orders o ON o.customer_id = c.customer_id
)
WHERE rn <= 2
ORDER BY name, order_date DESC;
```

```
name   order_id  order_date
-----  --------  ----------
Asha   111       2024-04-02
Asha   106       2024-03-05
John   112       2024-04-09
John   105       2024-02-20
Kabir  110       2024-03-30
Meena  104       2024-02-10
Priya  113       2024-04-15
Priya  109       2024-03-25
Ravi   114       2024-04-20
Ravi   108       2024-03-18
```

Connection to the real job

Referential integrity checks ("orders whose customer does not exist") are anti-joins. Audience building ("users who did X but not Y") is a semi-join combined with an anti-join. These come up in almost every analytics interview.

Applications

- **Data Analytics:** churned users, never-activated accounts, "bought A but not B".
- **Data Engineering:** orphan-row detection, incremental loads ("rows in source not yet in target").
- **Machine Learning:** building negative samples ("users who saw the ad but did not click").

COMMON MISTAKE	Using an inner join to answer a "has at least one" question and then counting rows. The count is inflated by duplicates. Use `EXISTS`, or `COUNT(DISTINCT key)`.

Check yourself

1. Which two anti-join forms are always NULL-safe?

Answer: `NOT EXISTS` and `LEFT JOIN ... WHERE right.key IS NULL`.

1. Why can a join duplicate customers when the question is "has at least one order"?

Answer: A join returns one row per matching order.

1. What is the SQL Server name of `LATERAL`?

Answer: `CROSS APPLY` / `OUTER APPLY`.

4.4 Join Cardinality and the Duplicate Pitfall

INTUITION	In a school, one teacher teaches many students. If you attach the teacher's salary to every student row and then add up the salary column, you pay the teacher once per student. Joining a parent to a child *repeats* the parent's values. Summing them afterwards over-counts. This is the **fan-out** problem.

Words you need

- **Cardinality**: how many rows on one side match one row on the other (1:1, 1:N, M:N).
- **Fan-out**: the row count grows after a join because a row matched several partners.
- **Grain**: what one row of the result means (from Chapter 3).

The rule of thumb

Before and after every join, ask: "How many rows should I have?" and **count them**. Joining a 1:N relationship from the 1 side multiplies the 1-side values. Joining N:M on a non-unique key explodes: the row count becomes the *product* per key.

Worked examples

Problem 4.4.1 Fan-out demo: every order has a flat shipping fee of 50. What is the total shipping charged? Predict: `14 orders × 50 = 700`.

Step 1	Create a `shipping` table with one row per order.
Step 2	Sum fees directly: correct.
Step 3	Join to `order_items` first and then sum: each order counted once per line.

```
CREATE TABLE shipping (order_id INTEGER PRIMARY KEY, fee INTEGER);
INSERT INTO shipping SELECT order_id, 50 FROM orders;

SELECT SUM(fee) AS correct_total FROM shipping;
```

```
correct_total
-------------
700
```

```
SELECT SUM(s.fee) AS inflated_total, COUNT(*) AS rows_after_join
FROM shipping s
JOIN order_items oi ON oi.order_id = s.order_id;
```

```
inflated_total  rows_after_join
--------------  ---------------
1000            20
```

- **Compute** Lines per order: 101 has 2, 102 has 1, 103 has 2, 104 has 1, 105 has 1, 106 has 2, 107 has 1, 108 has 1, 109 has 1, 110 has 2, 111 has 1, 112 has 1, 113 has 2, 114 has 2. Sum is 20 lines. `20 × 50 = 1000`.
- **Check** Rows after join `= 20` (the number of order lines), not 14.
- **Answer** The correct total is 700; the joined sum is 1000, which is 300 too high.
- **Meaning** The query ran without any error and gave a 43 percent over-statement. Fan-out is silent.

Problem 4.4.2 Fix it: aggregate each side to the grain of the join key *before* joining.

Step 1	Summarise `order_items` to one row per order in a derived table (a subquery in `FROM`, Chapter 5).
Step 2	Now the join is 1:1, so no fan-out.

```
SELECT SUM(s.fee) AS shipping_total, SUM(t.items_value) AS items_total, COUNT(*) AS rows_after_join
FROM shipping s
JOIN (SELECT order_id, SUM(qty * unit_price) AS items_value
      FROM order_items GROUP BY order_id) t ON t.order_id = s.order_id;
```

```
shipping_total  items_total  rows_after_join
--------------  -----------  ---------------
700             14560        14
```

- **Check** 14 rows, shipping `14 × 50 = 700`, items total 14560 (same as Chapter 3).
- **Answer** Both totals are right at the same time.
- **Meaning** Rule: when you need to sum two things from different grains, aggregate each to the common key, then join.

Problem 4.4.3 Many-to-many explosion: join `customers` to itself on `city` (a non-unique key). How many rows? Predict first.

Step 1	Each city with `k` customers gives `k × k` pairs.
Step 2	Pune has 3, Mumbai 2, Delhi 2, Bengaluru 1: `9 + 4 + 4 + 1 = 18`.

```
SELECT COUNT(*) AS pairs
FROM customers a JOIN customers b ON a.city = b.city;
```

```
pairs
-----
18
```

- **Check** $3^2 + 2^2 + 2^2 + 1^2 = 18$.
- **Answer** 18 rows from 8 customers.
- **Meaning** Joining on a non-unique key on both sides multiplies. If you see the row count jump, check whether the key is unique with `COUNT(*)` versus `COUNT(DISTINCT key)`.

Problem 4.4.4 A safety check you can reuse: verify a key is unique before you rely on it.

```
SELECT COUNT(*) AS n_rows, COUNT(DISTINCT order_id) AS n_distinct_keys FROM orders;
```

```
n_rows  n_distinct_keys
------  ---------------
14      14
```

```
SELECT COUNT(*) AS n_rows, COUNT(DISTINCT order_id) AS n_distinct_keys FROM order_items;
```

```
n_rows  n_distinct_keys
------  ---------------
20      14
```

- **Answer** In `orders` both numbers are 14, so `order_id` is a key. In `order_items` it is 20 rows and 14 distinct values, so `order_id` alone is **not** a key there.
- **Meaning** The first number equals the second only for a unique key. Run this test on every new table you join.

The picture

```
orders (1)          order_items (N)            joined result
+-----+-----+       +-----+-----+------+       +-----+-----+------+-----+
| 101 | 50  |       | 101 | 1   | 1500 |       | 101 | 50  | 1    |1500 |
+-----+-----+       | 101 | 3   |  160 |       | 101 | 50  | 3    | 160 |  <- 50 repeated
                    +-----+-----+------+       +-----+-----+------+-----+
SUM(fee) over the joined result = 50 + 50 = 100, but the real fee is 50.
```

Connection to the real job

Wrong revenue from fan-out is a classic production incident. For example: joining a one-row-per-order table to a one-row-per-order-item table and then summing a per-order value (such as shipping fee) counts it once per item, so the number comes out too large. Experienced analysts always reconcile: "does my total match the total from the source table?"

Applications

- **Data Analytics:** reconciling revenue between dashboards.
- **Data Engineering:** unit tests on pipelines (row counts before and after joins).
- **Data Science:** avoiding data leakage and duplicated training rows after feature joins.

COMMON MISTAKE	Trusting a total after a join without a sanity check. Always compare the result to a total computed from one table, and compare row counts before and after.

Check yourself

1. A 1:N join from the 1 side. What happens to 1-side values?

Answer: They repeat once per matching child row.

1. How do you test that a column is a key?

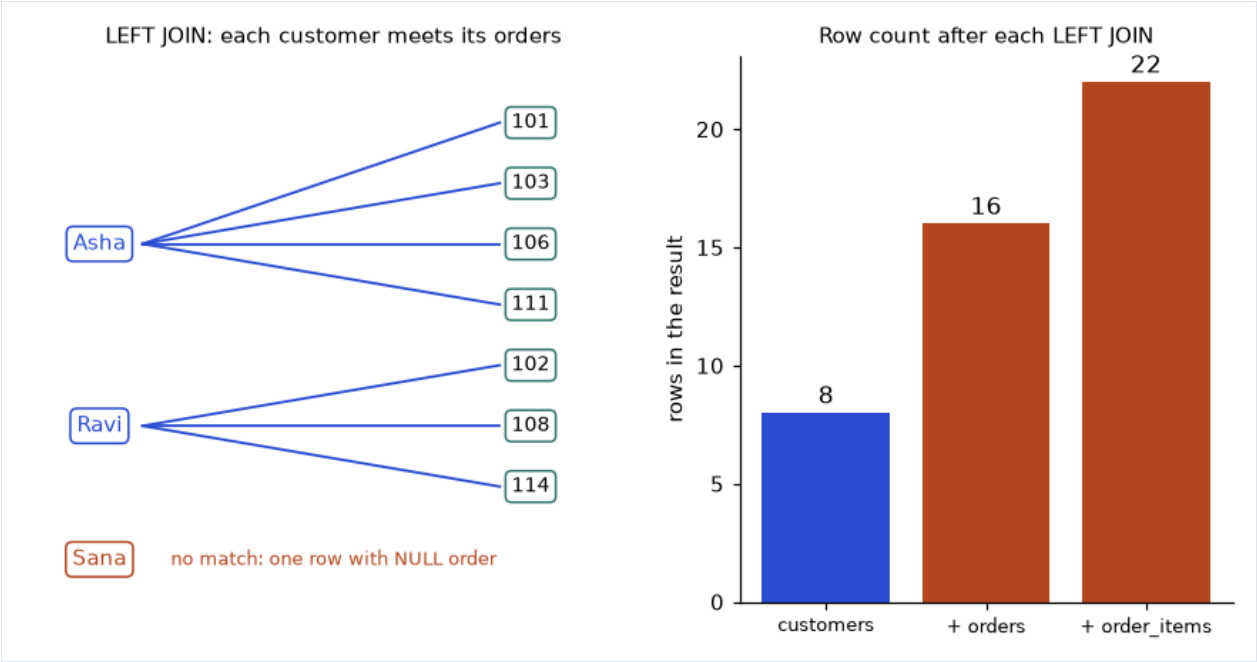
Answer: `COUNT(*)` equals `COUNT(DISTINCT col)` and no NULLs.

1. What is the fix for summing two grains?

Answer: Aggregate each side to the join key first, then join.

4.5 Seeing a Join: Row Matching and Row Counts After Each Join

INTUITION	A join does not *merge* tables. It creates one output row for every pair of rows that satisfy the `ON` test. If one customer has four orders, that customer appears four times in the output, and if each order has two items, the customer appears eight times. Once you see joins as 'one output row per matching pair', the duplicate pitfall of Section 4.4 stops being a surprise and becomes arithmetic you can do in your head.



Left: how a LEFT JOIN matches customers to orders in the shop database. Right: rows in the result after each successive join.

How to read it: on the left each line is one output row. Asha has four lines, Ravi three, and Sana has none, so she gets one row with NULL order columns. On the right the bars count rows: 8 customers become 16 rows once orders are joined (14 orders plus the 2 customers with no orders), and 22 rows once order items are joined (the 20 item lines plus the same 2 customers). The number of rows grows at every one-to-many step.

Problem 4.5.1 Count the rows after each join in the chain customers, orders, order items. Predict first: 8 customers; 14 orders plus the 2 customers without orders (Sana and Tom) gives 16; 20 item lines plus those 2 customers gives 22.

```
SELECT 'customers' AS stage, count(*) AS result_rows FROM customers
UNION ALL
SELECT '+ orders', count(*)
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
UNION ALL
SELECT '+ order_items', count(*)
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
                 LEFT JOIN order_items i ON i.order_id = o.order_id;
```

```
stage          result_rows
-------------  -----------
customers      8
+ orders       16
+ order_items  22
```

Step 1	Each stage is the count of the same chain stopped one join earlier. The first is just the customer table, 8 rows.
Step 2	After joining orders: every order matches its customer, giving 14 rows, plus one row each for the customers with no order. Customers 7 and 8 have none, so 14 + 2 = 16.
Step 3	After joining items: each order row becomes one row per item line. There are 20 item lines in total, all on real orders, so 20 + 2 unmatched customers = 22.
Rule	In a chain of `LEFT JOIN` s the row count equals the count of the most-detailed table that has matches, plus the unmatched rows from above. Compute it before you run the query, and compare.

Now the damage. Ask a seemingly innocent question after the three-table join: how many customers live in each city?

```
SELECT c.city, count(*) AS rows_counted, count(DISTINCT c.customer_id) AS real_customers
FROM customers c
LEFT JOIN orders o      ON o.customer_id = c.customer_id
LEFT JOIN order_items i ON i.order_id = o.order_id
GROUP BY c.city ORDER BY c.city;
```

```
city       rows_counted  real_customers
---------  ------------  --------------
Bengaluru  4             1
Delhi      3             2
Mumbai     6             2
Pune       9             3
```

```
SELECT c.city, count(*) AS customers
FROM customers c GROUP BY c.city ORDER BY c.city;
```

```
city       customers
---------  ---------
Bengaluru  1
Delhi      2
Mumbai     2
Pune       3
```

Step 1	`count(*)` after the joins counts *result rows*, not customers. Pune shows 9, Mumbai 6, Bengaluru 4, Delhi 3, which are the sums of each customer's item lines (or 1 for customers without orders).
Step 2	The correct customer counts are in the second result: Pune 3, Mumbai 2, Bengaluru 1, Delhi 2. The first query's `count(DISTINCT customer_id)` column agrees with them, which is the cheap fix when you must keep the joins.
Answer	Before aggregating a joined result, ask: what is one row of this result? If it is an order line, then any customer-level number summed over rows is multiplied by the number of lines.

COMMON MISTAKE

Using `count(*)` or `sum(customer_level_column)` after joining down to a finer grain. The three fixes, best first: (1) aggregate at the right grain in a subquery or CTE *before* joining, (2) do not join the finer table at all if you do not use its columns, (3) as a last resort use `count(DISTINCT key)`. A `SUM` has no `DISTINCT` fix that is safe, because two different orders can have the same amount and `sum(DISTINCT amount)` would drop one.

A habit that catches fan-out in ten seconds

After every join in a long query, add a temporary `SELECT count(*), count(DISTINCT <the grain key>)`. If the two numbers differ and you did not intend that, a join multiplied rows. Also keep a known total (here, the sum of all paid lines is 12,660) and compare it with the same total computed from your final result.

Practice with answers

Problem 4.5.2 Customers (8 rows) `LEFT JOIN` orders (14 rows, each belonging to one of 6 customers). How many rows does the result have? *(easy)*

Answer: 14 matched rows plus 2 customers with no order: 16. The result has at least as many rows as the left table in a `LEFT JOIN`.

Problem 4.5.3 The same customers `INNER JOIN` orders: how many rows, and which customers vanish? *(easy)*

Answer: 14 rows. Sana and Tom (customer ids 7 and 8) disappear because an inner join keeps only pairs that match.

Problem 4.5.4 A `CROSS JOIN` of 8 customers and 6 products returns how many rows, and when is that useful? *(medium)*

Answer: 8 x 6 = 48. It is useful for building a complete grid (every customer with every product, then `LEFT JOIN` actual sales to find the zeros), and dangerous when you forget the `ON` clause by accident.

Chapter recap

- A join matches rows from two tables using a key in `ON`.
- `INNER JOIN` keeps only matched pairs; `LEFT JOIN` keeps all left rows and fills NULL.
- `RIGHT JOIN` is a flipped `LEFT JOIN`; `FULL JOIN` keeps everything and is ideal for reconciliation.
- `CROSS JOIN` makes all pairs (`m × n`); use it to build grids.
- A self join uses two aliases of the same table (referrals, hierarchies).
- After a `LEFT JOIN`, use `COUNT(right_key)` and put right-table filters in `ON`.
- `EXISTS` and `NOT EXISTS` give semi- and anti-joins without duplicates; beware `NOT IN` with NULLs.
- Joins can multiply rows (fan-out). Count rows before and after, and aggregate to the key first.

Need	Pattern
All left rows	`A LEFT JOIN B ON ...`
No match	`WHERE NOT EXISTS (SELECT 1 FROM B WHERE B.k = A.k)`
At least one match	`WHERE EXISTS (...)`
Two lists compared	`A FULL JOIN B ON ...`
All pairs	`A CROSS JOIN B`
Is it a key?	`COUNT(*) = COUNT(DISTINCT k)`

Practice set

Problem 4.P.1 (easy) How many rows does `orders JOIN customers ON ...` return?

```
SELECT COUNT(*) AS n FROM orders o JOIN customers c ON c.customer_id = o.customer_id;
```

```
n
--
14
```

Answer: 14. Every order has exactly one customer (many-to-one), so no fan-out.

Problem 4.P.2 (easy) List products with their category and the number of order lines, including the Mug.

```
SELECT p.name, COUNT(oi.order_id) AS n_lines
FROM products p LEFT JOIN order_items oi ON oi.product_id = p.product_id
GROUP BY p.product_id, p.name ORDER BY n_lines DESC, p.name;
```

```
name       n_lines
---------  -------
Notebook   5
Keyboard   4
Mouse      4
Pen Pack   4
Desk Lamp  3
Mug        0
```

Answer: Notebook 5 lines, Pen Pack 4, Mouse 4, Keyboard 4, Desk Lamp 3, Mug 0 (they add to 20). Read the printed table to confirm.

Problem 4.P.3 (what is the output?) What does this return?

```
SELECT COUNT(*) AS n FROM customers c CROSS JOIN products p;
```

```
n
--
48
```

Answer: 48, because `8 × 6`.

Problem 4.P.4 (find the bug) A student wants all customers with their paid orders, including customers with none, and writes the query below. Which customers are lost and why? Fix it.

```
SELECT c.name, o.order_id
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'paid'
ORDER BY c.name, o.order_id;
```

```
name   order_id
-----  --------
Asha   101
Asha   103
Asha   106
Asha   111
John   105
John   112
Kabir  110
Priya  107
Priya  113
Ravi   102
Ravi   108
Ravi   114
```

```
SELECT c.name, o.order_id
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status = 'paid'
WHERE c.customer_id IN (3, 7, 8)
ORDER BY c.name;
```

```
name   order_id
-----  --------
Meena  NULL
Sana   NULL
Tom    NULL
```

Answer: The first query loses Meena, Sana and Tom, because `WHERE o.status = 'paid'` removes their NULL rows. The fix moves the condition into `ON`. The second query shows the three customers kept, each with a NULL order.

Problem 4.P.5 Customers who have placed orders in both February and April.

```
SELECT c.name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id AND o.order_date LIKE '2024-02%')
  AND EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id AND o.order_date LIKE '2024-04%');
```

```
name
----
Asha
John
```

Answer: Asha (103 in Feb, 111 in Apr) and John (105 in Feb, 112 in Apr).

Problem 4.P.6 For each order, show the customer name and the number of lines, for orders with more than one line.

```
SELECT o.order_id, c.name, COUNT(*) AS n_lines
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, c.name
HAVING COUNT(*) > 1
ORDER BY o.order_id;
```

```
order_id  name   n_lines
--------  -----  -------
101       Asha   2
103       Asha   2
106       Asha   2
110       Kabir  2
113       Priya  2
114       Ravi   2
```

Answer: Orders 101, 103, 106, 110, 113 and 114 have two lines each (6 orders).

Problem 4.P.7 List customers together with the name of the product they bought most units of (just show the join work: total units per customer and product, for customer 1).

```
SELECT p.name, SUM(oi.qty) AS units
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.customer_id = 1
GROUP BY p.name ORDER BY units DESC, p.name;
```

```
name       units
---------  -----
Mouse      3
Notebook   3
Desk Lamp  1
Keyboard   1
Pen Pack   1
```

Answer: Asha bought Mouse 3 units (106 has 2, 111 has 1), Notebook 3 units (101 has 2, 106 has 1), then Desk Lamp, Keyboard and Pen Pack with 1 each. Ties exist, so the second sort key `p.name` makes the order stable.

Problem 4.P.8 (scenario) A manager says: "Revenue by month went up 40 percent after the new join in the dashboard." You suspect fan-out. What two checks do you run? **Answer:** (1) Compare the dashboard total with the total from the source fact table for the same period. (2) Compare `COUNT(*)` before and after each join; if rows grew, find the non-unique key. Fix by aggregating to the key before joining.

Problem 4.P.9 (interview) What is the difference between `WHERE` and `ON` in a `LEFT JOIN`? **Answer:** `ON` decides which right rows match. Left rows with no match are still kept. `WHERE` runs after the join and removes rows, including the NULL-filled left rows, so a right-side filter in `WHERE` turns the left join into an inner join.

Problem 4.P.10 (interview) Find customers who never placed an order, and explain why `NOT IN` is risky. **Answer:** Use `NOT EXISTS` or `LEFT JOIN ... WHERE o.order_id IS NULL` (see 4.3.1). `NOT IN` returns no rows at all if the subquery returns a single NULL, because `x <> NULL` is UNKNOWN.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my second language. Teach me Chapter 4 topic: "SQL joins: inner, left, right, full, cross, self". Use two tiny tables of customers and orders and show the result of each join as a small table. After each join type ask me to predict the output before you reveal it, and check my understanding.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am learning SQL. Topic: join fan-out and duplicate rows after joins. Explain with a small example how a 1-to-many join repeats values and inflates SUM. Then give me 8 graded exercises where I must find the bug or fix the query, without solutions at first. Review my answers when I paste them.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a beginner in SQL. Quiz me one question at a time on anti-joins and semi-joins: NOT EXISTS, EXISTS, LEFT JOIN with IS NULL, IN, NOT IN and the NULL trap. Use a customers and orders schema. Wait for each answer, grade it, explain in simple English, and increase difficulty gradually. End with a score.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for analyst interviews. Review the multi-table SQL query I paste below. Check join types, ON versus WHERE for left joins, COUNT star after left join, duplicate rows from fan-out, and missing keys. Explain every issue in simple English and give a fixed query. My query: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am preparing for data-analyst interviews as a working professional moving into data roles. Run a mock interview on SQL joins: customers with no orders, products never sold, users who did A but not B, reconciliation between two tables, and wrong totals after joins. Ask one question at a time, grade me out of 10, show the model query, and give one tip each time.
```

CHAPTER 5

SUBQUERIES AND CTES (INCLUDING RECURSIVE CTES)

Building a big answer from small steps: queries inside queries, named steps with WITH, and loops that walk through trees and graphs.

Real questions have steps. "Which customers spent more than the average customer?" needs two answers: first the average, then the comparison. "Show the whole reporting chain under the CEO" needs a query that repeats until it reaches the bottom. SQL gives you three tools for this: **subqueries** (a query inside a query), **CTEs** (named steps written with `WITH`) and **recursive CTEs** (a step that feeds itself). After this chapter you can break any hard question into small readable steps, test each step alone, and handle trees such as org charts or category hierarchies.

5.1 Subqueries: Scalar, Derived Table, Correlated

INTUITION

When you cook, you do not do everything at once. You first chop onions, then fry them. A subquery is "do this small job first, and use its result in the main job". The result can be one value (the average), a list (customer ids), or a whole small table.

Words you need

- **Subquery**: a `SELECT` written inside brackets inside another query.
- **Scalar subquery**: returns exactly **one value** (one row, one column). It can be used anywhere a number can.
- **Derived table**: a subquery in `FROM` that returns a table. It must have an alias.
- **Correlated subquery**: refers to a column of the outer query, so it is (conceptually) re-run for every outer row.
- **Uncorrelated subquery**: stands alone; it runs once.

Real-world mapping

Kind	Example	Where it appears in the job
Scalar	`price > (SELECT AVG(price) FROM products)`	compare each row with a global benchmark
Derived table	`FROM (SELECT customer_id, SUM(...) ... GROUP BY ...) t`	aggregate first, then filter or join
`IN (subquery)`	`customer_id IN (SELECT ...)`	audience lists
Correlated	`price > (SELECT AVG(price) ... WHERE category = p.category)`	compare with the row's own group

Worked examples

Problem 5.1.1 (trivial) Products priced above the average product price. Predict: the average is `3450 / 6 = 575`. Which products exceed it?

Step 1	The inner query returns one number: `SELECT AVG(price) FROM products`.
Step 2	The outer query compares each price to it.

```
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

```
name       price
---------  -----
Keyboard   1500
Desk Lamp  900
Mouse      600
```

- **Check** Prices: 1500, 600, 80, 120, 900, 250. Above 575: 1500, 600, 900.
- **Answer** Keyboard, Desk Lamp and Mouse.
- **Meaning** You never had to type the number 575. If prices change, the query adapts.

Problem 5.1.2 (derived table) Which customers spent more than the *average paying customer*, counting paid orders only?

Step 1	Inner query: total paid spend per customer. It returns a small table (5 customers have paid orders).
Step 2	Take the average of those totals: `12660 / 5 = 2532`.
Step 3	Keep customers above 2532.

```
SELECT t.customer_id, t.spent
FROM (
  SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY o.customer_id
) AS t
WHERE t.spent > (SELECT AVG(spent) FROM (
                   SELECT SUM(oi.qty * oi.unit_price) AS spent
                   FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
                   WHERE o.status = 'paid' GROUP BY o.customer_id))
ORDER BY t.spent DESC;
```

```
customer_id  spent
-----------  -----
1            4560
2            3600
4            3300
```

- **Compute** Customer spends: 4560, 3600, 3300, 760, 440. Sum 12660. Average `12660 / 5 = 2532`.
- **Check** 4560, 3600 and 3300 are above 2532; 760 and 440 are not.
- **Answer** Customers 1, 2 and 4.
- **Meaning** The query repeats the same inner block twice. That hurts readability, and section 5.2 shows how a CTE fixes it.

Problem 5.1.3 (correlated) Which products cost more than the average price of **their own category**?

Step 1	For each product `p`, the inner query averages the prices where the category equals `p.category`. The link `p.category` is what makes it correlated.
Step 2	Compare.

```
SELECT p.name, p.category, p.price
FROM products p
WHERE p.price > (SELECT AVG(p2.price) FROM products p2 WHERE p2.category = p.category)
ORDER BY p.category;
```

```
name       category     price
---------  -----------  -----
Keyboard   Electronics  1500
Desk Lamp  Home         900
Pen Pack   Stationery   120
```

- **Compute** Electronics: `(1500 + 600) / 2 = 1050`, so the Keyboard (1500) is above. Stationery: `(80 + 120) / 2 = 100`, so the Pen Pack (120) is above. Home: `(900 + 250) / 2 = 575`, so the Desk Lamp (900) is above.
- **Check** Each category with two products has exactly one product above its average, which is always true for two different prices.
- **Answer** Keyboard, Pen Pack, Desk Lamp.
- **Meaning** Correlated subqueries are easy to read but, on huge tables, can be slow because they conceptually run once per row. Window functions (Chapter 8) often do the same job faster.

Problem 5.1.4 `IN` versus `EXISTS`, and a scalar subquery in `SELECT`. Show each customer's name with their number of orders, using a scalar correlated subquery.

```
SELECT c.name,
       (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS n_orders
FROM customers c
ORDER BY n_orders DESC, c.name;
```

```
name   n_orders
-----  --------
Asha   4
Priya  3
Ravi   3
John   2
Kabir  1
Meena  1
Sana   0
Tom    0
```

- **Check** 4, 3, 3, 2, 1, 1, 0, 0 add to 14 orders.
- **Answer** Asha 4; Priya and Ravi 3; John 2; Kabir and Meena 1; Sana and Tom 0.
- **Meaning** This gives zero for Sana and Tom without a `LEFT JOIN`. It is simple, but it runs the inner query for every customer.

IN versus EXISTS: which one?

- `IN (subquery)` is clear for a short list of values. The risk: `NOT IN` with NULLs (Chapter 2).
- `EXISTS (subquery)` stops at the first match and handles NULLs safely. Modern optimisers (PostgreSQL, SQL Server, MySQL 8) usually produce the same plan for both when they are logically equal. Use `EXISTS` / `NOT EXISTS` for "has / has no" questions, `IN` for readability with small lists.

The picture

```
 outer query:  SELECT ... FROM products p WHERE p.price > ( ??? )
                                                          |
                         inner query runs, returns ONE number (575)
                         --------------------------------------
 uncorrelated: runs once         -> 575 for every row
 correlated:   runs per outer row -> 1050 for Electronics, 100 for Stationery, 575 for Home
```

Connection to the real job

Comparing "each row with a benchmark" is everywhere: users above average spend, deliveries slower than the overall 95th percentile, products priced above category average. Interviews love the "second highest salary" question, which is a scalar subquery: `SELECT MAX(x) FROM t WHERE x < (SELECT MAX(x) FROM t)`.

Applications

- **Data Analytics:** compare segments with global or group benchmarks.
- **Data Engineering:** filtering a big table by a small list of keys (`IN`).
- **Data Science:** outlier detection with group averages.
- **Logistics:** find deliveries slower than the overall average.

COMMON MISTAKE	A scalar subquery that returns more than one row. PostgreSQL stops with `more than one row returned by a subquery used as an expression` . SQLite silently uses the first row. Always make sure the inner query returns a single value (an aggregate or a unique key).

Check yourself

1. What does "correlated" mean?

Answer: The inner query uses a column from the outer query.

1. Must a derived table have an alias?

Answer: Yes, in PostgreSQL and most databases.

1. What does a scalar subquery return?

Answer: One row and one column.

5.2 CTEs: Naming Your Steps with WITH

INTUITION	In Problem 5.1.2 we copied the same inner query twice. It was hard to read. Imagine a recipe where steps are numbered and each step has a name: "Step 1: chopped_onions". A **CTE** (Common Table Expression) is exactly that: you give a query a name at the top, then use the name below like a table.

How it works

```
WITH step1 AS ( SELECT ... ),
     step2 AS ( SELECT ... FROM step1 ... )
SELECT ... FROM step2 ...;
```

- The CTE exists only for that one statement. It is not stored.
- A later CTE can use earlier ones (**chained CTEs**). Each step is a small query you can test on its own by running `SELECT * FROM step1` .
- You can use one CTE several times in the main query (no copy-paste).
- In PostgreSQL 12 and later, a CTE used once is normally inlined like a subquery; a CTE used twice may be computed once. You can force it with `MATERIALIZED` or `NOT MATERIALIZED` . In older versions every CTE was a "fence" computed once. This matters for speed, not for correctness.

Real-world mapping

Step name	What it holds	Where it appears in the job
`paid_lines`	order lines of paid orders	cleaning step: filter bad rows first
`customer_spend`	one row per customer	aggregation step
`final`	ranking or share of total	presentation step

Worked examples

Problem 5.2.1 (trivial) Rewrite 5.1.2 with a CTE. Same answer, easier to read.

```
WITH customer_spend AS (
  SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY o.customer_id
)
SELECT customer_id, spent
FROM customer_spend
WHERE spent > (SELECT AVG(spent) FROM customer_spend)
ORDER BY spent DESC;
```

```
customer_id  spent
-----------  -----
1            4560
2            3600
4            3300
```

- **Check** Same output as 5.1.2: customers 1, 2, 4 with 4560, 3600, 3300.
- **Answer** The CTE `customer_spend` is defined once and used twice.
- **Meaning** Same logic, half the text, and each step has a name that tells the reader what it is.

Problem 5.2.2 (chained CTEs) For every paying customer show the customer's share of total paid revenue, in percent.

Step 1	`paid_totals` : revenue per customer.
Step 2	`grand` : the total across all customers.
Step 3	Divide: `spent / total × 100` .

```
WITH paid_totals AS (
  SELECT o.customer_id, SUM(oi.qty * oi.unit_price) AS spent
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY o.customer_id
),
grand AS (
  SELECT SUM(spent) AS total FROM paid_totals
)
SELECT p.customer_id, p.spent, g.total,
       ROUND(100.0 * p.spent / g.total, 1) AS share_pct
FROM paid_totals p CROSS JOIN grand g
ORDER BY p.spent DESC;
```

```
customer_id  spent  total  share_pct
-----------  -----  -----  ---------
1            4560   12660  36.0
2            3600   12660  28.4
4            3300   12660  26.1
5            760    12660  6.0
6            440    12660  3.5
```

- **Compute** `4560 / 12660 = 0.3602` , so 36.0 percent. `3600 / 12660 = 28.4` . `3300 / 12660 = 26.1` . `760 / 12660 = 6.0` . `440 / 12660 = 3.5` .
- **Check** `36.0 + 28.4 + 26.1 + 6.0 + 3.5 = 100.0` .

- **Answer** Customer 1 36.0 percent, customer 2 28.4, customer 4 26.1, customer 5 6.0, customer 6 3.5.
- **Meaning** The top three customers bring about 90 percent of paid revenue. (In Chapter 8, `SUM() OVER ()` computes the same share in one step.)

Problem 5.2.3 (combine ideas) The monthly revenue, and the month with the highest revenue. Use two CTEs and a filter on the max.

```
WITH monthly AS (
  SELECT substr(o.order_date,1,7) AS month, SUM(oi.qty * oi.unit_price) AS revenue
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'paid'
  GROUP BY substr(o.order_date,1,7)
)
SELECT month, revenue
FROM monthly
WHERE revenue = (SELECT MAX(revenue) FROM monthly);
```

```
month    revenue
-------  -------
2024-04  4900
```

- **Check** From Chapter 3: paid revenue by month is 2260, 2520, 2980, 4900. The maximum is April's 4900.
- **Answer** 2024-04 with 4900.
- **Meaning** If two months tied for the max, both would appear, which is usually what you want.

The picture

```
 orders+items --> [paid_totals] --> [grand] --\
                       |                      +--> final SELECT
                       +----------------------/
 Each box is a named step. You can run SELECT * FROM <box> to test it alone.
```

Connection to the real job

Production SQL (dbt models, dashboards) is written as long chains of CTEs: `source`, `cleaned`, `joined`, `aggregated`, `final`. Reviewers read top to bottom. Style rules used by many teams: lowercase keywords or uppercase, consistent; one CTE per idea; descriptive CTE names (`paid_orders`, not `a`, `b`); comments with `--` on non-obvious logic; and move repeated logic into a **view** (Chapter 9).

Applications

- **Data Analytics:** readable multi-step reports.
- **Data Engineering:** dbt models are chains of CTEs.
- **Data Science:** feature queries built step by step.
- **Interviews:** interviewers prefer CTEs to deeply nested subqueries because they are easier to explain.

COMMON MISTAKE	Assuming a CTE is faster than a subquery. It is mainly a readability tool. Also, in some databases a CTE used many times can be recomputed each time. Check the plan (Chapter 10) if speed matters.

Check yourself

1. Can a CTE be used after the statement ends?

Answer: No. It lives for one statement only.

1. How do you test a middle step?

Answer: Replace the final select with `SELECT * FROM that_step`.

1. What is a chained CTE?

Answer: A CTE that uses an earlier CTE.

5.3 Recursive CTEs

INTUITION

To list everyone under the CEO: first find the CEO. Then find the people who report to the CEO. Then find people who report to *those* people. Repeat until nobody new appears. That is a loop. Normal SQL has no loop, so SQL gives you the **recursive CTE**: a query that uses its own output as input in the next round.

Words you need

- **Anchor member** (base case): the starting rows. Runs once.
- **Recursive member**: refers to the CTE's own name; takes the rows from the previous round and produces new rows.
- **Termination**: the recursion ends when a round produces no new rows.
- **Hierarchy / tree**: rows pointing to a parent (employee to manager).
- **Graph**: nodes joined by edges; may contain cycles.

Shape

```
WITH RECURSIVE name(columns) AS (
    <anchor query>                -- starting rows
    UNION ALL                     -- or UNION to remove duplicates
    <recursive query using name>  -- builds the next layer
)
SELECT ... FROM name;
```

`RECURSIVE` is required in PostgreSQL and MySQL; SQL Server writes just `WITH`. SQLite accepts `WITH RECURSIVE`.

Real-world mapping

Use	Example	Where it appears in the job
Hierarchy	org chart, category tree	reporting lines, product categories
Date series	one row per day	calendar tables for gap-free charts
Graph traversal	routes, dependencies	lineage of tables in a data warehouse
Number series	1 to N	generating test data

Worked examples

Problem 5.3.1 (trivial) Count from 1 to 5, and also compute factorials, to see how rounds work.

Step 1	Anchor: `n = 1, f = 1`.
Step 2	Recursive: `n + 1`, `f × (n + 1)`, while `n < 5`. The `WHERE` is the stopping rule.

```
WITH RECURSIVE t(n, f) AS (
  SELECT 1, 1
  UNION ALL
  SELECT n + 1, f * (n + 1) FROM t WHERE n < 5
)
SELECT n, f FROM t;
```

```
n  f
-  ---
1  1
2  2
3  6
4  24
5  120
```

- **Check** `1! = 1`, `2! = 2`, `3! = 6`, `4! = 24`, `5! = 120`.
- **Answer** Five rows, one per round.
- **Meaning** Round 0 gives (1,1). Round 1 reads that row and creates (2,2). It continues until the `WHERE` fails and no new row is created.

Problem 5.3.2 (hierarchy) A small company. Show everyone under the CEO with their level and their full chain of command.

Step 1	Create `staff` with `manager_id`. The CEO has NULL.
Step 2	Anchor: the row with `manager_id IS NULL`, level 0.
Step 3	Recursive: join staff to the previous layer where `s.manager_id = tree.id`, level + 1, and build a path string.

```
CREATE TABLE staff (id INTEGER PRIMARY KEY, name TEXT, manager_id INTEGER);
INSERT INTO staff VALUES
 (1,'Meera',NULL),(2,'Arun',1),(3,'Bina',1),
 (4,'Chetan',2),(5,'Divya',2),(6,'Esha',3),(7,'Farid',4);

WITH RECURSIVE tree(id, name, level, path) AS (
  SELECT id, name, 0, name FROM staff WHERE manager_id IS NULL
  UNION ALL
  SELECT s.id, s.name, tree.level + 1, tree.path || ' > ' || s.name
  FROM staff s JOIN tree ON s.manager_id = tree.id
)
SELECT id, level, path FROM tree ORDER BY path;
```

```
id  level  path
--  -----  --------------------------
1   0      Meera
2   1      Meera > Arun
4   2      Meera > Arun > Chetan
7   3      Meera > Arun > Chetan > Farid
5   2      Meera > Arun > Divya
3   1      Meera > Bina
6   2      Meera > Bina > Esha
```

- **Check** Meera is level 0. Arun and Bina report to Meera: level 1. Chetan, Divya report to Arun and Esha to Bina: level 2. Farid reports to Chetan: level 3. Seven rows, as in `staff`.
- **Answer** Each person appears once, with the chain such as `Meera > Arun > Chetan > Farid`.
- **Meaning** One query walked four levels deep, though we never said how many levels there were.

Problem 5.3.3 (date series) A day-by-day count of orders for 5 to 11 March 2024, with zeros on empty days.

Step 1	Generate 7 dates with recursion: `date(d, '+1 day')`.
Step 2	`LEFT JOIN` orders by date and count.

```
WITH RECURSIVE days(d) AS (
  SELECT '2024-03-05'
  UNION ALL
  SELECT date(d, '+1 day') FROM days WHERE d < '2024-03-11'
)
SELECT days.d, COUNT(o.order_id) AS n_orders
FROM days LEFT JOIN orders o ON o.order_date = days.d
GROUP BY days.d
ORDER BY days.d;
```

```
d           n_orders
----------  --------
2024-03-05  1
2024-03-06  0
2024-03-07  1
2024-03-08  0
2024-03-09  0
2024-03-10  0
2024-03-11  0
```

- **Check** Orders in this window: 106 on March 5 and 107 on March 7 (108 is on the 18th, outside). So counts are 1, 0, 1, 0, 0, 0, 0.
- **Answer** Seven rows; only March 5 and March 7 have an order.
- **Meaning** Without a date series, a chart would skip the empty days and mislead the viewer. This is the "date spine".

PostgreSQL has `generate_series('2024-03-05'::date, '2024-03-11'::date, interval '1 day')` for this, which is shorter:

```
-- PostgreSQL syntax, not run here
SELECT d::date
FROM generate_series(DATE '2024-03-05', DATE '2024-03-11', INTERVAL '1 day') AS g(d);
```

Problem 5.3.4 (graph with a cycle) A table of one-way links between four services: A to B, B to C, C to D, B to D, D to A. Which services can be reached from A? The graph has a cycle (A, B, C, D, A), so `UNION ALL` would never stop.

Step 1	Use `UNION` (not `UNION ALL`). `UNION` removes duplicate rows, so a node already found is not added again, and the loop ends.
Step 2	Show the safer habit too: a depth limit.

```
CREATE TABLE links (src TEXT, dst TEXT);
INSERT INTO links VALUES ('A','B'),('B','C'),('C','D'),('B','D'),('D','A');

WITH RECURSIVE reach(node) AS (
  SELECT 'A'
  UNION
  SELECT l.dst FROM links l JOIN reach r ON l.src = r.node
)
SELECT node FROM reach ORDER BY node;
```

```
node
----
A
B
C
D
```

```
WITH RECURSIVE walk(node, depth) AS (
  SELECT 'A', 0
  UNION ALL
  SELECT l.dst, w.depth + 1 FROM links l JOIN walk w ON l.src = w.node
  WHERE w.depth < 4
)
SELECT depth, node FROM walk ORDER BY depth, node;
```

```
depth  node
-----  ----
0      A
1      B
2      C
2      D
3      A
3      D
4      A
4      B
```

- **Check** From A you reach B, then C and D, then A again (already found). So the set is A, B, C, D.
- **Answer** All four nodes. The second query shows every walk of length up to 4 (the same node can repeat because routes differ), and the `depth < 4` guard makes sure it stops.
- **Meaning** Always give recursion a stop: either `UNION` on a finite set, or a depth guard. PostgreSQL also has a `CYCLE` clause (version 14 and later) to detect loops.

The picture

```
 round 0 (anchor)   round 1            round 2          round 3
 [Meera]  -------->  [Arun, Bina] --->  [Chetan, Divya,  ---> [Farid]
                                         Esha]
 Each round finds the children of the previous round. Stop when a round is empty.
```

Connection to the real job

Data engineers use recursion for lineage ("which tables depend on this table?"), bill-of-materials explosions and category trees. Analysts use it to build date spines when no calendar table exists. For very large graphs, a graph database (Chapter 13) is better.

Applications

- **Data Analytics:** category roll-ups, org-chart reports.
- **Data Engineering:** lineage and dependency graphs; calendar tables.
- **Data Science:** friend-of-friend features on small graphs.
- **Operations:** train-route chains (which station connects to which).

COMMON MISTAKE	No stop condition. The query loops until the database kills it. Always add `WHERE depth < N` while developing, or use `UNION`. Another mistake: columns in the anchor and the recursive part with different types or order. They must match exactly.

Check yourself

1. What are the two parts of a recursive CTE?

Answer: The anchor (start) and the recursive member (next layer), joined by `UNION [ALL]`.

1. When does recursion stop?

Answer: When a round returns no new rows.

1. Why `UNION` in the cycle example?

Answer: It removes already-seen rows, so the cycle cannot grow forever.

5.4 Choosing the Right Tool, and Writing Readable SQL

INTUITION The same answer can be written five ways. A good engineer picks the way that the next reader (and you, six months later) can understand in one minute.

Decision table

Situation	Best tool
One number used as a filter	scalar subquery
"Has / has no" a match	`EXISTS` / `NOT EXISTS`
Several steps, or reuse one step twice	CTE
Per-row comparison with its group	window function (Chapter 8)
Tree, graph, date series	recursive CTE
Same logic needed by many queries	view (Chapter 9)
Very expensive step reused many times	temporary table (Chapter 9)

Worked examples

Problem 5.4.1 The "second highest price" interview question, three ways. Predict: the prices are 1500, 900, 600, 250, 120, 80, so the second highest is 900.

```
SELECT MAX(price) AS second_highest
FROM products
WHERE price < (SELECT MAX(price) FROM products);
```

```
second_highest
--------------
900
```

```
SELECT price AS second_highest FROM products ORDER BY price DESC LIMIT 1 OFFSET 1;
```

```
second_highest
--------------
900
```

```
WITH ranked AS (
  SELECT price, DENSE_RANK() OVER (ORDER BY price DESC) AS r FROM products
)
SELECT price AS second_highest FROM ranked WHERE r = 2;
```

```
second_highest
--------------
900
```

- **Check** All three print 900.
- **Answer** 900, three ways.
- **Meaning** The `LIMIT/OFFSET` way breaks when prices tie (two products at 1500 would make the answer 1500). The `DENSE_RANK` way is correct with ties, and the subquery way as well.

Problem 5.4.2 Style makeover. Here is hard-to-read SQL:

```
select c.name,sum(oi.qty*oi.unit_price) s from customers c join orders o on o.customer_id=c.customer_id join order_items oi
on oi.order_id=o.order_id where o.status='paid' group by c.name having sum(oi.qty*oi.unit_price)>2000 order by s desc;
```

```
name  s
----  ----
Asha  4560
Ravi  3600
John  3300
```

And the same query, readable:

```
-- Customers whose paid revenue is above 2000, biggest first
SELECT
    c.name,
    SUM(oi.qty * oi.unit_price) AS paid_revenue
FROM customers   AS c
JOIN orders      AS o  ON o.customer_id = c.customer_id
JOIN order_items AS oi ON oi.order_id   = o.order_id
WHERE o.status = 'paid'
GROUP BY c.name
HAVING SUM(oi.qty * oi.unit_price) > 2000
ORDER BY paid_revenue DESC;
```

```
name  paid_revenue
----  ------------
Asha  4560
Ravi  3600
John  3300
```

- **Check** Customers with paid revenue above 2000: Asha 4560, Ravi 3600, John 3300.
- **Answer** Same result. The second is easier to read and review.
- **Meaning** Style rules: one clause per line, aligned joins, meaningful aliases, a comment that states the purpose, uppercase keywords (or consistently lowercase), `AS` for aliases, explicit `ON`.

Connection to the real job

Code review of SQL is a real part of data teams. Clean SQL reduces bugs and onboarding time. Many companies use a formatter (such as `sqlfluff`) in their pipeline.

Applications

- **Data Analytics:** reusable report queries other people can maintain.
- **Data Engineering:** version-controlled SQL models with review.

COMMON MISTAKE	Writing 200-line queries with nested subqueries four levels deep. Break them into CTEs with names that explain each step.

Check yourself

1. Why is `LIMIT 1 OFFSET 1` risky for "second highest"?

Answer: Ties: duplicates of the top value make the answer wrong.

1. Name two style rules for readable SQL.

Answer: Descriptive aliases and one clause per line (also comments, CTEs).

1. When prefer EXISTS over IN?

Answer: When asking has / has no a match, especially when NULLs are possible.

Chapter recap

- A subquery is a query inside brackets; scalar means one value, derived means a table, correlated means it uses the outer row.
- `IN` and `EXISTS` often behave the same; `NOT IN` is dangerous with NULLs.
- A CTE names a step with `WITH`; chains read like a recipe.
- A CTE is mostly about readability; check the plan for speed.
- A recursive CTE has an anchor and a recursive member; it stops when no new rows appear.
- Always give recursion a stop: `UNION`, or a depth limit.
- Use recursion for trees, graphs and date series.
- Prefer clear names, comments and one clause per line.

Need	Pattern
Compare with a benchmark	`WHERE x > (SELECT AVG(x) FROM t)`
Named steps	`WITH a AS (...), b AS (... FROM a ...) SELECT ...`
Hierarchy	`WITH RECURSIVE t AS (anchor UNION ALL step) SELECT ...`
Date series	`WITH RECURSIVE d AS (SELECT date UNION ALL SELECT date(d,'+1 day') ... )`
Second highest	`WHERE x < (SELECT MAX(x) FROM t)` or `DENSE_RANK`

Practice set

Problem 5.P.1 (easy) Customers who signed up before the average signup date. Show names. (Dates are text, so compare using Julian day numbers.)

```
SELECT name, signup_date FROM customers
WHERE julianday(signup_date) < (SELECT AVG(julianday(signup_date)) FROM customers)
ORDER BY signup_date;
```

```
name   signup_date
-----  -----------
Asha   2024-01-05
Ravi   2024-01-12
Meena  2024-01-20
John   2024-02-02
```

Answer: Asha, Ravi, Meena and John. Counting days from 1 January: the signups are day 4, 11, 19, 32, 44, 60, 74 and 91. They add to 335, and `335 / 8 = 41.9`, which is about 11 February. Only the first four signed up before that. (PostgreSQL has no `julianday`; there you can average `EXTRACT(EPOCH FROM signup_date)`.)

Problem 5.P.2 (easy) Customers who have at least one cancelled or refunded order, using `IN`.

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE status <> 'paid')
ORDER BY name;
```

```
name
-----
Meena
Priya
```

Answer: Meena (order 104 cancelled) and Priya (order 109 refunded).

Problem 5.P.3 (what is the output?) What does this recursive query print?

```
WITH RECURSIVE c(n) AS (SELECT 10 UNION ALL SELECT n - 3 FROM c WHERE n > 0)
SELECT n FROM c;
```

```
n
--
10
7
4
1
-2
```

Answer: 10, 7, 4, 1, -2. The test `n > 0` is checked on the previous row: 1 is still above 0, so one more row (-2) is produced; then -2 fails the test and recursion stops.

Problem 5.P.4 (find the bug) A recursive query lists all descendants of a manager, but never ends. What is missing?

```
-- buggy idea: UNION ALL on a table with a cycle, no depth limit
```

Answer: Either a depth guard (`WHERE depth < 10`) or `UNION` so repeated rows are dropped. Data with a manager cycle (A manages B, B manages A) loops forever otherwise.

Problem 5.P.5 Products never ordered, using `NOT EXISTS`.

```
SELECT name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);
```

```
name
----
Mug
```

Answer: Mug.

Problem 5.P.6 For each customer with orders, the date of their first order, using a correlated subquery.

```
SELECT c.name,
       (SELECT MIN(o.order_date) FROM orders o WHERE o.customer_id = c.customer_id) AS first_order
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY first_order;
```

```
name   first_order
-----  -----------
Asha   2024-01-10
Ravi   2024-01-15
Meena  2024-02-10
John   2024-02-20
Priya  2024-03-07
Kabir  2024-03-30
```

Answer: Asha 2024-01-10, Ravi 2024-01-15, Meena 2024-02-10, John 2024-02-20, Priya 2024-03-07, Kabir 2024-03-30.

Problem 5.P.7 Number of employees at each level of the `staff` tree.

```
WITH RECURSIVE tree(id, level) AS (
  SELECT id, 0 FROM staff WHERE manager_id IS NULL
  UNION ALL
  SELECT s.id, t.level + 1 FROM staff s JOIN tree t ON s.manager_id = t.id
)
SELECT level, COUNT(*) AS headcount FROM tree GROUP BY level ORDER BY level;
```

```
level  headcount
-----  ---------
0      1
1      2
2      3
3      1
```

Answer: Level 0: 1, level 1: 2, level 2: 3, level 3: 1. Total 7.

Problem 5.P.8 (scenario) The CFO wants "revenue by month, with a column showing the previous month's revenue". Without window functions, how would you start? **Answer:** Build a CTE `monthly`, then self-join it on "month = previous month". Chapter 8 shows `LAG`, which is simpler. The CTE is the right first step in both approaches.

Problem 5.P.9 (interview) What is the difference between a subquery and a CTE? **Answer:** A CTE is a named subquery defined before the main query, which can be referenced several times and chained. Logically they give the same result. CTEs improve readability and are the only way to write recursion. Performance depends on the optimiser; do not assume a CTE is faster.

Problem 5.P.10 (interview) Explain how a recursive CTE works and how you prevent infinite loops. **Answer:** An anchor query produces the first rows. The recursive part joins the CTE to the base table to produce the next layer, repeated until a round returns nothing. Prevent infinite loops with a depth counter and `WHERE depth < N`, with `UNION` instead of `UNION ALL`, or with PostgreSQL's `CYCLE` clause.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my
second language. Teach me Chapter 5 topic: "subqueries: scalar, correlated, derived tables, IN versus EXISTS". Use a small
```

products and orders example. After each concept ask me to predict the result before you reveal it, and check my understanding with one question.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles. Topic: CTEs (WITH) and chained CTEs. Give me 8 graded exercises where I must break one long question into named steps, without solutions at first. After I paste my answers, review the naming, readability, and correctness, and show a cleaner version.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. Teach me recursive CTEs step by step: anchor member, recursive member, termination, UNION versus UNION ALL, depth limits. I am a beginner. Use an org chart, a date series and a small graph with a cycle. Draw each round as a small table, and after each example ask me a check question.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a career changer preparing for analyst interviews. Review the SQL I paste below. Suggest how to restructure nested subqueries into CTEs, point out correlated subqueries that could be rewritten with joins or window functions, and fix style problems (aliases, formatting, comments). Explain every change in simple English. My SQL: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. Run a mock SQL interview for me on subqueries and CTEs: second highest value, rows above group average, customers with no orders, org-chart traversal, generating a date series. One question at a time, wait for my answer, grade out of 10, show a model answer and the common wrong answer.

CHAPTER 6

CASE, STRINGS, DATES AND TIMES, JSON AND ARRAYS

The everyday tools for cleaning and reshaping values: if-then logic, text surgery, calendar maths, and semi-structured data.

Real data is messy. Names have extra spaces, emails come in capital letters, dates arrive as text, and half the interesting fields hide inside a JSON blob. Before you can count or join anything, you must clean and reshape the values. This chapter gives you the four toolboxes that do it: `CASE` (if-then-else inside SQL), text functions, date functions, and JSON/array functions. SQLite runs most of our examples. When PostgreSQL has a better or different tool (`DATE_TRUNC` , `SPLIT_PART` , `jsonb` , `UNNEST`), we show it and label it "PostgreSQL syntax, not run here".

6.1 CASE WHEN: If-Then-Else Inside a Query

INTUITION

A traffic officer looks at a vehicle and decides: if it is a truck, send it left; if a bus, send it right; otherwise straight. `CASE WHEN` does the same for each row: it tests conditions in order and returns the value of the first one that is true.

Words you need

- **Bucketing** (also *binning*): turning a number into a label such as small, medium, large.
- **Conditional aggregation**: using `CASE` inside `SUM` or `COUNT` (Chapter 3).
- **Manual pivot**: turning row values into columns using conditional aggregation.

Syntax

```
CASE WHEN condition1 THEN value1
     WHEN condition2 THEN value2
     ELSE default_value        -- if omitted, the default is NULL
END
```

Conditions are checked top to bottom; the first TRUE wins. All `THEN` values should have the same type. There is also the "simple" form `CASE column WHEN 'a' THEN ...` , which only tests equality.

Real-world mapping

Use of CASE	Example	Where it appears in the job
Bucketing	order size small / medium / large	customer segments, delivery-time bands
Recoding	`status` codes to labels	cleaning source-system codes
Conditional sum	`SUM(CASE WHEN ... )`	funnels, KPIs by segment
Manual pivot	one column per status	report layouts

Worked examples

Problem 6.1.1 (trivial) Label each product as `cheap` (under 200), `mid` (200 to 999) or `premium` (1000 or more).

Step 1	Write the conditions from lowest to highest.
Step 2	Because the first TRUE wins, the second test can just say `price < 1000` .

```
SELECT name, price,
       CASE WHEN price < 200  THEN 'cheap'
            WHEN price < 1000 THEN 'mid'
            ELSE 'premium' END AS band
FROM products
ORDER BY price;
```

```
name       price  band
---------  -----  -------
Notebook   80     cheap
Pen Pack   120    cheap
Mug        250    mid
Mouse      600    mid
Desk Lamp  900    mid
Keyboard   1500   premium
```

- **Check** 80 and 120 are below 200 (cheap). 250, 600, 900 are below 1000 (mid). 1500 is premium.
- **Answer** Notebook and Pen Pack cheap; Mug, Mouse and Desk Lamp mid; Keyboard premium.
- **Meaning** One column of numbers became three labels that a manager can read.

Problem 6.1.2 Bucket orders by total value: small below 500, medium 500 to 1499, large 1500 or more. How many orders and how much revenue are in each bucket?

Step 1	Compute each order's total in a CTE.
Step 2	Apply `CASE`, then `GROUP BY` the bucket.

```
WITH order_totals AS (
  SELECT order_id, SUM(qty * unit_price) AS total
  FROM order_items GROUP BY order_id
)
SELECT CASE WHEN total < 500  THEN '1 small'
            WHEN total < 1500 THEN '2 medium'
            ELSE '3 large' END AS bucket,
       COUNT(*) AS n_orders,
       SUM(total) AS revenue
FROM order_totals
GROUP BY bucket
ORDER BY bucket;
```

```
bucket    n_orders  revenue
--------  --------  -------
1 small   4         1600
2 medium  5         4400
3 large   5         8560
```

- **Compute** Order totals: 101:1660, 102:600, 103:1020, 104:400, 105:1500, 106:1280, 107:360, 108:900, 109:1500, 110:440, 111:600, 112:1800, 113:400, 114:2100. Small (under 500): 104, 107, 110, 113 give `400 + 360 + 440 + 400 = 1600`. Medium: 102, 103, 106, 108, 111 give `600 + 1020 + 1280 + 900 + 600 = 4400`. Large: 101, 105, 109, 112, 114 give `1660 + 1500 + 1500 + 1800 + 2100 = 8560`.
- **Check** `4 + 5 + 5 = 14` orders and `1600 + 4400 + 8560 = 14560`, the full revenue.
- **Answer** Small 4 orders / 1600, medium 5 / 4400, large 5 / 8560.
- **Meaning** Five large orders (36 percent of orders) bring 59 percent of revenue (`8560 / 14560`). The number prefix in the label (`1 small`) is a trick to sort buckets in the right order.

Problem 6.1.3 A manual pivot: one row per month, one column per order status.

```
SELECT substr(order_date,1,7) AS month,
       SUM(CASE WHEN status = 'paid'      THEN 1 ELSE 0 END) AS paid,
       SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled,
       SUM(CASE WHEN status = 'refunded'  THEN 1 ELSE 0 END) AS refunded
FROM orders
GROUP BY month
ORDER BY month;
```

```
month    paid  cancelled  refunded
-------  ----  ---------  --------
2024-01  2     0          0
2024-02  2     1          0
2024-03  4     0          1
2024-04  4     0          0
```

- **Check** Matches Chapter 3: Jan 2 paid; Feb 2 paid and 1 cancelled; Mar 4 paid and 1 refunded; Apr 4 paid. Column sums: `12 + 1 + 1 = 14`.
- **Answer** One compact table, months down, statuses across.
- **Meaning** This is the portable way to pivot. Chapter 7 returns to it.

Problem 6.1.4 The wrong version: no `ELSE`, and overlapping conditions. What does this return for the Mouse (600)?

```
SELECT name, price,
       CASE WHEN price >= 100 THEN 'A'
            WHEN price >= 500 THEN 'B' END AS grade
FROM products WHERE name IN ('Mouse', 'Notebook');
```

```
name      price  grade
--------  -----  -----
Mouse     600    A
Notebook  80     NULL
```

- **Answer** Mouse gets `A`, not `B`: the first TRUE wins, and `600 >= 100` is true. Notebook (80) matches nothing and gets NULL because there is no `ELSE`.
- **Fix** Order the tests from the most specific (`>= 500`) to the least, and add an `ELSE`.

The picture

```
  value 600 --> [price < 200?] no --> [price < 1000?] YES --> 'mid'  (stop)
  value 80  --> [price < 200?] YES --> 'cheap'                (stop)
  value 1500 -> [price < 200?] no --> [price < 1000?] no --> ELSE 'premium'
```

Connection to the real job

Segmentation (new / returning, small / large, fast / slow) is the first step of most analyses. A courier company buckets delivery times (under 1 day, 1 to 3 days, over 3 days) to show promise compliance, exactly the same `CASE` pattern.

Applications

- **Data Analytics:** customer segments, revenue bands, SLA bands.
- **Data Science and ML:** creating categorical features, flag columns (`is_high_value`).
- **Data Engineering:** mapping source codes to clean values.
- **Logistics:** delivery-time bands (under 1 day, 1 to 3 days, over 3 days).

COMMON MISTAKE	Wrong order of conditions, and forgetting `ELSE` so unmatched rows silently become NULL. Also mixing types: `THEN 1 ... ELSE 'none'` fails in PostgreSQL.

Check yourself

1. What does CASE return if no condition is true and there is no ELSE?

Answer: NULL.

1. If two conditions are both true, which one wins?

Answer: The first one, from the top.

1. How do you count rows meeting a condition inside one aggregate?

Answer: `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` or `COUNT(*) FILTER (WHERE cond)`.

6.2 String Functions

INTUITION	Text from forms, files and logs is like a hand-written note: extra spaces, wrong capitals, joined fields. String functions are scissors, glue and an eraser.

Words you need

- **String** (also *text*): a sequence of characters. **Literal**: `'abc'`.
- **Substring**: a piece of a string. **Position**: where a piece starts (counting from 1 in SQL, not 0).
- **Regular expression** (regex): a pattern language for text, for example `[0-9]+` means "one or more digits".

Function table

Task	PostgreSQL	SQLite
Join	`CONCAT(a,b)`, or the double-pipe operator	the double-pipe operator (see the text below the table)
Length	`LENGTH(s)`	`LENGTH(s)`
Upper / lower	`UPPER`, `LOWER`	same
Cut a piece	`SUBSTRING(s FROM 2 FOR 3)` or `SUBSTR(s,2,3)`	`SUBSTR(s,2,3)`
Find position	`POSITION('@' IN s)`	`INSTR(s,'@')`
Trim spaces	`TRIM(s)`, `LTRIM`, `RTRIM`	same
Replace	`REPLACE(s,'a','b')`	same
Pad left	`LPAD(s,5,'0')`	`printf('%05d', n)`
Split	`SPLIT_PART(s,'-',2)`	no built-in; use `SUBSTR` and `INSTR`
Regex	`s ~ 'pattern'`, `REGEXP_REPLACE`, `REGEXP_MATCHES`	`REGEXP` needs an extension; use `LIKE` / `GLOB`

The text-join operator is two vertical bars written together, as in `'a' || 'b'`. It is the standard SQL way to join text and works in PostgreSQL and SQLite.

Worked examples (messy contacts)

```
CREATE TABLE contacts (id INTEGER PRIMARY KEY, raw_name TEXT, email TEXT, phone TEXT);
INSERT INTO contacts VALUES
 (1,'  Asha  Rao ','ASHA.RAO@Example.COM','+91-98765-43210'),
 (2,'ravi kumar','ravi.k@example.com','+91-91234-56789'),
 (3,'MEENA IYER','meena@work.in','98765 00000');
```

Problem 6.2.1 (trivial) How long is each raw name, and how long after trimming? Predict: `' Asha Rao '` has 2 leading, 1 trailing spaces and a double space inside.

```
SELECT id, raw_name, LENGTH(raw_name) AS len_raw,
       TRIM(raw_name) AS trimmed, LENGTH(TRIM(raw_name)) AS len_trim
FROM contacts;
```

```
id  raw_name      len_raw  trimmed     len_trim
--  ------------  -------  ----------  --------
1     Asha  Rao   12       Asha  Rao   9
2   ravi kumar    10       ravi kumar  10
3   MEENA IYER    10       MEENA IYER  10
```

- **Compute** `' Asha Rao '`: 2 spaces + `Asha` (4) + 2 spaces + `Rao` (3) + 1 space = 12 characters. After `TRIM` (ends only): `Asha Rao` = 4 + 2 + 3 = 9.
- **Check** The output shows 12 and 9 for row 1. `ravi kumar` has 10 characters, `MEENA IYER` has 10, and trimming changes nothing.
- **Answer** `TRIM` removes spaces at both ends but not the double space in the middle.
- **Meaning** Extra spaces make `'Asha'` and `'Asha '` look equal to humans but different to `=` and `GROUP BY`, so trimming comes first in every cleaning job.

Problem 6.2.2 Clean the email domain: lowercase the email and take the part after `@`.

Step 1	`LOWER(email)` for case.
Step 2	`INSTR(email, '@')` gives the position of `@`.
Step 3	`SUBSTR(text, position + 1)` returns everything after it.

```
SELECT id,
       LOWER(email) AS email_clean,
       SUBSTR(LOWER(email), INSTR(email, '@') + 1) AS domain
FROM contacts;
```

```
id  email_clean          domain
--  -------------------  -----------
1   asha.rao@example.com  example.com
2   ravi.k@example.com    example.com
3   meena@work.in         work.in
```

- **Check** `ASHA.RAO@Example.COM`: `@` is at position 9, so the domain starts at 10: `Example.COM` in lowercase is `example.com`.
- **Answer** Domains: `example.com`, `example.com`, `work.in`.
- **Meaning** Grouping by domain tells you which companies your users come from.

Problem 6.2.3 Clean phone numbers to digits only, and pad an id to five digits with leading zeros.

```
SELECT id,
       REPLACE(REPLACE(REPLACE(phone, '+', ''), '-', ''), ' ', '') AS digits,
       printf('%05d', id) AS padded_id
FROM contacts;
```

```
id  digits        padded_id
--  ------------  ---------
1   919876543210  00001
2   919123456789  00002
3   9876500000    00003
```

- **Check** `+91-98765-43210` becomes `919876543210` (12 digits). `98765 00000` becomes `9876500000` (10 digits). The id `1` padded to 5 digits is `00001`.
- **Answer** Nested `REPLACE` calls remove each unwanted character.
- **Meaning** The two phones now differ in length (12 and 10): one has the country code and one does not. A length check like `LENGTH(digits)` is a quick data-quality test (Chapter 12).

Problem 6.2.4 Split a value: take the middle block of `+91-98765-43210`. In PostgreSQL `SPLIT_PART('+91-98765-43210', '-', 2)` returns `98765`. SQLite has no `SPLIT_PART`, so use `INSTR` and `SUBSTR`.

```
SELECT phone,
       SUBSTR(phone, INSTR(phone, '-') + 1,
              INSTR(SUBSTR(phone, INSTR(phone, '-') + 1), '-') - 1) AS middle_block
FROM contacts
WHERE phone LIKE '%-%-%';
```

```
phone           middle_block
--------------  ------------
+91-98765-43210  98765
+91-91234-56789  91234
```

- **Check** In `+91-98765-43210`, the first `-` is at position 4; after it the text is `98765-43210`; the next `-` is at position 6 of that text, so the length is 5, and the result is `98765`.
- **Answer** `98765` and `91234`.
- **Meaning** This is ugly. When you can choose, use `SPLIT_PART` or a regex in PostgreSQL.

```
-- PostgreSQL syntax, not run here
SELECT SPLIT_PART(phone, '-', 2)               AS middle_block,
       REGEXP_REPLACE(phone, '[^0-9]', '', 'g') AS digits_only,
       LPAD(id::text, 5, '0')                  AS padded_id
FROM contacts
WHERE email ~* '^[a-z.]+@example\.com$';   -- ~* means regex match, case-insensitive
```

Problem 6.2.5 Pattern search with `LIKE`: contacts whose email ends with `.com` (any case).

```
SELECT id, email FROM contacts WHERE LOWER(email) LIKE '%.com';
```

```
id  email
--  -------------------
1   ASHA.RAO@Example.COM
2   ravi.k@example.com
```

- **Answer** Contacts 1 and 2; contact 3 ends in `.in`.

- **Check** `ASHA.RAO@Example.COM` lowercased ends with `.com`. `meena@work.in` does not.

The picture

```
'ASHA.RAO@Example.COM'
 1234567890123...        INSTR(...,'@') = 9
 ASHA.RAO | @ | Example.COM
 SUBSTR(s, 1, 8)  = 'ASHA.RAO'   (the part before @: length = position - 1)
 SUBSTR(s, 10)    = 'Example.COM' (the part after @)
```

Positions count from 1. Characters before `@` are `9 - 1 = 8`.

Connection to the real job

Cleaning text is half of data work: standardise names, extract domains and area codes, build keys, search logs. A web analyst often parses URLs and log lines the same way (for example, take the path before `?`).

Applications

- **Data Analytics:** grouping by cleaned categories; email domains.
- **Data Engineering:** standardising keys before joins; parsing log fields.
- **Data Science and NLP:** text pre-cleaning before features.
- **Web analytics:** extracting page names from URLs.

COMMON MISTAKE	Joining on dirty text (`'Pune'` vs `'pune '`) so rows silently fail to match. Clean with `TRIM` and `LOWER` on both sides, or better, fix the data once at load time. Also: `NULL || 'x'` is NULL, so use `COALESCE` before joining text; PostgreSQL's `CONCAT` ignores NULLs.

Check yourself

1. In SQL, does the first character have position 0 or 1?

Answer: 1.

1. What does `TRIM(' a b ')` return?

Answer: `a b` (inner space kept).

1. What is `NULL || 'x'`?

Answer: NULL.

6.3 Dates and Times

INTUITION	Dates are the hardest "simple" data type: months have different lengths, leap years exist, and a timestamp means different clock times in Mumbai and London. A good analyst always asks: "which time zone, and what is the cut-off?"

Words you need

- **Date**: a calendar day. **Timestamp**: date plus time. `timestamptz` (PostgreSQL): timestamp stored in UTC and shown in your session time zone.
- **UTC**: the world reference time. India Standard Time is UTC + 5:30.
- **Interval**: a length of time (`7 days`, `1 month`).
- **Truncate**: cut a date down to the start of its period (month start, week start).
- **ISO format**: `YYYY-MM-DD`, which sorts correctly as text.

SQLite has no real date type. It stores dates as text (`'2024-03-05'`), numbers, or Julian days, and uses functions `date()` , `datetime()` , `strftime()` and `julianday()` . PostgreSQL has true `DATE` , `TIMESTAMP` , `TIMESTAMPTZ` and `INTERVAL` .

Function table

Task	PostgreSQL	SQLite
Today	`CURRENT_DATE`	`date('now')`
Add days	`d + INTERVAL '7 days'` or `d + 7`	`date(d, '+7 days')`
Difference in days	`d2 - d1`	`julianday(d2) - julianday(d1)`
Part of date	`EXTRACT(MONTH FROM d)`	`strftime('%m', d)`
Month start	`DATE_TRUNC('month', d)`	`date(d, 'start of month')`
Month end	`DATE_TRUNC('month', d) + INTERVAL '1 month - 1 day'`	`date(d, 'start of month', '+1 month', '-1 day')`
Weekday	`EXTRACT(DOW FROM d)` (0 = Sunday)	`strftime('%w', d)` (0 = Sunday)
Format	`TO_CHAR(d, 'YYYY-MM')`	`strftime('%Y-%m', d)`
Series	`generate_series(a, b, interval)`	recursive CTE (Chapter 5)

Worked examples

Problem 6.3.1 (trivial) For each order, show its month start and the date a 30-day return window closes. Show the first 4 orders.

```
SELECT order_id, order_date,
       date(order_date, 'start of month') AS month_start,
       date(order_date, '+30 days')       AS return_by
FROM orders ORDER BY order_id LIMIT 4;
```

```
order_id  order_date  month_start  return_by
--------  ----------  -----------  ----------
101       2024-01-10  2024-01-01   2024-02-09
102       2024-01-15  2024-01-01   2024-02-14
103       2024-02-03  2024-02-01   2024-03-04
104       2024-02-10  2024-02-01   2024-03-11
```

- **Check** Order 101 is on 2024-01-10. January has 31 days, so `+30 days` is 2024-02-09. Order 103 on 2024-02-03 plus 30 days: February 2024 has 29 days (a leap year), so 26 days take us to 2024-02-29, and 4 more to 2024-03-04.
- **Answer** The printed `return_by` values match: 2024-02-09 for order 101 and 2024-03-04 for order 103.
- **Meaning** The database knows month lengths and leap years, so you should never add days with your own arithmetic.

Problem 6.3.2 How many days passed from each customer's signup to their first order?

Step 1	In a CTE find the first order date per customer.
Step 2	Subtract the Julian day numbers (days since a fixed past date).

```
WITH first_orders AS (
  SELECT customer_id, MIN(order_date) AS first_order FROM orders GROUP BY customer_id
)
SELECT c.name, c.signup_date, f.first_order,
       CAST(julianday(f.first_order) - julianday(c.signup_date) AS INTEGER) AS days_to_first_order
FROM customers c JOIN first_orders f ON f.customer_id = c.customer_id
ORDER BY c.customer_id;
```

```
name   signup_date  first_order  days_to_first_order
-----  -----------  -----------  -------------------
Asha   2024-01-05   2024-01-10   5
Ravi   2024-01-12   2024-01-15   3
Meena  2024-01-20   2024-02-10   21
John   2024-02-02   2024-02-20   18
Priya  2024-02-14   2024-03-07   22
Kabir  2024-03-01   2024-03-30   29
```

- **Compute** Asha: Jan 5 to Jan 10 is 5 days. Ravi: Jan 12 to Jan 15 is 3. Meena: Jan 20 to Feb 10 is `11 + 10 = 21`. John: Feb 2 to Feb 20 is 18. Priya: Feb 14 to Mar 7 is `15 + 7 = 22` (February has 29 days: 15 days to Feb 29, then 7 more). Kabir: Mar 1 to Mar 30 is 29.
- **Check** These six numbers match the printed column.
- **Answer** 5, 3, 21, 18, 22, 29 days.
- **Meaning** On average `(5+3+21+18+22+29)/6 = 16.3` days. "Time to first purchase" is a standard activation metric.

Problem 6.3.3 Orders per weekday. In SQLite, `%w` gives 0 for Sunday to 6 for Saturday.

```
SELECT strftime('%w', order_date) AS dow,
       CASE strftime('%w', order_date)
            WHEN '0' THEN 'Sun' WHEN '1' THEN 'Mon' WHEN '2' THEN 'Tue'
            WHEN '3' THEN 'Wed' WHEN '4' THEN 'Thu' WHEN '5' THEN 'Fri'
            ELSE 'Sat' END AS day_name,
       COUNT(*) AS n_orders
FROM orders
GROUP BY dow ORDER BY dow;
```

```
dow  day_name  n_orders
---  --------  --------
1    Mon       4
2    Tue       4
3    Wed       1
4    Thu       1
6    Sat       4
```

- **Check** 2024-01-01 was a Monday, so 2024-01-10 is a Wednesday (1 order), as printed. Counts add to 14.
- **Answer** Mon 4, Tue 4, Wed 1, Thu 1, Sat 4; no orders on Fri or Sun.
- **Meaning** Day-of-week patterns drive staffing and campaign timing. Note that no row is shown for days with zero orders; use a date spine (Chapter 5) if you need them.

Problem 6.3.4 The classic trap: filtering a whole month. Show that `BETWEEN '2024-03-01' AND '2024-03-31'` is fine for dates but loses rows for timestamps. We simulate a timestamp column.

```
CREATE TABLE log (ts TEXT);
INSERT INTO log VALUES ('2024-03-31 09:15:00'), ('2024-03-31 23:59:59'), ('2024-04-01 00:00:00');

SELECT COUNT(*) AS between_version FROM log
WHERE ts BETWEEN '2024-03-01' AND '2024-03-31';

SELECT COUNT(*) AS half_open_version FROM log
WHERE ts >= '2024-03-01' AND ts < '2024-04-01';
```

```
between_version
---------------
0
half_open_version
-----------------
2
```

- **Compute** The text `'2024-03-31 09:15:00'` is greater than `'2024-03-31'` (longer, same start), so `BETWEEN` loses both March 31 rows. The half-open test keeps rows from the first moment of March up to, but not including, April 1.
- **Check** The half-open version returns 2 (the two March 31 rows); `BETWEEN` returns 0.
- **Answer** Use `>= start AND < next_start`.
- **Meaning** Half-open ranges work for dates, timestamps and any precision.

PostgreSQL versions of the same ideas, not run here:

```
-- PostgreSQL syntax, not run here
SELECT DATE_TRUNC('month', order_date)                    AS month_start,
       EXTRACT(DOW FROM order_date)                        AS weekday,
       order_date + INTERVAL '30 days'                     AS return_by,
       order_date - signup_date                            AS days_since_signup
FROM orders JOIN customers USING (customer_id);

-- time zones: store timestamptz, display in India time
SELECT created_at AT TIME ZONE 'Asia/Kolkata' FROM events;
```

The picture

```
 time line (UTC)      00:00     05:30 IST          12:00
 server timestamp  -->  |---------|------------------|
 same instant shown in Mumbai: 05:30 on the same day
 A "day" in Mumbai starts at 18:30 UTC of the previous day.
```

If your server stores UTC but the business day is Indian, grouping by UTC date puts late-evening orders on the wrong day. Convert first, then truncate.

Connection to the real job

Month-over-month growth, retention by week and "orders in the last 7 days" are all date logic. Time-zone errors are among the most common causes of dashboards that "do not match finance". Many systems (mobile apps, sensors, payment logs) store timestamps as milliseconds since 1970 (epoch time); converting them to dates is a typical first query.

Applications

- **Data Analytics:** monthly and weekly trends, age of customer, SLA days.
- **Data Engineering:** partitioning tables by date; incremental loads by `updated_at`.
- **Data Science and ML:** time-based features (day of week, days since last order).

- **Healthcare:** grouping hospital appointments by hour of the day.

COMMON MISTAKE	Using `BETWEEN` with an end date on timestamp columns (loses the last day), mixing time zones, and storing dates as text in free formats (`03/04/24` is March 4 or April 3?). Use ISO `YYYY-MM-DD` .

Check yourself

1. Which is safer for a month filter, `BETWEEN` or half-open?

Answer: Half-open (`>= start AND < next start`).

1. India Standard Time is how far from UTC?

Answer: UTC + 5:30.

1. How do you get the first day of the month in PostgreSQL?

Answer: `DATE_TRUNC('month', d)` .

6.4 JSON and Array Values

INTUITION	A relational table is a spreadsheet with fixed columns. But app events, API responses and logs arrive as **JSON**, a flexible text format with nested fields: `{"user":"asha","device":{"os":"android"}}` . You can store the whole blob in one column and still pull fields out with SQL.

Words you need

- **JSON** (JavaScript Object Notation): text made of objects `{"key": value}` , arrays `[1, 2, 3]` , strings, numbers, booleans and null.
- **Semi-structured data**: data whose fields vary per record.
- **Path**: an address inside JSON, like `$.device.os` (`$` is the root).
- **Array column**: PostgreSQL can store a list in one cell (`ARRAY['a','b']`).
- **Unnest / explode**: turning one row with a list into many rows, one per element.

Function table

Task	PostgreSQL (`jsonb`)	SQLite
Get field as JSON	`payload -> 'device'`	`payload -> '$.device'`
Get field as text	`payload ->> 'user'`	`payload ->> '$.user'`
Nested	`payload -> 'device' ->> 'os'`	`payload ->> '$.device.os'`
Array to rows	`jsonb_array_elements_text(payload -> 'tags')`	`json_each(payload, '$.tags')`
Contains	`payload @> '{"user":"asha"}'`	no direct operator
Build array	`ARRAY_AGG(x)` / `jsonb_agg(x)`	`json_group_array(x)`
Join text	`STRING_AGG(x, ', ')`	`GROUP_CONCAT(x, ', ')`
Array to rows	`UNNEST(arr)`	`json_each`
Index	`CREATE INDEX ... USING GIN (payload)`	expression index

PostgreSQL's `jsonb` stores JSON in a parsed binary form that supports indexes; `json` keeps the raw text. Prefer `jsonb` .

Worked examples

```
CREATE TABLE events (id INTEGER PRIMARY KEY, payload TEXT);
INSERT INTO events VALUES
 (1,'{"user":"asha","device":{"os":"android","ver":14},"tags":["promo","new"],"amount":250}'),
 (2,'{"user":"ravi","device":{"os":"ios","ver":17},"tags":[],"amount":600}'),
 (3,'{"user":"meena","device":{"os":"android","ver":13},"tags":["promo"]}');
```

Problem 6.4.1 (trivial) Pull the user, the operating system and the amount out of each event. Predict: event 3 has no `amount` field. What will that cell show?

```
SELECT id,
       payload -->> '$.user'         AS user,
       payload -->> '$.device.os'    AS os,
       payload -->> '$.amount'       AS amount
FROM events;
```

```
id  user   os       amount
--  -----  -------  ------
1   asha   android  250
2   ravi   ios      600
3   meena  android  NULL
```

- **Answer** Event 3's amount is NULL (a missing key gives NULL, not an error).
- **Check** Event 1: asha, android, 250; event 2: ravi, ios, 600; event 3: meena, android, NULL.
- **Meaning** You can query flexible data without a fixed schema, but missing keys become NULL, so count them.

Problem 6.4.2 Total `amount` and counts, by operating system.

```
SELECT payload -->> '$.device.os' AS os,
       COUNT(*) AS events,
       COUNT(payload -->> '$.amount') AS with_amount,
       SUM(payload -->> '$.amount') AS total_amount
FROM events
GROUP BY os
ORDER BY os;
```

```
os       events  with_amount  total_amount
-------  ------  -----------  ------------
android  2       1            250
ios      1       1            600
```

- **Compute** Android has events 1 and 3: amounts 250 and NULL, so `COUNT(amount) = 1` and `SUM = 250`. iOS has event 2: 600.
- **Check** `250 + 600 = 850` total across both groups.
- **Answer** Android: 2 events, 1 with amount, 250. iOS: 1 event, 1 with amount, 600.
- **Meaning** `SUM` ignores NULL. If event 3's missing amount really means zero, add `COALESCE`.

Problem 6.4.3 Explode the `tags` array: one row per (event, tag). Also count events per tag.

```
SELECT e.id, t.value AS tag
FROM events e, json_each(e.payload, '$.tags') AS t
ORDER BY e.id;
```

```
id  tag
--  -----
1   promo
1   new
3   promo
```

```
SELECT t.value AS tag, COUNT(*) AS n_events
FROM events e, json_each(e.payload, '$.tags') AS t
GROUP BY t.value ORDER BY n_events DESC;
```

```
tag    n_events
-----  --------
promo  2
new    1
```

- **Check** Event 1 has two tags, event 2 has none (an empty array gives no rows), event 3 has one. Total 3 rows. `promo` appears in events 1 and 3 (2), `new` in event 1 (1).
- **Answer** Three (event, tag) rows; `promo` 2, `new` 1.
- **Meaning** Event 2 vanished because an empty array gives zero rows. To keep such rows use a `LEFT JOIN` form. This "one row per element" shape is what you need to count or join tags.

Problem 6.4.4 The reverse: collapse rows into lists. For each product category, list its product names.

```
SELECT category, GROUP_CONCAT(name, ', ') AS products, COUNT(*) AS n
FROM (SELECT category, name FROM products ORDER BY name)
GROUP BY category
ORDER BY category;
```

```
category     products           n
-----------  -----------------  -
Electronics  Keyboard, Mouse    2
Home         Desk Lamp, Mug     2
Stationery   Notebook, Pen Pack 2
```

- **Check** Electronics holds Keyboard and Mouse; Home holds Desk Lamp and Mug; Stationery holds Notebook and Pen Pack. Alphabetical inside each group because we sorted before grouping.
- **Answer** One row per category with a comma list.
- **Meaning** SQLite does not guarantee the order of `GROUP_CONCAT`; sorting in a subquery works in practice. In PostgreSQL write `STRING_AGG(name, ', ' ORDER BY name)`.

```
-- PostgreSQL syntax, not run here
-- jsonb and arrays in PostgreSQL
SELECT payload ->> 'user'                     AS user_name,
       payload -> 'device' ->> 'os'           AS os,
       (payload ->> 'amount')::numeric        AS amount
FROM events
WHERE payload @> '{"tags": ["promo"]}';       -- contains this fragment (GIN index can help)

SELECT category, ARRAY_AGG(name ORDER BY name) AS names,
       STRING_AGG(name, ', ' ORDER BY name)    AS names_text
FROM products GROUP BY category;

SELECT e.id, tag
FROM events e, LATERAL jsonb_array_elements_text(e.payload -> 'tags') AS tag;

SELECT UNNEST(ARRAY[10, 20, 30]) AS n;        -- one row per element
```

The picture

```
one row, nested                json_each / UNNEST            rows
{"tags":["promo","new"]}  ----------------------->  (event 1, promo)
                                                     (event 1, new)
rows to list: GROUP_CONCAT / STRING_AGG / ARRAY_AGG works in the opposite direction.
```

Connection to the real job

Event data (clickstream, API logs, mobile telemetry) arrives as JSON. A data engineer lands the raw JSON in one column, then flattens it with these functions into proper tables (a common step in ELT). Many apps already store order details or settings as JSON; the same functions let you query them.

Applications

- **Data Analytics:** product events with flexible properties.
- **Data Engineering:** flattening raw JSON into columns during ELT.
- **Data Science and ML:** extracting features from event payloads.
- **GenAI:** storing LLM request and response metadata as JSON for analysis.

COMMON MISTAKE	Putting everything into one JSON column "for flexibility". You lose type checks, constraints and good indexing. Use JSON for truly variable fields; keep core fields as real columns. Another trap: `->` returns JSON (with quotes), `->>` returns plain text; comparing `payload -> 'user' = 'asha'` fails because the left side is `"asha"` with quotes.

Check yourself

1. What does `->>` return compared to `->`?

Answer: `->>` returns plain text; `->` returns JSON.

1. What does an empty array produce when you unnest it?

Answer: Zero rows.

1. Which function joins rows into one comma string in PostgreSQL?

Answer: `STRING_AGG`.

Chapter recap

- `CASE WHEN` tests conditions top to bottom and returns the first match; add `ELSE`.
- `CASE` inside `SUM` or `COUNT` builds conditional metrics and manual pivots.
- `TRIM`, `LOWER`, `REPLACE`, `SUBSTR` and `INSTR` clean text; positions start at 1.
- `NULL || text` is NULL; clean text on both sides before joining.
- Use ISO dates, half-open ranges (`>= start AND < next`) and never hand-count days.
- Know your time zone; convert before truncating to a day.
- JSON fields are read with `->` (JSON) and `->>` (text); missing keys give NULL.
- Unnest arrays to rows to count or join them; aggregate rows back to lists with `STRING_AGG` / `ARRAY_AGG`.

Task	PostgreSQL	SQLite
Bucket	`CASE WHEN ... END`	same
Month start	`DATE_TRUNC('month', d)`	`date(d,'start of month')`
Days between	`d2 - d1`	`julianday(d2)-julianday(d1)`

Task	PostgreSQL	SQLite
JSON text	`col ->> 'k'`	`col ->> '$.k'`
Explode array	`UNNEST`, `jsonb_array_elements_text`	`json_each`
Rows to text	`STRING_AGG(x, ', ')`	`GROUP_CONCAT(x, ', ')`

Practice set

Problem 6.P.1 (easy) Show each customer name in upper case and its length.

```
SELECT UPPER(name) AS name_upper, LENGTH(name) AS len FROM customers ORDER BY customer_id;
```

```
name_upper  len
----------  ---
ASHA        4
RAVI        4
MEENA       5
JOHN        4
PRIYA       5
KABIR       5
SANA        4
TOM         3
```

Answer: ASHA 4, RAVI 4, MEENA 5, JOHN 4, PRIYA 5, KABIR 5, SANA 4, TOM 3.

Problem 6.P.2 (easy) Classify each order's status as `ok` (paid) or `problem` (anything else) and count each.

```
SELECT CASE WHEN status = 'paid' THEN 'ok' ELSE 'problem' END AS kind, COUNT(*) AS n
FROM orders GROUP BY kind ORDER BY kind;
```

```
kind     n
-------  --
ok       12
problem  2
```

Answer: ok 12, problem 2.

Problem 6.P.3 (what is the output?) What does this print?

```
SELECT date('2024-01-31', '+1 month') AS a, date('2024-02-29', '+1 year') AS b;
```

```
a           b
----------  ----------
2024-03-02  2025-03-01
```

Answer: `a = 2024-03-02` in SQLite (31 January plus one month is "31 February", which rolls over to 2 March) and `b = 2025-03-01` (29 February 2025 does not exist, so it rolls to 1 March). PostgreSQL clamps to month end instead: `DATE '2024-01-31' + INTERVAL '1 month'` is `2024-02-29`. Month arithmetic differs by database; test it.

Problem 6.P.4 (find the bug) A teammate filters March data with `WHERE ts BETWEEN '2024-03-01' AND '2024-03-31'` on a timestamp column and the last day is missing. Fix it. **Answer:** `WHERE ts >= '2024-03-01' AND ts < '2024-04-01'`.

Problem 6.P.5 Show orders from the second half of each month (day 16 or later) with the day number.

```
SELECT order_id, order_date, CAST(strftime('%d', order_date) AS INTEGER) AS day_no
FROM orders
WHERE CAST(strftime('%d', order_date) AS INTEGER) >= 16
ORDER BY order_id;
```

```
order_id  order_date  day_no
--------  ----------  ------
105       2024-02-20  20
108       2024-03-18  18
109       2024-03-25  25
110       2024-03-30  30
114       2024-04-20  20
```

Answer: Orders 105 (day 20), 108 (18), 109 (25), 110 (30) and 114 (20). Order 113 is on day 15, so it is excluded.

Problem 6.P.6 Extract the first name and last name from `'ravi kumar'` using `INSTR` and `SUBSTR`.

```
SELECT SUBSTR(raw_name, 1, INSTR(raw_name, ' ') - 1) AS first_name,
       SUBSTR(raw_name, INSTR(raw_name, ' ') + 1)    AS last_name
FROM contacts WHERE id = 2;
```

```
first_name  last_name
----------  ---------
ravi        kumar
```

Answer: `ravi` and `kumar`. The space is at position 5, so the first name has `5 - 1 = 4` characters.

Problem 6.P.7 Using the `events` table, find events from Android devices with version 13 or higher.

```
SELECT id, payload ->> '$.user' AS user, payload ->> '$.device.ver' AS ver
FROM events
WHERE payload ->> '$.device.os' = 'android'
  AND (payload ->> '$.device.ver') >= 13;
```

```
id  user   ver
--  -----  ---
1   asha   14
3   meena  13
```

Answer: Events 1 (ver 14) and 3 (ver 13), as printed.

Problem 6.P.8 (scenario) A column `created_at` holds UTC timestamps. The business wants daily orders in India time. What do you do? **Answer:** Convert to `Asia/Kolkata` first (`created_at AT TIME ZONE 'Asia/Kolkata'` in PostgreSQL), then truncate to date and group. An order at 20:00 UTC on 31 March is 01:30 IST on 1 April, so it belongs to April in India.

Problem 6.P.9 (interview) What is the difference between `COALESCE` and `CASE WHEN x IS NULL`? **Answer:** `COALESCE(a, b, c)` is a short form that returns the first non-NULL argument. `CASE WHEN a IS NULL THEN b ELSE a END` does the same for two values but is longer, and it allows any conditions, not only NULL tests.

Problem 6.P.10 (interview) How would you find all users who have a particular tag in a JSON array column in PostgreSQL, and how would you make it fast? **Answer:** With `jsonb`: `WHERE payload @> '{"tags":["promo"]}'`, and create a GIN index on the column: `CREATE INDEX ON events USING`

`GIN (payload)`. Without the index the database scans every row.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my second language. Teach me Chapter 6 topic: "CASE WHEN, bucketing and manual pivot in SQL". Use a small orders table and show bucketing by value, conditional counts per month, and the order-of-conditions trap. After each example ask me to predict the output and check my answer.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: cleaning text with string functions (TRIM, LOWER, SUBSTR, INSTR, REPLACE, SPLIT_PART, REGEXP_REPLACE) in PostgreSQL. Give me 8 graded cleaning exercises on messy names, emails and phone numbers, with no solutions at first. Review my answers when I paste them and explain mistakes in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. Teach me date and time handling in SQL for a beginner: DATE_TRUNC, EXTRACT, intervals, half-open ranges, month arithmetic, and time zones (UTC and Asia/Kolkata). Use small numeric examples I can verify by hand, and ask one check question after each step. Finish with 5 quiz questions.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. Teach me JSON and array columns in PostgreSQL: jsonb, ->, ->>, @>, jsonb_array_elements, UNNEST, ARRAY_AGG, STRING_AGG, and GIN indexes. Use an events table with user, device and tags fields. I am a beginner. Give me small exercises, wait for my answers, and explain when NOT to use JSON columns.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. Run a mock SQL interview for a data-analyst role focused on CASE, string, date and JSON functions. Ask one practical question at a time (for example: month-over-month filter, cleaning emails, extracting JSON fields, last day of month), grade my answer out of 10, show the model solution in PostgreSQL, and mention one common mistake.
```

CHAPTER 7

SET OPERATIONS, PIVOT AND UNPIVOT, GROUPING SETS, ROLLUP AND CUBE

Stacking result sets, comparing them, turning rows into columns (and back), and producing subtotals in one query.

Joins put tables side by side. This chapter is about the other directions. How do you stack two lists into one? How do you find who is in list A but not in list B (churn, missing data)? How do you turn a long table (one row per month and category) into the wide table that managers love (months as columns)? And how do you add subtotal and grand-total rows to a report without writing five queries? After this chapter you can do all four, and you will know where SQLite stops and PostgreSQL continues.

7.1 UNION, UNION ALL, INTERSECT, EXCEPT

INTUITION

Think of two attendance sheets for two classes. Put both sheets in one pile: that is **UNION ALL**. Put them in one pile but write each name only once: **UNION**. Names that appear on both sheets: **INTERSECT**. Names on the first sheet but not on the second: **EXCEPT** (called `MINUS` in Oracle).

Words you need

- **Set operation**: combines the *rows* of two queries (not the columns, as a join does).
- **Compatible**: both queries must return the same number of columns, with compatible types, in the same order.
- **Duplicate-free**: `UNION`, `INTERSECT` and `EXCEPT` remove duplicate rows from the result; `UNION ALL` does not.

Definitions

Operator	Result	Removes duplicates?	Speed note
`A UNION ALL B`	all rows of A, then all rows of B	no	fastest
`A UNION B`	rows in A or B	yes	needs a sort or hash
`A INTERSECT B`	rows in both	yes	
`A EXCEPT B`	rows in A that are not in B	yes	order matters

Rules: column **names** come from the first query. `ORDER BY` goes once, at the very end. `NULL`s are treated as equal to each other when removing duplicates (unlike `=`). PostgreSQL also has `INTERSECT ALL` and `EXCEPT ALL`, which keep duplicate counts.

Real-world mapping

Operation	Example	Where it appears in the job
`UNION ALL`	stack `sales_2023` and `sales_2024`	combining monthly tables or sharded tables
`UNION`	unique email list from two sources	de-duplicated mailing list
`INTERSECT`	users active in both months	retained users
`EXCEPT`	users active in Feb but not April	churned users; reconciliation gaps

Worked examples

We will compare **February buyers** with **April buyers**. From the data: February orders are 103 (customer 1), 104 (customer 3) and 105 (customer 4). April orders are 111 (customer 1), 112 (customer 4), 113 (customer 5) and 114 (customer 2).

Problem 7.1.1 (trivial) Stack two tiny lists with and without duplicates. Predict: how many rows does `UNION` give, and how many does `UNION ALL` give, for `{1,2,2}` and `{2,3}`?

```
SELECT 1 AS n UNION ALL SELECT 2 UNION ALL SELECT 2
UNION ALL
SELECT 2 UNION ALL SELECT 3;
```

```
n
-
1
2
2
2
3
```

```
SELECT 1 AS n UNION SELECT 2 UNION SELECT 2
UNION
SELECT 2 UNION SELECT 3;
```

```
n
-
1
2
3
```

- **Check** Five values go in (1, 2, 2, 2, 3). `UNION ALL` keeps all 5. `UNION` keeps distinct values: 1, 2, 3.
- **Answer** 5 rows with `UNION ALL`, 3 with `UNION`.
- **Meaning** `UNION` quietly deletes duplicates. If you meant to keep every record (for example, stacking two transaction tables), `UNION` loses data. Use `UNION ALL` unless you want duplicates removed.

Problem 7.1.2 Compare the two groups of buyers with the four operators.

Step 1	Define each group with a query on `orders`.
Step 2	Combine.

```
-- February buyers
SELECT DISTINCT customer_id FROM orders WHERE order_date LIKE '2024-02%'
INTERSECT
-- April buyers
SELECT DISTINCT customer_id FROM orders WHERE order_date LIKE '2024-04%';
```

```
customer_id
-----------
1
4
```

```
SELECT DISTINCT customer_id FROM orders WHERE order_date LIKE '2024-02%'
EXCEPT
SELECT DISTINCT customer_id FROM orders WHERE order_date LIKE '2024-04%';
```

```
customer_id
-----------
3
```

```
SELECT DISTINCT customer_id FROM orders WHERE order_date LIKE '2024-04%'
EXCEPT
SELECT DISTINCT customer_id FROM orders WHERE order_date LIKE '2024-02%';
```

```
customer_id
-----------
2
5
```

```
SELECT customer_id FROM orders WHERE order_date LIKE '2024-02%'
UNION
SELECT customer_id FROM orders WHERE order_date LIKE '2024-04%'
ORDER BY customer_id;
```

```
customer_id
-----------
1
2
3
4
5
```

- **Compute** February set `A = {1, 3, 4}`. April set `B = {1, 4, 5, 2}`. Intersection `A ∩ B = {1, 4}`. `A − B = {3}`. `B − A = {2, 5}`. Union `A ∪ B = {1, 2, 3, 4, 5}`.
- **Check** Sizes: `|A ∪ B| = |A| + |B| − |A ∩ B| = 3 + 4 − 2 = 5`.
- **Answer** Retained: customers 1 and 4. Lost after February: customer 3 (Meena). New or returning in April: customers 2 and 5.
- **Meaning** `EXCEPT` is not symmetric: `A EXCEPT B` and `B EXCEPT A` answer different questions. This is a simple retention analysis. Chapter 12 builds a full cohort table.

Problem 7.1.3 `UNION ALL` to stack different sources and tag each row with its origin. Combine customers' cities and product categories into one "labels" list with a `source` column.

```
SELECT 'customer_city' AS source, city AS label FROM customers
UNION ALL
SELECT 'product_category', category FROM products
ORDER BY source, label
LIMIT 6;
```

```
source         label
-------------  ---------
customer_city  Bengaluru
customer_city  Delhi
customer_city  Delhi
customer_city  Mumbai
customer_city  Mumbai
customer_city  Pune
```

- **Check** The customer cities are Bengaluru, Delhi, Delhi, Mumbai, Mumbai, Pune, Pune, Pune (8 rows). Sorted, the first six rows are `customer_city` rows: Bengaluru, Delhi, Delhi, Mumbai, Mumbai, Pune.
- **Answer** The `source` literal column records where each row came from.
- **Meaning** Adding a literal "source" column is a standard habit when stacking tables.

Problem 7.1.4 Reconciliation with `EXCEPT` in both directions. Two lists of train stops, `scheduled` and `arrived` (from Chapter 4, created again here).

```
CREATE TABLE scheduled (stop TEXT);
CREATE TABLE arrived (stop TEXT);
INSERT INTO scheduled VALUES ('Pune'),('Lonavala'),('Kalyan'),('Thane');
INSERT INTO arrived VALUES ('Pune'),('Lonavala'),('Kalyan'),('Dadar');

SELECT 'scheduled_not_visited' AS issue, stop FROM (SELECT stop FROM scheduled EXCEPT SELECT stop FROM arrived)
UNION ALL
SELECT 'visited_not_scheduled', stop FROM (SELECT stop FROM arrived EXCEPT SELECT stop FROM scheduled);
```

```
issue                 stop
--------------------  -----
scheduled_not_visited  Thane
visited_not_scheduled  Dadar
```

- **Check** `Thane` is only in `scheduled`; `Dadar` is only in `arrived`.
- **Answer** Two issue rows.
- **Meaning** The same job as a `FULL JOIN` (Chapter 4). `EXCEPT` compares *whole rows* and treats NULLs as equal, which is handy when comparing two tables column for column.

Problem 7.1.5 A precedence trap. In standard SQL (and PostgreSQL) `INTERSECT` binds tighter than `UNION`. What does the following give? Predict with the standard rule, then see what SQLite does.

```
SELECT 1 AS n UNION SELECT 2 INTERSECT SELECT 2;
```

```
n
-
2
```

- **Compute** Standard rule: `1 UNION (2 INTERSECT 2)` is `{1} ∪ {2} = {1, 2}`. Left-to-right rule: `(1 UNION 2) INTERSECT 2` is `{1, 2} ∩ {2} = {2}`.
- **Check** The printed answer shows which rule SQLite follows.
- **Answer** SQLite evaluates compound operators from left to right with equal priority, so the result is `2`. PostgreSQL would return 1 and 2.
- **Meaning** Always use brackets (or CTEs) when mixing set operators, so the result does not depend on the database.

The picture

```
   A (Feb buyers)     B (April buyers)
   +-----------+-----------+
   |  3        | 1  4      |  5   2
   |           |           |
   +-----------+-----------+
    A EXCEPT B   INTERSECT   B EXCEPT A
       {3}        {1,4}       {2,5}
   UNION = everything inside either circle = {1,2,3,4,5}
```

Connection to the real job

`UNION ALL` is how you combine partitioned or monthly tables. `EXCEPT` and `INTERSECT` are standard tools for data validation: "rows in the source but not in the warehouse", "keys in both systems". In performance work: "endpoints that appear in the old test but not in the new one".

Applications

- **Data Analytics:** retained vs lost vs new customers; overlap between campaigns.

- **Data Engineering:** reconciliation between source and target; combining sharded tables.
- **Data Science:** removing test-set rows from a candidate training set (`EXCEPT`).
- **Operations:** diff of product codes or delivery areas between two weekly stock lists.

COMMON MISTAKE	Using `UNION` when `UNION ALL` is meant (silently loses duplicate records and is slower), and assuming columns are matched by name. They are matched by **position**, so keep the same column order in every query.

Check yourself

1. Which operator removes duplicates, `UNION` or `UNION ALL` ?

Answer: `UNION` .

1. Is `A EXCEPT B` the same as `B EXCEPT A` ?

Answer: No.

1. Are columns matched by name or by position?

Answer: By position.

7.2 Pivot and Unpivot

INTUITION	A **long** table is like a ledger: one line per fact (Electronics, January, 2100). A **wide** table is like a report card: one line per subject, with one column per month. People read wide tables; databases store and analyse long tables. **Pivot** converts long to wide. **Unpivot** converts wide to long.

Words you need

- **Long format** (tidy): one row per observation; few columns (category, month, revenue).
- **Wide format**: one column per value of a category (jan, feb, mar, apr).
- **Pivot key**: the column whose values become new column headers (month).

How: standard SQL has no PIVOT keyword

The portable method is the conditional aggregation you know from Chapters 3 and 6: one `SUM(CASE WHEN ... THEN value END)` per new column. Note: you must **know the column values in advance**. SQL cannot create columns whose names depend on the data in a plain query. (Dynamic pivots need generated SQL or a tool such as pandas `pivot_table` .)

Native keywords exist in some products: SQL Server and Oracle and Snowflake have `PIVOT` / `UNPIVOT` ; PostgreSQL has the `crosstab` function in the `tablefunc` extension; BigQuery has `PIVOT` .

Real-world mapping

Shape	Example	Where it appears in the job
Long	(category, month, revenue)	warehouse fact tables, ML training data
Wide	category x four month columns	Excel and dashboard layouts
Unpivot	wide survey columns to rows	cleaning spreadsheet exports

Worked examples

Problem 7.2.1 Build a wide table: paid revenue by category (rows) and month (columns).

Step 1	Join `order_items`, `orders` and `products`; keep paid orders.
Step 2	One conditional sum per month, wrapped in `COALESCE(..., 0)` so empty cells show 0 instead of NULL.
Step 3	Add a row total as another column.

```
CREATE TABLE cat_month AS
SELECT p.category,
       COALESCE(SUM(CASE WHEN substr(o.order_date,1,7) = '2024-01' THEN oi.qty*oi.unit_price END), 0) AS jan,
       COALESCE(SUM(CASE WHEN substr(o.order_date,1,7) = '2024-02' THEN oi.qty*oi.unit_price END), 0) AS feb,
       COALESCE(SUM(CASE WHEN substr(o.order_date,1,7) = '2024-03' THEN oi.qty*oi.unit_price END), 0) AS mar,
       COALESCE(SUM(CASE WHEN substr(o.order_date,1,7) = '2024-04' THEN oi.qty*oi.unit_price END), 0) AS apr,
       SUM(oi.qty*oi.unit_price) AS total
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id AND o.status = 'paid'
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category;

SELECT * FROM cat_month ORDER BY category;
```

```
category     jan   feb   mar   apr   total
-----------  ----  ----  ----  ----  -----
Electronics  2100  1500  1200  2700  7500
Home         0     900   900   1800  3600
Stationery   160   120   880   400   1560
```

- **Compute** Electronics: Jan `1500 + 600 = 2100` (orders 101 and 102), Feb `1500` (order 105), Mar `1200` (order 106; order 109 is refunded so excluded), Apr `600 + 1500 + 600 = 2700` (orders 111 and 114). Row total `2100 + 1500 + 1200 + 2700 = 7500`.
- **Check** Column totals `2260, 2520, 2980, 4900` and grand total 12660 match the paid revenue found in Chapter 3. The row totals are 7500 + 3600 + 1560 = 12660.
- **Answer** Electronics 2100 / 1500 / 1200 / 2700 (7500); Home 0 / 900 / 900 / 1800 (3600); Stationery 160 / 120 / 880 / 400 (1560).
- **Meaning** Home had no paid sales in January, and the table says so with a 0. This is the table a manager can read at a glance.

Problem 7.2.2 Unpivot: take `cat_month` back to long format. Predict: 3 categories x 4 months give 12 rows.

Step 1	For each month column, write a `SELECT category, 'month label', that_column`.
Step 2	Stack them with `UNION ALL`.

```
SELECT category, '2024-01' AS month, jan AS revenue FROM cat_month
UNION ALL SELECT category, '2024-02', feb FROM cat_month
UNION ALL SELECT category, '2024-03', mar FROM cat_month
UNION ALL SELECT category, '2024-04', apr FROM cat_month
ORDER BY category, month;
```

```
category     month    revenue
-----------  -------  -------
Electronics  2024-01  2100
Electronics  2024-02  1500
Electronics  2024-03  1200
Electronics  2024-04  2700
Home         2024-01  0
Home         2024-02  900
Home         2024-03  900
Home         2024-04  1800
Stationery   2024-01  160
Stationery   2024-02  120
Stationery   2024-03  880
Stationery   2024-04  400
```

- **Check** The output has `3 × 4 = 12` rows. For Electronics the four revenues are the first row's values read in order.
- **Answer** Twelve rows in long format.
- **Meaning** Long format is what charting libraries, `GROUP BY` and ML pipelines want. In PostgreSQL the shorter form is a `CROSS JOIN LATERAL (VALUES ...)`, shown below.

Problem 7.2.3 Pivot with `FILTER` (PostgreSQL and SQLite 3.30+) and a PostgreSQL native alternative. Count orders per status, one column per status, for the whole shop.

```
SELECT COUNT(*) FILTER (WHERE status = 'paid')      AS paid,
       COUNT(*) FILTER (WHERE status = 'cancelled') AS cancelled,
       COUNT(*) FILTER (WHERE status = 'refunded')  AS refunded
FROM orders;
```

```
paid  cancelled  refunded
----  ---------  --------
12    1          1
```

- **Check** 12 + 1 + 1 = 14 orders.
- **Answer** paid 12, cancelled 1, refunded 1.
- **Meaning** `FILTER` is shorter than `CASE`, but the idea is the same.

```
-- PostgreSQL syntax, not run here
-- Unpivot in PostgreSQL with VALUES + LATERAL
SELECT c.category, v.month, v.revenue
FROM cat_month c
CROSS JOIN LATERAL (VALUES ('2024-01', c.jan), ('2024-02', c.feb),
                           ('2024-03', c.mar), ('2024-04', c.apr)) AS v(month, revenue);

-- SQL Server / Snowflake / Oracle native PIVOT (not PostgreSQL):
-- SELECT * FROM (SELECT category, month, revenue FROM long_table)
-- PIVOT (SUM(revenue) FOR month IN ('2024-01','2024-02','2024-03','2024-04'));

-- PostgreSQL crosstab (needs: CREATE EXTENSION tablefunc;)
-- SELECT * FROM crosstab('SELECT category, month, revenue FROM long_table ORDER BY 1,2')
--   AS ct(category text, jan numeric, feb numeric, mar numeric, apr numeric);
```

The picture

```
LONG (unpivoted)                        WIDE (pivoted)
category     month    revenue           category     jan   feb   mar   apr
Electronics  2024-01  2100     --PIVOT-->  Electronics  2100  1500  1200  2700
Electronics  2024-02  1500     <-UNPIVOT-- Home            0   900   900  1800
Electronics  2024-03  1200                 Stationery    160   120   880   400
Electronics  2024-04  2700
... (12 rows)                             (3 rows)
```

Connection to the real job

Analysts pivot for reports and export to Excel or Power BI. Data engineers and scientists unpivot because tidy long data is easy to group and feed to models. Many "weird" Excel exports have one column per month; unpivoting is the first cleaning step.

Applications

- **Data Analytics:** cross-tab reports, month columns, status columns.
- **Data Engineering:** normalising wide exports into fact tables.
- **Data Science and ML:** reshaping to tidy format, creating one feature column per category (one-hot style).
- **Education:** students as rows, one column per subject (maths, science, English) in a mark sheet.

COMMON MISTAKE

Hard-coding months in the pivot, then forgetting to add the new month next quarter. Rows from months not listed vanish silently. Check that the pivot's grand total equals the unpivoted total (here 12660).

Check yourself

1. What replaces the missing `PIVOT` keyword in portable SQL?

Answer: Conditional aggregation (`SUM(CASE WHEN ...)`).

1. Which format is better for charting libraries and GROUP BY?

Answer: Long format.

1. How do you unpivot with plain SQL?

Answer: One `SELECT` per column, joined with `UNION ALL` .

7.3 GROUPING SETS, ROLLUP and CUBE

INTUITION

A finance report shows revenue per product, a subtotal per category, and a grand total at the bottom. Without special syntax you would write three queries and stack them. `ROLLUP` does it in one. `GROUPING SETS` lets you pick exactly which groupings you want. `CUBE` gives all combinations.

Words you need

- **Grouping set**: one particular `GROUP BY` list, such as `(category, product)` or `(category)` or `()` .
- **Subtotal row**: a row that summarises a group; the grouped-out columns appear as NULL.
- **Grand total**: the grouping set `()` , which has no columns, so everything is one group.

Definitions (PostgreSQL, SQL Server, MySQL 8, Oracle, Snowflake, BigQuery; not SQLite)

- `GROUP BY GROUPING SETS ((a, b), (a), ())` computes each listed grouping.
- `GROUP BY ROLLUP (a, b)` is shorthand for `GROUPING SETS ((a, b), (a), ())` . It rolls up from the right: n columns give n + 1 sets.

- `GROUP BY CUBE (a, b)` is shorthand for `GROUPING SETS ((a, b), (a), (b), ())`: all combinations, 2 to the power n sets.
- The function `GROUPING(col)` returns 1 when the column was rolled up in this row and 0 otherwise. Use it to tell a rolled-up NULL from a real NULL in the data.

Real-world mapping

Feature	Example	Where it appears in the job
`ROLLUP (category, product)`	product rows, category subtotals, grand total	finance and sales reports
`CUBE (region, product)`	totals for every slice	OLAP cubes, pivot-like analysis
`GROUPING SETS`	only the slices you ask for	dashboards with several summary levels

Worked examples

SQLite does not have these three, but we can **prove what they mean** by writing the same thing with `UNION ALL` (which SQLite runs), and then show the standard syntax.

Problem 7.3.1 (the long way) Revenue by category and product, with category subtotals and a grand total, using `UNION ALL`. All orders count here.

Step 1	Detail rows: group by `(category, product)`.
Step 2	Subtotals: group by `(category)`, with product shown as `'ALL'`.
Step 3	Grand total: no grouping, with both labels `'ALL'`.

```
SELECT p.category, p.name AS product, SUM(oi.qty*oi.unit_price) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category, p.name
UNION ALL
SELECT p.category, 'ALL', SUM(oi.qty*oi.unit_price)
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
UNION ALL
SELECT 'ALL', 'ALL', SUM(oi.qty*oi.unit_price)
FROM order_items oi
ORDER BY 1, 2;
```

```
category     product    revenue
-----------  ---------  -------
ALL          ALL        14560
Electronics  ALL        9000
Electronics  Keyboard   6000
Electronics  Mouse      3000
Home         ALL        3600
Home         Desk Lamp  3600
Stationery   ALL        1960
Stationery   Notebook   1120
Stationery   Pen Pack   840
```

- **Compute** Electronics: Keyboard 6000 and Mouse 3000, subtotal 9000. Home: Desk Lamp 3600, subtotal 3600 (the Mug never sold). Stationery: Notebook 1120 and Pen Pack 840, subtotal 1960. Grand total `9000 + 3600 + 1960 = 14560`.
- **Check** The grand total equals the total revenue from Chapter 3 (14560).
- **Answer** Nine rows in all: 5 detail rows, 3 subtotal rows and 1 grand-total row.
- **Meaning** Three scans of the data. In a big table, `ROLLUP` does it in one pass, which is why it exists.

The same result in PostgreSQL, one query:

```
-- PostgreSQL syntax, not run here
SELECT COALESCE(p.category, 'ALL')                AS category,
       COALESCE(p.name, 'ALL')                     AS product,
       SUM(oi.qty * oi.unit_price)                 AS revenue,
       GROUPING(p.category, p.name)                AS grouping_level
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY ROLLUP (p.category, p.name)
ORDER BY p.category NULLS LAST, p.name NULLS LAST;
```

`GROUPING(p.category, p.name)` is a bit mask: 0 means detail row, 1 means product rolled up (category subtotal), 3 means both rolled up (grand total).

Problem 7.3.2 `GROUPING SETS` and `CUBE`, counted by hand. How many result rows do we expect for `CUBE (category, month)` on our data? There are 3 categories and 4 months.

Step 1	`CUBE` has 4 grouping sets: `(category, month)`, `(category)`, `(month)`, `()`.
Step 2	Rows per set: `3 × 4 = 12` (only combinations that exist; with all 12 present), 3, 4, and 1.
Step 3	If every (category, month) pair exists, the total is `12 + 3 + 4 + 1 = 20`. Let us see whether that holds.

```
-- PostgreSQL syntax, not run here
SELECT p.category, substr(o.order_date::text, 1, 7) AS month, SUM(oi.qty*oi.unit_price) AS revenue
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id AND o.status = 'paid'
JOIN products p ON p.product_id = oi.product_id
GROUP BY CUBE (p.category, substr(o.order_date::text, 1, 7));
```

We can verify the count of 20 by running the four pieces in SQLite and counting them:

```
SELECT COUNT(*) AS cube_rows FROM (
  SELECT p.category, substr(o.order_date,1,7) AS m FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id AND o.status='paid'
    JOIN products p ON p.product_id = oi.product_id GROUP BY 1,2
  UNION ALL
  SELECT p.category, NULL FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id AND o.status='paid'
    JOIN products p ON p.product_id = oi.product_id GROUP BY 1
  UNION ALL
  SELECT NULL, substr(o.order_date,1,7) FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id AND o.status='paid'
    GROUP BY 2
  UNION ALL
  SELECT NULL, NULL
);
```

```
cube_rows
---------
19
```

- **Check** The detail set has fewer than 12 rows if a (category, month) pair has no paid sales. Home has no paid sales in January, so the detail set has 11 rows. Total: `11 + 3 + 4 + 1 = 19`.
- **Answer** 19 rows on our data (not 20), because the combination (Home, January) does not exist and `GROUP BY` only creates groups that occur in the data. The printed count agrees with this.
- **Meaning** `CUBE` and `ROLLUP` never invent empty groups. If you need empty combinations to appear as zero, build a grid with a `CROSS JOIN` first (Chapter 4).

Problem 7.3.3 Which one to pick?

Goal	Use
Hierarchy: product, category, grand total	`ROLLUP (category, product)`
Totals for every combination of two or three dimensions	`CUBE`
Only specific summary levels, e.g. by city and by category separately	`GROUPING SETS ((city), (category))`

- **Answer** The table above.
- **Meaning** `GROUPING SETS` is the general form, the other two are shorthand. For three columns, `CUBE` gives 8 sets; list only what you need to avoid wasted work.

The picture

```
ROLLUP (category, product)              grouping sets produced
-----------------------------------     ----------------------
Electronics | Keyboard | 6000            (category, product)
Electronics | Mouse    | 3000            (category, product)
Electronics | ALL      | 9000     <----  (category)          subtotal
Home        | Desk Lamp| 3600            (category, product)
Home        | ALL      | 3600     <----  (category)          subtotal
...                                       ...
ALL         | ALL      | 14560    <----  ()                  grand total
```

Connection to the real job

Management reports always have subtotals. BI tools (Power BI, Tableau) compute these on their own, but in a warehouse you often produce them in SQL. `CUBE` and `ROLLUP` are the SQL idea behind OLAP (Chapter 11).

Applications

- **Data Analytics:** financial statements, sales by region with subtotals.
- **Data Engineering:** pre-aggregated summary tables for dashboards.
- **Retail:** sales by store, by region, and overall in one result set.

COMMON MISTAKE

Mistaking a subtotal NULL for a real NULL. If a product name can be NULL in the data, `GROUP BY ROLLUP` shows two kinds of NULLs. Use `GROUPING(col)` to tell them apart. Also, trying `ROLLUP` in SQLite or older MySQL (`WITH ROLLUP` is the MySQL form) will fail; check your database.

Check yourself

1. How many grouping sets does `ROLLUP (a, b, c)` produce?

Answer: 4: (a,b,c), (a,b), (a), ().

1. How many does `CUBE (a, b, c)` produce?

Answer: 8.

1. What does `GROUPING(col) = 1` mean?

Answer: The column was rolled up in this row, so it is a subtotal NULL.

7.4 Reporting Lab: ROLLUP, Pivot and Unpivot You Can Run Anywhere

INTUITION	A finance report has detail lines, a subtotal under each group and a grand total at the bottom. `ROLLUP` is the keyword that produces all of those in one query. SQLite does not have it, but a report is only a stack of group-by queries at different levels, so `UNION ALL` builds the same thing. Doing it by hand once shows exactly what `ROLLUP` and `GROUPING()` do for you.

Problem 7.4.1 Build a revenue report by category and month with a subtotal per category and a grand total, using only portable SQL. First a view of paid lines, then three group-by levels. Predict first: 3 categories x months with sales, plus one subtotal row per category, plus one grand-total row.

```
CREATE VIEW paid_lines AS
SELECT p.category, substr(o.order_date, 1, 7) AS month, i.qty * i.unit_price AS amt
FROM orders o JOIN order_items i ON i.order_id = o.order_id JOIN products p ON p.product_id = i.product_id
WHERE o.status = 'paid';
```

```
SELECT category, month, revenue FROM (
  SELECT category, month, sum(amt) AS revenue FROM paid_lines GROUP BY category, month
  UNION ALL
  SELECT category, 'ALL', sum(amt) FROM paid_lines GROUP BY category
  UNION ALL
  SELECT 'ALL', 'ALL', sum(amt) FROM paid_lines
)
ORDER BY (category = 'ALL'), category, (month = 'ALL'), month;
```

```
category     month    revenue
-----------  -------  -------
Electronics  2024-01  2100
Electronics  2024-02  1500
Electronics  2024-03  1200
Electronics  2024-04  2700
Electronics  ALL      7500
Home         2024-02  900
Home         2024-03  900
Home         2024-04  1800
Home         ALL      3600
Stationery   2024-01  160
Stationery   2024-02  120
Stationery   2024-03  880
Stationery   2024-04  400
Stationery   ALL      1560
ALL          ALL      12660
```

Step 1	Level 0 groups by both columns: the detail rows. Level 1 groups by category only and writes the text `ALL` in the month column. Level 2 has no group at all, one grand-total row.
Step 2	All three branches have the same column list and types, so `UNION ALL` is legal. Use `UNION ALL`, not `UNION`, so identical totals are not merged.
Step 3	The three queries sit inside one outer `SELECT` because an `ORDER BY` on a `UNION` may only name result columns. The outer `ORDER BY` puts the `ALL` rows after the details by sorting on the boolean tests `(month = 'ALL')`, which are 0 or 1.
Check	Each category subtotal equals the sum of its detail rows, and the grand total equals the sum of the three subtotals.
PostgreSQL form	`GROUP BY ROLLUP (category, month)` gives the same rows, with `NULL` instead of `ALL`, in one pass over the data (not run here). Use `GROUPING(month)`, which is 1 when the `NULL` is a subtotal marker and 0 when the data itself was `NULL`, to label them.

COMMON MISTAKE

Using `COALESCE(month, 'ALL')` to label subtotal rows in a `ROLLUP` query. If the data has a real `NULL` month, it is also relabelled `ALL` and you cannot tell a data gap from a subtotal. Use `CASE WHEN GROUPING(month) = 1 THEN 'ALL' ELSE month END`. The same holds for the hand-made version here: if `month` could be NULL, the detail branch must handle it first.

Operator	Levels produced for (a, b)	Rows if a has 3 values and b has 4 (all combinations present)
`GROUP BY a, b`	(a,b)	12
`ROLLUP (a, b)`	(a,b), (a), ()	12 + 3 + 1 = 16
`CUBE (a, b)`	(a,b), (a), (b), ()	12 + 3 + 4 + 1 = 20
`GROUPING SETS ((a),(b))`	exactly the sets you list	3 + 4 = 7

Rule of thumb: `ROLLUP` is for hierarchies (country, state, city), where the order of columns matters. `CUBE` is for independent dimensions where you want every cross-total. `GROUPING SETS` is for when you know exactly which levels the report needs, and it is the cheapest.

Dynamic pivot: the column list comes from the data

Plain SQL cannot create columns whose names depend on data, but your program can write the SQL. Query the distinct values, build one `SUM(CASE ...)` per value, then run the generated text. This is the standard answer to the interview question "pivot when I do not know the months in advance".

Problem 7.4.2 Generate a pivot of sales by category with one column per month found in the data. Predict first: two month columns now, and three after a March row arrives.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE sales (category TEXT, month TEXT, amt INTEGER);
INSERT INTO sales VALUES ('Electronics','2024-01',2100),('Electronics','2024-02',1500),
 ('Stationery','2024-01',160),('Stationery','2024-02',120),('Home','2024-02',900);
""")
months = [r[0] for r in db.execute("SELECT DISTINCT month FROM sales ORDER BY month")]
cols = ",\n  ".join("sum(CASE WHEN month = '%s' THEN amt ELSE 0 END) AS \"m%s\"" % (m, m.replace("-", "_")) for m in months)
sql = "SELECT category,\n  " + cols + "\nFROM sales GROUP BY category ORDER BY category"
print(sql)
print()
cur = db.execute(sql)
print([d[0] for d in cur.description])
for row in cur: print(row)
db.execute("INSERT INTO sales VALUES ('Home','2024-03',250)")
months = [r[0] for r in db.execute("SELECT DISTINCT month FROM sales ORDER BY month")]
print("after a new month arrives, columns would be:", months)
```

```
SELECT category,
  sum(CASE WHEN month = '2024-01' THEN amt ELSE 0 END) AS "m2024_01",
  sum(CASE WHEN month = '2024-02' THEN amt ELSE 0 END) AS "m2024_02"
FROM sales GROUP BY category ORDER BY category

['category', 'm2024_01', 'm2024_02']
('Electronics', 2100, 1500)
('Home', 0, 900)
('Stationery', 160, 120)
after a new month arrives, columns would be: ['2024-01', '2024-02', '2024-03']
```

Step 1	The first query asks the data which months exist.
Step 2	A string join builds one aggregate column per month. Column aliases are quoted and cleaned (`-` becomes `_`), since a name like `2024-01` is not a legal bare identifier.
Step 3	The generated SQL is ordinary SQL, so you can print and inspect it. Electronics has 2100 in January and 1500 in February, as the data says.
Step 4	When March data appears, the list of months grows to three and the next run of the generator produces three columns. Nobody edits the query.

COMMON MISTAKE	Building a dynamic pivot by pasting *user-supplied* values into the SQL text. The month list above comes from your own table, but if it came from a web form, it would be an injection hole (Section 9.6). Validate values against the database, or against an allow-list, before they become identifiers. Placeholders cannot be used for column names.

Unpivot: wide back to long

The reverse trip, one row per (category, quarter), is needed whenever a spreadsheet-style table must be loaded into a tidy fact table or plotted. The portable way is one `SELECT` per column joined by `UNION ALL`:

Problem 7.4.3 Turn the wide `targets` table (one column per quarter) into a long table. Predict first: 2 categories x 3 quarters = 6 rows.

```
CREATE TABLE targets (category TEXT, q1 INTEGER, q2 INTEGER, q3 INTEGER);
INSERT INTO targets VALUES ('Electronics', 5000, 5500, 6000), ('Home', 2000, 2100, 2300);
SELECT category, 'q1' AS quarter, q1 AS target FROM targets
UNION ALL SELECT category, 'q2', q2 FROM targets
UNION ALL SELECT category, 'q3', q3 FROM targets
ORDER BY category, quarter;
```

```
category     quarter  target
-----------  -------  ------
Electronics  q1       5000
Electronics  q2       5500
Electronics  q3       6000
Home         q1       2000
Home         q2       2100
Home         q3       2300
```

Step 1	Each `SELECT` reads one wide column and writes its name as text in the `quarter` column.
Step 2	Six rows come back, as predicted: 2 categories times 3 quarters.
Native forms	SQL Server and Oracle write `UNPIVOT (target FOR quarter IN (q1, q2, q3))` . PostgreSQL writes `CROSS JOIN LATERAL (VALUES ('q1', q1), ('q2', q2), ('q3', q3)) AS v(quarter, target)` . BigQuery has `UNPIVOT` . Snowflake has `UNPIVOT` too. None of these four were run here.

SQLite can also do it by expanding a JSON object, which scales to many columns without repeating the pattern. It returns the same six rows:

```
SELECT t.category, j.key AS quarter, j.value AS target
FROM targets t, json_each(json_object('q1', t.q1, 'q2', t.q2, 'q3', t.q3)) AS j
ORDER BY t.category, j.key;
```

```
category     quarter  target
-----------  -------  ------
Electronics  q1       5000
Electronics  q2       5500
Electronics  q3       6000
Home         q1       2000
Home         q2       2100
Home         q3       2300
```

GOING DEEPER

A pivot is a `GROUP BY` with an aggregate per output column, so it is only correct when each (row key, column key) pair has *at most one* source row or you intend to aggregate duplicates. If the long table has two rows for (Home, 2024-02), `SUM` silently adds them and `MAX` silently drops one. Before pivoting, check uniqueness: `SELECT category, month, count(*) FROM sales GROUP BY 1, 2 HAVING count(*) > 1`. Wide tables are for people and charts; keep storage and joins long.

Practice with answers

Problem 7.4.4 A report needs revenue per (region, product) with a subtotal per region only and a grand total, but no per-product subtotal across regions. Which operator? *(easy)*

Answer: `ROLLUP (region, product)` gives (region, product), (region) and the grand total, which is exactly this. `CUBE` would add a per-product-across-regions row set you do not want.

Problem 7.4.5 `ROLLUP (a, b, c)` on 2 x 3 x 4 distinct combinations, all present, how many rows? *(medium)*

Answer: Levels: (a,b,c) = 24, (a,b) = 6, (a) = 2, () = 1. Total 24 + 6 + 2 + 1 = 33.

Problem 7.4.6 A teammate writes the hand-made rollup with `UNION` instead of `UNION ALL`. Name two problems. *(find the bug)*

Answer: First, `UNION` removes duplicate rows, so any two branches that happen to produce an identical row (same labels, same amount) would silently lose one, and the totals would no longer add up. Second, `UNION` must sort or hash all rows to find duplicates, which costs time for nothing. Use `UNION ALL` whenever you know the branches do not overlap.

Chapter recap

- `UNION ALL` stacks all rows; `UNION` also removes duplicates; use `UNION ALL` unless you need de-duplication.
- `INTERSECT` finds rows in both; `EXCEPT` finds rows in the first but not the second, and order matters.
- Set operations match columns by position and need compatible types; `ORDER BY` goes at the end.
- Use brackets when mixing set operators; databases differ in precedence.
- Pivot (long to wide) is `SUM(CASE WHEN ...)` per output column; the values must be known in advance.
- Unpivot (wide to long) is one `SELECT` per column joined by `UNION ALL`, or `LATERAL (VALUES ...)`.
- `GROUPING SETS`, `ROLLUP` and `CUBE` make subtotals in one pass; SQLite lacks them, emulate with `UNION ALL`.
- `CUBE` and `ROLLUP` never create empty groups.

Need	Pattern
Stack tables	`SELECT ... UNION ALL SELECT ...`
Who is in A not B	`A EXCEPT B` or anti-join
Who is in both	`A INTERSECT B`
Wide report	`SUM(CASE WHEN m='2024-01' THEN v END) AS jan`
Long format	`SELECT k,'jan',jan FROM t UNION ALL SELECT k,'feb',feb FROM t`
Subtotals	`GROUP BY ROLLUP (a, b)`

Practice set

Problem 7.P.1 (easy) How many rows: `SELECT 1 UNION ALL SELECT 1 UNION ALL SELECT 1` ?

```
SELECT 1 UNION ALL SELECT 1 UNION ALL SELECT 1;
```

```
1
-
1
1
1
```

Answer: 3. With `UNION` it would be 1.

Problem 7.P.2 (easy) Customers in Pune who also ordered in March, using `INTERSECT`.

```
SELECT customer_id FROM customers WHERE city = 'Pune'
INTERSECT
SELECT customer_id FROM orders WHERE order_date LIKE '2024-03%';
```

```
customer_id
-----------
1
```

Answer: Customer 1 (Asha, order 106). Pune customers are 1, 3, 8; March buyers are 1, 5, 2, 6.

Problem 7.P.3 (what is the output?) What does this return?

```
SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders
ORDER BY customer_id;
```

```
customer_id
-----------
7
8
```

Answer: 7 and 8, customers who never ordered (an anti-join written with `EXCEPT`).

Problem 7.P.4 (find the bug) A pivot query shows revenue for months 2024-01 to 2024-03 only, and the grand total is 10,000 lower than the source. What is likely wrong? **Answer:** A month is missing from the pivot columns, so its rows are dropped (here April). Reconcile the pivot grand total with `SELECT SUM(revenue)` from the long table, and add the missing column.

Problem 7.P.5 Pivot: number of orders per customer city, one column per city.

```
SELECT COUNT(*) FILTER (WHERE c.city = 'Pune')      AS pune,
       COUNT(*) FILTER (WHERE c.city = 'Mumbai')    AS mumbai,
       COUNT(*) FILTER (WHERE c.city = 'Delhi')     AS delhi,
       COUNT(*) FILTER (WHERE c.city = 'Bengaluru') AS bengaluru
FROM orders o JOIN customers c ON c.customer_id = o.customer_id;
```

```
pune  mumbai  delhi  bengaluru
----  ------  -----  ---------
5     4       2      3
```

Answer: Pune 5 (Asha 4 + Meena 1), Mumbai 4 (Ravi 3 + Kabir 1), Delhi 2 (John), Bengaluru 3 (Priya). Total 14.

Problem 7.P.6 Unpivot `cat_month` for one category (Home) to a three-column long table, using `UNION ALL`, for March and April only.

```
SELECT category, 'mar' AS month, mar AS revenue FROM cat_month WHERE category = 'Home'
UNION ALL
SELECT category, 'apr', apr FROM cat_month WHERE category = 'Home';
```

```
category  month  revenue
--------  -----  -------
Home      mar    900
Home      apr    1800
```

Answer: (Home, mar, 900) and (Home, apr, 1800).

Problem 7.P.7 Report: revenue per category with a grand total row, written with `UNION ALL` (so it runs in SQLite). All orders count.

```
SELECT p.category, SUM(oi.qty*oi.unit_price) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
UNION ALL
SELECT 'TOTAL', SUM(qty*unit_price) FROM order_items
ORDER BY revenue;
```

```
category     revenue
-----------  -------
Stationery   1960
Home         3600
Electronics  9000
TOTAL        14560
```

Answer: Stationery 1960, Home 3600, Electronics 9000, TOTAL 14560.

Problem 7.P.8 (scenario) You must compare the list of customer ids in last quarter's mailing with this quarter's, and report who dropped off, who is new, and who stayed. Which operators? **Answer:** `last EXCEPT this` for dropped, `this EXCEPT last` for new, `last INTERSECT this` for stayed. Or a `FULL JOIN` with `IS NULL` tests.

Problem 7.P.9 (interview) `UNION` versus `UNION ALL`: differences and when to prefer each. **Answer:** `UNION` removes duplicate rows (it needs sorting or hashing, so it is slower); `UNION ALL` keeps everything and is faster. Prefer `UNION ALL` unless you need unique rows, and when you know the inputs cannot overlap, use `UNION ALL` for speed.

Problem 7.P.10 (interview) Explain `ROLLUP` versus `CUBE` with an example for `(year, region)`. **Answer:** `ROLLUP (year, region)` gives rows for (year, region), (year), and grand total: a hierarchy. `CUBE (year, region)` also adds (region): all combinations. For 2 columns: ROLLUP has 3 grouping sets, CUBE has 4.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my second language. Teach me Chapter 7 topic: "SQL set operations: UNION, UNION ALL, INTERSECT, EXCEPT". Use two small customer lists and draw the result of each operator. After each operator ask me to predict the output, then check my answer.
```

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am learning SQL. Topic: pivot and unpivot using conditional aggregation and UNION ALL, with PostgreSQL crosstab and SQL Server PIVOT as extras. Give me 8 graded exercises that convert between long and wide tables on a sales dataset, with no solutions at first. Review my answers when I paste them and check that grand totals reconcile.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. Explain GROUPING SETS, ROLLUP and CUBE to a beginner. Use a sales table by region and product. Show how many grouping sets each produces, how the GROUPING function works, and the equivalent UNION ALL query. Ask me one check question after each idea and wait for my answer.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am preparing for analyst interviews. Review my SQL below for set operation mistakes (UNION versus UNION ALL, column order, brackets with INTERSECT) and for pivot mistakes (missing categories, wrong totals). Explain every problem simply and give a corrected query. My SQL: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. Run a mock SQL interview on retention-style set logic and reshaping: customers active in both months, customers lost, wide report from a long table, subtotals with ROLLUP. One question at a time, grade me out of 10, show the model solution in PostgreSQL, and give one common mistake to avoid.

CHAPTER 8

WINDOW FUNCTIONS IN DEPTH

Calculations across related rows without collapsing them: ranking, previous and next row, running totals, moving averages, top-N per group, gaps and islands.

`GROUP BY` squeezes many rows into one. But many real questions need the detail rows **and** a number computed from their neighbours: "each order and its rank within the customer", "each month and the change from last month", "a running total of revenue", "the top 2 products in every category". These cannot be written cleanly with `GROUP BY`. **Window functions** solve them. They are the most tested SQL topic in data interviews and the biggest step up from beginner to professional. By the end you can rank, compare to the previous row, build running totals and moving averages, pick top-N per group, remove duplicates, and find streaks.

For this chapter we save a helper query as a **view** (a named, saved query; Chapter 9 explains views). It gives one row per order with its total value.

```
CREATE VIEW order_totals AS
SELECT o.order_id, o.customer_id, o.order_date, o.status,
       SUM(oi.qty * oi.unit_price) AS total
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.customer_id, o.order_date, o.status;

SELECT COUNT(*) AS n_orders, SUM(total) AS all_revenue FROM order_totals;
```

```
n_orders  all_revenue
--------  -----------
14        14560
```

8.1 The OVER Clause: PARTITION BY and ORDER BY

INTUITION

Imagine a class mark sheet. A `GROUP BY` report prints one line per class: the class average. A window report keeps every student's line and **adds a column** with their class average next to their own mark. Each row looks through a "window" at the other rows of its class, and the rows are never merged.

Words you need

- **Window function**: a function followed by `OVER (...)`. It computes a value for each row using a set of related rows.
- **Partition**: a group of rows that belong together (like one class). `PARTITION BY customer_id` makes one partition per customer. No `PARTITION BY` means one partition: the whole table.
- **Window ordering**: `ORDER BY` inside `OVER` sorts the rows inside each partition, for functions that care about order.
- **Frame**: the sub-range of the partition used for each row (Section 8.4).

Shape

```
function(args) OVER (
    PARTITION BY col1, col2     -- optional: split into groups
    ORDER BY col3               -- optional: order inside each group
    ROWS BETWEEN ... AND ...    -- optional: frame
)
```

Any aggregate (`SUM` , `AVG` , `COUNT` , `MIN` , `MAX`) can be used as a window function by adding `OVER` . Window functions run **after** `WHERE` , `GROUP BY` and `HAVING` , and **before** `ORDER BY` and `LIMIT` . Because of this you cannot use them in `WHERE` ; wrap the query in a CTE and filter outside (Snowflake, BigQuery and DuckDB have `QUALIFY` for this).

Real-world mapping

Window idea	Example	Where it appears in the job
`SUM(x) OVER ()`	row's share of grand total	percent-of-total columns
`AVG(x) OVER (PARTITION BY k)`	row vs its group's average	outlier flags
`ROW_NUMBER() OVER (...)`	order within a user	"first purchase", dedup
`LAG(x) OVER (ORDER BY t)`	previous value	change since last period

Worked examples

Problem 8.1.1 (trivial) For each order, show its total and the share of all revenue. Show the first 4 orders. Predict: `SUM(total) OVER ()` equals 14560 on every row.

Step 1	`SUM(total) OVER ()` has empty brackets: one partition, the whole table, so every row sees 14560.
Step 2	Share = `total / 14560 × 100` .

```
SELECT order_id, total,
       SUM(total) OVER () AS grand_total,
       ROUND(100.0 * total / SUM(total) OVER (), 1) AS share_pct
FROM order_totals
ORDER BY order_id
LIMIT 4;
```

```
order_id  total  grand_total  share_pct
--------  -----  -----------  ---------
101       1660   14560        11.4
102       600    14560        4.1
103       1020   14560        7.0
104       400    14560        2.7
```

- **Compute** Order 101: `1660 / 14560 = 0.1140` , so 11.4 percent. Order 102: `600 / 14560 = 0.0412` , 4.1 percent.
- **Check** All 14 rows are still there (a `GROUP BY` would have given one row). Column `grand_total` is 14560 on each row.
- **Answer** Order 101 is 11.4 percent of revenue, 102 is 4.1, 103 is 7.0 (`1020 / 14560`), 104 is 2.7.
- **Meaning** You got detail and total on the same line, with no join back to a summary table.

Problem 8.1.2 `PARTITION BY` : compare each order to the average of **that customer's** orders. Show customers 1 and 5.

Step 1	`AVG(total) OVER (PARTITION BY customer_id)` makes one average per customer.
Step 2	Subtract to see whether the order is above or below the customer's norm.

```
SELECT customer_id, order_id, total,
       ROUND(AVG(total) OVER (PARTITION BY customer_id), 1) AS cust_avg,
       ROUND(total - AVG(total) OVER (PARTITION BY customer_id), 1) AS diff
FROM order_totals
WHERE customer_id IN (1, 5)
ORDER BY customer_id, order_id;
```

```
customer_id  order_id  total  cust_avg  diff
-----------  --------  -----  --------  ------
1            101       1660   1140.0    520.0
1            103       1020   1140.0    -120.0
1            106       1280   1140.0    140.0
1            111       600    1140.0    -540.0
5            107       360    753.3     -393.3
5            109       1500   753.3     746.7
5            113       400    753.3     -353.3
```

- **Compute** Customer 1: totals 1660, 1020, 1280, 600; sum 4560, average `4560 / 4 = 1140`. Customer 5: totals 360, 1500, 400; sum 2260, average `2260 / 3 = 753.3`.
- **Check** For customer 1 the diffs are `+520, −120, +140, −540`, and they add to 0 (a mean's deviations always cancel).
- **Answer** Customer 1 average 1140; customer 5 average 753.3, with its refunded order 109 (1500) well above it.
- **Meaning** The filter `WHERE customer_id IN (1,5)` ran **before** the window, so the averages use only those customers' rows, which is right here because partitions are per customer.

Problem 8.1.3 Window versus GROUP BY, side by side, and a named window. Count orders per customer in both ways.

```
SELECT customer_id, COUNT(*) AS n_orders
FROM order_totals GROUP BY customer_id ORDER BY customer_id;
```

```
customer_id  n_orders
-----------  --------
1            4
2            3
3            1
4            2
5            3
6            1
```

```
SELECT DISTINCT customer_id, COUNT(*) OVER w AS n_orders
FROM order_totals
WINDOW w AS (PARTITION BY customer_id)
ORDER BY customer_id;
```

```
customer_id  n_orders
-----------  --------
1            4
2            3
3            1
4            2
5            3
6            1
```

- **Check** Both give 4, 3, 1, 2, 3, 1 for customers 1 to 6.
- **Answer** Same numbers. The window version needed `DISTINCT` to remove repeated rows.
- **Meaning** If you want one row per group, use `GROUP BY`. Use windows when you want to keep every row. `WINDOW w AS (...)` names a window so that you can reuse it.

The picture

```
GROUP BY customer_id              PARTITION BY customer_id
---------------------             ------------------------------------
cust | n                          order | cust | total | cust_avg
1    | 4          (4 rows -> 1)  101   | 1    | 1660  | 1140   <- partition 1
3    | 1                         103   | 1    | 1020  | 1140
                                 ...   (all rows kept)
                                 104   | 3    | 400   | 400    <- partition 2
```

Connection to the real job

Percent-of-total, difference-from-average and "rank within group" columns are everyday dashboard needs. For example: each delivery's time next to the 90th percentile for its city.

Applications

- **Data Analytics:** share of total, deviation from group average, ranking customers.
- **Data Science:** group-wise features such as "order value divided by the customer's average".
- **Data Engineering:** de-duplicating records (Section 8.2).
- **Education:** compare each student's mark with the class average.

COMMON MISTAKE

Putting a window function in `WHERE` or `GROUP BY`. It fails (`window functions are not allowed in WHERE`). Compute it in a CTE or subquery, then filter. Another one: forgetting that `WHERE` runs first, so filtered-out rows are not in any window.

Check yourself

1. How many rows does a window function return per input row?

Answer: One (rows are not collapsed).

1. What does `OVER ()` with empty brackets mean?

Answer: One partition: the whole result.

1. Can you use a window function in WHERE?

Answer: No. Use a CTE or subquery.

8.2 Ranking Functions and Top-N per Group

INTUITION

In a race, runners get positions. If two runners tie for second place, what is the next runner's position, third or fourth? Different sports answer differently, and SQL gives you all the answers: `ROW_NUMBER` (never tie), `RANK` (ties share, then skip), `DENSE_RANK` (ties share, no skip).

Words you need

- **Tie**: two rows with equal values in the ordering column.
- **Percentile rank**: the position of a row as a fraction between 0 and 1.
- **Top-N per group**: the best N rows inside each partition.

Definitions

Function	Behaviour with ties	Values for 10, 20, 20, 30 (ascending)
`ROW_NUMBER()`	unique number, ties broken arbitrarily unless you add a tiebreaker	1, 2, 3, 4
`RANK()`	ties share a number; next number is skipped	1, 2, 2, 4
`DENSE_RANK()`	ties share a number; no skip	1, 2, 2, 3
`NTILE(n)`	splits ordered rows into `n` buckets of nearly equal size	e.g. `NTILE(2)`: 1, 1, 2, 2
`PERCENT_RANK()`	`(rank − 1) / (rows − 1)`	0, 0.33, 0.33, 1
`CUME_DIST()`	`rows up to and including this value / total rows`	0.25, 0.75, 0.75, 1

Real-world mapping

Function	Example	Where it appears in the job
`ROW_NUMBER`	latest record per key	de-duplication
`RANK` / `DENSE_RANK`	leaderboard with ties	competition tables, "top 3 salaries"
`NTILE(4)`	quartiles of customers by spend	RFM scoring, deciles
`PERCENT_RANK`	"this order is in the top 20 percent"	percentile labels

Worked examples

Problem 8.2.1 (trivial) Rank all 14 orders by total, biggest first, using all five functions. Predict: three pairs of orders tie (1500 twice, 600 twice, 400 twice).

Step 1	Order by `total DESC`, and `order_id` as tiebreaker for `ROW_NUMBER`.
Step 2	Add the other functions. `RANK` and `DENSE_RANK` must order by `total` only, otherwise the tiebreaker would hide the ties.

```
SELECT order_id, total,
       ROW_NUMBER() OVER (ORDER BY total DESC, order_id) AS row_num,
       RANK()       OVER (ORDER BY total DESC)           AS rnk,
       DENSE_RANK() OVER (ORDER BY total DESC)           AS dense_rnk,
       NTILE(4)     OVER (ORDER BY total DESC, order_id) AS quartile,
       ROUND(PERCENT_RANK() OVER (ORDER BY total DESC), 3) AS pct_rank
FROM order_totals
ORDER BY total DESC, order_id;
```

```
order_id  total  row_num  rnk  dense_rnk  quartile  pct_rank
--------  -----  -------  ---  ---------  --------  --------
114       2100   1        1    1          1         0.0
112       1800   2        2    2          1         0.077
101       1660   3        3    3          1         0.154
105       1500   4        4    4          1         0.231
109       1500   5        4    4          2         0.231
106       1280   6        6    5          2         0.385
103       1020   7        7    6          2         0.462
108       900    8        8    7          2         0.538
102       600    9        9    8          3         0.615
111       600    10       9    8          3         0.615
110       440    11       11   9          3         0.769
104       400    12       12   10         4         0.846
113       400    13       12   10         4         0.846
107       360    14       14   11         4         1.0
```

- **Compute** Sorted totals: 2100, 1800, 1660, 1500, 1500, 1280, 1020, 900, 600, 600, 440, 400, 400, 360. `RANK` : 1, 2, 3, 4, 4, 6, 7, 8, 9, 9, 11, 12, 12, 14. `DENSE_RANK` : 1, 2, 3, 4, 4, 5, 6, 7, 8, 8, 9, 10, 10, 11. `NTILE(4)` of 14 rows: the first two buckets get 4 rows, the last two get 3 (14 = 4+4+3+3). `PERCENT_RANK = (rank - 1) / 13` : for rank 4, `3 / 13 = 0.231` ; for rank 14, `13 / 13 = 1` .
- **Check** Rank 4 appears twice and the next rank is 6, as computed. The last `RANK` is 14 while the last `DENSE_RANK` is 11, which confirms the "skip". `ROW_NUMBER` and `NTILE` use the extra `order_id` tiebreaker, so they never tie.
- **Answer** See the printed table.
- **Meaning** If the question says "top 3 orders", then `RANK <= 3` returns 3 rows, but `RANK <= 4` returns 5 rows because of the tie. Always decide how ties should behave before choosing the function.

Problem 8.2.2 Top-N per group: the two largest orders of each customer.

Step 1	Rank inside each customer: `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC)` .
Step 2	Window functions cannot be filtered in `WHERE` , so put step 1 in a CTE and filter `rn <= 2` outside.

```
WITH ranked AS (
  SELECT customer_id, order_id, total,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC, order_id) AS rn
  FROM order_totals
)
SELECT customer_id, order_id, total, rn
FROM ranked
WHERE rn <= 2
ORDER BY customer_id, rn;
```

```
customer_id  order_id  total  rn
-----------  --------  -----  --
1            101       1660   1
1            106       1280   2
2            114       2100   1
2            108       900    2
3            104       400    1
4            112       1800   1
4            105       1500   2
5            109       1500   1
5            113       400    2
6            110       440    1
```

- **Compute** Customer 1: 1660 (101), 1280 (106). Customer 2: 2100 (114), 900 (108). Customer 3: only 104 (400). Customer 4: 1800 (112), 1500 (105). Customer 5: 1500 (109), 400 (113). Customer 6: only 110 (440).
- **Check** `2 + 2 + 1 + 2 + 2 + 1 = 10` rows.

- **Answer** Ten rows as printed.
- **Meaning** A customer with fewer than N orders simply returns fewer rows. This pattern is the answer to "top 3 products per category" and "latest 5 events per user".

Problem 8.2.3 Top product per category by revenue, keeping ties (`RANK`).

```
WITH rev AS (
  SELECT p.category, p.name, SUM(oi.qty * oi.unit_price) AS revenue
  FROM order_items oi JOIN products p ON p.product_id = oi.product_id
  GROUP BY p.category, p.name
), ranked AS (
  SELECT *, RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS r FROM rev
)
SELECT category, name, revenue FROM ranked WHERE r = 1 ORDER BY category;
```

```
category     name       revenue
-----------  ---------  -------
Electronics  Keyboard   6000
Home         Desk Lamp  3600
Stationery   Notebook   1120
```

- **Check** Electronics: Keyboard 6000 beats Mouse 3000. Home: Desk Lamp 3600 (Mug never sold, so is absent). Stationery: Notebook 1120 beats Pen Pack 840.
- **Answer** Keyboard, Desk Lamp, Notebook.
- **Meaning** The `GROUP BY` ran first, then the window ranked the grouped rows. Windows can sit on top of aggregates.

Problem 8.2.4 Deduplication: a table has repeated emails; keep only the newest row per email.

```
CREATE TABLE raw_users (id INTEGER, email TEXT, updated_at TEXT);
INSERT INTO raw_users VALUES
 (1,'a@x.com','2024-01-01'),(2,'a@x.com','2024-02-01'),
 (3,'b@x.com','2024-01-15'),(4,'a@x.com','2024-02-01');

WITH numbered AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC, id DESC) AS rn
  FROM raw_users
)
SELECT id, email, updated_at FROM numbered WHERE rn = 1 ORDER BY id;
```

```
id  email    updated_at
--  -------  ----------
3   b@x.com  2024-01-15
4   a@x.com  2024-02-01
```

- **Check** `a@x.com` has three rows, two share the latest date 2024-02-01 (ids 2 and 4). The tiebreaker `id DESC` picks id 4. `b@x.com` has one row (id 3).
- **Answer** Rows with id 3 and 4 survive.
- **Meaning** The tiebreaker matters: without it, the winner among ties is arbitrary and can change between runs. Dedup this way before loading into a warehouse.

The picture

```
PARTITION BY category ORDER BY revenue DESC
Electronics:  Keyboard 6000 (rn 1)   Mouse 3000 (rn 2)
Home:         Desk Lamp 3600 (rn 1)
Stationery:   Notebook 1120 (rn 1)   Pen Pack 840 (rn 2)
WHERE rn = 1 keeps the first row of each box.
```

Connection to the real job

"Top N per group", "latest record per key" and "remove duplicates" are three of the five most common SQL interview problems, and daily work for data engineers. `NTILE` drives RFM scoring (Chapter 12).

Applications

- **Data Analytics:** top products, top customers, deciles of spend.
- **Data Engineering:** dedup on `ROW_NUMBER = 1`, keep latest per key (CDC feeds).
- **Machine Learning:** train/validation splits by `NTILE`, stratified samples.
- **Logistics:** slowest 5 deliveries per city.

COMMON MISTAKE

`ROW_NUMBER` with no tiebreaker, and using `RANK` for "top 1 per group" without realising ties return several rows. Also, `ORDER BY` inside `OVER` only orders the window; the final result order needs its own `ORDER BY`.

Check yourself

1. Ranks of values 5, 5, 7 (descending 7, 5, 5) with `RANK`?

Answer: 1, 2, 2.

1. Same with `DENSE_RANK` if there were one more value of 3 after them?

Answer: 7 is 1, 5 and 5 are 2, 3 is 3 (no skip).

1. Why use a CTE to filter `ROW_NUMBER() = 1`?

Answer: Window functions cannot be in `WHERE`.

8.3 Offset Functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

INTUITION

Looking at today's temperature, you want yesterday's. In a spreadsheet you point at the cell above. `LAG` is "the cell above"; `LEAD` is "the cell below". `FIRST_VALUE` is the top of the group; `LAST_VALUE` the bottom.

Definitions

Function	Returns
`LAG(x, n, default)`	the value of `x` from `n` rows earlier (default 1) in the window order; `default` if there is none (else NULL)
`LEAD(x, n, default)`	the value `n` rows later
`FIRST_VALUE(x)`	`x` from the first row of the frame
`LAST_VALUE(x)`	`x` from the last row of the **frame** (see the trap below)
`NTH_VALUE(x, n)`	`x` from the n-th row of the frame

Real-world mapping

Function	Example	Where it appears in the job
`LAG(revenue)`	last month's revenue	MoM growth
`LAG(revenue, 12)`	same month last year	YoY growth
`LAG(order_date)`	days since previous order	churn and repeat-purchase analysis
`FIRST_VALUE`	first purchase category	acquisition analysis

Worked examples

Problem 8.3.1 Month-over-month growth of paid revenue.

Formula	`growth % = (this month − previous month) / previous month × 100`
Step 1	Monthly paid revenue in a CTE: 2260, 2520, 2980, 4900.
Step 2	`LAG(revenue)` brings the previous month's value onto each row.

```
WITH monthly AS (
  SELECT substr(order_date,1,7) AS month, SUM(total) AS revenue
  FROM order_totals WHERE status = 'paid' GROUP BY 1
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / LAG(revenue) OVER (ORDER BY month), 1) AS growth_pct
FROM monthly ORDER BY month;
```

```
month    revenue  prev_revenue  change  growth_pct
-------  -------  ------------  ------  ----------
2024-01  2260     NULL          NULL    NULL
2024-02  2520     2260          260     11.5
2024-03  2980     2520          460     18.3
2024-04  4900     2980          1920    64.4
```

- **Compute** Feb: `(2520 − 2260) / 2260 = 260 / 2260 = 0.1150`, 11.5 percent. Mar: `460 / 2520 = 0.1825`, 18.3 percent. Apr: `1920 / 2980 = 0.6443`, 64.4 percent. January has no previous month, so everything is NULL.
- **Check** `2260 + 260 = 2520`, `2520 + 460 = 2980`, `2980 + 1920 = 4900`.
- **Answer** Growth: NULL, 11.5, 18.3, 64.4 percent.
- **Meaning** April's jump is large. Before celebrating, ask whether April simply had more refunds fixed or a campaign. The query tells you *what*, not *why*.

Problem 8.3.2 Year-over-year with `LAG(revenue, 12)`: we build a 14-month sales table (2023-01 to 2024-02).

```
CREATE TABLE sales_m (month TEXT, revenue INTEGER);
INSERT INTO sales_m VALUES
 ('2023-01',100),('2023-02',110),('2023-03',120),('2023-04',130),('2023-05',140),('2023-06',150),
 ('2023-07',160),('2023-08',170),('2023-09',180),('2023-10',190),('2023-11',200),('2023-12',210),
 ('2024-01',130),('2024-02',121);

SELECT month, revenue,
       LAG(revenue, 12) OVER (ORDER BY month) AS same_month_last_year,
       ROUND(100.0 * (revenue - LAG(revenue, 12) OVER (ORDER BY month))
             / LAG(revenue, 12) OVER (ORDER BY month), 1) AS yoy_pct
FROM sales_m
WHERE month >= '2023-12'
ORDER BY month;
```

```
month    revenue  same_month_last_year  yoy_pct
-------  -------  --------------------  -------
2023-12  210      NULL                  NULL
2024-01  130      NULL                  NULL
2024-02  121      NULL                  NULL
```

Careful, this contains a trap: the `WHERE` runs before the window, so rows before 2023-12 are removed and `LAG(…, 12)` finds nothing. Run the fix, which filters after the window:

```
WITH calc AS (
  SELECT month, revenue,
         LAG(revenue, 12) OVER (ORDER BY month) AS last_year
  FROM sales_m
)
SELECT month, revenue, last_year,
       ROUND(100.0 * (revenue - last_year) / last_year, 1) AS yoy_pct
FROM calc WHERE month >= '2024-01' ORDER BY month;
```

```
month    revenue  last_year  yoy_pct
-------  -------  ---------  -------
2024-01  130      100        30.0
2024-02  121      110        10.0
```

- **Compute** January 2024 versus January 2023: `(130 - 100) / 100 = 0.30`, 30 percent. February: `(121 - 110) / 110 = 0.10`, 10 percent.
- **Check** The first query shows NULLs (the trap); the second shows 100 and 110 as last year's values.
- **Answer** YoY growth is 30.0 percent for January and 10.0 percent for February 2024.
- **Meaning** The lesson: filter *after* a window with a CTE if the window needs the rows you would filter out. Also `LAG(x, 12)` assumes every month is present; a missing month would shift the comparison. A calendar (date spine, Chapter 5) protects against that.

Problem 8.3.3 Days between a customer's consecutive orders, with `LAG` on a date. Customer 1.

```
SELECT order_id, order_date,
       LAG(order_date) OVER (ORDER BY order_date) AS prev_date,
       CAST(julianday(order_date) - julianday(LAG(order_date) OVER (ORDER BY order_date)) AS INTEGER) AS days_gap
FROM order_totals
WHERE customer_id = 1
ORDER BY order_date;
```

```
order_id  order_date  prev_date   days_gap
--------  ----------  ----------  --------
101       2024-01-10  NULL        NULL
103       2024-02-03  2024-01-10  24
106       2024-03-05  2024-02-03  31
111       2024-04-02  2024-03-05  28
```

- **Compute** Jan 10 to Feb 3 is `21 + 3 = 24` days. Feb 3 to Mar 5 is `26 + 5 = 31` (2024 is a leap year, February has 29 days). Mar 5 to Apr 2 is `26 + 2 = 28`.
- **Check** `24 + 31 + 28 = 83`, and Jan 10 to Apr 2 is `21 + 29 + 31 + 2 = 83` days.
- **Answer** Gaps of 24, 31 and 28 days.
- **Meaning** Customer 1 buys about monthly. Averaging these gaps gives the "repeat purchase interval".

Problem 8.3.4 The `LAST_VALUE` trap. For customer 1's orders, show the first and last order date in every row.

```
SELECT order_id, order_date,
       FIRST_VALUE(order_date) OVER (ORDER BY order_date) AS first_date,
       LAST_VALUE(order_date)  OVER (ORDER BY order_date) AS last_wrong,
       LAST_VALUE(order_date)  OVER (ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_right,
       NTH_VALUE(total, 2) OVER (ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS second_order_total
FROM order_totals WHERE customer_id = 1 ORDER BY order_date;
```

```
order_id  order_date  first_date  last_wrong  last_right  second_order_total
--------  ----------  ----------  ----------  ----------  ------------------
101       2024-01-10  2024-01-10  2024-01-10  2024-04-02  1020
103       2024-02-03  2024-01-10  2024-02-03  2024-04-02  1020
106       2024-03-05  2024-01-10  2024-03-05  2024-04-02  1020
111       2024-04-02  2024-01-10  2024-04-02  2024-04-02  1020
```

- **Check** The true last date is 2024-04-02. `last_wrong` equals each row's own date; `last_right` is 2024-04-02 everywhere. The second order (103) has total 1020.
- **Answer** With the default frame `LAST_VALUE` returns the current row. Fix by widening the frame.
- **Meaning** When a window has `ORDER BY`, the default frame ends at the *current row*, so "the last row of the frame" is the current row. Section 8.4 explains frames.

The picture

```
 month     revenue   LAG(revenue)   LAG(revenue, 12)
 2023-01     100        NULL           NULL
 2023-02     110        100            NULL
   ...
 2023-12     210        200            NULL
 2024-01     130        210            100     <- 12 rows above
 2024-02     121        130            110     <- 12 rows above
```

Read it as: `LAG` only looks backwards along the ordering you give in `OVER (ORDER BY ...)`, and `LEAD` looks forwards.

Connection to the real job

Growth rates (MoM, WoW, YoY), "time since last event", and churn signals use `LAG`. Product analytics (first-touch attribution) uses `FIRST_VALUE`.

Applications

- **Data Analytics:** growth rates, change since last period, first and last touch.

- **Data Science and ML:** lag features for time series (value yesterday, value a week ago).
- **Data Engineering:** detect changes between consecutive snapshots.
- **Healthcare:** change in average waiting time between consecutive months.

COMMON MISTAKE

Using `LAG(x, 12)` over data with missing months, and filtering with `WHERE` before the window removes the rows `LAG` needs. Also the `LAST_VALUE` default-frame trap above.

Check yourself

1. What does `LAG(x, 2)` return on the first row?

Answer: NULL (no row two above), unless you give a default.

1. Why does `LAST_VALUE` often look wrong?

Answer: The default frame ends at the current row.

1. Where do you filter if a window needs earlier rows?

Answer: Outside, in an outer query or CTE.

8.4 Frames: Running Totals, Moving Averages, ROWS vs RANGE

INTUITION

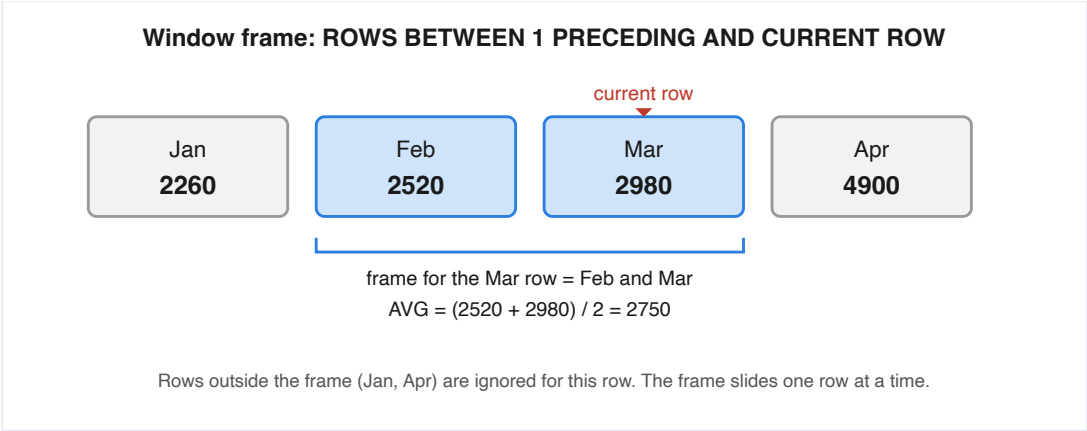
A running total is a pedometer: at each day it shows the sum so far. A moving average is a smoothing glass that looks at the last 7 days only. The *frame* says exactly which rows each row may look at.

Words you need

- **Frame**: the range of rows, inside the partition, used for this row's calculation.
- `UNBOUNDED PRECEDING`: from the first row of the partition. `CURRENT ROW`: this row. `n PRECEDING / n FOLLOWING`: n rows (or n units for `RANGE`) before or after. `UNBOUNDED FOLLOWING`: to the last row.
- **Peers**: rows with the same `ORDER BY` value.

Definitions

- `ROWS BETWEEN a AND b`: counts **physical rows**.
- `RANGE BETWEEN a AND b`: uses the **value** of the `ORDER BY` column; all peers are treated as one.
- `GROUPS` (SQLite 3.28+, PostgreSQL 11+): counts groups of peers.
- **Default frame** when `ORDER BY` is present: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. With no `ORDER BY`: the whole partition.



A moving-average frame: for the March row the frame covers February and March only.

How to read the picture: each box is a row ordered by month. The red arrow marks the current row (March). The blue shaded boxes are the frame `1 PRECEDING to CURRENT ROW`, which includes February and March. The average is taken only over those two boxes. For the next row (April) the frame slides one box to the right.

Real-world mapping

Frame	Example	Where it appears in the job
`UNBOUNDED PRECEDING to CURRENT ROW`	running total	cumulative revenue, year-to-date
`6 PRECEDING to CURRENT ROW`	7-day moving average	smoothing noisy metrics
whole partition	`UNBOUNDED PRECEDING to UNBOUNDED FOLLOWING`	share of group total, last value
`RANGE 30 PRECEDING`	last 30 days by date value	rolling-window KPIs

Worked examples

Problem 8.4.1 (trivial) Running total of monthly paid revenue, and a 2-month moving average.

```
WITH monthly AS (
  SELECT substr(order_date,1,7) AS month, SUM(total) AS revenue
  FROM order_totals WHERE status = 'paid' GROUP BY 1
)
SELECT month, revenue,
       SUM(revenue) OVER (ORDER BY month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
       ROUND(AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 1 PRECEDING AND CURRENT ROW), 1) AS avg_2m
FROM monthly ORDER BY month;
```

```
month    revenue  running_total  avg_2m
-------  -------  -------------  ------
2024-01  2260     2260           2260.0
2024-02  2520     4780           2390.0
2024-03  2980     7760           2750.0
2024-04  4900     12660          3940.0
```

- **Compute** Running total: `2260`, `2260 + 2520 = 4780`, `4780 + 2980 = 7760`, `7760 + 4900 = 12660`. Moving average: Jan has only itself, 2260. Feb `(2260 + 2520) / 2 = 2390`. Mar `(2520 + 2980) / 2 = 2750`. Apr `(2980 + 4900) / 2 = 3940`.
- **Check** The final running total 12660 equals the total paid revenue.
- **Answer** Running totals 2260, 4780, 7760, 12660; two-month averages 2260, 2390, 2750, 3940.
- **Meaning** At the start the frame is short (one row), so the first moving averages use fewer values. Decide whether to show or hide them.

Problem 8.4.2 ROWS versus RANGE with ties. Values 10, 20, 20, 30. What are the running sums?

Step 1	`ROWS`: add row by row: 10, 30, 50, 80.
Step 2	`RANGE` (the default): both 20s are peers, so each sees the sum including all of its peers: row 2 and row 3 both show `10 + 20 + 20 = 50`.

```
CREATE TABLE t (v INTEGER);
INSERT INTO t VALUES (10),(20),(20),(30);

SELECT v,
       SUM(v) OVER (ORDER BY v ROWS  BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_sum,
       SUM(v) OVER (ORDER BY v RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS range_sum,
       SUM(v) OVER (ORDER BY v) AS default_sum
FROM t;
```

```
v   rows_sum  range_sum  default_sum
--  --------  ---------  -----------
10  10        10         10
20  30        50         50
20  50        50         50
30  80        80         80
```

- **Check** `ROWS` gives 10, 30, 50, 80. `RANGE` and the default give 10, 50, 50, 80.
- **Answer** The two 20s get 30 and 50 under `ROWS` but 50 and 50 under `RANGE`.
- **Meaning** This is a famous bug: a "running total" on a date column with several rows on the same date jumps in a surprising way. Say `ROWS` when you mean physical rows.

Problem 8.4.3 Running total per customer and a Pareto (cumulative share) of product revenue.

```
WITH rev AS (
  SELECT p.name, SUM(oi.qty * oi.unit_price) AS revenue
  FROM order_items oi JOIN products p ON p.product_id = oi.product_id GROUP BY p.name
)
SELECT name, revenue,
       SUM(revenue) OVER (ORDER BY revenue DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative,
       ROUND(100.0 * SUM(revenue) OVER (ORDER BY revenue DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
             / SUM(revenue) OVER (), 1) AS cum_pct
FROM rev ORDER BY revenue DESC;
```

```
name       revenue  cumulative  cum_pct
---------  -------  ----------  -------
Keyboard   6000     6000        41.2
Desk Lamp  3600     9600        65.9
Mouse      3000     12600       86.5
Notebook   1120     13720       94.2
Pen Pack   840      14560       100.0
```

- **Compute** Cumulative: `6000`, `9600`, `12600`, `13720`, `14560`. Percent of 14560: `41.2`, `65.9`, `86.5`, `94.2`, `100.0`.
- **Check** The last value is 100.0 percent.
- **Answer** The top two products give 65.9 percent of revenue; the top three give 86.5 percent.
- **Meaning** This is the "80/20" (Pareto) view: a few products carry most revenue.

Problem 8.4.4 `RANGE` with a numeric offset: for each order, how many orders happened in the previous 30 days (including itself)? We verify the window answer with a plain correlated subquery.

```
SELECT order_id, order_date,
       COUNT(*) OVER (ORDER BY julianday(order_date) RANGE BETWEEN 30 PRECEDING AND CURRENT ROW) AS orders_30d,
       (SELECT COUNT(*) FROM orders o2
        WHERE julianday(o2.order_date) BETWEEN julianday(o.order_date) - 30 AND julianday(o.order_date)) AS check_count
FROM orders o ORDER BY order_date;
```

```
order_id  order_date  orders_30d  check_count
--------  ----------  ----------  -----------
101       2024-01-10  1           1
102       2024-01-15  2           2
103       2024-02-03  3           3
104       2024-02-10  3           3
105       2024-02-20  3           3
106       2024-03-05  3           3
107       2024-03-07  4           4
108       2024-03-18  4           4
109       2024-03-25  4           4
110       2024-03-30  5           5
111       2024-04-02  6           6
112       2024-04-09  5           5
113       2024-04-15  6           6
114       2024-04-20  6           6
```

- **Check** The two columns are identical on every row. For example order 106 (March 5): orders from February 4 to March 5 are 104 (Feb 10), 105 (Feb 20) and 106, plus nothing else (103 is February 3, one day too early), so 3.
- **Answer** `orders_30d` equals `check_count`.
- **Meaning** A date-based `RANGE` frame is the right tool for "last 30 days", because it is based on days, not on a fixed number of rows. PostgreSQL supports `RANGE BETWEEN INTERVAL '30 days' PRECEDING AND CURRENT ROW` on date columns (not run here).

Connection to the real job

Running totals (year-to-date), moving averages (7-day smoothing of daily active users) and rolling windows are in every dashboard. For a hospital: a rolling 7-day average of appointments per day.

Applications

- **Data Analytics:** YTD revenue, 7-day averages, Pareto charts.
- **Data Science and ML:** rolling-mean and rolling-sum features (the SQL version of pandas `rolling`).
- **Data Engineering:** incremental running balances.
- **Banking:** rolling 7-day count of transactions at a branch.

COMMON MISTAKE

Forgetting that the default frame with `ORDER BY` is `RANGE`, which treats ties as one, and that `LAST_VALUE` needs a widened frame. Write the frame explicitly.

Check yourself

1. Default frame when `OVER` has `ORDER BY`?

Answer: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

1. What is the frame for a 7-day moving average over daily rows?

Answer: `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.

1. Difference between ROWS and RANGE?

Answer: ROWS counts rows; RANGE uses values and treats ties as one.

8.5 Patterns: Gaps and Islands, Sessionization, Median

INTUITION	Look at a calendar where you ticked the days you went to the gym. A run of consecutive ticks is an **island**. The blank stretch between two islands is a **gap**. The question "what is my longest streak?" is an islands question. The trick: if you subtract the day's *position in the list* from the day itself, all days in one island give the same number.

Words you need

- **Island**: a maximal run of consecutive values (days, ids).
- **Gap**: missing values between two islands.
- **Session**: a burst of events by one user with pauses shorter than a time limit (for example 30 minutes).

Real-world mapping

Pattern	Example	Where it appears in the job
Islands	login streaks	engagement, subscription continuity
Gaps	missing days of data	data-quality (missing partitions)
Sessions	clicks grouped by 30-minute rule	web analytics
Median	typical order value	skew-resistant averages

Worked examples

Problem 8.5.1 Islands. A user logged in on these days of March 2024: 1, 2, 3, 5, 6, 9. Find each streak.

Step 1	Number the rows in date order: 1, 2, 3, 4, 5, 6.
Step 2	Subtract the row number from the day number: `1−1=0, 2−2=0, 3−3=0, 5−4=1, 6−5=1, 9−6=3` . Equal differences mean the same island.
Step 3	Group by that difference.

```
CREATE TABLE logins (day TEXT);
INSERT INTO logins VALUES ('2024-03-01'),('2024-03-02'),('2024-03-03'),
                          ('2024-03-05'),('2024-03-06'),('2024-03-09');

WITH numbered AS (
  SELECT day,
         julianday(day) - ROW_NUMBER() OVER (ORDER BY day) AS grp
  FROM logins
)
SELECT MIN(day) AS island_start, MAX(day) AS island_end, COUNT(*) AS days_in_streak
FROM numbered GROUP BY grp ORDER BY island_start;
```

```
island_start  island_end  days_in_streak
------------  ----------  --------------
2024-03-01    2024-03-03  3
2024-03-05    2024-03-06  2
2024-03-09    2024-03-09  1
```

- **Check** Three islands: March 1 to 3 (3 days), March 5 to 6 (2 days), March 9 (1 day). `3 + 2 + 1 = 6` logins.
- **Answer** The longest streak is 3 days.

- **Meaning** The "date minus row number" trick turns a hard problem into a plain `GROUP BY`. It needs one row per day (use `DISTINCT` first if there can be duplicates).

Problem 8.5.2 Gaps with `LEAD`: list the missing stretches.

```
SELECT day AS last_day_before_gap,
       LEAD(day) OVER (ORDER BY day) AS next_login,
       CAST(julianday(LEAD(day) OVER (ORDER BY day)) - julianday(day) AS INTEGER) - 1 AS missing_days
FROM logins
ORDER BY day;
```

```
last_day_before_gap  next_login  missing_days
-------------------  ----------  ------------
2024-03-01           2024-03-02  0
2024-03-02           2024-03-03  0
2024-03-03           2024-03-05  1
2024-03-05           2024-03-06  0
2024-03-06           2024-03-09  2
2024-03-09           NULL        NULL
```

- **Check** Rows where `missing_days` is above 0: March 3 to March 5 misses 1 day (the 4th); March 6 to March 9 misses 2 days (the 7th and 8th). Consecutive days show 0, and the last row shows NULL.
- **Answer** Two gaps, of 1 and 2 days.
- **Meaning** Filter `missing_days > 0` (in an outer query) to get the gap list, which is the standard check for missing data partitions.

Problem 8.5.3 Sessionization: events by one user at these times: 09:00, 09:10, 09:20, 10:15, 10:20, 12:00. A new session starts when the gap to the previous event is more than 30 minutes.

Step 1	`LAG` gives the previous event time; compute the gap in minutes.
Step 2	Mark a row with 1 if it starts a new session (first event or gap above 30), else 0.
Step 3	A running sum of those marks gives the session number.

```
CREATE TABLE clicks (user_id TEXT, ts TEXT);
INSERT INTO clicks VALUES
 ('u1','2024-03-05 09:00:00'),('u1','2024-03-05 09:10:00'),('u1','2024-03-05 09:20:00'),
 ('u1','2024-03-05 10:15:00'),('u1','2024-03-05 10:20:00'),('u1','2024-03-05 12:00:00');

WITH gaps AS (
  SELECT user_id, ts,
         (strftime('%s', ts) - strftime('%s', LAG(ts) OVER (PARTITION BY user_id ORDER BY ts))) / 60.0 AS gap_min
  FROM clicks
), flagged AS (
  SELECT *, CASE WHEN gap_min IS NULL OR gap_min > 30 THEN 1 ELSE 0 END AS new_session FROM gaps
), sessions AS (
  SELECT *, SUM(new_session) OVER (PARTITION BY user_id ORDER BY ts) AS session_no FROM flagged
)
SELECT user_id, session_no, MIN(ts) AS started, MAX(ts) AS ended, COUNT(*) AS events
FROM sessions GROUP BY user_id, session_no ORDER BY session_no;
```

```
user_id  session_no  started              ended                events
-------  ----------  -------------------  -------------------  ------
u1       1           2024-03-05 09:00:00  2024-03-05 09:20:00  3
u1       2           2024-03-05 10:15:00  2024-03-05 10:20:00  2
u1       3           2024-03-05 12:00:00  2024-03-05 12:00:00  1
```

- **Compute** Gaps in minutes: NULL, 10, 10, 55, 5, 100. Flags: 1, 0, 0, 1, 0, 1. Running sum: 1, 1, 1, 2, 2, 3.
- **Check** Events per session: 3, 2, 1, total 6.

- **Answer** Three sessions: 09:00 to 09:20 (3 events), 10:15 to 10:20 (2 events), 12:00 (1 event).
- **Meaning** "Flag the start, then running sum" is the general recipe for turning boundaries into group ids. Chapter 12 uses it on bigger data.

Problem 8.5.4 Median of order totals with windows (SQLite has no median function). The median is the middle value; for an even count it is the average of the two middle values.

```
WITH ordered AS (
  SELECT total,
         ROW_NUMBER() OVER (ORDER BY total) AS rn,
         COUNT(*) OVER () AS n
  FROM order_totals
)
SELECT AVG(total) AS median
FROM ordered
WHERE rn IN ((n + 1) / 2, (n + 2) / 2);
```

```
median
------
960.0
```

- **Compute** 14 sorted totals: 360, 400, 400, 440, 600, 600, 900, 1020, 1280, 1500, 1500, 1660, 1800, 2100. With `n = 14`: `(14+1)/2 = 7` and `(14+2)/2 = 8` (integer division). The 7th is 900 and the 8th is 1020. Median `(900 + 1020) / 2 = 960`.
- **Check** Seven values are at or below 900 and seven at or above 1020. The mean order total is `14560 / 14 = 1040`, a little above the median.
- **Answer** 960.
- **Meaning** Half of all orders are below 960 rupees. Median beats mean when a few huge orders pull the mean up.

```
-- PostgreSQL syntax, not run here
-- PostgreSQL: median and percentiles are built in
SELECT percentile_cont(0.5)  WITHIN GROUP (ORDER BY total) AS median,
       percentile_cont(0.9)  WITHIN GROUP (ORDER BY total) AS p90,
       percentile_disc(0.95) WITHIN GROUP (ORDER BY total) AS p95
FROM order_totals;
```

The picture

```
 days:      1  2  3  .  5  6  .  .  9
 row no:    1  2  3     4  5        6
 day - rn:  0  0  0     1  1        3
            [island A ] [island B ] [C]
```

Connection to the real job

These patterns are daily work: retention streaks, subscription continuity, "missing hour" checks on data feeds, web-session metrics, and medians for skewed data like revenue and waiting times. The percentiles you read in delivery or exam reports (p90, p95) correspond to `percentile_cont` or `percentile_disc` in SQL.

Applications

- **Data Analytics:** streaks, sessions per user, median basket size.
- **Data Engineering:** detect missing partitions (gaps), build sessions in clickstream pipelines.
- **Data Science and ML:** session features, longest-streak features.
- **Logistics:** median and 90th percentile of delivery times per city in SQL.

COMMON MISTAKE	Running the islands trick on data with duplicate days (the row number then outruns the date and splits islands). Remove duplicates first. And for sessions: choose and document the timeout (30 minutes is a convention, not a law).

Check yourself

1. In the islands trick, what stays equal inside one island?

Answer: `day − row_number`.

1. How do you turn "session start flags" into session numbers?

Answer: A running `SUM` of the flags.

1. Which PostgreSQL function gives the median?

Answer: `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`.

8.6 Frames in Depth: ROWS, RANGE and GROUPS When Values Tie

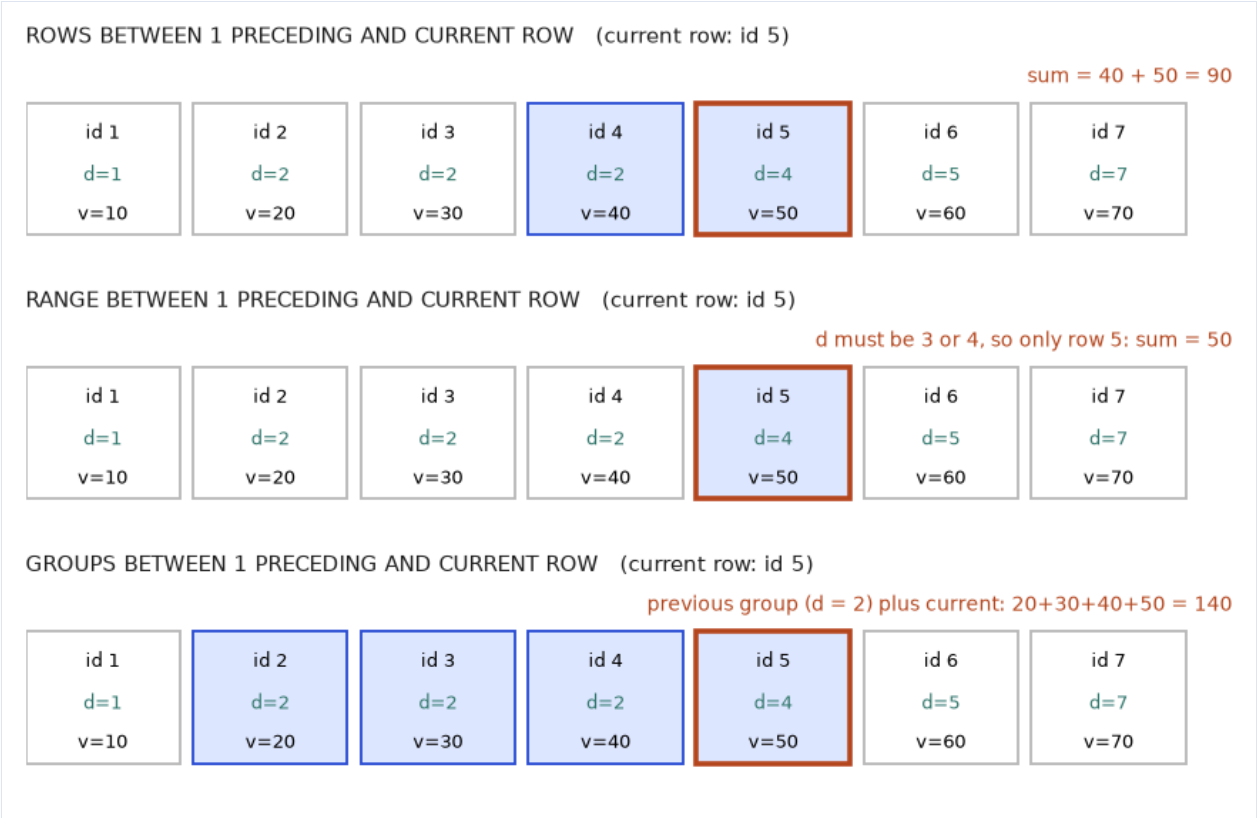
INTUITION	Section 8.4 treated a frame as 'the previous row and this one'. That is true for `ROWS`, which counts physical rows. But 'one before' can also mean 'one *unit of the sort key* before' (`RANGE`) or 'one *group of equal keys* before' (`GROUPS`). The three agree when keys are unique and disagree when keys tie or have gaps, which real data always does.

Take seven rows sorted by `d`, where three rows tie at d = 2 and there are gaps (no d = 3, no d = 6). The frame below ends at the current row and starts one step back, measured three different ways:

Problem 8.6.1 Compute a two-step sum with `ROWS`, `RANGE` and `GROUPS` frames, and the default frame. Predict first: for row id 5 (d = 4), ROWS adds the previous row, RANGE adds anything with d in [3, 4], GROUPS adds the whole previous group.

```
CREATE TABLE s (id INTEGER, d INTEGER, v INTEGER);
INSERT INTO s VALUES (1,1,10),(2,2,20),(3,2,30),(4,2,40),(5,4,50),(6,5,60),(7,7,70);
SELECT id, d, v,
  sum(v) OVER (ORDER BY d ROWS   BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_1,
  sum(v) OVER (ORDER BY d RANGE  BETWEEN 1 PRECEDING AND CURRENT ROW) AS range_1,
  sum(v) OVER (ORDER BY d GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS groups_1,
  sum(v) OVER (ORDER BY d) AS default_frame
FROM s ORDER BY id;
```

```
id  d  v   rows_1  range_1  groups_1  default_frame
--  -  --  ------  -------  --------  -------------
1   1  10  10      10       10        10
2   2  20  30      100      100       100
3   2  30  50      100      100       100
4   2  40  70      100      100       100
5   4  50  90      50       140       150
6   5  60  110     110      110       210
7   7  70  130     70       130       280
```



The frame for row id 5 (d = 4) under the three frame types.

How to read it: each box is a row with its id, sort key d and value v. The orange outline is the current row. Blue boxes are inside the frame. ROWS takes the one physical row before (id 4). RANGE takes rows whose d is within 1 of 4, which is only itself because there is no d = 3. GROUPS takes the whole previous group (the three rows with d = 2) because a group is a set of equal d values.

Row id 5	`rows_1` = 40 + 50 = 90. `range_1` : d between 3 and 4 gives only row 5, so 50. `groups_1` : the previous group (d = 2) is 20 + 30 + 40 = 90, plus 50 = 140.
Row id 2	The three rows with d = 2 are peers. `range_1` and `groups_1` give all of them 100 (the d = 1 row, plus all three d = 2 rows), but `rows_1` gives 30, 50, 70, depending on the arbitrary order inside the tie.
Default frame	With `ORDER BY` and no frame, SQL uses `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` . So the running total `default_frame` gives all three tied rows 100 (10 + 20 + 30 + 40), not 30, 60, 100.
Answer	Rows are not always the right unit. State the frame explicitly.

COMMON MISTAKE	Writing `sum(v) OVER (ORDER BY d)` for a running total when `d` has ties. The default frame is `RANGE` , so tied rows all show the same running total, which jumps by the whole tie group at once. If you want one step per row, add a unique tie-breaker (`ORDER BY d, id`) *or* write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` . Also, `RANGE` with a numeric offset (`RANGE BETWEEN 1 PRECEDING ...`) requires a single numeric or date `ORDER BY` key. PostgreSQL 11 and later, SQLite 3.28 and later support `GROUPS` ; MySQL and SQL Server do not support `GROUPS` , and SQL Server limits `RANGE` to `UNBOUNDED` and `CURRENT ROW` boundaries.

Problem 8.6.2 A table has daily revenue but some days are missing. You want a 7-day moving average that means 'the last seven *calendar days*', not 'the last seven rows'. Which frame? *(medium)*

Answer: A date-aware RANGE frame: `RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW` (PostgreSQL 11+). With missing days a ROWS frame of 6 PRECEDING would reach back further than a week. In engines without interval RANGE, join to a calendar table first so every day exists, then ROWS is correct. Divide by the number of days actually present if gaps remain.

Problem 8.6.3 Using the table above, what is `sum(v) OVER (ORDER BY d, id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` for ids 1 to 7? *(what is the output?)*

Answer: A strict running total in (d, id) order: 10, 30, 60, 100, 150, 210, 280. The unique tie-breaker makes the three d = 2 rows distinct, so the total rises by one row at a time: 30, 60, 100.

Chapter recap

- A window function adds a computed column without collapsing rows; the window is set by `OVER (PARTITION BY ... ORDER BY ... frame)`.
- Windows run after `WHERE` / `GROUP BY`, so filter on them in an outer query or CTE.
- `ROW_NUMBER` is unique, `RANK` skips after ties, `DENSE_RANK` does not; always add a tiebreaker to `ROW_NUMBER`.
- Top-N per group and dedup are `ROW_NUMBER() ... WHERE rn <= N` inside a CTE.
- `LAG` / `LEAD` look at neighbouring rows; use them for MoM, YoY and gaps between events.
- The default frame with `ORDER BY` is `RANGE ... CURRENT ROW`, so ties merge; state `ROWS` explicitly. `LAST_VALUE` needs a full frame.
- Running totals use `UNBOUNDED PRECEDING`; moving averages use `n PRECEDING`.
- Islands: `date - ROW_NUMBER()` is constant within a streak. Sessions: flag starts, then running `SUM`.
- Median: middle row(s) with `ROW_NUMBER`, or `percentile_cont` in PostgreSQL.

Need	Pattern
Share of total	`x / SUM(x) OVER ()`
Rank in group	`RANK() OVER (PARTITION BY g ORDER BY x DESC)`
Top N per group	CTE with `ROW_NUMBER()`, filter `rn <= N`
Dedup	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY updated DESC) = 1`
Previous value	`LAG(x) OVER (ORDER BY t)`
Running total	`SUM(x) OVER (ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
7-day average	`AVG(x) OVER (ORDER BY t ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`
Islands	`GROUP BY date - ROW_NUMBER() OVER (ORDER BY date)`

Practice set

Problem 8.P.1 (easy) For each product show its price and the average price of its category.

```
SELECT name, category, price,
       AVG(price) OVER (PARTITION BY category) AS category_avg
FROM products ORDER BY category, price;
```

```
name       category     price  category_avg
---------  -----------  -----  ------------
Mouse      Electronics  600    1050.0
Keyboard   Electronics  1500   1050.0
Mug        Home         250    575.0
Desk Lamp  Home         900    575.0
Notebook   Stationery   80     100.0
Pen Pack   Stationery   120    100.0
```

Answer: Electronics average 1050 (600 and 1500), Home 575 (250 and 900), Stationery 100 (80 and 120).

Problem 8.P.2 (easy) Number the customers by signup date, oldest first.

```
SELECT customer_id, name, signup_date,
       ROW_NUMBER() OVER (ORDER BY signup_date) AS signup_order
FROM customers;
```

```
customer_id  name   signup_date  signup_order
-----------  -----  -----------  ------------
1            Asha   2024-01-05   1
2            Ravi   2024-01-12   2
3            Meena  2024-01-20   3
4            John   2024-02-02   4
5            Priya  2024-02-14   5
6            Kabir  2024-03-01   6
7            Sana   2024-03-15   7
8            Tom    2024-04-01   8
```

Answer: Asha 1, Ravi 2, Meena 3, John 4, Priya 5, Kabir 6, Sana 7, Tom 8 (signup dates are all different and already in id order).

Problem 8.P.3 (what is the output?) What do `RANK` and `DENSE_RANK` return for the values 90, 80, 80, 70 in descending order?

```
SELECT v, RANK() OVER (ORDER BY v DESC) AS r, DENSE_RANK() OVER (ORDER BY v DESC) AS d
FROM (SELECT 90 AS v UNION ALL SELECT 80 UNION ALL SELECT 80 UNION ALL SELECT 70)
ORDER BY v DESC;
```

```
v   r  d
--  -  -
90  1  1
80  2  2
80  2  2
70  4  3
```

Answer: `RANK`: 1, 2, 2, 4. `DENSE_RANK`: 1, 2, 2, 3.

Problem 8.P.4 (find the bug) This query is meant to show the top order per customer but fails. Why, and what is the fix?

```
-- SELECT customer_id, order_id FROM order_totals
-- WHERE ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC) = 1;
```

Answer: Window functions are not allowed in `WHERE`. Put the `ROW_NUMBER()` in a CTE or subquery as a column `rn`, then filter `rn = 1` outside.

Problem 8.P.5 The previous order total for each order of customer 2, and whether the total went up.

```
SELECT order_id, order_date, total,
       LAG(total) OVER (ORDER BY order_date) AS prev_total,
       CASE WHEN total > LAG(total) OVER (ORDER BY order_date) THEN 'up' ELSE 'not up' END AS trend
FROM order_totals WHERE customer_id = 2 ORDER BY order_date;
```

```
order_id  order_date  total  prev_total  trend
--------  ----------  -----  ----------  ------
102       2024-01-15  600    NULL        not up
108       2024-03-18  900    600         up
114       2024-04-20  2100   900         up
```

Answer: Orders 102 (600, no previous so `not up`), 108 (900, up from 600), 114 (2100, up from 900).

Problem 8.P.6 Cumulative number of orders by date (running count), one row per order.

```
SELECT order_id, order_date,
       COUNT(*) OVER (ORDER BY order_date, order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS orders_so_far
FROM orders ORDER BY order_date, order_id;
```

```
order_id  order_date  orders_so_far
--------  ----------  -------------
101       2024-01-10  1
102       2024-01-15  2
103       2024-02-03  3
104       2024-02-10  4
105       2024-02-20  5
106       2024-03-05  6
107       2024-03-07  7
108       2024-03-18  8
109       2024-03-25  9
110       2024-03-30  10
111       2024-04-02  11
112       2024-04-09  12
113       2024-04-15  13
114       2024-04-20  14
```

Answer: The last row shows 14. The running count goes 1, 2, ..., 14 because every order has its own date.

Problem 8.P.7 Customers split into 2 halves by total paid spend (`NTILE(2)`, highest first).

```
WITH spend AS (
  SELECT customer_id, SUM(total) AS spent FROM order_totals WHERE status = 'paid' GROUP BY customer_id
)
SELECT customer_id, spent, NTILE(2) OVER (ORDER BY spent DESC) AS half FROM spend ORDER BY spent DESC;
```

```
customer_id  spent  half
-----------  -----  ----
1            4560   1
2            3600   1
4            3300   1
5            760    2
6            440    2
```

Answer: Five paying customers: the first 3 rows (1, 2, 4) get half 1 and the last 2 (5, 6) get half 2. `NTILE` gives the extra row to the first group.

Problem 8.P.8 (scenario) The product team wants "each user's first purchase category and the days between first and second purchase". Which functions? **Answer:** `FIRST_VALUE` (or `ROW_NUMBER = 1`) of category per user ordered by date; `LAG(order_date)` or `NTH_VALUE(order_date, 2)` for the second purchase, then a date difference. Put them in a CTE, filter in the outer query.

Problem 8.P.9 (interview) Find the 3rd highest distinct salary per department. Which function and why? **Answer:** `DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC)` and keep rank 3. `DENSE_RANK` handles ties without skipping; `ROW_NUMBER` would give a wrong answer when salaries tie.

Problem 8.P.10 (interview) Explain the difference between `ROWS` and `RANGE` frames, with an example where they differ. **Answer:** `ROWS` counts physical rows; `RANGE` includes all peer rows with the same `ORDER BY` value. For values 10, 20, 20, 30 a running sum with `ROWS` gives 10, 30, 50, 80, but `RANGE` gives 10, 50, 50, 80.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional in India moving into data roles, and English is my second language. Teach me Chapter 8 topic: "SQL window functions: OVER, PARTITION BY and ORDER BY". Use a small orders table. Show the difference between GROUP BY and a window function with tiny tables, then ask me to predict outputs for 5 queries, one at a time, and check my reasoning.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: ranking functions (ROW_NUMBER, RANK, DENSE_RANK, NTILE) and top-N per group and deduplication. Give me 8 graded exercises on an employees and sales schema you define, without solutions at first. When I paste my answers, review the tie handling and the use of CTEs, and explain mistakes simply.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. Teach me LAG, LEAD, FIRST_VALUE, LAST_VALUE, window frames (ROWS versus RANGE) running totals and moving averages. I am a beginner. Use monthly revenue numbers I can verify by hand, show the frame as a picture in text, and explain the LAST_VALUE trap. After each idea ask me one check question and wait.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. Teach me gaps and islands and sessionization in SQL step by step: row number minus date trick, LAG gap flags, running sum of flags. I am preparing for data-analyst interviews. Use small tables, make me predict intermediate columns, then give me 3 new practice problems and review my answers when I paste them.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. Run a mock SQL interview focused on window functions: second highest salary per department, month-over-month growth, 7-day moving average, remove duplicates keeping the latest, longest login streak. One question at a time, grade me out of 10, show the model solution in PostgreSQL, and name one common mistake each time.
```

CHAPTER 9

DDL, DML, CONSTRAINTS, UPSERTS AND TRANSACTIONS

Creating tables, changing data safely, letting the database enforce your rules, and keeping many users from corrupting each other's work.

Up to now you only read data. A real job also needs you to create tables, load rows, fix wrong values, and do all of it while other people and programs use the same database. One careless `DELETE` without a `WHERE`, or one bank-style transfer that stops half way, can cost a company real money. In this chapter you learn the commands that define and change data, the rules (constraints) that stop bad data from entering, and the transaction system that makes a group of changes all-or-nothing. By the end you can build a safe table, load and correct data, write an upsert, and explain ACID, isolation levels, locks and deadlocks in an interview.

All examples run in SQLite 3.51 unless I say **not run here**. SQLite is a little different from PostgreSQL in places (I will point these out), but the ideas are identical. We reuse the shop database from Chapter 2, plus a few small tables.

9.1 DDL: CREATE, ALTER, DROP, Data Types and Constraints

INTUITION

Think of building a school register. Before any student is written down, you print the form: columns for name, roll number, marks, and rules such as "roll number must be unique" and "marks must be 0 to 100." **DDL** (Data Definition Language) is the set of commands that prints the form. **Constraints** are the rules printed on it. The database then refuses any entry that breaks a rule, even if your Python code has a bug.

Words you need

- **DDL**: commands that define structure (`CREATE` , `ALTER` , `DROP`). **DML**: commands that change rows (`INSERT` , `UPDATE` , `DELETE`).
- **Constraint**: a rule the database checks on every change. **Primary key**: unique, never NULL, identifies a row. **Foreign key**: must match a primary key in another table (Chapter 1).

The constraints

Constraint	Rule it enforces	Example
`NOT NULL`	value must exist	`email TEXT NOT NULL`
`UNIQUE`	no two rows share the value	`email TEXT UNIQUE`
`PRIMARY KEY`	unique + not null, identifies the row	`rating_id INTEGER PRIMARY KEY`
`FOREIGN KEY`	value must exist in the parent table	`REFERENCES customers(customer_id)`
`CHECK`	any true/false test on the row	`CHECK (score BETWEEN 1 AND 5)`
`DEFAULT`	value used when you give none	`status TEXT DEFAULT 'new'`

Data types matter too. Common PostgreSQL types: `INTEGER` , `BIGINT` , `NUMERIC(10,2)` for exact money, `TEXT` , `BOOLEAN` , `DATE` , `TIMESTAMPTZ` (time with time zone), `JSONB` . Use `NUMERIC` for money, never floating point, because `0.1 + 0.2` is not exactly `0.3` in floating point. SQLite is loosely typed: it has only a few storage classes (`INTEGER` , `REAL` , `TEXT` , `BLOB`), and by default it lets you store the wrong type, as you will see below.

Real-world mapping

Item	Example	Where it appears in the job
`CHECK`	`price >= 0`	stopping bad rows from an orders pipeline

Item	Example	Where it appears in the job
`UNIQUE`	`(order_id, product_id)`	making a nightly load job safe to re-run
`FOREIGN KEY`	`orders.customer_id`	no order for a missing customer

Worked examples

Problem 9.1.1 Create a `ratings` table where `score` must be 1 to 5, `email` must exist and be unique, and `status` defaults to `'new'`. Insert one valid row. Predict first: the `SELECT` shows the default in `status`.

```
CREATE TABLE ratings (
  rating_id INTEGER PRIMARY KEY,
  email     TEXT NOT NULL UNIQUE,
  score     INTEGER NOT NULL CHECK (score BETWEEN 1 AND 5),
  status    TEXT NOT NULL DEFAULT 'new'
);
INSERT INTO ratings (email, score) VALUES ('a@x.com', 4);
SELECT * FROM ratings;
```

```
rating_id  email    score  status
---------  -------  -----  ------
1          a@x.com  4      new
```

- **Answer** One row. `rating_id` was auto-filled (SQLite) and `status` came from the `DEFAULT`, because we did not name it.

Now break each rule on purpose. Run these one at a time and read each error.

```
INSERT INTO ratings (email, score) VALUES ('a@x.com', 3);   -- same email
INSERT INTO ratings (email, score) VALUES ('b@x.com', 9);   -- score out of range
INSERT INTO ratings (email, score) VALUES (NULL, 3);        -- missing email
```

```
Error: stepping, UNIQUE constraint failed: ratings.email (19)
Error: stepping, CHECK constraint failed: score BETWEEN 1 AND 5 (19)
Error: stepping, NOT NULL constraint failed: ratings.email (19)
```

- **Answer** All three were rejected by the database, not by application code, and the table still has one row.

Problem 9.1.2 Change a table later with `ALTER TABLE`: `ALTER TABLE ratings ADD COLUMN note TEXT;` adds a column (old rows get NULL), `ALTER TABLE ratings RENAME COLUMN note TO comment;` renames it, and `DROP TABLE IF EXISTS s2;` removes a table without an error when it is missing. Run `PRAGMA table_info(ratings);` to list columns (in PostgreSQL use `\d ratings` in `psql`). Plain `DROP TABLE s2;` on a missing table gives `Error: in prepare, no such table: s2`.

Foreign keys and ON DELETE

A foreign key says "this value must exist in the parent." What should happen when the parent row is deleted? You choose.

Option	Meaning
`ON DELETE RESTRICT` (or no action)	refuse to delete the parent if children exist (the default behaviour)
`ON DELETE CASCADE`	delete the children too
`ON DELETE SET NULL`	keep children, set their FK to NULL

Problem 9.1.3 Show the difference between a delete that is refused and one that cascades. First, in the shop database, try to delete customer 1, who has orders:

```
PRAGMA foreign_keys = ON;
DELETE FROM customers WHERE customer_id = 1;
```

```
Error: stepping, FOREIGN KEY constraint failed (19)
```

Now a cascading design with shopping carts and their items:

```
PRAGMA foreign_keys = ON;
CREATE TABLE carts (cart_id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE cart_items (
  item_id INTEGER PRIMARY KEY,
  cart_id INTEGER NOT NULL REFERENCES carts(cart_id) ON DELETE CASCADE,
  price INTEGER CHECK (price >= 0)
);
INSERT INTO carts VALUES (1,'birthday'),(2,'groceries');
INSERT INTO cart_items(cart_id,price) VALUES (1,120),(1,340),(2,95);
DELETE FROM carts WHERE cart_id = 1;
SELECT * FROM cart_items;
```

```
item_id  cart_id  price
-------  -------  -----
3        2        95
```

- **Step 1** Deleting cart 1 removed its two items automatically.
- **Step 2** Cart 2 and its item stayed.
- **Answer** The restrict rule protected customer 1; the cascade rule cleaned up cart 1.

COMMON MISTAKE

In SQLite, foreign keys are **off by default** for each new connection. I ran the same cascade script in a new session *without* `PRAGMA foreign_keys = ON`, but deleted cart 2 (the `groceries` cart). The check `SELECT count(*) FROM cart_items` returned `3`: nothing cascaded, and the item of the deleted cart survived as an orphan row. PostgreSQL, MySQL (InnoDB) and SQL Server enforce foreign keys always. In SQLite, put the `PRAGMA` at the top of every script.

COMMON MISTAKE

Trusting SQLite types. In a plain SQLite table `CREATE TABLE s(n INTEGER); INSERT INTO s VALUES('abc');` succeeds, and `typeof(n)` says `text`. SQLite is "flexibly typed." Adding the keyword `STRICT` at the end of `CREATE TABLE` makes it refuse: `cannot store TEXT value in INTEGER column s2.n`. PostgreSQL always refuses. Do not build habits on the lenient behaviour.

Connection to the real job

A data engineer designs tables with constraints so pipelines cannot load garbage. An analyst reads constraint names to learn which columns can be trusted: if `email` is `UNIQUE NOT NULL`, you can join on it without fear of duplicates. In ML, a clean constrained feature table saves days of debugging.

Applications

- **Data Engineering:** `UNIQUE` keys make nightly loads safe to repeat; **Analytics:** a `NOT NULL UNIQUE` column is safe to join on.

Check yourself

1. Which constraint stops two customers from having the same email?

Answer: `UNIQUE` on the `email` column.

1. What does `ON DELETE CASCADE` do?

Answer: When the parent row is deleted, the child rows pointing to it are deleted too.

1. Why should money be `NUMERIC` and not floating point?

Answer: Floating point cannot store values like 0.1 exactly, so sums drift; `NUMERIC` is exact.

9.2 DML: INSERT, UPDATE, DELETE, Upserts, Views and Friends

INTUITION

Imagine a shared Google Sheet used by a team. You add rows, fix wrong cells, delete old rows. The risky moment is when you fix "all rows where..." and the filter is wrong. DML is the same, but the sheet has a million rows and no undo button, except the one you build yourself (transactions, section 9.3).

Words you need

- **Upsert**: "update + insert": update the row if it exists, insert it if not. **Conflict**: an insert that would break a `UNIQUE` or primary-key rule.
- **View**: a saved query used like a table (no data of its own). **Temp table**: disappears when your session ends.

INSERT, UPDATE, DELETE

```
INSERT INTO table (col1, col2) VALUES (v1, v2), (v3, v4);
UPDATE table SET col = expression WHERE condition;
DELETE FROM table WHERE condition;
```

Always name the columns in `INSERT`. If someone later adds a column, an unnamed `INSERT` breaks or fills the wrong column. And **always write the `WHERE` first**, then the rest.

Problem 9.2.1 The prices of the Home products rise by 50. Update and show the result. Predict: Desk Lamp 900 becomes 950, Mug 250 becomes 300.

```
UPDATE products SET price = price + 50 WHERE category = 'Home';
SELECT product_id, name, price FROM products WHERE category = 'Home';
```

```
product_id  name       price
----------  ---------  -----
5           Desk Lamp  950
6           Mug        300
```

- **Step 1** `WHERE category = 'Home'` selects products 5 and 6 only.
- **Step 2** `price = price + 50` reads the old price on the right and writes the new price on the left.
- **Check** 900 + 50 = 950 and 250 + 50 = 300.
- **Answer** Two rows changed, and no other row was touched.

`INSERT ... RETURNING product_id, name;` (SQLite 3.35+, PostgreSQL) gives back the new row in one step, avoiding a race with other sessions.

PROFESSOR'S NOTE

Before any risky `UPDATE` or `DELETE`, run the same `WHERE` as a `SELECT COUNT(*)` first. If you expect 2 rows and the count says 2,000,000, you just avoided a disaster. Better still, run it inside a transaction (section 9.3) and look before you commit.

COMMON MISTAKE

`DELETE FROM customers;` with no `WHERE` removes every row. `UPDATE products SET price = 0;` sets every price to zero. The database does not ask "are you sure?". Use a transaction, or a `SELECT` first.

TRUNCATE, COPY and bulk loading

`TRUNCATE TABLE t;` empties a table fast (PostgreSQL, MySQL, SQL Server; **not run here**; SQLite uses `DELETE FROM t;`). `COPY t FROM 'file.csv' WITH (FORMAT csv, HEADER);` bulk-loads a CSV into PostgreSQL (**not run here**; `\copy` in `psql`; SQLite: `.mode csv` then `.import`). Loading a million rows with one autocommitted `INSERT` each is slow because every commit waits for the disk: batch them in one transaction or use `COPY`.

Upsert: INSERT ... ON CONFLICT

The problem. A nightly job loads stock counts. Some products are already in the table, some are new. If you just `INSERT`, existing ones fail with a duplicate-key error. If you check first (`SELECT`), then insert, two jobs can both see "not there" and both insert. An upsert does the whole thing in one safe statement.

```
CREATE TABLE stock (product_id INTEGER PRIMARY KEY, qty INTEGER NOT NULL);
INSERT INTO stock VALUES (1,10),(2,20);

-- product 1 exists: add to quantity
INSERT INTO stock VALUES (1,5)
  ON CONFLICT (product_id) DO UPDATE SET qty = qty + excluded.qty;
-- product 3 is new: plain insert
INSERT INTO stock VALUES (3,7)
  ON CONFLICT (product_id) DO UPDATE SET qty = qty + excluded.qty;
-- product 2 exists, ignore the new value
INSERT INTO stock VALUES (2,99)
  ON CONFLICT (product_id) DO NOTHING;
SELECT * FROM stock;
```

```
product_id  qty
----------  ---
1           15
2           20
3           7
```

- **Step 1** The first statement conflicts on `product_id = 1`. `DO UPDATE` runs. `excluded` is a special name for "the row I tried to insert," so `excluded.qty` is 5, and `qty` is the existing 10. Result 10 + 5 = 15.
- **Step 2** Product 3 has no conflict. It is inserted as 7.
- **Step 3** Product 2 conflicts, but `DO NOTHING` skips the new row. Qty stays 20.
- **Check** 15, 20, 7 matches the output.
- **Answer** One statement handled update, insert and ignore, atomically.

The same syntax works in PostgreSQL. `MERGE` is the SQL-standard alternative (PostgreSQL 15+, SQL Server, Oracle). **Not run here.** In MySQL the equivalent is `INSERT ... ON DUPLICATE KEY UPDATE`.

COMMON MISTAKE

Using `INSERT OR REPLACE` (SQLite) or `REPLACE INTO` (MySQL) as an upsert. It **deletes** the old row and inserts a new one, so other columns you did not mention are reset to defaults. I ran `INSERT OR REPLACE INTO stock VALUES (1,1);` after the script above and product 1 went from 15 to 1: the whole row was replaced. Prefer `ON CONFLICT ... DO UPDATE`, which changes only the columns you name.

Views, temp tables and identity columns

Views give a name to a query so everyone uses the same definition of "revenue."

```
CREATE VIEW paid_revenue AS
SELECT o.order_id, SUM(i.qty * i.unit_price) AS total
FROM orders o JOIN order_items i USING (order_id)
WHERE o.status = 'paid'
GROUP BY o.order_id;

SELECT * FROM paid_revenue LIMIT 3;
```

```
order_id  total
--------  -----
101       1660
102       600
103       1020
```

Check by hand: order 101 has 1 keyboard at 1500 plus 2 notebooks at 80, so 1500 + 160 = 1660. Order 103: 900 + 120 = 1020.

Temp tables (`CREATE TEMP TABLE big AS SELECT ...`) hold intermediate results for your session only. **Identity:** PostgreSQL uses `id BIGINT GENERATED ALWAYS AS IDENTITY` (**not run here**); SQLite's `INTEGER PRIMARY KEY` auto-numbers, and ids are never guaranteed gap-free (I deleted id 2 from a 2-row table and the next insert got id 3 with `AUTOINCREMENT`). **Awareness, not run here:** `CREATE MATERIALIZED VIEW` plus `REFRESH MATERIALIZED VIEW` stores a heavy query's result; triggers and stored procedures hide logic in the database, so use them rarely.

Surrogate vs natural keys

A **natural key** is real-world data (an email) and can change. A **surrogate key** is a meaningless id the database makes (`customer_id`). Practice: surrogate primary key, plus `UNIQUE` on the natural key.

Connection to the real job

Upserts are in every incremental data pipeline: "load today's changes into the table without duplicating yesterday's." Views are how analytics teams share one definition of a metric. `COPY` and bulk loads are how data engineers fill warehouses.

Applications

- **Data Engineering:** idempotent loads with `ON CONFLICT` ; **Analytics:** shared views so dashboards agree; **ML:** temp tables to build a training set.

Check yourself

1. What does `excluded.qty` mean inside `ON CONFLICT DO UPDATE` ?

Answer: The value from the row that you tried to insert.

1. Why is `INSERT OR REPLACE` dangerous as an upsert?

Answer: It deletes the whole old row and inserts a new one, so unmentioned columns are lost.

1. What is the first thing to do before a risky `DELETE` ?

Answer: Run the same `WHERE` in a `SELECT COUNT(*)` , and ideally work inside a transaction.

9.3 Transactions and ACID

INTUITION	You move Rs 300 from Asha to Ravi. That is two steps: subtract from Asha, add to Ravi. If the power fails between the steps, Rs 300 vanishes. A **transaction** wraps both steps in one box that either happens fully or not at all. Think of a bank counter: deposit, stamp and receipt are many small actions but one business step with one result.

Words you need

- **Transaction**: a group of statements treated as one unit.
- **COMMIT**: make all the changes permanent.
- **ROLLBACK**: undo all the changes since `BEGIN`.
- **Savepoint**: a bookmark inside a transaction you can roll back to.
- **Autocommit**: the default mode in which each single statement is its own transaction.

ACID

Letter	Name	Plain meaning	If it were missing
A	Atomicity	all steps happen, or none	half a transfer
C	Consistency	rules (constraints) hold before and after	negative balances
I	Isolation	concurrent transactions do not see each other's half-done work	wrong totals in reports
D	Durability	once committed, it survives a crash	lost orders after power cut

Durability is usually done with a **write-ahead log (WAL)**: the database writes the change to a log file on disk first, and updates the table files later. After a crash it replays the log.

Problem 9.3.1 Transfer Rs 300 from Asha (id 1) to Ravi (id 2), first rolled back, then committed.

```
CREATE TABLE accounts (
  id INTEGER PRIMARY KEY, owner TEXT,
  balance INTEGER NOT NULL CHECK (balance >= 0));
INSERT INTO accounts VALUES (1,'Asha',1000),(2,'Ravi',500);

BEGIN;
UPDATE accounts SET balance = balance - 300 WHERE id = 1;
UPDATE accounts SET balance = balance + 300 WHERE id = 2;
SELECT * FROM accounts;
ROLLBACK;
SELECT * FROM accounts;

BEGIN;
UPDATE accounts SET balance = balance - 300 WHERE id = 1;
UPDATE accounts SET balance = balance + 300 WHERE id = 2;
COMMIT;
SELECT * FROM accounts;
```

```
id  owner  balance
--  -----  -------
1   Asha   700
2   Ravi   800
id  owner  balance
--  -----  -------
1   Asha   1000
2   Ravi   500
id  owner  balance
--  -----  -------
1   Asha   700
2   Ravi   800
```

- **Step 1** Inside the first transaction we see the new balances (700 and 800). This is *our own* view.
- **Step 2** `ROLLBACK` undoes both updates: 1000 and 500 again.
- **Step 3** The second run ends in `COMMIT`, so 700 and 800 become permanent.

- **Check** 700 + 800 = 1500 = 1000 + 500. Money is conserved.
- **Answer** Both steps happen together or not at all.

Problem 9.3.2 (find the bug) A script runs a transfer that should fail: Asha has only 700 but we try to move 5,000. We add 500 to Ravi first, then subtract 5,000 from Asha. The `CHECK (balance >= 0)` will reject the second step. Predict the final balances if the script continues after the error.

```
BEGIN;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
UPDATE accounts SET balance = balance - 5000 WHERE id = 1;
COMMIT;
SELECT * FROM accounts;
```

```
Runtime error near line 5: CHECK constraint failed: balance >= 0 (19)
id  owner  balance
--  -----  -------
1   Asha   700
2   Ravi   1300
```

- **Step 1** The first `UPDATE` worked: Ravi 800 to 1300.
- **Step 2** The second `UPDATE` failed. Only that **one statement** was undone, not the transaction.
- **Step 3** The `sqlite3` shell continued and ran `COMMIT`, which saved the half-done work.
- **Answer** Ravi gained 500 from nowhere. Total money rose from 1500 to 2000.
- **Fix** Your program must check for an error and issue `ROLLBACK` itself. In Python: `try: ... conn.commit() except: conn.rollback()`. In PostgreSQL an error inside a transaction puts the whole transaction in an "aborted" state, and `COMMIT` then behaves as a rollback. That is safer, but still handle errors in code.

COMMON MISTAKE	Assuming "an error cancels the whole transaction." Behaviour depends on the database and the shell. Always handle the error and roll back explicitly.

Savepoints (`SAVEPOINT s1; ... ROLLBACK TO s1;`) undo only part of a transaction. I ran a +1 update, a savepoint, a +100 update, `ROLLBACK TO s1`, then `COMMIT`: Ravi went from 1300 to 1301 only.

Connection to the real job

Every payment, order and booking system relies on atomic transactions. In data engineering, a pipeline loads a day of data in one transaction so dashboards never see half a day. Keep transactions **short**: a long open transaction holds locks and blocks others (section 9.4). It is the same as one slow customer at a bank counter holding up the whole queue.

Applications

- **Back ends:** checkout, transfers, seat booking. **Data Engineering:** swap a staging table in one transaction.

Check yourself

1. What does the A in ACID guarantee?

Answer: A transaction's steps all happen or none do.

1. Which ACID property is delivered by the write-ahead log?

Answer: Durability.

1. Why keep transactions short?

Answer: Open transactions hold locks and block other users; long ones cause waits and deadlocks.

9.4 Isolation Levels, MVCC, Locks and Deadlocks

INTUITION	Two clerks edit the same customer file in a bank at the same moment. Clerk 1 reads "balance 1000" and wants to subtract 100. Clerk 2 also read "1000" and subtracts 200. If both write back, one change is lost. **Isolation** is the set of rules for what each clerk may see and when one must wait. It is the same issue as two people updating one shared tally sheet: without a guard, the final value is wrong.

Words you need

- **Concurrent**: happening at the same time.
- **Dirty read**: reading another transaction's uncommitted data (which may be rolled back).
- **Non-repeatable read**: reading the same row twice in one transaction and getting two different values, because someone committed in between.
- **Phantom read**: running the same query twice and getting extra rows, because someone inserted rows in between.
- **Lock**: a flag that makes others wait before touching a row or table.
- **MVCC** (Multi-Version Concurrency Control): the database keeps old versions of a row, so readers see a stable snapshot and do not block writers.
- **Deadlock**: two transactions each wait for a lock the other holds, forever.

The four isolation levels (SQL standard)

Level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	no	possible	possible
REPEATABLE READ	no	no	possible (standard), not in PostgreSQL
SERIALIZABLE	no	no	no

Facts about real databases (check the version docs for details): PostgreSQL's default is **READ COMMITTED**; it treats READ UNCOMMITTED the same as READ COMMITTED, so it never allows dirty reads. Its REPEATABLE READ is a snapshot, and SERIALIZABLE may abort a transaction with a "serialization failure," which your code must **retry**. MySQL InnoDB defaults to REPEATABLE READ. You set the level with `BEGIN ISOLATION LEVEL REPEATABLE READ;` in PostgreSQL (**not run here**).

Higher isolation means fewer strange results and more waiting or retries. Most systems use READ COMMITTED and add explicit locks only where needed.

SQLite is different (and simpler)

SQLite allows **one writer at a time** for the whole database, and its transactions are serializable. That makes it easy to demonstrate. I used Python's `sqlite3` module with two connections, A and B, to the same file.

Problem 9.4.1 A updates a balance but has not committed. What does B see? Can B also write?

```
a.execute("BEGIN IMMEDIATE")
a.execute("UPDATE accounts SET balance = 700 WHERE id = 1")
print("A sees :", a.execute("SELECT balance FROM accounts").fetchone())
print("B sees :", b.execute("SELECT balance FROM accounts").fetchone())
b.execute("BEGIN IMMEDIATE")      # B also wants to write
```

```
A sees : (700,)
B sees : (1000,)
B error: database is locked
after commit B sees: (700,)
```

- **Step 1** A changed the balance to 700 but has not committed.
- **Step 2** A sees its own change, 700. B still sees 1000. **No dirty read.**
- **Step 3** B tries to start a write and gets `database is locked`, because A holds the write lock. (I set the wait time to 0 so the error appears at once. Normally you set a timeout and B simply waits.)
- **Step 4** After A commits, B sees 700.
- **Answer** Readers see committed data only, and a second writer must wait.

Problem 9.4.2 (lost update) Both connections read the balance (1000) into Python, subtract in Python and write back. A subtracts 100, B subtracts 200. The right answer is 700.

```
bal_a = a.execute("SELECT balance FROM accounts").fetchone()[0]   # 1000
bal_b = b.execute("SELECT balance FROM accounts").fetchone()[0]   # 1000
a.execute("UPDATE accounts SET balance = ?", (bal_a - 100,))      # writes 900
b.execute("UPDATE accounts SET balance = ?", (bal_b - 200,))      # writes 800
```

```
expected 700, got: 800
```

- **Step 1** Both read 1000.
- **Step 2** A writes 900. Then B writes 800, based on its old read, wiping A's change.
- **Answer** One withdrawal of 100 was lost. This is the **lost update** problem.

The fix is to let the database do the arithmetic in one statement:

```
a.execute("UPDATE accounts SET balance = balance - 100")
b.execute("UPDATE accounts SET balance = balance - 200")
```

```
expected 700, got: 700
```

`balance = balance - 100` reads and writes under one lock, so each change builds on the last. Other fixes: `SELECT ... FOR UPDATE` (PostgreSQL, **not run here**) locks the row while you read it; or use SERIALIZABLE and retry; or optimistic locking with a `version` column (`UPDATE ... WHERE id = 1 AND version = 7`, then check that 1 row changed).

COMMON MISTAKE	Read-modify-write in application code (`x = SELECT ...; UPDATE ... SET v = x - 100`). This is the classic cause of lost updates. Do the maths inside the SQL statement whenever possible.

Locks and MVCC in PostgreSQL

An `UPDATE` writes a **new version** of the row and keeps the old one for earlier transactions. Readers never block writers; writers block writers only on the *same row*. Dead versions are cleaned by `VACUUM` (Chapter 10). `ALTER TABLE` takes a strong table lock and can stall a busy table.

Deadlocks

A deadlock needs two transactions and two rows. Think of two cars on a narrow bridge, each waiting for the other to back up.

```
 time   Transaction T1                  Transaction T2
  1     UPDATE acct SET .. WHERE id=1   (locks row 1)
  2                                     UPDATE acct SET .. WHERE id=2  (locks row 2)
  3     UPDATE acct SET .. WHERE id=2   waits for T2
  4                                     UPDATE acct SET .. WHERE id=1  waits for T1
        --> each waits for the other: DEADLOCK
```

PostgreSQL detects this (after `deadlock_timeout`, 1 second by default), kills one transaction with `ERROR: deadlock detected`, and lets the other finish. The killed program must retry. **Not run here** (SQLite avoids it by having one writer).

How to avoid deadlocks (all databases):

1. Touch rows in the **same order** everywhere (always lower id first).
2. Keep transactions **short**; no user input or network calls in the middle.
3. Take stronger locks early (`SELECT ... FOR UPDATE`) when you know you will write.
4. **Retry** on deadlock and serialization errors. Your code should expect them.

Connection to the real job

A **hot row** (one counter row updated by every request) is a classic performance bottleneck: throughput is capped by how fast one lock can pass from transaction to transaction. Picture one bank counter: more customers join the queue, the number served per hour stays flat, and waiting times climb. The cure is the same as for any queue: shorter hold time, or spread the work across many rows.

Applications

- **Back ends:** inventory and wallets (atomic updates, `FOR UPDATE`). **Operations:** lock waits show as slow pages with low CPU use.

Check yourself

1. Which anomaly is "same query, extra rows the second time"?

Answer: Phantom read.

1. What does MVCC let readers do?

Answer: Read a consistent older version of rows without waiting for writers.

1. Name two ways to avoid deadlocks.

Answer: Lock rows in the same order everywhere; keep transactions short (also: retry on failure).

9.5 Transaction Anomaly Lab: Seeing Each Problem Happen

INTUITION	Section 9.4 gave you the names. Here you watch each problem happen, one step at a time, in a two-session timeline. Think of two cashiers sharing one cash register ledger. An anomaly is simply a moment where a cashier acts on a fact that stopped being true. Each isolation level is a rule about which stale facts a cashier is allowed to act on.

Every timeline below uses two sessions, **A** and **B**, and one table. Time runs downward. PostgreSQL timelines are **not run here** (no PostgreSQL server in this environment); I only state behaviour that the PostgreSQL manual documents. The SQLite examples were run for real.

Example 1: dirty read (and why you will rarely meet it)

t1	A: `BEGIN; UPDATE accounts SET balance = 700 WHERE id = 1;` (not committed)
t2	B: `SELECT balance FROM accounts WHERE id = 1;`
t3	A: `ROLLBACK;` (the 700 never existed)
Dirty read	If B saw 700 at t2, B acted on a number that was never real. PostgreSQL never allows this, even at READ UNCOMMITTED (it behaves as READ COMMITTED). MySQL InnoDB allows it only if you choose READ UNCOMMITTED.

Example 2: non-repeatable read

The balance starts at 1000. B is a report that reads the same row twice inside one transaction.

t1	B: `BEGIN; SELECT balance ...` returns 1000
t2	A: `UPDATE accounts SET balance = 900 WHERE id = 1; COMMIT;`
t3	B: `SELECT balance ...` again. READ COMMITTED returns 900. REPEATABLE READ returns 1000.
Meaning	At READ COMMITTED, B's report can contain two different values for one row. At REPEATABLE READ, B works from one snapshot.

SQLite runs in serializable mode, but in WAL mode (a journal mode that lets readers and one writer work at once) a transaction reads one snapshot, so I can show the REPEATABLE READ behaviour for real. I also try to write from the stale snapshot, which is the next lesson.

Problem 9.5.1 Show a stable snapshot and what happens when B tries to write after A committed. Predict first: B reads 1000 twice, then its write is refused, because its snapshot is out of date.

```
import sqlite3, tempfile
d = tempfile.mkdtemp(); p = d + "/bank.db"
setup = sqlite3.connect(p); setup.execute("PRAGMA journal_mode=WAL")
setup.execute("CREATE TABLE accounts (id INTEGER PRIMARY KEY, balance INTEGER)")
setup.execute("INSERT INTO accounts VALUES (1, 1000)"); setup.commit(); setup.close()
a = sqlite3.connect(p, isolation_level=None, timeout=0)
b = sqlite3.connect(p, isolation_level=None, timeout=0)
b.execute("BEGIN")
print("B first read :", b.execute("SELECT balance FROM accounts WHERE id=1").fetchone()[0])
a.execute("BEGIN IMMEDIATE")
a.execute("UPDATE accounts SET balance = 900 WHERE id = 1")
a.execute("COMMIT")
print("B second read:", b.execute("SELECT balance FROM accounts WHERE id=1").fetchone()[0])
try:
    b.execute("UPDATE accounts SET balance = balance - 200 WHERE id = 1")
except sqlite3.OperationalError as e:
    print("B write error:", e)
b.execute("ROLLBACK")
print("B retry read :", b.execute("SELECT balance FROM accounts WHERE id=1").fetchone()[0])
```

```
B first read : 1000
B second read: 1000
B write error: database is locked
B retry read : 900
```

Step 1	B opens a transaction and reads 1000. Its snapshot is fixed now.
Step 2	A commits 900. B reads again and still gets 1000: a repeatable read.
Step 3	B tries to change the row from a snapshot that is no longer current. SQLite refuses (the message is the same wording as an ordinary lock error). PostgreSQL does the same at REPEATABLE READ with `ERROR: could not serialize access due to concurrent update` (not run here).
Step 4	B rolls back and starts again. Its fresh snapshot reads 900 and the retry works.
Answer	A snapshot gives stable reads, and the price is that a stale writer must retry.

Example 3: phantom read

t1	B: `BEGIN; SELECT count(*) FROM orders WHERE status = 'pending';` returns 3
t2	A: `INSERT INTO orders ... 'pending'; COMMIT;`
t3	B: the same `SELECT count(*)` . READ COMMITTED returns 4 (a new "phantom" row appeared). PostgreSQL REPEATABLE READ returns 3.

The difference from a non-repeatable read is that no row B already saw changed. A *new* row joined the set. Row locks cannot stop this, because there is no row to lock yet. That is why the SQL standard lists phantoms separately.

Example 4: lost update (the application version)

You met this in Problem 9.4.2. In timeline form, with A subtracting 100 and B subtracting 200 from 1000:

t1	A: reads 1000. B: reads 1000.
t2	A: `UPDATE ... SET balance = 900`, commit.
t3	B: `UPDATE ... SET balance = 800`, commit. Final balance 800, should be 700.
Fixes	(1) `SET balance = balance - 200`, atomic. (2) `SELECT ... FOR UPDATE` so B waits at t1. (3) PostgreSQL REPEATABLE READ or SERIALIZABLE: B's write at t3 fails with a serialization error and you retry. (4) A `version` column checked in the `WHERE`.

COMMON MISTAKE	Believing "REPEATABLE READ means no lost updates" in every database. In PostgreSQL the second writer gets an error, which is safe if you retry. In MySQL InnoDB, plain `SELECT` reads a snapshot but an `UPDATE` acts on the latest committed row, so an application-level read, compute, write still loses an update. Use the atomic `SET x = x - 200` form, or `FOR UPDATE`, on both.

Example 5: write skew (the one that fools experts)

A hospital rule: at least one doctor must stay on call. Today Asha and Ravi are both on call. Each wants to leave.

t1	A (Asha): `BEGIN; SELECT count(*) FROM doctors WHERE on_call;` returns 2, so leaving is allowed.
t2	B (Ravi): `BEGIN; SELECT count(*) FROM doctors WHERE on_call;` returns 2, so leaving is allowed.
t3	A: `UPDATE doctors SET on_call = false WHERE name = 'Asha'; COMMIT;`
t4	B: `UPDATE doctors SET on_call = false WHERE name = 'Ravi'; COMMIT;`
Result	Zero doctors on call. A and B updated *different* rows, so no row conflict was detected, yet together they broke the rule.

This is **write skew**: each transaction made a decision from a read that the other transaction invalidated. Snapshot isolation (PostgreSQL REPEATABLE READ) allows it. PostgreSQL SERIALIZABLE detects it and aborts one transaction with SQLSTATE `40001`. You can also lock the rows that your decision depends on: `SELECT ... FROM doctors WHERE on_call FOR UPDATE` at t1 and t2 makes B wait until A finishes (not run here).

Which level prevents which problem (PostgreSQL)

Problem	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Dirty read	prevented	prevented	prevented
Non-repeatable read	possible	prevented	prevented
Phantom read	possible	prevented (snapshot)	prevented
Lost update (read then write in the app)	possible	error, retry	error, retry
Write skew	possible	possible	error, retry

MySQL InnoDB differs in details (its default is REPEATABLE READ, and it uses gap locks for locking reads), so check its manual before you rely on a cell of this table for InnoDB.

Deadlock reproduction and the retry loop

The deadlock timeline from 9.4 can be reproduced in PostgreSQL with two `psql` windows (not run here). The fix on the application side is always the same: catch the error, roll back, wait a little, and run the whole transaction again. PostgreSQL signals deadlocks with SQLSTATE `40P01` and

serialization failures with `40001`. This helper is generic Python and I ran it with a fake job that fails twice and then succeeds:

```
import time, random
random.seed(1)
class Retryable(Exception):
    pass
def run_with_retry(job, tries=5):
    for attempt in range(1, tries + 1):
        try:
            return job(attempt)
        except Retryable as e:
            wait = 0.01 * 2 ** attempt + random.random() * 0.01
            print("attempt", attempt, "failed:", e, "| wait about", round(wait, 2), "s")
            time.sleep(wait)
    raise RuntimeError("gave up after %d tries" % tries)
def job(attempt):
    if attempt < 3:
        raise Retryable("deadlock detected")
    return "committed on attempt %d" % attempt
print(run_with_retry(job))
```

```
attempt 1 failed: deadlock detected | wait about 0.02 s
attempt 2 failed: deadlock detected | wait about 0.05 s
committed on attempt 3
```

- **Step 1** The helper runs the whole job again, not just the failed statement. A rolled-back transaction leaves nothing behind, so the retry starts clean.
- **Step 2** The wait doubles each time (exponential backoff) plus a random bit (jitter) so two retrying sessions do not collide again in lockstep.
- **Step 3** There is a cap on attempts, because a bug that always fails must not loop forever.

Job queue: claim one task safely

Several workers pull tasks from one `jobs` table. The danger is two workers claiming the same task. The fix is a single statement that finds and marks the task together. In PostgreSQL the pattern is `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1`: rows locked by another worker are skipped instead of waited for (not run here). SQLite has one writer, so a plain atomic `UPDATE ... RETURNING` does the job, and I ran it:

Problem 9.5.2 Three tasks are queued and four workers each claim one. Predict first: workers 1 to 3 get ids 1 to 3, worker 4 gets nothing.

```
import sqlite3
db = sqlite3.connect(":memory:", isolation_level=None)
db.execute("CREATE TABLE jobs (id INTEGER PRIMARY KEY, status TEXT, worker TEXT)")
db.executemany("INSERT INTO jobs (status) VALUES (?)", [("queued",)] * 3)
claim = """UPDATE jobs SET status = 'running', worker = ?
           WHERE id = (SELECT id FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 1)
           RETURNING id"""
for w in ["w1", "w2", "w3", "w4"]:
    print(w, "claimed", db.execute(claim, (w,)).fetchall())
print(db.execute("SELECT * FROM jobs").fetchall())
```

```
w1 claimed [(1,)]
w2 claimed [(2,)]
w3 claimed [(3,)]
w4 claimed []
[(1, 'running', 'w1'), (2, 'running', 'w2'), (3, 'running', 'w3')]
```

- **Step 1** The inner `SELECT` picks the lowest queued id, and the outer `UPDATE` marks it in the same statement, so no other worker can slip in between.
- **Step 2** `RETURNING id` hands the claimed id back without a second query.
- **Step 3** Worker 4 finds nothing queued, so it gets an empty list and should sleep.

- **Answer** Each task was claimed exactly once.

GOING DEEPER	Without `SKIP LOCKED`, workers using `FOR UPDATE` line up behind the first locked row and the queue runs at the speed of one worker. With it, each worker takes the next free row, so throughput grows with the number of workers. Add an index `ON jobs (id) WHERE status = 'queued'` (a partial index, Chapter 10) so finding the next task stays fast as finished jobs pile up. If a worker dies after claiming, the task stays "running" forever, so real systems add a heartbeat or a `claimed_at` timeout column that lets another worker reclaim it.

9.6 Security: SQL Injection, Permissions and Row-Level Security

INTUITION	Imagine a bank clerk who types in whatever a customer says out loud, including the sentence "and also hand me the vault key." **SQL injection** is that: your program glues user text into a SQL command, so the text can become part of the command. The cure is to hand the database the command and the values *separately*, so the values can never be read as commands.

Words you need

- **SQL injection**: an attack where user input changes the meaning of a query.
- **Parameterized query** (prepared statement): the SQL text has placeholders (`?` in sqlite3, `%s` in psycopg) and the values travel separately.
- **Privilege**: permission to do one kind of action on one object. **Role**: a named bundle of privileges you can give to users.
- **Least privilege**: give every account only what it needs and nothing more.

Example 1: the attack, step by step

Problem 9.6.1 A web page lets users type a city and returns customer names. The developer builds the query with string joining. The table also holds a `secret` column that the page is never supposed to show. Predict first: a normal input returns two names, a crafted input returns the secrets.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT, city TEXT, secret TEXT);
INSERT INTO customers VALUES (1,'Asha','Pune','card-1111'),(2,'Ravi','Mumbai','card-2222'),(3,'Meena','Pune','card-3333');
""")
def bad(city):
    sql = "SELECT name FROM customers WHERE city = '" + city + "'"
    return db.execute(sql).fetchall()
def good(city):
    return db.execute("SELECT name FROM customers WHERE city = ?", (city,)).fetchall()
evil = "x' UNION SELECT secret FROM customers --"
print("bad normal :", bad("Pune"))
print("bad evil   :", bad(evil))
print("good evil  :", good(evil))
try:
    bad("x'; DROP TABLE customers; --")
except Exception as e:
    print("stacked    :", type(e).__name__, e)
```

```
bad normal : [('Asha',), ('Meena',)]
bad evil   : [('card-1111',), ('card-2222',), ('card-3333',)]
good evil  : []
stacked    : ProgrammingError You can only execute one statement at a time.
```

- **Step 1** With the input `Pune` the glued text is `WHERE city = 'Pune'`. Fine.
- **Step 2** With the evil input the glued text becomes `WHERE city = 'x' UNION SELECT secret FROM customers --'`. The quote closes the string early, `UNION` adds a second query, and `--` comments out the leftover quote. The page prints the card numbers.

- **Step 3** The parameterized version sends the same evil text as a plain value. No city is literally named `x' UNION ...`, so it returns an empty list.
- **Step 4** The last line shows that Python's sqlite3 refuses two statements in one call, which blocks the `DROP TABLE` trick. PostgreSQL drivers do not all behave this way when you send a simple query string, so never rely on it.

COMMON MISTAKE	Thinking an f-string is safe because "the input comes from our own dropdown." Anyone can send any HTTP request, dropdown or not. Also, escaping quotes by hand fails in many corner cases. Placeholders are the only reliable rule: `cursor.execute("... WHERE city = %s", (city,))` in psycopg (not run here, same idea as the `?` above).

What placeholders cannot do

Placeholders replace *values* only. They cannot stand in for a table name, a column name or `ASC` / `DESC`. If a user can pick the sort column, check the choice against a fixed allow-list in your code, then join the approved word into the text:

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE products (name TEXT, price INTEGER)")
db.executemany("INSERT INTO products VALUES (?, ?)", [("Mug", 250), ("Pen Pack", 120), ("Lamp", 900)])
ALLOWED = {"name": "name", "price": "price"}
def list_products(sort_by):
    col = ALLOWED.get(sort_by)
    if col is None:
        raise ValueError("bad sort column: " + sort_by)
    return db.execute("SELECT name FROM products ORDER BY " + col).fetchall()
print(list_products("price"))
try:
    list_products("price; DROP TABLE products")
except ValueError as e:
    print("rejected:", e)
```

```
[('Pen Pack',), ('Mug',), ('Lamp',)]
rejected: bad sort column: price; DROP TABLE products
```

Example 2: views as access filters

A **view** is a saved query that behaves like a table. Give analysts the view, not the table, and they can only see the columns and rows you chose. We hide `referrer_id` and show only Pune customers to a regional team:

```
CREATE VIEW pune_customers AS
SELECT customer_id, name, signup_date FROM customers WHERE city = 'Pune';
SELECT * FROM pune_customers;
SELECT referrer_id FROM pune_customers;
```

```
customer_id  name   signup_date
-----------  -----  -----------
1            Asha   2024-01-05
3            Meena  2024-01-20
8            Tom    2024-04-01
Parse error: no such column: referrer_id
  SELECT referrer_id FROM pune_customers;
         ^--- error here
```

- **Step 1** The first query returns the three Pune customers with only the three allowed columns.
- **Step 2** Asking the view for `referrer_id` fails, because the view never exposed it.
- **Answer** A view narrows both columns and rows. In PostgreSQL, you then `GRANT SELECT` on the view and not on the table (not run here). Note that in PostgreSQL a view runs with the privileges of its owner by default, so a view can expose rows the viewer could not read directly. That is the point of using it as a filter, and also the reason to be careful about who owns it.

Example 3: GRANT, REVOKE and roles (PostgreSQL, not run here)

```
-- one role per job, not per person
CREATE ROLE analyst NOLOGIN;
CREATE ROLE etl_loader NOLOGIN;
GRANT SELECT ON orders, order_items, products TO analyst;
GRANT SELECT, INSERT, UPDATE ON orders TO etl_loader;
-- people inherit a role
CREATE ROLE priya LOGIN PASSWORD '...';
GRANT analyst TO priya;
-- take a right away again
REVOKE UPDATE ON orders FROM etl_loader;
```

- **Step 1** Create roles that match jobs. Roles without `LOGIN` are groups.
- **Step 2** Grant only what each job needs: analysts read, the loader reads and writes `orders`, nobody can `DROP` anything.
- **Step 3** Add a person by granting the role, remove them by revoking it. You never edit per-table rights for one human again.

KEY IDEA	The application should connect with an account that can only run the statements the application needs (usually `SELECT`, `INSERT`, `UPDATE`, `DELETE` on a few tables), never as the database owner. Then even a successful injection cannot drop tables. Least privilege is the second wall behind parameterized queries.

Going deeper: row-level security (PostgreSQL, not run here)

Row-level security (RLS) lets the database filter rows per user, so every query, including a mistaken one, is filtered. A multi-tenant example, where each tenant sees only its own orders:

```
ALTER TABLE orders ADD COLUMN tenant_id INTEGER;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.tenant_id')::integer);
-- each request, after connecting:
SET app.tenant_id = '7';
SELECT count(*) FROM orders;  -- counts tenant 7 only
```

Two facts to remember. Table owners and superusers bypass RLS unless you also run `ALTER TABLE ... FORCE ROW LEVEL SECURITY` (superusers bypass it regardless). And a policy with `USING` filters reads and the matching rows for updates, while `WITH CHECK` controls what new rows may be written.

9.7 Programmability: Views, Triggers, Generated Columns, Identity and MERGE

INTUITION	So far the database stored data and you sent it commands. Programmability lets the database *react* and *derive* by itself, like a spreadsheet: a formula column that is always up to date, a rule that writes a log line whenever a cell changes. It is powerful and also easy to overuse, so this section ends with when not to.

Views and materialized views

A normal view stores only the query, so it is always fresh and as slow as the query. A **materialized view** stores the result, so it is fast and stale until refreshed. PostgreSQL has `CREATE MATERIALIZED VIEW` and `REFRESH MATERIALIZED VIEW name` (`CONCURRENTLY` keeps readers working but needs a unique index; not run here). SQLite has no materialized views, so I imitate one with a table and a refresh script, which shows exactly what staleness means.

Problem 9.7.1 Build a view and a "materialized" copy of paid revenue per customer, add a new paid order, and compare. Predict first: the view includes the new order, the copy does not until refreshed.

```
CREATE VIEW paid_rev_v AS
SELECT o.customer_id, sum(i.qty * i.unit_price) AS revenue
FROM orders o JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'paid' GROUP BY o.customer_id;

CREATE TABLE paid_rev_m AS SELECT * FROM paid_rev_v;

INSERT INTO orders VALUES (115, 6, '2024-04-25', 'paid', NULL);
INSERT INTO order_items VALUES (115, 1, 1, 1500);

SELECT v.customer_id, v.revenue AS view_now, m.revenue AS copy_stale
FROM paid_rev_v v JOIN paid_rev_m m USING (customer_id) WHERE v.customer_id = 6;
```

```
customer_id  view_now  copy_stale
-----------  --------  ----------
6            1940      440
```

```
BEGIN;
DELETE FROM paid_rev_m;
INSERT INTO paid_rev_m SELECT * FROM paid_rev_v;
COMMIT;
SELECT revenue FROM paid_rev_m WHERE customer_id = 6;
```

```
revenue
-------
1940
```

- **Step 1** Customer 6 had one paid order, 110: 4 x 80 + 1 x 120 = 440.
- **Step 2** After order 115 (1500) the view says 440 + 1500 = 1940, while the copy still says 440.
- **Step 3** The refresh inside one transaction replaces the copy, so readers never see it half empty. Now the copy says 1940.
- **Answer** Choose a materialized view when the query is expensive and slightly old data is acceptable, for example a dashboard refreshed every night.

Triggers: an audit log

A **trigger** is code that runs automatically when a row is inserted, updated or deleted. The classic use is an audit trail: who changed a price, from what to what.

Problem 9.7.2 Log every real price change in `products`. Update product 1 from 1500 to 1600, product 2 from 600 to 620, and product 3 to the same 80. Predict first: two log rows, because the trigger ignores the no-change update.

```
CREATE TABLE price_audit (
  audit_id   INTEGER PRIMARY KEY,
  product_id INTEGER, old_price INTEGER, new_price INTEGER
);
CREATE TRIGGER trg_price_audit
AFTER UPDATE OF price ON products
WHEN OLD.price <> NEW.price
BEGIN
  INSERT INTO price_audit (product_id, old_price, new_price)
  VALUES (OLD.product_id, OLD.price, NEW.price);
END;

UPDATE products SET price = 1600 WHERE product_id = 1;
UPDATE products SET price = 620  WHERE product_id = 2;
UPDATE products SET price = 80   WHERE product_id = 3;
SELECT * FROM price_audit;
```

```
audit_id  product_id  old_price  new_price
--------  ----------  ---------  ---------
1         1           1500       1600
2         2           600        620
```

- **Step 1** `AFTER UPDATE OF price` fires only when the `price` column is in the `SET` list.
- **Step 2** `WHEN OLD.price <> NEW.price` skips the third update, where nothing changed.
- **Step 3** `OLD` and `NEW` are the row before and after. The trigger writes both into the log inside the same transaction, so a rollback removes the log row too.
- **Answer** Two audit rows, ids 1 and 2.

COMMON MISTAKE

Hiding business rules in triggers nobody remembers. A trigger runs on every write, including bulk loads, so it can silently make a 10-minute load take 2 hours, and it makes behaviour hard to read from the application code. Use triggers for audit logs and invariants the database must guarantee. Put ordinary business logic where it can be tested and seen. In PostgreSQL a trigger calls a function (`CREATE FUNCTION ... RETURNS trigger LANGUAGE plpgsql`, not run here), and a bulk `COPY` still fires row triggers.

Generated columns

A **generated column** is computed by the database from other columns. `VIRTUAL` computes it when read; `STORED` saves it on write (SQLite supports both when you create the table; `ALTER TABLE ADD COLUMN` accepts only `VIRTUAL`). PostgreSQL supports `GENERATED ALWAYS AS (...) STORED` (virtual generated columns arrived only in PostgreSQL 18).

Problem 9.7.3 Add a `line_total` to `order_items` and use it. Predict first: it equals qty times unit price, with no extra storage and no way to get out of sync.

```
ALTER TABLE order_items ADD COLUMN line_total INTEGER GENERATED ALWAYS AS (qty * unit_price) VIRTUAL;
SELECT order_id, product_id, line_total FROM order_items WHERE order_id IN (101, 106);
SELECT sum(line_total) AS all_lines FROM order_items;
UPDATE order_items SET line_total = 5 WHERE order_id = 101;
```

```
order_id  product_id  line_total
--------  ----------  ----------
101       1           1500
101       3           160
106       2           1200
106       3           80
all_lines
---------
14560
Parse error: cannot UPDATE generated column "line_total"
```

- **Step 1** Order 101 has 1 x 1500 = 1500 and 2 x 80 = 160. Order 106 has 2 x 600 = 1200 and 1 x 80 = 80.
- **Step 2** The total of all 20 lines is the sum shown.
- **Step 3** Writing to a generated column is refused. The value cannot drift from its inputs, which is the whole benefit.

Identity columns and sequences

An **identity** column hands out the next number for you. PostgreSQL writes `id BIGINT GENERATED ALWAYS AS IDENTITY` and uses a **sequence** underneath (not run here). One fact surprises beginners: a sequence never goes backwards, so a rolled-back insert leaves a *gap*. Gaps are normal and harmless; never use ids as a gap-free counter such as an invoice number. SQLite shows a related choice, whether a deleted top id may be reused:

```
CREATE TABLE t_plain (id INTEGER PRIMARY KEY, v TEXT);
CREATE TABLE t_auto  (id INTEGER PRIMARY KEY AUTOINCREMENT, v TEXT);
INSERT INTO t_plain (v) VALUES ('a'),('b'),('c');
INSERT INTO t_auto  (v) VALUES ('a'),('b'),('c');
DELETE FROM t_plain WHERE id = 3;
DELETE FROM t_auto  WHERE id = 3;
INSERT INTO t_plain (v) VALUES ('d');
INSERT INTO t_auto  (v) VALUES ('d');
SELECT 'plain' AS tbl, max(id) AS newest FROM t_plain
UNION ALL SELECT 'autoincrement', max(id) FROM t_auto;
```

```
tbl           newest
------------  ------
plain         3
autoincrement  4
```

The plain table reused id 3, the `AUTOINCREMENT` table gave out 4. If other systems, such as logs or caches, remember id 3, reuse can attach old facts to a new row, so prefer the never-reuse behaviour for ids that leave your database.

MERGE, the standard upsert

`ON CONFLICT` (Chapter 9.2) is PostgreSQL and SQLite syntax. The SQL standard form is `MERGE`, available in PostgreSQL 15 and later, SQL Server, Oracle, Snowflake and BigQuery, but not in SQLite or MySQL (not run here):

```
MERGE INTO stock s
USING incoming n ON s.product_id = n.product_id
WHEN MATCHED AND n.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty + n.qty
WHEN NOT MATCHED THEN INSERT (product_id, qty) VALUES (n.product_id, n.qty);
```

Read it as a join with three outcomes. Matched rows can be updated or deleted by a condition, unmatched rows are inserted, and one statement does it all. `ON CONFLICT` cannot delete, and it needs a unique index to conflict on; `MERGE` works on any join condition. PostgreSQL 17 added a `RETURNING` clause to `MERGE`. Under concurrency `MERGE` in PostgreSQL can still raise a unique violation if two sessions insert the same key at once, so keep the retry loop from 9.5.

REAL JOB

Stored functions and procedures (PostgreSQL `CREATE FUNCTION`, `CREATE PROCEDURE`, `CALL`) put code next to the data and cut network round trips. Teams use them for batch maintenance and triggers. Most teams keep ordinary business logic in application code or in dbt models instead, because that code is easier to version, test and review.

Chapter recap

- DDL defines structure (`CREATE`, `ALTER`, `DROP`). Constraints (`NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`) make the database reject bad data.
- SQLite needs `PRAGMA foreign_keys = ON` each session and is loosely typed unless `STRICT`; PostgreSQL always enforces both.
- Use `NUMERIC` for money. Prefer surrogate primary keys plus a `UNIQUE` natural key.
- `INSERT`, `UPDATE`, `DELETE`: write the `WHERE` first and test it with a `SELECT COUNT(*)`.
- Upsert is `INSERT ... ON CONFLICT DO UPDATE / DO NOTHING`; do not use `REPLACE`, which deletes the whole row.
- A transaction groups statements: `BEGIN`, then `COMMIT` or `ROLLBACK`. Handle errors in code.
- ACID: atomic, consistent, isolated, durable. The write-ahead log gives durability.
- Isolation levels trade safety for speed: READ COMMITTED is the PostgreSQL default; SERIALIZABLE needs retries.
- Never read, then write back a computed value in app code; use `SET x = x - 100` or `FOR UPDATE`.
- Deadlocks: lock in a fixed order, keep transactions short, retry.

Cheat-sheet

Task	Pattern
Safe rerunnable create	`CREATE TABLE IF NOT EXISTS t (...)`, `DROP TABLE IF EXISTS t`
Add column	`ALTER TABLE t ADD COLUMN c TYPE`
Upsert	`INSERT ... ON CONFLICT (key) DO UPDATE SET c = excluded.c`
Insert and get id	`INSERT ... RETURNING id`
Atomic block	`BEGIN; ...; COMMIT;` or `ROLLBACK;`
Partial undo	`SAVEPOINT s; ... ROLLBACK TO s;`
Row lock (PostgreSQL)	`SELECT ... FOR UPDATE`
Isolation (PostgreSQL)	`BEGIN ISOLATION LEVEL SERIALIZABLE;`
Bulk load	`COPY` (PostgreSQL), `.import` (SQLite)

Practice set

Problem 9.P.1 (easy) Which constraint ensures `age` is between 0 and 120? **Answer:** `CHECK (age BETWEEN 0 AND 120)`.

Problem 9.P.2 (what is the output?) `CREATE TABLE t (a INTEGER PRIMARY KEY, b TEXT DEFAULT 'x'); INSERT INTO t (a) VALUES (5); SELECT * FROM t;` **Answer:** One row: `5` and `x`. The default fills `b`.

Problem 9.P.3 (find the bug) `DELETE FROM orders WHERE status = 'cancelled' OR 'refunded';` What is wrong? **Answer:** `OR 'refunded'` is not a comparison. Write `status IN ('cancelled','refunded')`. In SQLite the text `'refunded'` evaluates as 0 (false), so only cancelled rows would go, a silent bug. Check with a `SELECT COUNT(*)` first.

Problem 9.P.4 (find the bug) A function reads `bal = SELECT balance ...`, then runs `UPDATE accounts SET balance = <bal − 100>`. Two users run it at once from balance 1000. What can the final balance be, and what is the fix? **Answer:** 900 instead of 800 (lost update). Fix: `UPDATE accounts SET balance = balance − 100 WHERE id = 1 AND balance >= 100`, and check that one row changed.

Problem 9.P.5 (scenario) A nightly job loads 2 million rows with one `INSERT` each, autocommit on, and takes 3 hours. Suggest two improvements. **Answer:** Wrap batches (say 10,000 rows) in one transaction, so there are fewer commits and disk waits. Better, use `COPY` (PostgreSQL) or a multi-row `INSERT`. Also drop or defer heavy indexes during the load (Chapter 10).

Problem 9.P.6 (scenario) Your app logs `deadlock detected` about 5 times a day. Two functions update `accounts` row 1 then row 2, and row 2 then row 1. Fix it. **Answer:** Make both lock rows in the same order (lowest id first), keep the transaction short, and retry the transaction when the error occurs.

Problem 9.P.7 (interview) Explain the difference between READ COMMITTED and REPEATABLE READ with an example. **Answer:** In READ COMMITTED each statement sees data committed so far, so two identical `SELECT`s in one transaction can differ if someone commits between them

(non-repeatable read). In REPEATABLE READ the transaction sees one snapshot taken at its start, so both `SELECT`s agree. Cost: REPEATABLE READ may need retries on write conflicts.

Problem 9.P.8 (interview) Why does PostgreSQL not block a long report query when an `UPDATE` hits the same table? **Answer:** MVCC. The report reads a snapshot of old row versions, while the `UPDATE` writes new versions. Readers and writers do not block each other. The cost is dead rows that `VACUUM` must clean later.

Problem 9.P.9 (easy) Name the anomaly: a report reads a customer's balance twice in one transaction and gets 1000 then 900 because another session committed in between. **Answer:** Non-repeatable read. REPEATABLE READ prevents it by giving the transaction one snapshot.

Problem 9.P.10 (what is the output?) Two doctors are on call and the rule is "at least one." Each transaction checks the count (2), then sets its own doctor off call, and both commit at READ COMMITTED. How many are on call after? **Answer:** Zero. This is write skew. Each wrote a different row, so there was no row conflict. Fix with SERIALIZABLE plus retry, or lock the rows read with `SELECT ... FOR UPDATE`.

Problem 9.P.11 (find the bug) `db.execute(f"SELECT * FROM users WHERE name = '{name}'")`. What input returns every user, and how do you fix it? **Answer:** `' OR '1'='1` turns the filter into `name = '' OR '1'='1'`, which is true for every row. Fix: `db.execute("SELECT * FROM users WHERE name = ?", (name,))`.

Problem 9.P.12 (scenario) A page lets users sort by a column chosen in the URL. Why can you not use a placeholder for it, and what do you do? **Answer:** Placeholders carry values, not identifiers such as column names. Check the requested name against an allow-list dictionary in code and reject anything else, then place the approved word in the SQL text.

Problem 9.P.13 (scenario) Ten workers pull from a `jobs` table with `SELECT ... FOR UPDATE LIMIT 1` and throughput does not grow past one worker. Why, and what changes it? **Answer:** Every worker waits for the same first locked row. Add `SKIP LOCKED` so each worker skips rows that are locked and takes the next free one. Keep a partial index on the queued rows.

Problem 9.P.14 (what is the output?) In SQLite, a table `t(id INTEGER PRIMARY KEY)` without `AUTOINCREMENT` holds ids 1, 2, 3. You delete id 3 and insert a new row. What id does it get, and what with `AUTOINCREMENT`? **Answer:** Without it, 3 (the largest id plus one). With it, 4, because SQLite remembers the largest id ever used.

Problem 9.P.15 (find the bug) A trigger on `orders` sends an HTTP request to a billing service for every inserted row. Why is this dangerous? **Answer:** It runs inside the writing transaction: a slow or failing service blocks or rolls back inserts, and the request cannot be undone if the transaction later rolls back. A bulk load of 1 million rows makes 1 million calls. Write the event into an outbox table in the same transaction and let a separate worker send it.

Problem 9.P.16 (interview) Explain least privilege using the database accounts of a web application. **Answer:** The app connects with a role that has only the rights it uses (for example `SELECT, INSERT, UPDATE` on three tables), not the owner role. Migrations run under a different role that can alter tables. Then a leaked password or a successful SQL injection cannot drop tables or read unrelated data. Combine it with parameterized queries, because privileges limit the damage and placeholders prevent the attack.

Problem 9.P.17 (interview) When would you choose a materialized view over a normal view, and what is the cost? **Answer:** When the query is expensive (large joins or aggregates) and users accept data that is as old as the last refresh, such as a nightly dashboard. The cost is staleness, storage, and a refresh job that takes time and, without `CONCURRENTLY`, blocks readers in PostgreSQL while it runs.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning SQL for data roles. Teach me DDL and constraints: CREATE, ALTER, DROP, data types, PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL, DEFAULT and ON DELETE CASCADE. Use a small orders and customers example. Explain step by step in simple English, show each error message when a rule is broken, and after each idea ask me one question to check my understanding before moving on.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data engineering. Explain INSERT, UPDATE, DELETE and the upsert pattern INSERT ... ON CONFLICT in PostgreSQL, and how it differs from MERGE, REPLACE and INSERT OR REPLACE. Give 8 graded exercises with a small inventory table, no solutions at first. Then wait for my answers and correct them kindly, explaining every mistake.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional learning SQL. Teach me transactions and ACID with a bank-transfer example: BEGIN, COMMIT, ROLLBACK, savepoints and what happens when a statement fails halfway. Explain each ACID letter with an everyday analogy (a bank queue or a shop counter). Then quiz me one question at a time, with a short scenario each time, and only give the next question after I answer.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional who understands shared counters but is new to databases. Explain isolation levels (READ COMMITTED, REPEATABLE READ, SERIALIZABLE), dirty, non-repeatable and phantom reads, MVCC, row locks, lost updates and deadlocks in PostgreSQL. Use two-session timelines with numbers. Then give me 6 scenarios and ask which anomaly or lock problem occurs and how to fix it.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional preparing for data engineer interviews. Run a mock interview on SQL transactions, constraints and concurrency: ask me 8 questions, one at a time, from easy to hard, and wait for my answer each time. After each answer, score it out of 5, point out missing points, and give a model answer in simple English. Here is my last piece of work to review as well: [PASTE HERE]
```

CHAPTER 10

INDEXES, EXPLAIN AND QUERY OPTIMIZATION

Why one query takes 4 ms and another takes 400, how to read the database's plan, and how to fix slow SQL step by step, measuring before and after.

You already know how to find a bottleneck: measure, change one thing, measure again. This chapter applies that same habit to the database. A report that took 10 seconds in testing will take minutes with real data, and the usual cause is that the database reads every row when it could read a few. You will learn what an index is, how to see the database's plan with `EXPLAIN`, how to write queries the planner can speed up, and when an index makes things worse. By the end you can take a slow query, prove why it is slow, fix it, and show the before and after numbers.

Examples run in SQLite 3.51 using `EXPLAIN QUERY PLAN`. PostgreSQL's `EXPLAIN ANALYZE` is described but **not run here**, and I do not invent its numbers. Timings below are real runs on my laptop: your numbers will differ, but the ratios will look alike.

Running example. A table `big` with 200,000 rows, built so you can reproduce it:

```
CREATE TABLE big (id INTEGER PRIMARY KEY, customer_id INTEGER,
                  amount INTEGER, status TEXT, created TEXT);
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i+1 FROM n WHERE i < 200000)
INSERT INTO big
SELECT i, i % 5000, (i*37) % 1000,
       CASE WHEN i % 10 = 0 THEN 'refunded' ELSE 'paid' END,
       date('2024-01-01', '+' || (i % 365) || ' days')
FROM n;
```

There are 5,000 customers (so each has 40 rows), 10% of rows are `refunded`, and `created` spans one year. We also use the small shop tables from Chapter 2.

10.1 What an Index Is, and What It Costs

INTUITION

To find "Sampling" in a 500-page textbook you do not read from page 1. You open the **index** at the back, find "Sampling, p. 212," and jump. A database index is the same: a separate, sorted structure that says "customer 42 is in these rows." Without it, the database does a **full table scan**: it reads every row, like reading the whole book. An index turns an O(n) lookup into roughly O(log n).

Words you need

- **Full table scan** (PostgreSQL: *seq scan*; SQLite: `SCAN`): read every row.
- **Index scan / seek** (`SEARCH ... USING INDEX`): use the index to jump to matching rows.
- **B-tree**: the default index structure: a sorted, balanced tree where each step cuts the search area by a large factor. 200,000 rows need only 3 or 4 steps because each tree node holds hundreds of keys.
- **Selectivity**: the fraction of rows a filter keeps. `customer_id = 42` keeps 40 of 200,000 (0.02%): very selective. `status = 'paid'` keeps 90%: not selective.
- **Planner / optimizer**: the part of the database that picks the plan.

How it works

`CREATE INDEX idx_name ON table(column);` builds a sorted copy of that column plus a pointer to each row. Lookups use the sorted order to jump. The cost is paid on **writes**: every `INSERT`, `UPDATE` of an indexed column and `DELETE` must also change each index.

Real-world mapping

Item	Example	Where it appears in the job
Primary key index	`orders(order_id)`	automatic: every PK and `UNIQUE` has an index
Foreign key column	`orders(customer_id)`	joins and cascades; PostgreSQL does **not** index FKs for you
Time column	`events(created)`	dashboards filtering "last 7 days"
Search column	`users(email)`	login lookup

Worked examples

Problem 10.1.1 Find the rows of customer 42. Predict the plan before and after creating an index. Before: the planner must scan. After: it can search.

```
EXPLAIN QUERY PLAN SELECT * FROM big WHERE customer_id = 42;
CREATE INDEX idx_big_cust ON big(customer_id);
EXPLAIN QUERY PLAN SELECT * FROM big WHERE customer_id = 42;
```

```
QUERY PLAN
`--SCAN big
QUERY PLAN
`--SEARCH big USING INDEX idx_big_cust (customer_id=?)
```

- **Step 1** `SCAN big` means all 200,000 rows are read.
- **Step 2** After the index, `SEARCH ... USING INDEX` means the planner jumps to the 40 matching rows.
- **Answer** The plan changed from a scan to an index search.

How much faster? I ran each query in a Python loop and averaged:

```
no index  : 4.237 ms
with index: 0.029 ms
rows: 40
```

- **Compute** 4.237 / 0.029 is about 146.
- **Answer** The index made this lookup roughly 146 times faster, and the result is the same 40 rows.
- **Meaning** If a page runs this query 50 times per second, the scan version needs 50 × 4.2 = 210 ms of database time every second (about 21% of a core), while the index version needs 1.5 ms. That difference is what lets a system keep waiting times flat as users grow.
- **Check** 40 rows is what we predicted: 200,000 rows / 5,000 customers = 40.

Problem 10.1.2 (the cost side) Insert 100,000 rows into a table with 0, 2, and 4 indexes. Predict: more indexes, slower inserts.

```
0 indexes: insert 100,000 rows in 0.05 s
2 indexes: insert 100,000 rows in 0.12 s
4 indexes: insert 100,000 rows in 0.21 s
```

- **Compute** 0.21 / 0.05 = 4.2, so four indexes made loading about 4 times slower.
- **Answer** Each index adds write work, so reads and writes pull in opposite directions.
- **Meaning** For a log table that takes 5,000 inserts per second and is rarely queried, extra indexes are pure cost. Bulk load first, create indexes after.

When NOT to index

Situation	Why an index does not help
Tiny table (hundreds of rows)	a scan is as fast as a lookup
Low selectivity (`status` with 2 values, 90% `paid`)	the planner reads most rows anyway; it ignores the index
Column that is constantly updated	each update also rewrites the index
Write-heavy table with rare reads	write slowdown beats read speedup
Query returns a large share of the table	a sequential scan is cheaper than jumping 100,000 times

COMMON MISTAKE

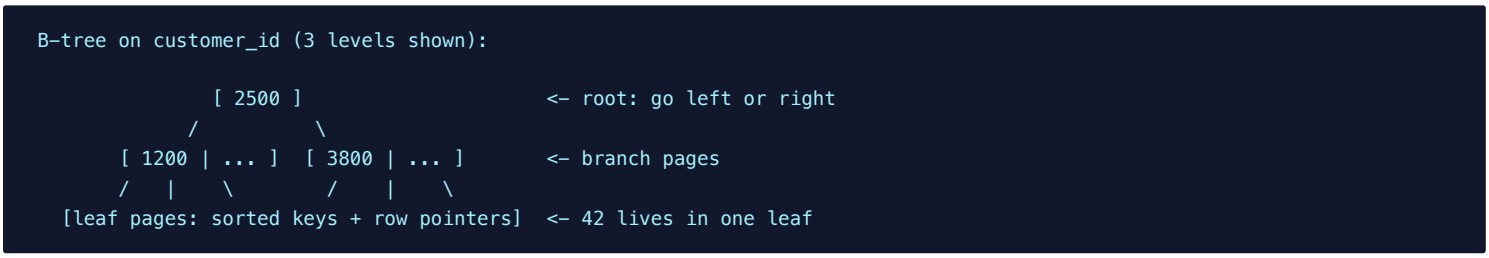
"Slow query, so add an index on every column." Each index costs disk space and write speed, and many will never be used. Check the plan first, add the index that serves a real query, and measure.

Index types beyond the default

Type	Good for	Notes
B-tree (default)	`=` , `<` , `>` , `BETWEEN` , `ORDER BY` , prefix `LIKE 'abc%'`	works for almost everything
Hash (PostgreSQL)	`=` only	rarely better than B-tree
GIN (PostgreSQL)	`JSONB` contents, arrays, full-text search	"which rows contain this word / key"
GiST (PostgreSQL)	geometry, ranges, nearest-neighbour	PostGIS uses it

GIN, GiST and hash indexes are PostgreSQL features and are **not run here**.

The picture



Start at the root, compare, go down. Each step skips most of the data. Leaf pages are linked in sorted order, so a range like `BETWEEN 100 AND 199` reads leaves one after another.

Connection to the real job

In every interview "how do you speed up a slow query?" starts with "look at the plan, then think about an index." On the job, missing indexes on foreign-key and timestamp columns are the number one cause of slow dashboards.

Applications

- **Data Analytics:** index the date column your dashboard filters on.
- **Data Engineering:** drop indexes during a huge load, rebuild after.
- **Back ends:** unique index on `email` makes login O(log n) and prevents duplicate accounts.
- **Operations:** a report that runs fast on 10,000 rows and crawls on 10,000,000 often exposes a missing index.

Check yourself

1. What does `SCAN big` in a plan mean?

Answer: The database reads every row of `big`.

1. Why do extra indexes slow down inserts?

Answer: Every insert must also add an entry to each index.

1. Why is an index on a 2-value column usually useless?

Answer: Low selectivity: the filter keeps about half the rows, so the planner reads most of the table anyway.

10.2 Composite, Covering, Partial and Expression Indexes

INTUITION	A phone book is sorted by **last name, then first name**. You can find "Sharma, Asha" fast. You can find all "Sharma". But you cannot find all "Asha" quickly: the first names are scattered across every last name. A **composite index** (an index on several columns) works exactly like this. Column order is the whole game.

Words you need

- **Composite index**: one index on two or more columns, `ON big(customer_id, amount)`.
- **Leftmost prefix rule**: a composite index helps filters on its first column, or first and second together, but not on the second column alone.
- **Covering index**: an index holding every column the query needs, so the database never reads the table itself (SQLite: `COVERING INDEX`; PostgreSQL: *Index Only Scan*).
- **Partial index**: an index on only the rows that match a condition.
- **Expression index**: an index on a calculated value, such as `lower(email)`.

Worked examples

Problem 10.2.1 Create `idx_big_cust_amt` on `(customer_id, amount)`. Which of these queries can use it?

```
CREATE INDEX idx_big_cust_amt ON big(customer_id, amount);
-- Q1
EXPLAIN QUERY PLAN SELECT * FROM big WHERE amount = 500;
-- Q2
EXPLAIN QUERY PLAN SELECT * FROM big WHERE customer_id = 42 AND amount > 900;
```

```
Q1:  `--SCAN big
Q2:  `--SEARCH big USING INDEX idx_big_cust_amt (customer_id=? AND amount>?)
```

- **Step 1** Q1 filters only on `amount`, the second column. By the leftmost prefix rule the index is unusable, so the result is a full scan.
- **Step 2** Q2 filters on the first column with `=` and the second with `>`. Both parts of the index are used.
- **Answer** Q2 uses the index, Q1 does not. The phone-book rule: you cannot look up by first name alone.
- **Check** Put the column you filter with `=` first, then the range column. For `WHERE customer_id = ? AND amount > ?` the order `(customer_id, amount)` is right. If you also filter on `amount` alone often, create a separate index on `amount`.

Problem 10.2.2 (covering) We need only `customer_id` and `amount` for customer 42. With the single-column index the planner must jump to the table for each row. With the composite index the answer is entirely inside the index.

```
EXPLAIN QUERY PLAN SELECT customer_id, amount FROM big WHERE customer_id = 42;
```

```
`--SEARCH big USING COVERING INDEX idx_big_cust_amt (customer_id=?)
```

`COVERING INDEX` says the table was never touched. A timing for 4,000 rows (customers 100 to 199) with `SELECT *` against selecting only the two covered columns:

```
SELECT *       : 2.645 ms
2 cols, covered: 0.918 ms
```

- **Compute** 2.645 / 0.918 is about 2.9.
- **Answer** Asking for only the needed columns, from a covering index, was about 3 times faster.
- **Meaning** `SELECT *` has a hidden cost: it blocks covering indexes and moves more data over the network. In PostgreSQL you can add non-key columns with `CREATE INDEX ... ON t(a) INCLUDE (b)` (**not run here**).

Problem 10.2.3 (partial index) Refunds are 10% of rows and we always look up refunds by customer. Index only those rows.

```
CREATE INDEX idx_refunded ON big(customer_id) WHERE status = 'refunded';
EXPLAIN QUERY PLAN SELECT * FROM big WHERE status = 'refunded' AND customer_id = 40;
```

```
`--SEARCH big USING INDEX idx_refunded (customer_id=?)
```

- **Step 1** The index holds 20,000 entries, not 200,000. That is 10 times smaller (`ANALYZE` showed `20000 40` for it, see 10.3).
- **Step 2** The query repeats the index's condition (`status = 'refunded'`), so the planner knows the partial index is valid.
- **Answer** Smaller index, faster writes for the other 90% of rows, same fast lookup. Both SQLite and PostgreSQL support partial indexes.

Problem 10.2.4 (expression index) `WHERE lower(status) = 'paid'` cannot use a plain index on `status`, because the index stores `status`, not `lower(status)`. Create an index on the expression:

```
CREATE INDEX idx_lower_status ON big(lower(status));
EXPLAIN QUERY PLAN SELECT * FROM big WHERE lower(status) = 'paid' AND id < 0;
```

```
`--SEARCH big USING INDEX idx_lower_status (<expr>=? AND rowid<?)
```

(I added `id < 0` only so the planner would choose to use it; without it, `paid` matches 90% of rows and a scan is cheaper, as the earlier table predicts.) The query's expression must match the index's expression exactly.

COMMON MISTAKE	Creating `(a, b)` and expecting `WHERE b = ...` to be fast. Always check the plan with `EXPLAIN`. Also remember that a multi-column index is only as useful as its leftmost columns.

Connection to the real job

Choosing the order of columns in a composite index is a daily decision for backend and data engineers. The rule of thumb: equality columns first, range column last, and add `ORDER BY` columns next if you want to avoid a sort.

Applications

- **Data Analytics:** a covering index on `(date, product_id, amount)` makes a daily-sales dashboard read only the index.
- **Back ends:** partial index `WHERE status = 'pending'` keeps a job-queue table lookup tiny.
- **Data Engineering:** expression index on `lower(email)` for case-insensitive de-duplication.

Check yourself

1. For an index on `(a, b)`, does `WHERE b = 5` use it?

Answer: No. Only filters that include the first column `a` can use it.

1. What does a covering index avoid?

Answer: Reading the table rows at all, since the index has every needed column.

1. Why must `WHERE lower(email) = ...` match an expression index exactly?

Answer: The index stores the computed value, so only the same expression can search it.

10.3 Reading EXPLAIN and EXPLAIN ANALYZE

INTUITION

A GPS can tell you its planned route before you drive: "highway, then city road, 22 minutes." That is `EXPLAIN`. After the trip, it can tell you how long each leg really took. That is `EXPLAIN ANALYZE`. A plan is the database's chosen route. When a query is slow, you read the plan to find which step is the slow leg, just as you read a delivery report to find the slowest leg of a journey.

Words you need

- **Query plan**: the tree of steps the database will run.
- **Node**: one step (scan a table, sort, join).
- **Estimated rows vs actual rows**: what the planner guessed vs what really happened. A big gap means bad **statistics**.
- **Statistics**: the database's summary of each column (row count, distinct values, common values), refreshed by `ANALYZE`.
- **Cost**: the planner's unit-less score for how expensive a step is; compare, do not trust as seconds.

The commands

Command	Does it run the query?	Use
`EXPLAIN SELECT ...` (PostgreSQL)	no	see the plan, with estimates
`EXPLAIN ANALYZE SELECT ...` (PostgreSQL)	**yes**	real times and actual row counts
`EXPLAIN QUERY PLAN SELECT ...` (SQLite)	no	a short plan summary

COMMON MISTAKE

Running `EXPLAIN ANALYZE` on an `UPDATE` or `DELETE` in production. It really executes the statement. Wrap it: `BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;`.

What to read in a PostgreSQL plan (not run here)

I do not show a made-up PostgreSQL plan with numbers. Here is what each part of a real `EXPLAIN ANALYZE` line means, using the standard shape:

```
Index Scan using idx_big_cust on big  (cost=0.42..48.9 rows=40 width=32)
                                      (actual time=0.02..0.05 rows=40 loops=1)
  Index Cond: (customer_id = 42)
Planning Time: ...
Execution Time: ...
```

(The `...` mean "your numbers.")

Piece	Meaning
node name (`Seq Scan`, `Index Scan`, `Index Only Scan`, `Bitmap Heap Scan`)	how rows are fetched
`cost=a..b`	startup cost .. total cost (estimate)

Piece	Meaning
`rows=` (estimated) vs `actual ... rows=`	planner guess vs reality: compare them first
`loops=`	how many times this node ran; multiply actual time by loops
`Filter:` / `Rows Removed by Filter:`	rows read and thrown away: large numbers mean a missing index
`Buffers:` (with `EXPLAIN (ANALYZE, BUFFERS)`)	pages read from cache vs disk
`Sort Method: external merge Disk`	the sort spilled to disk: more `work_mem` or an index

Read the plan **from the innermost, most-indented node outward**, and ask three questions: where is the time, where is the estimate wrong, and are many rows read to keep few?

Join algorithms

Algorithm	How it works	Good when
Nested loop	for each row on the left, look up matches on the right	left side is small and the right side has an index
Hash join	build a hash table from the smaller input, probe with the other	large inputs, equality joins, no index
Merge join	sort both sides and walk them together	both inputs already sorted (for example by an index)

PostgreSQL chooses among all three. SQLite uses nested loops, creating a temporary index when helpful. A hash or merge join in PostgreSQL is **not run here**.

Worked examples in SQLite

Problem 10.3.1 Join customers in Pune to their orders. Predict first: SQLite loops over customers filtered by city, and for each, looks for orders. Without an index on `orders.customer_id` it has to scan orders; with one, it can search.

```
EXPLAIN QUERY PLAN
SELECT c.name, o.order_id
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.city = 'Pune';
```

```
QUERY PLAN
|--SCAN c
`--SCAN o
```

Now index the foreign key and ask again:

```
CREATE INDEX idx_orders_cust ON orders(customer_id);
```

```
QUERY PLAN
|--SCAN c
`--SEARCH o USING COVERING INDEX idx_orders_cust (customer_id=?)
```

- **Step 1** The outer loop `SCAN c` reads 8 customers (tiny; an index is pointless here).
- **Step 2** The inner loop was `SCAN o` (14 orders, once per customer). With the index it becomes `SEARCH o`, a jump per customer.
- **Answer** Index the foreign key column on the child side. With 8 and 14 rows it makes no visible difference; with 5 million orders it is the difference between milliseconds and minutes.
- **Why it matters** A join is a loop inside a loop. The cost of the inner lookup is multiplied by the number of outer rows.

Problem 10.3.2 (sorting) Does `ORDER BY` cost anything?

```
EXPLAIN QUERY PLAN SELECT * FROM big WHERE customer_id = 42 ORDER BY amount;
```

```
|--SEARCH big USING INDEX idx_big_cust (customer_id=?)
`--USE TEMP B-TREE FOR ORDER BY
```

- **Step 1** The planner found the 40 rows via the index.
- **Step 2** It then sorts them in a temporary structure (`USE TEMP B-TREE FOR ORDER BY`).
- **Answer** Sorting 40 rows is cheap. Sorting 200,000 (`SELECT * FROM big ORDER BY amount` , which gives `SCAN big` plus the temp sort) is not. The composite index `(customer_id, amount)` from 10.2 would return the 40 rows already sorted and remove the sort step.

Statistics, ANALYZE and VACUUM

The planner guesses how many rows each filter keeps, using statistics. If the statistics are old (you just loaded 10 million rows), the guess is wrong and it picks a bad plan. `ANALYZE` refreshes them:

```
ANALYZE;
SELECT tbl, idx, stat FROM sqlite_stat1 WHERE tbl = 'big';
```

```
tbl  idx              stat
---  ---------------  -------------
big  idx_lower_status  200000 100000
big  idx_refunded      20000 40
big  idx_big_cust      200000 40
big  idx_created       200000 548
```

How to read `stat` : the first number is the number of rows in the index; the second is the average number of rows per key value. `idx_big_cust` has 40 rows per customer: the planner now *knows* a customer lookup is very selective. `idx_lower_status` has 100,000 rows per value (two values only), so the planner knows it is nearly useless. `idx_created` averages 548 rows per date.

In PostgreSQL, `autovacuum` runs `ANALYZE` and `VACUUM` in the background. `VACUUM` reclaims space from dead row versions left behind by `UPDATE` and `DELETE` (see MVCC in Chapter 9). After a big load, run `ANALYZE table;` yourself (**not run here**).

Connection to the real job

When a query suddenly gets slower with no code change, the first suspects are stale statistics, data growth that tipped the plan from index to scan, and bloat. Reading a plan is how you tell which.

Applications

- **Data Engineering:** `ANALYZE` after bulk loads; check plans of slow pipeline steps.
- **Data Analytics:** compare plans before and after rewriting a dashboard query.
- **Operations:** capture `EXPLAIN (ANALYZE, BUFFERS)` of the slowest query found in the monitoring dashboard and attach it to the ticket.

Check yourself

1. What is the difference between `EXPLAIN` and `EXPLAIN ANALYZE` in PostgreSQL?

Answer: `EXPLAIN` only shows the plan; `EXPLAIN ANALYZE` runs the query and adds real times and row counts.

1. If estimated rows is 10 and actual rows is 2,000,000, what is likely wrong?

Answer: Stale or missing statistics; run `ANALYZE` .

1. Which join algorithm suits a small outer table and an indexed inner table?

Answer: Nested loop.

10.4 Query Optimization: A Measure-First Method

INTUITION

When a bank queue is too slow you never "just make it faster." You time the current queue (the baseline), find the slow step, change one thing, and time it again. Query tuning works the same way. The database is the counter, the query is the customer request, and `EXPLAIN` shows where the time goes.

The method

1. **Measure** the slow query on realistic data volume (not 100 rows).
2. **Read the plan.** Find the node that reads many rows or takes the time.
3. **Change one thing**: add an index, rewrite the predicate, select fewer columns.
4. **Measure again**, compare to baseline in percent or as a ratio.
5. **Check the write side** and other queries before keeping a new index.

Sargable predicates

Sargable means "Search ARGument ABLE": the filter is written so an index can be used. The rule: **keep the indexed column bare on one side of the comparison.** Wrapping it in a function hides it from the index.

Not sargable (index ignored)	Sargable rewrite
`WHERE substr(created,1,10) = '2024-03-01'`	`WHERE created >= '2024-03-01' AND created < '2024-03-02'`
`WHERE amount + 10 = 510`	`WHERE amount = 500`
`WHERE created LIKE '%03%'`	(leading wildcard: no B-tree help; use full-text or trigram index in PostgreSQL)
`WHERE lower(email) = 'a@x.com'`	add an expression index, or store lowercase

Problem 10.4.1 Count the rows for one date. Compare the function version with the range version, with `idx_created` on `created`.

```
-- slow shape
SELECT * FROM big WHERE substr(created,1,10) = '2024-03-01';
-- fast shape
SELECT * FROM big WHERE created >= '2024-03-01' AND created < '2024-03-02';
```

```
one day, function on column: 10.966 ms
one day, range             : 0.682 ms
548 rows
```

- **Step 1** The function version computes `substr(...)` on all 200,000 rows, so the plan is `SCAN big` (we saw this earlier: it prints `SCAN big` even though `idx_created` exists).
- **Step 2** The range version asks the index directly: `SEARCH big USING INDEX idx_created (created>? AND created<?)`.
- **Compute** 10.966 / 0.682 is about 16.
- **Answer** Same 548 rows, about 16 times faster, with a rewrite and no new index.
- **Check** 200,000 rows / 365 dates = about 548 rows per date. It matches the `ANALYZE` statistic above.

COMMON MISTAKE

Assuming a bigger result means a bigger win. I also ran the range for a whole month (about 16,000 rows) and it took 23.6 ms, *slower* than the 20 ms scan. When the filter keeps a large share of rows, jumping around through an index loses to reading straight through the table. Always test with the real ranges your users use.

COMMON MISTAKE

`LIKE '2024-03%'` looks like a prefix search that a B-tree should help. In SQLite it still printed `SCAN big`, because by default SQLite's `LIKE` ignores case and the index uses case-sensitive order. In PostgreSQL the index may need `text_pattern_ops` or a "C" collation. The lesson: read the plan, never assume.

Other rewrites that matter

Habit	Better	Why
`SELECT *`	list the needed columns	less data moved; allows covering indexes
`OFFSET 100000 LIMIT 20`	`WHERE id > :last_id ORDER BY id LIMIT 20`	OFFSET still reads and drops the skipped rows (keyset pagination)
`IN (SELECT ...)` on large sets, or correlated subquery in `SELECT`	join or `EXISTS` (Chapter 5)	planner can pick a better strategy
`COUNT(*)` on a huge table for "does any row exist?"	`EXISTS (SELECT 1 ... LIMIT 1)`	stops at the first match
Many tiny queries in an app loop	one set-based query	each round trip has fixed latency
Filtering after a join	filter before it (`WHERE` on the base table)	smaller join input

Problem 10.4.2 (the N+1 problem) An app needs the total `amount` for customers 0 to 4,999. Version A runs one query per customer (5,000 queries). Version B runs one query.

```
-- B
SELECT sum(amount) FROM big WHERE customer_id < 5000;
```

```
99900000 99900000 N+1: 0.090 s, one query: 0.036 s
```

- **Step 1** Both versions give the same total, 99,900,000.
- **Compute** 0.090 / 0.036 = 2.5 times faster in a single process on one machine, using an index each time.
- **Answer** The set-based query is faster even here. Over a real network each of the 5,000 round trips adds latency (say 1 ms each means 5 s extra), so the gap is far larger in production.
- **Meaning** In practice this appears as one user action producing hundreds of SQL calls. Count the database calls per request; that is often the whole bottleneck.
- **Check** Sum of both totals match, so the rewrite is correct.

Statistics-driven decisions and limits

- **Join order**: the planner chooses it from statistics. You rarely force it; fix the statistics or the indexes instead.
- **Pre-aggregate**: for dashboards, compute daily totals once into a summary table or materialized view, rather than summing 100 million rows each time (Chapter 11).
- **Partitioning, sharding, replication, connection pooling, backup** are scaling tools: partitioning splits one big table by range (for example one partition per month) so scans can skip old months; sharding splits across servers; replication copies data to read-only replicas for reports; connection pooling (PgBouncer) reuses a few database connections for many clients, which matters because each PostgreSQL connection is a process. Backup is `pg_dump` (logical) or file-level base backups with WAL archiving (physical). All **not run here**. Learn them after the basics in this chapter feel easy.
- **Limits**: tuning fixes bad queries, not bad design. If a query must join ten tables to answer a simple question, revisit the schema (Chapter 11).

Connection to the real job

Analytics interviews ask "this query takes 10 minutes: what do you do?" The answer is the five-step method above, with these checks in order: plan, row counts, sargable filter, index, fewer columns, pre-aggregation. The skill-map goal "optimize a slow query from 10 s to under 1 s with `EXPLAIN ANALYZE`" is exactly this loop.

Applications

- **Data Analytics:** rewrite non-sargable date filters in dashboards.
- **Data Engineering:** replace row-by-row loops with set-based SQL in pipelines.
- **ML:** speed up feature extraction queries; build training sets from pre-aggregated tables.
- **Operations:** report database calls per request and the slowest plan node in the monitoring report.

Check yourself

1. Why is `WHERE year(created) = 2024` slow on an indexed column?

Answer: The function hides the column from the index, so every row is computed; rewrite as a date range.

1. What is the N+1 problem?

Answer: One query to get a list, then one more query per item, so N+1 calls instead of one.

1. Why can an index make a query slower?

Answer: If the filter keeps a large share of rows, jumping through the index costs more than a straight scan.

10.5 Performance Lab: Sargability, Pagination and Statistics You Can Measure

INTUITION

A phone book is sorted by surname. Finding 'Rao' takes seconds because you can jump. Finding everyone whose surname, *spelled backwards*, starts with 'oa' takes hours, because the sorting no longer helps and you must read every line. A query is **sargable** (Search ARGument ABLE) when its `WHERE` test is written so that an index can jump. This section makes you measure the difference instead of trusting it.

To measure work without depending on a fast or slow laptop, the lab counts **SQLite virtual-machine steps**: every query is compiled into small instructions, and we count how many run. Fewer steps means less work. The exact counts depend on your SQLite version (this run used 3.50.4), so compare the ratios, not the digits. PostgreSQL shows the same ideas through `EXPLAIN ANALYZE` (not run here).

Sargability: five rewrites that decide whether an index is used

The rule: an index stores the *column value*, so the test must compare the bare column with something. Wrap the column in a function or arithmetic and the database cannot use the sorted order.

Slow form (index ignored)	Fast form (index usable)	Why
`WHERE substr(created,1,7) = '2024-03'`	`WHERE created >= '2024-03-01' AND created < '2024-04-01'`	a date range is a contiguous slice of the sorted index
`WHERE lower(email) = 'a@x.com'`	expression index on `lower(email)`, same query	the index stores exactly what the query compares
`WHERE customer_id + 1 = 43`	`WHERE customer_id = 42`	move arithmetic to the constant side
`WHERE name LIKE '%son'`	`WHERE name LIKE 'son%'`, or a trigram index (PostgreSQL `pg_trgm`)	a B-tree can only follow a known prefix

Slow form (index ignored)	Fast form (index usable)	Why
`WHERE a = 1 OR b = 2`	`UNION ALL` of two queries, or one index on each column (PostgreSQL combines them with a bitmap OR)	a single B-tree cannot answer two different columns at once

Problem 10.5.1 Run the first three rewrites on a 50,000-row table and compare steps. Predict first: the slow forms touch all 50,000 rows (about 200,000 steps, four per row), the fast forms touch a small slice.

```
import sqlite3, random
random.seed(7)
db = sqlite3.connect(":memory:", isolation_level=None)
db.execute("CREATE TABLE big (id INTEGER PRIMARY KEY, customer_id INT, created TEXT, email TEXT, amount INT)")
rows = [(i, random.randint(1, 2000), "2024-%02d-%02d" % (random.randint(1, 12), random.randint(1, 28)),
         "user%d@Mail.com" % i, random.randint(10, 999)) for i in range(1, 50001)]
db.executemany("INSERT INTO big VALUES (?,?,?,?,?)", rows)

def cost(q, args=()):
    """count SQLite virtual-machine steps: a stable stand-in for 'work done'"""
    n = [0]
    def tick():
        n[0] += 1
        return 0
    db.set_progress_handler(tick, 1)
    db.execute(q, args).fetchall()
    db.set_progress_handler(None, 0)
    return n[0]

def plan(q):
    return db.execute("EXPLAIN QUERY PLAN " + q).fetchall()[0][3]

tests = [
  ("function on column", "SELECT count(*) FROM big WHERE substr(created, 1, 7) = '2024-03'",
                         "SELECT count(*) FROM big WHERE created >= '2024-03-01' AND created < '2024-04-01'", None),
  ("lower() on column",  "SELECT count(*) FROM big WHERE lower(email) = 'user77@mail.com'",
                         "SELECT count(*) FROM big WHERE lower(email) = 'user77@mail.com'",
                         "CREATE INDEX i_lemail ON big(lower(email))"),
  ("arithmetic on column", "SELECT count(*) FROM big WHERE customer_id + 1 = 43",
                         "SELECT count(*) FROM big WHERE customer_id = 42", None),
]
db.execute("CREATE INDEX i_created ON big(created)")
db.execute("CREATE INDEX i_cust ON big(customer_id)")
for name, bad, good, new_index in tests:
    print(name)
    print("  slow :", cost(bad), "steps |", plan(bad))
    if new_index:
        db.execute(new_index)
        print("  (created the expression index)")
    print("  fast :", cost(good), "steps |", plan(good))
```

```
function on column
  slow : 204116 steps | SCAN big USING COVERING INDEX i_created
  fast : 12322 steps | SEARCH big USING COVERING INDEX i_created (created>? AND created<?)
lower() on column
  slow : 200013 steps | SCAN big
  (created the expression index)
  fast : 18 steps | SEARCH big USING COVERING INDEX i_lemail (<expr>=?)
arithmetic on column
  slow : 200031 steps | SCAN big USING COVERING INDEX i_cust
  fast : 64 steps | SEARCH big USING COVERING INDEX i_cust (customer_id=?)
```

Step 1	The first pair filters by month. `substr(created,1,7)` is a function of the column, so the plan says `SCAN` and every row is read. The range form says `SEARCH ... (created>? AND created<?)`: the index jumps to 2024-03-01 and stops at 2024-04-01. Steps fall from 204,116 to 12,322, about 17 times fewer.
Step 2	The second pair uses the same query text. The only change is the new expression index on `lower(email)`, which stores the lowercase value. The scan (200,013 steps) becomes a search (18 steps). You changed the index, not the query.
Step 3	The third pair moves `+ 1` to the other side of the equals sign: 200,030 steps become 64.
Answer	Keep the column bare on one side of the comparison. If you cannot, index the expression.

COMMON MISTAKE	Writing a date filter as `WHERE DATE(created_at) = '2024-03-15'` or `WHERE EXTRACT(YEAR FROM created_at) = 2024` on a large table. It works and returns correct rows, so nothing warns you, yet it reads every row. Right: `WHERE created_at >= '2024-03-15' AND created_at < '2024-03-16'`. A second trap is a type mismatch, such as comparing a text column to a number (`WHERE phone = 5551234`): the database may convert every stored value and skip the index.

Problem 10.5.2 (find the slow part) A teammate reports that `SELECT * FROM big WHERE amount * 1.1 > 500` is slow even though `amount` is indexed. Fix it. Predict first: the multiplication is on the column side.

Idea	Divide the constant instead: `amount * 1.1 > 500` is the same as `amount > 500 / 1.1`.
Rewrite	`WHERE amount > 454.55` (500 / 1.1 = 454.545..., so use 454.55 for integers: `amount >= 455` because 455 x 1.1 = 500.5 is above 500 and 454 x 1.1 = 499.4 is not).
Check	Integer amounts 455 and up pass the original test, 454 and below fail. Both forms return the same rows, and the second can use an index on `amount`.

Pagination: OFFSET gets slower with every page, keyset does not

A web page shows 20 rows at a time. The obvious query is `ORDER BY created, id LIMIT 20 OFFSET 9980`. The database cannot skip 9,980 rows without walking past them, so page 500 costs 500 times what page 1 costs. **Keyset pagination** (also called the seek method) remembers the last row of the previous page and asks for rows after it, so the index jumps straight there.

Problem 10.5.3 Fetch page 1, page 100 and page 500 both ways. Predict first: OFFSET cost grows in proportion to the page number, keyset cost stays about the same.

```
import sqlite3, random
random.seed(7)
db = sqlite3.connect(":memory:", isolation_level=None)
db.execute("CREATE TABLE big (id INTEGER PRIMARY KEY, customer_id INT, created TEXT, email TEXT, amount INT)")
rows = [(i, random.randint(1, 2000), "2024-%02d-%02d" % (random.randint(1, 12), random.randint(1, 28)),
         "user%d@Mail.com" % i, random.randint(10, 999)) for i in range(1, 50001)]
db.executemany("INSERT INTO big VALUES (?,?,?,?,?)", rows)

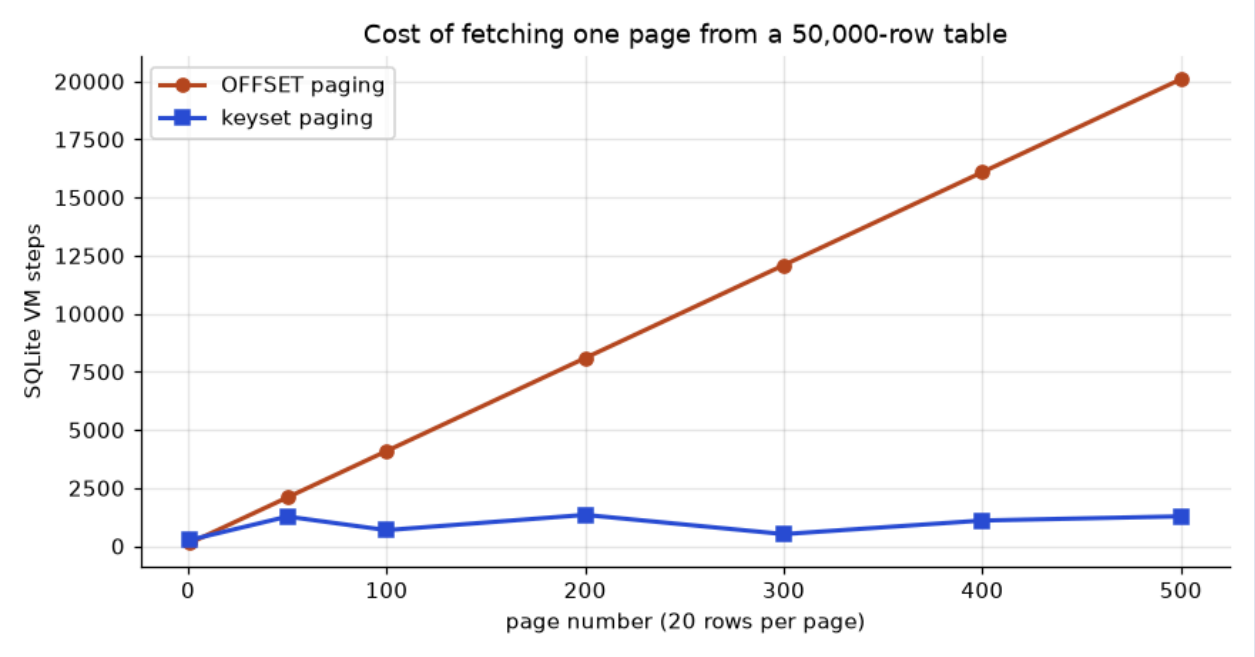
def cost(q, args=()):
    """count SQLite virtual-machine steps: a stable stand-in for 'work done'"""
    n = [0]
    def tick():
        n[0] += 1
        return 0
    db.set_progress_handler(tick, 1)
    db.execute(q, args).fetchall()
    db.set_progress_handler(None, 0)
    return n[0]

def plan(q):
    return db.execute("EXPLAIN QUERY PLAN " + q).fetchall()[0][3]

db.execute("CREATE INDEX i_ci ON big(created, id)")
for page in (1, 100, 500):
    off = (page - 1) * 20
    a = cost("SELECT id, created FROM big ORDER BY created, id LIMIT 20 OFFSET ?", (off,))
    last = db.execute("SELECT created, id FROM big ORDER BY created, id LIMIT 1 OFFSET ?", (max(off - 1, 0),)).fetchone()
    k = cost("SELECT id, created FROM big WHERE (created, id) > (?, ?) ORDER BY created, id LIMIT 20", last)
    print("page %3d  OFFSET: %6d steps   keyset: %5d steps" % (page, a, k))
```

```
page   1  OFFSET:    130 steps   keyset:   263 steps
page 100  OFFSET:   4090 steps   keyset:   765 steps
page 500  OFFSET:  20090 steps   keyset:   738 steps
```

Step 1	OFFSET steps grow by about 4,000 per 100 pages: 130, 4,090, 20,090. That is 200 steps per page of skipped rows.
Step 2	The keyset form uses `WHERE (created, id) > (last_created, last_id)`, a row-value comparison that the index on `(created, id)` answers by seeking. Its cost stays under about 1,400 steps whatever the page (it is not perfectly flat here because many rows share one date). On page 1 it is slightly more expensive, since there is no previous row to seek from.
Answer	Use keyset for feeds and infinite scroll. Keep OFFSET for small tables or when users must jump to page 37.



Work needed to fetch one page of 20 rows from a 50,000-row table, measured on SQLite 3.50.4.

How to read it: the orange line is OFFSET paging: it climbs in a straight line because each deeper page skips more rows. The blue line is keyset paging: it wobbles but never climbs. The vertical gap at page 500 is the price of skipping 9,980 rows.

COMMON MISTAKE

Keyset pagination on a column that is not unique. If you page by `created` alone and ten rows share one timestamp, rows can be skipped or repeated at the page boundary. Always add a unique tie-breaker (`created, id`) to both the `ORDER BY` and the `WHERE` . Also, row-value comparison `(a, b) > (x, y)` works in PostgreSQL, MySQL and SQLite but not in SQL Server, where you write `a > x OR (a = x AND b > y)` .

Statistics: why the planner sometimes guesses wrong

Before running a query the planner estimates how many rows each step returns, using **statistics** collected by `ANALYZE` . If the estimate is far from reality, it picks a bad plan: it scans when it should seek, or the other way round. The classic cause is **skew**: 99 percent of rows share one value, and statistics that only keep an average cannot see that.

Problem 10.5.4 A table has 50,000 events, 99 percent with status `done` and 1 percent `failed`, with an index on `status`. Compare the plans for both values before and after `ANALYZE`. Predict first: both look the same to a planner that only knows the average.

```
import sqlite3
db = sqlite3.connect(":memory:", isolation_level=None)
db.execute("CREATE TABLE ev (id INTEGER PRIMARY KEY, status TEXT, v INT)")
db.executemany("INSERT INTO ev VALUES (?,?,?)", [(i, 'done' if i % 100 else 'failed', i) for i in range(1, 50001)])
db.execute("CREATE INDEX i_status ON ev(status)")

def run(label):
    for st in ("done", "failed"):
        q = "SELECT count(*), sum(v) FROM ev WHERE status = '%s'" % st
        n = [0]
        def tick():
            n[0] += 1
            return 0
        db.set_progress_handler(tick, 1)
        db.execute(q).fetchall()
        db.set_progress_handler(None, 0)
        p = db.execute("EXPLAIN QUERY PLAN " + q).fetchall()[0][3]
        print(label, st.ljust(6), str(n[0]).rjust(7), "steps |", p)

run("before ANALYZE:")
db.execute("ANALYZE")
print(db.execute("SELECT tbl, idx, stat FROM sqlite_stat1").fetchall())
run("after  ANALYZE:")
```

```
before ANALYZE: done    297014 steps | SEARCH ev USING INDEX i_status (status=?)
before ANALYZE: failed    3013 steps | SEARCH ev USING INDEX i_status (status=?)
[('ev', 'i_status', '50000 25000')]
after  ANALYZE: done    298513 steps | SCAN ev
after  ANALYZE: failed  151513 steps | SCAN ev
```

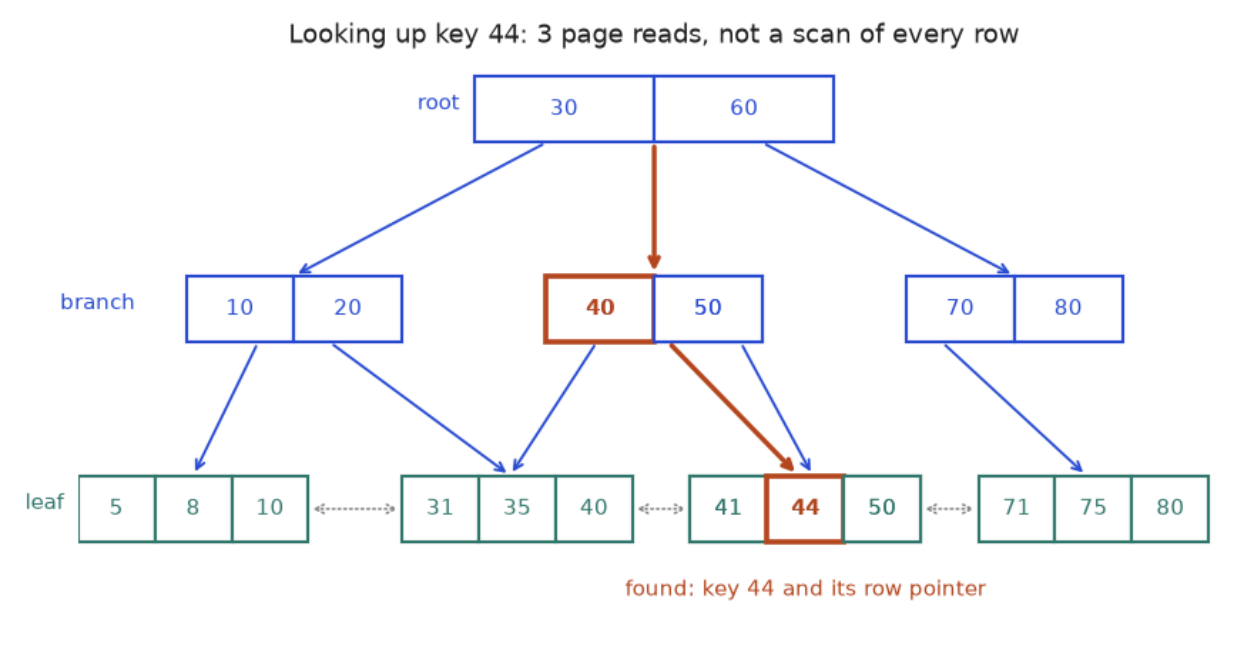
Step 1	Before `ANALYZE` SQLite has no statistics and uses the index for both values. For `failed` (500 rows) that is right: 3,013 steps. For `done` (49,500 rows) it is wasteful, 297,014 steps, because it hops from index to table 49,500 times.
Step 2	`ANALYZE` stores `'50000 25000'` : 50,000 rows, on average 25,000 rows per status value. Two values, so the average is exactly half the table.
Step 3	With that average the planner decides an index is not worth it and scans for *both*. That helps `done` slightly (298,513 steps, about the same) but makes `failed` 50 times worse (151,513 vs 3,013).
Answer	Statistics are summaries, and summaries hide skew. PostgreSQL keeps a list of most common values and a histogram, so it knows that `failed` is rare; SQLite builds with STAT4 do something similar. Always compare estimated and actual row counts in `EXPLAIN ANALYZE` (not run here); a 10x gap is your signal.

GOING DEEPER	PostgreSQL fixes a bad estimate in this order: (1) run `ANALYZE table` because statistics may simply be old, (2) raise the detail with `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000` , (3) add extended statistics for correlated columns (`CREATE STATISTICS s (dependencies) ON city, zip FROM t`), because the planner multiplies the selectivities of two columns as if they were independent, and city and zip are not, (4) as a last resort rewrite the query. The *autovacuum* daemon runs `ANALYZE` automatically, but after a bulk load you should run it yourself. None of these commands were run here.

10.6 Index Types, Join Algorithms and Partitioning

INTUITION

So far an index meant a B-tree: a sorted tree. Think of it as the index at the back of a textbook, sorted alphabetically. Other books need other indexes: a map index (find everything near this place), a keyword index for chapters that mention several words, a table of contents that only says which *block of pages* to look in. PostgreSQL offers each of these as an index type.



A B-tree lookup of key 44 in a tiny tree: root, branch, leaf.

How to read it: start at the root and follow the arrow whose range contains 44 (between 30 and 60). The branch sends you between 40 and 50, and the leaf holds 44 with a pointer to the table row. That is three page reads. A real tree holds hundreds of keys per page, so even a billion-row table is only four or five levels deep. The dotted arrows between leaves are why range queries (`BETWEEN` , `ORDER BY`) are fast: after finding the start, walk sideways.

Index types at a glance (PostgreSQL, not run here)

Type	Best for	Does not help with
B-tree (default)	equality, ranges, `ORDER BY` , prefix `LIKE 'abc%'`	searching inside arrays or text, or the last letters of a string
Hash	equality only, small and fast keys	ranges, sorting (rarely better than B-tree)
GIN	many values per row: arrays, `jsonb` , full-text search, trigrams	range sorting; writes are slower
GiST	geometry, nearest-neighbour, ranges overlap (`&&`), exclusion constraints	exact equality on plain numbers
BRIN	huge tables whose physical order follows the column (logs by timestamp); the index is tiny	randomly ordered columns
Partial / expression	only some rows, or a computed value (Section 10.2)	queries that do not repeat the same condition

Problem 10.6.1 Pick the index for each: (a) `WHERE tags @> ARRAY['sale']` on a text-array column, (b) 2 billion sensor rows appended in time order, queried by time window, (c) `WHERE email = ?`, (d) find the 10 nearest shops to a point. Predict first: four different answers.

(a)	GIN: it stores one entry per array element, so containment search is direct.
(b)	BRIN: it stores the min and max timestamp per block of pages, a few megabytes for terabytes of data. It only works because rows arrive in time order.
(c)	B-tree (hash would work for pure equality, but B-tree is the safe default and supports uniqueness).
(d)	GiST (or SP-GiST): it supports distance ordering with `ORDER BY location <-> point LIMIT 10`.

Join algorithms, measured

Section 10.3 listed the three join algorithms. SQLite uses nested loops, so we can measure one. We join 2,000 customers to 50,000 orders and keep customers 1 to 50:

```
import sqlite3, random
random.seed(7)
db = sqlite3.connect(":memory:", isolation_level=None)
db.execute("CREATE TABLE cust (customer_id INTEGER PRIMARY KEY, name TEXT)")
db.executemany("INSERT INTO cust VALUES (?,?)", [(i, "c%d" % i) for i in range(1, 2001)])
db.execute("CREATE TABLE big (id INTEGER PRIMARY KEY, customer_id INT, created TEXT, email TEXT, amount INT)")
rows = [(i, random.randint(1, 2000), "2024-%02d-%02d" % (random.randint(1, 12), random.randint(1, 28)),
         "user%d@Mail.com" % i, random.randint(10, 999)) for i in range(1, 50001)]
db.executemany("INSERT INTO big VALUES (?,?,?,?,?)", rows)


def cost(q):
    n = [0]
    def tick():
        n[0] += 1
        return 0
    db.set_progress_handler(tick, 1)
    db.execute(q).fetchall()
    db.set_progress_handler(None, 0)
    return n[0]

q = "SELECT count(*) FROM cust c JOIN big b ON b.customer_id = c.customer_id WHERE c.customer_id <= 50"
def plans(q):
    return " ; ".join(r[3] for r in db.execute("EXPLAIN QUERY PLAN " + q).fetchall())
print("no index   :", cost(q), "steps |", plans(q))
db.execute("CREATE INDEX i_cust ON big(customer_id)")
print("with index :", cost(q), "steps |", plans(q))
```

```
no index   : 251242 steps | SCAN b ; SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
with index : 8613 steps | SEARCH b USING COVERING INDEX i_cust (customer_id<?) ; SEARCH c USING INTEGER PRIMARY KEY
(rowid=?)
```

Step 1	Without an index on `big.customer_id` the plan reads: `SCAN b` (all 50,000 orders, outer loop) then `SEARCH c` by primary key for each one. SQLite picked the big table as the outer loop because that is the cheapest order it could find. Cost: 251,242 steps.
Step 2	With the index, the plan flips: `SEARCH b USING COVERING INDEX i_cust (customer_id<?)` reads only the orders of customers 1 to 50, then looks each customer up. Cost: 8,613 steps, about 29 times less.
Step 3	Notice that the planner also chose a different join *order*. Join order is part of the plan and it changes when indexes change.
Answer	A nested loop is fast exactly when the inner side can be probed with an index (here the primary key of `c`). A hash join (PostgreSQL) instead builds a hash table of the smaller input once and probes it, which is why it wins for big unindexed equality joins. A merge join wins when both inputs already arrive sorted. You rarely choose the algorithm; you make the right index or statistics exist and read the plan.

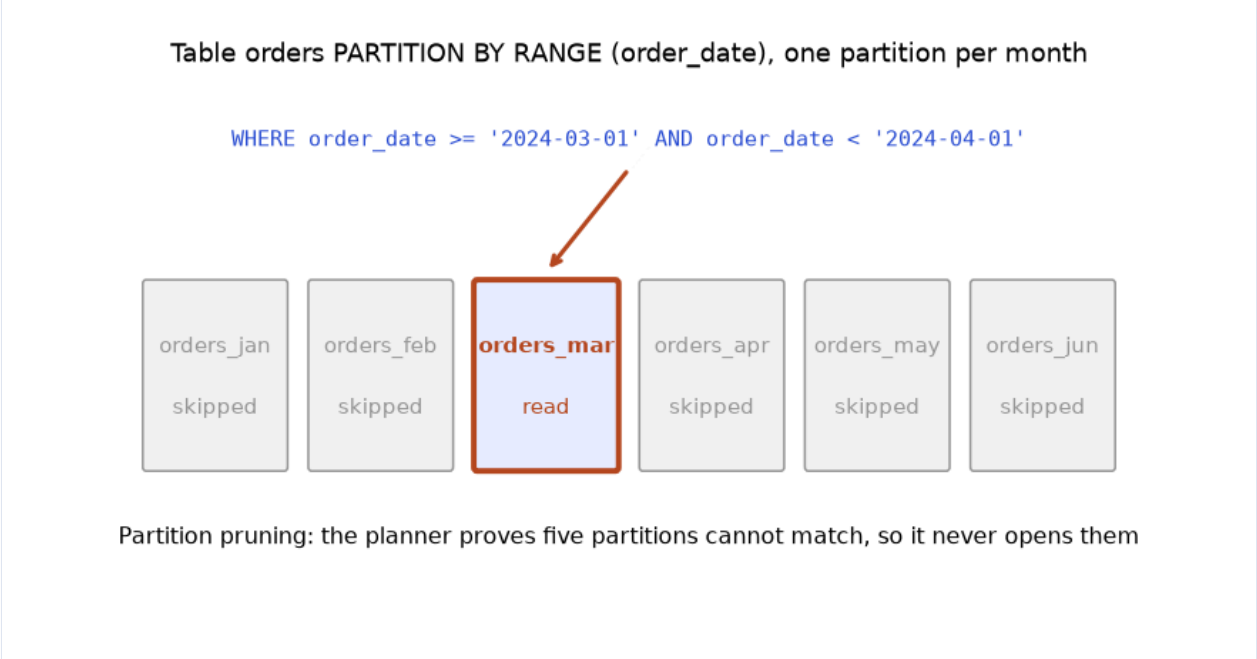
Partitioning: splitting one big table into many

Partitioning splits a table into physical pieces by a rule, usually a date range, while queries still address one table name. The pay-off is **partition pruning**: when the `WHERE` clause fixes the date, the planner skips whole pieces. A second pay-off is housekeeping: dropping an old month is `DROP TABLE orders_2023_01`, instant, instead of a huge `DELETE` that leaves dead rows.

```
-- PostgreSQL (not run here)
CREATE TABLE orders (
  order_id   bigint,
  order_date date NOT NULL,
  amount     numeric
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2024_03 PARTITION OF orders
  FOR VALUES FROM ('2024-03-01') TO ('2024-04-01');
CREATE TABLE orders_2024_04 PARTITION OF orders
  FOR VALUES FROM ('2024-04-01') TO ('2024-05-01');

-- retire old data without a giant DELETE
ALTER TABLE orders DETACH PARTITION orders_2024_03;
```



Partition pruning for a query that filters on March 2024.

How to read it: the filter at the top matches only the March box, so only that partition is opened. The other five are proven irrelevant from their boundaries, which is why `EXPLAIN` lists a single partition. If the filter did not mention `order_date`, every box would be read.

COMMON MISTAKE	Partitioning a table of 5 million rows 'for speed'. Partitioning helps when the table is large (hundreds of millions of rows, or too big to maintain) *and* most queries filter on the partition key. Otherwise it adds planning time and complexity. In PostgreSQL a primary key or unique index on a partitioned table must include the partition key, so `order_id` alone cannot be unique across partitions. Never partition just because the table is big; first check indexes and statistics.

N+1 and the slow-query log

The N+1 problem (Section 10.4) is the most common reason an application is slow with a fast database: one query lists 100 orders, then the code runs 100 more queries, one per order. The cure is one join or one `WHERE id IN (...)` query. To find such patterns in PostgreSQL, enable the `pg_stat_statements` extension (not run here): it adds up calls, total time and mean time per query shape, so a cheap query called 2 million times shows up at the top. Sort by `total_exec_time`, not by the slowest single query, because the biggest cost is often a million quick calls.

Practice with answers

Problem 10.6.2 A `WHERE status = 'open'` query is slow on a 300 million row table where 0.2 percent of rows are open. What index? *(medium)*

Answer: A partial index: `CREATE INDEX ON t (id) WHERE status = 'open'`. It holds only 0.2 percent of rows (600,000 entries), so it is tiny and fast to maintain. The query must repeat `status = 'open'` for the planner to use it.

Problem 10.6.3 What is wrong with `SELECT * FROM t ORDER BY created DESC LIMIT 20 OFFSET 100000` and what is the fix? *(medium)*

Answer: The database reads and discards 100,000 rows before returning 20. Fix: keyset pagination, `WHERE (created, id) < (:last_created, :last_id) ORDER BY created DESC, id DESC LIMIT 20`, backed by an index on `(created, id)`.

Problem 10.6.4 A query that was fast last month now takes minutes, and no code changed. List three causes in the order you would check them. *(interview)*

Answer: (1) Stale statistics after data growth or a bulk load: run `ANALYZE` and compare estimated with actual rows. (2) A plan that flipped because the data distribution changed (skew), seen by comparing old and new `EXPLAIN ANALYZE`. (3) Table or index bloat from many updates without vacuum, or a lock wait: check `pg_stat_activity` for blocked sessions before blaming the plan. Others: a missing index on a new join column, or the data growing beyond memory so a hash join spills to disk.

Problem 10.6.5 Why can a covering index make a query faster than any other index, and what is its cost? *(expert)*

Answer: If the index contains every column the query needs, the database never visits the table (an index-only scan), which avoids one random read per row. The costs are a larger index, slower writes (every insert and update of those columns also updates the index), and in PostgreSQL index-only scans still check the visibility map, so they work best on well-vacuumed, mostly static tables.

Chapter recap

- An index is a sorted structure (usually a B-tree) that lets the database jump to rows instead of scanning the table.
- Indexes speed reads and slow writes: in my test, four indexes made a 100,000-row load about 4 times slower.
- Do not index tiny tables, low-selectivity columns, or write-heavy tables with rare reads.
- Composite indexes follow the leftmost prefix rule: equality columns first, range last.
- A covering index (or `INCLUDE`) answers a query without reading the table; `SELECT *` prevents this.
- Partial and expression indexes keep the index small or support `lower(col)` style filters.
- `EXPLAIN` shows the plan; `EXPLAIN ANALYZE` runs it and shows real times. Compare estimated and actual rows, and watch for large "Rows Removed by Filter."
- Keep indexed columns bare in `WHERE` (sargable). Rewrite functions on columns into ranges.
- Fix N+1 loops with set-based queries; run `ANALYZE` after big changes.
- Measure first: baseline, one change, re-measure, check the side effects.

Cheat-sheet

Task	Pattern
Create index	`CREATE INDEX idx ON t(a, b);`
Partial index	`CREATE INDEX idx ON t(a) WHERE status = 'x';`
Expression index	`CREATE INDEX idx ON t(lower(email));`
Covering (PostgreSQL)	`CREATE INDEX idx ON t(a) INCLUDE (b);`
See plan	`EXPLAIN SELECT ...` / `EXPLAIN QUERY PLAN` (SQLite)
Real timing (PostgreSQL)	`EXPLAIN (ANALYZE, BUFFERS) SELECT ...`
Refresh statistics	`ANALYZE t;`

Task	Pattern
Sargable date filter	`col >= '2024-03-01' AND col < '2024-03-02'`
Keyset paging	`WHERE id > :last ORDER BY id LIMIT 20`

Practice set

Problem 10.P.1 (easy) Which is more selective: `WHERE gender = 'F'` or `WHERE email = 'a@x.com'`? Which deserves an index? **Answer:** The email filter (about one row). Index `email`. A two-value column has low selectivity.

Problem 10.P.2 (easy) What does `SEARCH big USING COVERING INDEX ...` tell you? **Answer:** The query was answered from the index alone, without reading the table.

Problem 10.P.3 (what is the output?) Table `t` has an index on `(a, b)`. Which queries can use it fully: `WHERE a=1 AND b=2`, `WHERE b=2`, `WHERE a=1`? **Answer:** The first and the third. The second (`b` alone) cannot.

Problem 10.P.4 (computation) A scan takes 8 ms. After an index the same query takes 0.05 ms. How many times faster, and how much database time is saved per second at 200 requests per second? **Answer:** 8 / 0.05 = 160 times faster. Saved per request 7.95 ms, so 200 × 7.95 = 1,590 ms per second of database time.

Problem 10.P.5 (find the bug) `WHERE amount + 10 = 510` is slow although `amount` is indexed. Why, and what is the fix? **Answer:** The arithmetic is on the column, so the index cannot be used. Write `WHERE amount = 500`.

Problem 10.P.6 (find the bug) A developer creates `CREATE INDEX i ON orders(status)` where 95% of rows are `paid`, and the planner ignores it. Explain. **Answer:** Low selectivity: filtering for `paid` keeps 95% of rows, so scanning the table is cheaper. The index may help for the rare values; use a partial index for those.

Problem 10.P.7 (computation) An index on a 100,000-row insert load adds 0.035 s per index in my test. Estimate extra time for 6 indexes loading 10 million rows, assuming linear scaling. **Answer:** 10,000,000 / 100,000 = 100 batches. 6 × 0.035 = 0.21 s per batch. 100 × 0.21 = 21 s extra. (A rough estimate only: real systems have disk and cache effects.)

Problem 10.P.8 (scenario) A dashboard query filters `created >= now() - 7 days`, groups by `product_id`, and sums `amount`. Propose an index. **Answer:** `CREATE INDEX ON sales(created, product_id, amount)`: the range column first, then the group and value columns, which also makes it covering. Check the plan; a pre-aggregated daily table may be even better.

Problem 10.P.9 (scenario) Page 1 of a list loads in 5 ms. Page 5,000 (`OFFSET 100000 LIMIT 20`) takes 800 ms. Why, and what is the fix? **Answer:** `OFFSET` reads and discards the first 100,000 rows each time. Use keyset pagination: `WHERE id > :last_seen_id ORDER BY id LIMIT 20`, with an index on `id`.

Problem 10.P.10 (interview) How do you debug a query that was fast last month and is slow today? **Answer:** Run `EXPLAIN (ANALYZE, BUFFERS)`. Compare estimated and actual rows (stale statistics, then `ANALYZE`). Check whether the plan flipped from an index to a scan because the table grew. Check lock waits and bloat (`VACUUM`). Check for a new code path with a non-sargable filter or an N+1 loop. Change one thing at a time and measure.

Problem 10.P.11 (interview) You added an index and one query got faster but the nightly load got slower. What do you do? **Answer:** Each index adds write cost. Measure both sides. Options: drop unused indexes (look at index usage statistics), make the index partial, drop and recreate it around the bulk load, or accept the trade if the read gain is larger.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional learning SQL for data roles. Teach me how a B-tree index works and what it costs: full table scan versus index search, selectivity, and when NOT to index. Use a 200,000-row orders table with real numbers, explain each step in simple English, and after each idea ask me one question to check my understanding before continuing.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data engineering. Explain composite indexes, the leftmost prefix rule, covering indexes, partial indexes and expression indexes in PostgreSQL. Then give me 8 graded exercises where I must choose the right index for a given query, without solutions. Wait for my answers and correct them kindly, explaining the reasoning.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional who reads business reports well but has never read a database plan. Teach me to read PostgreSQL EXPLAIN and EXPLAIN ANALYZE output node by node: scan types, cost, estimated versus actual rows, loops, Rows Removed by Filter, and join algorithms. I will paste a plan next; interpret it for me line by line: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional learning SQL query optimization. Review the slow query below. Find non-sargable filters, SELECT *, N+1 patterns, missing indexes and bad pagination. For each problem give a rewrite, the index you would create, and how I should measure before and after, with a baseline measurement. My query and table sizes: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional preparing for data analyst and data engineer interviews. Run a mock interview on indexes, EXPLAIN and query tuning: ask me 8 questions one at a time, from easy to hard, and wait for my answer each time. After each answer, score it out of 5, show what I missed, and give a short model answer in simple English.

CHAPTER 11

SCHEMA DESIGN, NORMALIZATION AND DATA WAREHOUSE MODELING

How to draw a database on paper before you build it, how to remove duplicated data safely, and when to break those rules on purpose for analytics.

Bad data usually starts as a bad table design. A customer's city stored in five places gives five different answers. A comma-separated list in one column makes "which orders contain a mouse?" painful. And a dashboard that joins ten tables for every click is slow and hard to understand. In this chapter you learn to design a schema in three steps: draw it (ER diagram), clean it (normalization), then reshape it for reporting (star schema and slowly changing dimensions). By the end you can take a messy spreadsheet and turn it into sound tables, and explain in an interview when you would break the rules.

We keep using the shop database from Chapter 2 (`customers` , `products` , `orders` , `order_items`). All SQL ran in SQLite 3.51, except where I write **not run here**.

11.1 ER Diagrams, Keys and Naming

INTUITION

Before building a house, an architect draws a plan: rooms and doors. An **ER diagram** (Entity-Relationship diagram) is the plan of a database: the things you store, and how they connect. Drawing takes ten minutes. Changing a live table with 50 million rows takes weeks.

Words you need

- **Entity**: a kind of thing you store (customer, product, order). It becomes a table.
- **Attribute**: a property of an entity (name, price). It becomes a column.
- **Relationship**: how two entities connect (a customer *places* orders). It becomes a foreign key, or a separate table.
- **Cardinality**: how many on each side: one-to-one (1:1), one-to-many (1:N), many-to-many (M:N).
- **Junction table** (also *bridge* or *associative* table): a table that turns M:N into two 1:N.

How to read the diagram

```
CUSTOMER 1 ----< places >---- N ORDER N ----< contains >---- M PRODUCT
(PK customer_id)              (PK order_id,                  (PK product_id)
                               FK customer_id)

M:N ORDER–PRODUCT becomes a junction table ORDER_ITEM:

+-----------+ 1     N +---------+ 1      N +------------+ N     1 +-----------+
| customers |---------| orders  |----------| order_items|---------| products  |
+-----------+         +---------+          +------------+         +-----------+
customer_id PK        order_id PK          (order_id, product_id) PK   product_id PK
                      customer_id FK       qty, unit_price
```

Read "1 --- N" as "one customer has many orders." The line sits on the "many" side where the foreign key lives. Rule: **the foreign key always goes on the "many" side.**

Relationship	How to build it	Example
1:N	foreign key on the N side	`orders.customer_id`
M:N	junction table with two FKs (+ own attributes)	`order_items(order_id, product_id, qty, unit_price)`

Relationship	How to build it	Example
1:1	FK with `UNIQUE`, or merge into one table	`user` and `user_profile`

Worked examples

Problem 11.1.1 A learning platform has students, courses and enrollments (a student takes many courses, a course has many students, and each enrollment has a grade). Design the tables.

Step 1	Entities: `students`, `courses`.
Step 2	Students to courses is M:N, and the grade belongs to the *pair*. So we need a junction table `enrollments`.
Step 3	Primary key of the junction table is the pair `(student_id, course_id)`, so a student cannot enroll twice in one course.

```
CREATE TABLE students (student_id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE courses  (course_id  INTEGER PRIMARY KEY, title TEXT NOT NULL);
CREATE TABLE enrollments (
  student_id INTEGER NOT NULL REFERENCES students(student_id),
  course_id  INTEGER NOT NULL REFERENCES courses(course_id),
  grade      INTEGER CHECK (grade BETWEEN 0 AND 100),
  PRIMARY KEY (student_id, course_id)
);
```

- **Answer** Three tables. Each fact lives once: a student's name in `students`, a grade in `enrollments`.
- **Check** If the grade were put in `students`, which course would it belong to? It has no home. That tells you it belongs on the relationship.

Problem 11.1.2 Where does the foreign key go for "an employee works in one department; a department has many employees"?

Step 1	One department, many employees: department is the "one" side.
Step 2	The FK goes on the "many" side: `employees.department_id REFERENCES departments(department_id)`.
Answer	The `employees` table holds the FK. Chapter 1 already did this with `departments` and `employees`.

Surrogate keys, data types and naming (small rules that matter)

- Use a **surrogate primary key** (`customer_id`) and a `UNIQUE` constraint on the natural key (see 9.2).
- Pick the smallest sensible type: `INTEGER` ids, `NUMERIC(12,2)` for money, `TIMESTAMPTZ` for event times (store UTC), `DATE` for dates, `BOOLEAN` for flags. Never store dates as text in PostgreSQL (SQLite has no date type; it stores text, as in our shop data).
- Naming: `snake_case`, same name for the same thing everywhere (`customer_id` in both tables), no spaces, no reserved words like `order` or `group`, tables as plural nouns, and booleans as `is_active`.

Connection to the real job

In design reviews and interviews you are often asked "draw the schema for X." The expected answer: entities, keys, cardinalities, the junction tables, and the reasons. Analysts read an ER diagram first when they join a new company's database.

Applications

- **Back ends:** model users, orders, payments before writing code.
- **Data Engineering:** read the source system's ER diagram before building a pipeline.
- **Data Analytics:** know which join key to use and which side repeats rows (Chapter 4).
- **ML:** features come from joins: the ER diagram tells you how rows multiply.

Check yourself

1. Where does the foreign key go in a 1:N relationship?

Answer: On the "many" side.

1. How do you model a many-to-many relationship?

Answer: With a junction table holding two foreign keys.

1. Why is `order` a bad table name?

Answer: It is a reserved SQL word (`ORDER BY`), so it needs quoting; use `orders` .

11.2 Normalization: 1NF, 2NF, 3NF and BCNF

INTUITION	Imagine writing a customer's phone number on every sales slip. When they change number, you must find and fix every slip, and if you miss one, two numbers exist for one person. **Normalization** is the discipline of writing each fact in **one place only**. The slogan: *every non-key column depends on the key, the whole key, and nothing but the key.*

Words you need

- **Functional dependency (FD)**: "A determines B," written `A → B` : if two rows have the same A, they must have the same B. Example: `product_id → product_name` .
- **Candidate key**: a minimal set of columns that identifies a row.
- **Anomaly**: a bug caused by repeated data. Three kinds: **update anomaly** (change in some places only), **insert anomaly** (cannot store a fact without another), **delete anomaly** (deleting a row loses an unrelated fact).
- **Normal form**: a level of tidiness. Each form includes the one before it.

The messy starting point

```
CREATE TABLE flat (
  order_id INT, product_id INT, product_name TEXT, category TEXT,
  customer_id INT, customer_city TEXT, qty INT,
  PRIMARY KEY (order_id, product_id));
INSERT INTO flat VALUES
 (101,1,'Keyboard','Electronics',1,'Pune',1),
 (101,3,'Notebook','Stationery',1,'Pune',2),
 (103,1,'Keyboard','Electronics',1,'Pune',1),
 (102,2,'Mouse','Electronics',2,'Mumbai',1);
```

Keyboard's name and category repeat in rows 1 and 3. Customer 1's city repeats three times. This is a spreadsheet pretending to be a table.

The forms, one at a time

Form	Rule	Violation looks like
1NF	each cell holds one value; no repeating groups; rows are identified by a key	`products = 'Keyboard,Mouse'` in one cell
2NF	in 1NF, and no non-key column depends on **part** of a composite key	`product_name` depends only on `product_id` , not on `(order_id, product_id)`
3NF	in 2NF, and no non-key column depends on **another non-key column**	`customer_city` depends on `customer_id` , which is not the key

Form	Rule	Violation looks like
BCNF	for every dependency `A → B`, A must be a candidate key	see the course example below

1NF in action. A table stores `products` as `'Keyboard,Mouse'` in one cell. Which orders contain a mouse?

```
CREATE TABLE orders_bad (order_id INT, customer TEXT, city TEXT, products TEXT);
INSERT INTO orders_bad VALUES
 (1,'Asha','Pune','Keyboard,Mouse'),(2,'Asha','Pune','Notebook'),(3,'Ravi','Mumbai','Mouse');
SELECT count(*) AS orders_with_mouse FROM orders_bad WHERE products LIKE '%Mouse%';
```

```
orders_with_mouse
_________________
2
```

- **Step 1** `LIKE '%Mouse%'` finds orders 1 and 3, so 2 is right today.
- **Step 2** It would also match a product named `Mouse Pad` or `Bluetooth Mouse`: wrong count with no error. It cannot use an index (Chapter 10, leading wildcard). You cannot join the list to `products`.
- **Answer** The count is right by luck. The fix for 1NF is one row per (order, product), which is exactly the `order_items` table you already know.

Problem 11.2.1 (find the functional dependency). Does `product_id → product_name` hold in `flat`? Test it with SQL: group by the left side and count distinct values on the right. Any group with more than 1 value is a violation.

```
SELECT product_id, COUNT(DISTINCT product_name) AS names
FROM flat GROUP BY product_id HAVING COUNT(DISTINCT product_name) > 1;
```

```
(no rows)
```

- **Step 1** No product has two names, so the FD holds in today's data.
- **Step 2** Now simulate the **update anomaly**: someone fixes the name in one row only, with a typo.

```
UPDATE flat SET product_name = 'Keybord' WHERE order_id = 101 AND product_id = 1;
-- run the same check again
```

```
product_id  names
----------  -----
1           2
```

- **Answer** Product 1 now has two names. The same fact is stored twice, so the copies drifted apart. This is why we split the table.
- **Meaning** Careful: data can only *disprove* an FD. A test that finds no violation shows the FD holds in this data, not that it must hold forever. The business rule decides.

Problem 11.2.2 (normalize to 3NF). Apply the rules to `flat`.

Step 1 (2NF)	`product_name` and `category` depend only on `product_id`, part of the key `(order_id, product_id)`. Move them to a `products` table keyed by `product_id`.
Step 2 (3NF)	`customer_city` depends on `customer_id`, which depends on `order_id`. Move customer and city into a table keyed by the customer, and keep `customer_id` on the order. (Our quick script keeps a combined orders table with `customer_city` for brevity; a full design has a separate `customers` table like Chapter 2.)
Step 3	What remains, `(order_id, product_id, qty)`, has only `qty`, which depends on the whole key.

```
CREATE TABLE p  AS SELECT DISTINCT product_id, product_name, category FROM flat;
CREATE TABLE o  AS SELECT DISTINCT order_id, customer_id, customer_city FROM flat;
CREATE TABLE oi AS SELECT order_id, product_id, qty FROM flat;
SELECT (SELECT count(*) FROM flat) AS flat_rows, (SELECT count(*) FROM p) AS p,
       (SELECT count(*) FROM o) AS o, (SELECT count(*) FROM oi) AS oi;
```

```
flat_rows  p  o  oi
---------  -  -  --
4          3  3  4
```

- **Check (lossless)** Join the pieces back; we must get the original 4 rows with the same facts:

```
SELECT o.order_id, p.product_name, oi.qty, o.customer_city
FROM oi JOIN p USING (product_id) JOIN o USING (order_id) ORDER BY 1,2;
```

```
order_id  product_name  qty  customer_city
--------  ------------  ---  -------------
101       Keyboard      1    Pune
101       Notebook      2    Pune
102       Mouse         1    Mumbai
103       Keyboard      1    Pune
```

- **Answer** Four rows, identical to the original facts. `p` has 3 rows (Keyboard stored once, not twice) and `o` has 3 orders. The city is stored once per order here and once per customer in a full design, so changing Pune takes one `UPDATE`.
- **Why it matters** A good decomposition is **lossless**: joining the parts gives back exactly the original, with no missing and no extra rows.

BCNF. 3NF still allows one odd case. Take a table `enroll(student, course, instructor)` where each instructor teaches only one course (`instructor → course`), but the key is `(student, course)`.

```
student  course  instructor
Asha     SQL     Rao
Ravi     SQL     Rao
Asha     Python  Iyer
Meena    Python  Iyer
```

Here `instructor → course`, but `instructor` is not a candidate key. That breaks BCNF. Rao's course is repeated in every student row. Test the FD:

```
SELECT instructor, COUNT(DISTINCT course) AS courses FROM enroll GROUP BY instructor;
```

```
instructor  courses
----------  -------
Iyer        1
Rao         1
```

Each instructor has exactly one course in this data, consistent with the rule. The fix: `teaches(instructor, course)` and `enrollment(student, instructor)`. BCNF is rarely a problem in practice; most designs that reach 3NF are already in BCNF. (4NF and 5NF deal with multi-valued dependencies; just know the names.)

COMMON MISTAKE	Treating normal forms as magic words. Ask the simple question instead: "If this fact changes, how many rows must I edit?" The answer should be one.

COMMON MISTAKE	Concluding an FD from a small sample. `city → state` looks true until a second `Springfield` appears. FDs come from business rules, so ask the business.

Connection to the real job

Operational (OLTP) databases are normalized to 3NF so writes are safe and fast. When you query them for analysis you re-join the pieces back, which is why Chapter 4 on joins matters so much.

Applications

- **Back ends:** 3NF keeps checkout, user and inventory data consistent.
- **Data Engineering:** you split wide source files (CSV exports) into clean tables when loading.
- **Data Analytics:** you read the normalized source to find where each metric lives.
- **Data quality:** FD checks with `GROUP BY ... HAVING COUNT(DISTINCT ...) > 1` find dirty data.

Check yourself

1. What does `A → B` mean?

Answer: Whenever two rows have the same A, they have the same B.

1. Which normal form removes dependency on part of a composite key?

Answer: 2NF.

1. Name the three anomalies.

Answer: Update, insert and delete anomalies.

11.3 Denormalization and OLTP vs OLAP

INTUITION

A restaurant kitchen is organised for cooking: ingredients stored once in separate shelves (normalized). But the menu card the customer reads has everything pre-combined: "Paneer wrap with fries and a drink, Rs 250" (denormalized). Cooking and menu-reading are different jobs, so you organise differently for each. In databases the two jobs are **OLTP** (taking orders) and **OLAP** (reading reports).

Words you need

- **OLTP** (Online Transaction Processing): many small, fast reads and writes (place an order, update an address). Like the billing counter of a shop.
- **OLAP** (Online Analytical Processing): fewer, heavy queries that scan lots of rows and aggregate (revenue by month and city).
- **Denormalization**: deliberately storing repeated or pre-joined data to make reads faster or simpler.
- **Data warehouse**: a database built for OLAP, fed from many OLTP systems.
- **Row store vs column store**: a row store keeps a whole row together (good for OLTP). A column store keeps each column together (good for scanning one column across many rows: ClickHouse, BigQuery, Snowflake, Redshift, and Parquet files).

OLTP vs OLAP

	OLTP	OLAP
Typical query	`UPDATE ... WHERE id = 7`	`SUM(revenue) GROUP BY month, city`
Rows touched	a few	millions
Latency goal	milliseconds	seconds to minutes
Users	thousands of app users	tens of analysts, dashboards
Design	normalized (3NF)	denormalized (star schema)

	OLTP	OLAP
Storage	row-oriented	often column-oriented
Example systems	PostgreSQL, MySQL	Snowflake, BigQuery, Redshift, ClickHouse

When to denormalize

Technique	Example	Gain	Risk
Copy a column	store `customer_city` on orders	skip one join	copies can drift
Pre-computed total	`orders.total`	no `SUM` per read	must update with items
Summary table	daily revenue per product	dashboards read 365 rows, not 100M	needs refresh job
Wide table	one flat row per event for ML	simple training queries	big, repeated data

Problem 11.3.1 Our dashboard shows revenue per customer city. Normalized, that needs `customers JOIN orders JOIN order_items`. Estimate the work, then reduce it with a summary table.

```
CREATE TABLE city_revenue AS
SELECT c.city, SUM(i.qty * i.unit_price) AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY c.city;
```

- **Step 1** Run the 3-table join once, store the result. The dashboard now reads the small summary table.
- **Step 2** With 100 million order items and 50 cities, each dashboard view reads 50 rows instead of scanning and joining 100 million.
- **Answer** Reads are far cheaper, in exchange for a refresh job and staleness: the table is only as fresh as its last refresh.
- **Check** The summary must equal the live query. Compare `SUM(revenue)` from both; they must match, otherwise the refresh has a bug.

COMMON MISTAKE

Denormalizing *first* "for speed." Normalize first, measure, and denormalize only the proven hot path. Every copy needs a plan to keep it in sync (a trigger, a refresh job, or one code path).

PROFESSOR'S NOTE

You will hear "NoSQL denormalizes everything." Document stores such as MongoDB embed related data inside one document, which is denormalization again. This document teaches relational design only.

Connection to the real job

Data engineers move data from OLTP databases (normalized) to a warehouse (denormalized). This is the **ETL** or **ELT** job (extract, transform, load): extract from the source, reshape, load into star schemas. Analysts live on the OLAP side; back-end developers live on the OLTP side. Knowing both lets you talk to both teams.

Applications

- **Data Analytics:** dashboards read pre-aggregated tables, not raw logs.
- **Data Engineering:** nightly jobs fill warehouse tables from operational ones.
- **ML:** training tables are wide and flat, with one row per example.
- **Operations:** heavy reports must not run on the primary OLTP database during the busiest sale hours; use a replica or warehouse.

Check yourself

1. Which workload uses normalized tables, OLTP or OLAP?

Answer: OLTP.

1. Give one risk of denormalization.

Answer: The repeated copies can become inconsistent unless you keep them in sync.

1. Why are column stores good for analytics?

Answer: A query reading one or two columns over millions of rows reads only those columns from disk.

11.4 Star and Snowflake Schemas

INTUITION	Imagine a receipt (the event: what was sold, how many, how much) surrounded by description cards: who bought (customer), what (product), when (date). The receipt is the **fact**. The cards are **dimensions**. Draw the receipt in the middle and the cards around it: a star.

Words you need

- **Fact table**: one row per business event (a sale line, a page view). Mostly numbers (measures: `qty`, `revenue`) and keys to dimensions. Usually the biggest table.
- **Dimension table**: descriptive attributes used to filter and group (`city`, `category`, `month`). Usually small and wide.
- **Grain**: what one row of the fact table means. **Decide the grain first.** Ours: one row per order line.
- **Measure**: a number you add up (`revenue`).
- **Surrogate key** in a dimension: a warehouse-made id (`cust_sk`), needed for slowly changing dimensions (11.5).

Build one from the shop

```
CREATE TABLE dim_product  AS SELECT product_id AS product_key, name AS product_name, category FROM products;
CREATE TABLE dim_customer AS SELECT customer_id AS customer_key, name AS customer_name, city FROM customers;
CREATE TABLE dim_date (date_key TEXT PRIMARY KEY, year INT, month INT, month_name TEXT);
INSERT INTO dim_date
SELECT DISTINCT order_date, CAST(strftime('%Y',order_date) AS INT), CAST(strftime('%m',order_date) AS INT),
       CASE strftime('%m',order_date) WHEN '01' THEN 'Jan' WHEN '02' THEN 'Feb' WHEN '03' THEN 'Mar' ELSE 'Apr' END
FROM orders;
CREATE TABLE fact_sales AS
SELECT o.order_id, o.order_date AS date_key, o.customer_id AS customer_key,
       i.product_id AS product_key, i.qty, i.qty * i.unit_price AS revenue
FROM orders o JOIN order_items i USING (order_id)
WHERE o.status = 'paid';
SELECT count(*) AS fact_rows, sum(revenue) AS total FROM fact_sales;
```

```
fact_rows  total
---------  -----
18         12660
```

Problem 11.4.1 Revenue for March by product category and customer city.

```
SELECT d.month_name, p.category, c.city, SUM(f.revenue) AS revenue
FROM fact_sales f
JOIN dim_date     d ON d.date_key     = f.date_key
JOIN dim_product  p ON p.product_key  = f.product_key
JOIN dim_customer c ON c.customer_key = f.customer_key
WHERE d.month = 3
GROUP BY d.month_name, p.category, c.city
ORDER BY revenue DESC;
```

```
month_name  category     city       revenue
----------  -----------  ---------  -------
Mar         Electronics  Pune       1200
Mar         Home         Mumbai     900
Mar         Stationery   Mumbai     440
Mar         Stationery   Bengaluru  360
Mar         Stationery   Pune       80
```

- **Step 1** The query starts at the fact table and joins to each dimension on its key: one join per dimension, always the same shape.
- **Step 2** Filters and groupings come from dimension columns (`month` , `category` , `city`). Sums come from the fact (`revenue`).
- **Check by hand** Order 106 (March, customer 1 in Pune) is 2 mice at 600 = 1200 plus a notebook at 80. The mouse line is Electronics/Pune = 1200, the notebook line is Stationery/Pune = 80. Both match. Order 108 (customer 2, Mumbai, Home) is 900, also matching.
- **Check (total)** All paid revenue directly from the source tables was also 12,660, equal to the fact total.
- **Answer** A star schema answers any "measure by attributes" question with the same simple pattern.

The picture

```
                    dim_date
                       |
dim_customer ------ fact_sales ------ dim_product
  (who)          (order_id, date_key,      (what)
                  customer_key,
                  product_key,
                  qty, revenue)
```

The fact table is in the middle. Each arm is one join. Dimensions are denormalized: `category` sits right in `dim_product` .

Star vs snowflake

A **snowflake schema** normalizes the dimensions further: `dim_product` points to `dim_category` , which points to `dim_department` . It saves some storage and keeps one copy of the category name, but needs more joins.

	Star	Snowflake
Dimension tables	denormalized, flat	normalized, split into sub-tables
Joins per query	fewer	more
Storage	slightly more	slightly less
Query simplicity	high	lower
Use	default for BI tools and warehouses	when a dimension is huge or shared, or hierarchies need their own maintenance

Storage is cheap and analyst time is not, so **star is the usual choice**.

Date dimension and calendar tables

A `dim_date` table with one row per day (year, quarter, month, week, `is_weekend`, holiday flags) is standard. It lets analysts group by "Q1" or "weekend" without date functions, and it supplies days with zero sales, which a plain `GROUP BY` on the fact table cannot show (the date spine idea from the analytics section).

Connection to the real job

Tools such as Power BI, Tableau and Looker expect star schemas. dbt projects name models `fact_` and `dim_`. In an interview "design a data model for an e-commerce / food-delivery / ride-share app" means: pick the grain, list the facts, list the dimensions, mention the date dimension.

Applications

- **Data Analytics:** BI dashboards built on facts and dimensions.
- **Data Engineering:** warehouse models in dbt, Snowflake, BigQuery.
- **ML:** feature tables made by joining a fact table to dimensions.
- **Healthcare:** a star model for appointments: `fact_appointments` with `dim_doctor`, `dim_patient` and `dim_date` makes trend reports simple.

Check yourself

1. What is the grain of `fact_sales` above?

Answer: One row per paid order line (an order and a product).

1. Fact or dimension: `revenue`? `city`?

Answer: `revenue` is a fact measure; `city` is a dimension attribute.

1. Why is a star usually preferred to a snowflake?

Answer: Fewer joins and simpler queries; the extra storage is cheap.

11.5 Slowly Changing Dimensions (SCD)

INTUITION

Asha lived in Pune and ordered in January. In March she moves to Delhi and orders again. If you simply overwrite her city with Delhi, then January's order looks as if it came from Delhi, and your "revenue by city" history is rewritten and wrong. A dimension attribute that changes *slowly and occasionally* is a **slowly changing dimension**. The question is: do you want history or not?

The types

Type	What it does	History kept?	Use when
SCD 0	never changes	n/a	fixed values (birth date)
SCD 1	overwrite the old value	No	corrections, typos, values nobody analyses by time
SCD 2	add a new row, close the old one with dates	Full	"as it was then" reporting (the common choice)
SCD 3	add a column `previous_city`	Only one previous value	rare, "current and previous" only

SCD 1: overwrite

```
CREATE TABLE d1 (customer_id INT PRIMARY KEY, city TEXT);
INSERT INTO d1 VALUES (1,'Pune');
UPDATE d1 SET city = 'Delhi' WHERE customer_id = 1;
SELECT * FROM d1;
```

```
customer_id  city
-----------  -----
1            Delhi
```

Pune is gone. January's orders will now report Delhi.

SCD 2: add a row

Each version of a customer gets its own row, a new surrogate key, a validity period and an `is_current` flag. The natural key `customer_id` repeats.

Problem 11.5.1 Asha (customer 1) moves from Pune to Delhi on 2024-03-10. Record it as SCD 2, then find the city on the date of each order.

```
CREATE TABLE dim_cust (
  cust_sk INTEGER PRIMARY KEY, customer_id INT, name TEXT, city TEXT,
  valid_from TEXT, valid_to TEXT, is_current INT);
INSERT INTO dim_cust (customer_id,name,city,valid_from,valid_to,is_current) VALUES
 (1,'Asha','Pune','2024-01-05','9999-12-31',1),
 (2,'Ravi','Mumbai','2024-01-12','9999-12-31',1);

BEGIN;
UPDATE dim_cust SET valid_to = '2024-03-10', is_current = 0
 WHERE customer_id = 1 AND is_current = 1;
INSERT INTO dim_cust (customer_id,name,city,valid_from,valid_to,is_current)
 VALUES (1,'Asha','Delhi','2024-03-10','9999-12-31',1);
COMMIT;
SELECT * FROM dim_cust ORDER BY cust_sk;
```

```
cust_sk  customer_id  name  city    valid_from  valid_to    is_current
-------  -----------  ----  ------  ----------  ----------  ----------
1        1            Asha  Pune    2024-01-05  2024-03-10  0
2        2            Ravi  Mumbai  2024-01-12  9999-12-31  1
3        1            Asha  Delhi   2024-03-10  9999-12-31  1
```

- **Step 1** Close the old row: set `valid_to` to the change date and `is_current = 0`.
- **Step 2** Insert a new row with a new `cust_sk` (3), the new city, `valid_from` = the change date, and the far-future `9999-12-31` as "open end."
- **Step 3** Both statements sit in one transaction (Chapter 9), so no reader sees a customer with zero or two current rows.

Now link orders to the version that was true on the order date:

```
CREATE TABLE fact_o (order_id INT, customer_id INT, order_date TEXT, revenue INT);
INSERT INTO fact_o VALUES (101,1,'2024-01-10',1660),(106,1,'2024-03-20',680);
SELECT f.order_id, f.order_date, d.city
FROM fact_o f
JOIN dim_cust d ON d.customer_id = f.customer_id
 AND f.order_date >= d.valid_from AND f.order_date < d.valid_to
ORDER BY f.order_id;
```

```
order_id  order_date  city
--------  ----------  -----
101       2024-01-10  Pune
106       2024-03-20  Delhi
```

- **Answer** The January order shows Pune, the March order shows Delhi. History is preserved.
- **Check** Each order matched exactly one dimension row: 2 orders in, 2 rows out. If a join like this ever returns more rows than the fact table, your validity periods overlap (a data bug). The interval is **inclusive at the start, exclusive at the end** (`>= valid_from AND < valid_to`) so the change date belongs to only one row.

- **Alternative** Many warehouses store the surrogate key in the fact table at load time (`customer_sk`), so the fact points straight to the right version and no date join is needed.

SCD 3 adds a column such as `previous_city`. It keeps one step of history only, and is used rarely. There are also hybrid types (SCD 4, 6); just know they exist.

COMMON MISTAKE	Forgetting to filter `is_current = 1` when you want today's customers: each moved customer is counted twice. And forgetting that `COUNT(DISTINCT customer_id)`, not `COUNT(*)`, counts people in an SCD 2 table.

COMMON MISTAKE	Using SCD 2 for everything. Every tracked column multiplies rows. Track only attributes the business analyses over time (city, plan tier, sales region), and use SCD 1 for typo fixes.

Connection to the real job

SCD 2 is a favourite interview topic for data engineering. dbt offers *snapshots* that implement SCD 2 for you. In customer analytics ("did churn differ by plan tier at the time?"), history decides whether the answer is right.

Applications

- **Data Engineering:** SCD 2 loads in warehouses; dbt snapshots.
- **Data Analytics:** "as-was" reporting of region or plan tier.
- **ML:** point-in-time correct features (no peeking at attributes the customer got later), which prevents data leakage.
- **Finance:** historical price lists and tax rates.

Check yourself

1. Which SCD type keeps full history?

Answer: Type 2.

1. Why does SCD 2 need a surrogate key?

Answer: The natural key (`customer_id`) repeats across versions, so each row needs its own unique key.

1. Why use `valid_to` as exclusive in the join?

Answer: So the change date matches only one version, not two.

11.6 Modelling Extras: Functional-Dependency Closure, BCNF by Algorithm, and Patterns You Will Meet

INTUITION	Section 11.2 found normalization problems by eye. Eyes miss things in a table with twelve columns. Database theory gives a mechanical test: write the rules of the data as **functional dependencies** (FDs), compute what each column set determines (its **closure**), and read off the keys and the violations. Then you decompose by algorithm, and prove with a query that nothing was lost.

Words you need

- **Functional dependency** `X -> Y`: whenever two rows agree on X they must agree on Y. 'course -> teacher' says each course has one teacher.
- **Closure** of a set X, written X^+: every column determined by X, found by applying the FDs again and again.
- **Superkey**: a column set whose closure is all columns. A **candidate key** is a minimal superkey.
- **BCNF**: every FD `X -> Y` has a superkey on the left. Any FD that breaks this is a violation.

Problem 11.6.1 A table `enrol(student, course, grade, teacher, office)` has the rules: (student, course) -> grade, course -> teacher, teacher -> office. Find the candidate key and the BCNF violations. Predict first: the key is (student, course), and both `course -> teacher` and `teacher ->`

`office` violate BCNF.

```
from itertools import combinations
attrs = {"student", "course", "grade", "teacher", "office"}
fds = [({"student", "course"}, {"grade"}), ({"course"}, {"teacher"}), ({"teacher"}, {"office"})]

def closure(x):
    """everything that x determines, by repeatedly applying the rules"""
    x = set(x)
    changed = True
    while changed:
        changed = False
        for left, right in fds:
            if left <= x and not right <= x:
                x |= right
                changed = True
    return x

print("closure of {course}          :", sorted(closure({"course"})))
print("closure of {student,course}  :", sorted(closure({"student", "course"})))
keys = [set(c) for n in range(1, 4) for c in combinations(sorted(attrs), n) if closure(c) == attrs]
minimal = [k for k in keys if not any(o < k for o in keys)]
print("candidate keys:", [sorted(k) for k in minimal])
for left, right in fds:
    ok = closure(left) == attrs
    print("%-18s -> %-8s left side is a superkey: %s" % (",".join(sorted(left)), ",".join(sorted(right)), ok))
```

```
closure of {course}          : ['course', 'office', 'teacher']
closure of {student,course}  : ['course', 'grade', 'office', 'student', 'teacher']
candidate keys: [['course', 'student']]
course,student     -> grade    left side is a superkey: True
course             -> teacher  left side is a superkey: False
teacher            -> office   left side is a superkey: False
```

Step 1	Closure of {course}: start with course. Rule 2 adds teacher. Rule 3 then adds office. Nothing else applies: {course, office, teacher}. It is missing student and grade, so course alone is not a key.
Step 2	Closure of {student, course}: rule 1 adds grade, rule 2 teacher, rule 3 office. That is all five columns, so it is a key, and it is minimal because neither student nor course alone reaches everything.
Step 3	Check each rule's left side. (student, course) is a superkey, so rule 1 is fine. course is not, so `course -> teacher` violates BCNF. teacher is not, so `teacher -> office` violates too.
Meaning	The violations are exactly the redundancy you would see: the office of 'Iyer' is repeated for every enrolment of every course Iyer teaches, and an office change has to be made in several rows.

The decomposition algorithm and the lossless check

Pick a violating FD X -> Y. Split the table into (X plus Y) and (everything except Y, which keeps X). Repeat on each piece until none violates. Here: split on `teacher -> office` first (teacher_office), then `course -> teacher` (course_teacher), leaving (student, course, grade). Because each split keeps the determinant X as the join column, the pieces rejoin exactly. The code builds the three tables and proves it:

```
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE enrol (student TEXT, course TEXT, grade TEXT, teacher TEXT, office TEXT);
INSERT INTO enrol VALUES
 ('Asha','SQL','A','Rao','B12'), ('Ravi','SQL','B','Rao','B12'),
 ('Asha','Stats','B','Iyer','C03'), ('Meena','Stats','A','Iyer','C03'),
 ('Ravi','ML','A','Iyer','C03');
-- BCNF decomposition from the closure analysis
CREATE TABLE teacher_office (teacher TEXT PRIMARY KEY, office TEXT);
CREATE TABLE course_teacher (course TEXT PRIMARY KEY, teacher TEXT REFERENCES teacher_office);
CREATE TABLE grade_of (student TEXT, course TEXT REFERENCES course_teacher, grade TEXT, PRIMARY KEY (student, course));
INSERT INTO teacher_office SELECT DISTINCT teacher, office FROM enrol;
INSERT INTO course_teacher SELECT DISTINCT course, teacher FROM enrol;
INSERT INTO grade_of SELECT student, course, grade FROM enrol;
""")
rebuilt = db.execute("""SELECT g.student, g.course, g.grade, c.teacher, t.office
  FROM grade_of g JOIN course_teacher c ON c.course = g.course JOIN teacher_office t ON t.teacher = c.teacher
  ORDER BY 1, 2""").fetchall()
original = db.execute("SELECT * FROM enrol ORDER BY 1, 2").fetchall()
print("rows original:", len(original), "| rows rebuilt:", len(rebuilt), "| identical:", rebuilt == original)
db.execute("UPDATE teacher_office SET office = 'D07' WHERE teacher = 'Iyer'")   # one place to change
print(db.execute("SELECT DISTINCT course, office FROM course_teacher c JOIN teacher_office t USING (teacher) ORDER BY
1").fetchall())
```

```
rows original: 5 | rows rebuilt: 5 | identical: True
[('ML', 'D07'), ('SQL', 'B12'), ('Stats', 'D07')]
```

Step 1	Three small tables, each with a key that is the left side of its FD.
Step 2	Joining them back gives the same 5 rows as the original: a **lossless** decomposition.
Step 3	The office of Iyer now lives in one row. One `UPDATE` changed it for both Stats and ML. In the original table you would have had to update 3 rows and could easily miss one.

COMMON MISTAKE

Splitting a table into two pieces that share no determining column, for example (student, grade) and (course, teacher). Joining them back on nothing, or on a non-key column, creates *spurious rows* that never existed. A decomposition is only safe when the shared columns form a key of at least one piece, which is exactly what the algorithm guarantees.

GOING DEEPER

BCNF can lose a dependency: some decompositions mean an FD can no longer be checked inside one table, which is why many designers stop at 3NF, which always preserves dependencies. 4NF goes one step further for multi-valued facts: if a person has independent sets of skills and languages, one table of (person, skill, language) must list every combination, and the fix is two tables (person, skill) and (person, language). In practice: aim for 3NF in transactional systems, check by FD closure when a design feels wrong, and denormalize deliberately (Section 11.3) only in analytics layers.

Pattern 1: soft delete without breaking uniqueness

Soft delete means marking a row deleted (`deleted_at`) instead of removing it, so history survives and mistakes can be undone. The trap: a normal `UNIQUE (email)` blocks a person who deleted their account from signing up again. A **partial unique index** enforces uniqueness only among live rows.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE users (id INTEGER PRIMARY KEY, email TEXT NOT NULL, deleted_at TEXT);
CREATE UNIQUE INDEX users_email_live ON users(email) WHERE deleted_at IS NULL;
INSERT INTO users (email) VALUES ('a@x.com');
UPDATE users SET deleted_at = '2024-05-01' WHERE email = 'a@x.com';
INSERT INTO users (email) VALUES ('a@x.com');     -- allowed: the old row is soft-deleted
""")
print(db.execute("SELECT id, email, deleted_at FROM users ORDER BY id").fetchall())
try:
    db.execute("INSERT INTO users (email) VALUES ('a@x.com')")   # a second live row with the same email
except sqlite3.IntegrityError as e:
    print("blocked:", e)
print("live users:", db.execute("SELECT count(*) FROM users WHERE deleted_at IS NULL").fetchone()[0])
```

```
[(1, 'a@x.com', '2024-05-01'), (2, 'a@x.com', None)]
blocked: UNIQUE constraint failed: users.email
live users: 1
```

Step 1	Row 1 is soft-deleted, so the live-only index ignores it, and a second row with the same email is accepted.
Step 2	A third insert would create two *live* rows with that email, and the index rejects it.
Note	Every query must now remember `WHERE deleted_at IS NULL`. Hide that behind a view (`CREATE VIEW live_users AS ...`), or the first forgotten filter shows deleted data. In PostgreSQL row-level security can enforce it.

Pattern 2: multi-tenant design that cannot cross tenants

In a SaaS product many customers (tenants) share tables. The worst bug is one tenant seeing another's data. Put `tenant_id` in every key, and make foreign keys include it, so the database itself refuses a cross-tenant reference.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("PRAGMA foreign_keys = ON")
db.executescript("""
CREATE TABLE customer (tenant_id INTEGER, customer_id INTEGER, name TEXT, PRIMARY KEY (tenant_id, customer_id));
CREATE TABLE orders (
  tenant_id INTEGER, order_id INTEGER, customer_id INTEGER,
  PRIMARY KEY (tenant_id, order_id),
  FOREIGN KEY (tenant_id, customer_id) REFERENCES customer (tenant_id, customer_id));
INSERT INTO customer VALUES (1, 10, 'Asha'), (2, 10, 'Zed');
INSERT INTO orders VALUES (1, 100, 10);     -- tenant 1 order for tenant 1 customer: fine
""")
try:
    db.execute("INSERT INTO orders VALUES (1, 101, 99)")      # tenant 1 pointing at a customer tenant 1 does not have
except sqlite3.IntegrityError as e:
    print("blocked:", e)
print(db.execute("SELECT * FROM orders").fetchall())
```

```
blocked: FOREIGN KEY constraint failed
[(1, 100, 10)]
```

Step 1	The primary key of `customer` is (tenant_id, customer_id). Both tenants have a customer 10, which is fine.
Step 2	The foreign key in `orders` is composite. A tenant-1 order can only point to a (1, customer) pair that exists, so a typo or an attack that points at tenant 2's customer is refused.
Step 3	The failing insert shows the database error. Add row-level security (Section 9.6) so queries are also filtered by tenant.

Pattern 3: polymorphic references, done safely

Comments that can belong to a post *or* a photo are often stored as `(target_type, target_id)`. That cannot have a foreign key, so orphans creep in. A safe design uses one nullable foreign key per parent and a `CHECK` that exactly one is set:

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("PRAGMA foreign_keys = ON")
db.executescript("""
CREATE TABLE post (post_id INTEGER PRIMARY KEY);
CREATE TABLE photo (photo_id INTEGER PRIMARY KEY);
CREATE TABLE comment (
  comment_id INTEGER PRIMARY KEY,
  post_id  INTEGER REFERENCES post,
  photo_id INTEGER REFERENCES photo,
  body TEXT,
  CHECK ((post_id IS NOT NULL) + (photo_id IS NOT NULL) = 1)   -- exactly one parent
);
INSERT INTO post VALUES (1); INSERT INTO photo VALUES (1);
INSERT INTO comment (post_id, body) VALUES (1, 'nice post');
""")
for sql in ["INSERT INTO comment (post_id, photo_id, body) VALUES (1, 1, 'two parents')",
            "INSERT INTO comment (post_id, body) VALUES (77, 'ghost post')",
            "INSERT INTO comment (body) VALUES ('orphan')"]:
    try:
        db.execute(sql); print("accepted:", sql[:44])
    except sqlite3.IntegrityError as e:
        print("blocked :", e)
```

```
blocked : CHECK constraint failed: (post_id IS NOT NULL) + (photo_id IS NOT NULL) = 1
blocked : FOREIGN KEY constraint failed
blocked : CHECK constraint failed: (post_id IS NOT NULL) + (photo_id IS NOT NULL) = 1
```

Step 1	Two parents set: the `CHECK` rejects it.
Step 2	A post that does not exist: the foreign key rejects it.
Step 3	No parent at all: the `CHECK` rejects it. The two-column design needs a new column per new parent type, but the database now guards every row.

Surrogate versus natural keys, and where this meets data engineering

A **natural key** comes from the real world (email, ISBN); a **surrogate key** is a meaningless number the database hands out. Natural keys change (people change email) and cost more to join on, so use a surrogate primary key and keep a `UNIQUE` constraint on the natural one, as Section 9.1 advised. For many-to-many relations the junction table's key is the pair of foreign keys. In a warehouse the same ideas reappear as layers: raw tables as loaded, staging models that rename and clean, and mart models shaped as stars. **Data Vault** is a warehouse modelling style that separates stable business keys (hubs), relationships (links) and changing attributes (satellites) so history is never rewritten; it suits many source systems, and costs more joins. A tool such as dbt expresses each layer as a SQL model (see the Data Engineering document).

Practice with answers

Problem 11.6.2 Given R(a, b, c) with FDs a -> b and b -> c, what is the closure of {a} and is a the key? *(easy)*

Answer: a determines b, b then determines c, so the closure is {a, b, c}: all columns. a is a key. The FD b -> c violates BCNF (b is not a key), but it is a transitive dependency, so it also violates 3NF.

Problem 11.6.3 Decompose R(a, b, c) from the previous problem. *(medium)*

Answer: Split on b -> c: R1(b, c) with key b, and R2(a, b) with key a. They share b, which is a key of R1, so the join is lossless.

Problem 11.6.4 Why does `UNIQUE (email)` plus soft delete cause trouble, and give two fixes. *(find the bug)*

Answer: A deleted row keeps its email, so the person cannot register again. Fix 1: a partial unique index where `deleted_at IS NULL` (PostgreSQL, SQLite). Fix 2: on deletion, rewrite the email to a unique tombstone, or move the row to an archive table. MySQL has no partial indexes, so use the second or a generated column.

Problem 11.6.5 Why should every table in a multi-tenant database include tenant_id in its primary key and foreign keys? *(interview)*

Answer: So the schema itself prevents cross-tenant references and makes per-tenant queries and sharding natural; a plain integer key would let a bug link tenant 1 to tenant 2's data, which is a data-leak class of defect. It costs a wider index and slightly larger keys.

Chapter recap

- An ER diagram lists entities, attributes and relationships. The foreign key goes on the "many" side; many-to-many needs a junction table.
- Use surrogate primary keys, sensible types and consistent `snake_case` names.
- Normalization stores each fact once. 1NF: single values. 2NF: no dependency on part of a key. 3NF: no dependency on non-key columns. BCNF: every determinant is a candidate key.
- An FD `A → B` can be tested with `GROUP BY A HAVING COUNT(DISTINCT B) > 1`.
- A good decomposition is lossless: joining the parts returns the original rows.
- OLTP uses normalized tables for fast safe writes; OLAP uses denormalized tables for fast big reads.
- Denormalize only measured hot paths, and own the sync plan.
- A star schema has one fact table (the grain) and flat dimension tables. A snowflake normalizes dimensions further. Star is the usual default.
- SCD 1 overwrites, SCD 2 adds a row with validity dates, SCD 3 adds a column.
- Always verify by totals: the star's `SUM(revenue)` must equal the source query.

Cheat-sheet

Topic	Rule or pattern
1:N	FK on the many side
M:N	junction table with composite PK
FD test	`SELECT a, COUNT(DISTINCT b) FROM t GROUP BY a HAVING COUNT(DISTINCT b) > 1`
2NF / 3NF	part-of-key and non-key dependencies move to their own table
Star query	`FROM fact JOIN dim ON key ... GROUP BY dim columns`
SCD 2 columns	`sk, natural_key, attrs, valid_from, valid_to, is_current`
SCD 2 join	`f.date >= d.valid_from AND f.date < d.valid_to`
Grain	"one row per ..." decided first

Practice set

Problem 11.P.1 (easy) Where does the foreign key go: one `author` writes many `books`? **Answer:** On `books` (`books.author_id`), the many side.

Problem 11.P.2 (easy) Is a cell containing `'red,green,blue'` in 1NF? **Answer:** No. Use one row per value in a child table.

Problem 11.P.3 (what is the output?) In a table `t(a, b)` with rows (1,'x'), (1,'y'), (2,'z'), what does `SELECT a, COUNT(DISTINCT b) FROM t GROUP BY a HAVING COUNT(DISTINCT b) > 1` return? **Answer:** One row: `a = 1`, count 2. So `a → b` does not hold.

Problem 11.P.4 (computation) A fact table has 50 million rows. A dimension has 200 cities. A summary table by city and day for 365 days has how many rows at most, and what is the reduction? **Answer:** 200 × 365 = 73,000 rows. 50,000,000 / 73,000 is about 685, so the dashboard reads about 685 times fewer rows.

Problem 11.P.5 (find the bug) A table has columns `(order_id, product_id, product_name, qty)` with PK `(order_id, product_id)`. Which normal form is violated and why? **Answer:** 2NF. `product_name` depends on `product_id` alone, only part of the key. Move it to `products`.

Problem 11.P.6 (find the bug) After an SCD 2 load, `COUNT(*)` of customers is 1,240 but the company has 1,100. What happened? **Answer:** Moved customers have two or more rows (one per version). Count current rows (`WHERE is_current = 1`) or `COUNT(DISTINCT customer_id)`.

Problem 11.P.7 (find the bug) The SCD 2 join returns 3 rows for 2 orders. Give the likely cause. **Answer:** Overlapping validity periods (for example both `valid_to = '2024-03-10'` and an inclusive `<=` in the join), so an order matches two versions. Make the end exclusive and check for overlaps.

Problem 11.P.8 (scenario) Design a star schema for ride-share trips. Name the grain, one fact table, and four dimensions. **Answer:** Grain: one row per completed trip. Fact `fact_trips` (`trip_id`, keys, `distance_km`, `fare`, `duration_min`). Dimensions: `dim_date` (and a time-of-day dimension), `dim_rider`, `dim_driver`, `dim_location` (pickup, and a second role for drop-off), `dim_vehicle_type`.

Problem 11.P.9 (scenario) Marketing wants "revenue by customer region as it was when the order happened" but the region team often reassigns customers. What do you do? **Answer:** Make region an SCD 2 attribute in the customer dimension (valid dates), and join the fact by order date or store the dimension's surrogate key in the fact at load time.

Problem 11.P.10 (interview) When would you deliberately denormalize? **Answer:** When measurement shows a read path is hot and the join cost is real, and I have a sync plan (summary table with a refresh job, trigger, or single write path). For analytics I model a star schema. I normalize first because it prevents anomalies, then relax for evidence.

Problem 11.P.11 (interview) Explain OLTP vs OLAP and why you would not run a big report on the production OLTP database. **Answer:** OLTP is many short transactions needing low latency; OLAP is few heavy scans. A big report scans many rows, uses CPU, memory and I/O, and can hold resources that slow checkout. Run it on a replica or in a warehouse.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning SQL for data roles. Teach me ER diagrams and relational schema design: entities, attributes, 1:1, 1:N and M:N relationships, junction tables, primary and foreign keys, naming conventions. Use a food-delivery app as the running example, draw ASCII diagrams, explain each step in simple English, and after each idea ask me one question before moving on.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data engineering. Teach me normalization from 1NF to BCNF with a messy single-table spreadsheet of orders. Show update, insert and delete anomalies with small numbers, then normalize step by step and prove the split is lossless with a join. Finish with 8 graded exercises without solutions, and wait for my answers.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional who understands everyday business processes but is new to data modeling. Explain OLTP versus OLAP, row stores versus column stores, and when to denormalize. Use a checkout system and a reporting dashboard as examples. Then give me 6 scenarios and ask me for each whether to normalize or denormalize, and why. Correct my answers kindly.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data engineer interviews. Teach me star and snowflake schemas: fact tables, dimensions, grain, measures and the date dimension. Then ask me to design a star schema for a business of your choice, and review my design when I answer. Check grain, keys, and whether my example queries work. My design: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data engineer interviews. Run a mock interview on slowly changing dimensions types 0 to 3 and schema design. Ask 8 questions one at a time, including a SQL question where I write an SCD 2 update and insert. After each answer, score it out of 5, show what I missed, and give a model answer in simple English.
```

CHAPTER 12

ANALYTICS SQL PATTERNS

Funnels, cohort retention, RFM, MoM and YoY, sessionization, deduplication and data-quality queries: the questions every analyst is asked in the first month of a job.

A manager never asks "write me a LEFT JOIN". She asks: "Of the people who viewed a product, how many paid?", "Do customers who joined in January still buy in April?", "Who are our best customers?", "Is revenue growing?". Each of these is a known **pattern**: a standard shape of SQL that thousands of analysts reuse. In this chapter you will learn the six most common patterns, built only from tools you already know (joins from Chapter 4, CTEs from Chapter 5, `CASE` and dates from Chapter 6, window functions from the window chapters). By the end you can answer all the questions above on real data, and check that your answer is not silently wrong.

We reuse the shop database from Chapter 2 (customers, products, orders, order_items) and add small new tables when a pattern needs them. All outputs below were produced with `sqlite3`. Date functions differ by database, so each dialect-specific piece is marked and the differences are collected in Chapter 14.

12.1 Funnel Analysis and Churn

INTUITION

A funnel is a leaking pipe. 100 people enter at the top (view a product). Some leave before the next stage (add to cart), more leave before checkout, and only a few come out at the bottom (pay). The analyst's job is to measure how many are left after each stage, and to find the stage where the pipe leaks most. It is like an online shop checkout: you check how many visitors complete step 1, step 2, step 3.

Words you need

Word	Meaning
Event	One row saying "user X did action Y at time T"
Funnel step	One stage in the ordered path (view, cart, checkout, pay)
Conversion rate	Users who reached a step, divided by users who reached an earlier step, as a percent
Churn	A customer who was active in one period and not active in the next

How it works

1. Turn the event log into **one row per user**, with the first time they did each step (`MIN` + `CASE`, the conditional aggregation of Chapter 3).
2. Mark a user as "reached step N" only if the earlier steps happened **earlier in time**.
3. Count users per step and divide.

Our running table for this section:

```
CREATE TABLE events (user_id INTEGER, event TEXT, ts TEXT);
INSERT INTO events VALUES
 (1,'view','2024-05-01 10:00:00'),(1,'cart','2024-05-01 10:05:00'),(1,'checkout','2024-05-01 10:09:00'),(1,'pay','2024-05-01 10:12:00'),
 (2,'view','2024-05-01 11:00:00'),(2,'cart','2024-05-01 11:30:00'),(2,'checkout','2024-05-01 11:40:00'),
 (3,'view','2024-05-01 12:00:00'),(3,'cart','2024-05-01 12:02:00'),
 (4,'view','2024-05-01 13:00:00'),
 (5,'view','2024-05-01 14:00:00'),(5,'view','2024-05-01 14:01:00'),(5,'cart','2024-05-01 14:03:00'),(5,'checkout','2024-05-01 14:05:00'),(5,'pay','2024-05-01 14:08:00'),
 (6,'view','2024-05-01 15:00:00'),(6,'pay','2024-05-01 15:02:00');
```

User 5 viewed twice (a repeat). User 6 paid without a cart or checkout (maybe a "buy now" link, or a tracking bug).

Worked examples

Problem 12.1.1 (tiny) How many rows and how many distinct users have a `view` event? Predict: user 5 has two views, so rows and users differ.

```
SELECT COUNT(*) AS views_rows, COUNT(DISTINCT user_id) AS viewers
FROM events WHERE event='view';
```

```
views_rows  viewers
----------  -------
7           6
```

- **Answer** 7 rows but 6 people.
- **Why it matters** A funnel counts **people**, not events. `COUNT(*)` would inflate step 1 and make every conversion rate look worse.

Problem 12.1.2 Build the strict funnel: a user counts at a step only if all earlier steps happened earlier in time.

Step 1	One row per user with the first time of each step.
Step 2	Use `CASE` to give 1 or 0 for "reached step N in order". A comparison with `NULL` gives `NULL`, which is not true, so the `ELSE 0` catches users who never did the step.
Step 3	Sum the 1s, then compute percentages.

```
WITH t AS (
 SELECT user_id,
  MIN(CASE WHEN event='view'     THEN ts END) t_view,
  MIN(CASE WHEN event='cart'     THEN ts END) t_cart,
  MIN(CASE WHEN event='checkout' THEN ts END) t_check,
  MIN(CASE WHEN event='pay'      THEN ts END) t_pay
 FROM events GROUP BY user_id),
s AS (
 SELECT user_id,
  1 AS s1,
  CASE WHEN t_cart  > t_view  THEN 1 ELSE 0 END AS s2,
  CASE WHEN t_check > t_cart  AND t_cart > t_view THEN 1 ELSE 0 END AS s3,
  CASE WHEN t_pay > t_check AND t_check > t_cart AND t_cart > t_view THEN 1 ELSE 0 END AS s4
 FROM t),
c AS (SELECT SUM(s1) a, SUM(s2) b, SUM(s3) c, SUM(s4) d FROM s)
SELECT 'view' step, a users, 100.0 AS pct_of_top, NULL AS pct_of_prev FROM c
UNION ALL SELECT 'cart', b, ROUND(100.0*b/a,1), ROUND(100.0*b/a,1) FROM c
UNION ALL SELECT 'checkout', c, ROUND(100.0*c/a,1), ROUND(100.0*c/b,1) FROM c
UNION ALL SELECT 'pay', d, ROUND(100.0*d/a,1), ROUND(100.0*d/c,1) FROM c;
```

```
step      users  pct_of_top  pct_of_prev
--------  -----  ----------  -----------
view      6      100.0
cart      4      66.7        66.7
checkout  3      50.0        75.0
pay       2      33.3        66.7
```

- **Check by hand** Users 1, 2, 3, 5 carted after viewing: 4. Of those, 1, 2, 5 checked out: 3. Of those, 1 and 5 paid: 2. User 6 paid but is not counted, because the cart and checkout steps are missing.
- **Answer** 6 viewers, 4 carted (66.7%), 3 reached checkout (75% of carts), 2 paid (66.7% of checkouts), overall 33.3%.
- **Meaning** The biggest leak, as a share of the previous step, is view to cart: 1 in 3 viewers leaves. With real traffic that is where you would start testing.

Problem 12.1.3 (combine) Monthly churn from the shop: customers with a paid order in March who have no paid order in April.

```
WITH a AS (SELECT DISTINCT customer_id, substr(order_date,1,7) ym
           FROM orders WHERE status='paid')
SELECT COUNT(*) AS active_mar,
  SUM(NOT EXISTS (SELECT 1 FROM a b WHERE b.customer_id=a.customer_id AND b.ym='2024-04')) AS churned,
  ROUND(100.0*SUM(NOT EXISTS (SELECT 1 FROM a b
        WHERE b.customer_id=a.customer_id AND b.ym='2024-04'))/COUNT(*),1) AS churn_pct
FROM a WHERE ym='2024-03';
```

```
active_mar  churned  churn_pct
----------  -------  ---------
4           1        25.0
```

- **Check** March buyers: customers 1, 2, 5, 6. April buyers: 1, 2, 4, 5. Only customer 6 is missing in April: 1 of 4, so 25%.
- **Answer** Churn from March to April is 25%.
- **Meaning** One in four active customers did not come back the next month. `NOT EXISTS` is the anti-join of Chapter 4. `SUM(boolean)` works in SQLite and MySQL; in PostgreSQL write `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` (not run here).

The picture

```
 6 users  view   ############################  100%
 4 users  cart   ##################            66.7%   (-2 left)
 3 users  check  ##############                50.0%   (-1 left)
 2 users  pay    #########                     33.3%   (-1 left)
```

Each bar is the number of distinct users who reached that step in the correct order. The shorter the next bar, the bigger the leak between the two steps.

Connection to the real job

Product and marketing analysts get funnel questions weekly. Data engineers build the "one row per user" table once so that dashboards do not recompute it.

Applications

- **Data Analytics:** checkout funnel, onboarding funnel, signup funnel.
- **Operations:** a hospital visit is a funnel (register, consult, test, pharmacy); the longest queue shows the weak step.
- **Data Science:** drop-off rates feed churn-prediction features.

COMMON MISTAKE

Counting events instead of distinct users, and ignoring the order of steps. User 6 paid without a cart. A sloppy "did they ever pay" count gives 3 payers, the strict funnel gives 2. Decide your rule first, write it down, then query.

Check yourself

1. Why do we use `MIN(ts)` per step? Answer: to get the first time the user did it, one value per user.
2. In 12.1.2, why is `pct_of_prev` for checkout `100.0*c/b`? Answer: it divides checkout users (3) by cart users (4) to get 75%.
3. What does a user with `t_cart` NULL get at step 2? Answer: 0, because `NULL > t_view` is not true and the `ELSE` branch runs.

12.2 Cohort Retention

INTUITION

A school follows each admission batch separately. "Of the 2024 batch, how many are still enrolled after 1 year, after 2 years?" You compare batches fairly because each is measured from its own start. A **cohort** is a group of customers who started in the same period. **Retention** is the share of the cohort still active n periods later.

How it works

1. Find each customer's **cohort**: the month of their first paid order.
2. For every later paid month compute **month_n** = months between the cohort month and that month.
3. Count distinct customers per (cohort, month_n). Divide by cohort size (month 0).

Worked examples

Problem 12.2.1 (tiny) In which month did customers 1 and 2 start? Predict: customer 1's first paid order is 101 (January 10), customer 2's is 102 (January 15).

```
SELECT customer_id, MIN(substr(order_date,1,7)) AS cohort
FROM orders WHERE status='paid' AND customer_id IN (1,2) GROUP BY customer_id;
```

```
customer_id  cohort
-----------  -------
1            2024-01
2            2024-01
```

- **Answer** Both belong to the 2024-01 cohort. `substr(order_date,1,7)` cuts 'YYYY-MM' from the text date. We reuse this `MIN` idea in the next problem.

Problem 12.2.2 Build the cohort table (months 0 to 3).

```
WITH paid AS (
  SELECT customer_id, substr(order_date,1,7) AS ym
  FROM orders WHERE status='paid' GROUP BY 1,2),
first AS (SELECT customer_id, MIN(ym) AS cohort FROM paid GROUP BY customer_id),
j AS (
  SELECT f.cohort,
   (CAST(substr(p.ym,1,4) AS INT)*12 + CAST(substr(p.ym,6,2) AS INT))
   -(CAST(substr(f.cohort,1,4) AS INT)*12 + CAST(substr(f.cohort,6,2) AS INT)) AS m,
   p.customer_id
  FROM paid p JOIN first f USING (customer_id))
SELECT cohort, COUNT(DISTINCT customer_id) AS size,
  COUNT(DISTINCT CASE WHEN m=0 THEN customer_id END) AS m0,
  COUNT(DISTINCT CASE WHEN m=1 THEN customer_id END) AS m1,
  COUNT(DISTINCT CASE WHEN m=2 THEN customer_id END) AS m2,
  COUNT(DISTINCT CASE WHEN m=3 THEN customer_id END) AS m3
FROM j GROUP BY cohort ORDER BY cohort;
```

```
cohort   size  m0  m1  m2  m3
-------  ----  --  --  --  --
2024-01  2     2   1   2   2
2024-02  1     1   0   1   0
2024-03  2     2   1   0   0
```

- **Step 1** Month number = `year*12 + month` for both dates, then subtract. This avoids a date-difference function that differs by dialect. In PostgreSQL you would use `DATE_TRUNC` and `AGE` or the same arithmetic (not run here).
- **Check** January cohort is customers 1 and 2. In February only customer 1 bought (m1 = 1). In March both (m2 = 2). In April both (m3 = 2). February cohort is customer 4: bought in February and April (m2 = 1). March cohort is customers 5 and 6: in April only customer 5 (m1 = 1).
- **Answer** The January cohort retains 50% in month 1, 100% in months 2 and 3.
- **Meaning** Retention does not have to fall every month. Customers skip a month and return.

Problem 12.2.3 Turn counts into percent with a window function.

```
WITH paid AS (
  SELECT customer_id, substr(order_date,1,7) AS ym
  FROM orders WHERE status='paid' GROUP BY 1,2),
first AS (SELECT customer_id, MIN(ym) AS cohort FROM paid GROUP BY customer_id),
j AS (
  SELECT f.cohort,
   (CAST(substr(p.ym,1,4) AS INT)*12 + CAST(substr(p.ym,6,2) AS INT))
   -(CAST(substr(f.cohort,1,4) AS INT)*12 + CAST(substr(f.cohort,6,2) AS INT)) AS m,
   p.customer_id
  FROM paid p JOIN first f USING (customer_id)),
c AS (SELECT cohort, m, COUNT(DISTINCT customer_id) n FROM j GROUP BY 1,2)
SELECT cohort, m AS month_n, n,
  ROUND(100.0*n/FIRST_VALUE(n) OVER (PARTITION BY cohort ORDER BY m),0) AS retention_pct
FROM c ORDER BY cohort, m;
```

```
cohort   month_n  n  retention_pct
-------  -------  -  -------------
2024-01  0        2  100.0
2024-01  1        1  50.0
2024-01  2        2  100.0
2024-01  3        2  100.0
2024-02  0        1  100.0
2024-02  2        1  100.0
2024-03  0        2  100.0
2024-03  1        1  50.0
```

- **Why** `FIRST_VALUE` Month 0 is the first row of each cohort, and it is the cohort size, so every row divides by it.
- **Meaning** This long form is what you feed into a heatmap in Power BI or Python.

The picture

```
 cohort    m0    m1    m2    m3
 2024-01   100%  50%   100%  100%
 2024-02   100%  --    100%   ?
 2024-03   100%  50%    ?     ?
           ('?' = month has not happened yet, we have data to April only)
```

Rows are start months, columns are months since start. Reading across shows one cohort ageing. Reading down compares cohorts at the same age.

Connection to the real job

Retention tables are the main product-health chart in SaaS, apps and e-commerce. Investors ask for them.

Applications

- **Data Analytics:** weekly active-user retention, repeat-purchase rate.
- **Data Science:** retention curves feed survival models and lifetime-value forecasts.
- **Marketing:** compare cohorts from different campaigns.

COMMON MISTAKE	Reading a missing cell as zero retention. In the real output the February cohort has `m3 = 0`, but month 3 of that cohort is May, and we only have data until April. The 0 means "not observed yet", not "everyone left". Compare the cohort's age with the last date in your data before you draw conclusions.

Check yourself

1. What is a cohort? Answer: customers who started in the same period.

2. Why `COUNT(DISTINCT customer_id)`? Answer: a customer with two orders in a month must count once.
3. What would a cohort of size 1 tell you? Answer: very little; its percentages jump between 0 and 100.

12.3 RFM Segmentation and Customer Lifetime Value

INTUITION

A shopkeeper knows three things about regulars: how recently they came, how often they come, and how much they spend. A customer who came yesterday, comes weekly and spends a lot is a champion. One who spent a lot but vanished six months ago needs a phone call. **RFM** = **R**ecency, **F**requency, **M**onetary value.

How it works

- **Recency** = days since the last paid order, measured against a fixed "as-of" date (never `now()` when you want a repeatable answer).
- **Frequency** = number of paid orders.
- **Monetary** = total paid money.
- Give each a score (1 to 3 here) with `CASE` thresholds, or with `NTILE` (equal-sized buckets) when you have many customers. Then name segments.

Worked examples

Problem 12.3.1 Compute raw R, F, M for each paying customer, as of 2024-04-30.

```
WITH rev AS (
  SELECT o.customer_id, o.order_id, o.order_date, SUM(oi.qty*oi.unit_price) AS amt
  FROM orders o JOIN order_items oi USING(order_id)
  WHERE o.status='paid' GROUP BY o.order_id),
rfm AS (
  SELECT customer_id,
    CAST(julianday('2024-04-30') - julianday(MAX(order_date)) AS INT) AS recency_days,
    COUNT(*) AS frequency, SUM(amt) AS monetary
  FROM rev GROUP BY customer_id)
SELECT * FROM rfm ORDER BY customer_id;
```

```
customer_id  recency_days  frequency  monetary
-----------  ------------  ---------  --------
1            28            4          4560
2            10            3          3600
4            21            2          3300
5            15            2          760
6            31            1          440
```

- **Step 1** The inner query sums line values per order first, so frequency counts orders and not order lines (the duplicate trap of Section 4.4).
- **Check** Customer 1's last paid order is 2024-04-02, which is 28 days before April 30. The monetary values add to 12,660, the paid total we know from Chapter 3.
- **Answer** Customer 2 is the most recent (10 days); customer 1 spends the most (4,560).
- **Meaning** Customers 3, 7 and 8 do not appear: they have no paid orders, so they have no RFM at all. Decide what to do with them separately.

Problem 12.3.2 Score and name segments. Thresholds are business choices; here recency <= 20 days scores 3, <= 45 scores 2; frequency >= 4 scores 3, >= 2 scores 2; monetary >= 3000 scores 3, >= 1000 scores 2.

```
WITH rev AS (
  SELECT o.customer_id, o.order_id, o.order_date, SUM(oi.qty*oi.unit_price) AS amt
  FROM orders o JOIN order_items oi USING(order_id)
  WHERE o.status='paid' GROUP BY o.order_id),
rfm AS (
  SELECT customer_id,
    CAST(julianday('2024-04-30') - julianday(MAX(order_date)) AS INT) AS recency_days,
    COUNT(*) AS frequency, SUM(amt) AS monetary
  FROM rev GROUP BY customer_id),
sc AS (
  SELECT *,
   CASE WHEN recency_days<=20 THEN 3 WHEN recency_days<=45 THEN 2 ELSE 1 END AS r,
   CASE WHEN frequency>=4 THEN 3 WHEN frequency>=2 THEN 2 ELSE 1 END AS f,
   CASE WHEN monetary>=3000 THEN 3 WHEN monetary>=1000 THEN 2 ELSE 1 END AS m
  FROM rfm)
SELECT customer_id, recency_days, frequency, monetary, r,f,m,
  CASE WHEN r=3 AND f>=2 AND m>=2 THEN 'Champion'
       WHEN r=1 THEN 'At risk' WHEN f=1 THEN 'New / one-time' ELSE 'Regular' END AS segment
FROM sc ORDER BY customer_id;
```

```
customer_id  recency_days  frequency  monetary  r  f  m  segment
-----------  ------------  ---------  --------  -  -  -  --------------
1            28            4          4560      2  3  3  Regular
2            10            3          3600      3  2  3  Champion
4            21            2          3300      2  2  3  Regular
5            15            2          760       3  2  1  Regular
6            31            1          440       2  1  1  New / one-time
```

- **Answer** Customer 2 is our only Champion. Customer 6 bought once.
- **Meaning** The `CASE` order matters: the first true branch wins (Chapter 6).

Problem 12.3.3 (combine) A simple lifetime-value estimate and the median-style split with `NTILE`.

Formula	Average order value (AOV) = paid revenue / paid orders = 12,660 / 12 = 1,055.
Formula	Simple CLV = AOV x orders per customer per year x expected years.

```
WITH rev AS (SELECT o.customer_id, SUM(oi.qty*oi.unit_price) m
  FROM orders o JOIN order_items oi USING(order_id)
  WHERE o.status='paid' GROUP BY o.customer_id)
SELECT customer_id, m, NTILE(2) OVER (ORDER BY m DESC) AS half FROM rev;
```

```
customer_id  m     half
-----------  ----  ----
1            4560  1
2            3600  1
4            3300  1
5            760   2
6            440   2
```

- **Check** 5 rows into 2 buckets gives 3 and 2 (the extra row goes to the first bucket).
- **Answer** The top half holds three customers who give 11,460 of 12,660: about 90.5% of revenue.
- **Meaning** A few customers carry most revenue, the usual "80/20" shape.
- **Limit** Real CLV uses retention curves (12.2) and margins, not just revenue. A formula with one AOV hides differences between customers.

COMMON MISTAKE	Computing recency with `now()`. Your answer changes every day and nobody can reproduce last month's report. Pass a fixed as-of date.

Connection to the real job

CRM and marketing teams use RFM to choose who gets a discount or a win-back email. It needs no machine learning, which is why it is still popular.

Applications

- **Marketing analytics:** target champions with loyalty offers, at-risk customers with win-back.
- **Data Science:** R, F, M are strong features for churn models.
- **Finance:** concentration risk (how much revenue comes from the top 3 customers).

Check yourself

1. What does high recency days mean? Answer: the customer bought long ago (bad).
2. Why aggregate by order before counting frequency? Answer: otherwise each order line counts as an order.
3. Difference between `CASE` scoring and `NTILE`? Answer: `CASE` uses thresholds you choose; `NTILE` forces equal-sized groups.

12.4 Period-over-Period: Running Totals, MoM and YoY

INTUITION	"Revenue is 2,980." Is that good? Only compared with something: last month (MoM, month-over-month), or the same month last year (YoY, year-over-year). YoY removes seasonality: December is always strong, so compare December with December.

How it works

- **Growth % =** (this period - previous period) / previous period x 100.
- `LAG(value, 1) OVER (ORDER BY period)` brings the previous row's value onto the current row.
- A **running total** is `SUM(...) OVER (ORDER BY period)`; a **moving average** adds a frame such as `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW`.

Worked examples

Problem 12.4.1 MoM growth, running total and 2-month moving average of paid revenue.

```
WITH m AS (
  SELECT substr(o.order_date,1,7) AS ym, SUM(oi.qty*oi.unit_price) AS rev
  FROM orders o JOIN order_items oi USING(order_id)
  WHERE o.status='paid' GROUP BY 1)
SELECT ym, rev,
  LAG(rev) OVER (ORDER BY ym) AS prev_rev,
  ROUND(100.0*(rev-LAG(rev) OVER (ORDER BY ym))/LAG(rev) OVER (ORDER BY ym),1) AS mom_pct,
  SUM(rev) OVER (ORDER BY ym) AS running,
  ROUND(AVG(rev) OVER (ORDER BY ym ROWS BETWEEN 1 PRECEDING AND CURRENT ROW),1) AS avg2
FROM m ORDER BY ym;
```

```
ym       rev   prev_rev  mom_pct  running  avg2
-------  ----  --------  -------  -------  ------
2024-01  2260                     2260     2260.0
2024-02  2520  2260      11.5     4780     2390.0
2024-03  2980  2520      18.3     7760     2750.0
2024-04  4900  2980      64.4     12660    3940.0
```

- **Compute** March: (2980 - 2520) / 2520 = 460 / 2520 = 0.1825, so 18.3%. April: (4900 - 2980) / 2980 = 1920 / 2980 = 0.6443, so 64.4%.

- **Check** The last running total, 12,660, equals the paid total. January has no previous month, so growth is `NULL`: correct, not zero.
- **Answer** Revenue grew in every month; April grew 64.4%.
- **Meaning** The average of the first row uses only one row because the frame cannot reach back.

Problem 12.4.2 (YoY, the trap) A table of monthly sales for two years, with 4 months each.

```
CREATE TABLE monthly_sales (ym TEXT, rev INTEGER);
INSERT INTO monthly_sales VALUES
('2023-01',1000),('2023-02',1100),('2023-03',1200),('2023-04',1500),
('2024-01',1300),('2024-02',1210),('2024-03',1800),('2024-04',1500);
SELECT ym, rev, LAG(rev,12) OVER (ORDER BY ym) AS wrong_lag12
FROM monthly_sales ORDER BY ym;
```

```
ym       rev   wrong_lag12
-------  ----  -----------
2023-01  1000
2023-02  1100
2023-03  1200
2023-04  1500
2024-01  1300
2024-02  1210
2024-03  1800
2024-04  1500
```

`LAG(rev,12)` counts 12 **rows** back, not 12 months. Here only 8 rows exist, so all values are `NULL`. If a month were missing, it would silently pick the wrong month. The safe method joins on the calendar:

```
SELECT c.ym, c.rev, p.rev AS last_year,
  ROUND(100.0*(c.rev-p.rev)/p.rev,1) AS yoy_pct
FROM monthly_sales c
LEFT JOIN monthly_sales p
  ON p.ym = strftime('%Y-%m', date(c.ym||'-01','-1 year'))
WHERE c.ym >= '2024-01' ORDER BY c.ym;
```

```
ym       rev   last_year  yoy_pct
-------  ----  ---------  -------
2024-01  1300  1000       30.0
2024-02  1210  1100       10.0
2024-03  1800  1200       50.0
2024-04  1500  1500       0.0
```

- **Check** March: (1800 - 1200) / 1200 = 0.5, so 50%. April: 0 change.
- **Answer** YoY growth is 30%, 10%, 50% and 0%.
- **Meaning** Compared with last year, April stood still, even though April grew strongly against March (MoM).
- **Fix for gaps** Build a **date spine** (a table with one row per month, from a recursive CTE, Chapter 5) and LEFT JOIN data onto it, so missing months appear as rows with 0 or `NULL` (the spine query is standard; PostgreSQL has `generate_series` instead, not run here).

The picture

```
revenue
4900 |                    ##
2980 |             ##     ##
2520 |      ##     ##     ##
2260 | ##   ##     ##     ##
     +----------------------------
       Jan  Feb    Mar    Apr
MoM compares each bar with the bar to its left.
YoY compares it with the same month a year earlier (different chart row).
```

Connection to the real job

Every business dashboard has a "vs last month" and "vs last year" number. Getting them wrong is a classic incident: a `LAG` over data with missing months.

Applications

- **Data Analytics:** weekly, monthly and yearly KPI reports.
- **Finance:** budget versus actual, year-to-date running totals.
- **Logistics:** compare this month's 95th-percentile delivery time with last month's (the same formula).

COMMON MISTAKE	Dividing by zero or by `NULL`. If last year was 0, `(c.rev-p.rev)/p.rev` fails or gives `NULL`. Guard with `NULLIF(p.rev,0)`.

Check yourself

1. Why is growth in the first month `NULL`? Answer: there is no earlier row to compare with.
2. When is `LAG(x,12)` wrong for YoY? Answer: when months are missing, since it counts rows, not months.
3. Why is `SUM() OVER (ORDER BY ym)` a running total? Answer: the default frame runs from the first row to the current row.

12.5 Sessionization and Gaps and Islands

INTUITION	You scroll a shopping app for 20 minutes, go for lunch, then return. Is that one visit or two? Analysts say a new **session** starts after a long silence, often 30 minutes. A similar puzzle: a gym app wants "longest streak of consecutive days". Both are "break a sorted list wherever there is a gap". Each unbroken stretch is an **island**; the breaks are **gaps**.

How it works (sessions)

1. For each click, compute the gap in minutes from the previous click of the same user (`LAG`).
2. Mark `1` when the gap is missing or longer than 30 minutes (a session start), else `0`.
3. A running `SUM` of those marks gives the session number.

Worked examples

Problem 12.5.1 Compute the gaps.

```
CREATE TABLE clicks (user_id INTEGER, ts TEXT);
INSERT INTO clicks VALUES
(1,'2024-05-01 09:00:00'),(1,'2024-05-01 09:10:00'),(1,'2024-05-01 09:25:00'),
(1,'2024-05-01 11:00:00'),(1,'2024-05-01 11:20:00'),
(1,'2024-05-02 08:00:00'),
(2,'2024-05-01 09:05:00'),(2,'2024-05-01 09:50:00');

SELECT user_id, ts,
  (strftime('%s',ts) - strftime('%s', LAG(ts) OVER (PARTITION BY user_id ORDER BY ts)))/60 AS gap_min
FROM clicks ORDER BY user_id, ts;
```

```
user_id  ts                  gap_min
-------  ------------------  -------
1        2024-05-01 09:00:00
1        2024-05-01 09:10:00  10
1        2024-05-01 09:25:00  15
1        2024-05-01 11:00:00  95
1        2024-05-01 11:20:00  20
1        2024-05-02 08:00:00  1240
2        2024-05-01 09:05:00
2        2024-05-01 09:50:00  45
```

- **Step 1** `strftime('%s', ...)` turns text time into seconds, so subtraction is exact (SQLite specific; PostgreSQL uses `EXTRACT(EPOCH FROM ...)`, not run here).
- **Check** 09:25 to 11:00 is 1h35m = 95 minutes.
- **Answer** The first click of each user has no gap; big gaps are 95, 1240 and 45.

Problem 12.5.2 Assign sessions (30-minute rule) and summarise.

```
WITH g AS (
 SELECT user_id, ts,
  (strftime('%s',ts) - strftime('%s', LAG(ts) OVER (PARTITION BY user_id ORDER BY ts)))/60 AS gap_min
 FROM clicks),
f AS (SELECT *, CASE WHEN gap_min IS NULL OR gap_min > 30 THEN 1 ELSE 0 END AS new_s FROM g),
s AS (SELECT *, SUM(new_s) OVER (PARTITION BY user_id ORDER BY ts) AS session_no FROM f)
SELECT user_id, session_no, MIN(ts) AS start_ts, MAX(ts) AS end_ts, COUNT(*) AS clicks,
  (strftime('%s',MAX(ts)) - strftime('%s',MIN(ts)))/60 AS minutes
FROM s GROUP BY user_id, session_no ORDER BY user_id, session_no;
```

```
user_id  session_no  start_ts            end_ts              clicks  minutes
-------  ----------  ------------------  ------------------  ------  -------
1        1           2024-05-01 09:00:00  2024-05-01 09:25:00  3       25
1        2           2024-05-01 11:00:00  2024-05-01 11:20:00  2       20
1        3           2024-05-02 08:00:00  2024-05-02 08:00:00  1       0
2        1           2024-05-01 09:05:00  2024-05-01 09:05:00  1       0
2        2           2024-05-01 09:50:00  2024-05-01 09:50:00  1       0
```

- **Check** User 1's clicks split at the 95 and 1240 minute gaps: 3 + 2 + 1 = 6 clicks. User 2 splits at 45 minutes.
- **Answer** 5 sessions in total. A single-click session has duration 0, a known limit: you cannot measure time on the last page.

Problem 12.5.3 Gaps and islands: longest login streak. Trick: subtract the row number (in days) from each date. In a run of consecutive days the result is constant.

```
CREATE TABLE logins (user_id INTEGER, d TEXT);
INSERT INTO logins VALUES (1,'2024-05-01'),(1,'2024-05-02'),(1,'2024-05-03'),
  (1,'2024-05-06'),(1,'2024-05-07'),(1,'2024-05-09');
WITH r AS (SELECT user_id, d, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY d) rn FROM logins),
i AS (SELECT *, date(d, '-'||rn||' days') AS grp FROM r)
SELECT user_id, MIN(d) AS streak_start, MAX(d) AS streak_end, COUNT(*) AS days
FROM i GROUP BY user_id, grp ORDER BY streak_start;
```

```
user_id  streak_start  streak_end  days
-------  ------------  ----------  ----
1        2024-05-01    2024-05-03  3
1        2024-05-06    2024-05-07  2
1        2024-05-09    2024-05-09  1
```

- **Why it works** For 05-01, 05-02, 05-03 the row numbers are 1, 2, 3. Subtracting gives 04-30 each time. After the gap (05-06 with row 4) we get 05-02, a new value, so a new island.
- **Answer** The longest streak is 3 days (May 1 to 3).
- **Limit** The trick needs one row per day (use `DISTINCT` first). In PostgreSQL write `d - rn` with a date column (not run here).

The picture

```
 time -->  9:00  9:10  9:25           11:00  11:20            next day 8:00
 gap:        -    10    15     95       -> 20          1240
 new_s:      1     0     0      1          0             1
 session:    1     1     1      2          2             3
```

The gap row above each click is the silence before it. A `1` starts a new session. The running sum counts how many starts we have seen so far.

Connection to the real job

Web and app analytics tools compute sessions in this way. A session is like one visit to a shop: a long gap between actions means the visitor left and later came back as a new visit.

Applications

- **Product analytics:** sessions per user, pages per session, session length.
- **Data Engineering:** building a `sessions` table from raw click logs.
- **Operations:** find outage windows (islands of consecutive error minutes).

COMMON MISTAKE	Forgetting `PARTITION BY user_id` in `LAG`. Then the first click of user 2 is compared with the last click of user 1, giving a nonsense gap.

Check yourself

1. Why do we mark a session start when the gap is `NULL`? Answer: the first click has no previous click, so it must start session 1.
2. What is constant inside an island? Answer: date minus row number.
3. Why does a one-click session have duration 0? Answer: start and end are the same timestamp.

12.6 Deduplication, Late Data and Data-Quality Queries

INTUITION

Before you trust a report, you inspect the ingredients, as a cook sniffs the milk. Duplicate rows, missing values, impossible numbers and orphan references are the four common problems. You find each with a short query. Keep these queries and run them after each data load; this is "testing for data".

How it works

- **Deduplication:** `ROW_NUMBER() OVER (PARTITION BY the-real-key ORDER BY the-tie-breaker)`, keep `rn = 1`.
- **Late data:** records that arrive after their period closed. Keep an `updated_at` or `loaded_at` column and pick the latest version per key.
- **Checks:** count NULLs, duplicates, out-of-range values; find orphans with an anti-join.

Running table:

```
CREATE TABLE signups_raw (id INTEGER, email TEXT, plan TEXT, updated_at TEXT, age INTEGER);
INSERT INTO signups_raw VALUES
(1,'asha@x.com','free','2024-05-01',29),
(2,'ravi@x.com','free','2024-05-01',31),
(3,'ravi@x.com','pro','2024-05-03',31),
(4,'Meena@x.com','pro','2024-05-02',NULL),
(5,'meena@x.com ','pro','2024-05-04',27),
(6,'john@x.com','free','2024-05-02',250),
(7,NULL,'free','2024-05-02',40);
```

Look carefully: Meena appears twice with different case and a trailing space. Ravi appears twice (an upgrade). Row 6 has age 250. Row 7 has no email.

Worked examples

Problem 12.6.1 Keep the newest row per person. Predict: 4 people, so 4 rows.

```
WITH r AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY lower(trim(email))
                               ORDER BY updated_at DESC, id DESC) AS rn
  FROM signups_raw WHERE email IS NOT NULL)
SELECT id, lower(trim(email)) AS email, plan, updated_at FROM r WHERE rn=1 ORDER BY id;
```

```
id  email        plan  updated_at
--  -----------  ----  ----------
1   asha@x.com   free  2024-05-01
3   ravi@x.com   pro   2024-05-03
5   meena@x.com  pro   2024-05-04
6   john@x.com   free  2024-05-02
```

- **Step 1** Normalise before comparing: `lower(trim(email))` makes 'Meena@x.com' and 'meena@x.com ' the same person.
- **Step 2** `ORDER BY updated_at DESC, id DESC`: newest first; `id` breaks ties so the result is stable.
- **Check** 7 rows, minus 1 null email, minus 2 older duplicates = 4.
- **Answer** Ravi is kept as `pro`, Meena as the May 4 row.
- **Meaning** Without the `lower(trim())`, Meena would survive twice and your user count would be wrong by one.

Problem 12.6.2 A data-quality report in one row.

```
SELECT COUNT(*) AS total_rows,
  SUM(email IS NULL) AS null_email,
  COUNT(email) - COUNT(DISTINCT lower(trim(email))) AS dup_extra_rows,
  SUM(age IS NULL) AS null_age,
  SUM(age < 0 OR age > 120) AS bad_age
FROM signups_raw;
```

```
total_rows  null_email  dup_extra_rows  null_age  bad_age
----------  ----------  --------------  --------  -------
7           1           2               1         1
```

- **Check** `COUNT(email)` ignores NULL: 6. Distinct cleaned emails: 4. So 2 extra rows. `age IS NULL` is only row 4. `age > 120` is only row 6. Note `age IS NULL` rows are not counted in `bad_age`, since a comparison with NULL is not true (Chapter 2).
- **Answer** 1 missing email, 2 extra duplicate rows, 1 missing age, 1 impossible age.
- **Meaning** Report these numbers to the data owner before building on the table.

Problem 12.6.3 (combine) Orphans and the median. Orphans: orders pointing to a customer that does not exist.

```
CREATE TABLE o2 (order_id INTEGER, customer_id INTEGER);
INSERT INTO o2 VALUES (1,1),(2,2),(3,99);
CREATE TABLE c2 (customer_id INTEGER PRIMARY KEY);
INSERT INTO c2 VALUES (1),(2);
SELECT o.* FROM o2 o LEFT JOIN c2 c USING (customer_id) WHERE c.customer_id IS NULL;
```

```
order_id  customer_id
--------  -----------
3         99
```

The median (the middle value, robust to outliers) of hospital appointment waiting times 120, 130, 150, 160, 170, 900 minutes:

```
CREATE TABLE waits (minutes INTEGER);
INSERT INTO waits VALUES (120),(130),(150),(160),(900),(170);
WITH o AS (SELECT minutes, ROW_NUMBER() OVER (ORDER BY minutes) rn, COUNT(*) OVER () n FROM waits)
SELECT AVG(minutes) AS median_minutes FROM o WHERE rn IN ((n+1)/2, (n+2)/2);
SELECT AVG(minutes) AS mean_minutes FROM waits;
```

```
median_minutes
--------------
155.0
mean_minutes
----------------
271.666666666667
```

- **Why the formula works** With n = 6, `(n+1)/2 = 3` and `(n+2)/2 = 4` (integer division), so we average the 3rd and 4th values: (150 + 160) / 2 = 155. With odd n, both expressions give the same row.
- **Answer** Orphan order 3; median 155 minutes against a mean of about 271.7 minutes.
- **Meaning** One very long wait (900 minutes) drags the mean far from the typical patient. This is why delivery and waiting-time reports quote percentiles. PostgreSQL has `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY minutes)` (not run here).

The picture

```
 raw table --> [dedupe] --> [checks: nulls, ranges, orphans] --> trusted table
                 |                    |
           ROW_NUMBER()          counts that must be 0
```

Connection to the real job

Data engineers wrap such queries in tests (dbt tests, Great Expectations) that run after every load. An analyst who runs them first avoids embarrassing corrections.

Applications

- **Data Engineering:** dedupe CDC (change data capture) feeds by `updated_at`.
- **Data Science:** clean training data before modelling.
- **Compliance:** prove that every order has a customer.

COMMON MISTAKE	Using `SELECT DISTINCT *` to dedupe. It removes only rows that are identical in every column; the Ravi rows differ in `plan`, so both stay. Pick the business key and a rule for the winner.

Check yourself

1. Why include a tie-breaker in `ORDER BY` of `ROW_NUMBER`? Answer: equal timestamps would give an unpredictable winner.
2. How do you find orphans? Answer: LEFT JOIN to the parent and keep rows where the parent key is NULL.
3. Why use median instead of mean for waiting times? Answer: outliers do not move the median much.

12.7 Advanced Pattern Library: Top-N, Graphs, Medians, Intervals and Division

INTUITION	Senior analysts do not invent a new query for every question. They recognise a shape: 'best per group', 'path through a network', 'typical value', 'merge overlapping ranges', 'who did all of these'. Each shape has a standard solution. This section is a pattern library: for each shape you get the picture, the query, real output on the shop data or a small table, and the trap.

Everything here ran on SQLite 3.50.4 against the shop database from Chapter 2 (or the small tables created in the example). PostgreSQL-only alternatives are shown but marked not run here.

Pattern 1: top N per group

Problem 12.7.1 Find the two best-selling products by paid revenue in each category. Predict first: Electronics has two products, so both appear; Home has only one sold product; Stationery has two.

```
WITH rev AS (
  SELECT p.category, p.name, sum(i.qty * i.unit_price) AS revenue
  FROM order_items i
  JOIN orders o   ON o.order_id = i.order_id AND o.status = 'paid'
  JOIN products p ON p.product_id = i.product_id
  GROUP BY p.category, p.name
),
ranked AS (
  SELECT *, row_number() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
  FROM rev
)
SELECT category, name, revenue, rn FROM ranked WHERE rn <= 2 ORDER BY category, rn;
```

```
category     name       revenue  rn
-----------  ---------  -------  --
Electronics  Keyboard   4500     1
Electronics  Mouse      3000     2
Home         Desk Lamp  3600     1
Stationery   Pen Pack   840      1
Stationery   Notebook   720      2
```

Step 1	The CTE `rev` sums paid revenue per product. Cancelled and refunded orders are filtered in the `JOIN` condition.
Step 2	`row_number() OVER (PARTITION BY category ORDER BY revenue DESC)` restarts the count at 1 inside each category.
Step 3	The outer `WHERE rn <= 2` keeps the top two. Electronics: Keyboard 4500 (1500 x 3 paid orders), Mouse 3000. Home: Desk Lamp 3600, and Mug never sold, so it has no row. Stationery: Pen Pack 840, Notebook 720.
Answer	Five rows. A group with fewer than N items returns what it has.

COMMON MISTAKE

Using `row_number()` when ties matter. If two products tie on revenue, `row_number()` breaks the tie arbitrarily, so a rerun can return a different product. Use `rank()` or `dense_rank()` to keep all tied rows (you may then get more than N rows), or add a unique second sort key such as the product id to make the order stable. Also, a window function cannot be used directly in `WHERE`, which is why the ranking sits in a CTE.

PostgreSQL and MySQL 8 also offer `CROSS JOIN LATERAL`: `FROM categories c CROSS JOIN LATERAL (SELECT ... WHERE category = c.category ORDER BY revenue DESC LIMIT 2) t` (not run here). It can be faster when you already have a small list of groups and an index on the sort column, because it stops after N rows per group instead of ranking everything.

Pattern 2: paths through a graph, with cycle protection

Problem 12.7.2 Flights connect cities, and the network has a loop (BLR back to PNQ). Find the three fastest routes from PNQ to BLR. Predict first: the direct-looking route through BOM, 0.5 + 1.7 = 2.2 hours, wins.

```
CREATE TABLE flights (src TEXT, dst TEXT, hours REAL);
INSERT INTO flights VALUES ('PNQ','BOM',0.5),('BOM','DEL',2),('PNQ','DEL',2.5),
  ('DEL','BLR',2.5),('BOM','BLR',1.7),('BLR','PNQ',1.4),('DEL','BOM',2);
```

```
WITH RECURSIVE route(city, hops, hrs, path) AS (
  SELECT 'PNQ', 0, 0.0, ',PNQ,'
  UNION ALL
  SELECT f.dst, r.hops + 1, r.hrs + f.hours, r.path || f.dst || ','
  FROM route r JOIN flights f ON f.src = r.city
  WHERE r.path NOT LIKE '%,' || f.dst || ',%'   -- cycle guard: never revisit a city
    AND r.hops < 4                               -- depth limit: a second safety net
)
SELECT trim(path, ',') AS path, hops, hrs FROM route WHERE city = 'BLR' ORDER BY hrs LIMIT 3;
```

```
path            hops  hrs
--------------  ----  ---
PNQ,BOM,BLR     2     2.2
PNQ,DEL,BLR     2     5.0
PNQ,BOM,DEL,BLR 3     5.0
```

Step 1	The recursive CTE starts at PNQ with an empty route. Each round follows one more flight and adds its hours.
Step 2	The path is kept as text, `,PNQ,BOM,` , and the guard `NOT LIKE '%,' || dst || ',%'` refuses to visit any city already in the path. That is the cycle protection.
Step 3	The depth limit `hops < 4` is a second safety net, because one bad row could still make a recursion run for ever.
Step 4	Filter to arrivals at BLR and sort by hours. Best is PNQ,BOM,BLR at 2.2 hours. Two routes tie at 5.0 hours (PNQ,DEL,BLR is 2.5 + 2.5, PNQ,BOM,DEL,BLR is 0.5 + 2 + 2.5).
Answer	A shortest path by total weight, found by listing every acyclic path and sorting. That is fine for small graphs, and a bad idea for large ones (see below).

To see why the guard matters, run the same walk with only a depth limit and no cycle check:

```
WITH RECURSIVE route(city, hops) AS (
  SELECT 'PNQ', 0
  UNION ALL
  SELECT f.dst, r.hops + 1 FROM route r JOIN flights f ON f.src = r.city
  WHERE r.hops < 5
)
SELECT count(*) AS rows_without_cycle_guard FROM route;
```

```
rows_without_cycle_guard
------------------------
41
```

With 7 flights and a loop, the walk produced 41 rows by depth 5, because it keeps travelling around the cycle. Without any limit it would never stop (SQLite stops when you press cancel; PostgreSQL runs until memory or time runs out unless you add a limit). PostgreSQL 14 added `CYCLE city SET is_cycle USING path` to do this bookkeeping for you (not run here).

GOING DEEPER	Listing every path grows exponentially with the size of the graph, so this pattern suits hundreds of nodes, not millions. Real shortest-path problems on large graphs belong in a graph database (Chapter 13) or in code (Dijkstra's algorithm). Inside SQL it is still the right tool for hierarchies, which are trees (no cycles), and for 'can I get from A to B in at most 3 steps' questions.

Pattern 3: bill of materials, exploding quantities

Problem 12.7.3 A bike needs 1 frame and 2 wheels. A wheel needs 1 rim, 32 spokes, 1 tyre and 2 bolts. A frame needs 3 tubes and 8 bolts. How many of each raw part for one bike? Predict first: spokes 2 x 32 = 64, bolts 8 + 2 x 2 = 12.

```
CREATE TABLE bom (parent TEXT, child TEXT, qty INTEGER);
INSERT INTO bom VALUES ('bike','frame',1),('bike','wheel',2),('wheel','rim',1),('wheel','spoke',32),
  ('wheel','tyre',1),('frame','tube',3),('frame','bolt',8),('wheel','bolt',2);
WITH RECURSIVE x(part, need) AS (
  SELECT 'bike', 1
  UNION ALL
  SELECT b.child, x.need * b.qty FROM x JOIN bom b ON b.parent = x.part
)
SELECT part, sum(need) AS total FROM x
WHERE part NOT IN (SELECT parent FROM bom)      -- keep only raw parts (leaves)
GROUP BY part ORDER BY part;
```

```
part   total
-----  -----
bolt   12
rim    2
spoke  64
tube   3
tyre   2
```

Step 1	Start with 1 bike. Each round joins the BOM and multiplies the quantity needed by the quantity per parent: `x.need * b.qty`.
Step 2	A part such as `bolt` is reached by two routes (via frame: 8, via wheel: 2 wheels x 2 bolts = 4), so it appears twice in the recursion. That is why we `sum()` at the end.
Step 3	Keep only leaves: parts that are never a parent. Result: bolt 12, rim 2, spoke 64, tube 3, tyre 2.
Answer	The multiplication in the recursive step is the whole trick. The same pattern computes org-chart cost roll-ups and recipe ingredient lists.

Pattern 4: median and percentiles, and why the mean lies

Problem 12.7.4 Ten page-load times in milliseconds contain two slow outliers. Compute the mean, the median and the 90th percentile. Predict first: the mean is dragged far above the typical value by the 900.

```
CREATE TABLE lat (id INTEGER PRIMARY KEY, ms INTEGER);
INSERT INTO lat (ms) VALUES (12),(15),(11),(250),(14),(16),(13),(18),(12),(900);
WITH s AS (SELECT ms, row_number() OVER (ORDER BY ms) AS rn, count(*) OVER () AS n FROM lat)
SELECT (SELECT avg(ms) FROM lat) AS mean_ms,
       (SELECT avg(ms) FROM s WHERE rn IN ((n + 1) / 2, (n + 2) / 2)) AS median_ms,
       (SELECT min(ms) FROM s WHERE rn >= 0.9 * n) AS p90_ms;
```

```
mean_ms  median_ms  p90_ms
-------  ---------  ------
126.1    14.5       250
```

Step 1	Sorted: 11, 12, 12, 13, 14, 15, 16, 18, 250, 900. The mean is 1261 / 10 = 126.1.
Step 2	The median of 10 values is the average of the 5th and 6th: (14 + 15) / 2 = 14.5. The trick `rn IN ((n+1)/2, (n+2)/2)` picks the one middle row when n is odd and the two middle rows when n is even (integer division).
Step 3	The 90th percentile by the nearest-rank method is the value at position 0.9 x 10 = 9, which is 250.
Answer	The mean says 126 ms, which describes nobody. The median 14.5 ms is the typical user and the p90 of 250 ms is what one user in ten feels. Report medians and percentiles for latency.

PostgreSQL has `percentile_cont(0.5) WITHIN GROUP (ORDER BY ms)`, which interpolates between neighbours, and `percentile_disc`, which returns an actual value. SQL Server, Oracle and Snowflake have similar functions. The nearest-rank query above is the portable version. The two methods can give slightly different p90 values, so state which one you use (not run here).

Pattern 5: merging overlapping intervals

Problem 12.7.5 Room bookings are given as start and end times. Merge overlapping or touching bookings into blocks per room, and also list every pair that overlaps. Predict first: room A has 09:00 to 12:00 (three chained bookings) and 13:00 to 14:00 (13:30 to 13:45 sits inside it).

```
CREATE TABLE booking (room TEXT, s TEXT, e TEXT);
INSERT INTO booking VALUES ('A','09:00','10:00'),('A','09:30','11:00'),('A','11:00','12:00'),
  ('A','13:00','14:00'),('A','13:30','13:45'),('B','09:00','09:30');
WITH m AS (
  SELECT *, max(e) OVER (PARTITION BY room ORDER BY s, e
                         ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prev_end
  FROM booking
),
g AS (
  SELECT *, sum(CASE WHEN prev_end IS NULL OR s > prev_end THEN 1 ELSE 0 END)
            OVER (PARTITION BY room ORDER BY s, e) AS grp
  FROM m
)
SELECT room, min(s) AS start, max(e) AS finish FROM g GROUP BY room, grp ORDER BY room, start;
```

```
room  start  finish
----  -----  ------
A     09:00  12:00
A     13:00  14:00
B     09:00  09:30
```

Step 1	Sort bookings by start inside each room. For each, `prev_end` is the latest end among all earlier bookings in that room, via `max(e)` over a frame that stops one row before the current one.
Step 2	A booking starts a *new* block when it begins after everything before it has finished (`s > prev_end`), or when it is the first. The running sum of those flags is a block number: the gaps-and-islands idea from Section 12.5.
Step 3	Group by block: min start, max end. Room A: 09:00 to 12:00 and 13:00 to 14:00. Room B: 09:00 to 09:30.
Step 4	The 11:00 booking starts exactly when 09:30-11:00 ends. We used `>` , so touching bookings merge. Use `>=` if back-to-back bookings should stay separate.
Overlap test	Two intervals overlap when `a.s < b.e AND b.s < a.e` , the one condition to memorise. The pair list below shows the two overlaps, and `a.rowid < b.rowid` stops each pair appearing twice.

```
SELECT a.room, a.s, a.e, b.s AS other_s, b.e AS other_e
FROM booking a JOIN booking b
  ON a.room = b.room AND a.rowid < b.rowid AND a.s < b.e AND b.s < a.e;
```

```
room  s      e      other_s  other_e
----  -----  -----  -------  -------
A     09:00  10:00  09:30    11:00
A     13:00  14:00  13:30    13:45
```

COMMON MISTAKE

Writing the overlap test as `a.s BETWEEN b.s AND b.e` . It misses the case where `a` starts before `b` and ends after it (a wraps b): neither of a's ends is between b's ends, yet they overlap. Always use the two-inequality form `a.s < b.e AND b.s < a.e` , which covers every arrangement. PostgreSQL has range types and the `&&` overlap operator (not run here).

Pattern 6: running distinct count

Problem 12.7.6 Visits by day: how many users per day, how many were seen for the first time that day, and how many distinct users so far? Predict first: day 1 has 2 users, day 2 adds user 3, day 3 adds user 4, so the running total is 2, 3, 4.

```
CREATE TABLE visit (day TEXT, uid INTEGER);
INSERT INTO visit VALUES ('d1',1),('d1',2),('d1',1),('d2',2),('d2',3),('d3',1),('d3',3),('d3',4);
WITH f AS (SELECT day, uid, row_number() OVER (PARTITION BY uid ORDER BY day) AS rn FROM visit)
SELECT day,
       count(DISTINCT uid)  AS daily_users,
       sum(rn = 1)          AS new_users,
       sum(sum(rn = 1)) OVER (ORDER BY day) AS cumulative_distinct
FROM f GROUP BY day ORDER BY day;
```

```
day  daily_users  new_users  cumulative_distinct
---  -----------  ---------  -------------------
d1   2            2          2
d2   2            1          3
d3   3            1          4
```

Step 1	`count(DISTINCT uid)` per day works for the daily number, but a window cannot do `count(DISTINCT ...) OVER` (PostgreSQL and SQLite reject it).
Step 2	The workaround: number each user's visits in time order with `row_number()`. The row with number 1 is the first time that user was ever seen, so `sum(rn = 1)` counts new users per day.
Step 3	A running sum of new users is the running distinct count: 2, 3, 4.
Answer	User 1 appears twice on day 1 (a duplicate event). The daily count is still 2 because of `DISTINCT`, and only the first of user 1's rows has `rn = 1` (the other has `rn = 2`), so user 1 counts once as new.

Pattern 7: relational division, 'who did all of these'

Problem 12.7.7 Which customers have bought every Electronics product (at least once, paid orders)? Predict first: Electronics has two products (Keyboard and Mouse), so we need customers with both: Asha and Ravi.

```
SELECT c.name
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id AND o.status = 'paid'
JOIN order_items i ON i.order_id = o.order_id
JOIN products p    ON p.product_id = i.product_id AND p.category = 'Electronics'
GROUP BY c.customer_id
HAVING count(DISTINCT p.product_id) = (SELECT count(*) FROM products WHERE category = 'Electronics');
```

```
name
----
Asha
Ravi
```

Step 1	Restrict to paid orders and to Electronics products with the join conditions.
Step 2	Group by customer and count *distinct* products bought. The distinct matters, because buying the keyboard twice is still one product.
Step 3	Compare to the total number of Electronics products from a subquery, so the query keeps working when the catalogue grows.
Answer	Asha and Ravi. John bought only the keyboard, so he fails the test.

The same shape answers 'users who completed every onboarding step', 'stores that stock all items on a list', and 'students who passed all required courses'. The pattern: count distinct matches per candidate, compare with the size of the required set.

Practice with answers

Problem 12.7.8 Why does the top-N pattern use a CTE or subquery instead of putting `rn <= 2` in the same query as the window function? *(easy)*

Answer: `WHERE` runs before window functions are computed, so `rn` does not exist yet at that stage. The window query goes in a CTE or subquery and the outer query filters it. (PostgreSQL has no `QUALIFY`, but BigQuery, Snowflake and DuckDB do and it removes the extra layer.)

Problem 12.7.9 In the bill of materials, what changes if a part can appear in two different parents at different depths and you forget to `sum()`? *(find the bug)*

Answer: You would list `bolt` twice (8 and 4) instead of once as 12, so purchasing would order too few bolts. The recursion correctly produces one row per path, and the final aggregation combines paths.

Problem 12.7.10 A table of 500 intervals per user needs the longest single block of continuous activity. Outline the query. *(medium)*

Answer: Merge intervals into blocks as in 12.7.5, then compute `max(finish - start)` per user over the blocks. In SQL: the grouped result is a CTE, and the outer query takes the maximum duration per user.

Problem 12.7.11 Why is a recursive path-listing query risky on a social network with millions of users? *(expert)*

Answer: The number of paths grows exponentially with depth (each person has many friends), so even depth 4 or 5 can create billions of rows, and the cycle guard that tests a growing path string costs more at every step. Use a graph database or an algorithm in code (breadth-first search with a visited set), or limit the depth to 2 or 3 and start from a single person.

Chapter recap

- A funnel counts distinct users who did each step in order; divide by the previous step to find the leak.
- A cohort is a group that started together; retention is the share of the cohort active n periods later. Unobserved months are not zeros.
- RFM scores recency, frequency and money; fix an as-of date and aggregate by order first.
- MoM compares with the previous period, YoY with the same period last year; `LAG(x,12)` counts rows, so use a calendar join when months can be missing.
- Sessions: gap from `LAG`, flag gap > threshold, running `SUM` gives the session number.
- Islands: date minus row number is constant inside a consecutive run.
- Dedupe with `ROW_NUMBER` over a normalised key; never rely on `DISTINCT *`.
- Data-quality queries count NULLs, duplicates, out-of-range values and orphans; run them after each load.

Pattern	Core SQL
Funnel	`MIN(CASE WHEN event=... THEN ts END)` per user, compare times
Retention	month index = `y*12+m` minus cohort index, `COUNT(DISTINCT)`
RFM	`MAX(date)`, `COUNT(*)`, `SUM(amt)` then `CASE` or `NTILE`
MoM	`LAG(x) OVER (ORDER BY period)`
YoY	self-join on `period - 1 year`
Session	`SUM(gap>30) OVER (PARTITION BY u ORDER BY ts)`
Island	`date - ROW_NUMBER()` as group key
Dedupe	`ROW_NUMBER() ... WHERE rn=1`
Orphans	`LEFT JOIN ... WHERE parent.id IS NULL`

Practice set

Problem 12.P.1 (easy) In the funnel data, how many users have a `cart` event? **Answer:** 4 (users 1, 2, 3, 5). Each user has one cart row, so `COUNT(*)` also gives 4, but `COUNT(DISTINCT user_id)` is the safe habit.

Problem 12.P.2 (easy) Overall view-to-pay conversion in the strict funnel. **Answer:** 2 paid of 6 viewers = 33.3%.

Problem 12.P.3 (what is the output?) The January cohort has 2 customers; 1 of them buys in February. What is month-1 retention? **Answer:** 1 / 2 = 50%, matching the table in 12.2.

Problem 12.P.4 (find the bug) A funnel query uses `COUNT(*)` on `events WHERE event='view'` and reports 7 viewers. What is wrong? **Answer:** User 5 viewed twice. Use `COUNT(DISTINCT user_id)`; the correct number is 6.

Problem 12.P.5 (find the bug) YoY with `LAG(rev,12)` returns NULL for every row of an 8-row table. Why, and what do you do? **Answer:** There are only 8 rows, so no row is 12 rows back; `LAG` counts rows, not months. Join the table to itself on the month one year earlier (12.4.2).

Problem 12.P.6 Using the click data, how many sessions exist if the timeout is 60 minutes instead of 30? **Answer:** 4. With a 60-minute rule the 45-minute gap of user 2 no longer splits, so user 2 has 1 session. User 1 still splits at 95 and 1240 minutes into 3 sessions. Total 3 + 1 = 4 (verified by running the query).

Problem 12.P.7 For each product category, which product has the highest paid revenue? **Answer:** Using `RANK() OVER (PARTITION BY category ORDER BY rev DESC)` and `rk = 1`: Electronics Keyboard 4500, Home Desk Lamp 3600, Stationery Pen Pack 840 (verified by running).

Problem 12.P.8 (scenario) Customer 3 has only a cancelled order. Does customer 3 belong to the January cohort, to churn, or to RFM? **Answer:** To none of them. All three patterns filter `status='paid'`, so customer 3 has no paid orders. Whether cancelled orders should count as activity is a business definition to agree before reporting.

Problem 12.P.9 (interview) Write the pattern to find users who logged in on at least 3 consecutive days. **Answer:** Make one row per user per day (`DISTINCT`), compute `d - ROW_NUMBER()` as the island key as in 12.5.3, group by user and key, and keep groups with `COUNT(*) >= 3`.

Problem 12.P.10 (interview) A table has duplicate rows per `order_id` from a retry bug. Delete-free way to get one row per order, keeping the latest `loaded_at`? **Answer:** `ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY loaded_at DESC)` in a CTE, then `WHERE rn = 1`. Mention a tie-breaker column.

Problem 12.P.11 (computation) Revenue was 2,520 in February and 2,980 in March. What is MoM growth, to one decimal? **Answer:** (2980 - 2520) / 2520 = 460 / 2520 = 0.1825 = 18.3%.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analytics and I know basic SQL joins, CTEs and window functions. Teach me funnel analysis in SQL step by step, using a small events table (user_id, event, ts). Show the "one row per user" step, the strict step order rule, and conversion percentages. After each step, ask me one question to check understanding before continuing.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning analytics SQL. Explain cohort retention in SQL with a tiny orders table. Cover cohort month, month index, the retention triangle and why missing cells are not zeros. Then give me 8 graded exercises without solutions, from easy to hard, and wait for my answers.
```

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional preparing for data analyst interviews. Quiz me one question at a time on analytics SQL patterns: RFM, MoM and YoY, sessionization, gaps and islands, deduplication. Give a small table with each question, grade my query out of 10, show a model answer and explain one common mistake.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional learning SQL for data work. Review my analytics query below for correctness: check joins for duplicate rows, window partitions, NULL handling, division by zero and time boundaries. Explain every problem in simple English and give a corrected version. My query and table sample: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles. Teach me data-quality queries in SQL: finding NULLs, duplicates, impossible values and orphan keys, and deduplicating with ROW_NUMBER. Use a messy 7-row sample table, show every query and its exact output, and then run a mock interview of five questions on data cleaning, grading each answer.

CHAPTER 13

BEYOND SQL

Document, key-value, wide-column, time-series, graph, search and vector databases: what each is good at, what each is bad at, and how to choose.

Sooner or later an interviewer or a senior engineer says: "Why not just put it in MongoDB?" or "We use Redis in front of Postgres. Why?" A strong data professional does not memorise product names. They ask one question: **how will this data be read and written?** Each database family below is built around one access pattern, and it gives up something else to be fast at that pattern. By the end of this chapter you can describe seven families in plain words, run small versions of their ideas in SQLite (so you feel how they work), and pick a store for a given problem using the CAP theorem and a simple decision table.

A promise about honesty: MongoDB, Redis, Cassandra, DynamoDB, Neo4j and Elasticsearch are not installed here. Their own commands are shown as short, standard snippets labelled "not run here". The ideas themselves are demonstrated with real SQLite output.

13.1 Document Stores (MongoDB)

INTUITION	A relational database is a filing cabinet of forms: the order form in one drawer, the item lines in another, joined by numbers. A document store is a folder per order: the customer, the status and all item lines are inside one folder. To read an order you open one folder, not three drawers. The price is that the same customer name is copied into many folders.

Words you need

Word	Meaning
Document	One record stored as JSON-like data (MongoDB stores it in a binary form called BSON)
Collection	A group of documents, like a table
Embed	Put related data inside the document
Reference	Store only the other document's id, like a foreign key
Aggregation pipeline	MongoDB's step-by-step query (`$match` , `$group` , ...)

How it works

You choose a shape per query pattern. **Embed** when the child data is always read with the parent and is small (order and its lines). **Reference** when the child is large, shared or changes often (a customer profile used by millions of orders). There is no join by default; MongoDB has `$lookup` , but it is slower than a relational join, so schema design avoids it.

We can feel this in SQLite, which can store JSON text (Chapter 6):

Worked examples

Problem 13.1.1 Store three orders as documents (items embedded) and read fields from them. Predict: three rows; order 104 has a coupon, the others do not.

```
CREATE TABLE orders_doc (id INTEGER PRIMARY KEY, doc TEXT);
INSERT INTO orders_doc VALUES
(101,'{"customer":{"id":1,"name":"Asha"},"status":"paid","items":[{"sku":"KB","qty":1,"price":1500},
{"sku":"NB","qty":2,"price":80}]}'),
(102,'{"customer":{"id":2,"name":"Ravi"},"status":"paid","items":[{"sku":"MS","qty":1,"price":600}]}'),
(104,'{"customer":{"id":3,"name":"Meena"},"status":"cancelled","items":
[{"sku":"NB","qty":5,"price":80}],"coupon":"SAVE10"}');
SELECT id, json_extract(doc,'$.customer.name') AS who, json_extract(doc,'$.status') AS status,
  json_array_length(doc,'$.items') AS n_items, json_extract(doc,'$.coupon') AS coupon
FROM orders_doc;
```

```
id   who    status     n_items  coupon
---  -----  ---------  -------  ------
101  Asha   paid       2
102  Ravi   paid       1
104  Meena  cancelled  1        SAVE10
```

- **Answer** One query on one table gives the whole order with no join. A missing key (`coupon` on 101) simply gives NULL. That flexibility (each document may have different fields) is the "schema-flexible" promise.
- **Meaning** Reading an order is one lookup. The equivalent in MongoDB (not run here): `db.orders.find({ "customer.name": "Asha" })` .

Problem 13.1.2 Compute the total of each order from the embedded items.

```
SELECT d.id, SUM(json_extract(j.value,'$.qty')*json_extract(j.value,'$.price')) AS total
FROM orders_doc d, json_each(d.doc,'$.items') j GROUP BY d.id;
```

```
id   total
---  -----
101  1660
102  600
104  400
```

- **Check** Order 101: 1 x 1500 + 2 x 80 = 1660. Order 104: 5 x 80 = 400. Both match Chapter 4's totals.
- **Answer** Totals are 1660, 600 and 400.
- **Meaning** The MongoDB version is an aggregation pipeline (not run here):

```
db.orders.aggregate([
  { $unwind: "$items" },
  { $group: { _id: "$_id", total: { $sum: { $multiply: ["$items.qty", "$items.price"] } } } }
])
```

`$unwind` is the same as `json_each` here: one row per array element.

The picture

```
 RELATIONAL                          DOCUMENT
 orders --< order_items >-- products    { _id:101, customer:{...},
 (3 tables, joins at read time)           items:[ {...}, {...} ] }
                                          (1 document, no join)
```

Connection to the real job

Application backends store user profiles, product catalogues and event payloads as documents. Data engineers meet MongoDB as a **source**: you export documents, flatten the arrays (as above) and load them into a warehouse.

Applications

- **Backend:** product catalogue where each product category has different attributes.
- **Data Engineering:** ingesting JSON from APIs and apps.
- **Content systems:** articles with comments embedded.

COMMON MISTAKE	Choosing a document store because "schema is annoying", then needing reports that join five collections. Flexible schema moves the work from write time to read time. If your questions are "revenue by customer city", a relational model is simpler.

Check yourself

1. When do you embed? Answer: when the child data is small and always read with the parent.
2. What is `$unwind` like in SQL? Answer: unnesting an array into rows, like `json_each` or `UNNEST`.
3. What is the cost of embedding customer details in every order? Answer: copies; a name change must be updated in many places.

13.2 Key-Value Stores and Caches (Redis)

INTUITION	A hotel key rack. You say the room number and get the key at once. You cannot ask "which keys are blue?". A **key-value store** gives the fastest possible "get by key" and nothing else. **Redis** keeps data in memory (RAM), so reads take well under a millisecond, but memory is small and costly compared with disk.

How it works

- Keys map to values. Redis values can be strings, hashes (small field-value maps), lists, sets, sorted sets (a set ordered by a score, used for leaderboards), and others.
- **TTL** (time to live): each key can expire automatically after N seconds.
- **Cache-aside pattern:** the application first asks the cache. On a **hit** it uses the value. On a **miss** it reads the database, writes the result to the cache with a TTL, and returns it.
- **Write-through:** every write goes to the cache and the database together, so the cache is never stale, at the cost of slower writes.
- **Invalidation:** deleting or updating a cached value when the source changes. It is famously hard: if you forget, users see stale data.

Worked examples

Problem 13.2.1 The speed benefit in numbers. A cache hit costs 1 ms. A miss costs 1 ms (the failed cache check) plus 20 ms (the database). With a 90% hit ratio, what is the average latency?

Formula	average = hit_ratio x hit_time + miss_ratio x miss_time
Plug in	0.9 x 1 + 0.1 x (1 + 20)
Compute	0.9 + 0.1 x 21 = 0.9 + 2.1 = 3.0
Answer	3.0 ms on average, against 20 ms with no cache, about 6.7 times faster.
Check	If the hit ratio were 0%, the formula gives 21 ms (slower than no cache by the 1 ms check). If it were 100%, it gives 1 ms. 3.0 lies between, as it should.
Meaning	For a service owner: the **tail** matters too. The 10% of requests at 21 ms still set your p95 and p99, so a cache improves the average more than the worst case.

Problem 13.2.2 TTL and rate limiting, shown with SQL tables (Redis equivalents below, not run here). First, a key-value table with expiry times (seconds since some clock). At time 1030, which keys are alive, and how long do they have left?

```
CREATE TABLE kv (k TEXT PRIMARY KEY, v TEXT, expires_at INTEGER);
INSERT INTO kv VALUES ('user:1','Asha',1000),('user:2','Ravi',1060);
SELECT k, v, expires_at-1030 AS ttl_left FROM kv WHERE expires_at > 1030;
```

```
k       v     ttl_left
------  ----  --------
user:2  Ravi  30
```

- **Answer** `user:1` expired at 1000, so only `user:2` is alive, with 30 seconds left.
- **Redis equivalent (not run here):** `SET user:2 Ravi EX 60` then `TTL user:2`.

Now a **fixed-window rate limit**: allow at most 3 requests per 60-second window per IP.

```
CREATE TABLE hits (ip TEXT, ts INTEGER);
INSERT INTO hits VALUES ('a',1),('a',2),('a',3),('a',4),('a',61),('b',5),('b',6);
SELECT ip, ts/60 AS minute_bucket, COUNT(*) AS n,
  CASE WHEN COUNT(*)>3 THEN 'BLOCK' ELSE 'ok' END AS verdict
FROM hits GROUP BY ip, ts/60;
```

```
ip  minute_bucket  n  verdict
--  -------------  -  -------
a   0              4  BLOCK
a   1              1  ok
b   0              2  ok
```

- **Check** IP `a` sent 4 requests in seconds 1 to 4 (bucket 0), over the limit of 3. The request at second 61 begins a new bucket.
- **Answer** Only IP `a` in the first minute is blocked.
- **Meaning** In Redis you do it with one counter per bucket (not run here): `INCR rate:a:0` and `EXPIRE rate:a:0 60`, then reject if the value is above 3. `INCR` is atomic, so two servers cannot both read "3" and both allow a request.

The picture

```
App --1. GET user:2--> [Cache RAM] --hit--> value (1 ms)
 |                          | miss
 |<--4. value---------------+-- 2. SELECT ... --> [Database] (20 ms)
 |                          <-- 3. SET user:2 value EX 60 --
```

Connection to the real job

Almost every large web system has a cache in front of the database. A shop that opens in the morning shows the "cold cache" problem: the first minute after a restart is slow because every request is a miss.

Applications

- **Web backend:** session storage, product page cache.
- **Gaming / analytics:** leaderboards with sorted sets.
- **Platform:** rate limiting, distributed locks, pub/sub messaging.
- **ML serving:** caching features or model answers.

COMMON MISTAKE	Treating Redis as the only copy of important data. By default it is a cache; if memory is lost, data can be lost (it has options to save to disk, but know what you configured). Keep the source of truth in a durable database.

Check yourself

1. What is a cache miss? Answer: the key is not in the cache, so the app must read the database.
2. Why is `INCR` good for rate limits? Answer: it is atomic, so concurrent requests cannot both read the same old value.
3. What does a TTL do? Answer: deletes the key automatically after a set time.

13.3 Wide-Column Stores (Cassandra, DynamoDB)

INTUITION	A post office with a PIN code for every street. Letters for one street are always stored together and in date order, so "latest 10 letters for Street 5" is one quick walk. But "all letters from Ravi, in any street" means walking every street. **Wide-column** stores spread data over many machines by a **partition key** and keep rows inside a partition sorted by a **clustering key**. They are built for huge write volume and predictable reads, and they refuse queries that do not follow the key.

Words you need

Word	Meaning
Partition key	Decides which machine(s) hold the row (hashed)
Clustering key	Decides the sort order of rows inside one partition
Query-first modelling	You design the table from the question you will ask, one table per question
Replication factor (N)	How many copies of each row exist
Quorum	A majority of the N copies

How it works

In Cassandra and DynamoDB you declare the key first: one table per access pattern, even if data is duplicated across tables. There are no joins. Reads must give the partition key. Consistency is **tunable**: with N copies, a write waits for W copies to confirm and a read asks R copies. If **R + W > N**, every read overlaps at least one copy that has the latest write.

Worked examples

Problem 13.3.1 Model "latest messages of a user". We imitate with a SQLite table whose primary key is (user_id, ts), which makes the rows of one user contiguous and time-sorted.

```
CREATE TABLE msgs_by_user (user_id INTEGER, ts INTEGER, body TEXT,
  PRIMARY KEY (user_id, ts)) WITHOUT ROWID;
INSERT INTO msgs_by_user VALUES (1,100,'hi'),(1,105,'yo'),(1,130,'ok'),(2,101,'hey'),(2,140,'bye');
SELECT * FROM msgs_by_user WHERE user_id=1 AND ts>=105 ORDER BY ts DESC LIMIT 2;
```

```
user_id  ts   body
-------  ---  ----
1        130  ok
1        105  yo
```

- **Answer** The two newest messages of user 1 from time 105 on.
- **Check** User 1 has messages at 100, 105, 130. From 105: 105 and 130, newest first: 130, 105. Correct.
- **The same in Cassandra's CQL (not run here):**

```
CREATE TABLE msgs_by_user (user_id int, ts int, body text,
  PRIMARY KEY ((user_id), ts)) WITH CLUSTERING ORDER BY (ts DESC);
SELECT * FROM msgs_by_user WHERE user_id = 1 AND ts >= 105 LIMIT 2;
```

Problem 13.3.2 What happens if you filter on a non-key column? Ask SQLite for the query plan (Chapter 10) of two queries.

```
EXPLAIN QUERY PLAN SELECT * FROM msgs_by_user WHERE user_id=1 AND ts>=105;
EXPLAIN QUERY PLAN SELECT * FROM msgs_by_user WHERE body='hey';
```

```
QUERY PLAN
`--SEARCH msgs_by_user USING PRIMARY KEY (user_id=? AND ts>?)
QUERY PLAN
`--SCAN msgs_by_user
```

- **Answer** The first query jumps straight to the right place (SEARCH). The second reads every row (SCAN).
- **Meaning** On one laptop a scan is cheap. In Cassandra, a scan means asking every machine, so the database refuses (or warns) unless you add `ALLOW FILTERING`. The lesson: design the key from the question.

Problem 13.3.3 Quorum arithmetic. N = 3 copies. Is W = 2, R = 2 strongly consistent? What about W = 1, R = 1?

Formula	consistent reads when R + W > N
Plug in	2 + 2 = 4 > 3 (yes). 1 + 1 = 2, which is not > 3 (no).
Answer	W = 2, R = 2 is consistent; W = 1, R = 1 is fast but a read may hit a copy that missed the write.
Meaning	You trade latency for freshness, per query.

The picture

```
partition key hash --> node
user_id=1 --> [Node A][Node B][Node C]  (3 copies)
inside the partition:  ts=100 | ts=105 | ts=130   (sorted)
```

Connection to the real job

Messaging, IoT events, shopping carts and activity feeds use these stores. DynamoDB is the managed AWS version; its "single-table design" is the same query-first idea (cloud service, not run here).

Applications

- **Backend:** user activity feeds, chat history, order history by user.
- **IoT:** sensor writes in huge volume.
- **Data Engineering:** a serving layer for pre-computed results.

COMMON MISTAKE	A "hot partition": choosing a partition key such as `date` so that all today's writes land on one node while others idle. Choose a key with many distinct, evenly used values (user id, not date).

Check yourself

1. What decides the machine that holds a row? Answer: the partition key.
2. Why are there no joins? Answer: data is spread over machines, so joins would be too slow; you duplicate data instead.
3. With N = 5, what is the smallest W if R = 3 and you want strong consistency? Answer: W = 3, because 3 + 3 = 6 > 5 (and W = 2 gives 5, which is not > 5).

13.4 Time-Series Databases

INTUITION	A weather station or a server produces a measurement every second: "temperature = 31 C" or "CPU = 63%". The data only grows, is almost never updated, and you mostly ask "what happened between 10:00 and 10:05?". A **time-series database** (TSDB) is a store tuned for this: fast appends, time-range reads, automatic downsampling and expiry. Examples: InfluxDB, TimescaleDB (an extension on PostgreSQL), Prometheus.

Words you need

Word	Meaning
Downsampling	Replacing fine data with coarser summaries (per second to per minute)
Retention	How long data is kept before auto-deletion
Cardinality	The number of distinct series (host x metric x label combinations)

Worked examples

Problem 13.4.1 Downsample raw CPU readings (two per minute) into one row per minute per host, keeping average and max (a max keeps spikes visible).

```
CREATE TABLE metrics (ts TEXT, host TEXT, cpu REAL);
INSERT INTO metrics VALUES
('2024-05-01 10:00:10','h1',40),('2024-05-01 10:00:40','h1',60),
('2024-05-01 10:01:10','h1',50),('2024-05-01 10:01:40','h1',90),
('2024-05-01 10:00:20','h2',20),('2024-05-01 10:01:20','h2',30);
SELECT substr(ts,1,16) AS minute, host, ROUND(AVG(cpu),1) AS avg_cpu, MAX(cpu) AS max_cpu, COUNT(*) AS n
FROM metrics GROUP BY minute, host ORDER BY host, minute;
```

```
minute           host  avg_cpu  max_cpu  n
---------------  ----  -------  -------  -
2024-05-01 10:00  h1    50.0     60.0     2
2024-05-01 10:01  h1    70.0     90.0     2
2024-05-01 10:00  h2    20.0     20.0     1
2024-05-01 10:01  h2    30.0     30.0     1
```

- **Check** h1 at 10:00: (40 + 60) / 2 = 50. At 10:01: (50 + 90) / 2 = 70.
- **Answer** Six raw rows became four summary rows.
- **Meaning** Keep raw data for a week, per-minute data for a year. TimescaleDB does this with "continuous aggregates"; InfluxDB with tasks (not run here).

Problem 13.4.2 Why averages hide spikes. Compare the whole-period summary with the per-minute view.

```
SELECT host, ROUND(AVG(cpu),1) AS avg_all, MAX(cpu) AS max_all FROM metrics GROUP BY host;
```

```
host  avg_all  max_all
----  -------  -------
h1    60.0     90.0
h2    25.0     30.0
```

- **Check** h1: (40 + 60 + 50 + 90) / 4 = 240 / 4 = 60.
- **Answer** h1 averages 60% but touched 90%.

- **Meaning** This is the same reason a train-delay report shows the worst delays and not just the average. Downsample with several statistics (avg, max, p95), never only the average.

The picture

```
raw (1s)        |||||||||||||||||||||| ... 7 days
1-minute rollup |    |    |    |    |    ... 1 year
1-hour rollup   |       |       |         ... 5 years
```

Each row is a coarser copy of the row above. Older data is smaller because fine detail is thrown away.

Connection to the real job

Prometheus plus Grafana is the standard monitoring stack. Monitoring data (server metrics, sensor readings, hospital vital signs) is time-series data.

Applications

- **Monitoring / SRE:** CPU, latency, error rates.
- **Finance:** tick prices.
- **IoT:** temperature and vibration sensors.

COMMON MISTAKE

Cardinality explosion. Putting `user_id` or `request_id` in a metric label creates one series per user. Millions of series exhaust memory. Labels must have few distinct values (service, region, status code).

Check yourself

1. What is downsampling? Answer: replacing fine-grained points with coarser summaries.
2. What is cardinality? Answer: the count of distinct series (unique label combinations).
3. Why store `MAX` too? Answer: the average hides short spikes.

13.5 Graph Databases (Neo4j)

INTUITION

Think of Facebook friends. "Who are friends of my friends who I don't know yet?" In SQL that is a join of the friends table with itself, and "friends of friends of friends" is three joins, and so on. A **graph database** stores the connections as first-class data, so following a link is cheap and the query for "3 hops away" is short. A graph has **nodes** (things) and **edges** (relationships between them).

How it works

Neo4j uses the query language **Cypher**, where you draw the pattern with ASCII art: `(a)-[:FOLLOWS]->(b)`. SQL can do it too with recursive CTEs (Chapter 5). We try that with a small follow network.

Worked examples

Problem 13.5.1 Who can Asha reach within 3 hops, and how many hops is the shortest?

```
CREATE TABLE follows (src TEXT, dst TEXT);
INSERT INTO follows VALUES ('Asha','Ravi'),('Ravi','Meena'),('Meena','John'),
  ('Asha','Priya'),('Priya','John'),('John','Kabir');
WITH RECURSIVE reach(person, hops, path) AS (
  SELECT 'Asha', 0, 'Asha'
  UNION ALL
  SELECT f.dst, r.hops+1, r.path||'>'||f.dst
  FROM reach r JOIN follows f ON f.src=r.person WHERE r.hops<3)
SELECT person, MIN(hops) AS shortest, COUNT(*) AS paths
FROM reach GROUP BY person ORDER BY shortest, person;
```

```
person  shortest  paths
------  --------  -----
Asha    0         1
Priya   1         1
Ravi    1         1
John    2         2
Meena   2         1
Kabir   3         1
```

- **Check** Asha to John: Asha > Priya > John is 2 hops; Asha > Ravi > Meena > John is 3 hops. So John has 2 paths with shortest 2. Kabir is only reachable through John: 3 hops by the short path.
- **Answer** Two people at one hop, two at two hops, one at three.
- **Meaning** The `hops < 3` limit stops the recursion, which also protects against cycles.

Problem 13.5.2 List the actual paths to John.

```
WITH RECURSIVE reach(person, hops, path) AS (
  SELECT 'Asha', 0, 'Asha'
  UNION ALL
  SELECT f.dst, r.hops+1, r.path||'>'||f.dst
  FROM reach r JOIN follows f ON f.src=r.person WHERE r.hops<3)
SELECT path FROM reach WHERE person='John';
```

```
path
--------------------
Asha>Priya>John
Asha>Ravi>Meena>John
```

- **Answer** The two paths found. In Cypher the same question is one line (not run here): `MATCH p = shortestPath((a:Person {name:'Asha'})-[:FOLLOWS*..3]->(j:Person {name:'John'})) RETURN p`.
- **Meaning** Fraud detection works like this: "accounts that share a phone number and a device with a known fraudster, within 3 hops".

The picture



Arrows mean "follows". Each arrow is one hop. Two different routes join at John.

Connection to the real job

Recommendation ("people who bought this also bought"), fraud rings, knowledge graphs, and network dependency maps (which service calls which) use graph queries. GenAI "GraphRAG" systems also use knowledge graphs.

Applications

- **Fraud analytics:** rings of accounts sharing devices.
- **Recommendation:** friends of friends, similar buyers.
- **IT operations:** impact analysis of a failing service.

COMMON MISTAKE	Using a graph database for ordinary tabular reporting. For "sum by month" it is slower and clumsier than SQL. Use a graph when the question is about **paths and relationships**, and depth is unknown or large.

Check yourself

1. What are nodes and edges? Answer: nodes are things, edges are relationships between them.
2. Why limit hops in a recursive CTE? Answer: to stop infinite loops on cycles and keep results small.
3. When is SQL enough? Answer: a few fixed hops, or no path questions at all.

13.6 Search Engines and Vector Databases

> **INTUITION (search):** The index at the back of a textbook: for each word it lists the pages. To find pages about "caching" you do not read the book; you look up the word. A search engine (Elasticsearch, OpenSearch, Solr) builds this **inverted index** for every word of every document.
>
> **INTUITION (vectors):** Search by *meaning*. "Cheap laptop" and "budget notebook" share no words, but an AI model can turn both into lists of numbers (**embeddings**) that are close together. A **vector database** finds the nearest lists to a query list.

How it works

Inverted index. For every word (term) store the list of documents that contain it (the **posting list**), with the count of occurrences (term frequency, tf). A query looks up each word and combines lists. **Analyzers** clean the text first: lowercase, split into words, remove stop-words, reduce words to stems ("running" to "run"). **Relevance** scores rank results; the standard score is **BM25**, which rewards words that appear often in a document, are rare in the whole collection, and appear in a short document.

Vector search. Two vectors are similar when the angle between them is small. **Cosine similarity** = dot(a, b) / (|a| x |b|); 1 means the same direction. Exact search compares the query with every vector, which is slow for millions. Vector databases (Pinecone, Milvus, Qdrant, the pgvector extension of PostgreSQL) use **approximate nearest neighbour (ANN)** indexes such as HNSW (a layered graph of vectors): much faster, a little less exact. (Details of embeddings belong to your GenAI study.)

Worked examples

Problem 13.6.1 Build a tiny inverted index and search for "fast car". Documents: 1 = "fast red car", 2 = "slow red bus", 3 = "fast blue car fast".

```
CREATE TABLE inv (term TEXT, doc_id INTEGER, tf INTEGER);
INSERT INTO inv
SELECT 'fast',1,1 UNION ALL SELECT 'red',1,1 UNION ALL SELECT 'car',1,1
UNION ALL SELECT 'slow',2,1 UNION ALL SELECT 'red',2,1 UNION ALL SELECT 'bus',2,1
UNION ALL SELECT 'fast',3,2 UNION ALL SELECT 'blue',3,1 UNION ALL SELECT 'car',3,1;
SELECT term, GROUP_CONCAT(doc_id) AS posting_list FROM inv GROUP BY term ORDER BY term;
SELECT doc_id, SUM(tf) AS score FROM inv WHERE term IN ('fast','car') GROUP BY doc_id ORDER BY score DESC;
```

```
term  posting_list
----  ------------
blue  3
bus   2
car   1,3
fast  1,3
red   1,2
slow  2
doc_id  score
------  -----
3       3
1       2
```

- **Check** Doc 3 has "fast" twice and "car" once: 2 + 1 = 3. Doc 1 has each once: 2. Doc 2 has neither, so it is never touched.
- **Answer** Doc 3 ranks first, doc 1 second; doc 2 is not returned.
- **Meaning** We never scanned documents. Counting tf is a crude score; BM25 improves it. SQLite has a real full-text engine, FTS5, which ranks by BM25:

```
CREATE VIRTUAL TABLE d5 USING fts5(body);
INSERT INTO d5(body) VALUES ('fast red car'),('slow red bus'),('fast blue car fast');
SELECT rowid, body FROM d5 WHERE d5 MATCH 'fast car' ORDER BY rank;
```

```
rowid  body
-----  ----------------
3      fast blue car fast
1      fast red car
```

Same order as our hand-made score. (PostgreSQL has built-in full text search; Elasticsearch scales it over many machines. Elasticsearch syntax: not run here.)

Problem 13.6.2 Find the animal nearest to a query vector `(1.0, 0.2, 0.0)` using cosine similarity. Three tiny 3-number vectors: cat (0.9, 0.1, 0.0), dog (0.8, 0.3, 0.1), car (0.1, 0.9, 0.4).

```
CREATE TABLE vec (id INTEGER, name TEXT, x REAL, y REAL, z REAL);
INSERT INTO vec VALUES (1,'cat',0.9,0.1,0.0),(2,'dog',0.8,0.3,0.1),(3,'car',0.1,0.9,0.4);
WITH q(x,y,z) AS (SELECT 1.0,0.2,0.0)
SELECT v.name,
 ROUND((v.x*q.x+v.y*q.y+v.z*q.z)/(sqrt(v.x*v.x+v.y*v.y+v.z*v.z)*sqrt(q.x*q.x+q.y*q.y+q.z*q.z)),4) AS cosine
FROM vec v, q ORDER BY cosine DESC;
```

```
name  cosine
----  ------
cat   0.9962
dog   0.9803
car   0.2774
```

- **Compute for cat** dot = 0.9 x 1.0 + 0.1 x 0.2 + 0 = 0.92. |cat| = √(0.81 + 0.01) = √0.82 = 0.9055. |q| = √(1 + 0.04) = 1.0198. Cosine = 0.92 / (0.9055 x 1.0198) = 0.92 / 0.9234 = 0.9962.
- **Answer** The cat vector is nearest, then dog, and car is far.
- **Meaning** A vector database does exactly this for millions of vectors using ANN indexes, so it does not compute every cosine. This is the retrieval step of RAG (retrieval-augmented generation).

The picture

```
Search:  "fast car" --> look up 'fast' -> [1,3]
                        look up 'car'  -> [1,3]   --> score, rank: 3, 1
Vectors:       ^ y
               |   car .
               |
               |  dog .
               | cat .   (small angle to query = similar)
               +-------------> x
```

Connection to the real job

Log search (ELK stack, OpenSearch) is a daily tool for operations and support teams. Vector stores power chatbot memory and document Q&A, which is the main use in GenAI jobs.

Applications

- **Search:** product search, log analysis, autocomplete.
- **GenAI:** RAG retrieval, semantic search, recommendations.
- **Data Engineering:** indexing documents from pipelines.

COMMON MISTAKE

Using a search engine as the main database. Elasticsearch is not built for strong transactions; keep the source of truth elsewhere and treat the index as a rebuildable copy. Also, never compare vectors from different embedding models; their numbers mean different things.

Check yourself

1. What does an inverted index map? Answer: each word to the list of documents containing it.
2. What does cosine similarity 1 mean? Answer: the vectors point in exactly the same direction.
3. Why use ANN? Answer: exact comparison with millions of vectors is too slow; ANN trades a little accuracy for large speed.

13.7 CAP, Consistency Models and Choosing a Store

INTUITION

Two bank branches share one ledger by phone. If the phone line breaks, each branch must choose: stop serving customers until the line is back (stay **consistent**, lose **availability**), or keep serving with their own copy and fix differences later (stay **available**, risk **inconsistency**). That choice, forced by a network failure, is the **CAP theorem**.

Words you need

Word	Meaning
Consistency (C)	Every read sees the latest write
Availability (A)	Every request to a working node gets an answer
Partition tolerance (P)	The system keeps working when the network between nodes breaks
Eventual consistency	Copies may differ for a while, but become equal if writes stop
Strong consistency	A read after a write always sees that write

How it works

CAP says: when a network partition happens (P is not optional on real networks), you must pick C or A. So the real choice is **CP** (refuse some requests to stay correct, e.g. most relational clusters with one primary, HBase, MongoDB with majority writes) or **AP** (always answer, maybe with old data, e.g. Cassandra at low consistency, DynamoDB with eventually consistent reads). Behaviour depends on configuration, so treat these labels as defaults. The **PACELC** extension adds: even when there is no partition (Else), you trade **L**atency against **C**onsistency, exactly the R and W choice in 13.3.

Worked examples

Problem 13.7.1 A shopping cart in an AP store. Two servers each hold a copy. During a partition, you add a book on server 1 and a pen on server 2. What does each show, and what should the merge produce after the line is restored?

Step 1	Server 1 cart: {book}. Server 2 cart: {pen}.
Step 2	After healing, a "last write wins" rule keeps only one cart, so one item is lost.
Step 3	A set-union merge keeps both: {book, pen}.
Answer	For carts, merge by union (never lose an add); for a bank balance, you need CP and a single agreed order.
Meaning	Eventual consistency is acceptable when a merge rule exists and a wrong read is cheap.

Problem 13.7.2 Choose a store for eight needs, and say why.

Need	Best fit	Reason
Orders, payments, money	PostgreSQL (relational)	ACID transactions, joins, constraints
Product catalogue with varied attributes	MongoDB	One document per product, flexible fields
Session store, rate limits, hot lookups	Redis	In-memory, TTL, atomic counters
Chat history, 1M writes per second	Cassandra / DynamoDB	Partitioned writes, query-first keys
CPU and latency metrics	Prometheus / TimescaleDB	Time-range reads, downsampling, retention
"Friends of friends", fraud rings	Neo4j	Path queries
Log and text search	Elasticsearch / OpenSearch	Inverted index, BM25
Meaning-based document retrieval	Vector DB or pgvector	Nearest-neighbour search

- **Answer** Eight needs, eight different best fits; the table is the answer.
- **Meaning** Real systems combine them: **polyglot persistence**. A typical shop: PostgreSQL as the source of truth, Redis in front for hot reads, Elasticsearch for product search, a warehouse for analytics. The cost of every extra store is more to operate, more to keep in sync and more failure modes. **Default to PostgreSQL** and add a store only when a measured need appears.

Also know **NewSQL / distributed SQL** (CockroachDB, Spanner, TiDB): relational databases that spread over many machines and offer SQL with transactions. They aim at CP with high availability, at higher latency per write. And **OLAP engines** (ClickHouse, Druid, Pinot, plus the warehouses of Chapter 14) answer huge aggregations over columns quickly; they are not for single-row updates.

The picture

```
          Consistency
             /\
            /  \
       CP  /    \  CA (only on one machine: no partition)
          /______\
  Partition tolerance ---- Availability
             AP
```

Each corner is a property. With a real network you must keep P, so you pick the left side (CP) or the bottom side (AP).

Connection to the real job

System-design interviews ask "which database and why?" Your answer: name the access pattern, the consistency need, the scale, and the operating cost, then give one trade-off.

Applications

- **Interviews:** "design a URL shortener, a chat app, a leaderboard".
- **Data Engineering:** choosing a serving layer for a pipeline's output.
- **Architecture:** deciding where the single source of truth lives.

COMMON MISTAKE	Saying "NoSQL is faster than SQL." Speed comes from fitting the access pattern and from giving up something (joins, transactions, flexible queries). A well-indexed PostgreSQL table beats a badly modelled NoSQL store for most workloads under a few thousand requests per second.

Check yourself

1. If the network between two data centres fails, what does CAP force? Answer: a choice between consistency (refuse some requests) and availability (answer, maybe stale).
2. What is eventual consistency? Answer: copies may differ now but become equal later if writes stop.
3. Which store would you use for money transfers? Answer: a relational database with ACID transactions.

13.8 Modelling for NoSQL: Embed or Reference, Partition Keys, Replica Lag and Friend Graphs

INTUITION	In SQL you model the *facts* and let queries combine them. In most NoSQL stores you model the *queries*: you list the questions the application will ask, then shape the data so each question is a single fast lookup. That is why the same shop is designed differently in a document store, a wide-column store and a graph, and why it can be painful when a new question appears. These examples show the design moves, with runnable stand-ins where a real server is not available. MongoDB, Redis, Cassandra and Neo4j commands are labelled *not run here*.

Example 1: embed or reference in a document store

An order has a customer and a list of items. In MongoDB you either **embed** the customer and items inside the order document, or **reference** them by id and look them up in another collection. Embedding makes reads one lookup and writes atomic; referencing avoids copying data that changes. SQLite's JSON functions let us feel the difference with real output:

Problem 13.8.1 Store three orders as JSON documents with the customer embedded, then count units per customer. Predict first: Asha has 3 + 1 = 4 units (two orders), Ravi 1.

```
CREATE TABLE doc_orders (id INTEGER PRIMARY KEY, doc TEXT);
INSERT INTO doc_orders VALUES
 (1, '{"customer": {"name": "Asha", "city": "Pune"}, "items": [{"sku": "kb", "qty": 1}, {"sku": "nb", "qty": 2}]}'),
 (2, '{"customer": {"name": "Asha", "city": "Pune"}, "items": [{"sku": "dl", "qty": 1}]}'),
 (3, '{"customer": {"name": "Ravi", "city": "Mumbai"}, "items": [{"sku": "ms", "qty": 1}]}');
SELECT json_extract(doc, '$.customer.name') AS customer, sum(json_extract(j.value, '$.qty')) AS units
FROM doc_orders, json_each(doc, '$.items') AS j
GROUP BY customer ORDER BY customer;
```

```
customer  units
--------  -----
Asha      4
Ravi      1
```

Step 1	Each document is self-contained: one read returns the order, the customer and the items. That is the speed benefit of embedding.
Step 2	The query walks each document's item array with `json_each` and sums quantities per customer name. Asha 4 units (1 + 2 + 1), Ravi 1.
Meaning	MongoDB's equivalent is an aggregation pipeline with `$unwind` and `$group` (not run here).

Now the price of embedding. Asha moves to Mumbai and we update *one* order document, as an application that forgot about the copies would:

```
UPDATE doc_orders SET doc = json_set(doc, '$.customer.city', 'Mumbai') WHERE id = 1;
SELECT id, json_extract(doc, '$.customer.name') AS name, json_extract(doc, '$.customer.city') AS city FROM doc_orders ORDER
BY id;
```

```
id  name  city
--  ----  ------
1   Asha  Mumbai
2   Asha  Pune
3   Ravi  Mumbai
```

Step 1	Order 1 now says Mumbai, order 2 still says Pune. The same customer has two cities: an update anomaly, exactly like the one normalization removes (Chapter 11).
Step 2	In a document store you accept this when the copy is meant to be a *snapshot*. An order should record the address it was shipped to, so the old city is correct for order 2.
Rule	Embed data that is read together and does not change independently or is a historical snapshot (order lines, shipping address). Reference data that changes in one place and is shared (customer profile, product price now).

COMMON MISTAKE	Embedding an unbounded list, such as every comment ever made inside the post document. Documents have a size limit (16 MB in MongoDB) and grow slowly into a performance problem, because every update rewrites a larger document. Reference unbounded children in their own collection.

Example 2: choosing a partition key (Cassandra, DynamoDB)

In wide-column stores the **partition key** decides which machine holds a row and how much data one partition contains. Rows with the same key sit together and are read together fast, but one giant partition becomes a hot spot. For sensor data the natural key is the sensor, which is wrong at scale. The numbers show why:

Problem 13.8.2 One sensor writes one reading per second. Compare 'partition by sensor' with 'partition by sensor and day'. Predict first: a year is 31.5 million rows per partition against 86,400.

```
from collections import Counter
# 1 million readings from 1,000 sensors, one reading per second for ~17 minutes each
sensors, seconds = 1000, 1000
rows_by_sensor = Counter()
for s in range(sensors):
    for t in range(seconds):
        rows_by_sensor[s] += 1
print("partition = sensor      : %d partitions, %d rows each" % (len(rows_by_sensor), max(rows_by_sensor.values())))

# at full speed one sensor makes 86,400 rows a day, 31.5 million a year
per_year = 86400 * 365
print("one sensor, one year    : %d rows in a single partition" % per_year)
print("partition = sensor+day  : %d rows per partition, %d partitions per sensor-year" % (86400, 365))
```

```
partition = sensor      : 1000 partitions, 1000 rows each
one sensor, one year    : 31536000 rows in a single partition
partition = sensor+day  : 86400 rows per partition, 365 partitions per sensor-year
```

Step 1	With the key (sensor) a year of data for one sensor sits in a single partition of 31,536,000 rows. Cassandra recommends keeping partitions well below 100 MB, and this one would be far over.
Step 2	Adding the day to the key caps a partition at 86,400 rows. Reading one day is one partition; reading a week is 7 partition reads, still cheap.
Step 3	The CQL shape is `PRIMARY KEY ((sensor_id, day), ts)` : the double brackets mean 'partition key is the pair', and `ts` orders rows inside the partition (not run here).
Rule	Choose a key that spreads data evenly *and* lets your main query hit one or a few partitions. Never one that grows without bound.

Example 3: replica lag, and reading your own writes

Most distributed stores copy each write to several machines and acknowledge before all copies have it. A reader hitting a lagging copy sees old data. This is **eventual consistency**. A tiny simulation with a seeded random network shows the symptom and one standard fix:

```
import random
random.seed(3)

class Replica:
    def __init__(self):
        self.data = {}

primary, follower = Replica(), Replica()
pending = []                                # writes still travelling to the follower

def write(key, value):
    primary.data[key] = value
    pending.append((key, value))            # replicated later, not instantly

def replicate_some():
    while pending and random.random() < 0.5:  # the network delivers some writes per tick
        k, v = pending.pop(0)
        follower.data[k] = v

write("profile:7", "new name")
print("read from follower right after the write :", follower.data.get("profile:7"))
replicate_some(); replicate_some(); replicate_some()
print("read from follower a few ticks later      :", follower.data.get("profile:7"))

# fix 1: read your own writes. After a write, read from the primary for a while.
write("profile:8", "x")
print("read-your-writes (primary) right after    :", primary.data.get("profile:8"))
```

```
read from follower right after the write : None
read from follower a few ticks later      : new name
read-your-writes (primary) right after    : x
```

Step 1	The write reaches the primary at once. The follower has not received it, so reading the follower returns `None`: the user saves their profile and sees the old one.
Step 2	A few ticks later the follower has caught up. 'Eventually' means exactly that: the copies agree if you wait.
Step 3	Fix: **read your own writes**. For a short time after a user writes, send that user's reads to the primary (or use a session token that records the version the user has written). Other users can tolerate staleness.
Meaning	Consistency is not all or nothing. Decide per feature: a bank balance needs the latest value, a like counter does not.

Example 4: friends of friends, in SQL and in Cypher

Graph databases shine at 'who is within two hops'. The SQL version is a self-join, which grows more expensive with each extra hop. A small friendship table (edges stored both ways) and the query to suggest friends to user 1, people two hops away who are not already friends:

```
CREATE TABLE friend (a INTEGER, b INTEGER);
INSERT INTO friend VALUES (1,2),(2,1),(1,3),(3,1),(2,4),(4,2),(3,4),(4,3),(4,5),(5,4),(5,6),(6,5);
SELECT DISTINCT f2.b AS suggestion
FROM friend f1 JOIN friend f2 ON f2.a = f1.b
WHERE f1.a = 1 AND f2.b <> 1 AND f2.b NOT IN (SELECT b FROM friend WHERE a = 1)
ORDER BY suggestion;
```

```
suggestion
----------
4
```

Step 1	User 1's friends are 2 and 3. Friends of 2 are 1 and 4. Friends of 3 are 1 and 4.
Step 2	Remove user 1 and anyone who is already a friend. What remains is user 4 (friend of both 2 and 3) as the single suggestion. User 5 is three hops away, so it is excluded.
Cypher	In Neo4j the same query is `MATCH (me:User {id: 1})-[:FRIEND]->()-[:FRIEND]->(s) WHERE NOT (me)-[:FRIEND]-(s) AND s <> me RETURN DISTINCT s` (not run here). Adding a third hop is one more arrow, while SQL needs another join.

SQL or NoSQL? A decision table

Situation	Good first choice	Why
Orders, payments, inventory with relationships and transactions	PostgreSQL or another relational database	joins, constraints and ACID are what the problem needs
Product catalogue where each item has different attributes	PostgreSQL with `jsonb`, or a document store	flexible shape with a relational core
Session data, caches, rate limits, leaderboards	Redis	in-memory speed, TTL, sorted sets
Billions of time-stamped events, simple read patterns	Cassandra, or a time-series or columnar engine	write throughput, partitioned by time
Deep relationship queries (recommendations, fraud rings)	graph database, or recursive SQL at small scale	hops are cheap in a graph
Full-text search and similarity search	search engine or vector index	ranking and approximate matching

The honest default for a new product is a relational database. Reach for a specialised store when you can name the specific query or scale that a relational database cannot serve, and keep the relational one as the source of truth.

Practice with answers

Problem 13.8.3 An e-commerce order document embeds the product's current price. The price changes. Is that a bug? *(medium)*

Answer: No, it is the desired snapshot: the order should keep the price the customer paid. The bug would be showing the live product price on an old invoice. Embed price at purchase, reference the product id for the current catalogue.

Problem 13.8.4 A table is partitioned by country and 40 percent of users are in one country. Problem? *(medium)*

Answer: A hot partition: one machine takes 40 percent of the traffic and storage, so scaling out does not help. Add a second key part (hash bucket, month, or user id range) so rows spread more evenly.

Problem 13.8.5 A user posts a comment and refreshes; the comment is missing for two seconds. Cause and fix? *(interview)*

Answer: Replica lag: the read went to a follower that had not received the write. Fix: read your own writes (route that user's reads to the primary for a short window, or use a session token / quorum read). Not every read needs it.

Problem 13.8.6 Why can friends-of-friends be slow in SQL on a large network and fast in a graph database? *(expert)*

Answer: SQL answers each hop with a join over the whole edge table (an index lookup per row, with intermediate results growing quickly). A native graph store keeps each node's neighbours next to it (index-free adjacency), so a hop follows pointers whose cost depends on the number of neighbours, not the table size. At small scale with an index on (a), SQL is fine.

Chapter recap

- Choose a store by **access pattern**, not by fashion; every store gives up something to be fast at one thing.
- Document stores keep a whole record in one document; embed what is read together, reference what is large or shared.

- Redis is an in-memory key-value store with TTL; use cache-aside and expect a cold-cache slowdown.
- Cassandra and DynamoDB are query-first: partition key spreads data, clustering key sorts it; no joins; R + W > N gives consistent reads.
- Time-series databases fit append-heavy, time-range data; downsample with avg and max; watch cardinality.
- Graph databases fit path questions; SQL can do a few hops with recursive CTEs.
- Search engines use inverted indexes and BM25; vector databases use embeddings, cosine similarity and ANN.
- CAP: during a partition choose consistency or availability; PACELC adds latency versus consistency. Default to PostgreSQL and add stores only for measured needs.

Family	Example	Strong at	Weak at
Relational	PostgreSQL	joins, ACID, ad-hoc queries	massive write scale on one node
Document	MongoDB	nested records, flexible fields	multi-document joins
Key-value	Redis	sub-ms lookup, TTL, counters	queries by value, large data
Wide-column	Cassandra, DynamoDB	huge writes, key lookups	ad-hoc queries
Time-series	Prometheus, TimescaleDB	time ranges, rollups	high cardinality
Graph	Neo4j	multi-hop paths	bulk reporting
Search	Elasticsearch	text, logs, relevance	transactions
Vector	pgvector, Qdrant	similarity by meaning	exact filters alone

Practice set

Problem 13.P.1 (easy) Which store family fits a leaderboard of top 10 players updated every second? **Answer:** Redis, using a sorted set (members ordered by score).

Problem 13.P.2 (easy) Define cache-aside in one sentence. **Answer:** The app checks the cache first; on a miss it reads the database, stores the result in the cache with a TTL and returns it.

Problem 13.P.3 (computation) Hit ratio 80%, hit 1 ms, miss 21 ms in total (1 ms check plus 20 ms database). Average latency? **Answer:** 0.8 x 1 + 0.2 x 21 = 0.8 + 4.2 = 5.0 ms.

Problem 13.P.4 (what is the output?) Using the `hits` table of 13.2.2, how many requests does IP `b` have in minute bucket 0, and is it blocked? **Answer:** 2 requests (seconds 5 and 6), not blocked (limit is 3).

Problem 13.P.5 (computation) N = 5 copies, W = 3. What is the smallest R for consistent reads? **Answer:** R + W > N means R > 2, so R = 3.

Problem 13.P.6 (find the bug) A team uses `date` as the Cassandra partition key for an events table. What goes wrong? **Answer:** All of today's writes go to one partition (hot partition) and one node. Use a key such as `device_id`, or `(device_id, date)`.

Problem 13.P.7 (find the bug) A Prometheus metric has the label `user_id`. What goes wrong? **Answer:** Cardinality explosion: one time series per user exhausts memory. Remove the label and log per-user data elsewhere.

Problem 13.P.8 Using the follows graph of 13.5, who are the people exactly 2 hops from Asha? **Answer:** Meena (Asha > Ravi > Meena) and John (Asha > Priya > John). The result table of 13.5.1 shows shortest = 2 for exactly these two.

Problem 13.P.9 (scenario) A web shop's product page is slow because the same 1,000 popular products are read 50,000 times per minute from PostgreSQL. What is your first fix and what is the risk? **Answer:** Put a Redis cache-aside layer with a short TTL in front. The risk is stale data (a price

change not shown) and a cache stampede (many misses at once when a key expires); use short TTLs, invalidate on update, and add random jitter to expiry times.

Problem 13.P.10 (interview) Explain CAP to a non-technical manager in three sentences. **Answer:** When the connection between our servers breaks, we must choose. Either we stop answering some requests so nobody sees wrong data (consistency), or we keep answering and may show slightly old data (availability). Banking picks the first, shopping carts often pick the second.

Problem 13.P.11 (interview) When would you NOT use MongoDB? **Answer:** When the data is highly relational and needs multi-row transactions and joins, such as accounting. Also when the main workload is analytical aggregation across many collections; a relational or columnar warehouse is simpler and faster.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data and AI roles and I know SQL basics. Explain how to choose between PostgreSQL, MongoDB, Redis, Cassandra, Elasticsearch, a time-series database and a graph database. Use one running example (an online shop) and a decision table. Then ask me five scenario questions one at a time and correct my answers.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning databases beyond SQL. Teach me Redis caching: cache-aside, write-through, TTL, invalidation, cache stampede and rate limiting. Use real numbers for hit ratio and latency, show the commands, and give me 8 graded exercises without solutions. Wait for my answers before revealing anything.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for system design interviews. Teach me the CAP theorem, PACELC, eventual versus strong consistency, and quorum reads and writes (R + W > N), step by step with small numeric examples and a shopping-cart story. After each part, check my understanding with a question.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data engineering. Explain how documents from MongoDB or JSON APIs are flattened into relational tables, including arrays, optional fields and nested objects. Give a small JSON sample and show the SQL to flatten it in PostgreSQL. Then review my own example: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into GenAI roles. Teach me inverted indexes, BM25, embeddings, cosine similarity and approximate nearest-neighbour search, and when to use a search engine versus a vector database. Use tiny numeric examples I can calculate by hand, then run a mock interview of five questions on retrieval systems and grade each answer.
```

CHAPTER 14

SQL DIALECT DIFFERENCES AND THE 40-PROBLEM INTERVIEW BANK

How the same idea is spelled in PostgreSQL, MySQL, SQL Server, BigQuery and Snowflake, and forty interview problems with answers.

You will meet more than one database in your career. Your first job may use SQL Server, the next BigQuery, a startup may run MySQL, and your own practice runs on PostgreSQL or SQLite. The core ideas (joins, grouping, windows) are the same everywhere, but small spelling differences cause errors on day one: `LIMIT` versus `TOP`, `||` versus `CONCAT`, `DATE_TRUNC` with its arguments in a different order. This chapter teaches you what is portable, where the dialects split, and how to port a query without panic. The second half is a bank of 40 interview problems that mixes SQL writing and database theory. Every query was run in SQLite on the shop database of Chapter 2. Dialect-specific syntax is marked "not run here", because only SQLite is installed on this machine; the dialect facts come from standard documentation knowledge and you should confirm details against the manual of your exact version.

14.1 The Portable Core and Why Dialects Exist

INTUITION

English in India, the UK and the US: same language, different words for "lift" and "elevator". SQL is the same. There is an official standard (ANSI/ISO SQL), but each vendor started before the standard was complete, added helpful extras, and never removed them. A **dialect** is one vendor's version of SQL.

Words you need

Word	Meaning
ANSI SQL	The official standard; vendors follow part of it
Dialect	A vendor's own spelling and extras (T-SQL for SQL Server, GoogleSQL for BigQuery)
Portable	Runs unchanged on most databases
Warehouse	A database tuned for analytics over huge data (BigQuery, Snowflake)

How it works

About 80% of everyday SQL is portable: `SELECT`, `WHERE`, `GROUP BY`, `JOIN`, `CASE`, `COALESCE`, `CTE`s, window functions, `UNION`. The differences cluster in six places: (1) limiting rows, (2) string and date functions, (3) types and auto-numbering, (4) upsert and procedural code, (5) quoting and case rules, (6) division and NULL behaviour.

Worked examples

Problem 14.1.1 Show the portable core in SQLite. Predict: two names with their category in brackets.

```
SELECT name||' ('||category||')' AS label FROM products LIMIT 2;
```

```
label
--------------------
Keyboard (Electronics)
Mouse (Electronics)
```

- **Answer** `||` joins strings in SQLite and PostgreSQL. `LIMIT 2` keeps two rows.
- **Meaning** Each of those two small things has a different spelling elsewhere (14.2).

Problem 14.1.2 The integer-division trap. In SQLite, what is `7/2`?

```
SELECT 7/2 AS int_div, 7*1.0/2 AS dec_div, 7%2 AS rem, 10/0 AS div_by_zero;
```

```
int_div  dec_div  rem  div_by_zero
-------  -------  ---  -----------
3        3.5      1
```

- **Answer** `7/2` is 3 because both numbers are integers. Multiply by 1.0 to get 3.5. Dividing by zero gives `NULL` in SQLite (blank above).
- **Dialects (not run here):** PostgreSQL and SQL Server also do integer division for two integers, and PostgreSQL and SQL Server raise an **error** on division by zero. MySQL's `/` returns a decimal (`7/2 = 3.5`) and has `DIV` for integers; division by zero gives NULL by default. BigQuery's `/` on integers returns a float; use `SAFE_DIVIDE(a, b)` to get NULL instead of an error.
- **Meaning** The same `7/2` can give 3 or 3.5. Percent calculations are where this bites: always force decimals (`100.0 * a / b`) and guard zeros (`NULLIF(b,0)`).

The picture



The big box is what you already learned. Each name below it adds its own extras around the core.

Connection to the real job

Job posts name the dialect. Interviewers often let you pick, but warehouse jobs expect BigQuery or Snowflake, and many enterprises use SQL Server. Knowing the split makes you quickly productive in any of them.

Applications

- **Data Analytics:** moving a report query from a Postgres test database to the company warehouse.
- **Data Engineering:** writing transformation code (dbt) that must run on the warehouse.
- **Consulting:** each client has a different database.

COMMON MISTAKE	Memorising one dialect's syntax as "SQL". When an error appears on a new platform, first suspect the dialect (function name, argument order, quoting) before suspecting your logic.

Check yourself

1. What is a dialect? Answer: one vendor's own version of SQL, with its own spellings and extras.
2. What does `7/2` return in SQLite? Answer: 3 (integer division).
3. Where do differences cluster? Answer: row limits, string/date functions, types, upsert, quoting, division/NULL behaviour.

14.2 PostgreSQL, MySQL and SQL Server Side by Side

INTUITION	A phrasebook. You do not learn a language; you look up the word for "tea". The tables below are your phrasebook for the three most common transactional databases.

How it works

These three are classic **row-oriented, OLTP** databases (Chapter 11): good at many small reads and writes. PostgreSQL follows the standard closely and has rich types (arrays, `jsonb`, ranges) and extensions. MySQL (InnoDB engine) is widely used for web applications; windows functions and CTEs arrived only in version 8.0, so older servers lack them. SQL Server is Microsoft's, programmed with **T-SQL** (variables like `@x`, `BEGIN...END` blocks, `#temp` tables).

Task	PostgreSQL	MySQL	SQL Server (T-SQL)
First 5 rows	`LIMIT 5`	`LIMIT 5`	`SELECT TOP 5 ...` or `OFFSET 0 ROWS FETCH NEXT 5 ROWS ONLY`
Concatenate	`a \	\	b `or` CONCAT`
Quote a name	`"col"`	` `col` `	`[col]`
Month start	`DATE_TRUNC('month', d)`	`DATE_FORMAT(d,'%Y-%m-01')`	`DATEFROMPARTS(YEAR(d),MONTH(d),1)` (newer: `DATETRUNC`)
Days between	`d2 - d1`	`DATEDIFF(d2,d1)`	`DATEDIFF(day,d1,d2)`
Auto number	`GENERATED ... AS IDENTITY`	`AUTO_INCREMENT`	`IDENTITY(1,1)`
Upsert	`INSERT ... ON CONFLICT DO UPDATE`	`INSERT ... ON DUPLICATE KEY UPDATE`	`MERGE`
Aggregate to text	`STRING_AGG(x, ',')`	`GROUP_CONCAT(x)`	`STRING_AGG(x, ',')`
NULL default	`COALESCE`	`COALESCE`, `IFNULL`	`COALESCE`, `ISNULL`
Case-insensitive match	`ILIKE`	`LIKE` (often case-insensitive by collation)	`LIKE` (depends on collation)
Lateral join	`LATERAL`	`LATERAL` (8.0.14+)	`CROSS APPLY`
Boolean type	`BOOLEAN`	`TINYINT(1)` alias	`BIT`

All entries in this table are "not run here". The portable alternative for most rows is the standard form: `COALESCE`, `CASE`, `CAST`, `FETCH FIRST n ROWS ONLY` (PostgreSQL and recent standard-following systems), and plain `JOIN`.

Worked examples

Problem 14.2.1 Show only the first 2 most expensive products. Predict: Keyboard and Desk Lamp.

```
SELECT name FROM products ORDER BY price DESC LIMIT 2;
```

```
name
---------
Keyboard
Desk Lamp
```

- **Answer** SQLite, PostgreSQL and MySQL: `LIMIT 2`. SQL Server (not run here): `SELECT TOP 2 name FROM products ORDER BY price DESC;`
- **Check** Prices are 1500, 900, 600, 250 ... so top 2 are Keyboard 1500 and Desk Lamp 900.
- **Meaning** Always include `ORDER BY` with a row limit; without it, "first 2" is arbitrary on every database.

Problem 14.2.2 Translate a "names in one cell" query. In SQLite we use `GROUP_CONCAT`:

```
SELECT GROUP_CONCAT(name, ', ') AS names FROM customers WHERE city='Pune';
```

```
names
---------------
Asha, Meena, Tom
```

- **Answer** Pune customers are Asha, Meena and Tom.
- **Translation (not run here):** PostgreSQL `STRING_AGG(name, ', ' ORDER BY name)`; SQL Server 2017+ `STRING_AGG(name, ', ') WITHIN GROUP (ORDER BY name)`; MySQL `GROUP_CONCAT(name ORDER BY name SEPARATOR ', ')`.
- **Meaning** The row order inside the text is not guaranteed unless you state it. Add the ordering clause in production.

The picture

```
Same intent: "give me the month of a date"
  PostgreSQL : DATE_TRUNC('month', d)
  MySQL      : DATE_FORMAT(d, '%Y-%m-01')
  SQL Server : DATEFROMPARTS(YEAR(d), MONTH(d), 1)
  SQLite     : date(d, 'start of month')     <- run in Chapter 6
```

Connection to the real job

Migration projects ("move from SQL Server to PostgreSQL") are a steady source of work. The checklist is this phrasebook plus data types and procedural code.

Applications

- **Backend teams:** MySQL and PostgreSQL behind web apps.
- **Enterprises:** SQL Server with reporting tools like Power BI.
- **Data Engineering:** reading from these systems into a warehouse.

COMMON MISTAKE

In MySQL, `||` is not string concatenation by default (it means logical OR), so `'a' || 'b'` does not give `'ab'`. Use `CONCAT`. In SQL Server, `+` with a NULL gives NULL, while `CONCAT` treats NULL as an empty string.

Check yourself

1. How do you take 5 rows in SQL Server? Answer: `SELECT TOP 5 ...` (or `OFFSET ... FETCH`).
2. Which function builds a month start in PostgreSQL? Answer: `DATE_TRUNC('month', d)`.
3. What replaces `LATERAL` in SQL Server? Answer: `CROSS APPLY`.

14.3 BigQuery and Snowflake: the Cloud Warehouses

INTUITION

The databases above are shops: many small customers, quick service. A warehouse is a cargo port: fewer, much bigger jobs ("sum a year of data"), served by machines rented only while the ship is unloading. **BigQuery** (Google) and **Snowflake** are **cloud data warehouses**: columnar storage (Chapter 11), no indexes to tune, and compute that scales out.

How it works

- **BigQuery** is serverless. In the on-demand model you pay by the amount of data your query **scans**, so `SELECT *` on a wide table costs real money and a `WHERE` on an unpartitioned column does not reduce the bill. You reduce cost by choosing only needed columns and by filtering on a **partition**

column (usually a date) and **clustering** columns. Tables are named `project.dataset.table`. Nested data uses `ARRAY` and `STRUCT` types, unnested with `UNNEST` (Chapter 6).

- **Snowflake** separates storage from compute. You create a **virtual warehouse** (a cluster of machines) of a chosen size; you pay while it runs, so you suspend it when idle. Data is stored in automatic **micro-partitions**; a **clustering key** can help on huge tables. Semi-structured data lives in a `VARIANT` column, read with `col:field::string`, expanded with `FLATTEN`. It offers Time Travel (query a table as it was earlier).
- **Both** support `QUALIFY` (filter on a window function without a subquery), CTEs, and `MERGE`. Primary and foreign keys are informational: they are not enforced as in PostgreSQL, so duplicates can appear and you must test for them (Chapter 12).

Task	BigQuery	Snowflake
Limit	`LIMIT n`	`LIMIT n` or `TOP n`
Month start	`DATE_TRUNC(d, MONTH)`	`DATE_TRUNC('month', d)`
Days between	`DATE_DIFF(d2, d1, DAY)`	`DATEDIFF(day, d1, d2)`
Safe division	`SAFE_DIVIDE(a,b)`	`DIV0(a,b)` or `a / NULLIF(b,0)`
Conditional	`IF(c,a,b)`	`IFF(c,a,b)`
Aggregate text	`STRING_AGG(x, ',')`	`LISTAGG(x, ',')`
Exclude columns	`SELECT * EXCEPT (col)`	`SELECT * EXCLUDE (col)`
Type names	`INT64`, `FLOAT64`, `STRING`	`NUMBER`, `FLOAT`, `VARCHAR`
Unquoted names	case-sensitive for tables, not for columns	stored UPPERCASE

All of this table is "not run here"; check your platform manual for the version you use. A notable BigQuery detail: argument order in `DATE_TRUNC` is reversed compared with PostgreSQL.

Worked examples

Problem 14.3.1 Cost thinking. A BigQuery table has 50 columns, 1 TB in total, and one `amount` column that is about 2% of the bytes. How much data does `SELECT amount FROM t` scan compared with `SELECT * FROM t`?

Formula	scanned bytes = size of the columns you select (columnar storage)
Plug in	`SELECT *`: 1 TB. `SELECT amount`: 2% of 1 TB.
Compute	0.02 x 1 TB = 0.02 TB = 20 GB (taking 1 TB = 1,000 GB)
Answer	20 GB instead of 1,000 GB: 50 times less, so about 50 times cheaper on a per-scan billing model.
Check	1000 / 20 = 50, and 1 / 0.02 = 50.
Meaning	In the warehouse world, "select only the columns you need" is a cost rule and not just style. In PostgreSQL it is a speed rule (Chapter 10).

Problem 14.3.2 Top-2 products per category with `QUALIFY`. SQLite has no `QUALIFY`, so we run the portable form with a CTE, and show the warehouse form beside it.

```
WITH r AS (
  SELECT p.category, p.name, SUM(oi.qty*oi.unit_price) AS rev
  FROM order_items oi JOIN products p USING(product_id) JOIN orders o USING(order_id)
  WHERE o.status='paid' GROUP BY p.product_id),
k AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY rev DESC) AS rn FROM r)
SELECT category, name, rev FROM k WHERE rn<=2 ORDER BY category, rn;
```

```
category     name       rev
-----------  ---------  ----
Electronics  Keyboard   4500
Electronics  Mouse      3000
Home         Desk Lamp  3600
Stationery   Pen Pack   840
Stationery   Notebook   720
```

- **Check** Electronics: Keyboard 4500 and Mouse 3000. Home has only one sold product that was paid (Desk Lamp 3600; Mug never sold). Stationery: Pen Pack 840, Notebook 720.
- **Answer** Five rows (one category has only one product).
- **Snowflake / BigQuery form (not run here):**

```
SELECT p.category, p.name, SUM(oi.qty*oi.unit_price) AS rev
FROM order_items oi JOIN products p USING (product_id) JOIN orders o USING (order_id)
WHERE o.status = 'paid'
GROUP BY p.category, p.name
QUALIFY ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(oi.qty*oi.unit_price) DESC) <= 2;
```

- **Meaning** `QUALIFY` is to window functions what `HAVING` is to aggregates: it filters after the window is computed, saving one CTE.

The picture

```
 ROW STORE (Postgres, MySQL, SQL Server)    COLUMN STORE (BigQuery, Snowflake)
 [id|name|city|amount]  one row together    id:     1 2 3 4 ...
 [id|name|city|amount]                      amount: 5 9 2 7 ...  <- SUM reads only this
 good: fetch one whole row                  good: scan one column over billions of rows
```

Connection to the real job

Most analytics-engineering jobs today use BigQuery or Snowflake with dbt. The cost model changes habits: you partition by date, avoid `SELECT *`, and watch warehouse sizes.

Applications

- **Data Analytics:** dashboards on a warehouse.
- **Data Engineering:** ELT transformations in the warehouse.
- **ML:** feature tables built in SQL, then exported to Python.

COMMON MISTAKE

Assuming `LIMIT 10` makes a BigQuery query cheap. The scan happens before the limit, so you can still pay for the whole table. Use a partition filter or a table preview instead. Also, do not trust primary keys in a warehouse: nothing stops duplicates.

Check yourself

1. What do you pay for in on-demand BigQuery? Answer: the bytes scanned.
2. What does `QUALIFY` filter? Answer: the result of window functions, like `HAVING` for aggregates.
3. How does Snowflake separate storage and compute? Answer: data lives once; you start or pause virtual warehouses (compute) independently.

14.4 Porting a Query: A Method

INTUITION

Translating a document. You do not translate word by word; you read for meaning, then rewrite in the other language. Porting SQL is the same: separate the logic (portable) from the spellings (dialect-specific).

How it works

1. Write the query in the standard core: CTEs, `CASE`, `COALESCE`, `JOIN`.
2. List dialect-specific pieces: row limit, dates, strings, division, types.
3. Replace them one by one using the phrasebook.
4. Run on a small sample in both systems and compare row counts and totals.

Worked examples

Problem 14.4.1 Port "orders per month with paid share". The SQLite version:

```
SELECT substr(order_date,1,7) AS ym, COUNT(*) AS orders,
  SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END) AS paid,
  ROUND(100.0*SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END)/COUNT(*),1) AS paid_pct
FROM orders GROUP BY substr(order_date,1,7) ORDER BY ym;
```

```
ym       orders  paid  paid_pct
-------  ------  ----  --------
2024-01  2       2     100.0
2024-02  3       2     66.7
2024-03  5       4     80.0
2024-04  4       4     100.0
```

- **Check** February has orders 103, 104 (cancelled), 105: 3 orders, 2 paid, 2/3 = 66.7%. March has 106 to 110: 5 orders, 4 paid (109 refunded), 80%.
- **Answer** Paid share by month: 100, 66.7, 80, 100.
- **Port to PostgreSQL (not run here):** replace `substr(order_date,1,7)` with `TO_CHAR(order_date,'YYYY-MM')` (if `order_date` is a real `DATE`). `ROUND(x, 1)` on a decimal works. Replace the paid count with `COUNT(*) FILTER (WHERE status='paid')`.
- **Port to SQL Server (not run here):** `FORMAT(order_date,'yyyy-MM')`; `100.0 * ...` already forces decimals; `ROUND` works.
- **Port to BigQuery (not run here):** `FORMAT_DATE('%Y-%m', order_date)` and `COUNTIF(status='paid')`.
- **Meaning** Only the month label and the conditional count changed. The shape (`GROUP BY`, `CASE`, ratio) is portable.

Problem 14.4.2 The classic NULL trap behaves the same on all databases. How many customers are not a referrer of anyone? Compare `NOT IN` and `NOT EXISTS`.

```
SELECT COUNT(*) AS wrong FROM customers
WHERE customer_id NOT IN (SELECT referrer_id FROM customers);
SELECT COUNT(*) AS right_ FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM customers r WHERE r.referrer_id=c.customer_id);
```

```
wrong
-----
0
right_
------
4
```

- **Check** Referrers are customers 1, 2, 3 and 5. So 8 - 4 = 4 customers (4, 6, 7, 8) are not referrers.
- **Answer** `NOT IN` returns 0 because `referrer_id` contains NULLs: `x NOT IN (..., NULL)` is never true. `NOT EXISTS` returns the correct 4.
- **Meaning** This is a logic issue, not a dialect issue, and it is a favourite interview question.

The picture

```
 query --> [portable core] ------------------> works everywhere
       \-> [dates][strings][limit][division] -> swap per dialect
```

Connection to the real job

dbt projects use macros (`dbt.date_trunc`) to hide dialect differences, so the same model runs on several warehouses. Knowing the phrasebook helps you read those macros.

Applications

- **Migrations:** SQL Server to PostgreSQL, on-prem to cloud.
- **Analytics engineering:** cross-warehouse dbt packages.
- **Interviews:** "write this in SQL Server" follow-up questions.

COMMON MISTAKE	Porting the syntax and not the semantics: for example text dates in SQLite (`'2024-03-18'` is just text) versus real `DATE` types elsewhere. Check types first.

Check yourself

1. First step in porting? Answer: write or identify the portable core, then list dialect-specific pieces.
2. How do you count conditionally in BigQuery? Answer: `COUNTIF(condition)` .
3. Why does `NOT IN` fail with NULLs? Answer: comparing with NULL gives unknown, so no row can be proven "not in".

14.5 The 40-Problem Interview Bank

INTUITION	An interview is a pattern-matching game. Forty well-chosen problems cover most of what is asked in analyst and data-engineer screens. Try each one yourself for five minutes before reading the answer. All outputs are from the shop database of Chapter 2 (customers, products, orders, order_items) run in SQLite.

How to use this bank

- **Part A (problems 1 to 24):** write the query. Try before reading.
- **Part B (problems 25 to 40):** explain in two or three sentences, the way you would speak in the interview.
- Say your plan aloud first ("I will join, filter paid, group by customer"). Interviewers score the plan.
- The checks given are by hand against the base data.

Part A: Write the query

Problem 14.1 (easy) How many customers live in Pune? **Answer:** `SELECT COUNT(*) FROM customers WHERE city='Pune';` returns 3 (Asha, Meena, Tom).

Problem 14.2 (NULL trap) How many orders have a coupon that is not 'SAVE10'? Then the version that treats no coupon as "not SAVE10". **Answer:** `WHERE coupon <> 'SAVE10'` returns 0 rows, because `NULL <> 'SAVE10'` is unknown. The fix is `WHERE coupon IS NULL OR coupon <> 'SAVE10'`, giving 11 (14 orders minus 3 SAVE10 orders).

Problem 14.3 What is the second most expensive product? **Answer:** `SELECT name, price FROM products ORDER BY price DESC LIMIT 1 OFFSET 1;` gives Desk Lamp 900 (Keyboard is 1500). For ties, use `DENSE_RANK() = 2` instead.

Problem 14.4 Which customers never ordered? **Answer:** `LEFT JOIN orders ... WHERE o.order_id IS NULL` (anti-join) gives Sana and Tom.

Problem 14.5 Difference between `COUNT(*)`, `COUNT(coupon)`, `COUNT(DISTINCT coupon)` on orders? **Answer:** 14, 3 and 1. `COUNT(coupon)` skips NULLs; the 3 coupons are all 'SAVE10', so 1 distinct value.

Problem 14.6 Paid revenue by category. **Answer:** Join `order_items`, `products`, `orders`, filter `status='paid'`, group by category. Electronics 7500, Home 3600, Stationery 1560. Check: sum is 12,660.

Problem 14.7 Customers with more than 2 paid orders. **Answer:** `GROUP BY customer ... HAVING COUNT(*) > 2` gives Asha 4 and Ravi 3.

Problem 14.8 Products that never sold. **Answer:** `WHERE NOT EXISTS (SELECT 1 FROM order_items ...)` gives Mug.

Problem 14.9 Number of orders per status. **Answer:** paid 12, refunded 1, cancelled 1 (`GROUP BY status`).

Problem 14.10 For each referred customer show the referrer's name, but only when they live in different cities. **Answer:** A self join (`customers c JOIN customers r ON r.customer_id = c.referrer_id WHERE c.city <> r.city`) gives Ravi (Mumbai) referred by Asha (Pune), John (Delhi) by Ravi

(Mumbai), Kabir (Mumbai) by Priya (Bengaluru). Meena and Tom share a city with their referrers, and customers with a NULL referrer drop out of the inner join.

Problem 14.11 Rank customers by paid revenue with no gaps for ties. **Answer:** `DENSE_RANK() OVER (ORDER BY rev DESC)` gives Asha 4560 (1), Ravi 3600 (2), John 3300 (3), Priya 760 (4), Kabir 440 (5).

Problem 14.12 Each customer's second order. **Answer:** `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)` and `rn = 2` returns: customer 1 order 103, customer 2 order 108, customer 4 order 112, customer 5 order 109 (a refunded order, because we did not filter status; say this aloud and ask the interviewer which orders count).

Problem 14.13 Days since the previous order for customer 1. **Answer:** `LAG(order_date)` and a date difference gives NULL, 24, 31, 28 days (orders 101, 103, 106, 111). Check: Jan 10 to Feb 3 is 24 days.

Problem 14.14 Customers who bought in every month January to April (relational division). **Answer:** Group paid orders by customer and `HAVING COUNT(DISTINCT month) = 4`. Only Asha.

Problem 14.15 Cumulative percent of revenue by customer, biggest first (Pareto). **Answer:** `SUM(rev) OVER (ORDER BY rev DESC) / SUM(rev) OVER ()` gives 36.0, 64.5, 90.5, 96.5, 100.0. The top 3 of 5 customers give 90.5% of revenue.

Problem 14.16 Customers whose April spend exceeded March spend. **Answer:** Conditional aggregation with `COALESCE(mar,0)`: customer 2 (900 to 2100), customer 4 (none to 1800) and customer 5 (360 to 400). The `COALESCE` matters for customer 4, who had no March orders.

Problem 14.17 Percent of paid revenue per category. **Answer:** Divide each category sum by a scalar subquery of the total: Electronics 59.2, Home 28.4, Stationery 12.3 (sums to 99.9 because of rounding).

Problem 14.18 Orders per month split by status (manual pivot). **Answer:** `SUM(status='paid')` and so on per month gives: Jan 2/0/0, Feb 2/1/0, Mar 4/0/1, Apr 4/0/0 (paid/cancelled/refunded). In PostgreSQL write `COUNT(*) FILTER (WHERE status='paid')` (not run here).

Problem 14.19 Which orders have more than one line? **Answer:** `GROUP BY order_id HAVING COUNT(*) > 1` gives 101, 103, 106, 110, 113, 114 (two lines each).

Problem 14.20 What does `SELECT 7/2` return in SQLite? How to get 3.5? **Answer:** 3. Use `7*1.0/2` or `CAST(7 AS REAL)/2`.

Problem 14.21 Compute the median waiting time (minutes) of 120, 130, 150, 160, 170, 900 for hospital appointments. **Answer:** Number the rows, take the average of the middle two (rows 3 and 4): (150 + 160) / 2 = 155, from Chapter 12.6. Mean is 271.7, which shows the outlier effect.

Problem 14.22 Remove duplicate sign-ups, keeping the newest per email. **Answer:** `ROW_NUMBER() OVER (PARTITION BY lower(trim(email)) ORDER BY updated_at DESC, id DESC)` and keep `rn = 1` (Chapter 12.6).

Problem 14.23 Longest login streak. **Answer:** Island trick: date minus `ROW_NUMBER()` as a group key; `COUNT(*)` per group (Chapter 12.5).

Problem 14.24 Customers whose every order is paid. **Answer:** `GROUP BY customer HAVING SUM(status <> 'paid') = 0` returns Asha, John, Kabir and Ravi. Meena (cancelled) and Priya (one refunded) are excluded; Sana and Tom have no orders so they never appear (decide if "no orders" should count).

Part B: Explain it

Problem 14.25 `WHERE` versus `HAVING`. **Answer:** `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation. You cannot use `SUM()` in `WHERE`.

Problem 14.26 INNER JOIN versus LEFT JOIN. **Answer:** INNER keeps only matching pairs. LEFT keeps every left row and fills right columns with NULL when there is no match. Putting a right-table filter in `WHERE` can silently turn a LEFT join into an inner join; put it in `ON`.

Problem 14.27 `UNION` versus `UNION ALL`. **Answer:** `UNION` removes duplicates (extra sort or hash); `UNION ALL` keeps them and is faster. Use

`UNION ALL` unless you need de-duplication.

Problem 14.28 `IN` versus `EXISTS`, and the `NOT IN` danger. **Answer:** `EXISTS` stops at the first match and is safe with NULLs. `NOT IN` returns nothing if the subquery contains a NULL (problem 14.4.2 above: 0 instead of 4).

Problem 14.29 `RANK`, `DENSE_RANK`, `ROW_NUMBER` on the values 100, 90, 90, 80. **Answer:** (verified by running) `ROW_NUMBER` gives 1, 2, 3, 4. `RANK` gives 1, 2, 2, 4 (gap after the tie). `DENSE_RANK` gives 1, 2, 2, 3.

Problem 14.30 `DELETE`, `TRUNCATE`, `DROP`. **Answer:** `DELETE` removes chosen rows, can be rolled back, and fires triggers. `TRUNCATE` empties the table quickly (rollback support depends on the database). `DROP` removes the table itself.

Problem 14.31 Primary key versus unique key. **Answer:** A table has one primary key (unique and not NULL) as its row identity. It can have many unique constraints; they can usually allow NULL (how many NULLs differs by dialect).

Problem 14.32 What are the ACID properties? **Answer:** Atomicity (all or nothing), Consistency (rules always hold), Isolation (concurrent transactions do not see each other's half-done work), Durability (committed data survives a crash). Example: a bank transfer debits and credits together or not at all.

Problem 14.33 Name the problem each isolation level prevents. **Answer:** Read committed prevents dirty reads. Repeatable read also prevents non-repeatable reads (the same row changing between two reads). Serializable also prevents phantom rows and write-skew type anomalies. Higher isolation costs concurrency.

Problem 14.34 When does an index NOT help? **Answer:** On tiny tables, on low-selectivity columns (a boolean), when the query uses a function on the column (`WHERE LOWER(email) = ...`), on `LIKE '%text'`, and on write-heavy tables where each index slows inserts.

Problem 14.35 What is a composite index and why does column order matter? **Answer:** An index on several columns, sorted by the first, then the second. An index on `(city, signup_date)` helps filters on `city` alone or both, but not on `signup_date` alone (leftmost-prefix rule).

Problem 14.36 Explain normalisation to 3NF in two sentences. **Answer:** 1NF: one value per cell, no repeating groups. 2NF: every non-key column depends on the whole key. 3NF: non-key columns depend only on the key, not on other non-key columns. Result: each fact stored once.

Problem 14.37 OLTP versus OLAP, and star schema. **Answer:** OLTP serves many small transactions on normalised row stores. OLAP serves large analytical scans, often on column stores. A star schema has one fact table (events with measures) surrounded by dimension tables (customer, product, date).

Problem 14.38 What is a deadlock and how do you reduce it? **Answer:** Two transactions each hold a lock the other needs, so neither can continue; the database kills one. Reduce it by locking rows in a consistent order, keeping transactions short, and retrying the victim.

Problem 14.39 CTE versus subquery versus temp table versus view. **Answer:** A CTE names a step for one query (readability). A subquery is inline. A temp table stores intermediate results for the session (reuse and indexing). A view saves a query's definition so others can reuse it; a materialized view also stores the result.

Problem 14.40 State CAP in one sentence and name one CP and one AP store. **Answer:** When the network splits, a distributed system must choose between consistency and availability. PostgreSQL with a single primary or HBase lean CP; Cassandra at low consistency levels or DynamoDB with eventually consistent reads lean AP (behaviour is configurable, see Chapter 13).

The picture

```
Interview answer shape:
1. Restate the question and the tables       (10 s)
2. Ask about NULLs, ties, which statuses     (10 s)
3. Say the plan: join -> filter -> group     (20 s)
4. Write the query, then check on 2 rows     (3 min)
5. Mention performance / edge cases          (30 s)
```

Practise this shape. Silence is the most common reason for failure; narrate while you type.

Connection to the real job

These patterns are the daily toolkit; the interview simply compresses it. Time how long you need for each problem and aim for under five minutes for 1 to 20 and under ten for 21 to 24.

Applications

- **Analyst screens:** problems 1 to 20.
- **Data engineer screens:** 21 to 24 and Part B.
- **System design rounds:** 37 and 40.

COMMON MISTAKE

Memorising answers instead of the reason. If the interviewer changes one word (ties, NULLs, a second table) a memorised query breaks. Know why each clause is there.

Check yourself

1. Which clause removes groups? Answer: `HAVING`.
2. What does `ROW_NUMBER` do with ties? Answer: gives unique numbers anyway, in an arbitrary order unless you add a tie-breaker.
3. Which isolation anomaly does read committed allow? Answer: non-repeatable reads (and phantoms); it prevents dirty reads.

14.6 Dialect Translation Tables: Common Tasks Across Seven Engines

INTUITION

Every SQL database speaks the same core and a different accent. The core (`SELECT` , `JOIN` , `GROUP BY` , windows, CTEs) moves anywhere. The accent shows in dates, strings, upserts, top-N and quoting, and those are exactly the places a copied query fails. These tables are a phrasebook for the accent. The SQLite column below was run; all other engines were **not run here** and follow their documentation.

PostgreSQL, MySQL and SQLite

Task	PostgreSQL	MySQL 8	SQLite
First day of month	`date_trunc('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`date(d, 'start of month')`
Add 7 days	`d + INTERVAL '7 days'`	`DATE_ADD(d, INTERVAL 7 DAY)`	`date(d, '+7 days')`
Days between	`d2 - d1` (dates)	`DATEDIFF(d2, d1)`	`julianday(d2) - julianday(d1)`
Join strings in a group	`string_agg(x, ', ')`	`GROUP_CONCAT(x SEPARATOR ', ')`	`group_concat(x, ', ')`
Concatenate	`a || b`	`CONCAT(a, b)` (|| means OR by default)	`a || b`
Replace NULL	`COALESCE(x, 0)`	`COALESCE(x, 0)` , `IFNULL`	`COALESCE(x, 0)` , `IFNULL`
Top N	`LIMIT n` or `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`LIMIT n`
Upsert	`INSERT ... ON CONFLICT (k) DO UPDATE`	`INSERT ... ON DUPLICATE KEY UPDATE`	`INSERT ... ON CONFLICT (k) DO UPDATE`
Auto id	`GENERATED ALWAYS AS IDENTITY`	`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`
Quote a name	`"name"`	`` `name` ``	`"name"` or `[name]`

Task	PostgreSQL	MySQL 8	SQLite
Boolean type	`BOOLEAN`	`BOOLEAN` is an alias of TINYINT(1)	0 and 1 stored as integers

SQL Server and Oracle

Task	SQL Server	Oracle
First day of month	`DATETRUNC(month, d)` (2022+) or `DATEFROMPARTS(YEAR(d), MONTH(d), 1)`	`TRUNC(d, 'MM')`
Add 7 days	`DATEADD(day, 7, d)`	`d + 7`
Days between	`DATEDIFF(day, d1, d2)`	`d2 - d1`
Join strings in a group	`STRING_AGG(x, ', ')` (2017+)	`LISTAGG(x, ', ') WITHIN GROUP (ORDER BY x)`
Concatenate	`a + b` or `CONCAT(a, b)`	`a || b`
Replace NULL	`ISNULL(x, 0)`, `COALESCE`	`NVL(x, 0)`, `COALESCE`
Top N	`SELECT TOP n ...` or `OFFSET 0 ROWS FETCH NEXT n ROWS ONLY`	`FETCH FIRST n ROWS ONLY` (12c+)
Upsert	`MERGE`	`MERGE`
Auto id	`IDENTITY(1,1)`	`GENERATED AS IDENTITY` (12c+)
Quote a name	`[name]` or `"name"`	`"name"`
Boolean type	`BIT`	no boolean column before 23ai; use NUMBER(1) or CHAR(1)

BigQuery and Snowflake

Task	BigQuery	Snowflake
First day of month	`DATE_TRUNC(d, MONTH)`	`DATE_TRUNC('month', d)`
Add 7 days	`DATE_ADD(d, INTERVAL 7 DAY)`	`DATEADD(day, 7, d)`
Days between	`DATE_DIFF(d2, d1, DAY)`	`DATEDIFF(day, d1, d2)`
Join strings in a group	`STRING_AGG(x, ', ')`	`LISTAGG(x, ', ')`
Concatenate	`a || b` or `CONCAT`	`a || b` or `CONCAT`
Replace NULL	`IFNULL(x, 0)`, `COALESCE`	`NVL(x, 0)`, `COALESCE`, `IFNULL`
Top N	`LIMIT n`	`LIMIT n` or `TOP n`
Top 1 per group	`QUALIFY ROW_NUMBER() OVER (...) = 1`	`QUALIFY ROW_NUMBER() OVER (...) = 1`
Upsert	`MERGE`	`MERGE`
Pivot	`PIVOT (SUM(x) FOR k IN ('a','b'))`	`PIVOT (SUM(x) FOR k IN ('a','b'))`
Quote a name	`` `name` ``	`"name"` (upper-case by default)

Problem 14.6.1 Write 'paid revenue per month' in SQLite (run), then translate it. Predict first: four months, with January smallest.

```
SELECT date(order_date, 'start of month') AS month, sum(i.qty * i.unit_price) AS revenue
FROM orders o JOIN order_items i ON i.order_id = o.order_id
WHERE o.status = 'paid'
GROUP BY month ORDER BY month;
```

```
month       revenue
----------  -------
2024-01-01  2260
2024-02-01  2520
2024-03-01  2980
2024-04-01  4900
```

Step 1	The only dialect-specific piece is the month bucket. The rest (`JOIN` , `WHERE` , `GROUP BY` , `sum`) is portable.
Step 2	Check: the four monthly totals 2,260 + 2,520 + 2,980 + 4,900 add to 12,660, the total of all paid order lines.
Translate	PostgreSQL and Snowflake: `date_trunc('month', o.order_date)` . BigQuery: `DATE_TRUNC(o.order_date, MONTH)` . MySQL: `DATE_FORMAT(o.order_date, '%Y-%m-01')` . SQL Server: `DATETRUNC(month, o.order_date)` or `DATEFROMPARTS(YEAR(o.order_date), MONTH(o.order_date), 1)` . Oracle: `TRUNC(o.order_date, 'MM')` . In PostgreSQL and SQL Server `order_date` must be a real date type for these to work, whereas in SQLite it is text.

Three quick SQLite checks of other cells in the table, run for real:

```
SELECT group_concat(name, ', ') AS pune_customers FROM (SELECT name FROM customers WHERE city = 'Pune' ORDER BY name);
SELECT date('2024-01-31', '+7 days') AS plus7, julianday('2024-03-01') - julianday('2024-02-01') AS days_between, 10 / 4 AS
int_div;
```

```
pune_customers
----------------
Asha, Meena, Tom
plus7       days_between  int_div
----------  ------------  -------
2024-02-07  29.0          2
```

Other engines you will meet

- **DuckDB**: an embedded analytical database (like SQLite for analytics). It reads Parquet and CSV files directly (`SELECT * FROM 'file.parquet'`), follows PostgreSQL syntax closely, and supports `QUALIFY` , `PIVOT` and `GROUP BY ALL` . Not run here.
- **Spark SQL, Trino/Presto, ClickHouse**: query engines for large files and warehouses. Core SQL is the same, but date functions (`date_trunc` , `date_add`), array and map types, and the absence of enforced constraints differ. They run queries over data, they do not give transactional guarantees the way PostgreSQL does.
- **CockroachDB, Google Spanner, YugabyteDB**: distributed SQL databases ('NewSQL'). They offer PostgreSQL-style SQL with transactions spread across machines, at the price of extra latency per commit. Use them when one machine is not enough but you still need ACID.
- **MariaDB**: a MySQL fork. Close to MySQL 5.7 and 8.0 with some differences (for example sequences, and different JSON handling).

COMMON MISTAKE	Assuming `||` concatenates strings everywhere. In MySQL (default mode) `||` is the logical OR, so `'a' || 'b'` returns 0, silently. In SQL Server `||` is a syntax error and `+` concatenates, but `+` with a NULL gives NULL while `CONCAT` ignores NULLs. Use `CONCAT(a, b)` when portability matters, and remember it treats NULL differently from `||` in PostgreSQL (where `||` with NULL gives NULL).

Practice with answers

Problem 14.6.2 Translate `SELECT name FROM customers ORDER BY signup_date DESC LIMIT 3` for SQL Server and Oracle. *(easy)*

Answer: SQL Server: `SELECT TOP 3 name FROM customers ORDER BY signup_date DESC`. Oracle 12c and later: `SELECT name FROM customers ORDER BY signup_date DESC FETCH FIRST 3 ROWS ONLY`. Both have to keep `ORDER BY`, otherwise 'top' is arbitrary.

Problem 14.6.3 Rewrite `SELECT dept, GROUP_CONCAT(name) FROM emp GROUP BY dept` (MySQL) for PostgreSQL, Snowflake and Oracle. *(easy)*

Answer: PostgreSQL: `string_agg(name, ',')`. Snowflake: `LISTAGG(name, ',')`. Oracle: `LISTAGG(name, ',') WITHIN GROUP (ORDER BY name)`. MySQL's default result is cut at 1024 characters (`group_concat_max_len`), which the others do not do.

Problem 14.6.4 A query using `QUALIFY` must run on PostgreSQL. How do you port it? *(medium)*

Answer: Put the window function in a CTE or subquery that exposes it as a column (`rn`), then filter in the outer `WHERE rn = 1`. `QUALIFY` is only a shortcut for that pattern.

Problem 14.6.5 Why do many teams write `CAST(x AS DATE)` rather than engine-specific date functions where possible? *(interview)*

Answer: `CAST` and `CURRENT_DATE`, `COALESCE`, `CASE`, `EXTRACT` are in the SQL standard, so they port unchanged. Engine-specific shortcuts (`IFNULL`, `NVL`, `DATEADD`) force a rewrite per engine. For code that must run on several engines, prefer standard functions or isolate dialect functions in one place (a dbt macro, a view).

Chapter recap

- Core SQL is portable; trouble sits in row limits, strings, dates, types, upsert, quoting and division.
- PostgreSQL: `LIMIT`, `||`, `DATE_TRUNC('month', d)`, `ON CONFLICT`. MySQL: `CONCAT`, `ON DUPLICATE KEY UPDATE`, windows only from 8.0. SQL Server: `TOP`, `+`, `CROSS APPLY`, `MERGE`.
- BigQuery bills by bytes scanned; partition and cluster, select only needed columns. Snowflake separates storage and compute; both support `QUALIFY` and treat keys as informational.
- Port a query by keeping the portable core, listing dialect pieces and replacing them one by one; then compare totals.
- `7/2` can be 3 or 3.5 depending on the database; always force decimals and guard zero.
- `NOT IN` with a NULL returns nothing; use `NOT EXISTS`.
- In interviews: clarify NULLs, ties and statuses, narrate a plan, then write and check the query.

Task	PostgreSQL	MySQL	SQL Server	BigQuery	Snowflake
Limit	`LIMIT`	`LIMIT`	`TOP`	`LIMIT`	`LIMIT` / `TOP`
Concat	`\`	\	`	`CONCAT`	`+` / `CONCAT`
Month start	`DATE_TRUNC('month',d)`	`DATE_FORMAT`	`DATEFROMPARTS`	`DATE_TRUNC(d,MONTH)`	`DATE_TRUNC('month',d)`
Upsert	`ON CONFLICT`	`ON DUPLICATE KEY`	`MERGE`	`MERGE`	`MERGE`
Safe divide	`NULLIF`	`/` gives NULL on 0	`NULLIF`	`SAFE_DIVIDE`	`DIV0`

Practice set

Problem 14.P.1 (easy) Write the SQL Server form of `SELECT name FROM products ORDER BY price DESC LIMIT 3`. **Answer:** `SELECT TOP 3 name FROM products ORDER BY price DESC;` (not run here).

Problem 14.P.2 (easy) Which dialect pieces change in "orders per month" when you move from SQLite to BigQuery? **Answer:** The month label (`FORMAT_DATE('%Y-%m', order_date)` instead of `substr`) and optionally `COUNTIF`. The `GROUP BY` / `CASE` shape stays.

Problem 14.P.3 (computation) A BigQuery table is 400 GB. You select 3 columns that are 5% of the bytes each. How much is scanned? **Answer:** 3 x 5% = 15% of 400 GB = 60 GB.

Problem 14.P.4 (what is the output?) `SELECT 9/2, 9/2.0, 9%2;` in SQLite. **Answer:** 4, 4.5, 1.

Problem 14.P.5 (find the bug) A MySQL query uses `'Hello ' || name` and returns 0 for every row. Why? **Answer:** In MySQL's default mode `||` is logical OR. Use `CONCAT('Hello ', name)`.

Problem 14.P.6 (find the bug) `SELECT name FROM customers WHERE customer_id NOT IN (SELECT referrer_id FROM customers)` returns no rows. Why, and what is the fix? **Answer:** `referrer_id` has NULLs. Use `NOT EXISTS`, or add `WHERE referrer_id IS NOT NULL` inside the subquery (gives 4 customers).

Problem 14.P.7 Find the top-1 product by paid revenue in each category on Snowflake. **Answer:** Aggregate by product, then `QUALIFY ROW_NUMBER() OVER (PARTITION BY category ORDER BY rev DESC) = 1` (not run here). Portable form: CTE with `rn = 1`. Result: Keyboard 4500, Desk Lamp 3600, Pen Pack 840.

Problem 14.P.8 Rank the values 50, 50, 40, 30 with `RANK` and `DENSE_RANK`. **Answer:** RANK 1, 1, 3, 4. DENSE_RANK 1, 1, 2, 3.

Problem 14.P.9 (scenario) Your company moves from SQL Server to BigQuery. Name four things in a query to check. **Answer:** `TOP` becomes `LIMIT`; `+` string concat becomes `||` / `CONCAT`; `DATEDIFF(day,a,b)` becomes `DATE_DIFF(b,a,DAY)`; `ISNULL` becomes `IFNULL` / `COALESCE`; also `SELECT *` costs money and keys are not enforced.

Problem 14.P.10 (interview) The same query returns 12,660 on PostgreSQL and 14,560 on a warehouse copy. What do you check first? **Answer:** The row filters (14,560 is the all-status total in the shop data: cancelled 400 and refunded 1500 added to 12,660), so check that the `status='paid'` filter or its data (case, spaces, new status values) is the same. Then check duplicate rows in the copy, since warehouses do not enforce keys.

Problem 14.P.11 (interview) Why can `LIMIT 10` still be expensive in BigQuery? **Answer:** Billing is by bytes scanned, and the scan happens before the limit; a partition filter or fewer columns reduces cost, `LIMIT` does not.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles and I know PostgreSQL-style SQL. Give me a phrasebook that translates 15 common tasks (row limit, string concat, date truncation, date difference, upsert, string aggregation, safe division) between PostgreSQL, MySQL, SQL Server, BigQuery and Snowflake. Then quiz me by giving a query in one dialect and asking me to port it, one at a time.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing for data analyst interviews. Run a mock SQL interview using a shop database with customers, products, orders and order_items. Ask one problem at a time, from easy to hard, let me write the query, then grade it out of 10, show a model answer, and name the single most important mistake I made. Start with problem 1.
```

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional learning cloud warehouses. Explain how BigQuery and Snowflake differ from PostgreSQL: columnar storage, partitioning and clustering, cost models, virtual warehouses, QUALIFY, semi-structured data, and unenforced keys. Use small numeric examples for cost, and end with five check questions that you ask me one at a time.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional learning SQL. Review the query below that I wrote for PostgreSQL and tell me exactly what would break or change if I ran it on MySQL, SQL Server and BigQuery. Explain each difference in simple English and show the corrected line for each dialect. My query: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional preparing for the theory part of data interviews. Quiz me one question at a time on SQL theory: ACID, isolation levels, indexes, normalization, OLTP vs OLAP, deadlocks, views vs CTEs, CAP. After each answer, grade me out of 10, correct any wrong parts in plain English, and tell me how to say the answer in 30 seconds.

CHAPTER 15

PROJECT LAB, PART 1: FUNNELS, COHORTS, RFM AND A SALES DATABASE

How to plan, build, test and present SQL portfolio projects, with three complete projects whose code and output were really run.

A hiring manager does not read your certificates. She opens your repository and asks one question: "Can this person take a vague business question and turn it into a correct, explained answer?" This chapter shows you how to build projects that answer yes. First you learn the working method that every good project shares: plan, layout, data, commits, tests, README, post. Then you build three projects end to end: a funnel and cohort-retention analysis, an RFM customer segmentation with a written report, and a sales database designed from scratch with twenty business queries. Every query below was run in SQLite 3.50 and the output shown is the real output. By the end you will have three finished repositories and the words to talk about them in an interview.

15.1 How to Run a Project in This Area

INTUITION	Think of a project like cooking a meal for guests, not like doing homework. You decide the menu first (the question), buy the ingredients (the data), cook step by step tasting as you go (tests), and finally you plate it nicely and write the menu card (README). A great dish with no menu card, or a menu card with no dish, both disappoint.

Plan the question and the success criteria

Write the question as one sentence a manager would say, then write how you will know you are right. Do this before any SQL.

Item	Weak version	Strong version
Question	"Analyse the shop data"	"Which signup channel loses the most visitors before payment, and do buyers come back?"
Success criteria	"Make some queries"	"Funnel counts reconcile with the orders table; cohort retention matches an independent Python check; 1-page report with 3 recommendations"
Out of scope	(nothing)	"No forecasting, no dashboards"

Repository layout

```
sql-project-name/
  README.md             the front door (template below)
  data/
    generate_data.py    creates the database; fixed random seed
    (no big files; the generator rebuilds them)
  sql/
    00_schema.sql       tables, constraints, indexes
    01_checks.sql       row counts, NULLs, duplicates
    02_analysis.sql     the real queries, numbered, commented
  tests/
    test_results.py     Python asserts that cross-check the SQL
  report/
    findings.md         the written answer for a manager
  images/               charts or screenshots for the README
```

Environment and dependencies

SQLite ships inside Python, so `python3` plus the `sqlite3` command-line tool is enough for every project in this chapter. Nothing to install. If you prefer PostgreSQL, keep the SQL portable and note the few date functions that change (Chapter 14). Record your versions in the README: `python3 --version` and `sqlite3 --version`.

Data sourcing or generation

Use public data (government open-data portals, Kaggle, UCI) when the licence allows, and write the licence and source URL in the README. When no good data exists, **generate** it with a script and a fixed seed, so anyone gets identical numbers. Say clearly that the data is synthetic. A generator also lets you plant a known pattern (for example "social channel converts badly") and check that your SQL finds it. That is a real test.

Commit rhythm and good commit messages

Commit after every working step, at least once per session. A message says what and why in the present tense:

```
Good:  Add strict funnel query; count users by order of events
Good:  Fix cohort month maths: use year*12+month, not date subtraction
Weak:  update
Weak:  final final v2
```

A history of ten honest commits shows how you think. One giant commit shows nothing.

Definition of done

- The question and success criteria are written in the README.
- One command rebuilds the database; another runs the tests; both are in the README.
- Every headline number is reproduced a second way (Python, a hand count, or a reconciliation to a total).
- Edge cases are handled and mentioned: NULLs, ties, incomplete months, cancelled orders.
- The report states what the data cannot tell you.
- A stranger can follow the README without asking you anything.

README template

```
# Project title: one line that states the result
**Question:** ...
**Answer in one paragraph:** the 3 headline numbers.

## Data
Source or generator, row counts, licence, what each table means.

## How to run
python3 data/generate_data.py shop.db
sqlite3 shop.db < sql/02_analysis.sql
python3 tests/test_results.py

## Method
Numbered steps. Key decisions and why (for example: why a "paid" order counts).

## Results
Table or image. Two or three sentences of meaning, in plain English.

## Checks
What I cross-checked and how.

## Limits
What this analysis cannot say.

## Next steps
Three extensions.
```

Demo and deploy options

SQL has no "server" to deploy, so show the work this way. (1) A public GitHub repository with a good README and a few result tables pasted in. (2) A screenshot or a chart image from matplotlib (not run in this chapter; see Chapter 13 for the Python bridge). (3) A short screen recording or GIF walking through the findings. (4) A free hosted notebook, if you like, that runs the same SQL. Option 1 plus the report is enough.

How to write the LinkedIn or blog post

Use four short paragraphs: **the question** (one sentence a non-technical person understands), **the surprising finding** (one number), **how you checked it** (one sentence; this earns trust), **the link**. Do not paste code. Do not claim business impact you did not measure; say "on synthetic data" when it is synthetic.

Testing and debugging checklist for SQL projects

1. Row counts: after every join, count again. A join that changes the row count unexpectedly means a duplicate key or a many-to-many.
2. Reconcile: pieces must add up to the whole (revenue by region equals total revenue).
3. Count people, not rows: use `COUNT(DISTINCT user_id)` for funnels and retention.
4. NULL check: `COUNT(*)` versus `COUNT(column)` for every column you group or join on.
5. Boundary dates: first and last day, and months that are only partly in the data (right-censoring).
6. Ties: if `ORDER BY` has ties and you use `LIMIT`, add a tie-breaker column, or the answer can change run to run.
7. Independent check: recompute one headline number in Python or by hand on a small sample.
8. Read the plan (`EXPLAIN QUERY PLAN`) if a query takes more than a second.

Check yourself

1. Why do we generate data with a fixed random seed? Answer: so everyone, including you next month, gets exactly the same numbers and the tests stay valid.
2. Name two ways to verify a headline number. Answer: recompute it in Python, and reconcile parts to a known total.
3. What does a commit message explain that the code cannot? Answer: why the change was made.

15.2 Project: Funnel and Cohort-Retention Analysis for an E-Commerce Shop

Goal and why employers care

Question: **where do visitors drop out before paying, does it differ by signup channel, and do buyers come back in later months?** Funnels tell a company where to fix the website. Retention tells it whether customers like the product after the first purchase. Almost every analyst job description for an online business contains these two words. Showing both, with checks, is a strong portfolio piece.

Skills used

- Conditional aggregation with `MIN(CASE ...)`: *Aggregation, GROUP BY, HAVING and FILTER* (Chapter 3).
- Joins to attach channel to users: *Joins* (Chapter 4).
- CTEs to build the answer in readable layers: *Subqueries and CTEs* (Chapter 5).
- Dates and `printf`: *CASE, Strings, Dates and Times, JSON and Arrays* (Chapter 6).
- The funnel and cohort patterns themselves: *Analytics SQL Patterns* (Chapter 12).

Data

Three tables: `users` (who and which channel), `events` (view, cart, checkout, pay with a timestamp), `orders` (what was paid). The generator plants a known pattern: social visitors convert badly, referral and email users buy again more. Save as `data/generate_data.py`.

```
import sqlite3, random, sys
from datetime import date, datetime, timedelta
random.seed(42)
con = sqlite3.connect(sys.argv[1])
con.executescript("""
DROP TABLE IF EXISTS users; DROP TABLE IF EXISTS events; DROP TABLE IF EXISTS orders;
CREATE TABLE users  (user_id INTEGER PRIMARY KEY, signup_date TEXT NOT NULL, channel TEXT NOT NULL);
CREATE TABLE events (event_id INTEGER PRIMARY KEY, user_id INTEGER NOT NULL, event TEXT NOT NULL, ts TEXT NOT NULL);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, user_id INTEGER NOT NULL, order_date TEXT NOT NULL, amount REAL NOT
NULL);
""")
start = date(2024, 1, 1); end = date(2024, 6, 30)
channels = [("search",.4,1.0),("social",.3,.7),("email",.2,1.3),("referral",.1,1.5)]  # name, share, quality
users=[]; events=[]; orders=[]
for uid in range(1, 3001):
    d = start + timedelta(days=random.randint(0, 150))
    r = random.random(); acc=0
    for ch,share,q in channels:
        acc+=share
        if r<=acc: break
    users.append((uid, d.isoformat(), ch))
    t = datetime(d.year,d.month,d.day,random.randint(8,22),random.randint(0,59))
    events.append((None,uid,'view',t.isoformat(' ')))
    if random.random() < .60*min(q,1.2):
        t += timedelta(minutes=random.randint(1,30)); events.append((None,uid,'cart',t.isoformat(' ')))
        if random.random() < .55:
            t += timedelta(minutes=random.randint(1,15)); events.append((None,uid,'checkout',t.isoformat(' ')))
            if random.random() < .70*min(q,1.1):
                t += timedelta(minutes=random.randint(1,10)); events.append((None,uid,'pay',t.isoformat(' ')))
                orders.append((None,uid,t.date().isoformat(),round(random.lognormvariate(3.9,.5),2)))
                p = .35*q                     # chance of buying again, shrinks each month
                for m in range(1,6):
                    if random.random() < p:
                        od = t.date()+timedelta(days=30*m+random.randint(-5,5))
                        if od<=end: orders.append((None,uid,od.isoformat(),round(random.lognormvariate(3.9,.5),2)))
                    p *= .75
con.executemany("INSERT INTO users VALUES (?,?,?)", users)
con.executemany("INSERT INTO events VALUES (?,?,?,?)", events)
con.executemany("INSERT INTO orders VALUES (?,?,?,?)", orders)
con.commit()
for t in ("users","events","orders"):
    print(t, con.execute(f"select count(*) from {t}").fetchone()[0])
```

```
users 3000
events 6345
orders 1209
```

Architecture

```
generate_data.py --> shop.db --> 01_checks.sql --> 02_funnel.sql --+
                                                   03_cohort.sql ---+--> findings.md
                                tests/test_results.py (Python re-computes cohort numbers)
```

Step-by-step build

1. **Generate and sanity-check.** Run `python3 data/generate_data.py shop.db`. Then check that the two tables tell the same story: every `pay` event should have an order.

```
SELECT (SELECT COUNT(*) FROM events WHERE event='pay') AS pay_events,
       (SELECT COUNT(DISTINCT user_id) FROM orders)    AS buyers;
```

```
pay_events  buyers
----------  ------
648         648
```

1. **Strict funnel by channel.** A user counts at a step only if the earlier steps happened earlier (Chapter 12.1). Predict first: which channel do you think is worst? Then run.

```
WITH t AS (
 SELECT user_id,
  MIN(CASE WHEN event='view'     THEN ts END) t_view,
  MIN(CASE WHEN event='cart'     THEN ts END) t_cart,
  MIN(CASE WHEN event='checkout' THEN ts END) t_check,
  MIN(CASE WHEN event='pay'      THEN ts END) t_pay
 FROM events GROUP BY user_id),
s AS (
 SELECT u.channel, 1 AS s1,
  CASE WHEN t_cart  > t_view THEN 1 ELSE 0 END AS s2,
  CASE WHEN t_check > t_cart AND t_cart > t_view THEN 1 ELSE 0 END AS s3,
  CASE WHEN t_pay > t_check AND t_check > t_cart AND t_cart > t_view THEN 1 ELSE 0 END AS s4
 FROM t JOIN users u USING(user_id))
SELECT channel, SUM(s1) viewers, SUM(s2) carted, SUM(s3) checkout, SUM(s4) paid,
       ROUND(100.0*SUM(s2)/SUM(s1),1) view_to_cart,
       ROUND(100.0*SUM(s3)/SUM(s2),1) cart_to_check,
       ROUND(100.0*SUM(s4)/SUM(s3),1) check_to_pay,
       ROUND(100.0*SUM(s4)/SUM(s1),1) overall
FROM s GROUP BY channel;
```

```
channel   viewers  carted  checkout  paid  view_to_cart  cart_to_check  check_to_pay  overall
--------  -------  ------  --------  ----  ------------  -------------  ------------  -------
email     606      427     246       187   70.5          57.6           76.0          30.9
referral  287      209     122       93    72.8          58.4           76.2          32.4
search    1201     724     389       272   60.3          53.7           69.9          22.6
social    906      379     201       96    41.8          53.0           47.8          10.6
```

Adding an `ALL` row with the same sums gives 3000, 1739, 958, 648 and an overall 21.6%. Social loses people at two places: view to cart (41.8%, against 60 to 73 elsewhere) and checkout to pay (47.8%, against 70 to 76). That matches the planted pattern, so the query works.

1. **Cohort retention.** Cohort = the month of a customer's first order. Retention at month k = share of that cohort who ordered again k months later. We turn each date into a month number `year*12 + month` so that "k months later" is simple subtraction.

```
WITH mi AS (
  SELECT user_id, order_date,
         CAST(substr(order_date,1,4) AS INTEGER)*12 + CAST(substr(order_date,6,2) AS INTEGER) AS mon
  FROM orders),
first_o AS (SELECT user_id, MIN(mon) AS first_mon FROM mi GROUP BY user_id),
last_data AS (SELECT MAX(mon) AS last_mon FROM mi),
act AS (SELECT DISTINCT m.user_id, f.first_mon, m.mon - f.first_mon AS k
        FROM mi m JOIN first_o f USING(user_id)),
sizes AS (SELECT first_mon, COUNT(*) size FROM first_o GROUP BY first_mon)
SELECT printf('%d-%02d', (s.first_mon-1)/12, (s.first_mon-1)%12+1) AS cohort,
       s.size,
       ROUND(100.0*SUM(a.k=1)/s.size,1) AS m1,
       CASE WHEN s.first_mon+2 <= (SELECT last_mon FROM last_data)
            THEN ROUND(100.0*SUM(a.k=2)/s.size,1) END AS m2,
       CASE WHEN s.first_mon+3 <= (SELECT last_mon FROM last_data)
            THEN ROUND(100.0*SUM(a.k=3)/s.size,1) END AS m3
FROM sizes s JOIN act a USING(first_mon)
GROUP BY s.first_mon ORDER BY s.first_mon;
```

```
cohort   size  m1    m2    m3
-------  ----  ----  ----  ----
2024-01  139   34.5  30.9  18.0
2024-02  124   37.1  25.8  23.4
2024-03  133   33.8  33.8  25.6
2024-04  131   33.6  32.1
2024-05  121   40.5
```

Read row one: 139 customers first bought in January; 34.5% (48 people) bought again in February. Blank cells are not zero. They are months the data has not reached yet (the data ends in June). Without the two `CASE` lines the same cells show a false `0.0`.

1. **Test with an independent calculation.** Save as `tests/test_results.py`. It recomputes retention in plain Python and asserts the SQL agrees.

```
import sqlite3, collections
con = sqlite3.connect("shop.db")
months = collections.defaultdict(set)
for u, d in con.execute("SELECT user_id, order_date FROM orders"):
    months[u].add(int(d[:4]) * 12 + int(d[5:7]))
cohort = collections.defaultdict(list)
for u, s in months.items():
    cohort[min(s)].append(u)
py = {c: (len(us), round(100 * sum(c + 1 in months[u] for u in us) / len(us), 1)) for c, us in cohort.items()}
rows = con.execute(open("sql/03_cohort.sql").read()).fetchall()
for label, size, m1, *_ in rows:
    y, m = map(int, label.split("-"))
    assert py[y * 12 + m] == (size, m1), label
print("all checks passed:", len(rows), "cohorts match")
```

```
all checks passed: 5 cohorts match
```

1. **Write the findings** (see README outline). Three sentences: social converts at 10.6% against 22.6% for search; about one in three first-time buyers returns the next month, and by month three 18 to 26% still buy; the fix to test first is the social checkout page.

Evaluation and acceptance criteria

- Funnel step counts never increase from one step to the next; `ALL` equals the sum of channels.
- Distinct buyers in `orders` equals paid users in the funnel (648 = 648 here).
- Cohort sizes add up to the number of buyers (139+124+133+131+121 = 648).
- The Python test passes; unseen cohort months show blank, not zero.
- Report names the worst channel with its number and one limit.

Common pitfalls

1. **Counting events, not users.** Symptom: step 1 is bigger than the user count. Detect: compare `COUNT(*)` with `COUNT(DISTINCT user_id)`.
2. **Ignoring step order.** Symptom: a user who paid without carting appears in the final step. Detect: the `t_cart > t_view` conditions; list users who break order.
3. **False zeros at the edge of the data.** Symptom: the newest cohorts show 0% retention. Detect: compare cohort month plus k with the last data month.
4. **Date maths with text.** Symptom: `'2024-03' - '2024-01'` gives 0. Use the `year*12+month` number.
5. **Averaging percentages across cohorts** instead of dividing summed counts. Detect: recompute from raw counts.

Three extensions

1. Weekly cohorts and a retention heatmap with matplotlib (not run here).
2. Time-to-convert: median minutes from `view` to `pay` per channel, using window functions (Chapter 8).
3. Add revenue per cohort (customer lifetime value so far) and compare channels.

Interview talking points

1. *Why count users, not events, in a funnel?* A person who views five times is still one person. Counting events makes step 1 too big and every conversion rate too small.
2. *What is right-censoring in a cohort table?* Young cohorts have not had time to reach month 3. A blank means "not observed yet", and a zero means "observed and nobody came". Mixing them gives a false story that new customers are worse.
3. *How did you know your SQL was right?* I recomputed retention in Python with plain sets and asserted that every cohort matches, and I reconciled paid events to the orders table.

README outline

Title with result; question; data (generator, 3 tables, counts); how to run (3 commands); funnel table; cohort table; 3 findings; checks (reconciliation, Python test); limits (synthetic data, no seasonality, one-month granularity); next steps.

15.3 Project: RFM Customer Segmentation in SQL with a Written Report

Goal and why employers care

Question: **who are the shop's best customers, who is slipping away, and what should marketing do for each group?** RFM (Recency, Frequency, Monetary value) is the oldest and most used customer segmentation: simple, explainable to a manager, and needs only an orders table. Employers like it because the real skill is the last step: turning scores into actions in a short report.

Skills used

- `GROUP BY` summaries and `julianday` date maths: *Aggregation, GROUP BY, HAVING and FILTER*; *CASE, Strings, Dates and Times, JSON and Arrays*.
- `NTILE` scoring: *Window Functions in Depth* (Chapter 8).
- `CASE` rules to name segments: Chapter 6.
- Percent-of-total with scalar subqueries: *Subqueries and CTEs*.
- RFM overview: *Analytics SQL Patterns*, section on RFM.

Data

Reuse `shop.db` from project 15.2 (the `orders` table: 1,209 orders from 648 customers, 2 January to 30 June 2024). Only customers who ordered are scored. The **snapshot date** is 2024-07-01: recency is measured from this day, not from today, so the answer never changes.

Architecture

```
orders --> rfm_base (R, F, M per customer) --> rfm (scores 1–4) --> seg (named segments)
                                                                     |
                                                                     +--> summary table --> report/findings.md
```

Step-by-step build

1. **Compute raw R, F, M** and look at the ranges first. Never score blindly.

```
CREATE TEMP VIEW rfm_base AS
SELECT user_id,
       CAST(julianday('2024-07-01') - julianday(MAX(order_date)) AS INTEGER) AS recency_days,
       COUNT(*) AS frequency,
       ROUND(SUM(amount),2) AS monetary
FROM orders GROUP BY user_id;

SELECT COUNT(*) customers, MIN(recency_days) min_r, MAX(recency_days) max_r,
       MIN(frequency) min_f, MAX(frequency) max_f, MIN(monetary) min_m, MAX(monetary) max_m FROM rfm_base;
```

```
customers  min_r  max_r  min_f  max_f  min_m  max_m
---------  -----  -----  -----  -----  -----  ------
648        1      181    1      5      16.08  345.52
```

Frequency only runs from 1 to 5. Quartiles on frequency would be mostly ties, so we use fixed business bins for F (1, 2, 3, 4 or more orders) and `NTILE(4)` for R and M.

1. **Score 1 to 4, where 4 is best.** For recency, a *smaller* number of days is better, so we sort descending: the oldest customers get score 1.

```
CREATE TEMP TABLE rfm AS
WITH sc AS (
 SELECT *,
  NTILE(4) OVER (ORDER BY recency_days DESC) AS r_score,
  NTILE(4) OVER (ORDER BY monetary)          AS m_score
 FROM rfm_base)
SELECT user_id, recency_days, frequency, monetary, r_score,
  CASE WHEN frequency=1 THEN 1 WHEN frequency=2 THEN 2 WHEN frequency=3 THEN 3 ELSE 4 END AS f_score,
  m_score
FROM sc;

SELECT r_score, COUNT(*) n, MIN(recency_days) mn, MAX(recency_days) mx FROM rfm GROUP BY 1 ORDER BY 1;
SELECT f_score, COUNT(*) n FROM rfm GROUP BY 1 ORDER BY 1;
```

```
r_score  n    mn   mx
-------  ---  ---  ---
1        162  100  181
2        162  58   98
3        162  31   58
4        162  1    31

f_score  n
-------  ---
1        238
2        285
3        103
4        22
```

Check: 4 groups of 162 = 648. Notice that 58 days appears in both score 2 and score 3. `NTILE` splits equal values arbitrarily at a boundary. This is a real property you must write in the report.

1. **Name the segments** with rules a manager can read. Order of the `WHEN` lines matters: the first match wins.

```
CREATE TEMP TABLE seg AS
SELECT *, CASE
  WHEN r_score>=3 AND f_score>=3 THEN 'Champions'
  WHEN f_score>=3 THEN 'Loyal but fading'
  WHEN r_score>=3 AND f_score=2 THEN 'Potential loyalists'
  WHEN r_score>=3 AND m_score>=3 THEN 'Big new spenders'
  WHEN r_score>=3 THEN 'New / small'
  WHEN m_score>=3 THEN 'At risk (big spenders)'
  ELSE 'Hibernating' END AS segment
FROM rfm;

SELECT segment, COUNT(*) customers, ROUND(100.0*COUNT(*)/(SELECT COUNT(*) FROM seg),1) pct_cust,
  ROUND(SUM(monetary)) revenue, ROUND(100.0*SUM(monetary)/(SELECT SUM(monetary) FROM seg),1) pct_rev,
  ROUND(AVG(recency_days)) avg_r, ROUND(AVG(frequency),1) avg_f
FROM seg GROUP BY segment ORDER BY revenue DESC;
```

```
segment                customers  pct_cust  revenue  pct_rev  avg_r  avg_f
---------------------  ---------  --------  -------  -------  -----  -----
Potential loyalists    169        26.1      18073.0  27.6     27.0   2.0
Champions              92         14.2      15919.0  24.3     25.0   3.3
At risk (big spenders) 94         14.5      12472.0  19.0     102.0  1.9
Hibernating            197        30.4      10239.0  15.6     111.0  1.2
Loyal but fading       33         5.1       5181.0   7.9      81.0   3.1
New / small            54         8.3       2658.0   4.1      45.0   1.0
Big new spenders       9          1.4       1032.0   1.6      43.0   1.0
```

1. **Reconcile.** Segments must cover every customer and every rupee: `SELECT SUM(monetary) FROM seg` returns 65574.04 and `SELECT ROUND(SUM(amount),2) FROM orders` returns 65574.04. Customers add up too: 169+92+94+197+33+54+9 = 648.

1. **Write the report** (below). Keep it to one page: headline, table, three actions, limits.

```
# Findings: customer segments (snapshot 1 July 2024, 648 customers)
Headline: 19.3% of customers (Champions plus Loyal but fading) bring 32.2% of revenue;
the 94 "At risk" big spenders hold 19.0% of revenue and have been silent ~100 days.
Actions: (1) Champions: early access, no discounts. (2) At risk: personal win-back
offer within 2 weeks. (3) Potential loyalists (2 orders, recent): nudge a third order.
Limits: synthetic data; scores are relative to this population; ties at quartile edges.
```

Check the two headline numbers: (92+33)/648 = 19.3%; (15919+5181)/65574 = 32.2%. Both match the table.

Evaluation and acceptance criteria

- Every customer has exactly one segment (`COUNT(DISTINCT user_id) = COUNT(*)` = 648).
- Segment revenue sums to total revenue to the paisa.
- Score groups are near equal size (162 each) for R and M.
- Segment names are mutually exclusive by construction (first-match `CASE`).
- The report has at least three actions, each tied to a number in the table.

Common pitfalls

1. **Recency direction reversed.** Symptom: "Champions" have the longest silence. Detect: check `avg_r` per segment; Champions must have a small number.
2. **Using today's date.** Symptom: results change every day. Fix: a fixed snapshot date in the README.
3. **Ties at quartile edges.** Symptom: reversing `ORDER BY` moves a few customers between segments (when I first wrote this with ascending order, Champions were 90, not 92). Detect: print MIN and MAX per score; accept small movement and say so.
4. `CASE` **order mistakes.** A broad rule above a narrow one hides the narrow segment. Detect: count per segment; a segment with 0 is suspicious.
5. **Including cancelled or refunded orders** in M. Decide the rule and write it down.

Three extensions

1. Compare 3-bin and 5-bin scoring and measure how many customers change segment.
2. Join with the funnel's channel to see which channel produces Champions.
3. Add a 90-day "customer lifetime value so far" and rank segments by it.

Interview talking points

1. *What is RFM?* Three numbers per customer: how recently they bought, how often, how much in total. Score each from 1 to 4, then name groups such as Champions.
2. *Why a fixed snapshot date?* So the report can be rebuilt and checked later with the same answer. With today's date every number moves daily.
3. *When does RFM fail?* When purchases are rare (cars, mattresses), when the data is short, or when F is mostly 1. Then use fewer bins or another method, such as the clustering in the machine-learning study documents.

README outline

Title; question; snapshot date and order rules; how to run; score definitions with ranges; segment rules table; summary table; one-page report; checks; limits; extensions.

15.4 Project: Design a Sales Database from Scratch, with 20 Business Queries

Goal and why employers care

Question: **can you design tables that protect the data, fill them, and answer the business?** Most tutorials start with a ready database. Employers want to see you build one: keys, constraints, seed data, indexes, and then queries that reconcile. This shows the whole chain from design to insight.

Skills used

- Primary keys, foreign keys, `CHECK`, `UNIQUE`, `NOT NULL`: *DDL, DML, Constraints, Upserts and Transactions* (Chapter 9) and *Databases and the Relational Model* (Chapter 1).
- Normalization and many-to-many tables: *Schema Design, Normalization and Data Warehouse Modeling* (Chapter 11).
- Joins, self join, anti-join: *Joins*. Aggregates: Chapter 3. Windows: *Window Functions in Depth*.
- Indexes on foreign keys: *Indexes, EXPLAIN and Query Optimization*.

Data

You write the schema, and a Python script seeds it with a fixed seed: 4 regions, 9 employees (one manager, eight reps), 60 customers, 4 categories, 24 products, 240 orders, 606 order lines.

Architecture

```
regions <--- employees (manager_id points back to employees)
   ^            ^
   |            |
customers <--- orders ---> order_items ---> products ---> categories
```

One order has many items; one product appears in many orders. `order_items` is the **bridge table** for that many-to-many link, and its primary key is the pair `(order_id, product_id)`.

Step-by-step build

1. **Write the schema** (`sql/00_schema.sql`). Each constraint is a business rule the database enforces for you.

```
PRAGMA foreign_keys = ON;
CREATE TABLE regions (region_id INTEGER PRIMARY KEY, region_name TEXT NOT NULL UNIQUE);
CREATE TABLE employees (
  employee_id INTEGER PRIMARY KEY, full_name TEXT NOT NULL,
  region_id   INTEGER NOT NULL REFERENCES regions(region_id),
  manager_id  INTEGER REFERENCES employees(employee_id),
  hired_on    TEXT NOT NULL);
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY, full_name TEXT NOT NULL,
  email  TEXT NOT NULL UNIQUE CHECK (email LIKE '%_@_%'),
  region_id INTEGER NOT NULL REFERENCES regions(region_id), joined_on TEXT NOT NULL);
CREATE TABLE categories (category_id INTEGER PRIMARY KEY, category_name TEXT NOT NULL UNIQUE);
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY, sku TEXT NOT NULL UNIQUE, name TEXT NOT NULL,
  category_id INTEGER NOT NULL REFERENCES categories(category_id),
  unit_price REAL NOT NULL CHECK (unit_price > 0),
  unit_cost  REAL NOT NULL CHECK (unit_cost >= 0),
  CHECK (unit_cost <= unit_price));
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
  employee_id INTEGER NOT NULL REFERENCES employees(employee_id),
  order_date TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('paid','shipped','cancelled','refunded')));
CREATE TABLE order_items (
  order_id   INTEGER NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
  product_id INTEGER NOT NULL REFERENCES products(product_id),
  quantity   INTEGER NOT NULL CHECK (quantity > 0),
  unit_price REAL NOT NULL CHECK (unit_price > 0),
  PRIMARY KEY (order_id, product_id));
CREATE INDEX ix_orders_customer ON orders(customer_id);
CREATE INDEX ix_orders_date     ON orders(order_date);
CREATE INDEX ix_items_product   ON order_items(product_id);
```

Note that `order_items.unit_price` copies the price at the time of sale. If the catalogue price changes next year, old orders must not change.

1. **Seed it.** Save this as `sql/seed.py` and run `python3 sql/seed.py sales.db`. It turns foreign keys on, runs the schema, and inserts rows with `random.seed(11)`, then prints the counts:

```
import sqlite3, random, sys
from datetime import date, timedelta
random.seed(11)
con = sqlite3.connect(sys.argv[1]); con.execute("PRAGMA foreign_keys=ON")
con.executescript(open("sql/00_schema.sql").read())
regions=["North","South","East","West"]
con.executemany("INSERT INTO regions VALUES (?,?)",list(enumerate(regions,1)))
emps=[(1,"Asha Rao",1,None,"2019-04-01")]
names=["Ravi","Meera","Kiran","Sana","Dev","Isha","Omar","Tara"]
for i in range(2,10): emps.append((i,f"{names[i-2]} Nair",(i%4)+1,1,f"202{random.randint(0,3)}-0{random.randint(1,9)}-15"))
con.executemany("INSERT INTO employees VALUES (?,?,?,?,?)",emps)
con.executemany("INSERT INTO categories VALUES (?,?)",[(1,"Stationery"),(2,"Electronics"),(3,"Home"),(4,"Toys")])
prods=[]
for i in range(1,25):
    cat=(i-1)%4+1; price=round(random.uniform(40,900),2); cost=round(price*random.uniform(.5,.85),2)
    prods.append((i,f"SKU-{i:03d}",f"Product {i}",cat,price,cost))
con.executemany("INSERT INTO products VALUES (?,?,?,?,?,?)",prods)
con.executemany("INSERT INTO customers VALUES (?,?,?,?,?)",
  [(i,f"Customer {i}",f"c{i}@example.com",random.randint(1,4),
(date(2023,1,1)+timedelta(days=random.randint(0,300))).isoformat()) for i in range(1,61)])
oid=0
for _ in range(240):
    oid+=1
    d=date(2024,1,1)+timedelta(days=random.randint(0,364))
    st=random.choices(["paid","shipped","cancelled","refunded"],[50,35,10,5])[0]
    con.execute("INSERT INTO orders VALUES (?,?,?,?,?)",(oid,random.randint(1,55),random.randint(2,9),d.isoformat(),st))
    for p in random.sample(range(1,25),random.randint(1,4)):
        con.execute("INSERT INTO order_items VALUES (?,?,?,?)",(oid,p,random.randint(1,5),prods[p-1][4]))
con.commit()
for t in ["regions","employees","customers","categories","products","orders","order_items"]:
    print(f"{t:12}", con.execute(f"select count(*) from {t}").fetchone()[0])
```

```
regions      4
employees    9
customers    60
categories   4
products     24
orders       240
order_items  606
```

1. **Prove the constraints work.** Try five bad inserts. A good schema rejects all of them. Predict the message for each.

```
INSERT INTO products VALUES (99,'SKU-099','Bad',1,100,150);
  -> CHECK constraint failed: unit_cost <= unit_price
INSERT INTO orders VALUES (999,1,2,'2024-05-01','lost');
  -> CHECK constraint failed: status IN ('paid','shipped','cancelled','refunded')
INSERT INTO orders VALUES (998,9999,2,'2024-05-01','paid');
  -> FOREIGN KEY constraint failed
INSERT INTO order_items VALUES (1,1,0,10);
  -> CHECK constraint failed: quantity > 0
INSERT INTO customers VALUES (500,'X','not-an-email',1,'2024-01-01');
  -> CHECK constraint failed: email LIKE '%_@_%'
```

SQLite does not enforce foreign keys unless you run `PRAGMA foreign_keys = ON` in every connection. This is a famous trap; without it the third insert would succeed silently.

1. **Answer 20 business queries** (`sql/02_queries.sql`). "Valid" orders are `paid` or `shipped`. Revenue is `SUM(quantity * unit_price)` from `order_items`. Here is the list, then the output of a few. All 20 were run.

#	Question	Technique
1	Orders per status	`GROUP BY`
2	Revenue per order, top 3	join, `SUM`
3	Total revenue and average order value	subquery
4	Monthly revenue	`substr` date
5	Revenue by category	4-table join
6	Top 5 products by units	`ORDER BY ... LIMIT`
7	Gross profit and margin by category	join to cost
8	Revenue by customer region	join regions
9	Customers with no orders	`LEFT JOIN ... IS NULL`
10	Top 5 customers	aggregate
11	Repeat customers	`HAVING COUNT(*)>=2`
12	Rep ranking inside region	`RANK() OVER`
13	Employee and manager	self join
14	Running total by month	window `SUM`
15	Month-over-month growth	`LAG`
16	Products never sold	`NOT EXISTS`
17	Cancel and refund rate by region	boolean sum
18	Products bought together	self join on items
19	Days from first to second order	`ROW_NUMBER`
20	Best product per category	window rank

```
-- Q3 Total revenue and average order value
SELECT ROUND(SUM(rev),2) AS total_revenue, ROUND(AVG(rev),2) AS avg_order_value, COUNT(*) AS orders
FROM (SELECT o.order_id, SUM(i.quantity*i.unit_price) AS rev
      FROM orders o JOIN order_items i USING(order_id)
      WHERE o.status IN ('paid','shipped') GROUP BY o.order_id);
```

```
total_revenue  avg_order_value  orders
-------------  ---------------  ------
675364.2       3326.92          203
```

```
-- Q7 Gross profit and margin by category
SELECT c.category_name, ROUND(SUM(i.quantity*(i.unit_price-p.unit_cost)),2) AS profit,
       ROUND(100.0*SUM(i.quantity*(i.unit_price-p.unit_cost))/SUM(i.quantity*i.unit_price),1) AS margin_pct
FROM order_items i JOIN orders o USING(order_id) JOIN products p USING(product_id)
     JOIN categories c USING(category_id)
WHERE o.status IN ('paid','shipped')
GROUP BY c.category_name ORDER BY profit DESC;
```

```
category_name  profit    margin_pct
-------------  --------  ----------
Stationery     82981.91  38.6
Toys           66945.8   35.3
Home           36971.86  22.5
Electronics    36214.01  34.1
```

```
-- Q17 Cancellation and refund rate by region (a boolean is 0 or 1 in SQLite, so SUM counts the true ones)
SELECT r.region_name, ROUND(100.0*SUM(o.status IN ('cancelled','refunded'))/COUNT(*),1) AS bad_pct, COUNT(*) AS orders
FROM orders o JOIN customers cu USING(customer_id) JOIN regions r ON r.region_id=cu.region_id
GROUP BY r.region_name ORDER BY bad_pct DESC;
```

```
region_name  bad_pct  orders
-----------  -------  ------
South        27.8     72
East         13.5     52
West         10.0     50
North        7.6      66
```

Other results: Q1 gives paid 116, shipped 87, cancelled 27, refunded 10 (total 240). Q9 gives 5 customers with no orders. Q11 gives 53 repeat customers. Q19 gives an average of 84.2 days between first and second order, over 53 customers. Q13 shows `Asha Rao` with a `NULL` manager (the top of the tree).

1. **Reconcile.** The categories in Q5 add to 214703.25+189905.23+164625.54+106130.18 = 675364.20, equal to Q3. The regions in Q8 add to 180927.38+178095.69+171097.51+145243.62 = 675364.20. Two independent splits of the same total is the best evidence that your joins did not duplicate rows.

Evaluation and acceptance criteria

- All 5 bad inserts fail with a constraint error; `PRAGMA foreign_key_check;` returns no rows.
- Row counts equal the seed script's counts.
- Revenue by category equals revenue by region equals total revenue.
- Every table has a primary key; every relationship has a foreign key and an index on the child column.
- All 20 queries run and each has a comment with the business question.

Common pitfalls

1. **Foreign keys silently off** (SQLite). Detect: try the bad insert; if it succeeds, the pragma is missing.
2. **Join fan-out.** Joining `orders` to `order_items` before summing an order-level amount counts the amount once per item. Detect: reconcile to a known total.
3. **Storing computed totals** in `orders`, which later disagree with the lines. Compute them in queries or views.
4. **Using the current price** instead of the price at sale time. Detect: change a product price, rerun Q3; the total must not change.
5. **Ties under `LIMIT`.** Q6 uses `ORDER BY units DESC, p.product_id` so the answer is stable.

Three extensions

1. Add a `payments` table (one order, many partial payments) and a view `v_order_revenue`.
2. Write a migration script that adds a `discount` column and backfills it inside a transaction (Chapter 9).
3. Add an `audit_log` table filled by a trigger when an order status changes.

Interview talking points

1. *Why a bridge table?* One order holds many products and one product appears in many orders. Putting both ids in one table with a combined key removes repeated data and keeps each fact once.

2. *Why constraints if the app already validates?* Apps have bugs and other apps write to the same database. The database is the last guard, and it never forgets.
3. *How do you know your 20 queries are right?* Two different splits of revenue reconcile to the same total, row counts match the seed script, and I tested one query by hand on three orders.

README outline

Title; the ER diagram (text tree above); table list with one line per rule; how to run (schema, seed, queries); constraint test results; the 20 questions table; three sample results; reconciliation proof; limits (synthetic, SQLite, no time zones); extensions.

15.5 Worked Extension: Compare Channels, Catch a Fan-Out and Test the Gap

INTUITION	A project is not finished when the queries run. It is finished when someone can trust the numbers and act on them. This walkthrough takes the funnel database from Section 15.2 (3,000 users, 6,345 events, 1,209 orders, built by its generator with seed 42) and does the three things a reviewer will ask: compare segments, check that totals reconcile, and say whether a gap is real. The SQL output below was produced by running the generator and these queries on SQLite 3.50.4.

Step 1: conversion by channel

Problem 15.5.1 What share of each channel's users ever bought? Predict first: the planted pattern says social is worst, referral and email best.

```
SELECT u.channel,
       count(DISTINCT u.user_id) AS users,
       count(DISTINCT o.user_id) AS buyers,
       round(100.0 * count(DISTINCT o.user_id) / count(DISTINCT u.user_id), 1) AS conv_pct
FROM users u LEFT JOIN orders o ON o.user_id = u.user_id
GROUP BY u.channel ORDER BY conv_pct;
```

```
channel   users  buyers  conv_pct
--------  -----  ------  --------
social    906    96      10.6
search    1201   272     22.6
email     606    187     30.9
referral  287    93      32.4
```

Step 1	A `LEFT JOIN` keeps users who never bought, so the denominator is all users in the channel. `count(DISTINCT o.user_id)` counts each buyer once however many orders they placed.
Step 2	Social converts 10.6 percent (96 of 906). Search is 22.6, email 30.9, referral 32.4.
Check	Buyers add up: 96 + 272 + 187 + 93 = 648, which equals the 648 users who reached the `pay` step in the funnel. Two independent routes agree.
Meaning	Social brings the most users after search but the fewest buyers per user. That is the first finding for the report.

Step 2: reconcile revenue, and see a fan-out happen

Now revenue by channel. First the total it must add up to:

```
SELECT round(sum(amount), 2) AS revenue, count(*) AS orders FROM orders;
```

```
revenue   orders
--------  ------
65574.04  1209
```

Problem 15.5.2 Compute revenue per channel correctly, then show what a careless extra join does to it. Predict first: the correct pieces add to 65,574.04, and the careless version inflates them several times over, because each user has several events.

```
SELECT u.channel, round(sum(o.amount), 1) AS revenue
FROM orders o JOIN users u ON u.user_id = o.user_id
GROUP BY u.channel ORDER BY u.channel;
```

```
channel   revenue
--------  -------
email     20798.5
referral  10904.0
search    26060.4
social    7811.1
```

Those four numbers add to 65,574.0, the total. Now someone adds a join to `events` 'to also show the number of events':

```
SELECT u.channel, round(sum(o.amount), 1) AS revenue
FROM orders o
JOIN users u  ON u.user_id = o.user_id
JOIN events e ON e.user_id = u.user_id          -- several events per user: duplicates each order
GROUP BY u.channel ORDER BY u.channel;
```

```
channel   revenue
--------  --------
email     83194.1
referral  43615.9
search    104241.8
social    31244.4
```

```
SELECT count(*) AS joined_rows, (SELECT count(*) FROM orders) AS orders FROM orders o JOIN events e ON e.user_id =
o.user_id;
```

```
joined_rows  orders
-----------  ------
4836         1209
```

Step 1	Each order row is matched with every event row of the same user. A user with 4 events makes each of their orders appear 4 times.
Step 2	The row count tells the story: 4,836 joined rows against 1,209 orders, a factor of 4.0 on average, so every revenue figure is about 4 times too large (email 83,194.1 against the true 20,798.5).
Step 3	No error is raised. The shares by channel look plausible, which is why this bug survives code review.

COMMON MISTAKE

Joining two tables that both have many rows per key (orders and events per user) and then summing. The rule: **aggregate each side to one row per key first, then join**, or do not join at all if you do not need the column. Detect it every time with the reconciliation habit: compare `count(*)` after the join with the count before, and compare the grand total with a query that has no joins.

```
SELECT u.channel, round(sum(o.amount), 1) AS revenue, count(*) AS orders
FROM orders o JOIN users u ON u.user_id = o.user_id
GROUP BY u.channel
UNION ALL
SELECT 'TOTAL', round(sum(amount), 1), count(*) FROM orders;
```

```
channel   revenue  orders
--------  -------  ------
email     20798.5  381
referral  10904.0  204
search    26060.4  478
social    7811.1   146
TOTAL     65574.0  1209
```

The final `TOTAL` row is the reconciliation built into the output. A reviewer who sees a total that equals the independent figure of 65,574.0 trusts the table.

Step 3: is the social gap real or noise?

Social converts 10.6 percent and search 22.6. With 906 and 1,201 users, could that gap be luck? The standard check is a two-proportion z-test (the Statistics document, Chapter 6, explains it fully). The code below computes it by hand:

Problem 15.5.3 Test social against search and give an interval for the gap. Predict first: a gap of 12 points on groups of about a thousand is far outside random wobble.

```
import math
# social: 96 buyers of 906 users.  search: 272 buyers of 1201 users.
x1, n1, x2, n2 = 96, 906, 272, 1201
p1, p2 = x1 / n1, x2 / n2
pool = (x1 + x2) / (n1 + n2)
se = math.sqrt(pool * (1 - pool) * (1 / n1 + 1 / n2))
z = (p2 - p1) / se
print("social %.1f%%  search %.1f%%  gap %.1f points" % (100 * p1, 100 * p2, 100 * (p2 - p1)))
print("z = %.1f" % z)
se_gap = math.sqrt(p1 * (1 - p1) / n1 + p2 * (1 - p2) / n2)
lo, hi = (p2 - p1) - 1.96 * se_gap, (p2 - p1) + 1.96 * se_gap
print("95%% interval for the gap: %.1f to %.1f points" % (100 * lo, 100 * hi))
```

```
social 10.6%  search 22.6%  gap 12.1 points
z = 7.2
95% interval for the gap: 9.0 to 15.2 points
```

Step 1	Pool the two groups: (96 + 272) / (906 + 1201) = 17.5 percent buyers overall. Under 'no difference' both channels share that rate.
Step 2	The standard error of the gap under that assumption combines both group sizes. The z score is the gap divided by it.
Step 3	A z of about 7.2 is enormous (anything beyond 2 or 3 is already unlikely by chance). The 95 percent interval for the gap runs from about 9.0 to 15.2 points and excludes zero.
Answer	The gap is real. Write it as: 'social converts 10.6 percent (96 of 906) versus 22.6 percent for search (272 of 1,201); the gap of 12.1 points has a 95 percent interval of 9.0 to 15.2 points.'

GOING DEEPER

Significance says the gap is not luck. It does not say why. Social users may be browsing from phones, or arriving from a different campaign, or just less intent. The data cannot separate those; only a test or a better variable can. Say so in the report, in the limits paragraph. Also note the multiple-comparison issue: if you test every pair of channels and every segment, some gaps will pass by chance. Decide the comparisons before you look.

A grading rubric for your own project

Score yourself 0 to 2 on each line. A score of 12 or more out of 16 is portfolio quality.

Line	0 points	2 points
Question	missing or vague	one sentence with a measurable answer

Line	0 points	2 points
Data	unexplained source	seeded generator or documented source, row counts shown
Correctness	totals never checked	every table reconciles to an independent total
Readability	one 80-line query	CTEs with clear names, one idea each
Segment sizes	rates without counts	every rate shows its n
Uncertainty	no comment on noise	at least one gap tested or labelled too small to trust
Writing	list of tables	three findings with numbers, a recommendation, limits
Reproducibility	cannot rerun	README with run order; fresh clone gives the same numbers

Practice with answers

Problem 15.5.4 Referral has 287 users and 93 buyers. Compute its conversion and say how many users you would need in each of social and referral to compare them with confidence. *(easy)*

Answer: 93 / 287 = 32.4 percent. The group sizes here (906 and 287) are already large enough, because the gap is more than 20 points. Small gaps need more users, big gaps need fewer.

Problem 15.5.5 A revenue table by channel adds to 262,296.2 but finance says 65,574.04. First check? *(find the bug)*

Answer: Compare row counts: `count(*)` of the final joined result with `count(*)` of `orders`. A ratio of 4 points to a join with `events`. Remove the join or aggregate events to one row per user first.

Problem 15.5.6 Why is 'average revenue per user' by channel computed with a `LEFT JOIN` from users, and what changes with an inner join? *(medium)*

Answer: A `LEFT JOIN` from `users` keeps non-buyers, so the denominator is every user, and the average is revenue per *visitor*. An inner join drops non-buyers, so the denominator is buyers only and the figure is revenue per *buyer*, much higher, and not comparable across channels with different conversion.

Chapter recap

- A project starts with a one-sentence question and measurable success criteria, before any SQL.
- Generate data with a fixed seed and plant a known pattern, so the test is real.
- A funnel counts people in time order. A cohort table counts people by first-purchase month. Blank cells are unobserved, not zero.
- RFM scores each customer on recency, frequency and monetary value; the real work is naming segments and writing actions.
- A database design is finished only when bad data is rejected: five failing inserts prove it.
- Always reconcile: pieces must add to the whole, and a headline number must be recomputed a second way.
- Ties, snapshot dates and unobserved periods are the three quiet bugs in analytics SQL.

Pattern	Code idea
Strict funnel	`MIN(CASE WHEN event=... THEN ts END)` per user, then compare times
Month number	`year*12 + month`
Censor young cohorts	`CASE WHEN first_mon+k <= last_mon THEN ... END`
Score quartiles	`NTILE(4) OVER (ORDER BY x)`
Percent of total	`100.0*SUM(x)/(SELECT SUM(x) FROM t)`
Enforce rules	`CHECK`, `UNIQUE`, `NOT NULL`, `REFERENCES`

Practice set

Problem 15.P.1 (easy) Write the success criteria for the question "Which region has the most cancellations?" in two lines. **Answer:** Example: "A table of cancellation rate per region that reconciles to the total cancelled count; a note on regions with fewer than 30 orders." Any measurable, checkable sentence is correct.

Problem 15.P.2 (easy) In the funnel, 3,000 viewers, 1,739 carted, 958 checked out, 648 paid. Compute the share of carted users who check out. **Answer:** 958 / 1739 = 0.5509, so 55.1%.

Problem 15.P.3 (easy) What does month number `year*12+month` give for 2024-03, and for 2024-01? What is the difference? **Answer:** 2024*12+3 = 24291 and 2024*12+1 = 24289; the difference is 2 months.

Problem 15.P.4 (medium) Find the bug: a cohort table shows 0.0% month-3 retention for the April cohort when the data ends in June. **Answer:** April plus 3 months is July, which the data does not contain. The cell must be NULL (blank), not 0.0. Compare `first_mon + 3` with the last data month.

Problem 15.P.5 (medium) A customer has recency 20 days. In `NTILE(4) OVER (ORDER BY recency_days)` they get 1, but you want 4 for the most recent. Fix it. **Answer:** Sort descending: `NTILE(4) OVER (ORDER BY recency_days DESC)`. Then the largest recency (oldest) falls into group 1 and the smallest falls into group 4.

Problem 15.P.6 (medium) Predict: `INSERT INTO orders VALUES (998,9999,2,'2024-05-01','paid');` in SQLite without `PRAGMA foreign_keys = ON`. **Answer:** It succeeds silently, leaving an orphan order for a customer that does not exist. Foreign key checks are off by default in SQLite connections.

Problem 15.P.7 (interview) A product manager says "retention is 34% for January." What three questions do you ask before agreeing? **Answer:** Retention at which month? Which definition of active (any order, or paid order)? Is the cohort fully observed? Also ask the cohort size, since 34% of 20 people is very noisy.

Problem 15.P.8 (interview, scenario) Revenue by category adds to 675,364.20 but revenue by region adds to 702,000. What happened and how do you find it? **Answer:** One of the joins duplicated rows (fan-out), most likely a join to a table with several rows per key. Count rows after each join, compare `COUNT(*)` with `COUNT(DISTINCT order_id)`, and fix by aggregating before joining.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. I am planning a SQL portfolio project on funnel and cohort-retention analysis for an online shop. Help me write the one-sentence question, three measurable success criteria, a repository layout and a definition of done. Ask me questions one at a time about my data and tools, and do not move on until each answer is clear.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. Review my funnel SQL for these problems: counting events instead of users, ignoring step order, false zeros in unobserved cohort months, and join duplicates. For each problem explain how to detect it with a small query and how to fix it. My query and a sample of my tables: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. I built an RFM segmentation in SQL. Review my scoring rules, my segment names and my report for logic mistakes, ties at quartile edges, the direction of recency, and weak recommendations. Then tell me what a hiring manager would still ask. My SQL and my report: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. Review my database design for a sales shop: tables, keys, constraints and indexes. Tell me which business rules are missing, which table might need splitting, and give me five bad INSERT statements I should test. Explain each point in simple English. My schema: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. My SQL project gives a number I do not trust: [DESCRIBE THE NUMBER]. Teach me a debugging routine step by step: row counts after each join, reconciliation to a total, an independent Python check, and NULL and boundary checks. Ask me to run each check and wait for my result before you suggest the next one.
```

CHAPTER 16

PROJECT LAB, PART 2: AN ANALYTICS CASE STUDY AND A QUERY-OPTIMIZATION LAB

Two more portfolio projects: answer twenty questions about a clinic and write findings you can defend, then take five slow queries and make them fast, with the plans and timings to prove it.

Chapter 15 gave you the working method and three projects. This chapter adds two projects that show two different strengths. The first, a 20-question case study, shows that you can turn a pile of tables into a short report that separates real findings from noise. This is the daily work of a data analyst. The second, a query-optimization lab, shows that you understand why a query is slow, can read a query plan, and can fix it and measure the fix. We use the working method of section 15.1 (question, success criteria, layout, checks, README) without repeating it. All code was run with Python 3 and SQLite 3.50. Timings depend on the machine, so read them as ratios, not as promises.

16.1 Project: A 20-Question SQL Analytics Case Study with a Findings Report

Goal and why employers care

Question: **a clinic loses money and patient time to missed appointments and long waits. What do the data say, and what should the clinic try first?** Interviews and take-home tests for analyst roles are almost always this shape: a dataset, a list of questions, and a request for "your recommendations". Employers watch three things: correct SQL, sensible checking, and honest writing. A report that says "this difference may be noise" earns more trust than one that shouts every number.

Skills used

- `GROUP BY`, `HAVING`, boolean sums: *Aggregation, GROUP BY, HAVING and FILTER* (Chapter 3).
- Joins to doctors and patients: *Joins* (Chapter 4).
- Bucketing with `CASE` and dates with `julianday` and `strftime`: *CASE, Strings, Dates and Times, JSON and Arrays* (Chapter 6).
- `RANK`, `LAG`, `ROW_NUMBER`: *Window Functions in Depth* (Chapter 8).
- Data-quality thinking: *Analytics SQL Patterns* (Chapter 12).

Data

Hospital-clinic appointments, generated. 12 doctors in 4 departments, 1,500 patients, 6,000 appointments in 2024. We plant two patterns: the longer ahead an appointment was booked, the more likely a no-show; and Monday has longer waits. Your SQL should find them. Save as `data/generate_data.py`.

```
import sqlite3, random, sys
from datetime import date, timedelta
random.seed(5)
con = sqlite3.connect(sys.argv[1])
con.executescript("""
DROP TABLE IF EXISTS appointments; DROP TABLE IF EXISTS patients; DROP TABLE IF EXISTS doctors;
CREATE TABLE doctors (doctor_id INTEGER PRIMARY KEY, name TEXT NOT NULL, department TEXT NOT NULL);
CREATE TABLE patients (patient_id INTEGER PRIMARY KEY, age INTEGER NOT NULL, gender TEXT NOT NULL, city TEXT NOT NULL);
CREATE TABLE appointments (
  appt_id INTEGER PRIMARY KEY, patient_id INTEGER NOT NULL REFERENCES patients,
  doctor_id INTEGER NOT NULL REFERENCES doctors,
  booked_on TEXT NOT NULL, appt_date TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('attended','no_show','cancelled')),
  wait_minutes INTEGER, fee REAL NOT NULL);
""")
depts = ["General","Cardiology","Orthopedics","Pediatrics"]
con.executemany("INSERT INTO doctors VALUES (?,?,?)", [(i, f"Dr. {chr(64+i)}", depts[(i-1)%4]) for i in range(1,13)])
con.executemany("INSERT INTO patients VALUES (?,?,?,?)",
  [(i, random.randint(1,85), random.choice("MF"), random.choice(["North","South","East","West"])) for i in range(1,1501)])
fees = {"General":300,"Cardiology":800,"Orthopedics":600,"Pediatrics":400}
rows = []
for a in range(1, 6001):
    d = random.randint(1,12); dep = depts[(d-1)%4]
    lead = random.choice([0,1,2,3,5,7,10,14,21,30])
    appt = date(2024,1,1) + timedelta(days=random.randint(0,364))
    booked = appt - timedelta(days=lead)
    p_no = 0.06 + 0.012*lead + (0.05 if dep == "Pediatrics" else 0)
    r = random.random()
    if r < p_no: st = "no_show"
    elif r < p_no + 0.08: st = "cancelled"
    else: st = "attended"
    wait = max(0, int(random.gauss(22 + 3*(d%4) + (10 if appt.weekday() == 0 else 0), 9))) if st == "attended" else None
    rows.append((a, random.randint(1,1500), d, booked.isoformat(), appt.isoformat(), st, wait, fees[dep]))
con.executemany("INSERT INTO appointments VALUES (?,?,?,?,?,?,?,?)", rows); con.commit()
print(con.execute("select count(*) from appointments").fetchone())
```

```
(6000,)
```

Note that `wait_minutes` is `NULL` for visits that did not happen. Remember that when averaging.

Architecture

```
generate_data.py --> clinic.db --> sql/01_checks.sql --> sql/02_questions.sql (Q1..Q20)
                                                                  |
                                       tests/test_results.py --+--> report/findings.md
```

Step-by-step build

1. **Check the data first** (Q1 and Q2). Never start with the interesting questions.

```
SELECT (SELECT COUNT(*) FROM appointments) appts,
       (SELECT COUNT(DISTINCT patient_id) FROM appointments) patients_seen,
       (SELECT COUNT(*) FROM doctors) doctors;

SELECT status, COUNT(*) n, ROUND(100.0*COUNT(*)/(SELECT COUNT(*) FROM appointments),1) pct
FROM appointments GROUP BY status ORDER BY n DESC;
```

```
appts  patients_seen  doctors
-----  -------------  -------
6000   1479           12

status    n     pct
--------  ----  ----
attended  4428  73.8
no_show   1057  17.6
cancelled 515   8.6
```

Check: 4428 + 1057 + 515 = 6000. Of 1,500 patients, 1,479 came at least once, so 21 never booked. Not a bug, just a fact.

1. **The main question: who misses appointments?** Q5 (department) and Q7 (booking lead time in bands). Predict before running: does a long wait between booking and visit raise no-shows?

```
-- Q5 No-show rate by department
SELECT d.department, ROUND(100.0*SUM(a.status='no_show')/COUNT(*),1) no_show_pct, COUNT(*) n
FROM appointments a JOIN doctors d USING(doctor_id)
GROUP BY d.department ORDER BY no_show_pct DESC;

-- Q7 No-show rate by lead-time band
SELECT CASE WHEN lead<=2 THEN '0-2 days' WHEN lead<=7 THEN '3-7 days'
            WHEN lead<=14 THEN '8-14 days' ELSE '15+ days' END band,
       ROUND(100.0*SUM(status='no_show')/COUNT(*),1) no_show_pct, COUNT(*) n
FROM (SELECT status, CAST(julianday(appt_date)-julianday(booked_on) AS INTEGER) lead FROM appointments)
GROUP BY band ORDER BY MIN(lead);
```

```
department   no_show_pct  n
-----------  -----------  ----
Pediatrics   21.7         1472
Orthopedics  16.4         1485
General      16.4         1507
Cardiology   16.0         1536

band       no_show_pct  n
---------  -----------  ----
0-2 days   7.9          1754
3-7 days   13.3         1830
8-14 days  21.0         1229
15+ days   35.0         1187
```

The lead-time effect is large and steady: 7.9% to 35.0%, a factor of 4.4, with at least 1,187 appointments in every band. Pediatrics is 5 points above the others on about 1,500 appointments, so it is also believable.

1. **Waiting time** (Q8 to Q12). Use only attended visits. The median needs a window function because SQLite has no `MEDIAN`.

```
-- Q9 Average wait by weekday (SQLite: %w gives 0 = Sunday ... 1 = Monday)
SELECT strftime('%w',appt_date) dow, ROUND(AVG(wait_minutes),1) avg_wait
FROM appointments WHERE status='attended' GROUP BY dow ORDER BY avg_wait DESC LIMIT 3;

-- Q11 Median wait
SELECT wait_minutes AS median_wait FROM (
  SELECT wait_minutes, ROW_NUMBER() OVER (ORDER BY wait_minutes) rn, COUNT(*) OVER () c
  FROM appointments WHERE status='attended') WHERE rn=(c+1)/2;

-- Q12 Share waiting over 45 minutes
SELECT ROUND(100.0*SUM(wait_minutes>45)/COUNT(*),1) pct_over_45, SUM(wait_minutes>45) n
FROM appointments WHERE status='attended';
```

```
dow  avg_wait
---  --------
1    36.5
0    26.3
6    26.1

median_wait
-----------
27

pct_over_45  n
-----------  ---
4.5          200
```

Monday (`1`) waits 36.5 minutes against 26.3 for the next highest day (Sunday), a gap of 10 minutes. The median is 27 minutes. Q8 by department gives Orthopedics 32.1, Cardiology 29.2, General 25.9, Pediatrics 22.9.

1. **Money** (Q13, Q14) and **repeat no-shows** (Q16, Q17).

```
-- Q14 Money lost to no-shows versus money earned
SELECT SUM(fee) lost_to_no_shows,
       (SELECT SUM(fee) FROM appointments WHERE status='attended') earned
FROM appointments WHERE status='no_show';

-- Q16, Q17 Patients with 2+ no-shows, and their share of all no-shows
SELECT COUNT(*) patients FROM (SELECT patient_id FROM appointments
  WHERE status='no_show' GROUP BY patient_id HAVING COUNT(*)>=2);
SELECT ROUND(100.0*SUM(c)/(SELECT COUNT(*) FROM appointments WHERE status='no_show'),1) pct_of_no_shows
FROM (SELECT COUNT(*) c FROM appointments WHERE status='no_show' GROUP BY patient_id HAVING COUNT(*)>=2);
```

```
lost_to_no_shows  earned
----------------  -------
545300.0          2340600.0

patients
--------
229

pct_of_no_shows
---------------
49.5
```

Lost fees are 545,300 against 2,340,600 earned: of all fees that were booked and not cancelled, 545300 / (545300 + 2340600) = 18.9% were lost. And 229 patients (about 15% of 1,500) cause 49.5% of no-shows, but be careful: in this data every patient has the same no-show chance, so people who happen to have 2 or more are simply the unlucky ones. Do not call them "serial offenders" without more evidence.

1. **The other questions** are variants of the same shapes. This table lists all 20 with the real answers.

#	Question	Answer from the data
1	Rows, patients, doctors	6,000; 1,479; 12
2	Status split	73.8% attended, 17.6% no-show, 8.6% cancelled
3	Three busiest months	May 540, July 536, August 520
4	Busiest weekday	Monday 911, Sunday 863, Friday 861
5	No-show by department	Pediatrics 21.7%, others 16.0 to 16.4%
6	No-show by lead days	7.2% at 0 days up to 37.6% at 30 days

#	Question	Answer from the data
7	Lead-time bands	7.9%, 13.3%, 21.0%, 35.0%
8	Wait by department	Orthopedics 32.1 minutes is the longest
9	Wait by weekday	Monday 36.5 minutes
10	Longest-wait doctors	Dr. C 32.4, Dr. K 32.1, Dr. G 31.8
11	Median wait	27 minutes
12	Waiting over 45 minutes	4.5% (200 visits)
13	Fees earned by department	Cardiology 920,000 is the largest
14	Lost to no-shows	545,300
15	No-show by age group	18.9% (0-17), 18.3%, 16.6%, 16.7% (60+)
16	Patients with 2+ no-shows	229
17	Their share of no-shows	49.5%
18	Month with biggest change in no-show rate	October, 16.2%, down 2.4 points
19	Top doctor per department (attended)	F, E, K, D (385, 401, 380, 356)
20	Same-day bookings	9.7% of all; no-show 7.2% against 18.7% for the rest

Reconcile: department fees in Q13 add up to 920,000 + 669,600 + 409,600 + 341,400 = 2,340,600, equal to the `earned` column in Q14.

1. **Write the findings report** (`report/findings.md`). One page, in this order: the three findings with numbers, the recommendations, the limits.

```
# Clinic findings, 2024 (6,000 appointments)
1. Booking far ahead drives no-shows: 7.9% when booked 0–2 days ahead, 35.0% at 15+ days.
2. Monday waits are 10 minutes longer than other days (36.5 vs about 26).
3. 545,300 in fees was lost to no-shows (18.9% of booked, non-cancelled fees).
Recommend: reminder calls for appointments booked 15+ days ahead; add one Monday slot
for the longest-wait department; test, do not assume, that reminders help.
Limits: synthetic data; no cause is proven; age differences (about 2 points on ~1,400
visits) are too small to act on; one year only.
```

Evaluation and acceptance criteria

- All 20 questions have a query, a result and a one-line answer.
- Status counts add to 6,000; department fees add to total fees.
- Each finding in the report shows its group size.
- At least one difference is explicitly labelled as too small to trust.
- The report has recommendations that each point to a finding, and a limits section.

Common pitfalls

1. **Averaging `NULL` waits as zero.** If you replaced `NULL` with 0 the average would drop. Detect: count rows with `COUNT(wait_minutes)` versus `COUNT(*)` .
2. **Comparing groups of very different size.** A 40% rate on 10 visits means little. Always print `n` next to a rate.
3. **Reading noise as a pattern.** Age groups differ by about 2 points. With about 1,400 visits per group the usual random wobble of a rate near 18% is about 1 point (`√(0.18 × 0.82 / 1400) ≈ 0.010`), so a 2-point gap is borderline. Say so.
4. **Causation language.** "Long lead causes no-shows" needs an experiment. Write "is associated with".

5. **Wrong weekday numbering.** SQLite `%w` starts at 0 for Sunday; other databases differ (Chapter 14). Detect: check one known date by hand.

Three extensions

1. Add a `reminders` table and compare no-shows with and without a reminder, with group sizes.
2. Build a rule-based risk score (lead band plus department) and test how many no-shows the top 20% of scores catch.
3. Re-run the study on a PostgreSQL copy and list every query you had to change (Chapter 14).

Interview talking points

1. *How do you decide whether a difference is real?* First look at group size. Then compare the gap with the usual random wobble of that rate. Large, steady, many-group patterns (like lead time) are credible; a small gap on one split is not.
2. *Why separate attended visits when computing waits?* Visits that did not happen have no wait. Putting them in as zero would make waits look shorter than they are.
3. *What would you do after the report?* Propose a small test, such as reminders for long-lead bookings in one department, and measure no-shows against a comparison group before rolling it out.

README outline

Title with the finding; question; data and generator; how to run; the 20-question table with answers; three findings; checks (sums reconcile); the report link; limits; extensions.

16.2 Project: A Query-Optimization Lab, from Slow to Fast

Goal and why employers care

Question: **why is this query slow, what is the smallest change that fixes it, and how do I prove the fix?** Anyone can add an index at random. Employers want someone who measures first, reads the plan, changes one thing at a time, and also counts the cost of the fix. This lab is the best way to practise the measure-first method of *Indexes, EXPLAIN and Query Optimization* (Chapter 10).

Skills used

- B-tree indexes, composite, covering: Chapter 10, sections on indexes.
- Reading plans: `EXPLAIN QUERY PLAN` (SQLite's version of `EXPLAIN`, Chapter 10).
- Sargable filters (a filter the index can use) and `EXISTS`: *Subqueries and CTEs* (Chapter 5).
- Timing method and checking that the result did not change.

Data

200,000 orders over two years and 20,000 customers. Generated with a seed so the table always holds the same rows. Save as `data/setup.py`.

```
import sqlite3, random, sys
from datetime import date, timedelta
random.seed(1)
con = sqlite3.connect(sys.argv[1])
con.executescript("""
DROP TABLE IF EXISTS orders; DROP TABLE IF EXISTS customers;
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT, city TEXT, email TEXT);
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER, order_date TEXT, status TEXT, amount REAL);
""")
cities = ["Pune","Delhi","Mumbai","Chennai","Kolkata","Jaipur","Surat","Indore"]
con.executemany("INSERT INTO customers VALUES (?,?,?,?)",
  [(i, f"Customer {i}", random.choice(cities), f"user{i}@example.com") for i in range(1, 20001)])
base = date(2023,1,1); st = ["paid"]*85 + ["refunded"]*5 + ["cancelled"]*10
rows = [(i, random.randint(1,20000), (base+timedelta(days=random.randint(0,729))).isoformat(),
         random.choice(st), round(random.uniform(5,500),2)) for i in range(1, 200001)]
con.executemany("INSERT INTO orders VALUES (?,?,?,?,?)", rows); con.commit()
print(con.execute("select count(*) from orders").fetchone(), con.execute("select count(*) from customers").fetchone())
```

```
(200000,) (20000,)
```

Architecture

```
setup.py --> lab.db --> bench.py "before" --> (apply ONE fix) --> bench.py "after"
                              |                                        |
                        plan + best-of-5 ms                      plan + best-of-5 ms
                              +----------------> results table + README
```

Step-by-step build

1. **Write the measuring helper.** Time each query five times and keep the best (the best run has the least interference from other programs). Print the plan with it. Save as `bench.py`; the five queries are named Q1 to Q5.

```
import sqlite3, time
con = sqlite3.connect("lab.db")
def timeit(sql, n=5):
    best = 1e9
    for _ in range(n):
        t = time.perf_counter(); rows = con.execute(sql).fetchall()
        best = min(best, time.perf_counter() - t)
    return best * 1000, rows
def report(label, sql, n=5):
    ms, rows = timeit(sql, n)
    print(f"[{label}] best of {n}: {ms:9.2f} ms   result={rows[0] if rows else None}")
    for r in con.execute("EXPLAIN QUERY PLAN " + sql): print("    plan:", r[3])

Q1 = "SELECT * FROM orders WHERE customer_id = 777"
Q2_BEFORE = "SELECT COUNT(*) FROM orders WHERE substr(order_date,1,7)='2024-03'"
Q2_AFTER  = "SELECT COUNT(*) FROM orders WHERE order_date>='2024-03-01' AND order_date<'2024-04-01'"
Q3_BEFORE = ("SELECT COUNT(*) FROM customers c WHERE c.customer_id<=300 AND "
             "(SELECT COUNT(*) FROM orders o WHERE o.customer_id=c.customer_id AND o.status='paid')>0")
Q3_AFTER  = ("SELECT COUNT(*) FROM customers c WHERE c.customer_id<=300 AND "
             "EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id AND o.status='paid')")
Q4 = ("SELECT order_id,order_date,amount FROM orders WHERE status='paid' "
      "ORDER BY order_date DESC, order_id DESC LIMIT 10")
Q5 = ("SELECT customer_id, SUM(amount) s FROM orders WHERE status='paid' AND order_date>='2024-01-01' "
      "GROUP BY customer_id ORDER BY s DESC LIMIT 5")
```

1. **Baseline.** With only primary keys, run the five queries. Predict: which will be slowest, and what will the plan word be? (Q3 asks for "has at least one paid order" but is written as a count inside a loop, for only the first 300 customers, because for all 20,000 it would run for minutes.)

```
[Q1 one customer] best of 5:      4.30 ms   result=(2261, 777, '2023-01-23', 'paid', 445.99)
    plan: SCAN orders
[Q2 month via substr()] best of 5:     10.81 ms   result=(8487,)
    plan: SCAN orders
[Q3 correlated COUNT, 300 customers] best of 1:   1603.07 ms   result=(300,)
    plan: SEARCH c USING INTEGER PRIMARY KEY (rowid<?)
    plan: CORRELATED SCALAR SUBQUERY 1
    plan: SCAN o
[Q4 latest 10 paid] best of 5:     12.25 ms   result=(198707, '2024-12-30', 88.14)
    plan: SCAN orders
    plan: USE TEMP B-TREE FOR ORDER BY
[Q5 top spenders 2024] best of 5:     31.15 ms   result=(19943, 4865.79)
    plan: SCAN orders
    plan: USE TEMP B-TREE FOR GROUP BY
    plan: USE TEMP B-TREE FOR ORDER BY
```

`SCAN orders` means "read every row, like reading a whole ledger to find one name". Q3 is the worst: for each of the 300 customers the inner query does a full `SCAN o` of 200,000 rows, about 60 million row reads in total.

1. **Fix one thing at a time, and re-measure after each.**

```
CREATE INDEX ix_orders_cust_status ON orders(customer_id, status);   -- Q1 and Q3
CREATE INDEX ix_orders_date        ON orders(order_date);             -- Q2
CREATE INDEX ix_orders_status_date ON orders(status, order_date);     -- Q4
CREATE INDEX ix_orders_cover       ON orders(status, order_date, customer_id, amount);  -- Q5
```

Results after each (plans abridged to the key line):

```
Q1  0.01 ms   SEARCH orders USING INDEX ix_orders_cust_status (customer_id=?)
Q3  0.12 ms   SEARCH o USING COVERING INDEX ix_orders_cust_status (customer_id=? AND status=?)
Q3  0.06 ms   same plan, rewritten with EXISTS
Q2  8.82 ms   SCAN orders USING COVERING INDEX ix_orders_date      <- substr() hides the column
Q2  0.15 ms   SEARCH orders USING COVERING INDEX ix_orders_date (order_date>? AND order_date<?)
Q4  0.01 ms   SEARCH orders USING INDEX ix_orders_status_date (status=?)   (no temp B-tree)
Q5  19.82 ms  SEARCH orders USING COVERING INDEX ix_orders_cover (status=? AND order_date>?)
              USE TEMP B-TREE FOR GROUP BY / FOR ORDER BY
```

Look at Q2 carefully. Adding the index alone moved it from 10.81 to 8.82 ms, almost no gain, because `substr(order_date,1,7)` wraps the column in a function and the index cannot be searched. The plan still says `SCAN`. Only the rewrite to a date range (called **sargable**, from "search ARGument able") made it a `SEARCH` and gave 0.15 ms.

1. **The summary table.** This is the main exhibit in your README. Speed-up = before / after.

Query	Problem	Fix	Before (ms)	After (ms)	Speed-up
Q1 one customer	full scan	index on `customer_id`	4.30	0.01	about 430x
Q2 one month	function on column	range filter + index	10.81	0.15	about 72x
Q3 has-paid-order loop	scan inside a loop	index; `EXISTS`	1603.07	0.06	about 27,000x
Q4 latest 10 paid	scan + sort	composite index	12.25	0.01	about 1,200x
Q5 top spenders	scan + group	covering index	31.15	19.82	1.6x

Q1 and Q4 have timer limits: 0.01 ms is near the clock's resolution, so say "much faster than 1 ms", not "430x exactly". Q5 shows an honest small win: the index removed the full scan but the grouping and sorting still happen, as the two `TEMP B-TREE` lines say.

1. **Count the cost.** Indexes make writes slower. Bulk-insert 200,000 rows into an in-memory table with 0, 1 and 4 of those indexes (best of 3):

```
0 extra indexes: bulk insert of 200,000 rows =      96 ms
1 extra indexes: bulk insert of 200,000 rows =     200 ms
4 extra indexes: bulk insert of 200,000 rows =     583 ms
```

Four indexes made the load about 6 times slower (583 / 96 = 6.1). Whether that is acceptable depends on the work: a reporting table loaded once a night loves indexes; a table written constantly may not.

1. **Check the answers did not change.** Every "after" result equals its "before" result (8487 rows in March, 300 customers, same top spender). A fast wrong answer is worthless.

Evaluation and acceptance criteria

- Each of the five queries has a before plan, an after plan, before and after timings, and an identical result.
- At least one query is fixed by rewriting (Q2), and at least one by an index alone (Q1).
- The README states the timing method (best of 5) and the machine caveat.
- The write cost of the indexes is measured and discussed.
- One failed or weak fix is reported honestly (Q5 at 1.6x).

Common pitfalls

1. **Function on an indexed column** (`substr` , `LOWER` , date functions). Detect: the plan still says `SCAN` . Fix: compare the raw column to a range or build an expression index.
2. **Wrong column order in a composite index.** `(status, order_date)` helps `WHERE status=... ORDER BY order_date` ; the reverse order would not. Detect: the plan shows `USE TEMP B-TREE FOR ORDER BY` .
3. `ORDER BY` **with ties.** In my first run of Q4 without the tie-breaker `order_id` , the "latest" row was id 1007 before the index and id 85648 after it. Many orders share the date 2024-12-30, so the database may return any of them. Add a unique tie-breaker column.
4. **Timing a cold, one-time run.** The first run may include disk caching. Detect: repeat; take the best of several.
5. **Adding every index you can think of.** Detect: measure bulk-load time and index count; drop indexes the plans never use.

Three extensions

1. Add a `LIKE 'abc%'` query and a `LIKE '%abc'` query; compare plans and explain why only one can use an index.
2. Create a partial index on `status='paid'` and measure its size against the full index (Chapter 10).
3. Repeat the lab on PostgreSQL with `EXPLAIN ANALYZE` and compare the plan words (not run here).

Interview talking points

1. *A query is slow. What do you do first?* Measure it and read the plan. I look for a full scan, a sort I did not expect, or a function wrapped around a column. I change one thing, then measure again.
2. *Why did the index not help the* `substr()` *query?* The index is sorted by the raw date text. A function on the column hides that order, so the database must compute the function for every row. Writing a date range lets it jump straight to the right place.
3. *What is the cost of an index?* Every insert, update and delete must also update each index, and the index takes disk space. In my lab four indexes made a bulk load about six times slower.

README outline

Title with best speed-up; question; data and setup; timing method; summary table; before and after plans for each query; write-cost test; lessons (sargable, column order, tie-breaker); limits (single machine, SQLite, synthetic data); extensions.

16.3 Worked Extensions: The Lead-Time Profile, a Risk Score and How Big a Test Must Be

INTUITION

The case study found that booking far ahead causes no-shows. A manager's next questions are always the same: 'how steep is it, exactly?', 'can you give me a list of who to phone?' and 'how long until we know if phoning works?'. This section answers all three on the same clinic data (generator in 16.1, seed 5). Numbers below come from running it on SQLite 3.50.4.

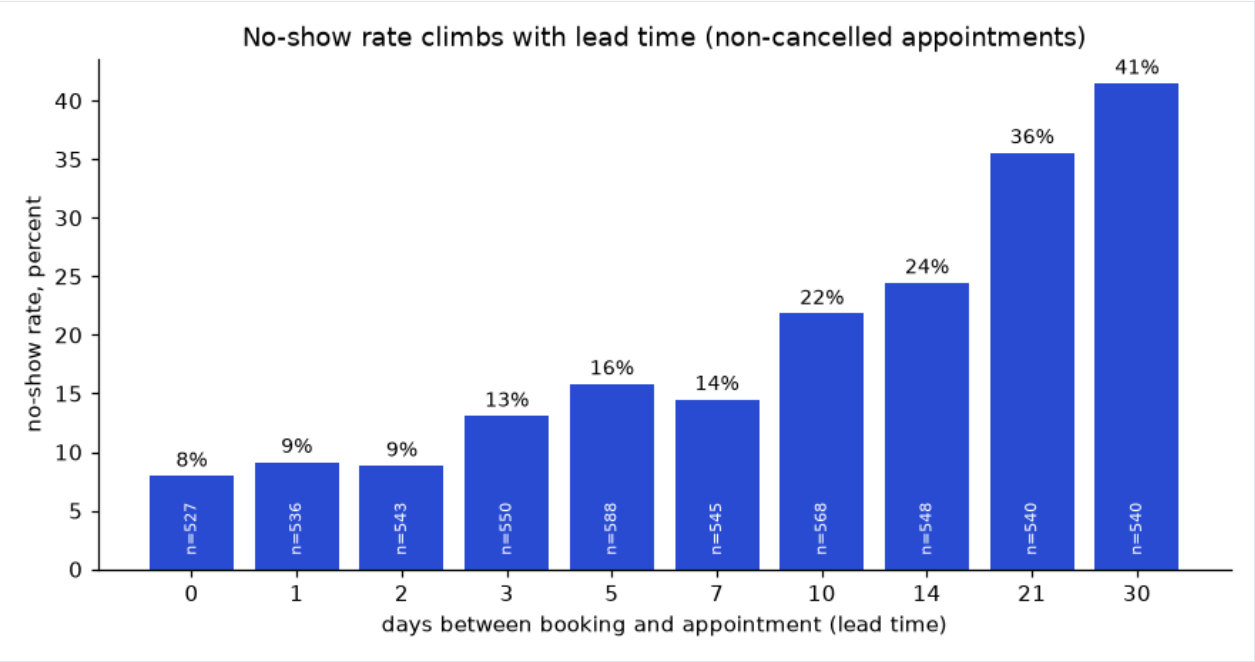
Extension 1: the full lead-time profile

Section 16.1 used lead-time bands. Bands hide the shape, so first look at every distinct lead value. Cancelled appointments are excluded because they never had a chance to be a no-show.

Problem 16.3.1 Compute the no-show rate for each lead time, with the group size. Predict first: the rate rises with lead time, from under 10 percent for same-week bookings to around 40 percent for 30 days.

```
SELECT cast(julianday(appt_date) - julianday(booked_on) AS INTEGER) AS lead_days,
       count(*) AS n,
       sum(status = 'no_show') AS no_shows,
       round(100.0 * sum(status = 'no_show') / count(*), 1) AS no_show_pct
FROM appointments
WHERE status <> 'cancelled'
GROUP BY lead_days ORDER BY lead_days;
```

```
lead_days  n    no_shows  no_show_pct
---------  ---  --------  -----------
0          527  42        8.0
1          536  49        9.1
2          543  48        8.8
3          550  72        13.1
5          588  93        15.8
7          545  79        14.5
10         568  124       21.8
14         548  134       24.5
21         540  192       35.6
30         540  224       41.5
```



No-show rate by lead time on the 5,485 non-cancelled appointments.

How to read it: each bar is one lead time, the label on top is the no-show rate and the small white text inside is the group size. All groups have 527 to 588 appointments, so none of the bars is a small-sample accident. The rate is nearly flat for 0 to 2 days (8 to 9 percent), then climbs, and the steepest jump is between 14 and 21 days (24.5 to 35.6 percent).

Step 1	`julianday` turns each date text into a day number, so the difference is the lead in days. `CAST ... AS INTEGER` drops the fraction.
Step 2	`sum(status = 'no_show')` adds the true (1) values: SQLite and MySQL treat a comparison as 0 or 1.
Check	Group sizes add up: 527 + 536 + 543 + 550 + 588 + 545 + 568 + 548 + 540 + 540 = 5,485 = 4,428 attended + 1,057 no-shows. No-shows add up to 1,057.
Meaning	The relationship is steady and monotone across ten groups of about 550 each. That is much stronger evidence than one gap between two bands.

COMMON MISTAKE	Leaving cancelled appointments in the denominator. A cancelled slot was never a no-show and never an attendance, so including it lowers every rate. The case study defines no-show rate among non-cancelled appointments; whichever definition you choose, state it in one line of the report, and use the same one everywhere.

Extension 2: a rule-based risk score and how well it ranks

Operations cannot phone 5,000 patients. They want a short list. A simple score: 2 points for lead of 14 days or more, 1 point for 5 to 13 days, plus 1 point for Pediatrics (the department with a higher planted rate). Then check how the score separates the groups.

Problem 16.3.2 Build the score and report no-show rate per score. Predict first: the rate should rise with the score, and the top two groups should hold most no-shows in a minority of appointments.

```
WITH t AS (
  SELECT a.status, d.department,
         cast(julianday(a.appt_date) - julianday(a.booked_on) AS INTEGER) AS lead
  FROM appointments a JOIN doctors d ON d.doctor_id = a.doctor_id
  WHERE a.status <> 'cancelled'
),
s AS (
  SELECT *, (CASE WHEN lead >= 14 THEN 2 WHEN lead >= 5 THEN 1 ELSE 0 END)
            + (department = 'Pediatrics') AS risk_score
  FROM t
)
SELECT risk_score, count(*) AS n, sum(status = 'no_show') AS no_shows,
       round(100.0 * sum(status = 'no_show') / count(*), 1) AS no_show_pct
FROM s GROUP BY risk_score ORDER BY risk_score DESC;
```

```
risk_score  n     no_shows  no_show_pct
----------  ----  --------  -----------
3           390   161       41.3
2           1679  485       28.9
1           1773  263       14.8
0           1643  148       9.0
```

Step 1	Score 3 (long lead and Pediatrics) has 41.3 percent no-shows (161 of 390). Score 0 has 9.0 percent (148 of 1,643). The score ranks well.
Step 2	Coverage: scores 2 and 3 together contain 390 + 1,679 = 2,069 appointments, 37.7 percent of 5,485. They hold 161 + 485 = 646 of the 1,057 no-shows, 61.1 percent.
Step 3	So phoning only the top 37.7 percent of bookings reaches 61.1 percent of all no-shows. That is a lift of 61.1 / 37.7 = 1.6 over calling people at random.
Answer	A transparent three-line rule already concentrates the problem. A model might do a little better, but this one can be explained to a receptionist.

COMMON MISTAKE	Judging the score on the same data used to invent its thresholds (14 and 5 days came from looking at the profile above). The lift is optimistic. Fix: invent the rule on the first half of the year and measure it on the second half (`WHERE appt_date >= '2024-07-01'`), as you would for any model. Expect a slightly smaller lift.

Extension 3: how big must a reminder test be?

The clinic has no reminder data, so it must run a test. Before running one, answer: how many bookings, and how many weeks? Let us plan for the group at highest risk, bookings made 14 or more days ahead, where 550 of 1,628 appointments were no-shows (33.8 percent). We hope reminders cut that by 5 points, and we want 95 percent confidence with 80 percent power (an 80 percent chance to detect a real 5-point drop).

Problem 16.3.3 Compute the sample size per arm and the time needed. Predict first: a 5-point change on a 34 percent base needs roughly 1,300 per arm, about 2,600 bookings in all, which is more than a year of such bookings.

```
import math
# baseline no-show rate for bookings made 14+ days ahead: 33.8 percent (550 of 1628)
p1 = 550 / 1628
p2 = p1 - 0.05            # we hope reminders cut it by 5 points
z_alpha, z_beta = 1.96, 0.8416      # 95 percent confidence, 80 percent power
d = p1 - p2
n_arm = (z_alpha + z_beta) ** 2 * (p1 * (1 - p1) + p2 * (1 - p2)) / d ** 2
print("baseline %.1f%%, target %.1f%%" % (100 * p1, 100 * p2))
print("needed per arm: %d, total: %d" % (math.ceil(n_arm), 2 * math.ceil(n_arm)))
per_week = 1628 / 52
print("such bookings per week: %.0f, weeks to fill: %.0f" % (per_week, 2 * math.ceil(n_arm) / per_week))
```

```
baseline 33.8%, target 28.8%
needed per arm: 1346, total: 2692
such bookings per week: 31, weeks to fill: 86
```

Formula	n per arm = $(z_{alpha} + z_{beta})^2$ x (p1(1 - p1) + p2(1 - p2)) / $(p1 - p2)^2$, with z_{alpha} = 1.96 and z_{beta} = 0.8416.
Plug in	p1 = 0.338, p2 = 0.288. Variance part: 0.338 x 0.662 + 0.288 x 0.712 = 0.2238 + 0.2051 = 0.4289. Numerator: 2.8016^2 x 0.4289 = 7.849 x 0.4289 = 3.366. Divide by 0.05^2 = 0.0025: about 1,346 per arm.
Time	The code gives the exact count and shows that at 31 qualifying bookings a week, 2,692 bookings take about 86 weeks, more than a year and a half.
Meaning	Run the test across all departments, or accept a larger effect to detect (a 10-point drop needs about a quarter of the sample), or extend the test over more than one clinic. Deciding this before the test is how you avoid a result nobody can trust.

GOING DEEPER

Sample size grows with the square of 1 over the effect. Halve the effect you want to detect and you need four times the people. That is why small improvements are expensive to prove and why analysts who promise 'we will see in two weeks' without this calculation are guessing. The same arithmetic sits behind A/B test planning in the Statistics document (Chapter 6).

Practice with answers

Problem 16.3.4 Group sizes are 527 and 540 and the rates are 8.0 and 41.5 percent. Is it worth testing whether that gap is real? *(easy)*

Answer: No. A 33.5-point gap on groups of about 530 is far beyond random wobble (a difference of 3 or 4 points would be borderline at that size). Spend the time on what to do about it.

Problem 16.3.5 The risk score gives scores 0 to 3. Why not phone only score 3? *(medium)*

Answer: Score 3 holds 390 appointments and 161 no-shows, only 15.2 percent of all no-shows. Reaching 61 percent needs scores 2 and 3, which is 2,069 calls. The choice is a trade-off between calls (cost) and no-shows reached (benefit); put both numbers in front of the manager.

Problem 16.3.6 Write the query that measures the rule only on appointments after 1 July. *(medium)*

Answer: Add `AND a.appt_date >= '2024-07-01'` to the `WHERE` of the CTE `t`, and keep the thresholds fixed from the first half. Report the new lift next to the old one.

Problem 16.3.7 The test runs, and reminders arm shows 29 percent against 34 percent in control, with 600 per arm. A manager says 'it works'. Your answer? *(interview)*

Answer: With 600 per arm the standard error of the gap is about 2.7 points, so a 5-point gap is only about 1.9 standard errors and an interval would run from roughly 0 to 10 points. It is promising, not proven. The planned test needed about 1,350 per arm; either keep running or report the interval honestly.

Chapter recap

- A case study answers a list of questions, but the product is the one-page report: findings with group sizes, recommendations, limits.
- Always print the group size next to a rate. Small groups and small gaps are noise until proven otherwise.
- Average only the rows where the value exists; `NULL` waits are not zeros.
- A median needs a window function in SQLite: `ROW_NUMBER` and `COUNT(*) OVER ()`.
- Optimization is measure, read the plan, change one thing, measure again, and check the result is identical.
- `SCAN` means every row is read; `SEARCH ... USING INDEX` means the database jumped to the right place.
- A function on an indexed column blocks the index; rewrite the filter as a range.
- Indexes cost write time and space: measure both sides.

Pattern	Code idea
Rate with group size	`ROUND(100.0*SUM(cond)/COUNT(*),1), COUNT(*)`
Bucket a number	`CASE WHEN x<=2 THEN '0–2' ... END`
Median in SQLite	`ROW_NUMBER() OVER (ORDER BY x)` and `rn=(c+1)/2`
Show the plan	`EXPLAIN QUERY PLAN <query>`
Sargable month filter	`d>='2024-03-01' AND d<'2024-04-01'`
Composite for filter + sort	`CREATE INDEX i ON t(filter_col, sort_col)`
Stable top-N	add a unique tie-breaker to `ORDER BY`

Practice set

Problem 16.P.1 (easy) 1,057 of 6,000 appointments are no-shows. What percent is that? **Answer:** 1057 / 6000 = 0.17617, so 17.6%.

Problem 16.P.2 (easy) What does `SCAN orders` in a plan mean, and what is the better word to see? **Answer:** The database reads every row of the table. `SEARCH orders USING INDEX ...` is better because it jumps to matching rows.

Problem 16.P.3 (easy) A table has 100 visits and 20 have `wait_minutes` NULL. Waits of the other 80 add to 2,400 minutes. What is `AVG(wait_minutes)`? What would it be if NULL were treated as 0? **Answer:** `AVG` ignores NULL: 2400 / 80 = 30. Treating NULL as 0 gives 2400 / 100 = 24, which wrongly understates the wait.

Problem 16.P.4 (medium) Rewrite for an index: `WHERE strftime('%Y', order_date) = '2024'`. **Answer:** `WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'`. The raw column is compared with a range, so an index on `order_date` can be searched.

Problem 16.P.5 (medium) Which index best serves `WHERE status='paid' ORDER BY order_date DESC LIMIT 10`? **Answer:** `CREATE INDEX i ON orders(status, order_date)`. The first column handles the equality filter and the second is already sorted, so no sort is needed.

Problem 16.P.6 (medium) Find the bug: "Afternoon visits have a 20% no-show rate against 14% in the morning, so move all visits to the morning." The afternoon has 15 visits. **Answer:** 20% of 15 is 3 visits, far too few to trust. Also this shows association, not cause. Get more data or a test before acting.

Problem 16.P.7 (interview) Your index made one report 50 times faster but the nightly load now takes twice as long. What do you do? **Answer:** Measure both. Options: drop indexes the plans do not use, make the index partial, drop and recreate it around the bulk load, or accept the trade if the report gain matters more. Decide with numbers, and write them down.

Problem 16.P.8 (interview, scenario) A manager asks, "Reminders cost money. Do they reduce no-shows?" You only have this year's data with no reminder column. What do you answer? **Answer:** The data cannot answer it. We can show that no-shows rise with booking lead time and suggest a small test: send reminders to a random half of long-lead bookings in one department, then compare no-show rates with group sizes.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. I am building a 20-question SQL case study on a dataset of clinic appointments (status, booking lead time, waiting minutes, fees). Help me choose good questions in easy, medium and hard order, and teach me how to write the one-page findings report with numbers, recommendations and limits. Ask me one question at a time and check my understanding.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. Review my findings report from a SQL case study. Check each claim for group size, noise versus real pattern, causal wording that goes too far, and numbers that do not reconcile. Rewrite the weakest paragraph in simple English and tell me what a hiring manager would still ask. My report and my queries: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. Teach me to read SQLite EXPLAIN QUERY PLAN output: SCAN, SEARCH, COVERING INDEX, USE TEMP B-TREE and CORRELATED SCALAR SUBQUERY. Use a 200,000-row orders table with real numbers, explain each plan line in plain words, and after every idea ask me to predict the plan for a new query before showing it.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data analyst roles. Review my query-optimization lab: my five slow queries, the plans, the timings, the indexes and my summary table. Tell me where my method is
```

weak (timing, ties, one change at a time, result checks, write cost) and which fix I should have tried instead. My lab notes: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional preparing for data analyst interviews. Run a mock interview on my two projects, a clinic analytics case study and a query-optimization lab. Ask me eight questions one at a time, from easy to hard, about my choices, my checks and my limits. After each answer, score me out of 5, show what I missed, and give a short model answer in simple English.

APPENDIX

CHEAT SHEET

One page per idea: every key pattern from Chapters 1 to 14, grouped by chapter. Print it, then try to need it less each week.

Use this appendix after you have worked through the chapters, not before. Each row is a reminder, not a lesson. If a row feels strange, go back to the named chapter and run the example again on the shop database. Syntax is PostgreSQL-style with SQLite notes unless a row says otherwise. Dialect-specific rows (BigQuery, Snowflake, SQL Server, MySQL) are not run here and should be checked against the manual of your version.

Chapter 1: Databases and the Relational Model

Idea	Pattern or rule
Open SQLite	`sqlite3 shop.db` then `.tables`, `.schema t`
psql basics	`\dt` list tables, `\d t` describe
Primary key	`customer_id INTEGER PRIMARY KEY`
Foreign key	`customer_id INTEGER REFERENCES customers(customer_id)`
SQLite FK switch	`PRAGMA foreign_keys = ON;` every session
Many-to-many	a bridge table with a composite primary key
Grain question	"One row per what?" before every query
NULL	unknown, not zero

Chapter 2: SELECT, WHERE, ORDER BY, LIMIT, DISTINCT, NULL

Idea	Pattern or rule
Top N	`ORDER BY x DESC LIMIT n` (always sort first)
Page k of size n	`LIMIT n OFFSET (k-1)*n`
Evaluation order	FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT
Mixed AND/OR	use brackets (`AND` binds first)
NULL test	`IS NULL`, `IS NOT NULL` (never `= NULL`)
Fill NULL	`COALESCE(x, 0)`
Safe divide	`x / NULLIF(y, 0)`
Integer division	`7/2 = 3`; use `7*1.0/2`
Trap	`x NOT IN (list with NULL)` returns no rows

Chapter 3: Aggregation, GROUP BY, HAVING, FILTER

Idea	Pattern or rule
Count flavours	`COUNT(*)` rows, `COUNT(c)` non-NULL, `COUNT(DISTINCT c)` kinds
Per group	`SELECT k, SUM(v) FROM t GROUP BY k`
Group filter	`HAVING COUNT(*) > 2`
Conditional sum	`SUM(CASE WHEN s='paid' THEN v ELSE 0 END)` or `FILTER (WHERE ...)`
Averages	never average averages; sum then divide
Check	grouped totals must equal the ungrouped total

Chapter 4: Joins

Need	Pattern
Matching rows only	`A JOIN B ON A.k = B.k`
Keep all of A	`A LEFT JOIN B ON ...`
Filter B on a LEFT join	put it in `ON`, not `WHERE`
Rows with no match	`LEFT JOIN ... WHERE B.k IS NULL` or `NOT EXISTS`
At least one match	`WHERE EXISTS (SELECT 1 ...)`
Compare two lists	`FULL JOIN`
All pairs	`CROSS JOIN` (m x n rows)
Hierarchy or pairs in one table	self join with two aliases
Fan-out test	`COUNT(*)` before and after the join; aggregate to the key first
Is it a key?	`COUNT(*) = COUNT(DISTINCT k)`

Chapter 5: Subqueries and CTEs

Need	Pattern
Compare with a benchmark	`WHERE x > (SELECT AVG(x) FROM t)`
Named steps	`WITH a AS (...), b AS (... FROM a) SELECT ...`
Recursive tree	`WITH RECURSIVE t AS (anchor UNION ALL step) SELECT ...`
Stop recursion	depth limit or `UNION`
Date series	recursive CTE with `date(d,'+1 day')` (PostgreSQL: `generate_series`)
Style	one clause per line, clear names, comments

Chapter 6: CASE, Strings, Dates, JSON

Task	PostgreSQL	SQLite
Buckets	`CASE WHEN ... THEN ... ELSE ... END`	same
Clean text	`TRIM`, `LOWER`, `REPLACE`, `SUBSTRING`	`trim`, `lower`, `replace`, `substr`
Month start	`DATE_TRUNC('month', d)`	`date(d,'start of month')`
Days between	`d2 - d1`	`julianday(d2)-julianday(d1)`
Date range	`d >= start AND d < next`	same
JSON text	`col ->> 'k'`	`col ->> '$.k'`
Array to rows	`UNNEST`	`json_each`
Rows to text	`STRING_AGG(x, ', ')`	`GROUP_CONCAT(x, ', ')`

Chapter 7: Sets, Pivot, ROLLUP

Need	Pattern
Stack	`UNION ALL` (keeps duplicates), `UNION` (removes)
In both / in first only	`INTERSECT` / `EXCEPT`
Pivot	`SUM(CASE WHEN m='2024-01' THEN v END) AS jan`
Unpivot	one `SELECT` per column with `UNION ALL`
Subtotals	`GROUP BY ROLLUP (a, b)`, `CUBE`, `GROUPING SETS` (SQLite: emulate with `UNION ALL`)

Chapter 8: Window Functions

Need	Pattern
Template	`fn() OVER (PARTITION BY g ORDER BY x ROWS BETWEEN ...)`
Rank family	`ROW_NUMBER` unique, `RANK` gaps, `DENSE_RANK` no gaps
Top N per group	CTE with `ROW_NUMBER()`, filter `rn <= N`
Previous / next	`LAG(x)`, `LEAD(x)`
Running total	`SUM(x) OVER (ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
Moving average	`AVG(x) OVER (ORDER BY t ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`
Share of total	`x / SUM(x) OVER ()`
Islands	group by `date - ROW_NUMBER()`
Buckets	`NTILE(n)`

Chapter 9: DDL, DML, Transactions

Task	Pattern
Create safely	`CREATE TABLE IF NOT EXISTS t (...)`
Constraints	`NOT NULL` , `UNIQUE` , `PRIMARY KEY` , `FOREIGN KEY` , `CHECK` , `DEFAULT`
Add column	`ALTER TABLE t ADD COLUMN c TYPE`
Safe update	write `WHERE` first, test with `SELECT COUNT(*)`
Upsert	`INSERT ... ON CONFLICT (k) DO UPDATE SET c = excluded.c`
Transaction	`BEGIN; ...; COMMIT;` or `ROLLBACK;`
Row lock	`SELECT ... FOR UPDATE`
ACID	atomic, consistent, isolated, durable
Isolation	read committed, repeatable read, serializable (needs retries)
Money	`NUMERIC` , never `REAL`

Chapter 10: Indexes, EXPLAIN, Optimization

Task	Pattern
Create	`CREATE INDEX idx ON t(a, b);` equality columns first, range last
Partial / expression	`... ON t(a) WHERE s='x'` , `... ON t(lower(email))`
Covering	`INCLUDE (b)` (PostgreSQL)
See the plan	`EXPLAIN` , `EXPLAIN ANALYZE` (SQLite: `EXPLAIN QUERY PLAN`)
Sargable	keep columns bare: `d >= x AND d < y` , not `DATE(d) = x`
Avoid	`SELECT *` , `LIKE '%x'` , N+1 query loops
Refresh stats	`ANALYZE t;`
Method	baseline, one change, re-measure, check write cost

Chapter 11: Schema Design and Warehouse Models

Topic	Rule
1:N	foreign key on the many side
M:N	junction table
1NF / 2NF / 3NF / BCNF	single values / whole key / only the key / every determinant is a key
FD test	`GROUP BY a HAVING COUNT(DISTINCT b) > 1`
OLTP vs OLAP	normalised writes vs denormalised big reads

Topic	Rule
Star schema	fact table (the grain) plus flat dimensions
SCD 1 / 2 / 3	overwrite / new row with validity dates / extra column
Verify	star `SUM()` equals source `SUM()`

Chapter 12: Analytics SQL Patterns

Pattern	Core SQL
Funnel	`MIN(CASE WHEN event=... THEN ts END)` per user; compare times; divide by previous step
Churn	active last month and `NOT EXISTS` this month
Cohort retention	month index = `y*12+m` minus cohort index; `COUNT(DISTINCT user)` ; `?` cells are not zero
RFM	`MAX(date)` , `COUNT(*)` , `SUM(amt)` with a fixed as-of date, then `CASE` or `NTILE`
MoM	`LAG(x) OVER (ORDER BY period)`
YoY	self join on the month one year earlier (`LAG(x,12)` counts rows, not months)
Session	`SUM(CASE WHEN gap>30 THEN 1 ELSE 0 END) OVER (PARTITION BY u ORDER BY ts)`
Streak	`date - ROW_NUMBER()` as group key
Dedupe	`ROW_NUMBER() OVER (PARTITION BY lower(trim(key)) ORDER BY updated DESC, id DESC)` , keep 1
Quality checks	NULL counts, `COUNT(c) - COUNT(DISTINCT c)` , range tests, `LEFT JOIN ... IS NULL` orphans
Median	middle one or two rows by `ROW_NUMBER` ; PostgreSQL `PERCENTILE_CONT(0.5)`

Chapter 13: Beyond SQL

Family	Example	Use it for	Key idea
Document	MongoDB	nested records, flexible fields	embed what is read together
Key-value	Redis	cache, sessions, counters, rate limits	cache-aside, TTL, atomic `INCR`
Wide-column	Cassandra, DynamoDB	huge writes, key lookups	partition key + clustering key, query-first
Time-series	Prometheus, TimescaleDB	metrics	downsample, retention, watch cardinality
Graph	Neo4j	paths, fraud rings	nodes, edges, Cypher
Search	Elasticsearch	text and logs	inverted index, BM25
Vector	pgvector, Qdrant	similarity by meaning	embeddings, cosine, ANN

Formula	Meaning
`avg = h x t_hit + (1-h) x t_miss`	average latency with a cache
`R + W > N`	quorum reads see the latest write

Formula	Meaning
`cos(a,b) = a.b / (abs(a) x abs(b))`	vector similarity
CAP	during a partition, choose consistency or availability

Chapter 14: Dialects and the Interview Bank

Task	PostgreSQL	MySQL	SQL Server	BigQuery	Snowflake
Limit	`LIMIT`	`LIMIT`	`TOP`	`LIMIT`	`LIMIT` / `TOP`
Concat	`\`	\	`	`CONCAT`	`+` / `CONCAT`
Month start	`DATE_TRUNC('month',d)`	`DATE_FORMAT`	`DATEFROMPARTS`	`DATE_TRUNC(d,MONTH)`	`DATE_TRUNC('month',d)`
Days between	`d2-d1`	`DATEDIFF(d2,d1)`	`DATEDIFF(day,d1,d2)`	`DATE_DIFF(d2,d1,DAY)`	`DATEDIFF(day,d1,d2)`
Upsert	`ON CONFLICT`	`ON DUPLICATE KEY`	`MERGE`	`MERGE`	`MERGE`
Text aggregate	`STRING_AGG`	`GROUP_CONCAT`	`STRING_AGG`	`STRING_AGG`	`LISTAGG`
Safe divide	`NULLIF`	`/` gives NULL	`NULLIF`	`SAFE_DIVIDE`	`DIV0`
Window filter	subquery	subquery	subquery	`QUALIFY`	`QUALIFY`

Interview step	Do this
1	Restate the tables and the question
2	Ask about NULLs, ties, statuses
3	Say the plan: join, filter, group
4	Write it, then check on two rows
5	Mention edge cases and performance

Expert Rows: Isolation, Security, Index Types, Patterns, Types and Dialects

These rows summarise the expert sections added to Chapters 2, 4, 7 to 16. Items marked (PG) are PostgreSQL; check the manual for MySQL and SQL Server details.

Isolation levels and anomalies (PG)

Problem	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Dirty read	prevented	prevented	prevented

Problem	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Non-repeatable read	possible	prevented	prevented
Phantom read	possible	prevented (snapshot)	prevented
Lost update (read, then write in the app)	possible	error, retry	error, retry
Write skew	possible	possible	error, retry

Retry on SQLSTATE 40001 (serialization failure) and 40P01 (deadlock): rerun the whole transaction with backoff and jitter. Atomic update: `SET x = x - 100`. Job queue: `FOR UPDATE SKIP LOCKED`.

Security and access

Task	Pattern
Stop SQL injection	placeholders for values (`?` or `%s`); allow-list for identifiers and sort direction
Least privilege	one role per job: `CREATE ROLE analyst NOLOGIN; GRANT SELECT ON t TO analyst;`
Hide columns or rows	`CREATE VIEW v AS SELECT ... WHERE ...`, grant on the view only
Row-level security (PG)	`ALTER TABLE t ENABLE ROW LEVEL SECURITY; CREATE POLICY p ON t USING (tenant_id = ...)`
Multi-tenant safety	`tenant_id` in every primary and foreign key

Index types and plans (Chapter 10)

Need	Use
Equality and ranges	B-tree (default)
Arrays, jsonb, full text	GIN
Geometry, nearest neighbour	GiST
Huge append-only table in time order	BRIN
Only a few rows matter	partial index
Search on a function	expression index, e.g. `lower(email)`
Slow `OFFSET`	keyset: `WHERE (a, id) > (:a, :id) ORDER BY a, id LIMIT n`
Bad row estimates	`ANALYZE`, raise statistics target, extended statistics
Remove old data fast	range partitions and `DETACH PARTITION`

Pattern library (Chapters 7, 12)

Question shape	Pattern
Best N per group	`row_number() OVER (PARTITION BY g ORDER BY x DESC)` in a CTE, then `rn <= N`
Path in a graph	recursive CTE, path string as cycle guard, depth limit
Bill of materials	recursive CTE multiplying `need * qty`, then `sum` over leaf parts

Question shape	Pattern
Median or p90	row_number and count, or `percentile_cont` (PG)
Overlap test	`a.s < b.e AND b.s < a.e`
Merge intervals	running `max(end)` of earlier rows; new block when start is greater
Running distinct count	`row_number() OVER (PARTITION BY uid ORDER BY ts) = 1` flag, then running sum
Did all of these	`HAVING count(DISTINCT x) = (SELECT count(*) FROM required)`
Subtotals	`ROLLUP`, `CUBE`, `GROUPING SETS`; label with `GROUPING()`
Unknown pivot columns	build the SQL in code from `SELECT DISTINCT`

NULL, ordering and types (Chapter 2)

Trap	Safe form
`col <> 'x'` drops NULL rows	`col IS DISTINCT FROM 'x'` (PG), `IS NOT` (SQLite)
`NOT IN` with a NULL in the subquery	`NOT EXISTS`
NULL sort position differs by engine	`ORDER BY x NULLS LAST`
Case-sensitive text compare	`lower(a) = lower(b)` or a case-insensitive collation
Integer division	`100.0 * a / b`

Modelling and join sanity (Chapters 4, 11)

Rule	Meaning
Rows after a join	one row per matching pair; one-to-many joins multiply rows
Check after each join	`count(*)` versus `count(DISTINCT key)`
BCNF test	every dependency X -> Y needs a superkey X; closure of X equal to all columns means superkey
Lossless split	the shared columns must be a key of one piece
Soft delete	partial unique index `WHERE deleted_at IS NULL`

Dialect quick map (Chapter 14.6)

Task	Common form	Differs in
Top N	`LIMIT n`	SQL Server `TOP n`, Oracle `FETCH FIRST n ROWS ONLY`
Upsert	`ON CONFLICT`	MySQL `ON DUPLICATE KEY UPDATE`, others `MERGE`
String aggregate	`string_agg`	MySQL `GROUP_CONCAT`, Oracle and Snowflake `LISTAGG`
Month bucket	`date_trunc('month', d)`	MySQL `DATE_FORMAT`, Oracle `TRUNC(d, 'MM')`
Top 1 per group shortcut	`QUALIFY`	BigQuery, Snowflake, DuckDB only

An Honest Closing Note

If you worked through all fourteen chapters and ran the examples yourself, you can now do real things. You can query and join several tables without silently doubling a total. You can summarise with grouping and window functions, build a funnel, a retention table and an RFM table, and spot duplicate or orphan rows before they reach a dashboard. You can read an `EXPLAIN` plan, add an index and measure its cost before and after. You can design a clean schema and explain a star schema. You can say, in plain words, when you would use PostgreSQL, Redis, MongoDB or a warehouse, and you can port a query between dialects. That is the working skill set of a junior analyst or analytics engineer, and it matches what most interviews test.

It is also important to be honest about the limits. All outputs here came from a tiny database, SQLite, with 14 orders. Real tables have hundreds of millions of rows, messy history and other people's mistakes. This document does not teach you replication, sharding, backup and recovery in depth, stored procedures and triggers beyond a mention, query tuning on a very large production system, database administration, security hardening, or the actual operation of MongoDB, Redis, Cassandra, Neo4j, Elasticsearch, BigQuery, Snowflake or SQL Server. Those parts were shown as short snippets labelled "not run here", and you must test them on the real system before you trust them. Also check the manual for your exact version, because syntax and features change. See `_missing_topics.md` for topics a strong professional still needs.

What to practise next, in this order. First, solve 100 problems on a public practice site (LeetCode, StrataScratch or DataLemur), mostly medium level, and time yourself with the interview shape of Chapter 14. Second, install PostgreSQL, load a real dataset of a few million rows (for example a public taxi or retail dataset), and practise `EXPLAIN ANALYZE` on slow queries until a 10-second query becomes a one-second query. Third, open a free BigQuery sandbox or a Snowflake trial and redo Chapter 12 there, so the phrasebook becomes muscle memory. Fourth, design a schema for a project of your own, for example a clinic appointment system or an online shop, and write the dashboard queries for it. Finally, pair SQL with Python (pandas) and a BI tool, because most jobs expect all three.

One rule from the start still holds: never move on until the current topic feels obvious. Rebuild a query from memory, check its totals, and explain it aloud. When you can do that without looking, you are ready for the next level.